

Zoox Compute Test Engineer

Study & Reference Guide

May 2026

Contents

How to Use This Guide	6
The Platform and the Mission	7
What the Zoox Compute Platform Is	7
The Four Manufacturing Test Phases	9
Design Verification vs Manufacturing Test	10
Which Chapter Matters at Which Phase	10
1 Python for Test Automation	12
1.1 CPython Memory Model	12
1.2 Numeric Types	13
1.3 Strings and Bytes	14
1.4 Collections	16
1.5 Control Flow	19
1.6 Comprehensions	21
1.7 Functions	22
1.8 Decorators	23
1.9 Dataclasses, Typing, Enum	25
1.10 Context Managers	26
1.11 Generators and itertools	27
1.12 Error Handling	28
1.13 Regular Expressions — the <code>re</code> Module	30
1.14 File I/O, CSV, JSON, YAML	32
1.15 subprocess — Wrapping CLI Tools	34
1.16 pytest	39
1.17 Logging	43
1.18 Concurrency	44
1.19 pyserial — Serial Port Instruments	47
1.20 pyvisa — SCPI Instruments (LAN, GPIB, USB)	48
1.21 Socket Programming — the <code>equipment_rpc</code> Pattern	49
1.22 NumPy for Measurement Data	50
1.23 OOP Patterns for Test Infrastructure	53
1.24 SQLite for Test Results	55
1.25 FastAPI — the Station Dashboard Pattern	55
1.26 PySide6 — Desktop Instrument GUIs	57
1.27 Modules, venv, and Packaging	58
1.28 Worked Problems: Test-Domain Patterns	59
1.29 Quick Reference: Pitfalls and Idioms	63
2 Bash and Shell Scripting	64
2.1 Shell Mechanics	65
2.2 Conditionals and Tests	68
2.3 Loops	69
2.4 The Text-Processing Toolkit	70

2.5	Robust Scripting	74
2.6	Signals and Traps (Hardware Safety)	76
2.7	Logging and Log Rotation	77
2.8	Scheduling: cron and systemd Timers	78
2.9	Debugging Bash Scripts	79
2.10	Manufacturing Test Scripts (Complete, End-to-End)	80
2.11	One-Liner Reference	83
3	Linux for Bring-up and Hardware Debug	84
3.1	Filesystem Hierarchy Reference	84
3.2	Device Enumeration	85
3.3	dmesg and journald: Kernel Messages	88
3.4	udev Rules	90
3.5	Kernel Modules	91
3.6	PCI Config Space and AER	91
3.7	Interrupts and IRQ Affinity	93
3.8	i2c-tools	94
3.9	MTD / Flash	95
3.10	Power and Thermal	95
3.11	perf and ftrace Basics	97
3.12	Filesystems, Mount, and /proc/mounts	98
3.13	Permissions and Linux Capabilities	99
3.14	Building and Booting: initramfs, ARM Device Tree, Serial Console	99
3.15	Recovering a Wedged Board	102
3.16	systemd Service Management	103
3.17	Process Management and Signals	104
3.18	Networking for Test Engineers	105
3.19	Hardware Debug Command Reference	106
4	PCIe: Architecture, Link Training, AER, and Diagnosis	109
4.1	Why this chapter is the long one	109
4.2	Architecture: the chain you actually debug	110
4.3	Generations, widths, encoding, and the GT/s → GB/s math	112
4.4	Config space is just a file you can read	113
4.5	Link training, the LTSSM, and the latched status bits	115
4.6	AER in depth — the bits are the layer	118
4.7	Power management: ASPM, L1 substates, and their failure modes	123
4.8	Enumeration and bring-up: when the link never reaches L0	124
4.9	Equalization, presets, and why links fall back	126
4.10	Completion Timeout: the ASPM / L1SS / CTO story	128
4.11	Lane margining at the receiver — the scope-free eye	129
4.12	The BERT: bit-error rate to a confidence level	132
4.13	Switches, retimers, and walking the whole chain	134
4.14	The diagnosis playbook	135
4.15	How this maps to manufacturing test	141
4.16	Command quick-reference	144
4.17	Accuracy notes & caveats	145
4.18	Toolkit cross-reference: how <code>computetest</code> realizes the chapter	146
5	NVMe and Storage	147
5.1	Why NVMe (and why the queue model matters to test)	147
5.2	The architecture, in one paragraph you can act on	147
5.3	Admin vs I/O command sets, and Identify	148
5.4	SMART / Health — your primary pass/fail gate (log page 0x02)	149

5.5	The other logs that a credible qualification reads (Get Log Page)	152
5.6	Device Self-Test (DST) — free coverage, with one fatal gotcha	154
5.7	nvme-cli usage with representative output	155
5.8	PCIe-attach implications (point to the PCIe chapter)	156
5.9	Stress and data integrity with fio	156
5.10	Failure signatures → root cause	157
5.11	Firmware update flow (you will own this)	158
5.12	Manufacturing-test flow (provision → test → soak → gate)	158
5.13	NVMe health check in Python (toolkit cross-reference)	159
6	GPUs: Architecture, ECC/RAS, and Manufacturing Test	160
6.1	GPU Architecture, the Parts That Matter for Test	160
6.2	The ECC Model in Depth	162
6.3	Clocks, Power, Thermal, and the Throttle Bitmask	165
6.4	The Manufacturing Test Sequence	167
6.5	XID Errors — the Codes That Matter and the Action for Each	168
6.6	Health Checks: nvidia-smi -q and DCGM	170
6.7	Failure Signatures → Root Cause	173
6.8	Manufacturing Flows: Screen vs RMA	174
7	DRAM and Memory (EDAC/RAS)	176
7.1	DDR4/DDR5 fundamentals relevant to test	176
7.2	ECC: SECCDED, on-die ECC, and scrubbing	177
7.3	The memory controller (where the counters come from)	177
7.4	EDAC sysfs — reading memory errors in Linux	178
7.5	rasdaemon and decoding an error to a physical DIMM silkscreen label	179
7.6	RAS concepts: CE vs UE, predictive failure, PPR, row-hammer	180
7.7	Stress and soak: stressapptest and memtester	181
7.8	Temperature and voltage dependence — run it hot	182
7.9	Failure signatures and RMA decisions	182
7.10	Manufacturing-test limits and the gate	183
7.11	Memory health check in Python (toolkit cross-reference)	184
8	Automotive and Serial Buses: GMSL, CAN, Ethernet, I2C/SPI/UART	186
8.1	GMSL — Cameras Over Coax	186
8.2	CAN and CAN-FD — The Vehicle Control Bus	197
8.3	Automotive Ethernet — One Pair, Master/Slave	200
8.4	I2C, SPI, UART — The Housekeeping Buses	203
8.5	Putting It Together at Zoox	206
9	Manufacturing Test Principles	207
9.1	The Four Phases, In Depth	207
9.2	The 10× Rule and Test-Coverage Allocation	209
9.3	Design Verification vs Manufacturing Test, In Depth	210
9.4	Test Limits — Guard-Banding and Cpk-Driven Limits	211
9.5	Per-Unit Data, Traceability, and SPC	212
9.6	Yield and Test Economics	213
9.7	Bring-Up vs Production	214
9.8	The Debug-to-Root-Cause Workflow	215
9.9	Reliability and Stress Screening	216
9.10	The CM Relationship: NPI to Mass-Production Ramp	217
9.11	The Test Station and Its Instruments	218
9.12	The Manufacturing-Test Tooling Landscape	220
9.13	Dashboards and Grafana for Manufacturing Test	223
9.14	Putting It Together — The Expert Mental Model	226

10	Math, Probability, and Statistics for Test	227
10.1	Probability Fundamentals	227
10.2	Distributions and When to Reach for Each	229
10.3	The “Sigma Level” Bridge	231
10.4	Geometry and Spatial Reasoning	231
10.5	Algebra and Signal Math	234
10.6	Vectors and Linear Algebra Basics	235
10.7	Core Physical Relationships	236
10.8	BER and Confidence — the PCIe BERT Math	237
10.9	Process Capability and Setting Limits	239
10.10	Statistical Process Control (SPC)	241
10.11	Gauge R&R / Measurement Systems Analysis	243
10.12	Sampling and AQL	245
10.13	Yield: FPY, RTY, Throughput, and Cost of Test	246
10.14	Reliability: Bathtub, MTBF, FIT, and Acceleration	247
10.15	Regression and Correlation	249
10.16	Fast Estimation and Mental Math for the Floor	249
10.17	One-Page Formula Sheet	250
11	Networking: Fundamentals to the Manufacturing Floor	252
11.1	The OSI and TCP/IP Models	252
11.2	Ethernet, MAC Addresses, and VLANs	254
11.3	IP Addressing and Subnetting	256
11.4	ARP, DHCP, and DNS	258
11.5	TCP vs UDP	259
11.6	Sockets and the Equipment RPC Pattern	261
11.7	Linux Networking Tools — Command-First Reference	263
11.8	PXE, TFTP, NFS, and HTTP — Test-Station Provisioning and DUT Netboot	267
11.9	Time Synchronization: NTP and PTP/IEEE 1588	270
11.10	NIC Manufacturing Test	273
11.11	Automotive Ethernet for Test Engineers	275
11.12	Layer-by-Layer Troubleshooting Methodology	277
11.13	HTTP, REST, and the FastAPI Dashboard	280
11.14	Pointer to the Automotive Chapter	281
12	Power, Bring-up, and Functional Safety	282
12.1	Board Power Architecture	282
12.2	Measuring Rails Correctly	285
12.3	Thermal Management and Thermal Test	288
12.4	DC and Transient Margining (Shmoo Testing)	290
12.5	Board Bring-up — Structured and Gated	291
12.6	Functional Safety (ISO 26262) — What a Compute Test Engineer Must Know	294
12.7	Correlating Rail Problems with Digital Failures	298
13	Control Systems	300
13.1	Feedback, Open-Loop, and Closed-Loop	300
13.2	PID — Intuition for Each Term	300
13.3	Stability, Poles, Bandwidth, and Margins	301
13.4	Sampling and the Nyquist Rate	303
13.5	Sensors and Actuators in an Autonomous-Vehicle Context	304
13.6	Where Test Engineers Touch Control Loops	305
13.7	Control Concepts Applied to the Compute Board — Summary	307
14	Embedded Systems for the Compute Test Engineer	308
14.1	What “Embedded” Means on a Compute Platform	308

14.2	Microcontrollers and the Memory Map	309
14.3	Bare-Metal, RTOS, and Real-Time	309
14.4	Interrupts and Concurrency on an MCU	310
14.5	Firmware Lifecycle: Build, Flash, Boot	311
14.6	Firmware Update in Manufacturing	312
14.7	Debug and Bring-Up Tooling	313
14.8	ESD Awareness and Handling	313
14.9	Low-Speed Control Buses (Cross-Reference)	314
14.10	Throughlines	314
15	Continuous Integration and Test-Engineering Quality Tooling	315
15.1	The shape of a quality pipeline (and why a test engineer must own it)	315
15.2	pytest, at the depth that matters in CI	316
15.3	unittest — when stdlib is enough, and migrating away	318
15.4	Unity — unit testing in C	318
15.5	Hypothesis — property-based testing	320
15.6	mutmut — mutation testing, and what it actually buys you	321
15.7	coverage.py — what to measure, what <i>not</i> to	322
15.8	The static-analysis stack — ruff, black, mypy	323
15.9	GitHub Actions, from first principles	325
15.10	Test data: fixtures, fakes, corpus, contract tests	327
15.11	The pipeline you build at the bench (a concrete worked example)	328
15.12	Reading CI output: what to ignore, what to fix today	329
15.13	What this entire stack does for you	329

How to Use This Guide

This is a **day-to-day technical reference** for the Compute (Manufacturing Test & Diagnostics) role — not interview prep, and not an onboarding plan. The onboarding and career material (the first 90 days — technical *and* relational — working with contract manufacturers, the leadership principles) lives in the companion *Success* guide; live PCIe root-causing lives in the *PCIe Debug* guide. Read the opener for context, then use the chapters as deep references while you work a board, a bring-up, or a yield issue.

- **Chapters 1–3 — the toolbox:** Python for test automation, Bash, and the parts of Linux you actually debug hardware from.
- **Chapters 4–8 — the interfaces, at register level:** PCIe, NVMe, GPUs, DRAM/memory, and the automotive/serial buses (GMSL, CAN, Automotive Ethernet, I²C/SPI/UART).
- **Chapters 9–13 — the discipline:** manufacturing-test principles, the math and statistics behind limits and confidence, networking on the floor, power/bring-up/ functional safety, and control-systems fluency.

A companion `toolkit/` implements much of the PCIe/NVMe/GPU/GMSL material described here (the BERT engine, AER decode, lane margining, the chain monitor, the interface checks). Read a section, then go read and run the code that does it.

The Platform and the Mission

This is the context opener for the rest of the guide. It answers three questions before any register appears: *what* you are testing (the compute platform and the autonomy data path it serves), *where* you test it (the four manufacturing phases), and *why the discipline is shaped the way it is* (Design Verification (DV) vs Manufacturing Test — different goals, statistics, and consumers). Everything here is deliberately shallow on detail; each interface and technique gets its own deep chapter, and this opener tells you which one to reach for at which phase.

The one-sentence version of the job: **prove every interface on a compute board works — at speed, under load, at temperature — and catch the bad units before they cost 10× more to find downstream.** Hold that sentence; the whole guide serves it.

What the Zoox Compute Platform Is

Zoox builds a purpose-built robotaxi — a symmetric, bidirectional vehicle with no steering wheel and no human driver to fall back on. The **compute platform** is the brain of that vehicle: a set of **server-grade compute assemblies** that ingest sensor data, run the perception → prediction → planning → control stack, and command the vehicle’s motion. You are testing data-center-class hardware that has been put inside a car.

That phrase — *server-grade compute in a vehicle* — is the whole tension of the job. Data-center parts assume a clean rack: stable power, filtered air, a bench tech a few feet away. A robotaxi gives them none of that. The same silicon now lives in a sealed, vibrating, temperature-cycled enclosure, on a vehicle power bus, expected to run a safety-critical workload for years with no one watching. Manufacturing test is what bridges that gap: it proves the data-center part survives the automotive environment *before* it carries a passenger.

The shape of the platform:

- **Server-class SoCs/CPUs + GPUs** for the AI/perception workload — data-center-grade compute (lots of PCIe lanes, Error-Correcting Code (ECC) memory, high power and heat) in an automotive enclosure. (Deep dive: the **GPU chapter**.)
- **Custom PCIe devices.** Zoox designs its own PCIe cards — sensor-interface cards, Gigabit Multimedia Serial Link (GMSL) camera aggregators, Field-Programmable Gate Array (FPGA)/accelerator cards, switch/fan-out boards. *Custom* is the load-bearing word: there is no vendor datasheet test plan and no supplier-provided diagnostic. *You* write the test that proves the board works. This is the highest-leverage part of the job — there is no fallback, so the quality of the test is entirely on you. (Deep dive: the **PCIe chapter**.)
- **Storage and memory.** Non-Volatile Memory Express (NVMe) SSDs (OS, logging, maps and models — sensor logging alone is enormous) and ECC Dynamic Random-Access Memory (DRAM) (DDR4/DDR5). (Deep dives: the **NVMe** and **Memory** chapters.)
- **High-speed links — every one a SerDes that can train wrong, drift with temperature, or fail a marginal lane.** PCIe (Gen3/4/5) between everything; **GMSL** from cameras; **automotive Ethernet** for radar/lidar/inter-module; **Controller Area Network (CAN)/Controller Area**

Network Flexible Data-Rate (CAN-FD) on the vehicle bus. Most of what you debug lives here. (Deep dives: the **PCIe** and **automotive/ serial-bus** chapters.)

- **Redundancy and safety.** Because a single compute fault could mean loss of vehicle control, the platform is built with redundancy — redundant compute paths, ECC on memory, dual-path links where it matters. Your tests don't just find defects; they **verify that the safety and detection mechanisms actually work** (ECC really corrects, the redundant link really carries traffic, bus-off recovery really recovers). More on this framing in the **functional-safety** chapter (ISO 26262 / Automotive Safety Integrity Level (ASIL)).
- **Linux, everywhere.** The platform runs Linux and your test stations run Linux. Your command-line debugging fluency is load-bearing, not incidental.

Don't assume x86 + a discrete GPU — the ARM64/Tegra reality. The “server-grade compute” framing above can read as “an x86 host with a discrete datacenter GPU,” and some of the fleet is exactly that. But NVIDIA's automotive compute (Jetson **Orin / Thor** class) is an **ARM64 (Tegra) SoC** — an *integrated* GPU, LPDDR shared with the CPU, no PCIe link to the GPU — and a **Jetson dev kit is the cheap bring-up stand-in** you'll actually have on the bench. The practical point for *your tooling*: assumptions baked into an x86-plus-discrete-GPU test program quietly break on a Tegra. What flips —

- **GPU telemetry:** `nvidia-smi/NVML` → `tegrastats/jtop` (there is no `nvidia-smi` on L4T); the iGPU has no GPU-ECC, no row-remapping, no PCIe replay counter to read (**GPU chapter**).
- **Memory RAS:** x86 EDAC/MCA + ACPI-EINJ error injection assume an x86/ACPI platform; a device-tree ARM SoC like Tegra has no ACPI-EINJ and the LPDDR controller exposes RAS differently (**Memory chapter**).
- **What's even present:** no CXL; no discrete-GPU ECC; GMSL only with a camera carrier; PCIe is there, but a Tegra's lane count and generation differ from a server's.

A check that “passes” only because the registers it reads **don't exist** on the SoC is a silent escape. Know the ISA and the GPU class of the unit in front of you before you trust a number — the lesson the toolkit hit standing up its Tegra/`tegrastats` path.

The autonomy data path (what you are protecting)

```
Cameras (e.g. AR0820) --MIPI CSI-2--> GMSL serializer (MAX96717)
--coax up to 15 m through the harness--> GMSL deserializer (MAX96712)
--CSI-2 / PCIe--> Compute SoC --PCIe--> GPU(s) + NVMe + NIC
Lidar / Radar --Automotive Ethernet (100/1000BASE-T1)--> Compute
Vehicle bus --CAN / CAN-FD--> Compute
```

Why this picture matters to a test engineer: a single weak PCIe lane, a GMSL link that won't lock at 85 °C, or a NVMe drive that throttles under sustained logging can *silently* degrade perception. In a robotaxi that is a safety issue, not a customer-annoyance issue. Manufacturing test is the gate that keeps a marginal board from ever reaching a vehicle. That is the weight behind “ship only good units” — and the reason the false-fail-vs-escape trade is asymmetric here (see the Manufacturing Test (MT) chapter on yield and test economics).

Where you sit

You are in the **Compute program**, on **manufacturing test & diagnostics**. Your daily orbit:

- **Electrical Engineering (EE) / hardware design** — they design the boards; you test them. You receive schematics, board files, and bring-up reports; when your test finds a failure, EE is who you hand the evidence to. The sharper your evidence (decoded Advanced Error Reporting (AER), per-lane margin, eye/thermal correlation), the faster the fix.

- **Software / firmware** — drivers, Board Support Package (BSP), firmware images. Your tests run on their Linux image and flash their firmware; a new firmware drop means you re-qualify the test.
- **Manufacturing / New Product Introduction (NPI) / operations** — the people who run your tests on the line. Your test must be fast, robust, and operator-proof.
- **Contract Manufacturers (CMs)** — external partners who build at volume. You release test programs *to them* and support them remotely (see the MT chapter on the Contract Manufacturer (CM) relationship).
- **Quality / reliability** — yield data, Return Merchandise Authorization (RMA)/field returns, corrective action.

You are the person who turns “the hardware team thinks it works” into “we have data proving 1,000 units work, and a gate that stops the ones that don’t.”

The Four Manufacturing Test Phases

This is the backbone of the whole job: **Printed Circuit Board Assembly (PCBA) → module → system → vehicle**. Every test you write lives at one of these phases, and the central skill is **placing each test at the earliest phase that can catch its defect**. This section is the orientation; the phases get worked in depth in the MT chapter (The Four Phases, In Depth).

Phase	What the unit is	Who runs it	What you’re proving
PCBA	Bare PCB + components, off the SMT line	CM, usually	Built correctly: right parts, good solder, no shorts/opens, powers on, basic boot
Module	PCBA + heatsink + enclosure + connectors = a functional unit	CM or Zoxx	Every interface works at speed, under load, at temperature; firmware loaded; calibrated
System	Multiple modules integrated into the compute box/rack	Zoxx	Modules talk to each other; full-load power/thermal; system boots and runs the stack
Vehicle	Compute installed in the car with real sensors	Zoxx	End-to-end: real cameras lock over GMSL, sensors stream, vehicle-level End-of-Line (EOL) checks

The reason the phase matters is economic and it is steep: **the cost to find and fix a defect rises roughly 10× at each phase you let it escape to** — rework a board in the line (1×), tear open a module (10×), isolate which module in a system (100×), pull compute from a vehicle (1000×), or roll a truck for a field return (10,000×). That single curve is *why* you “shift left”: push coverage to the earliest phase that can detect a given defect class. The catch is that “earliest phase that can detect it” is not always “earliest phase you can introduce it” — a marginal Gen4 lane that only fails at 85 °C *cannot* be caught at room-temperature PCBA test; it needs a thermal stress at module test. Knowing which defects are catchable where is the core of “correct test coverage.” (Worked allocation matrix and the full 10× table: MT chapter, the phases and the 10× rule.)

Design Verification vs Manufacturing Test

These two modes share instruments, code, and physics but differ in **goal, statistics, and who consumes the result**. Conflating them is a classic mistake; fluently moving a test between them is the high-leverage skill. Full treatment in the MT chapter, on DV vs MT; here is the framing you carry into every later chapter.

One line. *DV asks “how good is this **design**, and where are its edges?” (measure everything, small N , characterize). MT asks “is **this unit** good enough — fast?” (go/no-go against limits, huge N , capture the few parameters that let you tune those limits later).*

Dimension	DV	MT
Question	How good is the <i>design</i> ? Where are its margins?	Is <i>this unit</i> good enough — and fast?
Output	Characterization, margin maps, the limits themselves	A go/no-go verdict (+ a few captured parameters)
Sample size	Small N (tens to a few hundred)	Every unit, at volume
Method	Characterization, shmoo, margining, corner/stress sweeps	Go/no-go against fixed limits, fast
Time budget	Hours-days per unit acceptable	Seconds-minutes per unit (takt-bound)
Who consumes it	Test/EE engineers, the design itself	The line, quality, the safety case
Who runs it	Engineers in the lab	Operators on the line / at the CM

Intent and consumers. DV’s customer is the *design* and the EE team: its job is to find the design’s edges and produce the data that *sets* the limits. MT’s customer is the *line, quality, and the safety case*: its job is a fast, repeatable verdict on each unit, plus enough captured data to keep the limits honest over time. That is why limits exist (so a unit can be judged good-enough without re-characterizing it) and why takt time exists (so the line keeps moving — a test that overruns takt makes the station the bottleneck and forces more capital).

The bridge you will use constantly: *capture the parameter, not just the verdict.* The same measurement code that DV sweeps-and-plots to characterize a design is what MT compares-to-a-limit for a fast pass/fail. DV uses the captured distribution to *set* a data-driven limit; MT *checks* that limit per unit and keeps capturing, which is what later feeds Statistical Process Control (SPC), C_{pk} , and guard-banding. **You cannot set a good limit on data you didn’t keep** (MT chapter, on DV-vs-MT and on yield/economics).

Which Chapter Matters at Which Phase

A map so you know where to turn when you are standing at a station at a given phase. The register-level detail is deferred to the named chapters — this table only points.

Phase	What you’re chasing	Primary chapters
PCBA	Build correctness: shorts/opens, missing/wrong parts, basic boot	MT (ICT/AOI/boundary-scan, DFT); Power/Bring-Up
Module	Every interface at speed, under load, hot; firmware; calibration	PCIe (link/AER/margining); NVMe; GPU; Memory; Automotive buses; Instruments (rails, scope, thermal forcer)
System	Inter-module links, system power/thermal budget, full-stack boot	PCIe (switch/retimer tree-walk); Networking; Power/Bring-Up

Phase	What you're chasing	Primary chapters
Vehicle (End-of-Line (EOL))	Real harnesses: GMSL camera lock, sensor streaming, CAN, vehicle checks	Automotive buses (GMSL/CAN/Auto-Ethernet); Networking
All phases	Limits, yield, SPC, traceability, CM correlation, debug-to-root-cause	MT; Math & Statistics (Cpk/SPC/Gage R&R/BER); Bash/Linux (the debug toolbox)

Two cross-cutting chapters underpin every phase. The **Math & Statistics** chapter holds the derivations the discipline rests on — capability (C_{pk}/P_{pk}), SPC and the Western Electric rules, Gage R&R, yield/Rolled Throughput Yield (RTY)/cost-of-test, and the Bit Error Rate (BER)/Bit Error Rate Test (BERT) confidence math. The **Bash/Linux** chapter is the command-line debug toolbox you reach for on every failure. The MT chapter (next) is the discipline itself: the phases in depth, DV-vs-MT, limits and guard-banding, yield and economics, the tooling and dashboards the line runs on, CM correlation, traceability, bring-up vs production, the debug-to-root-cause workflow, and the NPI-to-mass-production ramp.

Chapter 1

Python for Test Automation

1.1 CPython Memory Model

Everything in CPython is an object on the heap. A variable is a name that binds to an object; assignment copies the reference, not the object.

```
a = [1, 2, 3]
b = a # b and a point at the same list
b.append(4)
print(a) # [1, 2, 3, 4] -- alias bug

b = a[:] # shallow copy breaks the alias
b = list(a) # equivalent
import copy
b = copy.deepcopy(a) # deep copy for nested structures
```

CPython uses reference counting as the primary GC mechanism. Every object carries a `ob_refcnt` field; it drops to zero the moment the last reference goes away and the object is immediately reclaimed. A cyclic garbage collector (generation-based) handles reference cycles that reference counting cannot break. `sys.getrefcount(x)` returns the count + 1 (the argument slot itself adds one).

Small-integer interning: CPython pre-allocates integers in `[-5, 256]`, so `a is b` is `True` for those values. Outside that range, identical int literals may or may not be the same object depending on the compile unit — never use `is` to compare integers or strings in production code.

```
x = 256; y = 256
print(x is y) # True (interned)
x = 257; y = 257
print(x is y) # True inside one code block (compiler optimization)
# may be False if constructed at runtime
```

Pass-by-object-reference: Functions receive a reference to the object, not a copy of it. Rebinding the parameter name inside the function does not affect the caller's binding. Mutating the object does.

```
def bad_reset(lst):
    lst = [] # rebinds local name; caller unchanged

def good_clear(lst):
    lst.clear() # mutates the object; caller sees it
```

1.1.1 Mutable vs Immutable and Hashability

Type	Mutable	Hashable
int, float, bool, str, bytes, tuple (if contents hashable), frozenset	No	Yes
list, dict, set, bytearray	Yes	No

Hashability gates dict key and set membership eligibility. A `tuple` is only hashable if every element is hashable:

```
t = (1, [2, 3])
hash(t) # TypeError: unhashable type: 'list'
```

Custom classes are hashable by default (id-based hash). If you define `__eq__`, Python sets `__hash__ = None` — you must also define `__hash__` explicitly if you want instances to be usable as dict keys.

1.2 Numeric Types

1.2.1 Integers

Python `int` has arbitrary precision — no overflow. Bit-manipulation for hardware registers is exact.

```
REG = 0xABCD_1234

def extract_field(value: int, lsb: int, width: int) -> int:
    """Extract a contiguous bit field from an integer register value."""
    mask = (1 << width) - 1
    return (value >> lsb) & mask

# PCIe Link Status register (16-bit)
LINK_STATUS = 0x1043
speed_enc = extract_field(LINK_STATUS, 0, 4) # bits [3:0]
width_enc = extract_field(LINK_STATUS, 4, 6) # bits [9:4]
print(f"speed=0x{speed_enc:X} width=x{width_enc}")

# Formatting
val = 0b1010_1100
print(f"hex={val:#010x} bin={val:#010b} dec={val}")
# hex=0x000000ac bin=0b10101100 dec=172
```

Bitwise operators: `&` AND, `|` OR, `^` XOR, `~` NOT, `<<` left-shift, `>>` right-shift. All work at arbitrary width.

```
# Set bit N
def set_bit(val, n): return val | (1 << n)
def clear_bit(val, n): return val & ~(1 << n)
def toggle_bit(val, n): return val ^ (1 << n)
def test_bit(val, n): return bool(val & (1 << n))
```

1.2.2 Floats and IEEE 754

`float` is a C double — 64-bit IEEE 754. Never use `==` for float comparison.

```
import math

EXPECTED_VOLTAGE = 3.3
```

```

measured = 3.299_987

# bad
assert measured == EXPECTED_VOLTAGE

# good: rel_tol handles large magnitudes, abs_tol handles near-zero
assert math.isclose(measured, EXPECTED_VOLTAGE, rel_tol=1e-4)

# absolute tolerance for specs where relative doesn't make sense
assert math.isclose(current_A, 0.0, abs_tol=1e-6) # "near zero"

```

rel_tol defaults to 1e-9; abs_tol defaults to 0.0. For hardware specs you usually want abs_tol when the expected value is near zero and rel_tol otherwise.

```

from decimal import Decimal, ROUND_HALF_UP

# Exact decimal arithmetic for spec limits
limit = Decimal("3.3000")
reading = Decimal("3.2997")
within = abs(reading - limit) <= Decimal("0.0010")

```

1.2.3 Integers as Enum/IntFlag for Registers

```

from enum import IntFlag, auto

class LinkSpeed(IntFlag):
    GEN1 = 0x1
    GEN2 = 0x2
    GEN3 = 0x4
    GEN4 = 0x8
    GEN5 = 0x10

cap = 0x1F # all speeds supported
if LinkSpeed.GEN4 in LinkSpeed(cap):
    print("GEN4 capable")

```

1.3 Strings and Bytes

1.3.1 Core String Methods (Daily Reference)

```

s = " PCIe Gen4 x16 link "
s.strip() # "PCIe Gen4 x16 link"
s.lstrip() / s.rstrip()
s.upper() / s.lower() / s.title()
s.startswith("PCIe") / s.endswith("link ")
s.find("Gen") # 7; returns -1 if not found
s.index("Gen") # 7; raises ValueError if not found
s.replace("Gen4", "Gen5")
s.split() # ["PCIe", "Gen4", "x16", "link"] (whitespace default)
s.split(":") # split on delimiter
":".join(["a", "b"]) # "a:b"
s.count("e") # 2
"42".zfill(6) # "000042"
s.partition("Gen") # (" PCIe ", "Gen", "4 x16 link ") -- key idiom

```

```
# Python 3.9+
s.removeprefix(" PCIe ")
s.removesuffix(" ")
```

`str.partition(sep)` is the idiomatic way to split once and keep the separator context — heavily used when parsing key: value lines from `lspci/dmesg` output.

1.3.2 f-String Format Spec

```
val = 3.141592653
f"{val:.4f}" # "3.1416"
f"{val:10.4f}" # " 3.1416" (width 10, right-aligned)
f"{val:<10.4f}" # "3.1416 " (left-aligned)
f"{val:+.3f}" # "+3.142"
reg = 0xDEAD_BEEF
f"0x{reg:08X}" # "0xDEADBEEF"
f"{reg:#010x}" # "0xdeadbeef"
f"{42:06b}" # "101010"
n = 1_234_567
f"{n:,}" # "1,234,567"
f"{n:_}" # "1_234_567"

# Expressions and calls inside braces
items = [3, 1, 4, 1, 5]
f"sorted: {sorted(items)!r}"
f"{'OK' if val > 0 else 'FAIL'}"

# Multi-line (no backslash needed inside braces)
result = (
    f"device={device!r} "
    f"speed={speed_gbps:.2f} Gb/s "
    f"status={'PASS' if pass_ else 'FAIL'}"
)
```

1.3.3 Bytes and Encoding

Serial ports, binary firmware files, and SCPI instrument responses all work in bytes.

```
# str <-> bytes
raw = b"\x41\x42\x43"
raw.decode("ascii") # "ABC"
"ABC".encode("ascii") # b'ABC'

# Error modes
"café".encode("ascii", errors="ignore") # b'caf'
"café".encode("ascii", errors="replace") # b'caf?'
"café".encode("ascii", errors="backslashreplace") # b'caf\\xe9'

# Struct: pack/unpack binary protocol frames
import struct
frame = struct.pack(">HHI", 0x0001, 0x0004, 0xDEADBEEF) # big-endian
cmd, length, payload = struct.unpack(">HHI", frame)

# struct format character quick reference:
# Byte order prefix: > big-endian < little-endian = native ! network (big)
# B unsigned byte (1) b signed byte (1)
# H unsigned short (2) h signed short (2)
# I unsigned int (4) i signed int (4)
```



```

# Q unsigned long long (8) q signed long long (8)
# f float (4) d double (8) s char[] (prefix with count: "4s")
# x pad byte (no corresponding Python value)

# struct.calcsize tells you the on-wire byte count before you allocate a buffer
assert struct.calcsize(">HHI") == 8

# unpack_from reads from a buffer at an offset without copying
# (useful when parsing frames inside a larger bytearray)
raw = bytearray(16)
raw[4:12] = frame
cmd2, length2, payload2 = struct.unpack_from(">HHI", raw, offset=4)

# iter_unpack: parse a binary log file of fixed-width records efficiently
RECORD_FMT = struct.Struct("<IHH") # compile once; use .unpack_from / .iter_unpack
with open("telemetry.bin", "rb") as f:
    data = f.read()
for ts_ms, temp_raw, flags in RECORD_FMT.iter_unpack(data):
    process(ts_ms, temp_raw * 0.125, flags)

# bytes are immutable; bytearray is mutable
buf = bytearray(256)
buf[0] = 0xAA
buf[1:3] = b"\xBB\xCC"

```

1.4 Collections

1.4.1 list

list is a dynamic array. Appends and pops from the right are $O(1)$ amortized; inserts/deletes at position 0 are $O(n)$.

```

readings = []
readings.append(3.301) # O(1)
readings.insert(0, 3.295) # O(n) -- avoid in hot loops
readings.extend([3.302, 3.300])
readings.sort(reverse=True)
readings.sort(key=lambda x: abs(x - 3.3)) # sort by distance from nominal
top3 = readings[:3] # slicing; creates a new list

# Sorting stability: sort by secondary key first, primary key second
pairs = [(1, "b"), (2, "a"), (1, "a")]
pairs.sort(key=lambda t: (t[0], t[1])) # (1, 'a'), (1, 'b'), (2, 'a')

```

1.4.2 tuple

Immutable, hashable (if contents are), lighter than list. Prefer tuples for fixed-length records.

```

Point = (x, y, z) = 1.0, 2.0, 3.0 # unpacking
a, *rest, z = [1, 2, 3, 4, 5] # star unpacking

# Swap without temp
x, y = y, x

```

1.4.3 NamedTuple

```
from typing import NamedTuple

class LinkStatus(NamedTuple):
    bdf: str
    speed: str
    width: int
    retrain_count: int = 0

ls = LinkStatus("0000:03:00.0", "16 GT/s", 16)
print(ls.speed) # "16 GT/s"
print(ls._asdict()) # regular dict since 3.8 (dicts keep insertion order since 3.7)
bdf, speed, *_ = ls # still iterable/unpackable
```

1.4.4 dict

Python 3.7+ guarantees insertion order. Keys must be hashable.

```
config = {"device": "nvme0", "timeout": 30, "retries": 3}

# Access patterns
config.get("missing", "default") # safe get
config.setdefault("log_level", "INFO") # insert only if absent

# Iteration
for key, val in config.items(): pass
for key in config: pass # same as config.keys()

# Merge (Python 3.9+)
defaults = {"timeout": 30, "retries": 3}
overrides = {"timeout": 60, "verbose": True}
merged = defaults | overrides # {"timeout":60,"retries":3,"verbose":True}
defaults |= overrides # in-place

# dict comprehension
squares = {x: x**2 for x in range(10)}

# Deleting
val = config.pop("retries", None) # safe pop
del config["device"] # KeyError if absent
```

1.4.5 set

Unordered, no duplicates, O(1) lookup. Use for device/ID bookkeeping.

```
active = {"nvme0", "nvme1", "nvme2"}
failed = {"nvme1"}

healthy = active - failed # difference
all_seen = active | {"nvme3"} # union
common = active & {"nvme0", "nvme3"} # intersection
xor = active ^ failed # symmetric difference

active.add("nvme3")
active.discard("nvme99") # no KeyError if missing
"nvme0" in active # O(1)

# frozenset: hashable set (usable as dict key)
```

```
device_combo = frozenset({"nvme0", "pcie0"})
```

1.4.6 defaultdict and Counter

```
from collections import defaultdict, Counter

# Grouping test results by device
results_by_dev = defaultdict(list)
for dev, result in raw_results:
    results_by_dev[dev].append(result)

# Counting error codes from dmesg
errors = defaultdict(int)
for line in dmesg_lines:
    if "error" in line.lower():
        code = parse_error_code(line)
        errors[code] += 1

# Counter is a specialized subclass
error_counts = Counter(parse_error_code(l))
for l in dmesg_lines:
    if "error" in l.lower():
        error_counts.update(parse_error_code(l))
top5 = error_counts.most_common(5)
error_counts.update(more_lines) # merge counts
```

1.4.7 deque

Double-ended queue, $O(1)$ at both ends. Essential for sliding-window and ring-buffer patterns.

```
from collections import deque
import time

# Ring buffer for live telemetry
class RingBuffer:
    def __init__(self, maxlen):
        self._buf = deque(maxlen=maxlen)

    def push(self, val):
        self._buf.append(val) # old values auto-evicted

    def values(self):
        return list(self._buf)

# Sliding-window error burst detection
def find_error_bursts(log_lines, window_seconds=60, threshold=5):
    from datetime import datetime
    errors = deque()
    bursts = []
    for line in log_lines:
        if "[ERROR]" not in line:
            continue
        ts = datetime.strptime(line[:19], "%Y-%m-%d %H:%M:%S")
        errors.append(ts)
        while errors and (ts - errors[0]).total_seconds() > window_seconds:
            errors.popleft()
        if len(errors) >= threshold:
            bursts.append((errors[0], len(errors)))
    return bursts
```

1.4.8 OrderedDict for LRU Cache

```
from collections import OrderedDict

class LRUCache:
    def __init__(self, capacity: int):
        self._cap = capacity
        self._cache = OrderedDict()

    def get(self, key):
        if key not in self._cache:
            return -1
        self._cache.move_to_end(key)
        return self._cache[key]

    def put(self, key, val):
        if key in self._cache:
            self._cache.move_to_end(key)
            self._cache[key] = val
        if len(self._cache) > self._cap:
            self._cache.popitem(last=False) # evict oldest
```

1.5 Control Flow

1.5.1 if / elif / else

```
if speed == "16GT/s" and width == "x16":
    status = "PASS"
elif speed == "16GT/s":
    status = "WARN - degraded width"
else:
    status = "FAIL"
```

Python uses `and`, `or`, `not` (not `&&`, `||`, `!`). These are **short-circuit** operators: `and` stops at the first falsy operand, `or` stops at the first truthy operand, and they return the operand value (not necessarily a bool).

```
x = a or "default" # x is a if a is truthy, else "default"
y = a and a.strip() # avoids calling .strip() on None
```

Truthiness: empty containers (`[]`, `{}`, `()`, `set()`, `""`), `0`, `0.0`, `None`, and `False` are falsy. Everything else is truthy. Test for emptiness Pythonically:

```
if not devices: # GOOD: empty list/dict/str is falsy
    ...
if len(devices) == 0: # works but verbose
    ...
```

1.5.2 Ternary expression

```
status = "PASS" if measured <= limit else "FAIL"
# Chaining is legal but hurts readability; prefer a lookup or a function:
# grade = "A" if s >= 90 else "B" if s >= 80 else "C"
```

1.5.3 for loops and the iteration helpers

```
for device in devices: # iterate elements directly
    test(device)

for i, device in enumerate(devices): # index + element
    print(f"{i}: {device}")

for i, device in enumerate(devices, start=1): # 1-based index
    print(f"Test {i}: {device}")

for name, result in zip(names, results): # parallel iteration (stops at shortest)
    print(f"{name}: {result}")

# range variants
for i in range(10): ... # 0..9
for i in range(2, 10): ... # 2..9
for i in range(0, 100, 5): ... # 0, 5, 10, ..., 95
for i in range(10, 0, -1): ... # 10, 9, ..., 1 (descending)
```

Prefer `enumerate` over manual index counters, and `zip` over indexing two lists in lockstep. Avoid `for i in range(len(xs))` unless you genuinely need the index.

1.5.4 for/else and while/else

The `else` block runs **only if the loop completed without hitting `break`**. This replaces the “found” flag pattern.

```
for dev in devices:
    if dev.degraded:
        print(f"FAIL: {dev}")
        break
else:
    # Reached only if we NEVER broke out -> all devices passed
    print("All devices passed link check")

retries = 0
while retries < max_retries:
    if connect():
        break
    retries += 1
    time.sleep(0.5 * (2 ** retries)) # exponential backoff
else:
    raise ConnectionError("Max retries exceeded") # ran out of retries
```

1.5.5 break, continue, pass

- `break` exits the innermost loop immediately.
- `continue` skips to the next iteration.
- `pass` is a no-op placeholder (an empty body that is syntactically required).

1.5.6 Walrus operator `:=` (Python 3.8+)

Assigns and returns a value in a single expression. Useful in `while` conditions and comprehensions to avoid computing something twice.

```
# Read lines until EOF without a separate pre-read
while (line := f.readline()):
```

```

process(line)

# Filter + transform without calling strip() twice
cleaned = [c for raw in data if (c := raw.strip())]
# Keeps only non-empty stripped lines, computing the strip exactly once.

# Reuse an expensive result inside the condition
if (n := len(devices)) > 8:
    print(f"Too many devices: {n}")

```

1.5.7 match / case (Python 3.10+): structural pattern matching

More than a switch: it destructures and binds.

```

match command:
    case "start":
        start_test()
    case "pause":
        pause_test()
    case str(x) if x.startswith("set_"): # guard clause with binding
        set_parameter(x[4:])
    case _: # wildcard (default)
        print(f"Unknown command: {command}")

# Destructuring dicts and matching on values
match event:
    case {"type": "error", "code": code, "message": msg}:
        log_error(code, msg)
    case {"type": "result", "value": float(v)} if v > threshold:
        handle_high_value(v)
    case {"type": "result", "value": float(v)}:
        handle_normal(v)

# Destructuring sequences
match point:
    case (0, 0):
        print("origin")
    case (x, 0):
        print(f"on x-axis at {x}")
    case (x, y):
        print(f"at {x}, {y}")

```

1.6 Comprehensions

List, dict, set comprehensions and generator expressions share the same syntax template: `[expr for item in iterable if condition]`. The `if` clause is a filter — it controls which items enter the loop. The conditional expression `val if cond else other` is different: it transforms the value for every item.

```

# Filter: only passing items
passing_bdfs = [bdf for bdf, status in results.items() if status == "PASS"]

# Transform every item
gb_values = [bytes_val / 1e9 for bytes_val in byte_readings]

# Both: filter then transform
gb_values = [bytes_val / 1e9 for bytes_val in byte_readings

```

```

if bytes_val is not None]

# Nested loops (order matches for-loop reading order)
combos = [(dev, speed)
for dev in devices
for speed in ["GEN3", "GEN4", "GEN5"]]

# Dict and set comprehensions
error_map = {dev: count for dev, count in errors.items() if count > 0}
seen_codes = {parse_code(l) for l in log_lines}

# Generator expression - lazy, no list allocated
total = sum(v**2 for v in readings if not math.isnan(v))

```

Avoid deeply nested comprehensions (more than two for clauses) — a regular loop is clearer and easier to debug.

1.7 Functions

1.7.1 LEGB Scope and Closures

```

GLOBAL_TIMEOUT = 30 # module-level global

def make_threshold_checker(threshold):
    """Closure: threshold is captured in the enclosing scope."""
    def check(value):
        return value >= threshold # 'threshold' from enclosing scope
    return check

is_hot = make_threshold_checker(85.0)
is_hot(90.0) # True

# nonlocal to mutate enclosing variable
def make_counter():
    count = 0
    def inc():
        nonlocal count
        count += 1
    return count
    return inc

```

1.7.2 Argument Forms

```

def send_command(
    port: str, # positional-or-keyword
    cmd: bytes, # positional-or-keyword
    /, # everything left of / is positional-only
    timeout: float = 1.0, # keyword-or-positional with default
    *, # everything right of * is keyword-only
    retries: int = 3,
    verbose: bool = False,
) -> bytes:
    ...

```

Mutable default trap — the default is evaluated once at definition time:

```

# WRONG
def log_event(msg, history=[]):
    history.append(msg) # all callers share the same list

# CORRECT: sentinel pattern
_SENTINEL = object()
def log_event(msg, history=_SENTINEL):
    if history is _SENTINEL:
        history = []
    history.append(msg)
    return history

```

Late-binding in lambdas:

```

# WRONG: all lambdas capture the same 'i' variable
fns = [lambda: i for i in range(5)]
fns[0]() # 4, not 0

# CORRECT: bind at definition via default argument
fns = [lambda i=i: i for i in range(5)]
fns[0]() # 0

```

```

# *args and **kwargs
def run_all(*test_fns, **options):
    for fn in test_fns:
        fn(**options)

# Unpacking into call
args = ("nvme0", b"identify")
kwargs = {"timeout": 2.0, "retries": 1}
send_command(*args, **kwargs)

```

1.8 Decorators

A decorator is a callable that takes a callable and returns a callable. `functools.wraps` preserves the wrapped function's metadata (name, docstring, annotations) — essential for pytest fixtures and logging.

```

import functools, time, logging

log = logging.getLogger(__name__)

def timed(fn):
    @functools.wraps(fn)
    def wrapper(*args, **kwargs):
        t0 = time.perf_counter()
        result = fn(*args, **kwargs)
        log.debug("%s took %.3fs", fn.__name__, time.perf_counter() - t0)
        return result
    return wrapper

@timed
def read_temperature(device: str) -> float:
    ...

```


1.8.1 Decorator Factory (Three-Layer Pattern)

When a decorator needs its own arguments, add an outer function that captures them and returns the actual decorator.

```
def retry(times=3, delay=0.5, exceptions=(Exception,)):
    """Decorator factory: @retry(times=5, delay=1.0)"""
    def decorator(fn):
        @functools.wraps(fn)
        def wrapper(*args, **kwargs):
            last_exc = None
            for attempt in range(times):
                try:
                    return fn(*args, **kwargs)
                except exceptions as exc:
                    last_exc = exc
                    log.warning("%s attempt %d/%d failed: %s",
                                fn.__name__, attempt + 1, times, exc)
                    time.sleep(delay)
                    raise last_exc
            return wrapper
        return decorator

@retry(times=5, delay=0.1, exceptions=(IOError, TimeoutError))
def read_register(addr: int) -> int:
    ...
```

1.8.2 Class-Based Decorator (Stateful)

```
class CallCount:
    """Count how many times a function is called."""
    def __init__(self, fn):
        functools.update_wrapper(self, fn)
        self._fn = fn
        self.count = 0

    def __call__(self, *args, **kwargs):
        self.count += 1
        return self._fn(*args, **kwargs)

@CallCount
def poll_status(): ...

poll_status()
print(poll_status.count) # 1
```

1.8.3 Stacking Order

Decorators apply bottom-up; the top decorator is the outermost wrapper.

```
@timed # outermost: runs first/last
@retry(3) # middle
@validate # innermost: runs first on the actual call
def measure(): ...
# equivalent to: timed(retry(3)(validate(measure)))
```



```
class NVMeDevice(TypedDict):
    DevicePath: str
    ModelNumber: str
    Firmware: str
    SectorSize: int
```

Type hints are not enforced at runtime by default — use `mypy` or `pyright` for static analysis. At Zoox, `Optional[X]` means `Union[X, None]`.

1.9.3 Enum

```
from enum import Enum, IntFlag, auto

class TestResult(Enum):
    PASS = "pass"
    FAIL = "fail"
    SKIP = "skip"
    ERROR = "error"

    def is_terminal(self) -> bool:
        return self in (TestResult.PASS, TestResult.FAIL)

# Membership lookup by value (safe)
result = TestResult("pass") # TestResult.PASS

class ErrorFlags(IntFlag):
    NONE = 0
    LINK_DOWN = auto() # 1
    CRC_ERROR = auto() # 2
    TIMEOUT = auto() # 4
    POWER_FAULT = auto() # 8

flags = ErrorFlags.LINK_DOWN | ErrorFlags.CRC_ERROR # 3
if ErrorFlags.CRC_ERROR in flags:
    ...
```

`IntFlag` members support bitwise operations and integer comparison — ideal for hardware status register decoding.

1.10 Context Managers

`__enter__` runs on with entry; `__exit__` runs on exit whether or not an exception occurred. The three arguments to `__exit__` are the exception type, value, and traceback; returning a truthy value suppresses the exception.

```
class InstrumentSession:
    def __init__(self, resource_string: str):
        self._addr = resource_string
        self._inst = None

    def __enter__(self):
        import pyvisa
        rm = pyvisa.ResourceManager()
        self._inst = rm.open_resource(self._addr)
```

```

self._inst.timeout = 5000
return self._inst

def __exit__(self, exc_type, exc_val, exc_tb):
    if self._inst:
        self._inst.close()
    return False # don't suppress exceptions

with InstrumentSession("TCPIP::192.168.1.10::INSTR") as inst:
    inst.write("*RST")
    voltage = float(inst.query("MEAS:VOLT? DC"))

```

1.10.1 @contextmanager

```

from contextlib import contextmanager, suppress, ExitStack

@contextmanager
def exclusive_device(path: str):
    import fcntl
    fd = open(path, "rb")
    try:
        fcntl.flock(fd, fcntl.LOCK_EX | fcntl.LOCK_NB)
    yield fd
    finally:
        fcntl.flock(fd, fcntl.LOCK_UN)
    fd.close()

# suppress: swallow specific exceptions
with suppress(FileNotFoundError):
    Path("/tmp/stale.lock").unlink()

# ExitStack: dynamic number of context managers
def open_all_devices(paths):
    with ExitStack() as stack:
        handles = [stack.enter_context(open(p, "rb")) for p in paths]
    return process_all(handles)
# all handles closed even if process_all raises

```

1.11 Generators and itertools

A generator function contains `yield`. Calling it returns a generator object; execution is suspended at each `yield` and resumed on the next `next()` call. Generator objects implement the iterator protocol (`__iter__` returns self, `__next__` advances).

```

def poll_temperature(device: str, interval_s: float = 1.0):
    """Infinite generator: yields (timestamp, temp_C) until stopped."""
    import time
    while True:
        temp = read_device_temp(device)
        yield time.monotonic(), temp
        time.sleep(interval_s)

# Consume with for loop + break condition
for ts, temp in poll_temperature("gpu0"):
    record(ts, temp)

```

```

if temp > 95.0:
    trigger_thermal_alert()
    break

# yield from: delegate to sub-generator
def chain_logs(*paths):
    for path in paths:
        yield from open(path) # yields lines from each file in sequence

# Generator expression vs list comprehension
# Generator: lazy, no memory allocation
total = sum(float(line.split()[-1]) for line in open("data.csv"))
# List: eager, all values in memory -- only when you need random access

```

1.11.1 itertools

```

import itertools

# chain: flatten iterables without loading all into memory
all_lines = itertools.chain(file1, file2, file3)

# groupby: group consecutive equal-key items -- MUST be pre-sorted
data = sorted(results, key=lambda r: r.device)
for device, group in itertools.groupby(data, key=lambda r: r.device):
    device_results = list(group)

# product: Cartesian product for parametric test matrices
devices = ["nvme0", "nvme1"]
speeds = ["GEN3", "GEN4"]
qd_values = [1, 4, 16, 32]
test_cases = list(itertools.product(devices, speeds, qd_values))
# [("nvme0", "GEN3", 1), ("nvme0", "GEN3", 4), ...]

# islice: lazy slice of any iterator
first_100_errors = list(itertools.islice(
    (l for l in log_lines if "ERROR" in l), 100
))

# zip_longest: pair sequences of unequal length
from itertools import zip_longest
for a, b in zip_longest(baseline, current, fillvalue=None):
    compare(a, b)

# accumulate: running totals
from itertools import accumulate
import operator
cumulative_bytes = list(accumulate(transfer_sizes))
running_max = list(accumulate(readings, func=max))

```

1.12 Error Handling

1.12.1 Exception Hierarchy Best Practices

```

# Most specific first -- Python checks handlers top-down

```

```

try:
    result = read_register(addr)
except TimeoutError:
    log.error("register read timed out: addr=0x%X", addr)
    raise
except IOError as exc:
    log.error("IO error reading addr=0x%X: %s", addr, exc)
    return None
except Exception:
    log.exception("unexpected error reading addr=0x%X", addr)
    raise

# Bare raise re-raises the current exception without wrapping
try:
    connect()
except ConnectionError:
    cleanup()
    raise # preserves original traceback

# Exception chaining
try:
    raw = serial_read()
except serial.SerialTimeoutException as exc:
    raise InstrumentError("read timed out") from exc

# Swallow-and-log (rare in test code -- prefer letting failures propagate)
try:
    cache.invalidate()
except Exception:
    log.warning("cache invalidate failed", exc_info=True)

```

1.12.2 Custom Exception Hierarchy

```

class TestInfraError(Exception):
    """Base for all test infrastructure errors."""

class InstrumentError(TestInfraError):
    def __init__(self, msg, resource=None):
        super().__init__(msg)
        self.resource = resource

class DeviceNotFoundError(TestInfraError):
    pass

class SpecViolation(TestInfraError):
    def __init__(self, param, value, limit):
        super().__init__(
            f"{param}={value} violates limit {limit}"
        )
        self.param, self.value, self.limit = param, value, limit

```

Easier to Ask Forgiveness than Permission (EAFP) is idiomatic Python: try the operation, handle the exception. Look Before You Leap (LBYL) uses guard checks before attempting the operation. EAFP is preferred when the operation is likely to succeed; LBYL is fine for configuration validation at startup.

1.13 Regular Expressions — the re Module

Parsing tool output (`lspci`, `dmesg`, `nvme`, `ethtool`, instrument replies) is most of the string work in test automation, and `re` is the workhorse. Always write patterns as **raw strings** (`r"..."`) so backslashes reach the regex engine untouched — `r"\d"`, never `"\d"`.

1.13.1 Module functions vs. compiled patterns

Call	Returns	Use for
<code>re.search(p, s)</code>	first <code>Match</code> anywhere, else <code>None</code>	“is it in there / pull one field”
<code>re.match(p, s)</code>	<code>Match</code> only if <code>s</code> starts with <code>p</code>	anchored-at-start check
<code>re.fullmatch(p, s)</code>	<code>Match</code> only if <code>p</code> matches <i>all</i> of <code>s</code>	strict validation
<code>re.findall(p, s)</code>	list of strings (tuples if >1 group)	collect every hit
<code>re.finditer(p, s)</code>	iterator of <code>Match</code> (with positions)	every hit, lazily, with spans
<code>re.sub(p, repl, s)</code>	new string	replace / redact / normalize
<code>re.subn(p, repl, s)</code>	(new_string, n)	replace + count
<code>re.split(p, s)</code>	list	split on a pattern
<code>re.escape(s)</code>	string	treat data/user text as a literal

`re.compile(pattern, flags)` once and reuse the object in loops — clearer, and it attaches flags to the pattern. The module-level convenience functions (`re.search`, `re.match`, etc.) do cache compiled patterns internally (up to 512 unique patterns in CPython’s dict-based cache), but explicit `compile()` at module level is still preferred: it makes the pattern’s scope and flags obvious, avoids re-hashing strings on every call, and keeps patterns testable in isolation. The single biggest gotcha: **match is anchored at the start of the string, search is not**. Reach for `search` unless you specifically mean “starts with.”

```
import re
pat = re.compile(r"count=(\d+)")
pat.search("nvme0: reset count=7") # <re.Match...>, .group(1) == "7"
pat.match("nvme0: reset count=7") # None -- string does not START with count=
```

1.13.2 The Match object

```
m = re.search(r"(?P<dev>\w+):.*count=(?P<n>\d+)", "nvme0: reset count=7")
m.group(0) # whole match: 'nvme0: reset count=7'
m.group(1), m.group("dev") # both 'nvme0' (group 1 == named group 'dev'); group 2 ('7') is in
    ↪ groups() below
m.groups() # ('nvme0', '7')
m.groupdict() # {'dev': 'nvme0', 'n': '7'}
m.span(2) # (start, end) indices of group 2
m["dev"] # subscript form (3.6+)

# Walrus folds the match-test and the capture into one line:
if (m := pat.search(line)):
    handle(int(m.group(1)))
```

1.13.3 Pattern syntax you reach for

Classes `\d` `\D` digit/non `\w` `\W` word/non `\s` `\S` space/non `.` any (not `\n`)
`[abc]` set `[^abc]` negated `[a-f0-9]` ranges
Anchors `^` start `$` end `\b` word-boundary `\B` non-boundary `\A` `\Z` string start/end
Quantify `*` 0+ `+` 1+ `?` 0/1 `{m}` exactly `{m,n}` range (append `?` for LAZY: `*?` `+?` `??`)
Groups (...) capture `(?:...)` non-capture `(?P<name>...)` named `\1` / `(?P=name)` backref
Alternate `a|b`

Lookahead (?=...) followed-by (?!...) not-followed-by
Lookbehind (?<=...) preceded-by (?<!...) not-preceded-by (fixed-width only)

Flags, as a constructor arg or inline: `re.I` ignorecase, `re.M` multiline (`^/$` match each line), `re.S` dotall (`.` also matches `\n`), `re.X` verbose, `re.A` ASCII-only `\w\d\s`. Combine with `|` (`re.I | re.M`); set inline at the front (`?im`) or scoped (`?i:abc`) (3.6+).

1.13.4 Greedy vs. lazy (the classic bug)

```
xml = "<tag>value</tag>"
re.findall(r"<.*>", xml) # ['<tag>value</tag>'] greedy: longest
re.findall(r"<.*?>", xml) # ['<tag>', '</tag>'] lazy: shortest
```

`*`, `+`, `{m,n}` are greedy by default; append `?` for lazy. (3.11+ also has possessive `*+ / ++` and atomic groups (`?>...`) that never give matches back — see pitfalls.)

1.13.5 Substitution — string, backreference, or callable

```
re.sub(r"\s+", " ", raw).strip() # collapse runs of whitespace
re.sub(r"(\d{4})-(\d{2})-(\d{2})", r"\3/\2/\1", s) # reorder via backrefs (\g<1> also works)

# The replacement can be a FUNCTION -- compute each replacement:
def redact(m): return m.group(0)[:4] + "..."
safe = re.sub(r"SN[0-9A-F]{8}", redact, log) # mask serials in a shared log
text, n = re.subn(r"\bFAIL\b", "PASS", report) # also returns how many it changed
```

1.13.6 Splitting that keeps the delimiters

```
re.split(r"[,\t/]+", line) # split on any run of comma / tab / pipe
re.split(r"(\d+)", "ch12blk3") # ['ch', '12', 'blk', '3', ''] -- capture group keeps the splitters;
↳ trailing '' (string ends on a match)
```

1.13.7 Performance and pitfalls

- **Compile patterns used in hot loops** (parsing thousands of dmesg lines per unit).
- **Catastrophic backtracking:** nested quantifiers over overlapping classes — e.g. `(a+)+$` against `"aaaa...!"` — are exponential and can wedge a test station. Fix by being specific, anchoring, or using atomic groups (`?>...`) / possessive `++` (3.11+).
- `.` excludes newline unless `re.S`; `^/$` are whole-string unless `re.M`.
- `re.X` (verbose) strips unescaped whitespace in the pattern — escape real spaces as `\` or put them in `[ ]`, or a “prettified” pattern silently stops matching. (This is a common self-inflicted bug.)
- **Don’t regex structured formats.** Parse JSON/CSV with `json/csv`, `key=value` with `str.split`, nested grammars with a real parser. Regex is for flat, line-oriented text.

1.13.8 Worked examples — parsing the tools you live in

```
import re

# 1) A dmesg / log line -> named fields.
# NOTE: no re.X here -- the spaces between fields are LITERAL and must match.
LOG = re.compile(
    r"(?P<ts>\d{4}-\d{2}-\d{2} \d{2}:\d{2}:\d{2}) "
    r"\[(?P<level>\w+)\]"
```



```

r"(?P<dev>\S+): "
r"(?P<msg>.*)"
)
m = LOG.search("2024-01-15 08:43:12 [ERROR] nvme0: link reset count=7")
m.groupdict() # {'ts': '...', 'level': 'ERROR', 'dev': 'nvme0', 'msg': 'link reset count=7'}

# 2) PCIe LnkSta / LnkCap out of `lspci -vvv`
LNKSTA = re.compile(r"LnkSta:\s+Speed\s+(?P<speed>\S+),\s+Width\s+x(?P<width>\d+)")
def parse_lnksta(vvv):
    m = LNKSTA.search(vvv)
    return {"speed": m["speed"], "width": int(m["width"])} if m else {}

# 3) A BDF: domain:bus:device.function (validate + capture)
BDF = re.compile(r"^(?P<dom>[0-9a-f]{4}):(?P<bus>[0-9a-f]{2}):(?P<dev>[0-9a-f]{2})\. (?P<fn>\d)$")

# 4) key=value pairs -> dict (SMART / sysfs style dumps)
kv = dict(re.findall(r"(\w+)=\S+", "temp=41 spare=100 used=2")) # {'temp': '41', ...}

# 5) Verbose mode done RIGHT: comments allowed, literal spaces escaped as '\ '
AER = re.compile(r"""
    (?P<dev>\S+)\ # device id, then a literal space
    AER:\ # the 'AER:' tag
    (?P<kind>Corrected|Uncorrected)\ error
    """, re.VERBOSE)

```

1.13.9 Quick token reference

Token	Meaning	Token	Meaning
\d \w \s	digit / word / space	* + ?	0+, 1+, 0-or-1
\D \W \S	negated classes	{m,n}	m to n times
.	any char but newline	*? +?	lazy quantifiers
^ \$	line start / end	(...)	capture group
\b \B	word boundary / not	(?:...)	non-capturing
[...] [^...]	set / negated set	(?P<n>...)	named group
\A \Z	string start / end	(?=...) (?!...)	look-ahead +/-

Use named groups ((?P<name>...)) whenever you index by meaning — the parse becomes self-documenting and survives reordering.

1.14 File I/O, CSV, JSON, YAML

1.14.1 Path Operations

```

from pathlib import Path

results_dir = Path("/home/tester/results")
run_dir = results_dir / "run_2024-01-15"
run_dir.mkdir(parents=True, exist_ok=True)

log_file = run_dir / "nvme_stress.log"
log_file.write_text("run started\n", encoding="utf-8")
content = log_file.read_text()

```

```

# glob and rglob
csv_files = list(results_dir.rglob("*.csv"))
latest = max(csv_files, key=lambda p: p.stat().st_mtime)

# Atomic write: write temp file, then rename (os.replace is atomic on POSIX)
import os, tempfile

def atomic_write(path: Path, content: str):
    dir_ = path.parent
    with tempfile.NamedTemporaryFile("w", dir=dir_, delete=False,
        suffix=".tmp") as f:
        f.write(content)
        tmp = Path(f.name)
        os.replace(tmp, path)

```

1.14.2 CSV

```

import csv

# Write results
fieldnames = ["device", "iteration", "bandwidth_GBps", "latency_us", "result"]
with open(run_dir / "results.csv", "w", newline="", encoding="utf-8") as f:
    writer = csv.DictWriter(f, fieldnames=fieldnames)
    writer.writeheader()
    for row in test_results:
        writer.writerow(row)

# `newline=""` is required on Windows; harmless on POSIX

# Read and process
with open(run_dir / "results.csv", newline="", encoding="utf-8") as f:
    reader = csv.DictReader(f)
    rows = [row for row in reader]
    failures = [r for r in rows if r["result"] == "FAIL"]

```

1.14.3 JSON

```

import json

# Serialize test record
record = {
    "run_id": "2024-01-15T08:43:00",
    "device": "nvme0",
    "metrics": {"bw_GBps": 6.8, "iops_k": 750},
    "passed": True,
}
with open(run_dir / "record.json", "w") as f:
    json.dump(record, f, indent=2)

# Deserialize
with open(run_dir / "record.json") as f:
    data = json.load(f)

# String variants
s = json.dumps(record)
data = json.loads(s)

```

```

# Custom serializer for non-serializable types
class TestEncoder(json.JSONEncoder):
    def default(self, obj):
        if isinstance(obj, Path):
            return str(obj)
        if hasattr(obj, "isoformat"):
            return obj.isoformat()
        return super().default(obj)

json.dumps(record, cls=TestEncoder)

```

1.14.4 YAML (PyYAML)

YAML is the standard format for station configuration files at Zoox — more human-editable than JSON, supports comments.

```

import yaml

# Station config (config/station.yaml):
# station_id: rack-3-slot-2
# devices:
# - type: nvm
#   path: /dev/nvme0
#   expected_speed: "16 GT/s"
# - type: pcie
#   bdf: "0000:03:00.0"
# timeouts:
# connect: 5
# read: 2

def load_station_config(path: str | Path) -> dict:
    with open(path, "r", encoding="utf-8") as f:
        return yaml.safe_load(f) # ALWAYS safe_load; yaml.load() is unsafe

def save_station_config(config: dict, path: str | Path) -> None:
    with open(path, "w", encoding="utf-8") as f:
        yaml.safe_dump(config, f, default_flow_style=False, sort_keys=False)

# Usage
cfg = load_station_config("/etc/zoox/station.yaml")
for dev in cfg["devices"]:
    print(dev["type"], dev.get("bdf", dev.get("path")))

```

`yaml.safe_load` only constructs basic Python objects (dict, list, str, int, float, bool, None). `yaml.load(f, Loader=yaml.FullLoader)` allows arbitrary Python objects — use only when you control the YAML source.

1.15 subprocess — Wrapping CLI Tools

1.15.1 Core Patterns

```

import subprocess
from pathlib import Path

def run_tool(
    args: list[str],

```

```

timeout: float = 15,
input_data: str | None = None,
) -> tuple[int, str, str]:
    """Run a CLI tool; return (returncode, stdout, stderr). Never raises on nonzero exit."""
    try:
        proc = subprocess.run(
            args,
            capture_output=True,
            text=True,
            timeout=timeout,
            check=False,
            input=input_data,
        )
        return proc.returncode, proc.stdout, proc.stderr
    except FileNotFoundError:
        return 127, "", f"command not found: {args[0]}"
    except subprocess.TimeoutExpired as exc:
        # subprocess.run() kills the child and re-raises; exc.stdout/stderr hold
        # any partial output captured before the timeout (bytes if text=False,
        # str if text=True -- may be None if nothing was flushed yet).
        partial_out = (exc.stdout or "").strip()
        return 124, partial_out, f"timed out after {timeout}s: {' '.join(str(a) for a in args)}"

# ALWAYS pass args as a list -- never use shell=True with untrusted input
rc, out, err = run_tool(["lspci", "-s", bdf, "-vvv"])
if rc != 0:
    raise RuntimeError(f"lspci failed: {err}")

```

Shell quoting bugs and injection are the main hazards of `shell=True`. Use a list; let subprocess handle escaping.

1.15.2 lspci Parsing

```

LNKCAP_PAT = re.compile(
    r"LnkCap:.*?Speed\s+(?P<cap_speed>\S+),\s+Width\s+x(?P<cap_width>\d+)"
)
LNKSTA_PAT = re.compile(
    r"LnkSta:.*?Speed\s+(?P<cur_speed>\S+),\s+Width\s+x(?P<cur_width>\d+)"
)

def get_pcie_link_info(bdf: str) -> dict:
    rc, out, err = run_tool(["lspci", "-s", bdf, "-vvv"])
    if rc != 0:
        raise RuntimeError(f"lspci -s {bdf} -vvv: {err.strip()}")

    cap = LNKCAP_PAT.search(out)
    sta = LNKSTA_PAT.search(out)
    return {
        "cap_speed": cap.group("cap_speed") if cap else None,
        "cap_width": int(cap.group("cap_width")) if cap else None,
        "cur_speed": sta.group("cur_speed") if sta else None,
        "cur_width": int(sta.group("cur_width")) if sta else None,
    }

```

1.15.3 PCIe Link Status via sysfs (no lspci process overhead)

```
def read_link_status(bdf: str) -> dict:
    base = Path(f"/sys/bus/pci/devices/{bdf}")
    if not base.exists():
        raise FileNotFoundError(f"No PCI device at {bdf}")
    return {
        "current_speed": (base / "current_link_speed").read_text().strip(),
        "max_speed": (base / "max_link_speed").read_text().strip(),
        "current_width": (base / "current_link_width").read_text().strip(),
        "max_width": (base / "max_link_width").read_text().strip(),
    }
```

1.15.4 NVMe — nvme-cli

```
import json as _json

def nvme_list() -> list[dict]:
    rc, out, _ = run_tool(["nvme", "list", "-o", "json"])
    if rc != 0:
        return []
    data = _json.loads(out)
    # nvme-cli < 2.11: top-level "Devices" is a flat list of controller dicts.
    # nvme-cli >= 2.11: output restructured to {"Devices": [{"Subsystems": [...]}]}.
    # Probe for both layouts so the caller gets a consistent flat list.
    devices = data.get("Devices", [])
    if devices and "Subsystems" in devices[0]:
        flat = []
        for sub in devices:
            for ctrl in sub.get("Subsystems", []):
                flat.extend(ctrl.get("Controllers", []))
        return flat
    return devices

def nvme_smart_log(device: str) -> dict:
    rc, out, err = run_tool(["nvme", "smart-log", device, "-o", "json"])
    if rc != 0:
        raise RuntimeError(f"nvme smart-log {device}: {err.strip()}")
    return _json.loads(out)

def nvme_id_ctrl(device: str) -> dict:
    rc, out, _ = run_tool(["nvme", "id-ctrl", device, "-o", "json"])
    return _json.loads(out) if rc == 0 else {}
```

1.15.5 dmesg and ethtool

```
def get_dmesg_errors(keyword: str = "nvme", since_boot: bool = True) -> list[str]:
    args = ["dmesg", "--level=err,warn"]
    if since_boot:
        args.append("-T") # human-readable timestamps
    rc, out, _ = run_tool(args, timeout=5)
    return [l for l in out.splitlines() if keyword in l.lower()]

# Live-streaming dmesg (Popen for continuous output)
def stream_dmesg(callback, keyword=None):
    with subprocess.Popen(
        ["dmesg", "--follow", "-T"],
```

```

stdout=subprocess.PIPE,
text=True,
) as proc:
for line in proc.stdout:
if keyword is None or keyword in line:
callback(line.rstrip())

def get_ethtool_stats(iface: str) -> dict[str, int]:
rc, out, _ = run_tool(["ethtool", "-S", iface])
stats = {}
for line in out.splitlines():
line = line.strip()
if ":" in line:
key, _, val = line.partition(":")
try:
stats[key.strip()] = int(val.strip())
except ValueError:
pass
return stats

```

1.15.6 Privileged Commands (sudo without a TTY)

Many compute-platform tools (`nvme format`, `setpci`, `rescan`) require root. In a non-interactive test runner there is no TTY, so `sudo -S` reads the password from stdin. The better option for automated test stations is a targeted `sudoers` entry so no password is required at all.

```

import subprocess, os

def run_privileged(args: list[str], timeout: float = 30) -> tuple[int, str, str]:
    """Run a command under sudo, no TTY required.

    On a test station, configure /etc/sudoers.d/test-runner with:
    tester ALL=(ALL) NOPASSWD: /usr/sbin/nvme, /usr/sbin/setpci
    Then sudo will not prompt for a password and this wrapper is straightforward.
    """
    cmd = ["sudo", "--non-interactive"] + args
    try:
        proc = subprocess.run(
            cmd,
            capture_output=True,
            text=True,
            timeout=timeout,
            check=False,
        )
        return proc.returncode, proc.stdout, proc.stderr
    except subprocess.TimeoutExpired as exc:
        return 124, (exc.stdout or ""), f"timed out: {' '.join(args)}"
    except FileNotFoundError:
        return 127, "", "sudo not found"

```

`--non-interactive` (equivalent to `-n`) makes `sudo` fail immediately with a clear error instead of hanging waiting for a TTY password prompt — critical when running inside a Continuous Integration (CI) job or a test daemon.

1.15.7 Environment Isolation for Subprocess Calls

`subprocess.run` inherits the parent's environment by default. Passing an explicit `env=` dict prevents ambient variables (proxy settings, `LD_PRELOAD`, debug flags) from poisoning tool output.

```

import os

def clean_env(**extra: str) -> dict[str, str]:
    """Return a minimal environment for subprocess calls.

    Keeps PATH, HOME, USER, TERM; discards proxy, debug, and LD_* variables.
    Pass extra={"NVME_DEBUG": "1"} to inject variables for one call.
    """
    base_keys = {"PATH", "HOME", "USER", "LOGNAME", "SHELL", "TERM",
                 "LANG", "LC_ALL", "DBUS_SESSION_BUS_ADDRESS"}
    env = {k: v for k, v in os.environ.items() if k in base_keys}
    env.update(extra)
    return env

rc, out, err = subprocess.run(
    ["nvme", "smart-log", "/dev/nvme0", "-o", "json"],
    capture_output=True, text=True,
    env=clean_env(),
).returncode, ..., ... # unpack as normal

```

1.15.8 Popen for Long-Running Background Processes

`subprocess.run` blocks until the process exits. Use `Popen` when you need to start something, do other work concurrently, then wait — or when you need to send stdin incrementally.

```

import subprocess, signal, time

class BackgroundProcess:
    """Start a long-running subprocess, stream its stdout, stop it cleanly."""

    def __init__(self, args: list[str]):
        self._args = args
        self._proc: subprocess.Popen | None = None

    def start(self):
        self._proc = subprocess.Popen(
            self._args,
            stdout=subprocess.PIPE,
            stderr=subprocess.PIPE,
            text=True,
        )

    def stop(self, timeout: float = 5.0):
        if not self._proc:
            return
        self._proc.send_signal(signal.SIGINT)
        try:
            self._proc.wait(timeout=timeout)
        except subprocess.TimeoutExpired:
            self._proc.kill()
            self._proc.wait()

    def read_lines(self):
        """Yield lines from stdout as they arrive (blocks between lines)."""
        for line in self._proc.stdout:
            yield line.rstrip()

    def __enter__(self):
        self.start()

```

```

return self

def __exit__(self, *_):
    self.stop()

# Capture a fio run while the test loop does other work
with BackgroundProcess(["fio", "--filename=/dev/nvme0", "--rw=read",
    "--name=read_bw", "--output-format=json"]) as fio:
    time.sleep(10) # let it run
# stdout/stderr are still accessible via fio._proc after __exit__

```

1.16 pytest

pytest is the standard test runner. Key concepts: fixtures for setup/teardown, parametrize for data-driven tests, monkeypatch for dependency substitution, and `conftest.py` for shared fixtures.

1.16.1 Fixtures

```

# conftest.py (shared across a test directory tree)
import pytest
from pathlib import Path

@pytest.fixture(scope="session")
def station_config():
    """Load station config once per session."""
    return load_station_config(Path("config/station.yaml"))

@pytest.fixture(scope="module")
def nvme_device(station_config):
    """Return path to the primary NVMe device under test."""
    devs = [d for d in station_config["devices"] if d["type"] == "nvme"]
    if not devs:
        pytest.skip("no NVMe device in station config")
    return devs[0]["path"]

@pytest.fixture
def fresh_results_dir(tmp_path):
    """Per-test temp directory pre-populated with a results subdir."""
    d = tmp_path / "results"
    d.mkdir()
    return d

# Yield fixture: teardown after yield
@pytest.fixture
def serial_port(station_config):
    import serial
    port_cfg = station_config.get("serial_port", {})
    ser = serial.Serial(
        port=port_cfg["path"],
        baudrate=port_cfg.get("baud", 115200),
        timeout=port_cfg.get("timeout", 1),
    )
    yield ser
    ser.close()

```


Fixture scopes: function (default), class, module, session. Higher scopes reduce setup overhead but require fixtures to be safe for reuse. Use `yield` for teardown.

1.16.2 parametrize

```
import pytest

GEN_SPEEDS = ["GEN3", "GEN4", "GEN5"]
QUEUE Depths = [1, 4, 16, 32]

@pytest.mark.parametrize("speed", GEN_SPEEDS)
@pytest.mark.parametrize("qd", QUEUE_Depths)
def test_nvme_throughput(nvme_device, speed, qd):
    # Cartesian: 3 speeds * 4 qd = 12 test cases
    bw = measure_bandwidth(nvme_device, speed=speed, queue_depth=qd)
    assert bw >= 3.0, f"BW {bw:.2f} GBps below limit at {speed} qd={qd}"

@pytest.mark.parametrize("cmd,expected", [
    (b"*IDN?\n", "Zoox"),
    (b"MEAS:VOLT?\n", None), # None = don't check value
    (b"SYST:ERR?\n", "+0,"),
])
def test_instrument_commands(serial_port, cmd, expected):
    serial_port.write(cmd)
    resp = serial_port.readline().decode().strip()
    if expected is not None:
        assert expected in resp

# pytest.approx for floating-point
def test_voltage_within_spec(measured_voltage):
    assert measured_voltage == pytest.approx(3.3, abs=0.05)
```

1.16.3 Mocking Hardware

```
from unittest.mock import MagicMock, patch, call

# patch WHERE IT IS USED, not where it is defined
@patch("mymodule.tests.subprocess.run")
def test_lspci_parse(mock_run):
    mock_run.return_value = MagicMock(
        returncode=0,
        stdout=FAKE_LSPCI_OUTPUT,
        stderr="",
    )
    info = get_pcie_link_info("0000:03:00.0")
    assert info["cur_speed"] == "16 GT/s"
    assert info["cur_width"] == 16

# side_effect: raise on N-th call, succeed on others
call_count = 0
def flaky_read(*args, **kwargs):
    global call_count
    call_count += 1
    if call_count < 3:
        raise IOError("transient error")
    return MagicMock(returncode=0, stdout="ok", stderr="")

@patch("mymodule.subprocess.run", side_effect=flaky_read)
```

```
def test_retry_behavior(mock_run):
    rc, out, _ = run_tool(["some_cmd"])
    assert mock_run.call_count == 3
    assert out == "ok"

# monkeypatch: cleaner for simple attribute/env substitution
def test_with_env_override(monkeypatch, tmp_path):
    monkeypatch.setenv("ZOOX_RESULTS_DIR", str(tmp_path))
    monkeypatch.setattr("mymodule.DEFAULT_TIMEOUT", 0.1)
    # monkeypatch reverts all changes after the test automatically
```

1.16.4 Fixture Parametrization

Parametrizing a fixture lets you run every test that depends on it against multiple values without touching the test itself — the right approach when the variation is about the resource (which device), not the test logic.

```
# conftest.py
import pytest

@pytest.fixture(params=["nvme0", "nvme1"])
def nvme_dev(request):
    """Parametric fixture: each dependent test runs twice, once per device."""
    return f"/dev/{request.param}"

# test_nvme.py
def test_identify(nvme_dev):
    # runs as test_identify[nvme0] and test_identify[nvme1]
    rc, out, _ = run_tool(["nvme", "id-ctrl", nvme_dev, "-o", "json"])
    assert rc == 0
```

Use `pytest.fixture(params=...)` when the parameter is the *resource*. Use `@pytest.mark.parametrize` when the parameter is *test data* (inputs, thresholds, expected values). Mixing both is fine — you get the Cartesian product.

1.16.5 Accessing the request Fixture

The built-in request fixture exposes test context. The most common uses in hardware test code:

```
@pytest.fixture
def result_log(request, tmp_path):
    """Named log file per test, in a shared tmp directory."""
    log_file = tmp_path / f"{request.node.name}.log"
    return log_file

@pytest.fixture(scope="session")
def shared_results_dir(request, tmp_path_factory):
    """Session-scoped temp dir (tmp_path is only function-scoped)."""
    return tmp_path_factory.mktemp("results")

@pytest.fixture
def device_under_test(request):
    """Read device path from command-line option, skip if not given."""
    dev = request.config.getoption("--device", default=None)
    if dev is None:
        pytest.skip("--device not specified")
    return dev
```

`tmp_path` is function-scoped (new dir per test). `tmp_path_factory` is session-scoped — use it inside session or module fixtures when you need a shared directory that persists across tests in the run.

1.16.6 Mocking subprocess at the Fixture Level

Mocking `subprocess.run` inline in every test is noisy. Promote it to a fixture so all tests in a module share the same mock setup cleanly:

```
# conftest.py
import pytest
from unittest.mock import MagicMock, patch

FAKE_LSPCI = """\
0000:03:00.0 Non-Volatile memory controller: ...
  LnkCap: Port #0, Speed 32GT/s, Width x4
  LnkSta: Speed 32GT/s, Width x4
"""

@pytest.fixture
def mock_lspci(monkeypatch):
    """Replace subprocess.run with a canned lspci response."""
    fake = MagicMock(returncode=0, stdout=FAKE_LSPCI, stderr="")
    monkeypatch.setattr("mymodule.parsers.lspci.subprocess.run",
        lambda *a, **kw: fake)
    return fake

def test_link_speed_parsed(mock_lspci):
    info = get_pcie_link_info("0000:03:00.0")
    assert info["cur_speed"] == "32GT/s"
    assert info["cur_width"] == 4
```

Prefer `monkeypatch.setattr` over `@patch` inside fixtures — `monkeypatch` is pytest-native, automatically reverts after the test, and doesn't require the `with patch(...)` boilerplate.

1.16.7 conftest.py Patterns

```
# Markers for slow/hardware tests
# conftest.py at project root:
import pytest

def pytest_addoption(parser):
    parser.addoption("--run-hardware", action="store_true",
        help="Run tests that require physical hardware")

def pytest_configure(config):
    config.addinvalue_line("markers",
        "hardware: requires physical hardware (--run-hardware)")
    config.addinvalue_line("markers",
        "slow: slow integration test")

def pytest_collection_modifyitems(config, items):
    if not config.getoption("--run-hardware"):
        skip = pytest.mark.skip(reason="needs --run-hardware")
        for item in items:
            if "hardware" in item.keywords:
                item.add_marker(skip)
```

```
# pytest.ini / pyproject.toml
# [tool.pytest.ini_options]
# testpaths = ["tests"]
# markers = ["hardware", "slow", "regression"]
# addopts = "-ra -q --tb=short"
```

1.17 Logging

```
import logging
from logging.handlers import RotatingFileHandler
from pathlib import Path

def setup_logging(
    name: str,
    log_dir: Path,
    level: int = logging.DEBUG,
    max_bytes: int = 10 * 1024 * 1024, # 10 MB
    backup_count: int = 5,
) -> logging.Logger:
    log = logging.getLogger(name)
    log.setLevel(level)

    fmt = logging.Formatter(
        "%(asctime)s %(name)s %(levelname)s %(message)s",
        datefmt="%Y-%m-%dT%H:%M:%S",
    )

    # Console handler
    ch = logging.StreamHandler()
    ch.setLevel(logging.INFO)
    ch.setFormatter(fmt)

    # Rotating file handler
    log_dir.mkdir(parents=True, exist_ok=True)
    fh = RotatingFileHandler(
        log_dir / f"{name}.log",
        maxBytes=max_bytes,
        backupCount=backup_count,
    )
    fh.setLevel(logging.DEBUG)
    fh.setFormatter(fmt)

    log.addHandler(ch)
    log.addHandler(fh)
    return log

log = setup_logging("nvme_test", Path("logs"))

# %-style args -- NOT f-strings -- so the string is only formatted when
# the message will actually be emitted (avoids overhead at low log levels)
log.debug("reading register addr=0x%X", addr)
log.info("test %s: %s in %.3fs", test_name, result, elapsed)
log.warning("retry %d/%d for %s", attempt, max_tries, device)
log.error("FAIL: %s = %.4f outside [%.4f, %.4f]", param, val, lo, hi)
log.exception("unexpected exception") # includes traceback
```

1.18 Concurrency

1.18.1 The GIL

CPython's Global Interpreter Lock (GIL) serializes bytecode execution to one thread at a time. Threads are appropriate for I/O-bound work (disk, network, serial port, SCPI) — one thread blocks in the kernel and releases the GIL while others run. They don't parallelize CPU-bound computation. Use `multiprocessing` for CPU-bound work (waveform FFT, large numpy reductions), or `concurrent.futures.ProcessPoolExecutor`.

1.18.2 Threads for Instrument I/O

```
import threading
import time
from collections import deque

class DataCollector:
    """Thread-safe background sampler for any polling read function."""

    def __init__(
        self,
        read_fn,
        interval_s: float = 0.1,
        max_history: int = 1000,
    ):
        self._read_fn = read_fn
        self._interval = interval_s
        self._lock = threading.Lock()
        self._history = deque(maxlen=max_history)
        self._latest = None
        self._active = False
        self._thread: threading.Thread | None = None
        self._stop_event = threading.Event()

    def start(self):
        self._active = True
        self._stop_event.clear()
        self._thread = threading.Thread(target=self._loop, daemon=True)
        self._thread.start()

    def stop(self, timeout: float = 5.0):
        self._active = False
        self._stop_event.set()
        if self._thread:
            self._thread.join(timeout=timeout)

    def _loop(self):
        while not self._stop_event.wait(timeout=self._interval):
            try:
                value = self._read_fn()
                with self._lock:
                    self._latest = value
                    self._history.append((time.monotonic(), value))
            except Exception as exc:
                # swallow transient read errors; log if you want
                _ = exc
```

```

@property
def latest(self):
    with self._lock:
        return self._latest

def snapshot(self) -> list[tuple[float, object]]:
    with self._lock:
        return list(self._history)

def __enter__(self):
    self.start()
    return self

def __exit__(self, *_):
    self.stop()

# Usage
with DataCollector(lambda: read_psu_voltage("CH1"), interval_s=0.05) as sampler:
    run_stress_test()
    data = sampler.snapshot()

```

Lock vs RLock: Lock is not reentrant — a thread that already holds it will deadlock if it tries to acquire it again. RLock allows the same thread to acquire it multiple times. Use RLock when a method that holds the lock may call other methods that also acquire it.

```

# threading.Event: clean cancellable loop without sleep polling
stop_event = threading.Event()

def monitoring_thread(stop_event):
    while not stop_event.wait(timeout=1.0):
        check_health()

t = threading.Thread(target=monitoring_thread, args=(stop_event,), daemon=True)
t.start()
# later:
stop_event.set()
t.join()

```

1.18.3 ThreadPoolExecutor

```

from concurrent.futures import ThreadPoolExecutor, as_completed

def test_all_devices(devices: list[str], timeout: float = 30) -> dict[str, str]:
    results = {}
    with ThreadPoolExecutor(max_workers=len(devices)) as ex:
        future_to_dev = {ex.submit(run_device_test, dev): dev for dev in devices}
        for fut in as_completed(future_to_dev, timeout=timeout):
            dev = future_to_dev[fut]
            try:
                results[dev] = fut.result()
            except Exception as exc:
                results[dev] = f"ERROR: {exc}"
    return results

```

1.18.4 asyncio for Async Instrument I/O

Use `asyncio` when managing many concurrent I/O operations in a single thread — e.g., polling dozens of SCPI instruments, streaming multiple serial ports simultaneously.

```
import asyncio

async def query_instrument(addr: str, cmd: str, timeout: float = 2.0) -> str:
    reader, writer = await asyncio.wait_for(
        asyncio.open_connection(addr, 5025), timeout=timeout
    )
    try:
        writer.write((cmd + "\n").encode())
        await writer.drain()
        data = await asyncio.wait_for(reader.readline(), timeout=timeout)
        return data.decode().strip()
    finally:
        writer.close()
        await writer.wait_closed()

async def poll_all(instruments: list[str], cmd: str) -> list[str]:
    tasks = [query_instrument(addr, cmd) for addr in instruments]
    return await asyncio.gather(*tasks, return_exceptions=True)

# Run from synchronous code
results = asyncio.run(poll_all(instrument_list, "MEAS:VOLT?"))
```

Asyncio is cooperative — `await` points are where the event loop can run other coroutines. A coroutine that never awaits blocks the entire event loop.

`asyncio.timeout` (Python 3.11+) replaces the nested `wait_for` pattern with a cleaner context manager that raises `TimeoutError` directly:

```
import asyncio

async def query_instrument_311(addr: str, cmd: str, timeout: float = 2.0) -> str:
    async with asyncio.timeout(timeout):
        reader, writer = await asyncio.open_connection(addr, 5025)
    try:
        writer.write((cmd + "\n").encode())
        await writer.drain()
        data = await reader.readline()
        return data.decode().strip()
    finally:
        writer.close()
        await writer.wait_closed()
```

The `timeout` covers the entire block, not just one `await`. On Python 3.10 and earlier, keep using `wait_for`.

Using `asyncio` from synchronous `pytest` tests — `asyncio.run()` works but creates and destroys an event loop per call. For test suites, install `pytest-asyncio` and mark coroutines directly:

```
# pip install pytest-asyncio
# pyproject.toml: [tool.pytest.ini_options] asyncio_mode = "auto"

import pytest

@pytest.mark.asyncio
async def test_all_instruments_respond(instrument_addrs):
    results = await poll_all(instrument_addrs, "*IDN?")
    for r in results:
```

```
assert not isinstance(r, Exception), f"instrument error: {r}"
```

asyncio_mode = "auto" (pytest-asyncio 0.21+) marks all `async def` test functions automatically; you can drop the `@pytest.mark.asyncio` decorator.

1.19 pyserial — Serial Port Instruments

```
import serial
import serial.tools.list_ports
import time

def list_serial_ports() -> list[str]:
    return [p.device for p in serial.tools.list_ports.comports()]

class SerialInstrument:
    """Thin wrapper for RS-232/RS-485 instruments."""

    def __init__(
        self,
        port: str,
        baudrate: int = 115200,
        timeout: float = 1.0,
        terminator: str = "\n",
    ):
        self._ser = serial.Serial(
            port=port,
            baudrate=baudrate,
            bytesize=serial.EIGHTBITS,
            parity=serial.PARITY_NONE,
            stopbits=serial.STOPBITS_ONE,
            timeout=timeout,
        )
        self._term = terminator.encode()

    def __enter__(self):
        if not self._ser.is_open:
            self._ser.open()
        return self

    def __exit__(self, *_):
        self._ser.close()

    def write_cmd(self, cmd: str) -> None:
        self._ser.write((cmd + "\n").encode("ascii"))

    def read_line(self) -> str:
        raw = self._ser.readline()
        return raw.decode("ascii", errors="replace").strip()

    def query(self, cmd: str, delay_s: float = 0.05) -> str:
        self._ser.reset_input_buffer()
        self.write_cmd(cmd)
        time.sleep(delay_s)
        return self.read_line()

@property
```



```

def in_waiting(self) -> int:
    return self._ser.in_waiting

with SerialInstrument("/dev/ttyUSB0", baudrate=9600) as inst:
    idn = inst.query("*IDN?")
    voltage = float(inst.query("MEAS:VOLT?"))

```

`serial.SerialTimeoutException` is raised when `write` or `readline` exceeds the configured timeout. Wrap in the custom `InstrumentError` hierarchy for consistent error handling across instrument types.

1.20 pyvisa — SCPI Instruments (LAN, GPIB, USB)

```

import pyvisa

class SCPIInstrument:
    """VISA resource wrapper with standard SCPI commands."""

    def __init__(self, resource_string: str, timeout_ms: int = 5000):
        self._addr = resource_string
        self._timeout_ms = timeout_ms
        self._rm: pyvisa.ResourceManager | None = None
        self._inst = None

    def __enter__(self):
        self._rm = pyvisa.ResourceManager()
        self._inst = self._rm.open_resource(self._addr)
        self._inst.timeout = self._timeout_ms
        return self

    def __exit__(self, *_):
        if self._inst:
            self._inst.close()
        if self._rm:
            self._rm.close()

    def write(self, cmd: str) -> None:
        self._inst.write(cmd)

    def query(self, cmd: str) -> str:
        return self._inst.query(cmd).strip()

    def reset(self) -> None:
        self.write("*RST")

    def measure_dc_voltage(self, channel: int = 1) -> float:
        return float(self.query(f"MEAS:VOLT:DC? (@{channel})"))

    def measure_current(self, channel: int = 1) -> float:
        return float(self.query(f"MEAS:CURRE:DC? (@{channel})"))

# Supported VISA address strings:
# LAN: "TCPIP::192.168.1.10::INSTR"
# LAN raw socket: "TCPIP::192.168.1.10::5025::SOCKET"
# GPIB: "GPIB0::14::INSTR"
# USB: "USB0::0x0957::0x0607::MY47000419::INSTR"

```

```

with SCPIInstrument("TCPIP::10.0.1.50::INSTR") as psu:
    psu.reset()
    psu.write("VOLT 3.3; CURR 2.0; OUTP ON")
    v = psu.measure_dc_voltage()
    assert 3.25 <= v <= 3.35, f"PSU voltage {v:.3f} V out of range"

```

`pyvisa.errors.VisaIOError` covers most instrument communication errors (timeout, resource not found, etc.). Wrap it in `InstrumentError` when you want a unified exception hierarchy.

1.21 Socket Programming — the `equipment_rpc` Pattern

A common manufacturing-test design is a small TCP RPC server that fronts an instrument: the GUI or test runner sends a JSON command, the server executes it against the hardware and returns JSON. This decouples the UI process from the hardware process and lets multiple clients share one instrument.

1.21.1 TCP server

```

import socket
import json

def start_server(host="0.0.0.0", port=5000):
    server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    # AF_INET = IPv4; SOCK_STREAM = TCP (reliable, ordered byte stream). UDP would use SOCK_DGRAM.
    server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    # SO_REUSEADDR lets you re-bind a port still in TIME_WAIT after a restart,
    # avoiding "Address already in use" for ~60 s.
    server.bind((host, port))
    server.listen(5) # backlog: max queued pending connections
    print(f"Listening on {host}:{port}")
    while True:
        conn, addr = server.accept() # blocks until a client connects
        with conn:
            data = conn.recv(4096).decode()
            request = json.loads(data)
            response = handle_request(request)
            conn.sendall(json.dumps(response).encode())

def handle_request(req):
    cmd = req.get("command")
    if cmd == "read_voltage":
        return {"status": "ok", "value": 48.01}
    return {"status": "error", "message": f"unknown command: {cmd}"}

```

1.21.2 TCP client

```

def send_command(host, port, command):
    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as sock:
        sock.settimeout(5.0) # ALWAYS set a timeout; a hung instrument must not block forever
        sock.connect((host, port))
        sock.sendall(json.dumps(command).encode())
        # TCP is a stream, not messages: read until the peer closes (or use a length prefix).
        chunks = []
        while True:
            chunk = sock.recv(4096)

```

```

if not chunk: # empty bytes -> peer closed the connection
    break
chunks.append(chunk)
return json.loads(b"".join(chunks).decode())

```

Two essentials: always `settimeout`, and remember that `recv` returns whatever bytes are available, so you must loop and reassemble. For request/response framing, prefer a newline-delimited protocol or a fixed-length header that states the body length.

1.22 NumPy for Measurement Data

```

import numpy as np

readings = np.array([3.298, 3.302, 3.299, 3.301, 3.300, 3.297, 3.303])

# Descriptive statistics
mean = readings.mean()
std = readings.std(ddof=1) # ddof=1 for sample std dev
p95 = np.percentile(readings, 95)
rms = np.sqrt(np.mean(readings**2))

# Spec check with boolean masking
# CRITICAL: hardware specs use & not 'and' with numpy arrays
LOW, HIGH = 3.25, 3.35
mask = (readings >= LOW) & (readings <= HIGH) # element-wise AND
passing = readings[mask]
n_fail = (~mask).sum()

# Threshold detection
hot_idx = np.where(readings > 3.305)[0]

# Rolling window - prefer sliding_window_view (NumPy 1.20+, memory-safe)
def rolling_mean(arr: np.ndarray, window: int) -> np.ndarray:
    # sliding_window_view is zero-copy and bounds-checked; no manual stride math.
    windows = np.lib.stride_tricks.sliding_window_view(arr, window_shape=window)
    return windows.mean(axis=1)

# as_strided is the older alternative (still common in the wild) but dangerous:
# incorrect shape/strides can read out-of-bounds memory without raising an error.
# Only use it when sliding_window_view is unavailable or you need non-contiguous strides.
# shape = (arr.size - window + 1, window)
# strides = (arr.strides[0], arr.strides[0])
# windows = np.lib.stride_tricks.as_strided(arr, shape=shape, strides=strides)

# Histogram for distribution analysis
counts, edges = np.histogram(readings, bins=20)
bin_centers = (edges[:-1] + edges[1:]) / 2 # midpoint of each bin

# Vectorized register field extraction (batch decode)
raw_regs = np.array([0x1043, 0x2086, 0x30C9], dtype=np.uint32)
speeds = (raw_regs >> 0) & 0xF # bits [3:0]
widths = (raw_regs >> 4) & 0x3F # bits [9:4]

# Cumulative distribution (empirical CDF) -- useful for latency reporting
sorted_latencies = np.sort(readings)
cdf = np.arange(1, len(sorted_latencies) + 1) / len(sorted_latencies)

```

```

p50 = sorted_latencies[np.searchsorted(cdf, 0.50)]
p99 = sorted_latencies[np.searchsorted(cdf, 0.99)]

# np.diff: detect sudden jumps (link speed changes, power events)
deltas = np.diff(readings)
jump_idx = np.where(np.abs(deltas) > 0.005)[0] # indices where delta > threshold

# np.polyfit: linear drift check over time
t = np.arange(len(readings), dtype=float)
slope, intercept = np.polyfit(t, readings, deg=1)
# if abs(slope) > threshold_per_sample: flag as drift

```

1.22.1 pandas for Test Records

```

import pandas as pd

# Load a multi-run CSV
df = pd.read_csv("results.csv")

# Boolean indexing
failures = df[df["result"] == "FAIL"]
slow_runs = df[df["latency_us"] > 100]
nvme_fails = df[(df["device"] == "nvme0") & (df["result"] == "FAIL")]

# Aggregation by device
summary = (
    df.groupby("device")
    .agg(
        runs=("iteration", "count"),
        mean_bw=("bandwidth_GBps", "mean"),
        p95_latency=("latency_us", lambda s: s.quantile(0.95)),
        fail_count=("result", lambda s: (s == "FAIL").sum()),
    )
    .reset_index()
)

# Pivot table for speed vs queue_depth heatmap
pivot = df.pivot_table(
    values="bandwidth_GBps",
    index="speed",
    columns="queue_depth",
    aggfunc="mean",
)

# Save
summary.to_csv("summary.csv", index=False)

# Merge baseline with current run
baseline = pd.read_csv("baseline.csv")
current = pd.read_csv("current.csv")
merged = baseline.merge(current, on=["device", "speed", "queue_depth"],
    suffixes=("_base", "_cur"))
merged["bw_delta_pct"] = (
    (merged["bandwidth_GBps_cur"] - merged["bandwidth_GBps_base"])
    / merged["bandwidth_GBps_base"] * 100
)
regressions = merged[merged["bw_delta_pct"] < -5.0]

```

1.22.2 Practical pandas Patterns for Multi-Device Test Runs

```
import pandas as pd

# --- Loading multiple per-device JSON result files into one DataFrame ---
import json
from pathlib import Path

def load_run(run_dir: Path) -> pd.DataFrame:
    frames = []
    for jf in sorted(run_dir.glob("*.json")):
        data = json.loads(jf.read_text())
        frames.append(pd.json_normalize(data)) # flattens nested dicts
    return pd.concat(frames, ignore_index=True) if frames else pd.DataFrame()

df = load_run(Path("/results/run_2024-01-15"))

# --- Add pass/fail column based on spec limits ---
SPEC = {"bandwidth_GBps": (3.0, None), "latency_us": (None, 150)}

def apply_spec(df: pd.DataFrame, spec: dict) -> pd.DataFrame:
    df = df.copy()
    mask = pd.Series(True, index=df.index)
    for col, (lo, hi) in spec.items():
        if col not in df.columns:
            continue
        if lo is not None:
            mask &= df[col] >= lo
        if hi is not None:
            mask &= df[col] <= hi
        df["pass"] = mask
    return df

df = apply_spec(df, SPEC)

# --- Detect first failure per device ---
first_fail = (
    df[~df["pass"]]
    .sort_values("iteration")
    .groupby("device")
    .first()
    .reset_index()[["device", "iteration", "bandwidth_GBps", "latency_us"]]
)

# --- Pivot + percent-from-nominal heatmap ---
nominal_bw = 6.5 # GBps reference
heat = df.pivot_table(
    values="bandwidth_GBps",
    index="speed",
    columns="queue_depth",
    aggfunc="mean",
)
heat_pct = (heat / nominal_bw - 1) * 100 # percent deviation from nominal

# --- pd.cut: bin latency readings into spec tiers ---
bins = [0, 50, 100, 150, float("inf")]
labels = ["excellent", "good", "marginal", "fail"]
df["latency_tier"] = pd.cut(df["latency_us"], bins=bins, labels=labels, right=False)
print(df["latency_tier"].value_counts())
```

`pd.json_normalize` flattens nested dicts (e.g., `{"metrics": {"bw": 6.8}}` becomes column `metrics.bw`)
— eliminates the manual unpacking loop. `pd.cut` is cleaner than a chain of `np.where` for tiered spec checking.

1.23 OOP Patterns for Test Infrastructure

1.23.1 Abstract Base Class for Instrument Hierarchy

```
from abc import ABC, abstractmethod
from typing import Optional
import time, logging

log = logging.getLogger(__name__)

class Instrument(ABC):
    """Base class for all DUT-connected instruments."""

    def __init__(self, resource: str, timeout_s: float = 5.0):
        self.resource = resource
        self.timeout_s = timeout_s
        self._connected = False

    @abstractmethod
    def connect(self) -> None: ...

    @abstractmethod
    def disconnect(self) -> None: ...

    @abstractmethod
    def reset(self) -> None: ...

    def __enter__(self):
        self.connect()
        return self

    def __exit__(self, *_):
        self.disconnect()

    @property
    def is_connected(self) -> bool:
        return self._connected

    @classmethod
    def from_config(cls, cfg: dict) -> "Instrument":
        """Factory: instantiate from a YAML config dict."""
        return cls(cfg["resource"], timeout_s=cfg.get("timeout", 5.0))

    def measure_with_retry(
        self,
        measure_fn,
        retries: int = 3,
        delay_s: float = 0.5,
    ):
        last = None
        for attempt in range(retries):
            try:
                return measure_fn()
            except Exception as exc:
```

```

last = exc
log.warning("%s.measure retry %d/%d: %s",
self.__class__.__name__, attempt + 1, retries, exc)
time.sleep(delay_s)
raise last

```

1.23.2 @property and Validation

```

class SpecWindow:
    def __init__(self, low: float, high: float):
        if low > high:
            raise ValueError(f"low={low} > high={high}")
        self._low = low
        self._high = high

    @property
    def low(self) -> float:
        return self._low

    @low.setter
    def low(self, v: float):
        if v > self._high:
            raise ValueError(f"new low={v} > high={self._high}")
        self._low = v

    def contains(self, v: float) -> bool:
        return self._low <= v <= self._high

    def __repr__(self) -> str:
        return f"SpecWindow(low={self._low}, high={self._high})"

```

1.23.3 MRO and Mixins

Python uses C3 linearization for Method Resolution Order (MRO). When using multiple inheritance, keep base classes narrow (mixins), avoid diamond-of-death data attributes, and always use `super()`.

```

class LogMixin:
    def log_info(self, msg, *args):
        logging.getLogger(type(self).__name__).info(msg, *args)

class RetryMixin:
    def with_retry(self, fn, times=3, delay=0.5):
        for i in range(times):
            try:
                return fn()
            except Exception:
                if i == times - 1:
                    raise
            time.sleep(delay)

class PowerSupply(LogMixin, RetryMixin, Instrument):
    def measure_voltage(self):
        return self.with_retry(lambda: float(self.query("MEAS:VOLT?")))

```

1.24 SQLite for Test Results

SQLite is a serverless, file-based database: zero setup, the whole database is one file, and it survives restarts. Ideal for a single-server dashboard with one writer and fast reads. Reach for PostgreSQL/MySQL only when many clients must write concurrently.

```
import sqlite3
import json, time

conn = sqlite3.connect("dashboard.db") # opens or creates the file (no server process)

conn.execute("""
    CREATE TABLE IF NOT EXISTS stations (
        hostname TEXT PRIMARY KEY,
        data TEXT,
        updated REAL
    )
""")

# Upsert: insert, or overwrite the row if the primary key already exists
conn.execute(
    "INSERT OR REPLACE INTO stations (hostname, data, updated) VALUES (?, ?, ?)",
    (hostname, json.dumps(data), time.time()),
)
conn.commit() # writes to disk; without it the change is lost
```

The `?` placeholders are **parameterized queries**: they prevent SQL injection and handle quoting. Never build SQL by string interpolation/f-strings.

```
# Query one row
cur = conn.execute("SELECT data FROM stations WHERE hostname = ?", (hostname,))
row = cur.fetchone() # a tuple, or None if not found
if row:
    station = json.loads(row[0])

# Query many rows + aggregate
cur = conn.execute("""
    SELECT keyword, AVG(duration_s) AS avg_s
    FROM keyword_timings
    WHERE model = ?
    GROUP BY keyword
""", (model,))
averages = {kw: avg for kw, avg in cur.fetchall()} # fetchall returns a list of tuples

conn.close()
```

Set `conn.row_factory = sqlite3.Row` to access columns by name (`row["hostname"]`) instead of by index. Use a `with conn:` block to wrap a transaction that auto-commits on success and rolls back on exception.

1.25 FastAPI — the Station Dashboard Pattern

FastAPI is a modern Python web framework. A typical use is a monitoring dashboard: each test station POSTs a heartbeat with its status, and the server stores it and serves a live view.

```
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel
from typing import Any, Optional
```



```

app = FastAPI(title="Station Dashboard")
# FastAPI() is the app object; routes are registered on it. On each request, FastAPI
# finds the handler whose path/method match and calls it.

@app.get("/api/stations")
def list_stations():
    # GET = read, no side effects. Returning a dict/list -> FastAPI serializes it to JSON.
    return [{"hostname": "STATION-1", "state": "RUN"}]

```

1.25.1 Pydantic models: automatic request validation

You declare the expected shape as a class; FastAPI validates the incoming JSON against it and rejects bad input with a 422 automatically.

```

class HeartbeatPayload(BaseModel):
    model_config = {"extra": "allow"} # accept unknown fields (forward-compatible payloads)
    hostname: str # required; must be a string
    state: str = "IDLE" # optional, defaults to "IDLE"
    keyword_index: int = 0 # optional int (bad type -> 422)
    failures: list[Any] = [] # optional list
    actuator_connected: Optional[bool] = None

@app.post("/heartbeat")
def receive_heartbeat(payload: HeartbeatPayload):
    # If validation fails, the function is never called; FastAPI returns 422.
    data = payload.model_dump() # convert the model back to a plain dict
    db.upsert(payload.hostname, data)
    return {"status": "ok"}

```

1.25.2 Path and query parameters

```

@app.get("/api/stations/{hostname}")
def get_station(hostname: str): # {hostname} in the path -> path parameter
    station = db.get(hostname)
    if station is None:
        raise HTTPException(status_code=404, detail="Station not found")
    return station

@app.get("/api/summary")
def summary(period: str = "24h"): # not in the path -> query parameter (?period=week)
    if period not in {"24h", "week", "month"}:
        raise HTTPException(status_code=400, detail="Invalid period")
    return db.summary(period)

```

1.25.3 Serving static files and the dashboard page

```

from fastapi.staticfiles import StaticFiles
from fastapi.responses import FileResponse

app.mount("/static", StaticFiles(directory="static"), name="static") # /static/app.js etc.

@app.get("/")
def dashboard():
    return FileResponse("static/index.html") # the browser UI; correct Content-Type set for you

```

1.25.4 Lifespan (startup/shutdown)

```
import asyncio
from contextlib import asynccontextmanager

@asynccontextmanager
async def lifespan(app: FastAPI):
    task = asyncio.create_task(background_purge()) # before yield = startup
    yield
    task.cancel() # after yield = shutdown
    try:
        await task
    except asyncio.CancelledError:
        pass

app = FastAPI(lifespan=lifespan)
```

1.25.5 Running it and key concepts

```
uvicorn server:app --reload --host 0.0.0.0 --port 8080 # dev: auto-reload
uvicorn server:app --host 0.0.0.0 --port 8080 --workers 1 # prod (single worker if state is
↳ in-memory)
```

Talking points if asked: sync route handlers (`def`, not `async def`) run in a thread pool, which is fine for fast handlers doing dict ops and a SQLite write. Pydantic removes all the manual if "hostname" not in data validation. FastAPI auto-generates interactive OpenAPI docs at `/docs`. Use a single worker when the server keeps in-memory caches, since multiple workers would each hold a separate copy.

1.26 PySide6 — Desktop Instrument GUIs

PySide6 is the official Qt binding for Python (LGPL, free for commercial use). It is the standard choice for responsive desktop instrument-control GUIs with live graphing, where a web app would add unacceptable HTTP latency between operator and hardware.

1.26.1 Signals and slots: Qt's event system

A **signal** is an event an object can emit; a **slot** is a function that receives it. Objects connect signals to slots, decoupling emitter from receiver.

```
from PySide6.QtCore import QObject, Signal

class ServoController(QObject):
    state_changed = Signal(str, str) # declares a signal carrying (state, detail)
    error_occurred = Signal(str)
    stopped = Signal() # carries nothing

    def go(self):
        self.state_changed.emit("RAMP", "Ramping to 65 ft-lb") # notify every connected slot

servo = ServoController()
servo.state_changed.connect(lambda s, d: print(f"{s}: {d}")) # connect a slot
servo.error_occurred.connect(show_error_dialog) # many slots may connect
servo.go() # fires all connected slots in order
```

Emitting a signal with no connections is harmless (does nothing). Multiple slots can connect to one signal, and one slot can serve many signals.

1.26.2 QTimer: periodic work without blocking the UI

```
from PySide6.QtCore import QTimer

self.tick_timer = QTimer()
self.tick_timer.timeout.connect(self.handle_tick) # timeout fires every interval
self.tick_timer.setInterval(500) # milliseconds
self.tick_timer.start()
# ... later:
self.tick_timer.stop()
```

Use `QTimer` instead of `time.sleep()` in a loop: the timer fires on the main event loop, so the GUI stays responsive (buttons, repaints, dialogs work between ticks). `time.sleep()` freezes the entire UI. And use `QTimer` rather than a background thread when the callback touches widgets, because Qt widgets may only be modified from the main thread.

1.26.3 The loop-variable capture pattern in connections

```
from PySide6.QtWidgets import QCheckBox
for iface in ["RS232", "RS485", "ETH", "CAN", "ANALOG"]:
    cb = QCheckBox(iface)
    cb.toggled.connect(lambda checked, i=iface: self.on_toggle(i, checked))
    # i=iface captures the CURRENT iface; without it every callback would see "ANALOG".
```

1.26.4 Threading rule of thumb

Do slow I/O on a worker thread, then deliver results to the GUI by emitting a signal (Qt marshals it to the main thread). Never update widgets directly from a worker thread.

1.27 Modules, venv, and Packaging

1.27.1 Virtual Environments

```
python3 -m venv .venv
source .venv/bin/activate # macOS/Linux
# .venv\Scripts\activate.bat # Windows

pip install -r requirements.txt
pip install -e . # editable install of local package
```

Always activate a venv before running test scripts. The `-e .` (editable) install makes `import mypackage` resolve to your source tree without reinstalling.

1.27.2 pyproject.toml

```
[build-system]
requires = ["setuptools>=68", "wheel"]
build-backend = "setuptools.build_meta"
```

```

[project]
name = "zoox-compute-tests"
version = "0.1.0"
requires-python = ">=3.11"
dependencies = [
    "pytest>=7.4",
    "pyvisa>=1.14",
    "pyserial>=3.5",
    "numpy>=1.26",
    "pandas>=2.0",
    "PyYAML>=6.0",
]

[project.optional-dependencies]
dev = ["mypy", "ruff", "pytest-cov"]

[tool.pytest.ini_options]
testpaths = ["tests"]
addopts = "-ra -q --tb=short"

[tool.mypy]
python_version = "3.11"
strict = true

```

1.27.3 Package Layout

```

zoox-compute-tests/
src/
zoox_tests/
__init__.py
instruments/
__init__.py
serial_inst.py
visa_inst.py
parsers/
lspci.py
nvme.py
dmesg.py
fixtures/
conftest.py
tests/
test_nvme.py
test_pcie.py
pyproject.toml
requirements.txt

```

`__init__.py` marks a directory as a package. Relative imports (`from .parsers import lspci`) work within the package; use absolute imports in test files.

1.28 Worked Problems: Test-Domain Patterns

These are the recurring algorithm patterns that show up in Zoox platform test code. Each maps to a real problem in hardware test automation.

1.28.1 Sliding Window: Error Burst Detection

Detect windows where error rate exceeds a threshold — used in link stability monitoring.

```
from collections import deque
from datetime import datetime

def max_errors_in_window(log_lines: list[str], window_s: float) -> int:
    """Return the maximum number of errors in any window_s-second window."""
    times = []
    for line in log_lines:
        if "[ERROR]" in line:
            ts = datetime.strptime(line[:19], "%Y-%m-%d %H:%M:%S")
            times.append(ts.timestamp())

    if not times:
        return 0

    max_count = 0
    left = 0
    for right in range(len(times)):
        while times[right] - times[left] > window_s:
            left += 1
        max_count = max(max_count, right - left + 1)
    return max_count
```

1.28.2 Two-Pointer: Merge Sorted Measurement Runs

```
def merge_sorted_runs(a: list[float], b: list[float]) -> list[float]:
    result = []
    i = j = 0
    while i < len(a) and j < len(b):
        if a[i] <= b[j]:
            result.append(a[i]); i += 1
        else:
            result.append(b[j]); j += 1
    result.extend(a[i:])
    result.extend(b[j:])
    return result
```

1.28.3 Stack: Balanced Protocol Framing

Validate that protocol open/close tags are balanced — e.g., PCIe Transaction Layer Packet (TLP) framing in log analysis.

```
def is_balanced(s: str) -> bool:
    stack = []
    pairs = {"(": ")", "[": "]", "{": "}"
    for ch in s:
        if ch in "([{":
            stack.append(ch)
        elif ch in ")]}":
            if not stack or stack[-1] != pairs[ch]:
                return False
            stack.pop()
    return not stack
```

1.28.4 Hash Map: Two-Sum (Threshold Pairing in Readings)

Find all pairs of readings whose sum meets a target — used when looking for correlated events in two channels.

```
def two_sum_indices(readings: list[float], target: float) -> list[tuple[int, int]]:
    seen = {} # value -> index
    pairs = []
    for i, val in enumerate(readings):
        complement = target - val
        if complement in seen:
            pairs.append((seen[complement], i))
        seen[val] = i
    return pairs
```

1.28.5 Bit-Field Decoder: Register Status Report

Decode a 32-bit hardware status register into a structured dict — the everyday pattern for PCIe or Non-Volatile Memory Express (NVMe) register parsing.

```
from dataclasses import dataclass
from typing import ClassVar

@dataclass
class PCIeStatusReg:
    raw: int

    FIELDS: ClassVar[dict[str, tuple[int, int]]] = {
        "correctable_errors": (0, 8),
        "uncorrectable_errs": (8, 8),
        "link_bandwidth_mgmt": (16, 1),
        "link_auto_bw": (17, 1),
        "link_speed": (18, 4),
        "link_width": (22, 6),
    }

    def decode(self) -> dict[str, int]:
        return {
            name: (self.raw >> lsb) & ((1 << width) - 1)
            for name, (lsb, width) in self.FIELDS.items()
        }

reg = PCIeStatusReg(0x0043_0200)
print(reg.decode())
```

1.28.6 State Machine: Link Training State Parser

Parse dmesg output tracking PCIe link training state transitions.

```
def parse_link_states(log_lines: list[str]) -> list[str]:
    """Extract ordered list of PCIe link training states from dmesg."""
    import re
    STATE_PAT = re.compile(r"PCIe.*?link.*?state[:\s]+(\w+)", re.IGNORECASE)
    states = []
    prev = None
    for line in log_lines:
        m = STATE_PAT.search(line)
        if m:
            state = m.group(1).upper()
            if state != prev:
```

```

states.append(state)
prev = state
return states

```

1.28.7 Anagram Groups: Duplicate Config Detection

Group config file names that are anagrams (same characters, different order) — useful when auditing config sets for accidental duplicates.

```

from collections import defaultdict

def group_anagrams(names: list[str]) -> list[list[str]]:
    groups = defaultdict(list)
    for name in names:
        key = "".join(sorted(name.lower()))
        groups[key].append(name)
    return [g for g in groups.values() if len(g) > 1]

```

1.28.8 OOP Composite: Test Suite with Mixed Test Types

```

from abc import ABC, abstractmethod

class TestNode(ABC):
    def __init__(self, name: str):
        self.name = name

    @abstractmethod
    def run(self) -> dict: ...

    @abstractmethod
    def count(self) -> int: ...

class LeafTest(TestNode):
    def __init__(self, name: str, fn):
        super().__init__(name)
        self._fn = fn

    def run(self) -> dict:
        import traceback
        try:
            self._fn()
            return {"name": self.name, "result": "PASS"}
        except AssertionError as e:
            return {"name": self.name, "result": "FAIL", "reason": str(e)}
        except Exception:
            return {"name": self.name, "result": "ERROR",
                    "reason": traceback.format_exc()}

    def count(self) -> int:
        return 1

class TestSuite(TestNode):
    def __init__(self, name: str):
        super().__init__(name)
        self._children: list[TestNode] = []

    def add(self, node: TestNode) -> "TestSuite":
        self._children.append(node)

```

```

return self

def run(self) -> dict:
    results = [child.run() for child in self._children]
    passed = sum(1 for r in results if r["result"] == "PASS")
    return {
        "name": self.name,
        "result": "PASS" if passed == len(results) else "FAIL",
        "children": results,
        "summary": f"{passed}/{len(results)}",
    }

def count(self) -> int:
    return sum(c.count() for c in self._children)

```

1.29 Quick Reference: Pitfalls and Idioms

Pattern	Wrong	Right
Float compare	<code>x == 3.3</code>	<code>math.isclose(x, 3.3, abs_tol=0.01)</code>
Mutable default	<code>def f(lst=[]):</code>	<code>def f(lst=None): lst = lst or []</code>
Late-bind lambda	<code>[lambda: i for i in r]</code>	<code>[lambda i=i: i for i in r]</code>
numpy boolean	<code>(a > 0) and (a < 10)</code>	<code>(a > 0) & (a < 10)</code>
groupby without sort	<code>groupby(data, key)</code>	<code>groupby(sorted(data, key=key), key)</code>
subprocess injection	<code>run("lspci -s {bdf}", shell=True)</code>	<code>run(["lspci", "-s", bdf])</code>
YAML unsafe load	<code>yaml.load(f)</code>	<code>yaml.safe_load(f)</code>
patch target	<code>patch("mod.original_module.func")</code>	<code>patch("mod.tests.func")</code> (where used)
is for equality	<code>x is "hello"</code>	<code>x == "hello"</code>
Bare except	<code>except:</code>	<code>except Exception:</code>
Suppress all errors	<code>except Exception: pass</code>	<code>log + re-raise or specific handling</code>
<code>str(Path(...))</code>	<code>str(Path("/a") + "/b")</code>	<code>str(Path("/a") / "b")</code>
pyproject build-backend	<code>"setuptools.backends.legacy:build"</code>	<code>"setuptools.build_meta"</code>
nvme list JSON (≥ 2.11)	<code>data["Devices"][0]["DevicePath"]</code>	check for "Subsystems" key first
rolling window (numpy)	<code>np.lib.stride_tricks.as_strided(...)</code>	<code>sliding_window_view(arr, window) (1.20+)</code>
sudo in CI/daemon	<code>subprocess.run(["sudo", ...])</code> hangs	add <code>--non-interactive</code> ; configure NOPASSWD
TimeoutExpired output	<code>discard exc.stdout</code>	<code>exc.stdout</code> or <code>""</code> has partial output
fixture vs parametrize	<code>@parametrize</code> for resource variants	<code>@fixture(params=...)</code> for resources, <code>@parametrize</code> for data
<code>tmp_path</code> in session fixture	<code>tmp_path</code> (function-scoped, fails)	<code>tmp_path_factory.mktemp(...)</code>

Chapter 2

Bash and Shell Scripting

Shell fluency is a daily multiplier on this job. Python orchestrates tests; Bash is how you interrogate hardware the moment it misbehaves — “the board failed” becomes “endpoint 0000:03:00.0 trained Gen3 x8, BadTLP Advanced Error Reporting (AER) counter climbing, here is the sysfs value” in the time it takes to type one pipeline. This chapter is a working reference organized around the problems you actually hit on the station.

2.0.1 When to Reach for Bash vs. Python

The boundary matters. Choosing wrong costs you either time (bash where you need Python) or maintainability (Python where bash would have been five lines).

Reach for Bash when	Reach for Python when
Interactive hardware query at a prompt	Structured logic — state machines, retries with backoff
Piping Command-Line Interface (CLI) tools (<code>lspci</code> , <code>nvme</code> , <code>ethtool</code> , <code>dmesg</code>)	Instrument comms (serial/SCPI/USB/TCP)
System-admin one-offs under ~100 lines	Parsing structured data robustly (JSON, CSV, XML)
Cron/systemd wrappers, Continuous Integration (CI) scaffolding	Anything a teammate will maintain
“Do I see 4 GPUs right now?”	“Assert exactly 4 GPUs every run, log to DB, fail the test”
Environment setup, deployment glue	pytest fixtures, PySide6 GUIs

The one-sentence rule: **Bash glues programs together and pokes at the system; Python owns logic, state, and anything maintained.** The moment you find yourself building data structures or needing real error handling in Bash, stop and rewrite in Python.

In practice the two interoperate constantly. Your ATF is Python; it shells out for the parts where no clean library exists:

```
# Bash one-liner at the prompt -- fastest path to "do I have 4 GPUs?":  
lspci | grep -i nvidia | wc -l
```

```
# Same check, hardened, inside a pytest fixture:  
import subprocess  
result = subprocess.run(["lspci"], capture_output=True, text=True, timeout=10)  
gpu_count = result.stdout.count("NVIDIA")  
assert gpu_count == 4, f"Expected 4 GPUs, found {gpu_count}"
```

And Bash scripts call back into Python for the parts Bash handles badly:

```
# Bash orchestrates the station; Python does structured analysis:
rate=$(python3 /opt/atf/analyze.py --input "$RESULTS_CSV" --metric pass_rate)
[[ "${rate%.*}" -ge 95 ]] || die "Yield below threshold: ${rate}%"
```

Reading sysfs attributes directly from Python (`pathlib.Path.read_text()`) is usually *better* than shelling out to parse `lspci` text — zero subprocess overhead, no parsing fragility. On the bench, interactively, Bash wins for speed of thought.

2.1 Shell Mechanics

2.1.1 Variables and Quoting

```
# Assignment: NO SPACES around =. "X = 1" tries to RUN a command named X.
DEVICE="/dev/nvme0n1"
BDF="0000:03:00.0"
LOG="/var/log/mfg_test/${date +%Y%m%d_%H%M%S}.log"
readonly MAX_RETRIES=5 # readonly: any later reassignment is an error
declare -i counter=0 # integer type: arithmetic happens automatically
export PATH="/opt/atf/bin:$PATH" # export: child processes inherit this variable

# Reference with ${}: braces are optional but eliminate ambiguity.
echo "$DEVICE"
echo "${BDF}_suffix" # WITHOUT braces: $BDF_suffix would look up the var named BDF_suffix
```

Quoting is the single largest source of Bash bugs:

```
echo "$DEVICE" # double quotes: variables EXPAND -> /dev/nvme0n1
echo '$DEVICE' # single quotes: LITERAL, no expansion -> $DEVICE
echo "$BDF and ${date}" # double quotes still allow $(...) substitution
echo 'cost is $5' # single quotes protect literal dollar signs

# ALWAYS double-quote variable expansions. Unquoted = word-split + glob-expand:
path="/mnt/my board/file"
rm $path # runs: rm /mnt/my board/file (THREE args; disaster)
rm "$path" # runs: rm "/mnt/my board/file" (one arg; correct)
```

2.1.2 Command Substitution and Arithmetic

```
# Prefer $(...) over backticks: nestable, readable, unambiguous.
SPEED=$(cat /sys/bus/pci/devices/$BDF/current_link_speed)
GPU_COUNT=$(lspci | grep -c 'NVIDIA')
NOW=$(date +%s) # epoch seconds

# Integer arithmetic with (( )) or $( ( )). Bash has NO floating point.
count=$((count + 1))
pct=$(( passed * 100 / total )) # integer division: 7/2 == 3, not 3.5
(( failed > 0 )) && echo "some failed" # (( )) as a test: nonzero == true

# Floating point: pipe to bc or use awk BEGIN.
pct=$(echo "scale=2; $passed * 100 / $total" | bc)
margin=$(awk -v a="$volts" -v b="$nominal" 'BEGIN{printf "%.3f", a-b}')

```

2.1.3 Parameter Expansion (Built-in String Surgery)

This replaces a surprising amount of `sed/awk`. These show up constantly when massaging BDFs, paths, and log lines:

```
echo "${BDF:-default}" # value of BDF, or "default" if unset/empty (no assignment)
echo "${BDF:=default}" # same, but ALSO assigns "default" to BDF
echo "${BDF:?must set}" # error and exit with "must set" if unset/empty (guard at top of script)
echo "${BDF:+yes}" # "yes" if BDF is set/non-empty, else empty
echo "${#BDF}" # string LENGTH
echo "${BDF^^}" # UPPERCASE (bash 4+)
echo "${BDF,,}" # lowercase (bash 4+)

# Substring removal. # strips a PREFIX, % strips a SUFFIX. Double the char for "longest match".
filepath="/var/log/test/results.csv"
echo "${filepath##*/}" # results.csv (## = longest leading */ -> basename equivalent)
echo "${filepath#*/}" # var/log/test/results.csv (# = shortest leading match)
echo "${filepath%/*}" # /var/log/test (% = shortest trailing /* -> dirname equivalent)
echo "${filepath%.csv}" # /var/log/test/results (strip known extension)
echo "${filepath##*.}" # csv (just the extension)

# In-place substitution (no sed needed for simple cases):
echo "${filepath/test/prod}" # /var/log/prod/results.csv (first match)
echo "${filepath//o/O}" # /var/log/test/results.csv (all matches: //)

# Substrings by offset:length:
s="0000:03:00.0"
echo "${s:5:2}" # 03 (2 chars from index 5 -- the bus number)
echo "${s:0:4}" # 0000 (the PCI domain)
```

2.1.4 Exit Codes

Every command returns an integer status. **0 = success, non-zero = failure**. This is the bedrock of pass/fail logic.

```
lspci -s "$BDF" &>/dev/null # discard all output; check only the exit status
echo $? # 0 = found, 1 = not found

# Use exit codes directly in conditionals -- no need to compare $? explicitly:
if lspci -s "$BDF" &>/dev/null; then echo "present"; else echo "absent"; fi

# Chain on success (&&) or failure (||):
mount /mnt/usb && cp results.csv /mnt/usb/ || echo "backup failed"

# Conventional codes:
# 0 success
# 1-125 application-defined failure
# 126 command exists but is not executable
# 127 command not found
# 128+N killed by signal N (130 = Ctrl+C/SIGINT, 137 = SIGKILL, 143 = SIGTERM)
exit 0 # PASS
exit 1 # generic FAIL
```

2.1.5 Redirection and File Descriptors

Three standard file descriptors: **0 = stdin, 1 = stdout, 2 = stderr**. Redirection rewires where they point. This matters enormously for logging test output cleanly.

```

command > file # stdout -> file (TRUNCATE / overwrite)
command >> file # stdout -> file (APPEND)
command 2> errlog # fd 2 (stderr) -> errlog
command 2>&1 # redirect stderr to wherever stdout currently points
command > out 2>&1 # ORDER MATTERS: stdout -> out first, THEN stderr -> same place
command &> file # bash shorthand: both stdout and stderr -> file
command < input.txt # stdin FROM file
command 2>/dev/null # discard only errors (when a sysfs file may not exist)
command &>/dev/null # discard everything (just give me the exit code)

# The ordering trap:
do_test > test.log 2>&1 # CORRECT: both streams land in test.log
do_test 2>&1 > test.log # WRONG: stderr goes to the original stdout (terminal),
# then stdout goes to the file. stderr never reaches the file.

```

2.1.6 Here-Documents and Here-Strings

```

# Here-doc: feed a multi-line block as stdin.
# QUOTE the delimiter to DISABLE variable expansion (everything is literal):
cat << 'EOF' > /etc/test/config.ini
[psu]
voltage = 48.0
current_limit = 20.0
EOF

# Unquoted delimiter: variables ARE expanded:
cat << EOF
Testing board $SERIAL at $(date)
Expected speed: $EXPECTED
EOF

# Here-string (<<<): a single string as stdin. Handy with grep/read/bc:
grep -q "16 GT/s" <<< "$SPEED" && echo "Gen4"
read -r dom bus dev func <<< "${BDF//[[:.]]/ }" # 0000:03:00.0 -> dom=0000 bus=03 dev=00 func=0 (4
↪ fields)

```

2.1.7 Pipes and tee

A pipe (|) connects stdout of one command to stdin of the next. Process substitution (< <(...)) lets you feed command output as a file to a redirect, which keeps the receiving shell in the current process rather than a subshell.

```

lspci | grep -i nvidia | wc -l # count NVIDIA devices
dmesg | tail -50 | grep -i error # recent kernel errors

# tee: write to a file AND pass the stream onward (log without losing data):
run_test.sh | tee test.log | grep -i fail # full output to file, failures to terminal
echo "started $(date)" | tee -a runs.log # -a appends instead of truncating

# Process substitution: feed command output as a "file" via a named pipe:
diff <(lspci -t) <(cat expected_topology.txt) # compare live topology vs saved baseline
while IFS= read -r line; do
  process "$line"
done <<(lspci -d 10de:) # loop body runs in THIS shell, not a subshell -- counters survive

```

The subshell trap is one of the most common Bash bugs in test scripts:

```
count=0
lspci -d 10de: | while read -r _; do (( count++ )); done
echo "$count" # prints 0 -- the pipe created a subshell; count was discarded!

# Fix: use process substitution so the while loop runs in the current shell:
while read -r _; do (( count++ )); done <<(lspci -d 10de:)
echo "$count" # correct
```

2.2 Conditionals and Tests

Use `[[ ... ]]` (a Bash keyword) rather than `[ ... ]` (POSIX `test`). `[[ ]]` does not word-split or glob-expand its operands, supports `&&/||/=`, and avoids a class of subtle bugs from unquoted variables.

2.2.1 String, Numeric, and File Tests

```
# String tests:
[[ "$status" == "PASS" ]] # equality (== or =)
[[ "$status" != "FAIL" ]] # inequality
[[ -z "$var" ]] # true if empty or unset (zero length)
[[ -n "$var" ]] # true if non-empty
[[ "$str" == *.log ]] # GLOB match (right side is a pattern: * ? [ ])
[[ "$bdf" =~ ^[0-9a-f]{4}:[0-9a-f]{2}:[0-9a-f]{2}\.[0-9]$ ]] # =~ REGEX (ERE)
echo "${BASH_REMATCH[0]}" # after =~, captured groups land in BASH_REMATCH[]
# DO NOT quote the right side of =~ if it contains regex metacharacters --
# quoting forces a literal string comparison instead.

# Numeric tests (use -eq family inside [[ ]] -- == compares as strings):
[[ "$count" -eq 4 ]] # equal
[[ "$count" -ne 0 ]] # not equal
[[ "$count" -gt 10 ]] # greater than
[[ "$count" -lt 100 ]] # less than
[[ "$count" -ge 4 ]] # >=
[[ "$count" -le 4 ]] # <=
# Or use (( )) arithmetic context, which reads more naturally:
(( count == 4 ))
(( count > 10 && count < 100 ))
(( temp >= 70 )) && echo "OVERHEAT"
# Gotcha: a leading-zero operand is parsed as OCTAL in a numeric context, so
# [[ "08" -eq 8 ]] ERRORS ("value too great for base" -- 8 is not an octal digit),
# it is not TRUE. Force base 10 with (( 10#08 == 8 )). [[ "08" == 8 ]] is FALSE (string).

# File tests:
[[ -e "$path" ]] # exists (file, dir, symlink, device)
[[ -f "$file" ]] # exists AND is a regular file
[[ -d "$dir" ]] # exists AND is a directory
[[ -L "$path" ]] # is a symbolic link
[[ -b "$dev" ]] # is a block device (e.g. /dev/nvme0n1)
[[ -c "$dev" ]] # is a character device (e.g. /dev/ttyUSB0)
[[ -r "$file" ]] # readable
[[ -w "$file" ]] # writable
[[ -x "$file" ]] # executable
[[ -s "$file" ]] # exists AND has size > 0 (non-empty)
[[ "$f1" -nt "$f2" ]] # f1 is NEWER than f2 (modification time)

# Manufacturing guard:
```

```
[[ -c /dev/ttyUSB0 ]] || die "Programmer cable not detected on /dev/ttyUSB0"
```

2.2.2 Branching

```
# Combining conditions:
[[ "$speed" == "16 GT/s" && "$width" == "16" ]] # AND (both must be true)
[[ "$speed" != "16 GT/s" || "$width" != "16" ]] # OR (either is true)
[[ ! -f "$lockfile" ]] # NOT

if [[ "$status" == "PASS" ]]; then
    log "OK"
elif [[ "$status" == "FAIL" ]]; then
    log "FAILED" "ERROR"
else
    log "UNKNOWN status: $status" "WARN"
fi

# case (switch) -- cleaner than a long if/elif chain; patterns are GLOBS:
case "$device_type" in
    gpu) test_gpu "$bdf" ;;
    nvme|ssd) test_nvme "$bdf" ;; # multiple patterns with |
    nic|eth*) test_nic "$bdf" ;; # glob: anything starting with "eth"
    *) die "Unknown device type: $device_type" ;; # default/fallthrough
esac
```

2.3 Loops

```
# for over a word list (command substitution, glob, or array):
for dev in $(lspci -d 10de: -D | awk '{print $1}'); do # -D shows full BDF with domain
    speed=$(cat "/sys/bus/pci/devices/$dev/current_link_speed" 2>/dev/null)
    echo "$dev: $speed"
done

# C-style for (counter loops):
for ((i = 0; i < 16; i++)); do
    echo "Lane $i"
done

# for over files via a glob -- always guard against an empty match:
for file in /var/log/mfg_test/*.log; do
    [[ -f "$file" ]] || continue # if glob matched nothing it stays literal; skip it
    process "$file"
done

# while with an arithmetic condition (retry loop with backoff):
retries=0
while (( retries < MAX_RETRIES )); do
    try_connect && break
    (( retries++ ))
    sleep 1
done
(( retries < MAX_RETRIES )) || die "Failed to connect after $MAX_RETRIES tries"

# until (runs WHILE the condition is FALSE) -- wait for DUT to come up:
```

```
until ping -c1 -W1 192.168.100.10 &>/dev/null; do
    echo "Waiting for device to boot..."
    sleep 2
done
```

2.3.1 Reading Input Line by Line (the Right Way)

```
# while IFS= read -r line is the correct idiom.
# IFS= stops leading/trailing whitespace from being trimmed.
# -r stops backslashes from being interpreted as escape sequences.

while IFS= read -r line; do
    echo "Processing: $line"
done < /opt/test/serials.txt

# Read a COMMAND's output line by line. Use process substitution <<(...)
# so the loop body runs in the CURRENT shell (variables set inside survive after the loop):
while IFS= read -r line; do
    bdf=$(awk '{print $1}' <<< "$line")
    echo "Found device: $bdf"
done << (lspci -d 10de:)
```

2.4 The Text-Processing Toolkit

grep filters, sed edits streams, awk handles fields/aggregation, cut/sort/uniq/tr do quick column and frequency work, find/xargs act on files. Master these and you can extract any value from lspci, nvme, ethtool, dmesg, or a results CSV.

2.4.1 grep

```
grep 'error' /var/log/syslog # case-sensitive (BRE by default)
grep -i 'error' log.txt # case-INsensitive
grep -c 'FAIL' results.txt # COUNT matching lines (not print them)
grep -n 'TODO' main.py # prefix each hit with its line NUMBER
grep -rn 'pyvisa' src/ # -r recursive, -n line numbers (search a whole tree)
grep -v 'DEBUG' app.log # INVERT: print lines that do NOT match
grep -l 'error' *.log # list only FILENAMES that contain a match
grep -L 'PASS' *.log # filenames that do NOT contain a match (find failures)
grep -w 'fail' log.txt # whole WORD only (won't match "failure" or "default")
grep -A3 -B2 'error' log.txt # context: 3 lines After, 2 Before each match
grep -C2 'error' log.txt # -C: context both sides (2 before AND 2 after)
grep -o 'Gen[0-9]' log.txt # -o: print ONLY the matched text, not the whole line
grep -q 'PASS' log.txt # -q: QUIET -- no output, just the exit code (use in if tests)
grep -m1 'error' big.log # stop after the FIRST match (fast on huge logs)

# Regex flavors:
grep -E 'error|warn|critical' log # -E: Extended Regex (ERE): | + ? () without backslashes
grep -P '\d{4}-\d{2}-\d{2}' log # -P: Perl-Compatible Regex: \d \w \s lookarounds

# The killer PCRE trick: -oP with \K to extract a value after an anchor.
# \K discards everything matched before it, so the anchor is required but not printed.
grep -oP 'Speed \K[^\,]+' lspci.txt # -> "16GT/s" from "LnkSta: Speed 16GT/s, Width x16"
grep -oP 'Width \Kx[0-9]+' lspci.txt # -> "x16"
grep --color=always 'error' f | less -R # keep color highlights when paging
```



```
# Pull link speed and width for one device:
lspci -s 03:00.0 -vvv | grep -oP 'LnkSta:.*Speed \K[^\,]+' # -> 16GT/s
lspci -s 03:00.0 -vvv | grep -oP 'LnkSta:.*Width \Kx[0-9]+' # -> x16
```

2.4.2 sed

sed edits text as it streams past. Substitution (s///) is the core; it also prints line ranges, deletes lines, and inserts text.

```
sed 's/old/new/' file.txt # substitute FIRST occurrence per line
sed 's/old/new/g' file.txt # g: GLOBAL -- every occurrence on each line
sed 's/old/new/gi' file.txt # gi: global + case-insensitive
sed -i 's/old/new/g' file.txt # -i: IN-PLACE edit (edits the file; no stdout)
sed -i.bak 's/old/new/g' file.txt # -i.bak: in-place, save original as file.txt.bak

sed -n '10,20p' file.txt # -n suppresses default print; p prints lines 10-20
sed -n '/START/,/END/p' file.txt # print everything between two patterns (inclusive)
sed -n '$p' file.txt # print only the last line ($ = last line in sed)
sed '/^$/d' file.txt # DELETE blank lines
sed '/^#/d' file.txt # delete comment lines (starting with #)
sed '/^\s*$/d; /\s*$/d' file.txt # strip comments AND blank lines (two commands)
sed 's/[[:space:]]*$//' file.txt # strip TRAILING whitespace
sed '1i\HEADER' file.txt # INSERT "HEADER" before line 1
sed '$a\FOOTER' file.txt # APPEND "FOOTER" after the last line
sed '3d' file.txt # delete line 3
sed -n '2~3p' file.txt # print every 3rd line starting at line 2 (GNU step)

# Capture groups with -E (ERE). \1 \2 ... refer to parenthesized groups.
# You can use any delimiter after s: s#...#...# avoids escaping / in paths.
sed -E 's/([0-9]+)GT\s/Speed=\1/g' lspci.txt # "16GT/s" -> "Speed=16"
sed -E 's#^(0000:[0-9a-f:.]+).*#\1#' devs.txt # keep only the BDF

echo "0000:03:00.0" | sed 's/[.:]/ /g' # "0000 03 00 0" -- split BDF into fields
```

2.4.3 awk

awk reads line by line, splits each line into fields (\$1, \$2, ...; \$0 = whole line; NF = field count; NR = record/line number), and runs pattern { action } rules. It is the right tool for columnar data and aggregation.

```
awk '{print $1}' file.txt # first whitespace-delimited field of every line
awk '{print $NF}' file.txt # the LAST field ($NF = field at position NF)
awk '{print $(NF-1)}' file.txt # second-to-last field
awk -F: '{print $1, $3}' /etc/passwd # -F: set the field separator (here ":") -> username, uid
awk -F, '{print $1, $4}' results.csv # CSV with comma separator
awk -F'\t' '{print $2}' data.tsv # tab-separated

awk 'NR > 2' nvme_list.txt # skip the first 2 header lines -- print the rest
awk 'NR >= 10 && NR <= 20' file.txt # line range (equivalent to sed -n '10,20p')
awk 'NR % 2 == 0' file.txt # even-numbered lines only

awk '$3 > 100' data.txt # print lines where field 3 (numeric) exceeds 100
awk '$4 == "FAIL"' results.csv # lines where field 4 equals the string FAIL
awk -F, '$4=="FAIL" {print $1, $2}' # combine condition with custom print action
awk '/error/ {print NR: " $0}' # for lines matching /error/, print "lineno: full line"
awk '!/DEBUG/' app.log # print lines that do NOT match (negate with !)

# Aggregation -- awk shines here. BEGIN runs before input, END runs after.
```



```

awk '{sum += $3} END {print "Total:", sum}' data.txt
awk '{sum += $1; n++} END {printf "avg=%.2f\n", sum/n}' nums.txt
awk -F, 'NR>1 {c[$4]++} END {for (k in c) print k, c[k]}' r.csv # frequency by category
awk '{gsub(/old/, "new"); print}' file # global substitution then print

# ALWAYS pass shell variables in with -v (never interpolate "$x" inside the awk program):
threshold=70
awk -v t="$threshold" '$2 > t {print $1, "OVERHEAT", $2}' temps.txt

```

Multi-line awk to walk `lspci -vvv` output and emit a clean Bus/Device/Function (BDF)/Speed/Width table:

```

lspci -vvv 2>/dev/null | awk '
BEGIN { OFS = "\t"; print "BDF", "Speed", "Width" }
/^[0-9a-f]/ { bdf = $1 } # non-indented hex line = new device
/LnkSta:/ && !/LnkSta2:/ { # link-status line (skip secondary LnkSta2)
gsub(/,/,"") # drop commas so fields are clean
for (i = 1; i <= NF; i++) {
if ($i == "Speed") speed = $(i+1)
if ($i == "Width") width = $(i+1)
}
print bdf, speed, width
}
'

```

2.4.4 cut, sort, uniq, tr

```

# cut: extract columns by delimiter or character position
cut -d: -f1 /etc/passwd # field 1, ":" delimited (all usernames)
cut -d, -f2,4 data.csv # fields 2 AND 4
cut -d, -f2-5 data.csv # fields 2 through 5
cut -c1-10 file.txt # characters 1-10 of each line

# sort
sort file.txt # ascending, lexicographic
sort -n nums.txt # NUMERIC sort (9 < 10; not the case lexicographically)
sort -h sizes.txt # human-numeric: understands 2K, 5M, 1G
sort -r file.txt # REVERSE (descending)
sort -u file.txt # sort and de-duplicate in one step
sort -t, -k3 -rn data.csv # CSV: field 3, reverse numeric
sort -k2,2 -k1,1n file # multi-key: by field 2, then numerically by field 1

# uniq -- ONLY collapses ADJACENT duplicates; always sort first.
sort file | uniq # remove duplicate lines
sort file | uniq -c # prefix each unique line with its count
sort file | uniq -c | sort -rn # the classic FREQUENCY-COUNT pipeline
sort file | uniq -d # show ONLY lines that appeared more than once
sort file | uniq -u # show ONLY lines that appeared exactly once

# tr -- translate or delete characters (operates on stdin only)
tr 'a-z' 'A-Z' < file # uppercase
tr -d '\r' < dos.txt > unix.txt # strip carriage returns (DOS->Unix line endings)
tr -s ' ' < file # SQUEEZE runs of spaces into a single space
tr -s '\t' '\n' < file # turn runs of space/tab into newlines (one token/line)
tr -cd '[:print:]\n' < f # keep only printable chars + newlines (strip binary noise)

```

Top-10 most common error messages in a test log:

```
grep '\[ERROR\]' test.log \
| sed 's/.*\[ERROR\] //' \ # strip prefix up to and including [ERROR]
| sort \ # group identical messages (REQUIRED before uniq)
| uniq -c \ # count consecutive duplicates
| sort -rn \ # rank by count, highest first
| head -10 # top 10
```

2.4.5 find and xargs

```
find . -name '*.py' -type f # all regular .py files
find . -iname '*.LOG' -type f # case-insensitive name match
find /var/log -name '*.log' -mtime +30 # modified MORE than 30 days ago
find /var/log -name '*.log' -mtime -1 # modified within the LAST 1 day
find . -type f -size +100M # files larger than 100 MB
find . -type f -newer reference_file # modified after a reference file
find . -type d -empty # empty directories
find . -maxdepth 1 -name '*.csv' # only the current directory (no recursion)

# Acting on results:
find /var/log -name '*.log' -mtime +30 -delete # delete old logs (find's own -delete)
find . -name '*.py' -exec wc -l {} + # {} + : one invocation for all files
find . -name '*.tmp' -exec rm {} \; # {} \; : one invocation PER FILE

# xargs: turn stdin into arguments for a command.
# The SAFE pairing for filenames with spaces: -print0 on find + -0 on xargs.
find . -name '*.pyc' -print0 | xargs -0 rm
find . -name '*.py' -print0 | xargs -0 grep -l 'import pyvisa'
pgrep -f pytest | xargs -r kill -15 # -r: skip the kill if there are no PIDs
```

2.4.6 tee, diff, comm, paste, column

```
# tee: split a stream to a file AND onward (logging without losing the stream)
lspci -vvv | tee lspci.txt | grep NVIDIA

# diff: line-by-line comparison
diff -u fileA fileB # unified (git-style) diff
diff <(lspci -t) <(cat expected_topology.txt) # compare live command vs saved baseline

# comm: compare two SORTED files. Three columns: only-in-1, only-in-2, in-both.
# The NUMBER you pass = the column you SUPPRESS.
comm -23 <(sort a) <(sort b) # lines ONLY in a (suppress col 2 and 3)
comm -12 <(sort a) <(sort b) # lines in BOTH (suppress col 1 and 2)
comm -13 <(sort a) <(sort b) # lines ONLY in b

# paste: join lines side by side
paste -d, file1 file2 # merge two files into comma-separated columns

# column: align output into a readable table
mount | column -t
printf 'BDF Speed Width\n0000:03:00.0 16GT/s x16\n' | column -t
```

2.5 Robust Scripting

2.5.1 The Header Every Production Script Starts With

```
#!/bin/bash
set -euo pipefail
IFS=$'\n\t'
```

`set -euo pipefail` is the single most important line in any production script. Each flag prevents a different class of silent, dangerous failure:

```
# -e (errexit): EXIT IMMEDIATELY if any command returns non-zero. Without it, a script
# sails on after a failed step -- catastrophic when you're controlling a PSU
# or actuator and step 3 failed but step 4 ("ramp voltage") still runs.
#
# -u (nounset): Treat an UNSET variable as an error. Catches typos:
# rm -rf "/data/$DEVCE/" -- with typo'd $DEVCE expanding to "",
# that becomes rm -rf "/data/" . With -u it errors out instead.
#
# -o pipefail: A pipeline's exit status is the FIRST non-zero command, not just the last.
# Without it: broken_tool | grep PASS reports SUCCESS (grep ran fine)
# even though broken_tool crashed. Your test would falsely "pass."
#
# IFS=$'\n\t': Internal Field Separator = newline + tab only (drop the default space).
# Stops filenames and values WITH SPACES from splitting into multiple words
# in unquoted expansions and for-loops.
```

Caveats so `set -e` does not surprise you:

```
# A command whose failure is EXPECTED must be made explicit, or it kills the script:
grep PASS log.txt || true # "|| true" intentionally swallows the failure
count=$(grep -c PASS log.txt) || count=0 # handle the no-match exit code explicitly

# Commands tested by if/while/for do NOT trigger -e exit -- they are the test.
if grep -q FAIL log.txt; then ...; fi # grep returning non-zero here is fine; it's the condition
```

2.5.2 Logging, Fatal-Exit, and Self-Locating Helpers

```
# Resolve the script's own directory regardless of where it is called from:
readonly SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
readonly LOG="/var/log/mfg_test/${date +%Y%m%d_%H%M%S}.log"
mkdir -p "$(dirname "$LOG")"

log() { # timestamped, leveled, tees to console + file
  local level="${2:-INFO}"
  echo "[$(date +%Y-%m-%d %H:%M:%S)][$level] $1" | tee -a "$LOG"
}

die() { # log a fatal message and exit non-zero
  log "$1" "FATAL"
  exit 1
}

# Input validation up front -- fail fast with a clear usage message:
[[ $# -ge 1 ]] || die "Usage: $0 <bdf> [expected_speed]"
BDF="$1"
EXPECTED="${2:-16 GT/s}"
```

2.5.3 Functions, Locals, and Return Codes

```
check_link() {
    local bdf="$1" # ALWAYS declare function variables 'local'
    local expected="$2" # avoids clobbering globals with the same name
    local actual
    actual=$(cat "/sys/bus/pci/devices/$bdf/current_link_speed" 2>/dev/null) || {
        log "Cannot read link speed for $bdf" "ERROR"
        return 1 # functions return an EXIT STATUS (0-255), not a value
    }
    [[ "$actual" == "$expected" ]] # function's return status = result of this test
}

# Call it and branch on the status:
if check_link "$BDF" "$EXPECTED"; then
    log "PASS: $BDF at $EXPECTED"
else
    log "FAIL: $BDF not at $EXPECTED"
fi

# To return a VALUE (string/number), echo it and capture with $(...):
get_link_speed() { cat "/sys/bus/pci/devices/$1/current_link_speed" 2>/dev/null; }
speed=$(get_link_speed "$BDF")
```

2.5.4 Flag Parsing with getopt

```
usage() { echo "Usage: $0 -s SERIAL [-v] [-c CONFIG]"; exit 1; }

VERBOSE=0; CONFIG="/opt/test/default.json"
while getopt "s:c:v" opt; do
    case "$opt" in
        s) SERIAL="$OPTARG" ;; # -s takes an argument (note the colon after s in the string)
        c) CONFIG="$OPTARG" ;; # -c takes an argument
        v) VERBOSE=1 ;; # -v is a boolean flag
        *) usage ;;
    esac
done
shift $((OPTIND - 1)) # remove parsed flags; $@ now contains positional arguments
[[ -n "${SERIAL:-}" ]] || die "-s SERIAL is required"
```

2.5.5 Arrays

```
# Indexed arrays:
devices=("gpu0" "gpu1" "gpu2" "gpu3")
echo "${devices[0]}" # gpu0
echo "${devices[@]}" # all elements (always use quotes: "${devices[@]}")
echo "${#devices[@]}" # element count
echo "${!devices[@]}" # the indices (0 1 2 3)
devices+=("gpu4") # append

# Capture command output into an array, one element per line:
mapfile -t bdfs <<(lspci -d 10de: -D | awk '{print $1}') # -t strips the newlines
echo "Found ${#bdfs[@]} GPUs"

# Iterate safely (quoted @ preserves elements with spaces intact):
for d in "${devices[@]"; do echo "$d"; done
```

```
# Associative arrays (bash 4+) -- key/value pairs, ideal for a results table:
declare -A results
results["0000:01:00.0"]="PASS"
results["0000:02:00.0"]="FAIL"
for bdf in "${!results[@]}"; do
    log "$bdf -> ${results[$bdf]}"
done
```

2.6 Signals and Traps (Hardware Safety)

When a script controls real hardware – a PSU, actuators, a pressure rig, heaters – signal handling is a **safety requirement, not a nicety**. If an operator hits Ctrl+C during a stress run and the script dies without zeroing the supply, you can damage a board.

A trap registers a handler that runs when the script exits or receives a signal.

```
#!/bin/bash
set -euo pipefail

cleanup() {
    trap '' SIGINT SIGTERM # ignore further signals -- prevent re-entry
    log "Restoring safe state..."
    set_voltage 0 || true # zero the supply; || true so one failure won't abort cleanup
    disable_psu || true
    release_actuators || true
    log "Safe state reached."
}

# EXIT catches everything: normal exit, set -e error exit, and AFTER signal handlers run.
# Without trapping EXIT, a normal successful exit would skip cleanup.
trap cleanup EXIT
trap cleanup SIGINT # Ctrl+C
trap cleanup SIGTERM # kill / systemctl stop
trap cleanup SIGHUP # controlling terminal closed (SSH dropped)

set_voltage 48
run_stress_test
log "Test complete."
# cleanup() runs automatically here via the EXIT trap.
```

Critical distinctions:

```
# EXIT is the catch-all. Trap this first.
# Per-signal handlers with conventional exit codes (128+N):
trap 'log "Interrupted"; cleanup; exit 130' SIGINT # 128 + 2
trap 'log "Terminated"; cleanup; exit 143' SIGTERM # 128 + 15

# SIGKILL (kill -9) and SIGSTOP CANNOT be trapped or ignored.
# NEVER send kill -9 to a hardware-control script if you can avoid it.
# Send SIGTERM first; give the trap time to run; -9 is the last resort.

# List all signal names and numbers:
kill -l
```

2.7 Logging and Log Rotation

2.7.1 Structured Logging Pattern

```
# Timestamped log file with a die() that flushes the message before exiting:
readonly LOG_DIR="/var/log/mfg_test"
readonly LOG="${LOG_DIR}/test_$(date +%Y%m%d_%H%M%S)_${SERIAL:-unknown}.log"
mkdir -p "$LOG_DIR"

log() {
    local ts level msg
    ts=$(date '+%Y-%m-%d %H:%M:%S')
    level="${2:-INFO}"
    msg="[${ts}] [${level}] $1"
    echo "$msg" | tee -a "$LOG"
    [[ "$level" == "ERROR" || "$level" == "FATAL" ]] && echo "$msg" >&2 # also to stderr
}

die() { log "$1" "FATAL"; exit 1; }

# Capture a command's output AND status while preserving both streams:
run_and_log() {
    local cmd_output exit_code
    cmd_output=$(("$@" 2>&1)
    exit_code=$?
    log "$cmd_output"
    return $exit_code
}
```

2.7.2 Log Rotation

Test stations accumulate logs. Two approaches:

```
# 1. logrotate (standard system tool; config in /etc/logrotate.d/):
# Create /etc/logrotate.d/mfg-test:
#
# /var/log/mfg_test/*.log {
# daily
# rotate 30 # keep 30 rotated copies
# compress # gzip old logs
# delaycompress # don't compress the most recent rotated log (still readable if open)
# missingok # don't error if no log file exists
# notifempty # don't rotate an empty file
# create 640 testeng testeng # permissions and ownership for new log file
# }

# 2. Find + compress inline (for ad-hoc cleanup from a script or cron):
find /var/log/mfg_test -name '*.log' -mtime +30 -print0 | xargs -0 gzip
find /var/log/mfg_test -name '*.gz' -mtime +90 -delete # then delete very old compressed logs

# 3. A simple self-managing log directory (cap at N most recent files):
clean_old_logs() {
    local dir="$1" keep="${2:-50}"
    ls -lt "${dir}"/*.log 2>/dev/null | tail -n "+${(keep+1)}" | xargs -r rm
}
```

2.8 Scheduling: cron and systemd Timers

2.8.1 cron

```
crontab -e # edit YOUR user's crontab (opens $EDITOR)
crontab -l # list current entries
crontab -r # REMOVE ALL entries (no confirmation -- be careful)
sudo crontab -e -u testeng # edit another user's crontab

# Five time fields, then the command:
# minute (0-59) hour (0-23) day-of-month (1-31) month (1-12) day-of-week (0-7, 0/7=Sun)
# * = every */N = every Nth a-b = range a,b,c = list

0 * * * * /opt/test/pcie_verify.sh >> /var/log/pcie.log 2>&1 # top of every hour
0 */6 * * * /opt/test/nvme_health.sh >> /var/log/nvme.log 2>&1 # every 6 hours
30 7 * * 1-5 /opt/test/station_self_check.sh # weekdays 07:30
*/5 * * * * /opt/test/check_all_stations.sh # every 5 minutes
0 6 * * * /opt/test/yield_report.sh | mail -s "Yield" team@zoox.com # daily email
@reboot /opt/test/startup_init.sh >> /var/log/startup.log 2>&1 # once at boot
```

Cron gotchas – memorize these, they bite everyone at least once:

```
# 1. PATH is MINIMAL. Always use absolute paths: /usr/bin/python3, not python3.
#
# 2. NO interactive-shell environment. Set variables in the crontab directly:
# PYTHONPATH=/opt/atf
# SHELL=/bin/bash
# 0 * * * * /usr/bin/python3 /opt/test/check.py
#
# 3. The working directory is $HOME. Use absolute paths for everything.
#
# 4. Output goes NOWHERE unless redirected. Always end with: >> logfile 2>&1
#
# 5. cron runs with /bin/sh. For bash-specific syntax, use a #!/bin/bash script.
#
# 6. % is special in crontab (means newline). Escape it as \% or put it in a script.

# Debugging cron:
systemctl status cron # is the daemon running? (or 'crond' on RHEL)
grep CRON /var/log/syslog # did your job fire?
# Golden rule: run the command manually first with the cron-minimal environment, then schedule.
```

2.8.2 systemd Timers

Timers over cron: logs land in `journalctl`, you get dependency ordering, `RandomizedDelaySec` prevents 50 stations all hammering the dashboard at :00, and `Persistent=true` catches up a missed job after downtime. A timer is a pair of unit files.

```
# Step 1: the .service -- WHAT to run.
sudo tee /etc/systemd/system/pcie-check.service >/dev/null << 'EOF'
[Unit]
Description=Hourly PCIe Health Check

[Service]
Type=oneshot
ExecStart=/opt/test/pcie_verify.sh
User=testeng
EOF
```

```

# Step 2: the .timer -- WHEN to run it.
sudo tee /etc/systemd/system/pcie-check.timer >/dev/null << 'EOF'
[Unit]
Description=Run PCIe health check every hour

[Timer]
OnCalendar=hourly
# Other forms:
# OnCalendar=*-*-* 06:00:00 -- daily at 06:00
# OnCalendar=Mon..Fri 07:30 -- weekdays at 07:30
# OnCalendar=*:0/5 -- every 5 minutes
RandomizedDelaySec=30 # spread firings over a 0-30s window (thundering herd)
Persistent=true # if the machine was off, run ASAP on next boot

[Install]
WantedBy=timers.target
EOF

# Step 3: load, enable, start.
sudo systemctl daemon-reload
sudo systemctl enable --now pcie-check.timer

# Inspect:
systemctl list-timers # all timers + next/last run times
systemctl status pcie-check.timer
journalctl -u pcie-check.service -n 50 --no-pager

```

2.9 Debugging Bash Scripts

```

# set -x: PRINT every command (after expansion) before running it. The #1 debug tool.
set -x
speed=$(cat /sys/bus/pci/devices/$BDF/current_link_speed)
set +x # turn tracing back off

# Trace from the command line without editing the script:
bash -x ./myscript.sh # trace execution
bash -v ./myscript.sh # print each line as READ (before expansion)
bash -n ./myscript.sh # SYNTAX CHECK ONLY -- catches unclosed quotes/blocks

# Richer trace prefix: show file, line number, and function name:
export PS4='+(${BASH_SOURCE}:${LINENO}): ${FUNCNAME[0]:+${FUNCNAME[0]}(): }'
bash -x ./myscript.sh

# Trace ONLY one function, restore caller's options afterward:
noisy_fn() {
    local saved; saved=$(set +o) # snapshot current shell options as eval-able string
    set -x
    # ... body under trace ...
    eval "$saved" # restore exactly what was set before
}

# shellcheck: STATIC analysis (like pylint for bash). Run it on every script you write.
# sudo apt install shellcheck
shellcheck ./myscript.sh
# Catches: unquoted variables (SC2086), useless cat, wrong [ ] vs [[ ]], unreachable code,
# $(...) in single quotes, and dozens of subtle quoting/expansion bugs.

```



```
# Treat its warnings as errors; clean production scripts pass with zero findings.
```

2.10 Manufacturing Test Scripts (Complete, End-to-End)

2.10.1 pcie_verify.sh – Topology Matches Expected Config

```
#!/bin/bash
# pcie_verify.sh -- verify every expected PCIe device is present at expected speed/width.
# Exit 0 = all pass; exit 1 = any fail. Safe to drive from cron, systemd, or the ATF.
set -euo pipefail

readonly EXPECTED_CONFIG="${1:-/opt/test/expected_topology.json}"
readonly LOG="/var/log/mfg_test/pcie_$(date +%Y%m%d_%H%M%S).log"
mkdir -p "$(dirname "$LOG")"

log() { echo "[$(date '+%H:%M:%S')] $*" | tee -a "$LOG"; }
die() { log "FATAL: $*"; exit 1; }
trap 'log "Script complete (exit $?)." EXIT' EXIT

[[ -f "$EXPECTED_CONFIG" ]] || die "Config not found: $EXPECTED_CONFIG"
command -v jq >/dev/null || die "jq is required but not installed"

# expected_topology.json format:
# [{"bdf": "0000:01:00.0", "type": "gpu", "speed": "16 GT/s", "width": "16"}, ...]
total=0; passed=0; failed=0

# < <(...) keeps counters in THIS shell, not a subshell (the classic bash pipe-subshell trap).
while IFS= read -r device; do
    bdf=$(jq -r '.bdf' <<< "$device")
    type=$(jq -r '.type' <<< "$device")
    exp_speed=$(jq -r '.speed' <<< "$device")
    exp_width=$(jq -r '.width' <<< "$device")
    (( total++ ))

    if ! lspci -s "$bdf" &>/dev/null; then
        log "FAIL: $type at $bdf NOT FOUND"; (( failed++ )); continue
    fi

    # Read directly from sysfs -- faster and more reliable than parsing lspci text.
    actual_speed=$(< "/sys/bus/pci/devices/$bdf/current_link_speed" 2>/dev/null) ||
        ↪ actual_speed="UNKNOWN"
    actual_width=$(< "/sys/bus/pci/devices/$bdf/current_link_width" 2>/dev/null) || actual_width="0"

    if [[ "$actual_speed" == "$exp_speed" && "$actual_width" == "$exp_width" ]]; then
        log "PASS: $type $bdf -> $actual_speed x$actual_width"; (( passed++ ))
    else
        log "FAIL: $type $bdf -> expected ${exp_speed} x${exp_width}, got ${actual_speed}
            ↪ x${actual_width}"
        (( failed++ ))
    fi
done < <(jq -c '.[ ]' "$EXPECTED_CONFIG")

log "Results: $passed/$total passed, $failed failed"
(( failed == 0 )) || die "PCIe verification FAILED"
log "ALL DEVICES PASSED"
```

2.10.2 nvme_health.sh – SMART Pass/Fail Across All NVMe Drives

```
#!/bin/bash
# nvme_health.sh -- gate manufacturing pass/fail on NVMe SMART health.
set -euo pipefail
log() { echo "[$(date '+%H:%M:%S')] $*"; }
rc=0

for dev in $(nvme list -o json | jq -r '.Devices[].DevicePath'); do
    log "Checking $dev..."
    smart=$(nvme smart-log "$dev" -o json)

    crit=$(jq '.critical_warning' <<< "$smart") # 0 = healthy; any bit set = fault flag
    # nvme-cli JSON reports temperature in Kelvin; subtract 273 to get Celsius.
    temp_k=$(jq '.temperature' <<< "$smart")
    temp=$(( temp_k - 273 )) # Celsius
    pct_used=$(jq '.percent_used' <<< "$smart") # endurance consumed
    media_err=$(jq '.media_errors' <<< "$smart") # uncorrectable media errors

    # Manufacturing pass criteria for a brand-new board:
    if (( crit != 0 )); then log "FAIL: $dev critical_warning=$crit"; rc=1; continue; fi
    if (( media_err != 0 )); then log "FAIL: $dev media_errors=$media_err"; rc=1; continue; fi
    if (( temp >= 70 )); then log "FAIL: $dev temp=${temp}C -- too hot"; rc=1; continue; fi
    if (( pct_used >= 5 )); then log "WARN: $dev percent_used=${pct_used}% (used drive?)"; fi

    log "PASS: $dev (temp=${temp}C wear=${pct_used}% media_err=$media_err)"
done
exit $rc
```

2.10.3 link_check.sh – Single Device, Scriptable Verdict

```
#!/bin/bash
# link_check.sh <bdf> [expected_speed] [expected_width]
# Minimal: exits 0 on pass, 1 on fail, 2 on missing device.
set -euo pipefail
BDF="${1:?Usage: $0 <bdf> [speed] [width]}"
EXP_SPEED="${2:-16 GT/s}"
EXP_WIDTH="${3:-16}"
SYS="/sys/bus/pci/devices/$BDF"

[[ -d "$SYS" ]] || { echo "FAIL: $BDF not enumerated"; exit 2; }
speed=$(< "$SYS/current_link_speed") # < file is a fast read, no cat subprocess
width=$(< "$SYS/current_link_width")

if [[ "$speed" == "$EXP_SPEED" && "$width" == "$EXP_WIDTH" ]]; then
    echo "PASS: $BDF $speed x$width"; exit 0
else
    echo "FAIL: $BDF got $speed x$width, expected $EXP_SPEED x$EXP_WIDTH"; exit 1
fi
```

2.10.4 aer_cycle.sh – Clear AER, Stress, Report Errors

```
#!/bin/bash
# aer_cycle.sh <bdf> -- arm, apply stress, report which AER error bits re-set.
# Root-cause pattern: clear -> stress -> read. The bits that re-latch ARE the failure mode.
set -euo pipefail
BDF="${1:?need a BDF}"
```

```

echo "=== AER BEFORE ==="
for f in /sys/bus/pci/devices/$BDF/aer_dev_correctable \
/sys/bus/pci/devices/$BDF/aer_dev_fatal \
/sys/bus/pci/devices/$BDF/aer_dev_nonfatal; do
[[ -r "$f" ]] && { printf "%-30s\n" "${f##*/}"; cat "$f"; echo; }
done

# Apply stress (traffic generator, GPU load, NVMe fio, thermal soak):
run_stress "$BDF" 60 # 60-second stress run

echo "=== AER AFTER STRESS ==="
lspci -s "$BDF" -vvv | grep -A12 'Advanced Error Reporting' | grep -E 'CESta|UESta'
# A clean board: all status bits clear.
# The SPECIFIC bits that re-set (BadTLP, BadDLLP, RxErr, ReplayTimeout vs.
# uncorrectable Malformed TLP) are the root-cause classification.

```

The sysfs counters are the right long-term polling interface. Each key maps to a PCIe spec error:

```

# Read all three AER counter files at once and label them:
BDF=0000:03:00.0
SYS=/sys/bus/pci/devices/$BDF
for f in aer_dev_correctable aer_dev_nonfatal aer_dev_fatal; do
[[ -r "$SYS/$f" ]] || continue
echo "--- $f ---"
cat "$SYS/$f"
done

# Expected output on a healthy device (ALL counts should be zero or absent):
# --- aer_dev_correctable --- (the kernel emits short tokens, not prose labels)
# RxErr 0 <- receiver error: physical-layer noise on the lane
# BadTLP 0 <- bad Transaction Layer Packet; framing error
# BadDLLP 0 <- bad Data-Link Layer Packet
# Rollover 0 <- REPLAY_NUM rollover; internal counter, not a real error event
# Timeout 0 <- replay-timer timeout: retrain fired; marginal link under load
# NonFatalErr 0 <- advisory non-fatal
# CorrIntErr 0 <- corrected internal error
# HeaderOF 0 <- header log overflow
# TOTAL_ERR_COR 0

# If any counter is non-zero and rising, classify:
# Growing RxErr / BadTLP -> physical layer: SI problem, marginal trace, bad connector, retimer
# Growing Replay Timer Timeout -> link retrain under traffic; check eye diagram
# Any Fatal error counter -> endpoint was reset; check lspci for link status after the event

```

Polling counters over time to detect a rate (useful for a pass/fail threshold on a production test):

```

# Poll an AER correctable counter at T=0 and T=60s; fail if any counter accumulated.
snap() {
grep -E 'BadTLP|Timeout' "/sys/bus/pci/devices/$1/aer_dev_correctable" \
| awk '{sum += $2} END {print sum+0}'
}
before=$(snap "$BDF")
run_stress "$BDF" 60
after=$(snap "$BDF")
delta=$(( after - before ))
(( delta == 0 )) || echo "FAIL: $delta correctable AER events during stress (BDF=$BDF)"

```

2.11 One-Liner Reference

Goal	Command
Count NVIDIA GPUs present	<code>lspci -d 10de: \ wc -l</code>
List degraded PCIe links	see loop below
Watch live for kernel errors	<code>dmesg -w \ grep -i 'error\ fail\ reset'</code>
AER correctable count for one BDF	<code>grep TOTAL /sys/bus/pci/devices/0000:03:00.0/aer_dev_correctable</code>
NVMe temp in Celsius	<code>printf '%d C\n' "\$((\$(nvme smart-log -o json /dev/nvme0 \ jq .temperature) - 273))"</code>
Count unique IPs in access log	<code>awk '{print \$1}' access.log \ sort -u \ wc -l</code>
Top 10 largest files recursively	<code>find . -type f -exec du -h {} + \ sort -rh \ head -10</code>
Lines in file1 not in file2	<code>comm -23 <(sort file1) <(sort file2)</code>
Pass rate from a results CSV	<code>awk -F, '\$NF=="PASS"{p++} END{printf "%.1f%\n", p/NR*100}' r.csv</code>
Average of a column	<code>awk '{s+=\$1;n++} END{printf "%.2f\n",s/n}' nums.txt</code>
Count files by extension	<code>find . -type f \ sed 's/.*\./' \ sort \ uniq -c \ sort -rn</code>
Replace in all Python files	<code>find . -name '*.py' -exec sed -i 's/old_fn/new_fn/g' {} +</code>
Top error messages	<code>grep ERROR log \ sort \ uniq -c \ sort -rn \ head</code>
Bytes received on eth0	<code>cat /sys/class/net/eth0/statistics/rx_bytes</code>
Which process holds port 8080	<code>ss -tlnp \ grep :8080</code>
Kill all pytest runners	<code>pgrep -f pytest \ xargs -r kill -15</code>
Follow log for ERROR lines	<code>tail -f /var/log/test.log \ grep --line-buffered ERROR</code>
All hwmon sensor temps	<code>paste <(cat /sys/class/thermal/thermal_zone*/type) <(cat /sys/class/thermal/thermal_zone*/temp) \ awk '{printf "%-20s %.1f C\n", \$1, \$2/1000}'</code>

```
# List every PCIe device running below its maximum link speed or width:
for d in /sys/bus/pci/devices/*; do
  cs=$(cat "$d/current_link_speed" 2>/dev/null) || continue
  ms=$(cat "$d/max_link_speed" 2>/dev/null) || continue
  cw=$(cat "$d/current_link_width" 2>/dev/null)
  mw=$(cat "$d/max_link_width" 2>/dev/null)
  [[ "$cs" == "$ms" && "$cw" == "$mw" ]] && continue
  echo "DEGRADED ${d##*/} speed ${cs}/${ms} width x${cw}/x${mw}"
done
```

Chapter 3

Linux for Bring-up and Hardware Debug

The compute boards you test are running Linux. The test station running the test is running Linux. The virtual filesystems `/sys`, `/proc`, and `dmesg` are where the kernel exposes its live view of every device, driver, interrupt, and error counter – all as plain files you read or write with ordinary shell tools. This chapter covers the parts of Linux a Compute Test Engineer lives in daily: hardware enumeration, device-tree debugging, driver management, sysfs-based hardware interrogation, and recovering a wedged board.

Everything in Linux is a file. A PCIe link width is a file. An Advanced Error Reporting (AER) error counter is a file. A thermal zone temperature is a file. That abstraction is exactly why the command-line toolkit is so powerful: querying hardware state is `cat /sys/...`, and scripting it is `read_text()` in Python.

3.1 Filesystem Hierarchy Reference

Path	What lives there
<code>/</code>	Root of the whole tree
<code>/bin</code> , <code>/usr/bin</code>	Essential user binaries (<code>ls</code> , <code>cp</code> , <code>grep</code>) – often merged in modern distros
<code>/sbin</code> , <code>/usr/sbin</code>	System binaries (<code>fdisk</code> , <code>modprobe</code> , <code>ip</code> , <code>reboot</code>)
<code>/etc</code>	System configuration (<code>/etc/fstab</code> , <code>/etc/passwd</code> , <code>/etc/systemd/</code>)
<code>/home/<user></code>	Per-user home directories
<code>/var/log</code>	System and service logs – CHECK HERE FIRST when debugging
<code>/run</code> , <code>/var/run</code>	Runtime data (PID files, sockets); a tmpfs cleared on boot
<code>/proc</code>	Virtual FS: kernel + per-process info, generated on read, not on disk
<code>/sys</code>	Virtual FS: device/driver tree; hardware attributes you read and write
<code>/dev</code>	Device files (<code>/dev/nvme0n1</code> , <code>/dev/ttyUSB0</code> , <code>/dev/null</code>)
<code>/tmp</code>	Temporary files (often cleared on reboot)
<code>/opt</code>	Optional/third-party software; a common home for <code>/opt/atf</code> , <code>/opt/test</code>
<code>/mnt</code> , <code>/media</code>	Mount points for removable media
<code>/boot</code>	Kernel image, initramfs, GRUB config

Path	What lives there
/lib, /usr/lib	Shared libraries and kernel modules

3.2 Device Enumeration

3.2.1 PCIe: lspci

`lspci` is the daily driver. Understand its flags deeply because every PCIe debug session starts here.

```
lspci # one-line summary per device
lspci -nn # append numeric [vendor:device] IDs (essential for grep and scripts)
lspci -D # show the full DOMAIN in the BDF (0000:03:00.0, not just 03:00.0)
lspci -vvv # MAXIMUM verbosity: capabilities, LnkCap/LnkSta, AER, ASPM, MSI
lspci -k # show kernel driver and modules bound to each device
lspci -t # TREE view of bus topology (parent ports -> endpoints)
lspci -d 10de: # filter by VENDOR ID (10de = NVIDIA)
lspci -d ::0302 # filter by CLASS code (0302 = 3D controller)
lspci -d ::0200 # CLASS 0200 = Ethernet
lspci -d ::0108 # CLASS 0108 = NVMe
lspci -s 0000:03:00.0 -vvv # full detail on ONE specific device by BDF
lspci -s 03:00.0 -vvv | grep -i 'lnkcap\|lnksta' # capable vs. actual link, side by side
```

Representative `lspci -s 03:00.0 -vvv` fragment showing a healthy PCIe Gen4 x16 link:

```
03:00.0 3D controller: NVIDIA Corporation Device 2342 (rev a1)
...
LnkCap: Port #0, Speed 16GT/s, Width x16, ASPM L0s L1, ...
LnkSta: Speed 16GT/s (ok), Width x16 (ok)
...
Capabilities: [100 v2] Advanced Error Reporting
UESta: DLP- SDES- TLP- FCP- CmpltT0- CmpltAbt- ...
CESta: RxErr- BadTLP- BadDLLP- Rollover- Timeout- ...
```

If `LnkSta` shows Speed 8GT/s when `LnkCap` says 16GT/s, or Width x8 when capable of x16, the link degraded. That is always abnormal and requires investigation.

Degraded-link failure signatures and initial triage:

```
LnkCap: Speed 16GT/s, Width x16 / LnkSta: Speed 8GT/s (downgraded), Width x16 (ok)
-> Speed degraded only. Common cause: endpoint or root port could not complete
equalization at Gen4. Check BIOS PCIe Gen setting, retimer firmware, and dmesg
for "dpc" (Downstream Port Containment) or equalization errors at boot.

LnkCap: Speed 16GT/s, Width x16 / LnkSta: Speed 16GT/s (ok), Width x8 (downgraded)
-> Width degraded only. One or more lanes failed during link training. Check for
physical damage, bent connector pins, or an open-circuit trace.

LnkCap: Speed 16GT/s, Width x16 / LnkSta: Speed 2.5GT/s (downgraded), Width x1 (downgraded)
-> Worst case: trained to Gen1 x1. Device is present but barely functional.
Almost always a physical problem: wrong slot, bad connector, or broken retimer.

lspci -vvv also shows ASPM state (L0s, L1) and whether ASPM is enabled.
-> If ASPM L1 is active and you're seeing ReplayTimeout AER events, try disabling
ASPM as a diagnostic step: setpci -s 03:00.0 CAP_EXP+0x10.W=0x0000
(writes 0 to LnkCtl register -- clears ASPM bits; confirm with lspci -vvv after).
```

3.2.1.1 setpci: Direct Config Space Access

```
# setpci reads and writes PCI config space registers directly.
# Sizes: .B = byte, .W = word (2B), .L = long (4B).
setpci -s 03:00.0 COMMAND.W # read the Command register (offset 0x04)
setpci -s 03:00.0 04.W # same by hex offset
setpci -s 03:00.0 COMMAND.W=0x0146 # WRITE -- use with care; wrong writes brick a device

# Common registers (use: setpci --dumpregs for the full list of named registers):
# 0x00.L = vendor/device ID
# 0x04.W = command (bus master, mem space, I/O space enable)
# 0x06.W = status (signaled target abort, master abort, etc.)
# 0x08.B = revision ID
# 0x3D.B = interrupt pin
# Use setpci when you need to poke a register that sysfs does not expose directly.
# Extended capabilities (AER, ASPM, SR-IOV) live at offset >= 0x100 and are best
# reached via: setpci -s 03:00.0 CAP_EXP+<offset>.<width>
```

3.2.2 sysfs: /sys/bus/pci

sysfs is the scripting goldmine. Everything `lspci` shows is also available as individual files – readable directly from Bash or Python without spawning a subprocess.

```
ls /sys/bus/pci/devices/ # every PCIe function, named by BDF

BDF=0000:03:00.0
SYS=/sys/bus/pci/devices/$BDF

cat $SYS/vendor # 0x10de (NVIDIA)
cat $SYS/device # numeric device ID
cat $SYS/class # 0x030000 = VGA/display, 0x010802 = NVMe, 0x020000 = Ethernet
cat $SYS/revision # silicon revision
cat $SYS/current_link_speed # "16 GT/s" -- the speed this link actually trained to
cat $SYS/current_link_width # "16" -- lane width actually negotiated
cat $SYS/max_link_speed # "16 GT/s" -- maximum the device is capable of
cat $SYS/max_link_width # "16"
cat $SYS/numa_node # which NUMA node this device is attached to
cat $SYS/irq # legacy IRQ line
cat $SYS/enable # 1 if device is enabled
readlink $SYS/driver # symlink -> bound driver (e.g. ../drivers/nume)
xxd $SYS/config | head # RAW PCI config space (vendor/device, command/status, BARs)

# Degradation check in one line:
[[ "$(cat $SYS/current_link_speed)" == "$(cat $SYS/max_link_speed)" ]] || echo "speed degraded"

# Kernel-tracked AER error counters (Linux 4.19+, requires CONFIG_PCIEAER):
cat $SYS/aer_dev_correctable # RxErr, BadTLP, BadDLLP, Rollover, Replay Timer, NonFatalErr
cat $SYS/aer_dev_nonfatal
cat $SYS/aer_dev_fatal

# These are the right way to track AER on a production station -- no setpci required.
# All entries are "name <tab> count" -- zero counts are still printed.
# TOTAL_ERR_COR at the end may not equal the sum of individual counts
# (multiple errors can arrive in a single ERR_COR message).

# Administrative actions (use deliberately -- these perturb a live link):
echo 1 | sudo tee $SYS/remove # hot-remove the device from the kernel
echo 1 | sudo tee /sys/bus/pci/rescan # re-scan the bus to rediscover it
echo 1 | sudo tee $SYS/reset # function-level reset (FLR), if the device supports it
```

Enumerate all degraded links in one loop:

```
for d in /sys/bus/pci/devices/*; do
cs=$(cat "$d/current_link_speed" 2>/dev/null) || continue
ms=$(cat "$d/max_link_speed" 2>/dev/null) || continue
cw=$(cat "$d/current_link_width" 2>/dev/null)
mw=$(cat "$d/max_link_width" 2>/dev/null)
[[ "$cs" == "$ms" && "$cw" == "$mw" ]] && continue
echo "DEGRADED ${d##*/} speed ${cs}/${ms} width x${cw}/x${mw}"
done
```

Python is preferred for production test code – pathlib reads these files directly with zero parsing fragility:

```
from pathlib import Path

def enumerate_pcie() -> list[dict]:
    """Read every PCIe device from sysfs -- no subprocess, no text parsing."""
    devices = []
    for dev in sorted(Path("/sys/bus/pci/devices").iterdir()):
        def attr(name: str) -> str | None:
            try:
                return (dev / name).read_text().strip()
            except (FileNotFoundError, PermissionError):
                return None
        speed, max_speed = attr("current_link_speed"), attr("max_link_speed")
        width, max_width = attr("current_link_width"), attr("max_link_width")
        devices.append({
            "bdf": dev.name,
            "vendor": attr("vendor"),
            "device": attr("device"),
            "speed": speed,
            "width": f"x{width}" if width else None,
            "max_speed": max_speed,
            "max_width": f"x{max_width}" if max_width else None,
            "degraded": bool(max_speed) and (speed != max_speed or width != max_width),
        })
    return devices

def degraded_links() -> list[dict]:
    return [d for d in enumerate_pcie() if d["degraded"]]
```

3.2.3 USB, Block, CPU, and Full Inventory

```
# USB
lsusb # summary list
lsusb -v # verbose (includes VID/PID, class, endpoints)
lsusb -t # topology TREE (shows hub hierarchy)

# Block devices / storage
lsblk # tree of block devices, partitions, mountpoints
lsblk -o NAME,SIZE,TYPE,FSTYPE,MOUNTPOINT,MODEL,SERIAL
nvme list # NVMe namespaces with model/serial/size
nvme list -o json | jq '.' # JSON form (for scripting)

# CPU
lscpu # architecture, sockets/cores/threads, cache sizes, flags
nproc # number of online logical CPUs (use this in scripts)
cat /proc/cpuinfo | grep 'model name' | head -1 # CPU model string
```



```

# Full hardware inventory (lshw is called out in the Zoox JD)
lshw # complete hardware tree (very detailed)
lshw -short # one-line-per-device summary table
lshw -json # JSON output -- parse this in Python, not the text form
lshw -class network # only NICs
lshw -class storage # only storage controllers
lshw -class display # only GPUs
lshw -class disk

# DIMM slot inventory
dmidecode --type memory # slot info: type, speed, size, manufacturer, part number
dmidecode --type bios # BIOS vendor/version/release date
dmidecode --type system # manufacturer, product name, serial, UUID
dmidecode --type processor # CPU socket details

```

3.3 dmesg and journald: Kernel Messages

dmesg is usually the **first** place to look when a board does something unexpected. Hardware faults, driver bind/unbind, PCIe AER events, Non-Volatile Memory Express (NVMe) controller resets, thermal throttling, and OOM kills all land here.

3.3.1 dmesg

```

dmesg # full ring buffer
dmesg -T # human-readable wall-clock timestamps (not seconds since boot)
dmesg -w # FOLLOW live (like tail -f) -- watch events as they happen
dmesg -l err,warn # only error and warning level messages
dmesg | tail -50 # most recent messages
dmesg -T | grep -i 'pcie\|aer' # PCIe / AER events
dmesg -T | grep -i 'nvme' # NVMe resets, timeouts, controller errors
dmesg -T | grep -i 'thermal\|thrott' # thermal throttling events
dmesg -T | grep -iE 'error|fail|fault|timeout|reset' # broad hardware-trouble sweep
sudo dmesg -C # CLEAR the ring buffer (arm before a stress run, re-read after)
journalctl -k -b # same kernel messages, persisted across reboots (this boot)
journalctl -k -b -1 # previous boot's kernel messages

```

3.3.2 Decoding Common dmesg Signatures

```

PCIe AER correctable error (recoverable, but non-zero = investigate):
[ 42.185] pcieport 0000:00:01.0: AER: Corrected error received: id=0108
[ 42.185] 0000:03:00.0: PCIe Bus Error: severity=Corrected, type=Physical Layer,
id=0300(Receiver ID)
[ 42.185] 0000:03:00.0: device [10de:2342] error status/mask=00000001/00002000
[ 42.185] 0000:03:00.0: [0] RxErr

-> BadTLP / BadDLLP / RxErr: physical layer noise (marginal lanes, power integrity).
If the count is static and never rises again, it may be a boot-transient.
If the count climbs under GPU/NVMe load, it is an SI / signal integrity issue.

-> ReplayTimeout: retrain happening; link is marginal under traffic.
Cross-check: cat current_link_speed -- if it dropped to 8GT/s from 16GT/s, the
link down-trained due to too many errors.

-> Uncorrectable Fatal (UESta: MalformedTLP+ or SurpriseDown+):

```

The endpoint reset. Expect "device disabled" or driver unbind messages to follow.
Immediately do: `lspci -s $BDF && cat aer_dev_fatal` to get the full picture.

PCIe AER uncorrectable non-fatal:

```
[ 55.300] 0000:03:00.0: PCIe Bus Error: severity=Uncorrected (Non-Fatal), type=Transaction Layer
[ 55.300] 0000:03:00.0: device [10de:2342] error status/mask=00000020/00000000
[ 55.300] 0000:03:00.0: [5] SDES (First)
-> SDES = Surprise Down Error Status (AER UESa bit 5, 0x20). The link partner dropped off
the bus -- a Data-Link-layer surprise-down (power loss or a PERST#/reset), not symbol noise.
Action: check power delivery to the slot and the PERST#/reset signal, then the device's link
↪ state.
```

NVMe timeout / reset:

```
[ 88.002] nvme nvme0: I/O 23 QID 1 timeout, disable controller
[ 88.002] nvme nvme0: Device shutdown timeout. Resetting the device.
-> Power delivery to the M.2 slot, PCIe lane integrity, or firmware bug on NVMe side.
Follow-up: nvme smart-log /dev/nvme0 to check media_errors and critical_warning.
If media_errors > 0 on a brand-new drive, it is a bad unit.
```

NVMe reset after link retrain:

```
[ 89.010] nvme nvme0: controller is down; will reset: CSTS=0x3, PCI_STATUS=0x0110
-> CSTS fatal status + PCI_STATUS bit 8 (Master Data Parity Error; 0x100 -- 0x10 alone is bit 4,
the benign Capabilities List bit). The NVMe lost
its PCIe link before completing an I/O. Root cause is usually in the PCIe lane,
not the NVMe itself.
```

Thermal throttling:

```
[ 312.001] CPU0: Package temperature above threshold, cpu clock throttled
-> Normal if the board is hot; abnormal at idle or if the thermal solution is misapplied.
Follow-up: sensors; cat /sys/class/thermal/thermal_zone*/temp; check fan RPM.
```

Driver bind failure:

```
[ 5.223] nvidia: probe of 0000:03:00.0 failed with error -12
-> Error -12 = ENOMEM. Driver could not allocate memory.
Too many GPUs for available system RAM, or IOMMU aperture exhausted.
Other common errors: -22 = EINVAL (bad parameter); -5 = EIO (hardware not responding).
```

OOM kill:

```
[1234.5] Out of memory: Kill process 4321 (pytest) score 847 or sacrifice child
-> Station is memory-constrained under load. free -h to check swap usage.
If swap is full too, either add RAM or reduce test parallelism.
```

MCE (Machine Check Exception):

```
[ 10.555] mce: [Hardware Error]: CPU 0: Machine Check: 0 Bank 1: b2000000000000800
[ 10.555] mce: [Hardware Error]: TSC 0 ADDR 0xffff8800deadbeef MISC 0x68
-> Corrected MCE: the hardware fixed it; monitor count with 'mcelog' or 'rasdaemon'.
-> Uncorrected MCE (MCACOD in bank): CPU internal fault, ECC memory error, or
interconnect fault. An uncorrected MCE usually triggers a kernel panic.
Action: check dimm slot with dmidecode; run memtest86+.
```

3.3.3 journalctl

`journalctl` extends `dmesg` – it persists across reboots and covers all `systemd` unit output:

```
journalctl -u atf-dashboard # all logs for one service
journalctl -u atf-dashboard -f # FOLLOW live (like tail -f)
journalctl -u atf-dashboard -n 100 --no-pager # last 100 lines without a pager
journalctl -u atf-dashboard --since '1 hour ago' # time window ("yesterday", "2 hours ago")
journalctl -u atf-dashboard -p err # priority err and above (emerg..err)
```

```
journalctl -k # kernel messages only (persisted dmesg)
journalctl -b # since the current boot
journalctl -b -1 # PREVIOUS boot (debug a crash/reboot)
journalctl --since "2026-04-01" --until "2026-04-02"
journalctl --disk-usage # how much space the journal uses
sudo journalctl --vacuum-time=7d # trim to the last 7 days
```

3.4 udev Rules

udev runs in userspace and handles everything that happens when the kernel detects a new device: creating /dev nodes, setting permissions, running scripts, loading firmware. When a device node appears at an unpredictable name (/dev/ttyUSB0 on one boot, /dev/ttyUSB1 on another) or with the wrong permissions, udev rules fix it.

```
# Inspect existing rules:
ls /lib/udev/rules.d/ # system rules (from packages -- read but do not edit)
ls /etc/udev/rules.d/ # local overrides (your rules go here)

# Watch udev events in real time (plug in a device while this runs):
udevadm monitor --environment --udev

# Query a device's properties (everything available for rule matching):
udevadm info -a -n /dev/ttyUSB0 # -a walks up the hierarchy (parent chain)
udevadm info -a -n /dev/nvme0 # NVMe
udevadm info /sys/bus/pci/devices/0000:03:00.0 # PCIe device by sysfs path
```

A rule that gives every FTDI serial adapter (VID 0403) a stable symlink and allows the testeng group to access it without sudo:

```
# /etc/udev/rules.d/99-test-instruments.rules
# Assign a stable symlink and set group permissions for all FTDI adapters.
SUBSYSTEM=="tty", ATTRS{idVendor}=="0403", ATTRS{idProduct}=="6001", \
    SYMLINK+="ttyFTDI%n", GROUP="testeng", MODE="0664"

# A specific device by serial number (use: udevadm info -a to find ATTRS{serial}):
SUBSYSTEM=="tty", ATTRS{idVendor}=="0403", ATTRS{serial}=="A50285BI", \
    SYMLINK+="psu_console", GROUP="testeng", MODE="0664"

# NVMe device: set group ownership so testeng user can run nvme commands without sudo.
SUBSYSTEM=="nvme", KERNEL=="nvme[0-9]*", GROUP="testeng", MODE="0660"

# Reload rules and re-trigger without rebooting:
sudo udevadm control --reload-rules
sudo udevadm trigger

# Test a rule (dry run -- shows what WOULD happen):
udevadm test /sys/bus/usb/devices/1-1.2/1-1.2:1.0 # use the path from udevadm monitor
```

Rule file naming: files are processed in lexical order. Use 99- prefix for your rules so they run after distro rules (which set defaults you may want to override).

3.5 Kernel Modules

Drivers live as loadable kernel modules. When a device does not appear or behaves incorrectly, the module is the first thing to inspect.

```
lsmod # all currently loaded modules + use counts + dependents
lsmod | grep nvme # is the nvme driver loaded?
modinfo nvme # metadata: file path, parameters, dependencies, version
modinfo -p nvme # just the tunable parameters
sudo modprobe nvme # load a module (and its dependencies)
sudo modprobe -r nvme # unload (fails if in use: use count > 0)
sudo modprobe nvme io_timeout=30 # load with a parameter
sudo rmmod nvme # low-level remove (no dep handling -- prefer modprobe -r)
dmesg | grep -i nvme # kernel messages as the driver loaded, bound, or errored
```

Persisting module configuration:

```
# Blacklist a module (e.g. prevent the open-source nouveau from loading so nvidia can bind):
echo "blacklist nouveau" | sudo tee /etc/modprobe.d/blacklist-nouveau.conf

# Set a parameter at load time:
echo "options nvme io_timeout=30" | sudo tee /etc/modprobe.d/nvme.conf

# Force-load a module at boot:
echo "nvme" | sudo tee /etc/modules-load.d/nvme.conf

# After any modprobe.d change, rebuild the initramfs so the change takes effect early:
sudo update-initramfs -u # Debian/Ubuntu
sudo dracut -f # RHEL/Fedora
```

3.5.1 DKMS: Driver Modules Across Kernel Updates

Out-of-tree drivers (proprietary GPU, custom hardware) break when the kernel upgrades unless they are managed by DKMS (Dynamic Kernel Module Support), which rebuilds them automatically:

```
dkms status # show all DKMS-managed modules and their build status
sudo dkms install nvidia/535.86.10 # manually trigger a build
sudo dkms remove nvidia/535.86.10 --all # remove

# If dkms autoinstall fails after a kernel upgrade:
sudo dkms autoinstall # rebuild all registered modules for the running kernel
# Check /var/lib/dkms/<name>/<version>/build/make.log for compiler errors
```

3.6 PCI Config Space and AER

The 256-byte (or 4096-byte extended) PCI configuration space holds device identity, capabilities, command/status, and – in the extended capabilities – AER, Active State Power Management (ASPM), Single Root I/O Virtualization (SR-IOV). Understanding it is necessary for low-level debug.

```
# Read raw config space:
xxd /sys/bus/pci/devices/0000:03:00.0/config | head -20
# Bytes 0-1: Vendor ID
# Bytes 2-3: Device ID
# Bytes 4-5: Command register (bus master, mem/IO space enable)
# Bytes 6-7: Status register
# Byte 8: Revision ID
# Byte 9-11: Class code
```

```

# Bytes 16-39: Base Address Registers (BARs 0-5)
# Bytes 44-47 (0x2C): Subsystem Vendor ID (0x2C) + Subsystem ID (0x2E)
# Byte 52 (0x34): Capabilities Pointer
# Byte 60: Interrupt line
# Byte 61: Interrupt pin

# setpci: read/write specific registers
setpci -s 03:00.0 COMMAND.W # read Command register
setpci -s 03:00.0 04.W # same by hex offset

# AER registers live in the Extended Capability block (PCIe extended config space, offset >=
  ↪ 0x100).
# The kernel-tracked counters in sysfs are the preferred interface:
cat /sys/bus/pci/devices/0000:03:00.0/aer_dev_correctable

```

AER correctable error register bits (from the PCIe spec):

Bit	Name	What it means at the bench
0	RxErr	Receiver error; physical layer noise on the lane
6	BadTLP	Bad TLP received; framing or data corruption
7	BadDLLP	Bad DLLP; data-link layer issue
8	Rollover	Error counter rolled over (not a real error event)
12	ReplayTimeout	Replay timer expired; link retrain under traffic
13	NonFatalErr	Advisory non-fatal; escalated from COR

Real counter file output – cat /sys/bus/pci/devices/0000:03:00.0/aer_dev_correctable:

```

# CLEAN board (all zeros -- the only acceptable result at station):
Receiver Error 0
Bad TLP 0
Bad DLLP 0
RELAY_NUM Rollover 0
Replay Timer Timeout 0
Advisory Non-Fatal 0
Corrected Internal Error 0
Header Log Overflow 0
TOTAL_ERR_COR 0

# MARGINAL board (BadTLP climbing under GPU workload):
Receiver Error 0
Bad TLP 47 <- non-zero: physical layer / SI issue
Bad DLLP 0
RELAY_NUM Rollover 0
Replay Timer Timeout 12 <- retrain is happening under load
Advisory Non-Fatal 0
TOTAL_ERR_COR 59

```

Failure signatures and what to do next:

```

Growing RxErr or BadTLP under load
-> Lane marginality: SI problem, marginal trace, bad connector, failed retimer.
Action: check lspci LnkSta for width downgrade; run eye-diagram test; inspect
board layout / connector seating.

```

Growing Replay Timer Timeout (without RxErr/BadTLP)
 -> Link retrain at speed; can be ASPM-related or thermal drift on the retimer.
 Action: check `current_link_speed`; disable ASPM and retest (`setpci LNKCTL.W`).

Any Fatal counter (UESta: MalformedTLP+ or SurpriseDown+)
 -> The endpoint was reset. The driver may have unbound.
 Action: check `dmesg` for "AER: broadcast error_detected" or "device disabled";
 check `lspci` for device presence; trigger FLR if driver is still bound.

UESta: Surprise Down
 -> Device disappeared from the bus mid-operation.
 Action: check power delivery to the slot; check PCIe reset (PERST#) signal.

PCIe link recovery flow (device appears but is degraded or re-enumerating):

```
BDF=0000:03:00.0
SYS=/sys/bus/pci/devices/$BDF

# 1. Confirm the device is present:
lspci -s "$BDF" -vvv | head -5

# 2. Check current vs maximum link:
echo "Speed: $(cat $SYS/current_link_speed) / $(cat $SYS/max_link_speed)"
echo "Width: x$(cat $SYS/current_link_width) / x$(cat $SYS/max_link_width)"

# 3. Attempt a function-level reset (FLR) -- if the device supports it:
echo 1 | sudo tee $SYS/reset

# 4. If FLR does not recover it, hot-remove and rescan:
echo 1 | sudo tee $SYS/remove
sleep 1
echo 1 | sudo tee /sys/bus/pci/rescan

# 5. If still wrong after rescan, pull AER state:
cat $SYS/aer_dev_correctable
cat $SYS/aer_dev_fatal
dmesg -T | grep -i "$BDF"

# Note: remove+rescan does not power-cycle the device; it only removes/re-adds the
# kernel's representation. A true power-cycle requires PERST# or BMC control.
```

3.7 Interrupts and IRQ Affinity

```
cat /proc/interrupts # interrupt counts per IRQ, broken out PER CPU
# Columns: IRQ#, one count column per CPU, then the controller type and device/driver name.
# Uses:
# - Is the device's interrupt firing at all? (zero count under load = wrong/dead IRQ)
# - Are interrupts spread across CPUs or piled on CPU0? (CPU0 pile-up caps throughput)
# - MSI/MSI-X vectors appear by name: confirm an NVMe or NIC negotiated multiple queues.

cat /proc/softirqs # software interrupts (NET_RX/NET_TX for NIC throughput)

# Watch NVMe IRQs increment live under fio load:
watch -n1 'cat /proc/interrupts | grep -i nvme'
```

```

# IRQ affinity: which CPUs service a given IRQ
cat /proc/irq/<NUM>/smp_affinity # bitmask (hex)
cat /proc/irq/<NUM>/smp_affinity_list # human-readable CPU list (e.g. "0-3")

# Set affinity (route IRQ 42 to CPUs 2 and 3):
echo "c" | sudo tee /proc/irq/42/smp_affinity # 0xc = binary 1100 = CPUs 2,3

# irqbalance daemon automatically spreads IRQs across CPUs.
# On a test station running latency-sensitive workloads you may want to stop it:
sudo systemctl stop irqbalance
sudo systemctl disable irqbalance
# Then set affinity manually to pin NIC/NVMe interrupts to the CPUs that run the test.

```

3.8 i2c-tools

I2C is ubiquitous on compute boards for power management ICs, temperature sensors, EEPROMs, fan controllers, and voltage regulators. `i2c-tools` gives you direct bus access from userspace.

```

# Enumerate I2C adapters:
ls /dev/i2c-* # typically /dev/i2c-0 through /dev/i2c-N
i2cdetect -l # list adapters with their names and parent (from device tree)

# Scan for devices on a bus (bus number = the N in /dev/i2c-N):
i2cdetect -y 0 # scan bus 0 (-y skips interactive prompt; -r uses receive-byte probe instead of
↳ quick-write)
# Output: a 7-bit address grid. -- = no device. UU = in use by a kernel driver. 0x68 = present.

# Read registers:
i2cget -y 0 0x48 0x00 b # bus 0, address 0x48, register 0x00, byte read
i2cget -y 0 0x48 0x00 w # word (16-bit) read

# Write a register:
i2cset -y 0 0x48 0x01 0x60 b # write 0x60 to register 0x01 on address 0x48

# Dump all readable registers of a device:
i2cdump -y 0 0x48 b # bus 0, address 0x48, byte mode

# Read a raw register without specifying a register (useful for some sensors):
i2cget -y 0 0x48 # returns whatever the device sends next

```

Practical use cases on a compute board:

```

# Read a TMP102 temperature sensor at address 0x48 on I2C bus 0.
# TMP102 register 0x00 is a 16-bit two's complement value, MSB first; 0.0625 C/LSB:
raw=$(i2cget -y 0 0x48 0x00 w)
# raw is big-endian: swap bytes and shift
temp_raw=$(( ( (0x${raw:2:2}) | ((0x${raw:4:2}) << 8) ) >> 4 ))
echo "Temperature: $(awk "BEGIN{printf \"%.2f\\n\", $temp_raw * 0.0625}") C"

# Read a PMBus power rail voltage (device at 0x40, VOUT_MODE + VOUT_COMMAND registers):
i2cget -y 1 0x40 0x20 b # VOUT_MODE
i2cget -y 1 0x40 0x21 w # VOUT_COMMAND (raw voltage word -- decode per PMBus spec)

```

The `i2c-dev` kernel module must be loaded for `/dev/i2c-*` to exist:

```

sudo modprobe i2c-dev
echo "i2c-dev" | sudo tee /etc/modules-load.d/i2c-dev.conf # persist across reboots

```

3.9 MTD / Flash

Many compute boards have NOR or NAND flash for firmware (UEFI/BIOS, BMC, Field-Programmable Gate Array (FPGA) images). Linux exposes these via the MTD (Memory Technology Device) subsystem.

```
cat /proc/mtd # MTD partitions present (name, size, erase size)
# Example output:
# dev: size erasesize name
# mtd0: 00200000 00001000 "bios"
# mtd1: 00800000 00001000 "bmc"

ls /dev/mtd* # mtdN = raw partition, mtdblockN = block device interface

# Read a partition to a file (firmware backup before update):
sudo dd if=/dev/mtd0 of=bios_backup.bin bs=4096 # or use nanddump for NAND

# Tools from mtd-utils:
sudo apt install mtd-utils

flashcp -v new_bios.bin /dev/mtd0 # program a NOR flash partition (verify after)
nandwrite -p /dev/mtd1 new_bmc.bin # write NAND (without bad-block skip: add -k)
nanddump -f saved.bin /dev/mtd1 # read NAND to a file (skips bad blocks by default)

mtdinfo /dev/mtd0 # geometry: size, erase size, bad-block handling
flash_eraseall /dev/mtd0 # erase entire partition (DESTRUCTIVE)
flash_eraseall -j /dev/mtd0 # erase and format as JFFS2

# Verify a flash region matches a known-good image:
sudo dd if=/dev/mtd0 bs=4096 | md5sum
md5sum reference.bin
# A mismatch after flashing = incomplete write or read-back error.
```

3.10 Power and Thermal

3.10.1 hwmon: Hardware Monitor via sysfs

```
ls /sys/class/hwmon/ # hwmon0, hwmon1, ... each is a sensor chip
cat /sys/class/hwmon/hwmon0/name # chip name (e.g. "coretemp", "nct6779", "ina3221")

# Temperature sensors (in_tempN_*):
cat /sys/class/hwmon/hwmon0/temp1_input # temperature in millidegrees C (e.g. 45000 = 45 C)
cat /sys/class/hwmon/hwmon0/temp1_max # configured max (millidegrees)
cat /sys/class/hwmon/hwmon0/temp1_crit # critical threshold
cat /sys/class/hwmon/hwmon0/temp1_label # what is being measured (e.g. "Package id 0")

# Fan speed:
cat /sys/class/hwmon/hwmon0/fan1_input # RPM

# Voltage / current (INA3221, INA219, etc.):
cat /sys/class/hwmon/hwmon0/in1_input # millivolts
cat /sys/class/hwmon/hwmon0/curr1_input # milliamps
```



```
# Print all labeled temperatures in human-readable form:
paste <(cat /sys/class/thermal/thermal_zone*/type) \
<(cat /sys/class/thermal/thermal_zone*/temp) | \
awk '{printf "%-22s %.1f C\n", $1, $2/1000}'

# sensors command (lm-sensors package) aggregates hwmon for a human view:
sensors # all chips
sensors -u # raw values (what sysfs actually contains)
sensors coretemp-isa-0000 # one chip by name
```

Thermal failure signatures and triage:

```
Symptom: Board reboots or NVMe resets after 10-20 minutes under load.
- Check dmesg: "CPU0: Package temperature above threshold, cpu clock throttled"
- Check sensors: if Package temp approaches temp1_crit (often 95-105 C), thermal solution
  is inadequate.
- Check fan RPM via fan1_input: if zero, the fan is dead or the connector is unseated.
- Check INA3221 current: if input current is below expected draw, PSU may be current-limiting.
Action: re-apply thermal paste; verify heatsink mounting pressure; confirm fan direction.

Symptom: Temperatures read correctly in sensors but PCIe errors appear at moderate load.
- Check /sys/class/hwmon/hwmonN/in1_input for 12V / 3.3V rails on the board's INA sensors.
- A rail sagging 5% or more under load causes PCIe receiver noise (BadTLP / RxErr AER events).
Action: measure rail voltage with a scope; check bulk capacitance near the PCIe slot.

Symptom: All temperatures look fine but thermal throttling message appears.
- Some platforms trigger throttle based on the PCH or memory temperature, not just the CPU.
- cat /sys/class/thermal/thermal_zone*/type to identify which zone is triggering.
Action: check the specific zone's _crit threshold; check airflow around that component.
```

3.10.2 cpufreq: CPU Frequency and Thermal Throttling

```
# Current frequency per CPU (in kHz):
cat /sys/devices/system/cpu/cpu0/cpufreq/scaling_cur_freq
cat /sys/devices/system/cpu/cpu*/cpufreq/scaling_cur_freq # all CPUs

# Available governors:
cat /sys/devices/system/cpu/cpu0/cpufreq/scaling_available_governors
# performance, powersave, schedutil, ondemand, conservative

# Change governor for all CPUs:
echo "performance" | sudo tee /sys/devices/system/cpu/cpu*/cpufreq/scaling_governor
# Use "performance" on test stations to prevent thermal throttling from affecting results.

# Min/max limits:
cat /sys/devices/system/cpu/cpu0/cpufreq/scaling_min_freq
cat /sys/devices/system/cpu/cpu0/cpufreq/scaling_max_freq
echo 3600000 | sudo tee /sys/devices/system/cpu/cpu0/cpufreq/scaling_max_freq # 3.6 GHz cap

# Check for throttling:
dmesg | grep -i throttl
cat /sys/devices/system/cpu/cpu0/cpufreq/scaling_cur_freq # if << max, likely throttled
```

3.10.3 RAPL: Running Average Power Limit

RAPL exposes CPU and Dynamic Random-Access Memory (DRAM) power consumption counters. On test stations they help characterize power draw during load.

```

# RAPL domains are exposed via powercap:
ls /sys/class/powercap/

# Energy counters (in microjoules -- monotonically increasing, wraps at max_energy_range_uj):
cat /sys/class/powercap/intel-rapl:0/energy_uj # package 0 energy
cat /sys/class/powercap/intel-rapl:0:0/energy_uj # core domain
cat /sys/class/powercap/intel-rapl:0:1/energy_uj # uncore/DRAM domain
cat /sys/class/powercap/intel-rapl:0/name # domain name

# Measure power over an interval (two readings, then compute delta):
e1=$(cat /sys/class/powercap/intel-rapl:0/energy_uj); sleep 1
e2=$(cat /sys/class/powercap/intel-rapl:0/energy_uj)
awk "BEGIN{printf \"Package power: %.2f W\\n\", ($e2-$e1)/1e6}"

# Power limits (current TDP envelope):
cat /sys/class/powercap/intel-rapl:0/constraint_0_power_limit_uw # PL1 in microwatts
cat /sys/class/powercap/intel-rapl:0/constraint_1_power_limit_uw # PL2

# turbostat (linux-tools package) gives a unified per-CPU view of frequency + power + temp:
sudo turbostat --interval 1 # 1-second samples with per-package and per-core columns

```

3.11 perf and ftrace Basics

3.11.1 perf

perf reads hardware performance counters (available via the PMU) and software tracepoints. Even a brief `perf stat` run identifies whether a workload is CPU-bound, memory-bound, or dominated by cache misses.

```

# Count events for a command:
perf stat ./workload

# Key output lines:
# task-clock (msec): CPU time consumed
# instructions: total instructions executed
# cycles: total cycles
# IPC (insn/cycle): < 1 = stalled (usually memory or branch prediction); > 2 = good
# cache-misses: LLC miss rate
# branch-misses: branch predictor misses

# Live top (hottest functions right now):
sudo perf top # system-wide
sudo perf top -p <pid> # one process

# Record a call-graph profile:
sudo perf record -g -p <pid> sleep 10 # record for 10 s
sudo perf report # interactive browser of the hotspots

# Count specific events:
perf stat -e cache-misses,cache-references,branch-misses,instructions ./workload

# perf on a device driver:
sudo perf trace -a -e 'irq:irq_handler_entry' sleep 5 # trace all IRQ handler entries for 5s

```

3.11.2 ftrace

ftrace is built into the kernel and requires no extra packages. It traces kernel functions and tracepoints with very low overhead – useful for debugging driver behavior or latency spikes.

```

# Mount debugfs if not already mounted:
sudo mount -t debugfs none /sys/kernel/debug

cd /sys/kernel/debug/tracing

# Function tracing:
echo function | sudo tee current_tracer # enable the function tracer
echo "nvme_*" | sudo tee set_ftrace_filter # trace only nvme_ functions
echo 1 | sudo tee tracing_on # start recording
sleep 5
echo 0 | sudo tee tracing_on # stop
cat trace # read the log

# Event tracing (tracepoints):
ls available_events | grep nvme # available NVMe tracepoints
echo "nvme:nvme_sq" | sudo tee set_event # enable a specific tracepoint
echo 1 | sudo tee tracing_on
sleep 5
echo 0 | sudo tee tracing_on
cat trace | head -50

# Latency tracing (find the highest-latency function call):
echo wakeup | sudo tee current_tracer
echo 1 | sudo tee tracing_on
sleep 5
cat trace | head -100 # look for "LATENCY TRACE" header and max latency value

# Reset tracer:
echo nop | sudo tee current_tracer
echo "" | sudo tee set_ftrace_filter

```

3.12 Filesystems, Mount, and /proc/mounts

```

# Inventory:
lsblk -o NAME,SIZE,TYPE,FSTYPE,MOUNTPOINT,MODEL,SERIAL
sudo fdisk -l # disks and partition tables
sudo parted -l # GPT-aware, more detail
blkid # UUIDs and filesystem types (used in /etc/fstab)

# Usage:
df -h # filesystem usage: how full each FS is (human-readable)
df -i # INODE usage (a FS can be "full" on inodes with space left)
du -sh /var/log/* # size of each item under a directory
du -sh /opt/test 2>/dev/null # total size of a tree

# Mount / unmount:
sudo mount /dev/sdb1 /mnt/usb # mount a partition
sudo mount -o ro /dev/sdb1 /mnt/usb # read-only (forensics / imaging)
sudo umount /mnt/usb # unmount (device must not be in use)
mount | column -t # all current mounts, aligned
cat /etc/fstab # persistent mount config (entries mounted at boot)

# Create / check (destructive -- double-check the target device!):
sudo mkfs.ext4 /dev/sdb1
sudo fsck /dev/sdb1 # check / repair (FS must be UNMOUNTED)

```

```
# SMART health -- the storage equivalent of AER for PCIe:
sudo smartctl -H /dev/sda # overall: PASSED / FAILED
sudo smartctl -a /dev/sda # all SMART attributes (reallocated sectors, pending, offline)
sudo nvme smart-log /dev/nvme0 # NVMe: temp, percentage_used, media_errors, critical_warning
sudo nvme error-log /dev/nvme0 # NVMe error log entries
```

3.13 Permissions and Linux Capabilities

```
# Permission string from ls -l: -rwxr-xr-x
# char 1: type ( - file, d dir, l symlink, c char device, b block device )
# chars 2-4: OWNER permissions rwx
# chars 5-7: GROUP permissions rwx
# chars 8-10: OTHER permissions rwx
# r = read (4) w = write (2) x = execute (1)

id # your uid, gid, and group memberships
whoami # current username
groups testeng # groups a user belongs to

chmod 755 script.sh # rwxr-xr-x standard for executable scripts
chmod 644 config.json # rw-r--r-- standard for regular files
chmod 600 id_ed25519 # rw----- required for SSH private keys
chmod +x deploy.sh # add execute for everyone
chmod -R 755 /opt/test # recursive
chmod u+s /usr/bin/tool # SETUID: binary runs with file owner's privileges
chmod g+s shared_dir/ # SETGID on a dir: new files inherit the directory's group
chmod +t /tmp # STICKY bit: only owner can delete their files in this dir

sudo chown testeng:eng file
sudo chown -R testeng:eng /opt/test/
sudo usermod -aG dialout testeng # add to the dialout group (access /dev/ttyUSB* without sudo)

# Linux Capabilities: grant specific root-like privileges without full root.
# More secure than setuid root or running as root.
getcap /usr/bin/ping # show capabilities on a file
sudo setcap cap_net_raw+eip /opt/atf/packet_gen # grant raw socket access
sudo setcap cap_sys_rawio+eip /opt/atf/hwmon_reader # grant raw I/O port access
# Relevant capabilities for test tools:
# cap_net_raw : raw/packet sockets (tcpdump, custom NIC test)
# cap_sys_rawio : raw I/O port access (pcimem, direct MMIO)
# cap_ipc_lock : lock memory pages (latency-sensitive tests)
# cap_sys_admin : broad admin (loading modules, mounting, sysfs writes) -- avoid; too broad
capsh --print # capabilities of the current process
```

3.14 Building and Booting: initramfs, ARM Device Tree, Serial Console

3.14.1 initramfs

The initramfs is a minimal filesystem baked into the boot image. It loads early drivers (NVMe, RAID, filesystem) before the real root is accessible.

```

# Rebuild initramfs (after changing kernel parameters or blacklists):
sudo update-initramfs -u # Debian/Ubuntu: update for the running kernel
sudo update-initramfs -u -k all # rebuild for ALL installed kernels
sudo dracut -f # RHEL/Fedora equivalent
sudo dracut -f --kver $(uname -r) # specific kernel version

# Inspect contents of an initramfs:
mkdir /tmp/initrd_inspect && cd /tmp/initrd_inspect
gzip -cd /boot/initrd.img-$(uname -r) | cpio -idmv 2>/dev/null | head -50
# Or with unmkinitramfs (Debian):
unmkinitramfs /boot/initrd.img-$(uname -r) /tmp/initrd_inspect

# Force-include a specific module in initramfs (e.g. a new NVMe driver for early boot):
echo "nvme_core" >> /etc/initramfs-tools/modules # Debian/Ubuntu
sudo update-initramfs -u

# GRUB command line (kernel boot parameters):
cat /proc/cmdline # the ACTUAL parameters the running kernel was given
# Edit /etc/default/grub (GRUB_CMDLINE_LINUX_DEFAULT), then:
sudo update-grub # regenerate /boot/grub/grub.cfg

```

3.14.2 ARM Device Tree

On ARM compute boards (common at Zoox), the kernel does not discover hardware by probing bus addresses; instead, a Device Tree Blob (DTB) describes every peripheral – I2C buses, SPI controllers, PCIe root ports, GPIO pins, power domains – and the kernel reads it at boot.

```

# Find the loaded DTB:
ls /boot/*.dtb /boot/dtbs/ # Debian/Ubuntu path
ls /proc/device-tree/ # live kernel-parsed device tree (virtual FS)

# Inspect the live device tree:
ls /proc/device-tree/ # top-level nodes
cat /proc/device-tree/model | tr -d '\0' # board model string
cat /proc/device-tree/compatible | tr '\0' '\n' # compatible strings (vendor,chip)

# Decompile a DTB to DTS (human-readable source):
dtc -I dtb -O dts -o board.dts /boot/board.dtb
# Now you can grep or read board.dts to understand the hardware topology.

# Recompile a DTS back to DTB:
dtc -I dts -O dtb -o board.dtb board.dts

# Overlay DTBOs (dynamic device tree overlays -- common for optional add-ons):
ls /boot/overlays/ # available overlays
# Active overlays are applied by U-Boot or the boot loader config:
cat /boot/config.txt # Raspberry Pi example (dtoverlay=...)
# On Tegra/Qualcomm boards, overlays may be applied by the early-stage bootloader

# Debugging DT binding failures:
dmesg | grep -i 'of|dtb|device tree|compatible'
ls /sys/bus/platform/devices/ # platform devices created by the DT
cat /sys/bus/platform/devices/3110000.i2c/driver # which driver is bound (symlink)

```

Common device tree debug patterns:

```

# "The I2C temperature sensor is not detected":
cat /proc/device-tree/i2c@3110000/tmp102@48/compatible # is the node present?
dmesg | grep "tmp102" # did the driver probe?

```

```
i2cdetect -y 0 # is the device actually on the bus?

# "The PCIe controller won't enumerate devices":
dmesg | grep -i 'pci|pcie' # look for DWC/IPROC/Tegra PCIe init messages
ls /proc/device-tree/pcie/ # does the DT have a PCIe node?
cat /proc/device-tree/pcie/compatible | tr '\0' '\n' # is the compatible string matched by a
↳ driver?

# "I need to add a new peripheral":
# 1. Write or modify the DTS (add the device node under the correct bus node).
# 2. Build/install the driver module.
# 3. Compile the DTS to DTB.
# 4. Boot with the new DTB (copy to /boot, update U-Boot env or GRUB).
# 5. Check dmesg for driver probe.
```

3.14.3 Serial Console

Serial console is the lifeline when SSH is not available – during bring-up, firmware flashing, early boot failures, or a wedged board.

```
# Connect to a device via USB serial adapter:
# First: identify the port
ls -l /dev/ttyUSB* /dev/ttyACM* # USB serial adapters
dmesg | grep -i tty | tail -10 # driver messages when the cable was plugged in

# minicom (feature-rich, saves settings):
sudo apt install minicom
minicom -s # setup: set port to /dev/ttyUSB0, baud to 115200
minicom -D /dev/ttyUSB0 -b 115200 # connect directly
# Exit minicom: Ctrl+A, then X

# screen (simple, always installed):
sudo screen /dev/ttyUSB0 115200
# Exit screen: Ctrl+A, then K (kill), confirm

# picocom (minimal, good for scripts):
sudo apt install picocom
picocom -b 115200 /dev/ttyUSB0
# Exit: Ctrl+A then Ctrl+X

# cu (basic, part of uucp):
cu -l /dev/ttyUSB0 -s 115200

# Save serial output to a file while also viewing it:
picocom -b 115200 /dev/ttyUSB0 | tee serial_$(date +%Y%m%d_%H%M%S).log

# Common baud rates for embedded boards: 115200, 57600, 38400, 19200
# Most Tegra, Qualcomm, and similar platforms default to 115200 8N1.

# If the port exists but garbled output:
# 1. Verify baud rate in the bootloader config (U-Boot: "bdfinfo" shows baudrate).
# 2. Check parity/stop bits: 8N1 is standard but some boards use 8E1.
# 3. Check for flow control: disable RTS/CTS unless the board requires it.
```

3.15 Recovering a Wedged Board

This is the escalating sequence when a board stops responding.

3.15.1 Step 1: Diagnose Before Acting

```
# Can you still ping it?
ping -c3 192.168.100.10

# Can you reach it via serial console?
# If yes: try to connect and check dmesg / systemd status before resetting.

# Is it SSH-responsive but sluggish?
ssh testeng@192.168.100.10 'uptime; free -h; dmesg -T | tail -20'
```

3.15.2 Step 2: Graceful Recovery

```
# Attempt graceful reboot via SSH:
ssh testeng@192.168.100.10 'sudo reboot'

# If SSH hangs: send magic SysRq (requires ssh access or serial console):
# Enable SysRq first (usually on by default): echo 1 > /proc/sys/kernel/sysrq
echo b > /proc/sysrq-trigger # immediate reboot (no sync or unmount)
echo s > /proc/sysrq-trigger # sync filesystems to disk
echo u > /proc/sysrq-trigger # remount all filesystems read-only
echo o > /proc/sysrq-trigger # power off

# The "REISUB" sequence via serial console or keyboard (safe, remounts FS before reboot):
# Hold Alt+SysRq and press: R E I S U B (one per second)
# R: unraw keyboard; E: SIGTERM all; I: SIGKILL all; S: sync; U: remount ro; B: reboot
```

3.15.3 Step 3: Hard Power Cycle

```
# If the board has a BMC / IPMI:
ipmitool -H 192.168.100.20 -U admin -P password chassis power status
ipmitool -H 192.168.100.20 -U admin -P password chassis power off
ipmitool -H 192.168.100.20 -U admin -P password chassis power on
ipmitool -H 192.168.100.20 -U admin -P password chassis power cycle # off then on

# If GPIO-controlled power on the test fixture:
# NOTE: The /sys/class/gpio sysfs ABI is deprecated since kernel 4.8.
# Prefer libgpiod (gpiocp / gpiocp / gpiomon) for new code.
# The sysfs interface still works on most distros but may disappear in future kernels.
echo out > /sys/class/gpio/gpio17/direction
echo 0 > /sys/class/gpio/gpio17/value # assert reset or cut power
sleep 2
echo 1 > /sys/class/gpio/gpio17/value # release

# Modern equivalent with libgpiod (install: apt install gpiod):
# gpiocp gpiocp0 17=0 # assert
# sleep 2
# gpiocp gpiocp0 17=1 # release

# If a relay-controlled PSU:
# Write the PSU control script that sets the relay line low/high.
```


3.15.4 Step 4: Post-Recovery Triage

```
# After the board comes back up, check the previous boot's evidence:
journalctl -b -1 # all messages from the previous boot
journalctl -k -b -1 # kernel messages from the previous boot
journalctl -b -1 -p err # only errors
journalctl -b -1 --since "$(date -d '2 hours ago' '+%Y-%m-%d %H:%M:%S')"
```

```
# Check for filesystem errors from an unclean shutdown:
dmesg | grep -i 'ext4|xfs|btrfs|journal|filesystem' # FS recovery messages
sudo fsck /dev/sda1 # run fsck on the root FS (unmounted; from live USB)
```

```
# Check for hardware errors:
dmesg -T | grep -iE 'mce|error|uncorrect|fatal'
# MCE = Machine Check Exception: the CPU detected an internal hardware fault
# (corrected vs. uncorrected; uncorrected MCE on memory or cache = failing hardware)
```

3.15.5 Filesystem Full / Inode Full

Both present as “no space left on device” from the application’s perspective:

```
df -h # which FS is full (check % column)
df -i # which FS has run out of inodes (% iuse column)
```

```
# Find the biggest consumers (when disk is full):
du -sh /var/log/* | sort -rh | head -20 # largest directories under /var/log
find /var/log -name '*.log' -size +100M # log files over 100 MB
journalctl --disk-usage # how much the journal uses
sudo journalctl --vacuum-time=7d # trim journal to 7 days
```

```
# When inodes are exhausted (many small files):
find /tmp -type f | wc -l # count files in /tmp
ls /var/spool/ # sometimes a runaway print queue, cron output, etc.
```

3.16 systemd Service Management

systemd is PID 1 on modern Linux. It starts services in dependency order, restarts crashed services, and collects all their logs into the journal.

```
systemctl start atf-dashboard
systemctl stop atf-dashboard
systemctl restart atf-dashboard
systemctl reload atf-dashboard # re-read config without restart (if the service supports it)
systemctl status atf-dashboard # state + PID + uptime + last log lines
systemctl enable atf-dashboard # start at boot
systemctl disable atf-dashboard
systemctl enable --now atf-dashboard # enable AND start in one command
systemctl is-active atf-dashboard # prints "active"; exit 0 if running (use in scripts)
systemctl is-enabled atf-dashboard # will it start at boot?
systemctl list-units --type=service # all loaded services
systemctl list-units --failed # services that FAILED (check this after a bad boot)
systemctl cat atf-dashboard # display the effective unit file(s)
```


3.16.1 Writing a Service Unit

```
# Place in /etc/systemd/system/ for services you create.
# /lib/systemd/system/ comes from packages; override with drop-ins, don't edit those.
sudo tee /etc/systemd/system/atf-dashboard.service >/dev/null << 'EOF'
[Unit]
Description=ATF Manufacturing Test Dashboard
After=network.target
# Wants=postgresql.service # soft dependency: start postgres too, but don't fail if it dies
# Requires=postgresql.service # hard dependency: if postgres stops, this stops too

[Service]
# Type options:
# simple : the ExecStart process IS the service (default)
# forking : daemon double-forks; systemd waits for the fork
# oneshot : runs once and exits (use for scripts, health checks)
# notify : service tells systemd via sd_notify when ready (most robust)
Type=simple
ExecStart=/usr/bin/python3 -m uvicorn dashboard.server:app --host 0.0.0.0 --port 8080
ExecReload=/bin/kill -SIGHUP $MAINPID
WorkingDirectory=/opt/atf/dashboard
Restart=on-failure
RestartSec=5
StartLimitBurst=5
StartLimitIntervalSec=60
User=testeng
Group=testeng
Environment=PYTHONUNBUFFERED=1
# StandardOutput=journal (default)
# StandardError=journal (default)

[Install]
WantedBy=multi-user.target
EOF

sudo systemctl daemon-reload # REQUIRED after creating or editing unit files
sudo systemctl enable --now atf-dashboard

# Override ONE setting without editing the unit (creates a drop-in override.conf):
sudo systemctl edit atf-dashboard
# Add: [Service] \n Environment=DEBUG=1
# Then: sudo systemctl daemon-reload && sudo systemctl restart atf-dashboard
```

3.17 Process Management and Signals

```
ps aux # all processes (USER PID %CPU %MEM ... COMMAND)
ps -ef --forest # process tree showing parent/child relationships
ps aux | grep '[p]ython' # the [p] trick excludes the grep process itself
pgrep -f test_runner # PIDs matching full command line
pgrep -la python # -l name, -a full command line
pidof python3 # exact name match
top # live: M = sort by mem, P = by CPU, 1 = per-core, k = kill
htop # nicer interactive view (if installed)

# Signals:
kill -15 <pid> # SIGTERM: ask politely (default). Process can clean up. Use first.
```

```

kill -9 <pid> # SIGKILL: force. Cannot be caught/ignored. No cleanup. Last resort.
kill -2 <pid> # SIGINT: same as Ctrl+C
kill -1 <pid> # SIGHUP: many daemons reload config on this
kill -0 <pid> # no signal sent; tests whether the PID exists (exit 0 = exists)
killall python3 # kill by name
pkill -f 'pytest' # kill by command-line pattern

# Priority:
nice -n 10 ./job # start at lower priority (higher number = nicer = lower prio)
sudo renice -n -5 -p <pid> # raise priority of running process (needs root for negative)
taskset -c 0,1 ./workload # pin to CPUs 0 and 1

# Job control:
./script.sh & # background
nohup ./script.sh & # keep running after logout (output -> nohup.out)
jobs # list background jobs in this shell
fg %1 # bring job 1 to foreground
disown %1 # detach from shell so it survives shell exit

# Diagnosing a hung process:
strace -p <pid> # show system calls -- the blocking call IS the root cause
cat /proc/<pid>/status # state, memory, thread count
cat /proc/<pid>/cmdline | tr '\0' ' ' # full command line
ls -l /proc/<pid>/fd # open file descriptors (what it has open)

```

strace deserves emphasis: when a Python test or instrument driver hangs, `strace -p <pid>` shows the exact system call it is stuck in. A `read()` on a socket fd means it is waiting for an instrument response that never came. A `poll()` on a serial fd means the device is not replying. That observation localizes a “test just hangs” bug in seconds, without touching a line of code.

3.18 Networking for Test Engineers

```

# ip (modern, always installed):
ip -br link # one-line-per-interface with state
ip addr show # interfaces with IP addresses
ip route show # routing table (look for "default via <gateway>")
ip neigh show # ARP/neighbor table (IP -> MAC)
ip -s link show eth0 # statistics: RX/TX packets, errors, drops

# Make temporary changes:
sudo ip addr add 192.168.1.50/24 dev eth0
sudo ip link set eth0 mtu 9000 # jumbo frames

# ethtool (NIC link-layer truth):
ethtool eth0 # speed, duplex, link, autoneg
ethtool -S eth0 # hardware statistics: rx_errors, tx_errors, drops, CRC
# ANY non-zero error counter = PHY, cable, SFP, or driver issue
ethtool -i eth0 # driver name + driver/firmware versions
ethtool -m eth0 # SFP/QSFP optical power, temp (if supported)
ethtool -p eth0 10 # blink the port LED for 10 s (find the right cable)

# ethtool failure signatures:
# rx_crc_errors > 0 -> physical layer issue (bad cable, SFP, or switch port)
# rx_missed_errors > 0 -> NIC ring buffer overrun under load; increase ring size:
# ethtool -G eth0 rx 4096
# tx_errors / tx_carrier_errors -> link dropped during TX; check cable and SFP power levels

```

```

# "Link detected: no" -> no carrier; check cable, switch port, SFP insertion
# Speed 100Mb/s when 10Gb expected -> auto-neg fell back; check switch config or force speed:
# ethtool -s eth0 speed 10000 duplex full autoneg off

# Sockets (ss replaces netstat):
ss -tuln # TCP+UDP listening, numeric -- "what ports are open?"
ss -tlnp '( sport = :8080 )' # who is listening on 8080
lsof -i :8080 # alternative: what process holds port 8080

# Connectivity checklist (work bottom up):
# 1. ip link show (state UP? carrier?)
# 2. ethtool eth0 (link detected? speed/duplex correct?)
# 3. ip addr (correct IP/mask?)
# 4. ping <gateway>
# 5. dig <hostname>
# 6. ss -tuln (target port open?)
# 7. curl -v http://host:port/health
# 8. sudo iptables -L (rule blocking it?)

# Packet capture:
sudo tcpdump -i eth0 -c 100 -w cap.pcap # capture 100 packets to file (open in Wireshark)
sudo tcpdump -i eth0 port 502 # Modbus/TCP filter
tcpdump -r cap.pcap | head

# Throughput test:
iperf3 -s # server
iperf3 -c 192.168.1.100 -t 30 # TCP bandwidth test, 30 s
iperf3 -c 192.168.1.100 -u -b 10G # UDP at 10 Gbps target (loss/jitter)

```

3.19 Hardware Debug Command Reference

The quick-reach section for when a board is on the fixture throwing errors. Organized by the question you are answering.

3.19.1 “Is the Device Present and on What Link?”

```

lspci -nn # all devices with numeric IDs
lspci -d 10de: -nn # only NVIDIA devices
lspci -t # topology tree
lspci -s 03:00.0 -vvv | grep -i 'lnkcap\|lnksta' # capable vs actual speed/width
cat /sys/bus/pci/devices/0000:03:00.0/current_link_speed
cat /sys/bus/pci/devices/0000:03:00.0/max_link_speed
lspci -k -s 03:00.0 # which driver is bound
lsusb -t; nvme list; lsblk # USB tree, NVMe namespaces, block devices
lshw -short # one-line full inventory

```

3.19.2 “Is the Link Degraded or Erroring?”

```

# Degradation = current != max:
for d in /sys/bus/pci/devices/*; do
  c=$(cat "$d/current_link_speed" 2>/dev/null) || continue
  m=$(cat "$d/max_link_speed" 2>/dev/null) || continue
  [[ "$c" != "$m" ]] && echo "SPEED DEGRADED ${d##*/}: $c (max $m)"
done

```

```
# AER error counters (kernel-tracked):
cat /sys/bus/pci/devices/0000:03:00.0/aer_dev_correctable
cat /sys/bus/pci/devices/0000:03:00.0/aer_dev_nonfatal
cat /sys/bus/pci/devices/0000:03:00.0/aer_dev_fatal
lspci -s 03:00.0 -vvv | grep -A12 'Advanced Error Reporting'
dmesg -T | grep -iE 'aer|pcie|corrected|malformed'
```

3.19.3 “What Is the Kernel Saying?”

```
dmesg -T | tail -50
dmesg -T | grep -iE 'error|fail|fault|timeout|reset'
dmesg -T | grep -i nvme # NVMe resets/timeouts
dmesg -T | grep -i 'thermal|throttling' # thermal throttling
dmesg -w # follow live during a stress run
sudo dmesg -C # clear before a test, re-read after
journalctl -k -b # persisted kernel log for this boot
```

3.19.4 “Storage Health?”

```
nvme list
sudo nvme smart-log /dev/nvme0 # temp, percentage_used, media_errors, crit
sudo nvme error-log /dev/nvme0 # error log entries
sudo smartctl -H /dev/sda # SATA: PASSED / FAILED
sudo smartctl -a /dev/sda # full SMART attributes
lsblk -o NAME,SIZE,TYPE,MOUNTPOINT,MODEL,SERIAL
```

3.19.5 “Network Link?”

```
ip -br link # up/down at a glance
ethtool eth0 # speed, duplex, autoneg
ethtool -S eth0 | grep -iE 'err|drop|crc' # error counters (all should be 0)
ethtool -i eth0 # driver + firmware version
cat /sys/class/net/eth0/carrier # 1 = link detected
ping -c4 <gateway>; ss -tuln
```

3.19.6 “Thermals, Power, CPU, Load?”

```
sensors # lm-sensors: temps, fans, voltages
paste <(<cat /sys/class/thermal/thermal_zone*/type) \
    <(<cat /sys/class/thermal/thermal_zone*/temp) | \
    awk '{printf "%-22s %.1f C\n", $1, $2/1000}'
sudo turbostat --interval 1 --num_iterations 3 # per-package power, frequency, temp
lscpu; nproc # CPU topology
free -h # memory usage
uptime # load averages (compare to nproc)
cat /proc/interrupts | grep -iE 'nvme|eth' # are device IRQs firing and spread?
```

3.19.7 “Perf Bottleneck?”

```
vmstat 1 5 # CPU vs memory (si/so) vs IO-wait (wa)
iostat -x 1 # per-disk %util / await
mpstat -P ALL 1 # per-core usage (find one pegged core)
```

```
pidstat 1 # per-process CPU over time
strace -p <pid> # what a hung/slow process is blocked on
sudo perf top -p <pid> # which functions are eating CPU
```

3.19.8 “Driver/Firmware OK?”

```
lspci -k -s 03:00.0 # is the right driver bound?
lsmod | grep <driver_name> # is the module loaded?
modinfo <driver_name> | grep version # what version?
ethtool -i eth0 # NIC firmware version
nvme id-ctrl /dev/nvme0 | grep -i 'fr\|sn\|mn' # NVMe firmware rev, serial, model
dmesg | grep -i <driver_name> # driver init / bind / error messages
```

3.19.9 The Mental Checklist

When a board fails on the fixture, drive toward the **failure tuple**, not a one-word verdict. “It failed” is useless; “endpoint 0000:03:00.0 trained Gen3 x8 instead of Gen4 x16, correctable AER BadTLP counter climbing at 3/sec under load, package temp 78 C, NVMe SMART clean” is something an Electrical Engineering (EE) can act on.

1. **Present?** `lspci -nn / sysfs` – does it enumerate at all?
2. **Right link?** `current_link_speed / current_link_width` vs `max_*` – degraded?
3. **Erroring?** AER counters + `dmesg -T` – which specific bits, how fast accumulating?
4. **Healthy peripherals?** `nvme smart-log`, `ethtool -S` – clean counters?
5. **Environment?** sensors / thermal zones, `vmstat / iostat` – hot or resource-starved?
6. **Driver/firmware?** `lspci -k`, `ethtool -i`, `modinfo` – right versions bound?
7. **Capture the evidence:** tee every output into a timestamped log file; hand the decoded failure tuple to the EE, not a one-word verdict.

Chapter 4

PCIe: Architecture, Link Training, AER, and Diagnosis

4.1 Why this chapter is the long one

The Zoox JD’s bonus qualification reads “*strong expertise in PCIe troubleshooting — especially with GPUs and similar electronic assemblies.*” On the compute platform PCIe is the spine: it carries every GPU, every Non-Volatile Memory Express (NVMe) drive, every custom sensor-interface card and switch/fan-out board. Most of what you debug on the line is a SerDes link that trained wrong, drifted with temperature, or quietly dropped a lane — and almost all of those wear a PCIe costume. So this chapter goes deep and stays practical: the **register offsets and bit-fields you actually read and write**, real `lspci/setpci` output, the **failure signature** → **root-cause-layer** mapping, and how each finding lands in **manufacturing test (MT)**.

The organizing idea, repeated until it is reflex:

A PCIe failure is never errors=5. It is a *tuple*: which Advanced Error Reporting (AER) bits, on which lane, in which direction, with which header log, at which retrain count, with how much eye margin, at what temperature and load. That tuple is the root cause. Everything here exists to collect that tuple efficiently and read it.

Two cross-references so this chapter doesn’t re-derive what lives elsewhere:

- The **Bit Error Rate (BER) confidence math** (Poisson model, the “3/BER” rule, the chi-squared upper bound) is derived in the **Math & Statistics chapter**, . Here we *use* it — when you see a confidence number, that’s where the formula came from.
- The companion **toolkit/** implements this chapter: **aer.py** (decode + Write-1-to-Clear (W1C) clear), **linkstate.py** (speed/width + retrain/latch monitor), **ber.py** (the confidence math), **bert.py** (the arm→stress→read→decide loop), **margin.py** (lane margining + the Transmit (TX)-preset sweep), **topology.py/diagnostics.py** (the whole-chain walk). Read a section, then read the code that does it.

4.2 Architecture: the chain you actually debug

PCIe is a **point-to-point, serial, packet-based** interconnect. Each *link* connects exactly two ports — there is no shared bus. Multiple **lanes** (x1, x2, x4, x8, x16) are bonded for bandwidth; each lane is a differential pair in each direction (TX+/TX−, Receive (RX)+/RX−), so a link is **full-duplex** — it sends and receives simultaneously, and the two directions are independent. *That independence matters for test*: a bit-error problem can exist on one direction of one link and not the other, which is why errors are evaluated **per receiver**, not per link.

4.2.1 Topology and the four port types

The hierarchy fans out from the CPU:

```
CPU / Root Complex
|
[Root Port] <- a Downstream Port (DSP); owns the link below it
| link (its own LTSSM, AER, equalization)
[Switch Upstream Port] <- USP: faces the root
/ | \
[DSP] [DSP] [DSP] <- each switch Downstream Port: its own link/LTSSM/AER
| | |
Endpoint Endpoint Endpoint (GPU, NVMe, NIC, custom card)
```

Port type	Code (Device/Port Type — PCIe cap +0x02, bits [7:4])	Role
Endpoint	0x0	A leaf: GPU, NVMe controller, NIC, custom FPGA card
Root Port	0x4	A Root Complex egress port; a downstream port
Switch USP	0x5	The single port of a switch that faces the root
Switch DSP	0x6	One of N switch ports that face leaves; downstream

The term **Downstream Port (DSP)** = root port *or* switch downstream port; it **owns the link below it** (and the link’s bandwidth-change latches). **Upstream Port (USP)** = an endpoint, or a switch’s upstream port. This distinction is load-bearing for diagnosis: the DSP is where speed/width and the Link Bandwidth Management Status (LBMS)/Link Autonomous Bandwidth Status (LABS) latches for that link live, and the root port is the only place the kernel assembles the OS-level error story.

A **switch** is internally one USP bridged to N DSPs; **every one of those ports has its own Link Training and Status State Machine (LTSSM), its own AER capability, and its own link registers**. A fallback or an error cluster can be on *any* segment — so when you debug a device behind a switch you walk the whole tree (`lspci -tv`) and check each link, not just the endpoint.

4.2.2 BDF: how every function is addressed

Every PCIe function is named by **Bus/Device/Function (BDF) = domain:bus:device.function**, e.g. 0000:03:00.0. Domain (a.k.a. segment) is usually 0000; bus is assigned during enumeration; device is 0–31 on a bus; function is 0–7 (multi-function devices, and Single Root I/O Virtualization (SR-IOV) VFs, use the function digit). Config reads/writes are addressed by BDF; AER’s **Error Source ID** reports the BDF of the requester that caused an error. Throughout this chapter BDF=0000:03:00.0 is the running example.

4.2.3 The layer model

Like networking, PCIe is layered — and as in networking, the fastest first question on any failure is “*which layer?*”

Layer	Packet	Job	What breaks here
Transaction (TL)	TLP	Carries reads, writes, completions, messages	Protocol: wrong address, timeout, malformed packet
Data Link (DLL)	DLLP	Reliability: sequence numbers, LCRC, ACK/NAK, replay, flow-control credits	Link-level: corrupted TLPs get NAK'd and replayed
Physical (PL)	PLP / ordered sets	SerDes, 8b/10b or 128b/130b coding, equalization, lane bonding, the LTSSM	SI: symbol errors, closed eye, retraining

- **TLP — Transaction Layer Packet.** The payload-carrying unit. Types: **Memory Read/Write** (access a device’s Base Address Register (BAR) space — a write is *posted*, no completion; a read is *non-posted*, needs a Completion), **Config Read/Write** (BDF-addressed access to config space), **Completion** (the response to a non-posted request — carries read data or a status code), **I/O** (legacy), and **Message** (Message Signaled Interrupt (MSI)/Message Signaled Interrupt Extended (MSI-X) interrupts, error signaling, power management).
- **DLLP — Data Link Layer Packet.** Small, link-local, never routed: ACK/NAK, flow-control credit updates, power-management. DLLPs are **not retried** — a corrupted one is just dropped (which is why a *Bad DLLP* correctable is a pure signal-integrity tell,).
- **PLP / ordered sets** — the physical-layer framing and training sequences (TS1/TS2, SKP, EIEOS) that the LTSSM uses to bring a link up and keep it locked.

Reliability is at the Data Link Layer (DLL), by replay. Every TLP gets a **sequence number** and an **Link CRC (LCRC)**. The receiver checks the LCRC, and ACKs good TLPs / NAKs bad ones; an un-ACKed TLP is **replayed** from a retry buffer. This is the mechanism behind the correctable-error counters you live in (Bad TLP, Replay Timer Timeout, REPLAY_NUM Rollover) — they are the DLL telling you the physical layer is corrupting packets and the link is retrying.

Flow control is credit-based, at the TL. The receiver advertises buffer space (credits) per TLP class — **Posted, Non-Posted, Completion** — and the transmitter may not send without credits. This prevents receiver overflow without per-packet ACKs at the transaction layer. A credit-accounting bug surfaces as a **Flow Control Protocol** or **Receiver Overflow** uncorrectable.

4.3 Generations, widths, encoding, and the GT/s → GB/s math

This is a place to be precise — getting the bandwidth math wrong in front of Electrical Engineering (EE) is an easy own-goal, and the numbers drive your pass/fail expectations.

Gen	Raw rate	Encoding	Coding overhead	Effective BW/lane	x16 bandwidth (one dir)
Gen1	2.5 GT/s	8b/10b	20%	250 MB/s	4.0 GB/s
Gen2	5.0 GT/s	8b/10b	20%	500 MB/s	8.0 GB/s
Gen3	8.0 GT/s	128b/130b	~1.5%	~984.6 MB/s	~15.75 GB/s
Gen4	16 GT/s	128b/130b	~1.5%	~1.969 GB/s	~31.5 GB/s
Gen5	32 GT/s	128b/130b	~1.5%	~3.938 GB/s	~63 GB/s
Gen6	64 GT/s	PAM4 + 256B FLIT + FEC/CRC	~few % (FEC, CRC, FLIT framing)	~7.56 GB/s	~121 GB/s

The math, worked. “GT/s” is **giga-transfers per second** — one bit per transfer for the Non-Return-to-Zero (NRZ) gens (Gen1–5), so the raw bit rate per lane equals the GT/s number in Gbit/s. Apply the coding efficiency, then divide by 8 for bytes:

```
Gen3 x1: 8.0 GT/s x (128/130) = 7.877 Gbit/s -> /8 = 0.9846 GB/s per lane
Gen4 x16: 16 GT/s x (128/130) x 16 / 8 = 31.5 GB/s (one direction)
Gen5 x16: 32 GT/s x (128/130) x 16 / 8 = 63 GB/s
```

Two facts to internalize:

- **8b/10b** (Gen1/2) maps 8 data bits to 10 line bits to guarantee DC balance and enough transitions for clock recovery → a flat **20% overhead**. **128b/130b** (Gen3–5) sends 128 payload bits under a 2-bit sync header and uses *scrambling* (not block-coding) for DC balance → only **~1.5% overhead**.
- **The Gen2→Gen3 jump is bigger than 1.6×**. The raw rate went 5→8 GT/s (1.6×) *and* the overhead dropped 20%→1.5%, so per-lane bandwidth roughly **doubled** (500 → ~985 MB/s). That combined step is why “Gen3” is such a watershed and why everything above it needs equalization.

Gen6 changes the signaling and the error model. Gen6 uses **Pulse Amplitude Modulation 4-level (PAM4)** (4 voltage levels → 2 bits per symbol, so 64 GT/s at the same ~32 GBaud as Gen5), moves to a fixed **256-byte Fixed-size Link Packet (FLIT)** with **FEC (lightweight LDPC Forward Error Correction (FEC)) + a strong CRC + replay** instead of LCRC-retry, and **eliminates standalone DLLPs** (ACK/NAK and flow-control move *inside* the FLIT). The 256-byte FLIT budgets roughly **236 B of TLP payload, ~6 B of DLP (the old DLLP content), 8 B CRC, and ~6 B FEC** — so the DLLP information is still there, just carried in the FLIT rather than as separate, droppable link packets. Because PAM4 has a *raw* symbol-error rate orders of magnitude worse than NRZ (three eyes instead of one), the spec leans on FEC to pull the post-correction error rate back down — FEC keeps the link-retry probability under ~1e-5. The practical consequence for your tools: **an AER-correctable Bit Error Rate Test (BERT) under-measures a Gen6 link**, because most symbol errors are FEC-corrected and never become Bad-TLP/replay events. A true Gen6 error rate comes from **FEC corrected/uncorrected symbol counters** (and the CRC/retry rate), not AER correctables. The toolkit flags this (`bert.py` emits a note when `current_link_speed >= 6`); you check the **Flit Mode Status** bit before trusting LCRC-retry accounting (the LTSSM section covers reading it).

Bandwidth → time, for sizing a BERT. Gen4 x16 ≈ 31.5 GB/s ≈ 2.5×10^{11} bit/s; an NVMe Gen3 x4 link ≈ 3.5 GB/s. These set how long a confidence BERT must run: 3×10^{12} clean bits is ≈ **12 s** on Gen4 x16 and ≈ **110 s** on Gen3 x4.

4.4 Config space is just a file you can read

Every PCIe function exposes a **256-byte** legacy config space, extended to **4096 bytes** for PCIe (where AER and the other extended capabilities live). On Linux it is literally a file:

```
/sys/bus/pci/devices/0000:03:00.0/config # raw 4096-byte config space
/sys/bus/pci/devices/0000:03:00.0/current_link_speed # "16.0 GT/s"
/sys/bus/pci/devices/0000:03:00.0/current_link_width # "16"
/sys/bus/pci/devices/0000:03:00.0/max_link_speed # capability ceiling
/sys/bus/pci/devices/0000:03:00.0/max_link_width
```

You can read any register by `pread-ing config` at the right offset — which is exactly what the toolkit’s back-end does (`read_config(bdf, offset, size)`), so it never fragile-parses `lspci` text. Reading the extended space (`offset ≥ 0x100`) needs root.

4.4.1 The header and the layout you navigate

```
0x000 +-----+
| Type 0 (EP) or Type 1 hdr | Vendor/Device ID, Command, Status, Class, BARs...
0x040 +-----+
| Capability list | linked list via "next pointer" bytes:
| - PCI Express Cap (ID 0x10) | -> Link Cap/Ctrl/Status, Dev Ctrl/Status 2
| - MSI / MSI-X |
| - Power Management |
0x100 +-----+ <- Extended config space (PCIe-only, root needed)
| Extended Capability list | another linked list:
| - AER (0x0001) | -> the error registers
| - Secondary PCIe (0x0019) | -> Gen3+ equalization control
| - Lane Margining (0x0027) | -> receiver eye margin per lane (Gen4+)
| - DPC (0x001D), ACS, SR-IOV ...
0xFFFF +-----+
```

The header registers you check first on any device:

Offset	Register	Size	What to verify
0x00	Vendor ID	16-bit	Expected vendor (0x10DE NVIDIA, 0x144D Samsung NVMe, 0x8086 Intel)
0x02	Device ID	16-bit	Expected GPU/NVMe/card model
0x04	Command	16-bit	Bit 2 = Bus Master Enable (must be set for DMA)
0x06	Status	16-bit	Capabilities-list present (bit 4); error summary bits
0x08	Revision ID	8-bit	Silicon revision
0x09	Class Code	24-bit	0x030000 VGA, 0x030200 3D controller, 0x010802 NVMe
0x0E	Header Type	8-bit	0x00 = Type 0 (endpoint), 0x01 = Type 1 (bridge/switch port)
0x10–0x27	BAR 0–5	32-bit ea	Base Address Registers — where the device’s MMIO is mapped

Memory-Mapped I/O (MMIO) vs the BARs, and why “unassigned” = dead. A modern PCIe device’s registers are mapped into the CPU’s memory space (MMIO); the BAR tells the OS where. If `lspci -vvv` shows **Region 0: Memory at <unassigned> or BAR ... can't assign**, the device is **visible but non-functional** — software cannot reach its registers. On a custom card this usually means the BIOS couldn’t fit the requested window (often a 64-bit-prefetch/above-4G setting). It is a top “detected but does nothing” cause.

4.4.2 Walking the capability lists by hand (once)

Capabilities are linked lists, and walking one once cements how `setpci`’s aliases and the toolkit’s `find_ext_cap()` work.

- **Legacy caps** start at the byte pointed to by config 0x34. Each cap is `[cap_id (1B)][next_ptr (1B)] ...`; follow `next` until it’s 0. The **PCI Express Capability** has ID 0x10 — Link/Device control and status hang off it.
- **Extended caps** start at 0x100. Each has a 32-bit header: `[cap_id:16][cap_version:4][next_offset:12]`. Follow `next_offset` until 0 (or the header reads 0x00000000/0xFFFFFFFF). **AER = ID 0x0001**, **Secondary PCIe = 0x0019**, **Lane Margining = 0x0027**, **Downstream Port Containment (DPC) = 0x001D**.

```
# toolkit/backend.py - the extended-cap walk (paraphrased)
offset = 0x100
while offset:
    header = read_config(bdf, offset, 4)
    if header in (0, 0xFFFFFFFF):
        break
    if (header & 0xFFFF) == cap_id: # low 16 bits = capability ID
        return offset # found it (e.g. AER at 0x100)
    offset = (header >> 20) & 0xFFF # next-capability offset
```

`setpci` gives you symbolic aliases so you don’t hard-code these: **CAP_EXP** = the PCIe capability base, **ECAP_AER** = the AER extended-cap base, **ECAP_VSEC** = a vendor-specific extended cap. So `setpci -s $BDF CAP_EXP+0x12.W` reads Link Status regardless of where the PCIe cap physically sits.

4.5 Link training, the LTSSM, and the latched status bits

4.5.1 The LTSSM

The **LTSSM** brings a link up and maintains it. Knowing the states maps a “stuck” symptom directly onto a physical cause:

State	What happens	“Stuck here” means
Detect	Look for a partner (sense far-end Receive termination)	No device / no power / PERST# (PERST#) not released / missing AC-cap or termination / dead PHY — the classic “ <i>device not detected.</i> ”
Polling	Establish bit + symbol lock, exchange TS1/TS2, agree polarity (always at 2.5 GT/s)	REFCLK absent/wrong, SSC mismatch, RX DC offset, gross SI, inverted polarity.
Configuration	Negotiate link width + lane numbers (x16→x8 falls out here)	Dead/noisy high lanes, lane-reversal unsupported, bifurcation mismatch , scrambler problem.
L0	Normal operation — data flows. This is where you want to live.	—
Recovery	Re-enter training to change speed, run equalization (Gen3+), or recover from errors	Looping Recovery = marginal SI / equalization (EQ) preset mismatch / RX won’t lock at the new rate / thermal drift. The #1 <i>dynamic</i> failure.
L0s / L1 / L2	Low-power link states	A too-slow exit can manufacture a CTO.

Speed changes happen in Recovery, and so does **equalization**. For Gen3+ the link first trains to Gen1 in Polling, reaches L0, then renegotiates up through Recovery — equalizing at each higher rate. **A link that *trains* to Gen4 but keeps re-entering Recovery is marginal even though a snapshot shows Gen4.** This is why your monitor watches the training bit over the soak, not just the final state.

4.5.2 Reading link speed, width, and the latches (PCIe Capability)

The negotiated link state lives in the **PCIe Capability** (not AER). Offsets are relative to the PCIe-cap base (CAP_EXP):

Register	Offset	Key fields
Link Capabilities (LnkCap)	+0x0C	Max speed [3:0], max width [9:4], DLL Link Active Reporting Capable (bit 20)
Link Control (LnkCtl)	+0x10	ASPM control [1:0], Retrain Link (bit 5) , Link Disable (bit 4), Common Clock (bit 6)
Link Status (LnkSta)	+0x12	Current speed [3:0] , current width [9:4] , Link Training (bit 11) , DLLA (bit 13) , LBMS (bit 14) , LABS (bit 15)
Device Control 2 (DevCtl2)	+0x28	CTO Value [3:0] , CTO Disable (bit 4)
Device Capabilities 2 (DevCap2)	+0x24	Supported CTO ranges, CTO-disable-supported
Link Control 2 (LnkCtl2)	+0x30	Target Link Speed [3:0] (force a gen for retrain)

Register	Offset	Key fields
Link Status 2 (LnkSta2)	+0x32	Flit Mode Status (bit 10) — set in Gen6 FLIT mode

Speed code → **GT/s**: 1=2.5 (Gen1), 2=5 (Gen2), 3=8 (Gen3), 4=16 (Gen4), 5=32 (Gen5), 6=64 (Gen6). Both `LnkCap[3:0]` (max) and `LnkSta[3:0]` (current) use this code.

The easiest read of the **current** speed/width is `sysfs (current_link_speed / current_link_width)`, which mirrors `LnkSta[3:0] / [9:4]`. But knowing the register lets you read it via `setpci` when `sysfs` is stale, or on a port behind a switch:

```
setpci -s $BDF CAP_EXP+0x12.W # raw Link Status word (CLS/NLW/LT/DLLA/LBMS/LABS)
setpci -s $BDF CAP_EXP+0x0c.L # Link Capabilities (max speed/width, DLLARC bit 20)
```

4.5.3 The latched bits a snapshot misses

A 5 ms poll on “is it Gen4 x16 *right now*” misses a link that bounced to Recovery and back between polls. `LnkSta` carries **latches that survive between reads** — these catch the transients:

LnkSta bit	Name	What it tells you
11	Link Training (LT)	Set <i>while</i> retraining (in Recovery). Flicking = the link is bouncing.
13	DLLA (DL Link Active)	Drops to 0 on a link-down (DL_Down). A latched link-down detector — <i>but only meaningful if LnkCap bit 20 is set</i> .
14	LBMS (LBMS)	W1C latch : set on a completed software retrain or when hardware dropped speed/width to correct unreliable link operation — the “couldn’t hold the rate” tell.
15	LABS (LABS)	W1C latch : set when hardware changed speed/width autonomously for reasons other than reliability (power / bandwidth-on-demand) — not a reliability fault by itself.

LBMS is the gem. Spec-precisely, LBMS (bit 14) is the bit hardware sets when it dropped speed/width **to correct unreliable link operation** — so LBMS latching after a soak (with no software-initiated retrain in the window) means the link **couldn’t hold the rate** and renegotiated down. LABS (bit 15) is the *autonomous, non-reliability* change (power / bandwidth-on-demand). A one-shot “current speed = Gen4 x16” check *passes* a unit that bounced; the LBMS latch *fails* it correctly — the canonical “**trains fine, marginal under load/temperature**” catch, for one extra register read. Arm **both** adjacent W1C bits before the soak (and read a set LBMS as the reliability tell):

```
setpci -s $BDF CAP_EXP+0x12.W=0xc000 # write 1 to bits 14,15 -> clear LBMS/LABS (arm)
# ... run stress / thermal soak ...
setpci -s $BDF CAP_EXP+0x12.W # re-read: bit14(LBMS)=reliability downgrade, bit15(LABS)=autonomous;
↪ either => renegotiated
```

Read the latch at the downstream port, not the endpoint. LBMS/LABS describe a *link*, and a link is owned by the **downstream port** above it (root port or switch DSP). On the **endpoint** side of the link those bits are `RsvdZ` — they read as 0 forever. Arm and re-read at

the DSP's BDF (resolve it with `lspci -t`, or the toolkit's `link_chain()` / `read_port_type()`), not at the endpoint you happen to be probing. Point this check at the wrong end and your bandwidth-change detector silently never fires — it polls a hard-wired 0 and *PASS*es every soak. (The toolkit was reading \$BDF at the endpoint; the fix was to resolve the owning downstream port. It's the most common way the LBMS/LABS check gets *written* but never *works*.)

What the words actually look like, decoded by hand (Link Status is 16-bit; width field is bits [9:4], so x16 = field value 0x10 sits at bit 8 = 0x0100):

```
# Healthy Gen4 x16, link up, freshly armed, held the rate through the soak:
$ setpci -s 0000:03:00.0 CAP_EXP+0x12.W
2104
0x2104 = 0010 0001 0000 0100b
[3:0] = 0x4 -> Current Link Speed = 16 GT/s (Gen4)
[9:4] = 0x10 -> Current Link Width = x16 (the 0x0100 bit)
bit11 (LT) = 0 -> not training right now
bit13 (DLLLA) = 1 -> data link active (the 0x2000 bit) -- link is up
bit14 (LBMS) = 0, bit15 (LABS) = 0 -> nothing renegotiated. PASS.

# Same link AFTER a hot soak -- it bounced down and recovered:
$ setpci -s 0000:03:00.0 CAP_EXP+0x12.W
e104
0xE104 = 1110 0001 0000 0100b
[3:0]=0x4 (16 GT/s), [9:4]=0x10 (x16) -> ends Gen4 x16 again (a snapshot would PASS)
bit13 (DLLLA) = 1 -> link is up
bit14 (LBMS) = 1 -> dropped speed/width to correct unreliable operation -> could NOT hold the rate
↪ -> FAIL
bit15 (LABS) = 1 -> ALSO an autonomous (non-reliability) bandwidth change in the window
```

The point: both reads end at “Gen4 x16,” so the speed/width fields alone pass the unit. Only the **latched** LBMS/LABS bits (the high nibble: 0xE... with bits 14-15 set vs 0x2... with them clear) separate the healthy link from the one that dropped out and clawed back during the soak.

The toolkit's `linkstate.check_link(..., watch_s=...)` does exactly this: it arms the latches, polls `LnkSta` over the window counting **retrains** (rising edges of LT), tracks the **minimum** speed/width seen, and flags **bw_changed** if LBMS/LABS latched. A link that ends at Gen4 x16 but retrained 40 times during the soak fails — a one-snapshot test would pass it.

Gen6 / Flit mode. When `LnkSta2+0x32` bit 10 (Flit Mode Status) is set, the error model is FEC-based, not LCRC-retry — switch your mental model before trusting AER correctable counts on Gen6 silicon.

The full LTSSM state is *not* in standard config space. Detect/Polling/Config/Recovery live in **vendor-specific** registers (Broadcom/PLX, Intel) or a PHY/debug interface — and on NVIDIA GPUs, partially via `nvidia-smi`. What you *can* observe portably: the LT bit flicking, **dmesg** link-down/up and speed-change lines, and recovery-entry counters where a device/switch exposes them.

4.6 AER in depth — the bits are the layer

AER is the extended capability (ID 0x0001) where the hardware **latches** correctable and uncorrectable errors. Its entire diagnostic value is that **the bit names the layer**, which names the root-cause class. Your tool must never just say **errors=5** — it must say *which bits*, because that’s the first fork in the debug tree.

4.6.1 AER register map (offsets relative to the AER cap base)

Offset	Register	Notes
+0x00	Capability Header	Cap ID 0x0001
+0x04	Uncorrectable Error Status (UNCOR_STATUS)	W1C . Non-zero after a clean run = FAIL
+0x08	Uncorrectable Error Mask	1 = not <i>reported</i> (may still latch in status)
+0x0C	Uncorrectable Error Severity	1 = Fatal (→ ERR_FATAL, link reset / DPC), 0 = Non-Fatal
+0x10	Correctable Error Status (COR_STATUS)	W1C . Recovered errors; rising count = SI concern
+0x14	Correctable Error Mask	1 = masked
+0x18	Adv. Error Cap & Control (ERR_CAP)	First Error Pointer [4:0] (which UNCOR bit the header log belongs to); ECRC gen/chk enables
+0x1C	Header Log (16 bytes)	First 4 DWORDs of the TLP that caused the first uncorrectable error
+0x2C	Root Error Command	Root ports / RCEC only
+0x30	Root Error Status	Root ports only
+0x34	Error Source ID	Root ports only — requester ID of the COR / UNCOR source

Root-port-only caveat (often gotten wrong). Root Error Command (+0x2C), Root Error Status (+0x30), and Error Source ID (+0x34) exist **only on Root Ports and Root Complex Event Collectors (RCECs)** — *not* on endpoints or switch downstream ports. To learn “which requester caused this,” read **Error Source ID on the root port above the device**, not on the device. The root port is where the kernel AER driver assembles the OS-level story.

4.6.2 Correctable Error Status (COR_STATUS, +0x10) — the SI early-warning system

Bit	Mask	Name	Meaning / typical root cause
0	0x0001	Receiver Error	Raw PHY symbol / 8b10b / 128b130b / sync-header error. Pure SI (loss, jitter, crosstalk, marginal EQ).
6	0x0040	Bad TLP	TLP with bad LCRC /sequence → NAK’d & replayed. SI (corruption on the wire).
7	0x0080	Bad DLLP	An ACK/NAK/FC/PM DLLP failed CRC. SI (DLLPs aren’t retried, just dropped).

Bit	Mask	Name	Meaning / typical root cause
8	0x0100	REPLAY_NUM Rollover	Replay counter wrapped (4 consecutive replays of one TLP). SI / marginal link.
12	0x1000	Replay Timer Timeout	No ACK before REPLAY_TIMER expired → replay. Classic marginal-link symptom.
13	0x2000	Advisory Non-Fatal	An uncorrectable error was <i>demoted</i> to advisory. Read UNCOR_STATUS to see what was demoted.
14	0x4000	Corrected Internal Error	Device-internal corrected error (its own RAM ECC). Device-side, not the link.
15	0x8000	Header Log Overflow	More uncorrectables than the 1-deep log could hold → <i>many</i> uncorrectables; go fix those.

The single most important correctable signature: *Replay Timer Timeout + Bad TLP + REPLAY_NUM Rollover clustering* = the link is repeatedly corrupting TLPs and retrying = **physical-layer / signal integrity**. See that cluster and you are in (SI), not (protocol). The toolkit's `aer.CORRECTABLE_BITS` table carries exactly these names and meanings so a decode reads like a sentence.

4.6.3 Uncorrectable Error Status (UNCOR_STATUS, +0x04) — any bit set after a clean run = FAIL

Bit	Mask	Name	Meaning / typical root cause
4	0x0000_0010	Data Link Protocol	ACK/sequence-number protocol violation. Protocol/IP bug , sometimes severe SI corrupting seq#s.
5	0x0000_0020	Surprise Down	Link dropped to DL_Down unexpectedly. Device/power lost, cable yanked, far-end PHY died, hot-unplug.
12	0x0000_1000	Poisoned TLP Received	TLP arrived with the EP (poison) bit set (sender marked its own data bad). Upstream data corruption , not this link's SI.
13	0x0000_2000	Flow Control Protocol	Credit accounting violated the rules. Firmware/IP bug (or corrupted FC DLLPs).

Bit	Mask	Name	Meaning / typical root cause
14	0x0000_4000	CTO	A non-posted request (usually a read) never got its completion before the CTO timer fired. Upstream hung / wrong address / ASPM-L1 exit too slow / switch dropped it / FW.
15	0x0000_8000	Completer Abort	The completer refused the request (returned CA). Target-side: bad address, permission, completer-internal error.
16	0x0001_0000	Unexpected Completion	A completion with no matching outstanding request. Switch mis-routing / duplicate tags / firmware.
17	0x0002_0000	Receiver Overflow	More TLP data than advertised credits. Flow-control/IP bug (transmitter overran).
18	0x0004_0000	Malformed TLP	Structurally invalid TLP (bad length/byte-enables/addr/type). FW/IP bug or severe corruption. Usually Fatal by default.
19	0x0008_0000	ECRC Error	ECRC mismatch (only if ECRC gen+check enabled). End-to-end corruption an LCRC didn't catch through a switch/retimer.
20	0x0010_0000	Unsupported Request	Target doesn't support the request (access to an unimplemented BAR/region). Address-map / enumeration / driver bug.
21	0x0020_0000	ACS Violation	ACS blocked a peer-to-peer TLP. IOMMU/ACS policy (often <i>expected</i> under virtualization).
22	0x0040_0000	Uncorrectable Internal	Device-internal uncorrectable (its own logic/RAM). Device-side.
23–31	—	MC-Blocked / AtomicOp-Egress / TLP-Prefix-Blocked / IDE / PCRC	Newer-spec integrity, encryption, translation errors. Decode them anyway so an unexpected high bit reads as a name, not “unknown bit 24.”

The fork this table encodes: **Receiver Error / Bad TLP / Replay Timer** cluster → physical SI (reseat, temperature, margining, equalization). **Completion Timeout (CTO) / Malformed / Unexpected Completion / FCP** → transaction/protocol (upstream device, switch, firmware). *Where* the bit sits is the first half of triage.

Decode the full word. A naive decoder that stops at bit 22 shows an IDE or AtomicOp error as “unknown bit 24” and sends a tech down the wrong path. Decode bits 23–31 even though they’re rare on an ordinary compute board — being able to *name* an unexpected bit is half of triage.

4.6.4 Severity and mask — read them *before* you stress

UNCOR_SEVERITY (+0x0C) selects **Fatal(1)/Non-Fatal(0) per bit**; a Fatal error triggers ERR_FATAL → link reset / DPC. *_MASK selects whether the error is *reported* (generates a message/interrupt); masked errors may still latch in status. **Read severity + mask in the arm step** so you know which bits will trigger DPC or a link reset *under you* mid-stress — otherwise the device vanishes and you blame the wrong thing.

```
setpci -s $BDF ECAP_AER+0x0c.L # UNCOR severity (1=Fatal -> ERR_FATAL/DPC)
setpci -s $BDF ECAP_AER+0x08.L # UNCOR mask
setpci -s $BDF ECAP_AER+0x14.L # COR mask
```

4.6.5 The write-1-to-clear discipline (your tool’s core primitive)

AER status bits are **write-1-to-clear (W1C)**: writing a 1 clears a bit, writing 0 does nothing. Two consequences drive the tool design:

1. **You must arm (clear) before a measurement.** The latches accumulate **from boot** — through power-on glitches, enumeration, link training, and every prior test. If you read without arming first you’re reading history, not your stress window. **Arm → stress → read** is the only valid sequence.
2. **Clear only the bits that are set, and verify.** The elegant W1C trick: read the status register, then **write the value you just read back to it**. Because only the set bits are 1 in that value, the write clears exactly those and touches nothing else. Then re-read and confirm 0.

```
// toolkit aer_clear_one() (conceptual; real path is pread/pwrite on /sys/.../config)
uint32_t sts = read_config(fd, aer_base + AER_COR_STATUS, 4);
if (sts != 0) { // only act if something is set
    write_config(fd, aer_base + AER_COR_STATUS, sts, 4); // W1C: write back the set bits
    uint32_t after = read_config(fd, aer_base + AER_COR_STATUS, 4);
    uint32_t stuck = after & sts; // a bit that REFUSES to clear is a finding
}
```

The blunt form `setpci -s $BDF ECAP_AER+0x04.L=0xffffffff` (write-all-ones) also clears everything — writing 1 to an already-clear or reserved bit is harmless. But **write-back-the-read-value** is the precise, auditable form: it only ever clears what was set, and **a bit that won’t clear is itself a result** — a hardware fault re-latching it every cycle (a stuck PHY, a persistent internal error). Don’t paper over it by re-clearing in a loop; record it. The toolkit returns `ClearResult(cleared, stuck)` for exactly this.

4.6.6 Who else is clearing your latches (the reconciliation trap)

The kernel `pcieport/AER` driver attaches to **root ports and RCECs** and will read-and-clear AER status out from under you to print its `dmesg` lines. If both your tool and the kernel clear the same W1C bits, each hides errors from the other and your counts are wrong and irreproducible. Three options, in order of preference:

1. **Read the kernel's own counters instead of racing it.** With CONFIG_PCIEAER_STATS the kernel exposes monotonic, per-named-error counters it maintains:

```
cat /sys/bus/pci/devices/$BDF/aer_dev_correctable # RxErR/BadTLP/... tallies
cat /sys/bus/pci/devices/$BDF/aer_dev_nonfatal
cat /sys/bus/pci/devices/$BDF/aer_dev_fatal
```

These survive between polls and aren't cleared by your `setpci` — **for a station tool this is often the robust source**: delta them across the stress window and never touch the raw W1C bits. 2. **Own the registers in a window the kernel won't report in** (mask the class you're counting, arm, stress, read raw status, restore). 3. **Disable kernel AER for an A/B test** (`pci=noaer` on the cmdline) to prove the kernel path isn't the noise source — diagnostic only, you never ship with AER off.

Pick **one owner per run** and say which one in the record. The cleanest station design reads `aer_dev_*` (kernel-owned) and never clears the raw bits.

4.6.7 The Device Status fallback when AER is absent

Not every function has an AER capability (some endpoints, many simple devices). **Every PCIe function has a Device Status register** in the PCIe Capability (+0x0A) with coarse error-detected summary bits — your universal fallback:

DevSta bit	Mask	Meaning
0	DEVSTA_CORR	Correctable Error Detected (no per-type breakdown)
1	DEVSTA_NONFATAL	Non-Fatal error detected
2	DEVSTA_FATAL	Fatal error detected
3	DEVSTA_UR	Unsupported Request detected

The toolkit codifies the preference explicitly: `aer.error_source()` returns `"aer"` (rich per-type), else `"devstatus"` (coarse summary), else `"none"`. The BERT honors it — a device with **no error source at all returns status "skip"**, never a false pass from zero errors it couldn't actually measure. That “skip not pass” rule is a manufacturing-safety property: a test must never claim a unit good on the strength of a measurement it didn't make.

4.7 Power management: ASPM, L1 substates, and their failure modes

PCIe links save power by parking in low-power states. The state machine for *active* power management (the link decides, in hardware) is **Active State Power Management (ASPM)**:

State	Description	Exit latency	Power saved
L0	Active — normal operation	0	none
L0s	Standby — one direction idle	<1 μ s	low
L1	Both directions idle, PLL may stay on	2–4 μ s (spec target; real parts advertise up to 32–64 μ s)	medium
L1.1	L1 substate — common-mode kept, clock/PLL off	order ~10–20 μ s	high
L1.2	L1 substate — common-mode <i>off</i> too (lowest power)	order ~100 μ s (CLKREQ + common-mode re-establish)	highest
L2	Link off — full retrain on wake	>100 μ s	maximum

ASPM is controlled by `LnkCtl[1:0]`; the **L1 substates (L1SS)** have their own control capability (`L1SubCtl1/2`). They are great for idle power and **a recurring source of two failure modes in test**:

- **Latency that manufactures a CTO.** If the link is in L1/L1.1/L1.2 and the time to wake it and return to L0 exceeds the CTO timer, an in-flight read **times out** — surfacing as a **Completion Timeout** uncorrectable that *looks* like a link error but is really power-management latency.
- **Transitions that look like errors / link unavailability.** Entering and exiting low-power states briefly makes the link unavailable; under a tight poll this can read as a transient or a retrain.

Manufacturing Test (MT) implication: usually disable ASPM during test. ASPM transitions add noise that can masquerade as link errors, and lane margining *requires* ASPM off. Disable it during the stress window and re-run to confirm observed errors aren't ASPM-related:

```
# runtime, per-policy:
echo performance | sudo tee /sys/module/pcie_aspm/parameters/policy
# or globally at boot: pcie_aspm=off on the kernel cmdline
```

Reset primitives you'll use alongside power management: Hot Reset (via a downstream port's Bridge Control register — resets everything below the port), and **Function Level Reset (FLR)** (resets a single function without disturbing the link — the clean way to re-init a device between test phases).

4.8 Enumeration and bring-up: when the link never reaches L0

If `lspci -nn` doesn't show the device (or shows it with no BARs), the link never reached **L0** — the LTSSM is stuck early, and you can't read AER on a link that never came up. This branch is about the *physical bring-up* facts.

Stuck in (LTSSM)	First moves
Detect	Far-end power off? PERST# (PERST#) released at the right time after rails are up? AC-coupling cap / termination missing on a lane? Dead PHY?
Polling	REFCLK present (100 MHz, SSC if expected)? SSC mismatch ? RX DC offset? Inverted polarity? Gross SI (impedance/crosstalk)?
Configuration	Dead/noisy high lanes (configures smaller width), lane-reversal unsupported, bifurcation mismatch , scrambler problem. A link that comes up x8 instead of x16 fell out here.

```
# 1. Is it there at all? Right Vendor:Device at the expected BDF?
lspci -nn | grep -i <vendorID>
dmesg | grep -iE 'link (training|up|down)|not ready|timed out|Bifurcation'

# 2. Present but dead to software -- are BARs assigned?
lspci -vvv -s $BDF | grep -iE 'Region|BAR|ignoring' # "can't assign"/"ignoring BAR" = dead

# 3. Rails + REFCLK + PERST# (where the scope comes in, sec 14):
# scope PHY supply rails at power-on (sequencing + level); confirm REFCLK at the slot;
# confirm PERST# deasserts at the right time after rails are up.

# 4. Bifurcation vs the schematic:
# does the BIOS/strap split (e.g. 2x8 or 4x4) MATCH the board's lane assignment?
```

Bifurcation is the #1 custom-card enumeration bug. Bifurcation splits one wide root port into multiple narrower links — $x16 \rightarrow 2 \times x8$, $x16 \rightarrow 4 \times x4$, $x8 \rightarrow 2 \times x4$ — configured in **BIOS/strap** and it *must match the board layout*. Zoox's compute platform almost certainly uses it to fan a single x16 root port out to multiple NVMe drives or custom devices. A mismatch shows up as “device not detected” or “trains at the wrong width.” On a custom card, verify the BIOS bifurcation setting against the schematic's lane assignment **before** you suspect a dead PHY. (Ghost devices from a previous config sometimes need a cold boot to clear.)

The custom-card bring-up order (the JD's “custom PCIe devices” — for an off-the-shelf GPU you have NVIDIA's tools; for Zoox's own card you have a schematic and a problem):

1. **Does it enumerate?** `lspci -nn` — right Vendor/Device ID at the expected BDF? If not: rails, Reference Clock (REFCLK), PCIe Reset signal (PERST#), LTSSM stuck in Detect/Polling, or a layout error.
2. **Right class/BARs?** `lspci -vvv -s $BDF` — are BARs assigned? Unassigned = dead to software even though visible.
3. **Right link?** Current vs max speed/width. $x16 \rightarrow x8$ or Gen4 $\rightarrow$ Gen3 on a new board is your first Signal Integrity (SI) finding.
4. **Does it respond?** Read a known register over MMIO; for a custom Field-Programmable Gate Array (FPGA) the design team gives you a scratch/ID register to prove the datapath.
5. **Driver / interrupts?** `lspci -k` — does the driver bind? Is `/proc/interrupts` counting during activity? (MSI/MSI-X: the device writes a special address to signal an interrupt; MSI-X scales to 2048 vectors with per-vector masking. A device that looks fine in `lspci` but never interrupts is dead from the OS's view.)

6. **Errors under stress?** Now the BERT/AER monitor, hot and cold (,).
7. **Margin?** Lane margining for the per-lane eye.

4.9 Equalization, presets, and why links fall back

At Gen3 (8 GT/s) and above the channel — PCB traces, vias, connectors, cables — attenuates the high-frequency content of the signal until the raw eye is **closed**. **Equalization** reopens it.

- **TX equalization** shapes each bit with three finite impulse response (FIR) coefficients under a fixed-swing constraint:

```
pre-cursor (C-1): boost the bit BEFORE a transition (compensates loss to come)
cursor (C0): the main bit amplitude ("drive strength")
post-cursor(C+1): boost the bit AFTER a transition (cancels Inter-Symbol Interference (ISI) from
↳ the prev bit)
constraint: |C-1| + |C0| + |C+1| = const (full-swing budget)
```

More emphasis on the cursors = stronger transitions but a weaker main bit. **Too much** → **overshoot/ringing**; **too little** → **closed eye**. - The spec defines **11 TX presets (P0–P10)** bundling coefficient ratios for different channel losses (P0 ≈ no emphasis for a short channel; higher presets add post/pre-emphasis for lossy channels). During training the hardware negotiates a preset *per lane*. - **RX equalization: Continuous-Time Linear Equalizer (CTLE)** (continuous-time linear EQ — frequency-dependent gain that boosts high frequencies) + **Decision Feedback Equalizer (DFE)** (decision-feedback EQ — cancels Inter-Symbol Interference (ISI) using past bit decisions). The receiver *adapts* these during training, which is why a marginal link can *still train* — the RX compensates until it can't. **That is exactly why lane margining matters more than pass/fail-on-link-up.**

The 4-phase EQ handshake runs in Recovery.Equalization at Gen3+:

Phase	Who tunes what	What happens
0	DSP TX -> USP RX	Link is now at the higher rate. The DSP transmits using the preset values it sent in its training sets; the USP applies those starting presets (and RxHint) to its own transmitter. Coarse setup, no coefficient requests yet.
1	both directions	Both partners exchange the FS (Full Swing) / LF (Low Frequency) bounds — the upper/lower limits on TX coefficients — and confirm they can hold the higher rate at a coarse BER.
2	USP tunes the DSP's TX	The upstream port evaluates <i>its own</i> receiver and requests TX coefficient changes from the downstream port; the downstream adjusts its transmitter until the USP's Receive is optimized.
3	DSP tunes the USP's TX	The downstream port evaluates <i>its own</i> receiver and requests TX coefficient changes from the upstream port; fine-tuning until the DSP's RX hits its eye target.

The phase ownership is a localization lever. Phase 2 optimizes the **downstream-pointing** direction (DSP transmitter → USP receiver); Phase 3 optimizes the **upstream-pointing** direction (USP transmitter → DSP receiver). A board that equalizes one direction fine but not the other (an asymmetric channel — a long TX trace on one side, a connector on one side only)

shows up as an EQ failure or downgrade you can map back to a phase, which maps to a direction, which maps to a physical leg. This is the same per-direction thinking the receiver error model uses (per-receiver, never per-link) — the channel is two independent halves and EQ tunes each half separately.

Why a link falls back to a lower speed/width — the single most common PCIe finding. EQ couldn't reach a usable eye at the higher gen, so the LTSSM negotiated down. Causes: trace length/loss, **via stubs**, impedance discontinuities, connector seating, **temperature** (a board that equalizes fine at 25 °C can fail at 55–85 °C), power-supply noise, a too-aggressive or too-weak TX preset, or a BIOS **Target Link Speed** cap (LnkCtl2[3:0]).

4.9.1 Reading and overriding the negotiated presets

The **Secondary PCIe** extended capability (ID 0x0019) holds the Gen3+ **Lane Equalization Control** — per lane, the DSP TX preset, USP TX preset, and RX preset hints. Reading it tells you which preset each lane negotiated; on a board where one lane is marginal, a different preset there is a clue.

```
lspci -vvv -s $BDF | grep -A8 'Secondary PCI Express' # per-lane EQ presets (if decoded)

# Force a gen to test EQ at a specific rate, then retrain:
setpci -s $BDF CAP_EXP+0x30.W # LnkCtl2: [3:0] = Target Link Speed
setpci -s $BDF CAP_EXP+0x30.W=0x0003 # cap target at Gen3 (3) -- example
setpci -s $BDF CAP_EXP+0x10.W=0x0020 # set Retrain Link (LnkCtl bit5) -> forces Recovery
```

Retrain warning. Writing **Retrain Link** or **Target Link Speed** *perturbs a live link* — it will blip the device. Do it on a dev station or with the DUT quiesced, **never on a unit mid-soak on a production line.**

4.9.2 The TX-preset characterization sweep (the X-ES pattern, generalized)

Before lane margining was standard, the way to find the best equalization for a board was to sweep TX presets and measure link quality at each — the grid-search a signal-integrity engineer does manually on a bench, automated. At X-ES this was a script that iterated TX pre-emphasis on a PLX/Broadcom switch, retrained, ran a quick functional check, then ran the BERT to measure quality — producing a (**preset, lane**) → **BER** matrix that told you which preset gave the best eye for that specific board revision. Once characterized, the optimal preset was locked into switch firmware for production.

The toolkit generalizes it as `margining.characterize_equalization()`: sweep P0..P10, set the preset via the Secondary PCIe cap, toggle Retrain Link, run a short BERT + margining per preset, and return an `EqSweep` whose `.best` is the preset with the **largest eye margin** (tie-broken by fewest errors). On real hardware the per-step register write is the part you validate per switch model; the *concept* is what transfers.

Why this still matters at Zook. Any time the PCB layout, connector, or cable changes — a new board revision, a different riser, a new cable assembly — you must **re-characterize the equalization**. EQ is sensitive to everything the channel does, and a board that passes at 25 °C can fail in burn-in. This is Design Verification (DV)/New Product Introduction (NPI) work that feeds the production setting.

4.10 Completion Timeout: the ASPM / L1SS / CTO story

Completion Timeout (UNCOR bit 14) is the uncorrectable bit **most often misdiagnosed as SI** when it's actually **power-management latency or a hung completer**. A non-posted request (usually a memory read) didn't get its completion before the CTO timer fired. The header log names *what* read and *who* issued it; this chapter finds *why it was slow*.

The CTO timer lives in Device Control 2. The timeout *range* is programmable in DevCtl2 (+0x28), **CTO Value field [3:0]**; DevCap2 (+0x24) advertises supported ranges and whether CTO can be disabled.

```
setpci -s $BDF CAP_EXP+0x28.W # DEVCTL2: [3:0]=CTO value, [4]=CTO disable
setpci -s $BDF CAP_EXP+0x24.L # DEVCAP2: supported CTO ranges, CTO-disable-supported
```

Default CTO ranges run ~50 μ s up to ~64 s. A too-*short* CTO can manufacture timeouts on a perfectly good but slow completer; a too-*long* one can hang the CPU on a dead one.

The ASPM/L1 connection. The classic “random hang / occasional CmplTO under no obvious stress” is **L1-exit latency**: the link went into L1 (or deeper L1.1/L1.2), and the wake time to L0 exceeded the CTO timer, so an in-flight read timed out.

4.10.1 The decisive A/B test

```
# A: reproduce the CmplTO with ASPM as-shipped (record rate + header log).
# B: disable ASPM and re-run the EXACT same stress:
# boot with pcie_aspm=off (or at runtime:)
echo performance | sudo tee /sys/module/pcie_aspm/parameters/policy
```

- **Timeouts vanish with pcie_aspm=off** → root cause is **L1-exit latency**. Fix is an ASPM/L1SS policy or exit-latency-advertisement change. **Do not** margin lanes or reseal connectors — it's not the wire.
- **Timeouts persist with ASPM off** → the completer is genuinely **hung or mis-addressed**. Back to the header log and the upstream device.

This one A/B routinely saves a day of chasing the wrong layer. The high-leverage move is to rule out ASPM/L1SS **first** — it's a 10-minute reboot — before spending an hour on margining and reseating. CmplTO *looks* like a link error in AER and `dmesg`, and the instinct is to suspect the wire; resist it.

4.11 Lane margining at the receiver — the scope-free eye

This is the modern technique that turns “sweep presets and run a BERT” into a spec-standard, push-button measurement, and it’s a genuinely strong thing to bring to Zoox.

Lane Margining at the Receiver is a **mandatory PCIe Gen4+ feature** (extended capability ID 0x0027). It commands a receiver — *while the link stays in L0* — to **shift its sampling point in time** (left/right of the data eye) and, on capable receivers, in **voltage** (up/down), **per lane**, and report when errors start. Stepping the offset outward until errors exceed a limit measures the **eye margin directly, on-die, with no oscilloscope**.

```
voltage
^ . . . . . <- step up until errors (voltage margin)
| . .
sample + . DATA EYE .
point | . .
| . . . . . <- step down until errors
+-----+-----+----> time (UI)
^ ^
step left step right (timing margin, fractions of a UI)
```

Timing margining is required at Gen4 (16 GT/s); **voltage margining is mandatory at Gen5 (32 GT/s)** and up.

The real Linux tool is pcilmr — part of pciutils (≥ 3.11 , Feb 2024; further developed through 3.13), no vendor SDK required. *This is the answer to “how do I margin a lane today.”* There is no generic kernel sysfs “margin this lane” interface; pcilmr drives the capability registers from user space and hardcodes vendor quirks (e.g. Ice Lake) a hand-rolled sequence would miss.

```
sudo pcilmr --scan # links that can be margined (negotiated >=16 GT/s)
sudo pcilmr --margin -TV 0000:03:00.0 # all lanes, timing (T) + voltage (V)
sudo pcilmr --margin -TV -r 1,2,3,6 0000:03:00.0 # near RX(1), retimer RXs(2-5), far RX(6)
sudo pcilmr -o ./csv --full # margin every ready link, CSV out for limit-setting
```

Key flags: `-e <errlimit>` (default 4), `-d <dwell-sec>` (default 1), `-l <lanes>`, `-r <recv#>` (1 = the port’s own RX ... 6 = far-end RX, **including retimers 2–5**), `-t/-T` (timing), `-v/-V` (voltage), `-g` (grade in %Unit Interval (UI) or ps), `-c` (capabilities only). **Requires root**, the link in **D0**, and **ASPM + HW-autonomous features disabled** during the test (pcilmr does the latter and warns).

4.11.1 Converting steps to UI and mV

What pcilmr does internally (useful when you parse its CSV or hand-roll a fallback):

$$\text{timing_margin_UI} = \frac{\text{passing_timing_steps}}{\text{NumTimingSteps}} \times \frac{\text{MaxTimingOffset}}{100}$$

$$\text{voltage_margin_mV} = \frac{\text{passing_voltage_steps}}{\text{NumVoltageSteps}} \times \text{MaxVoltageOffset}$$

MaxTimingOffset is a **percent of a UI** (so /100 gives UI); **MaxVoltageOffset** is in mV (or 10 mV units on some parts — verify per silicon). The conversion is anchored on the **UI**: at 16 GT/s one UI ≈ 62.5 ps ($1/16\text{e9}$ s), at 32 GT/s ≈ 31.25 ps — so a “% of UI” margin halves in picoseconds when you double the rate even though the percentage looks the same. Spec eye targets pcilmr grades against: at **16 GT/s** min timing **30% UI** (≈ 18.75 ps) / recommended 38% UI (≈ 23.75 ps), min voltage **15 mV** / recommended 21 mV; at **32 GT/s** min timing 30% UI (≈ 9.375 ps), min voltage 15 mV. That shrinking-ps-at-fixed-%UI is *why* a board that margins comfortably at Gen4 can fail at Gen5 with the identical channel — same percentage, half the absolute eye.

The toolkit ships a **mock margining backend** (`margining.margin_link`) so you can demo the sweep on a laptop — it derives a believable per-lane margin from an injected BER with seeded per-lane variation, so a marginal link shows **one weak lane** (the realistic failure shape). The default manufacturing limit is `DEFAULT_MIN_TIMING_UI = 0.25 UI`. The robust real-hardware path is to **shell out to `pcilmr` and parse its CSV** (richer alternatives: OCP `pci_lmt`, Google `pcie_lmt` for Gen4/5/6, Oxide `lmar`). Either way the output is the per-lane **timing/voltage margin** number.

Why this lands at Zoox. Walking in able to say “*I’d add receiver lane margining to module test so we get a per-lane eye margin number and set a data-driven limit, instead of pass/fail on link-up*” is a high-leverage coverage improvement, not a script. It’s the spec-blessed successor to the pre-emphasis sweep — and the **per-lane** number is what lets DV *set* an eye limit that MT then *checks* per unit.

4.11.2 Retimers and redrivers — and localizing the bad segment

Long channels (a board-to-board cable, a riser) insert repeaters:

- A **retimer** is a **protocol-aware** repeater: it recovers the data, re-equalizes, and re-transmits a clean eye, **resetting the jitter/loss budget** and **creating an independent link segment on each side** (PCIe allows **up to 2 retimers per link**). It shows up as a PCIe device with its own link/error state — *and crucially* it appears in lane margining as **additional Receiver Numbers** (`pcilmr -r 2..5`). That lets you margin the retimer’s RX and **localize a marginal eye** to “**before vs after the retimer**” — i.e. board trace vs cable. On Zoox’s cabled board-to-board links that’s a huge lever.
- A **redriver** is an **analog** booster: **invisible to software**, no link state, you cannot margin or query it. If a link has a redriver and a marginal eye, you’re back to the scope.

```
# Margin the near RX (recu#1), retimer RXs (2..5 if present), and the far RX (recu#6):
sudo pcilmr --margin -TV -r 1,2,3,6 0000:03:00.0
```

4.11.3 Decoding the AER Header Log — turning a bit into a sentence

When an uncorrectable error latches, AER captures the **first 4 DWORDs of the offending TLP** in the **Header Log** (`+0x1C`), and the **First Error Pointer** (`ERR_CAP[4:0]`, `+0x18`) says which uncorrectable bit that log belongs to. Decoding it turns “CTO” into “a 4-byte memory read of address `0x05010000` by requester `00:04.0` timed out” — the difference between a guess and a root cause.

Where to get the raw DWORDs:

```
lspci -vvv -s $BDF | grep -A1 'Header Log' # HeaderLog: 04000001 00200a03 05010000 00050100
dmesg | grep -A4 'PCIe Bus Error' | grep 'TLP Header' # the kernel prints the same 4 DWORDs
setpci -s $BDF ECAP_AER+0x1c.L ECAP_AER+0x20.L ECAP_AER+0x24.L ECAP_AER+0x28.L # raw
```

A worked decode (`DW0 = 0x04000001`):

```
DW0 = 0x04000001
byte0 = 0x04 = 0b0000_0100
[7:5] Fmt = 0b000 -> 3-DWORD header, NO data (a memory READ request)
[4:0] Type = 0b00000 -> Memory class
=> Memory Read Request, 32-bit address (3DW header)
Length [9:0] = 0x001 -> 1 DWORD requested (a 4-byte read)

DW1 = 0x00200a03
[31:16] Requester ID = 0x0020 -> bus 0x00, dev 0x04, fn 0 (i.e. 00:04.0)
[15:8] Tag = 0x0a
[7:0] Byte enables = 0x03 (first-DW BE / last-DW BE)

DW2 = 0x05010000 -> Address[31:2] (3DW header => 32-bit address)
=> target address ~ 0x05010000
```

```
DW3 = 0x00050100 -> next captured DWORD (for a 4DW/64-bit header, DW2/DW3 = Address[63:32]/[31:2])
```

Read it as a sentence: “A 4-byte memory read of 0x05010000 by requester 00:04.0 (tag 0x0a) triggered the first uncorrectable error.” Now go look: is 0x05010000 inside a BAR that exists? Is 00:04.0 the device you expect? If the bit set was **CTO**, that read never completed — so the **completer of 0x05010000** is the suspect (hung endpoint, slow L1 exit, or a switch that dropped the completion), not the requester’s link SI.

Decode shortcuts. Fmt [7:5]: 000=3DW/no-data, 001=4DW/no-data, 010=3DW/with-data, 011=4DW/with-data, 100=TLP-prefix. Type [4:0]: 00000=Memory, 00010=I/O, 00100/00101=Config Type0/1, 01010=Completion. A **Completion** TLP’s DW1/DW2 carry Completer ID, Completion Status, Byte Count, and Lower Address instead of an address — so an **Unexpected Completion** header log tells you *who completed* and *what status*, which is exactly the mis-routing / duplicate-tag clue.

The toolkit captures the header log + First Error Pointer for every uncorrectable, so a failure arrives with its requester/type/address already decoded — the single highest-value diagnostic addition over errors=N.

4.12 The BERT: bit-error rate to a confidence level

This is your tool’s core, and it’s the X-ES pattern: a **C engine counts errors and bits fast**; a **Python layer decides whether you’ve proven the link good**. The goal of a manufacturing BERT is *not* to measure the exact BER — it’s to **prove $\text{BER} < \text{target}$ (e.g. $1\text{e-}12$) at a stated confidence, in the shortest time**, and to fail fast when it can’t.

4.12.1 The math, in one paragraph (full derivation: Math chapter)

Model errors as **Poisson** with mean $\lambda = n \cdot p$, where n = bits transferred and p = the target BER. The **confidence that the true BER is below p** , given E errors observed:

$$\text{CL} = 1 - \text{PoissonCDF}(E; np) = 1 - \sum_{k=0}^E e^{-np} \frac{(np)^k}{k!}$$

Two results run the whole test:

- **Zero-error case** (the common one): set $E=0$ and solve for the bits needed, $n = -\ln(1 - \text{CL})/p$. For 95% confidence at $p = 1\text{e-}12$: $n \approx 3.0 \times 10^{12}$ bits — the **“3/BER” rule** (since $-\ln(0.05) \approx 2.996$). That’s the target your tool transfers cleanly.
- **Reporting the bound** (any error count): the exact upper bound on BER you can claim at confidence CL after n bits with E errors is the chi-squared form $\text{BER}_{\text{upper}} = \chi_{\text{inv}}^2(\text{CL}, 2E+2)/(2n)$, which reduces to the zero-error formula when $E=0$.

This is why the math is in **Python** (`ber.py` — regularized incomplete-gamma / chi-squared inverse, with a pure-Python fallback so it runs on a bare laptop) while the **C engine just counts** — large np products and factorials need care that belongs in the high-level layer, not the tight counting loop.

4.12.2 Bits → time

The engine accounts n either by **measuring** a real workload’s throughput or **theoretically** from the link: $n \approx \text{link_speed} \times \text{link_width} \times \text{efficiency} \times \text{time}$ (`backend.link_bits_per_second`). Sanity points: Gen4 x16 ≈ 31.5 GB/s $\approx 2.5 \times 10^{11}$ bit/s, so 3×10^{12} bits $\approx \sim 12$ s of clean traffic; NVMe Gen3 x4 ≈ 3.5 GB/s, so $\approx \sim 110$ s. Those are realistic module-test durations — which is the point of choosing a confidence target rather than running forever.

4.12.3 The loop the tool runs (arm → stress → read → decide)

`bert.run_bert()` implements exactly this:

1. **Idle baseline (begin)**. Quiesce the link, clear AER, read what’s set with *no* traffic. Bits set at idle are a **constant fault** (or severe marginality), not a per-error rate — recorded and excluded from the rate count so a real fault can’t run the count away.
2. **Arm**. Clear AER status with the W1C read-back primitive and confirm zero; start the measurement window clean.
3. **Stress + read**. Drive traffic (or account theoretical bits) and poll AER. Each distinct correctable **bit** set per poll counts as one error of that type (a tighter lower bound than 1-per-poll, exact at the low rates where a unit passes), then clear-and-re-arm fast so the next error can latch. Any **uncorrectable** → immediate fail.
4. **Decide (sequential)**. Feed (`errors`, `bits`, `p`, `CL`) to `ber.sequential_decision`:
 - **pass** — proved $\text{BER} \leq \text{target}$ at the confidence level (enough clean bits);
 - **reject/fail** — even the optimistic lower bound on BER exceeds the target → fail fast, no extra runtime will rescue it;
 - **continue** — undecided; extend the window if the takt budget allows (`extend_budget` x the zero-error target, hard-capped by `max_seconds`).

5. **Idle baseline (end).** Errors still present with no traffic = a real fault → fail regardless of the rate verdict.

The diagnostics ride along: which AER bits, the per-type correctable counts, the uncorrectable decode, the Gen6/FLIT note, and the idle-fault flag — so a failure comes with its root-cause evidence already attached, not just a number. (For high error rates a Python poll loop can't clear-and-recount fast enough; that's the `engine="c"` path — the C binary counts in tight increments while the same Python “conductor” makes the sequential decision and bounds the runtime.)

The DV ↔ MT bridge (why MT must capture the parameter, not just the verdict).

The *same* BERT engine serves both modes. **DV:** run it across voltage/temperature corners and TX presets and *plot the surface*. **MT:** run it once to a confidence target and emit pass/fail. The captured (**errors, bits, BER-bound**) is what later feeds Statistical Process Control (SPC) and guard-banding — *you cannot set a good limit on data you didn't keep*.

4.13 Switches, retimers, and walking the whole chain

Zoox builds custom switch/fan-out boards and cabled board-to-board links, so you rarely debug a single point-to-point link in isolation — you debug a **tree**. Walk all of it.

```
lspci -tv # the tree: root ports -> switches -> endpoints
# For EVERY link in the path (root port, switch USP, each switch DSP, endpoint),
# check speed/width AND AER -- not just the endpoint:
for bdf in 0000:00:1c.0 0000:02:00.0 0000:03:00.0 ; do
    echo "== $bdf =="
    cat /sys/bus/pci/devices/$bdf/current_link_speed /sys/bus/pci/devices/$bdf/current_link_width
    setpci -s $bdf ECAP_AER+0x10.L ECAP_AER+0x04.L # COR / UNCOR status on this link
done
```

A switch is itself a unit-under-test: bad switch SerDes (→ correctable cluster on a DSP), a switch-firmware flow-control bug (→ FCP / RX_OVER), or a switch that drops completions (→ **CTO at the endpoint above it**, even though the endpoint's own link is clean). On the **root port** (only), **Error Source ID** (+0x34) gives the requester ID of the COR/ UNCOR source — on a tree with switches that's how you attribute an error to the *true* originating device rather than the port that happened to log it.

The toolkit makes this first-class. `topology.analyze_chain()` decomposes an endpoint's path into **per-BDF error directions** (each BDF's AER reports *one direction of one link* — its receiver side) and **per-link downgrade targets** (speed/width is a per-link property owned by the downstream port, read *once*). `diagnostics.diagnose_chain()` then runs **one** stress window (the endpoint BERT — its traffic traverses every link), monitors AER per-direction on every BDF, checks each link's downgrade at its downstream port, and watches `dmesg` for PCIe/ AER events across the whole window. Any error/downgrade/uncorrectable on any segment fails the chain, **pinned to the exact BDF+direction or link**. This is the data model behind .

4.14 The diagnosis playbook

The bench artifact: given a misbehaving link, localize it to a layer and a physical cause, in a defensible order, with the exact commands.

4.14.1 The 30-second decision tree

```
0. ARM. Clear AER (W1C). Record: LnkCap both ends, current speed/width, DPC state,
   ASPM/L1SS state, severity+mask, temperature.

1. Does it ENUMERATE? (lspci -nn)
   NO -> LTSSM stuck (Detect/Polling/Config). Cause set: power/REFCLK/PERST#/
   bifurcation/dead PHY/AC-cap. Evidence: dmesg "training" stuck; rails on
   scope; BIOS bifurcation vs schematic; reseal changes it. (sec 7)

2. Right SPEED and WIDTH? (current vs max; lspci prints "downgraded")
   SPEED low (Gen4->Gen3) -> EQUALIZATION / SI margin / BIOS gen-cap / THERMAL.
   Evidence: dmesg gen-change; retest hot AND cold; pcilmr margin shrinks. (sec 8,sec 10)
   WIDTH low (x16->x8) -> connector / dead lane / bifurcation / lane-reversal.
   Evidence: per-lane pcilmr finds the dead lane; reseal; BIOS bifurcation. (sec 7)

3. ERRORS under stress? Decode WHICH AER bits. (sec 5)
   Correctable {RxErr,BadTLP,BadDLLP,ReplayTO,RollOver} -> PHYSICAL / SI. (sec 5,sec 8,sec 10)
   Evidence: count rises with temperature; pcilmr margin low on one lane;
   reseal/cable swap changes it; a better TX preset improves it.
   Uncorrectable {CmplTO,UnexpCmpl,MalformedTLP,FCP,RxOverflow} -> PROTOCOL/FW/UPSTREAM.
   Evidence: header-log decode names requester/type/address; reproducible
   regardless of temp/reseal; tied to a traffic pattern or FW/driver. (sec 5,sec 9,sec 12)
   {PoisonedTLP,ECRC} -> DATA CORRUPTION upstream / through a switch or retimer. (sec 12)
   {SurpriseDown} -> POWER / mechanical. (sec 14)

4. Errors ONLY hot or ONLY under load -> THERMAL SI or POWER droop. (sec 8,sec 14)
5. Nothing reproduces but field-flaky -> MARGIN is the discriminator: a pcilmr
   margin below limit on one lane is a latent SI fail a pass/fail link-up test misses. (sec 10)
```

The reason it works: **different root causes leave different evidence in different registers**. SI = correctable clusters that move with temperature and shrink the eye margin; protocol = uncorrectable bits whose header log names a transaction; power = Surprise Down / load-correlated bursts; firmware = reproducible, temperature-independent, version-tied uncorrectables.

Failure-signature → root-cause quick lookup. The same content as the tree, indexed the way a symptom actually arrives at the bench — by what you *observe* first:

Observed signature	Most likely root cause	The one test that confirms it
Device absent from <code>lspci -nn</code>	LTSSM stuck pre-L0 (power/REFCLK/PERST# (PERST#)/bifurcation/dead PHY)	<code>dmesg</code> “training”/“timed out”; scope rails+REFCLK+PERST#; BIOS bifurcation vs schematic
Present, Region N: <unassigned> / “can’t assign BAR”	BIOS couldn’t fit the MMIO window (often 64-bit/above-4G)	Toggle above-4G decoding / resize BAR; re-enumerate
Speed ... (downgraded), width OK	EQ / SI margin / BIOS gen-cap / thermal	Retest hot AND cold; <code>pcilmr</code> margin; check <code>LnkCtl2 Target Link</code> <code>Speed cap</code>
Width ... (downgraded), speed OK	Dead lane / connector / bifurcation / lane-reversal	Per-lane <code>pcilmr</code> finds the dead lane; reseal; BIOS bifurcation
Ends at full speed/width but <code>ABWMgmt+</code> (LABS) latched	Trained high, could NOT hold it (marginal under load/temp)	Arm LBMS/LABS, soak, re-read; count LT retrains over the window
<code>Train+</code> flickering across reads	Link bouncing through Recovery	Poll <code>LnkSta</code> over the soak; correlate with temperature/load

Observed signature	Most likely root cause	The one test that confirms it
COR cluster RxErr+BadTLP+ReplayTO	Physical / SI on the wire	Count rises with temp; one-lane <code>pcilmr</code> low; reseal/cable swap moves it; better preset helps
Bad DLLP heavy, little else	SI on the DLLP path (same SI workup)	Same as above; DLLPs aren't retried so this is a pure-SI tell
UNCOR CTO, no SI cluster	ASPM/L1-exit latency OR hung/mis-addressed completer	<code>pcie_aspm=off</code> A/B: vanishes => L1 latency; persists => header-log the completer
UNCOR Malformed, UnexpCmpl, FCP, RxOverflow	Protocol / firmware / IP bug	Reproducible regardless of temp/reseat; tied to traffic pattern or FW/driver version
UNCOR Poisoned TLP / ECRC	Data corruption upstream / through a switch or retimer	Walk the chain; check Error Source ID at the root port; margin each segment
UNCOR Surprise Down	Power lost / browned out / connector opened / hot-unplug	Scope the device rail at the load step; reseal; swap PSU
Device vanishes then re-enumerates	DPC contained a fatal OR a power glitch reset it	<code>dmesg</code> DPC containment + trigger reason vs a clean re-enumerate; rail scope at the event
Bursts correlated with load steps	Rail droop/ripple at a current transient	Scope rail AC-coupled under the same load profile; back off one load
Nothing reproduces but field-flaky	Latent marginal eye that still trains	<code>pcilmr</code> per-lane margin vs a UI limit — link-up alone passes it

4.14.2 Arm before you measure (the full checklist)

```

BDF=0000:03:00.0
# 1. Static facts BEFORE you touch anything (these don't change under test):
lspci -vvv -s $BDF | sed -n '/LnkCap:/p; /LnkSta:/p; /LnkCap2:/p; /LnkSta2:/p'
cat /sys/bus/pci/devices/$BDF/{max,current}_link_speed
cat /sys/bus/pci/devices/$BDF/{max,current}_link_width
# 2. Severity + mask FIRST (so you know which bits trip DPC/reset under you mid-test):
setpci -s $BDF ECAP_AER+0x0c.L # uncorrectable severity (1=Fatal -> ERR_FATAL/DPC)
setpci -s $BDF ECAP_AER+0x08.L ECAP_AER+0x14.L # UNCOR mask, COR mask
# 3. DPC + ASPM/L1SS state (these change what a fatal error DOES to your test):
lspci -vvv -s $BDF | grep -A2 -iE 'DPC:|ASPM|L1SubCtl1'
# 4. ARM: clear correctable + uncorrectable status (WIC):
setpci -s $BDF ECAP_AER+0x10.L=0xffffffff # clear COR_STATUS
setpci -s $BDF ECAP_AER+0x04.L=0xffffffff # clear UNCOR_STATUS
# 5. VERIFY the arm took (both should now read 0):
setpci -s $BDF ECAP_AER+0x10.L ECAP_AER+0x04.L

```

4.14.3 Decoding `lspci -vvv` for a GPU

`lspci -vvv` already decodes the PCIe-cap and AER words for you — the skill is knowing which lines carry a verdict. A realistic (lightly edited) capture of a degraded GPU link, with the load-bearing lines called out:

```

03:00.0 VGA compatible controller: NVIDIA Corporation ... (rev a1)
...
Capabilities: [68] Express (v2) Endpoint, MSI 00
DevCap: MaxPayload 256 bytes, PhantFunc 0
DevCtl: ... MaxPayload 256 bytes, MaxReadReq 512 bytes
DevSta: CorrErr+ NonFatalErr- FatalErr- UnsuppReq- <- coarse summary (DevStatus fallback)
LnkCap: Port #0, Speed 16GT/s, Width x16, ASPM L0s L1 <- what THIS end can do
LnkCtl: ASPM L1 Enabled; ... Retrain- CommClk+ <- ASPM L1 on -> CmplTO suspect
LnkSta: Speed 8GT/s (downgraded), Width x8 (downgraded) <- what it IS: TWO findings

```

```

TrErr- Train- SlotClk+ DLAActive+ BWMgmt+ ABWMgmt+ <- LBMS+ AND LABS+ latched!
LnkCap2: Supported Link Speeds: 2.5-16GT/s, ...
LnkCtl2: Target Link Speed 16GT/s, ...
LnkSta2: Current De-emphasis Level: -6dB, EqualizationComplete+ EqualizationPhase3+
Capabilities: [100 v2] Advanced Error Reporting
UESta: DLP- SDES- TLP- FCP- CmpltTO- CmpltAbrt- UnxCmplt- RxOF- MalfTLP- ECRC- UnsupReq- ...
UEMsk: ... (which UNCOR bits are masked from reporting)
UESvrt: DLP+ SDES+ ... FCP+ ... MalfTLP+ ... <- which bits are FATAL (-> DPC/reset)
CESta: RxErr+ BadTLP+ BadDLLP- Rollover- Timeout+ AdvNonFatalErr- <- COR bits set: SI cluster
CEMsk: ...
AERCap: First Error Pointer: 00, ECRGGenCap+ ECRGGenEn- ECRChkCap+ ECRChkEn-
Capabilities: [bb0 v1] Lane Margining at the Receiver <- present => pcilmr can margin this link

```

How to read it, top to bottom:

- **LnkSta: Speed 8GT/s (downgraded), Width x8 (downgraded)** — the headline. `lspci` prints (downgraded) whenever current < `LnkCap`. **Two** tags = a speed finding *and* a width finding; chase both (speed → equalization/SI/thermal; width → connector/dead-lane/ bifurcation). Compare against the *other* end's `LnkCap` — the negotiated max is the **min** of the two ends, so a GPU advertising 16GT/s behind a root port capped at 8GT/s is a config ceiling, not a defect.
- **DLActive+ BWMgmt+ ABWMgmt+** on the `LnkSta` line — `lspci`'s names for **DLLA**, **LBMS**, and **LABS**. `BWMgmt+` (LBMS) is the gem: spec-precisely it's the bit set when hardware dropped speed/width **to correct unreliable operation** — the link could not hold the higher rate. (`ABWMgmt+/LABS` is the *autonomous, non-reliability* change: power / bandwidth-on-demand.) A snapshot of “Speed 8GT/s” alone doesn't tell you *whether it ever tried 16*; `BWMgmt+` does.
- **Train-** is the LT bit at the instant of capture; if it flickers **Train+** across repeated reads the link is bouncing through Recovery (the live-retrain tell a single read misses).
- **CESta: RxErr+ BadTLP+ ... Timeout+** — the correctable cluster. `RxErr + BadTLP + Replay Timer Timeout` together is the canonical **physical-layer / SI** signature.
- **UESvrt:** — read this in the **arm** step: it tells you which uncorrectable bits are Fatal and will trip DPC / a link reset *under you* mid-stress.
- **EqualizationComplete+ EqualizationPhase3+** on `LnkSta2` — the link finished all four EQ phases. If you ever see it stuck below Phase 3 with a downgrade, EQ itself failed to converge (channel too lossy for any preset at that rate).
- **Lane Margining at the Receiver** capability present → this link is margin-able with `pcilmr` (it negotiated ≥ 16 GT/s).

The +/- suffix is set/clear throughout. `CESta/UESta` are the decoded AER status words — read which bits, then go to the AER section.

4.14.4 dmesg signatures

```
dmesg | grep -iE 'pcie|aer|link|train|dpc|bifurcation'
```

dmesg line (paraphrased)	Reading
pcieport AER: Corrected error received + BadTLP/Timeout	Correctable SI cluster — /
... AER: Uncorrected (Non-Fatal) error + TLP Header: ...	Uncorrectable; decode the header log
pci link is down / ... Link Down	Link-down event — Surprise Down or DPC; correlate timestamp
... downgraded link to 8 GT/s / ... not capable of link width x16	Speed/width fallback at training — /
pcieport DPC: containment event + trigger reason	DPC fired on a fatal error —

dmesg line (paraphrased)	Reading
... timed out during enumeration	LTSSM never reached L0 —

4.14.5 Link-down vs link-degrade — the discriminator

- **Link-down** = the link dropped out of L0 entirely. Evidence: **DLLLA** drops (if capable), **Surprise Down** (UNCOR 5) latches, **dmesg** “link is down,” the device may vanish. Causes: power/mechanical, a far-end PHY dying, DPC containment.
- **Link-degrade** = the link stayed up but at a lower speed/width, or it’s bouncing. Evidence: **LnkSta** current < max, **LABS** latched (autonomous downgrade — the “couldn’t hold it” tell,), **LT** flicking (retrains), correctable clusters rising. Causes: equalization/SI margin, thermal, connector/dead-lane.

A snapshot conflates these; the **latches separate them**. LABS set + final state Gen4 x16 = it degraded and recovered during the soak = a **fail** a one-shot test passes.

4.14.6 DPC — when the device vanishes on purpose

DPC (DPC, ext-cap ID 0x001D) is a root/switch downstream-port mechanism that, on a Fatal/Non-Fatal error, **automatically disables the link** to contain the error (stops bad TLPs propagating, stops a hung read from hanging the CPU). When DPC fires, **the device disappears**, then the kernel attempts recovery and re-enumerates.

```
lspci -vvv -s <root-or-switch-DSP-bdf> | grep -A3 'DPC:' # Enabled? Trigger reason?
dmesg | grep -iE 'DPC|containment' # WHY it fired = the evidence
# trigger reason: uncorrectable / ERR_NONFATAL / ERR_FATAL / RP-PIO / SW-trigger
# RP-PIO_* logs the offending root-port read that tripped RP-PIO
```

For MT it’s a **double-edged sword**: it cleanly *captures and isolates* a fatal event (great forensics — the trigger reason tells you *why*) **but it also yanks the device out from under your test**. Decide per station: **DPC on** = a contained fatal is a clean, attributed event, but your stress run ends when it fires; **DPC off** (**pcie_ports=native** to let the OS own it, or firmware-disabled) = the link stays up so you keep stressing past the first fatal. Know which one your root ports are set to *before* a run so “the device vanished” is read correctly (DPC containing a real fatal vs a power glitch vs a hot-unplug).

4.14.7 Confirming signal integrity and localizing the lane

Once you’ve decoded a correctable cluster or a fallback, confirm SI — the discriminators are **temperature dependence**, **eye margin**, and **reseat/cable sensitivity**:

1. **Decode which correctable bit(s)** — RxErr/BadTLP/ReplayTO cluster confirms PHY-layer.
2. **Margin the lanes** (**pcilmr -TV**) — a margin **below limit on one lane** localizes it to a connector pin, a via, a SerDes, or an AC-cap on that lane.
3. **Sweep temperature** — count correctables hot vs cold; **margin shrinking with temperature** is the SI fingerprint (the reason to test hot).
4. **Reseat / swap the cable** — if the symptom moves with the connector/cable, it’s mechanical/contact SI, not silicon.
5. **Try a better TX preset** — if a preset change improves margin/error rate, the channel was under-equalized.

4.14.8 The correctable-error storm

A flood of correctable errors during stress (the screen scrolling AER lines) is the common “link works but is marginal” failure. Triage:

```
# 1. ARM both status registers (sec 13.2). 2. Run stress 10 min (gpu-burn / fio / iperf3).
# 3. Read the kernel-maintained counters (don't race the W1C bits, sec 5.6):
cat /sys/bus/pci/devices/$BDF/aer_dev_correctable # which named errors, and how many
# 4. Classify the cluster:
# BadTLP + ReplayTO + RollOver -> SI (reseat, temperature, margining, preset) sec 8/sec 10
# Bad DLLP heavy -> SI on the DLLP path (same SI workup)
# AdvisoryNonFatal -> an uncorrectable got demoted; read UNCOR_STATUS sec 5.3
# 5. Correlate: temperature (thermal chamber?), load step (rail droop?), one lane (margining?).
```

For MT a correctable rate isn't automatically a fail — set a defensible limit (e.g. the BERT's confidence target, or < N correctables/hour). But a *cluster that grows with temperature or localizes to one lane* is a reject even if the count is “low,” because it's a latent SI defect that will worsen in the field.

4.14.9 Power, mechanical, and the load-vs-temperature split

When AER shows **Surprise Down**, or correctables come in **bursts that correlate with load steps**, suspect **power integrity or a mechanical/contact problem**, not the SerDes channel.

Symptom	Reading	Confirm with
Surprise Down under load	Far end lost power / browned out, or a connector momentarily opened	Scope the PHY/device rail at the load step; reseal; swap PSU
Correctable bursts at load steps	Rail droop/ripple at a current transient corrupts the eye transiently	Scope rail AC-coupled under the same load profile; correlate timestamps
Errors only under combined load (GPU+NVMe+link busy)	System power budget / shared-rail droop	Measure the rail with everything loaded; back off one load
Device vanishes then re-enumerates	DPC fired OR a power glitch reset it	dmesg for DPC containment vs a clean re-enumerate; rail scope at the event

The discriminator (power droop vs thermal SI): power droop is **load-correlated** (it tracks current transients and improves when you reduce load even at the same temperature); thermal SI is **temperature-correlated** (it tracks junction temperature and improves when you cool the part even at the same load). **Vary load at fixed temperature, then temperature at fixed load, to separate them.** A lot of “digital” PCIe failures are power/SI failures — the engineer who reaches for the **scope and the AER decode together** closes the intermittent ones.

4.14.10 Self-testing the AER pipeline with aer-inject

Before you trust a station's AER decode/clear/count path, validate it with **no bad hardware** by *injecting* a known error and asserting the pipeline reports exactly that (needs a kernel with CONFIG_PCIEAER_INJECT → /dev/aer-inject).

```
cat > badtlp.aer <<'EOF'
AER
PCI_ID 0000:03:00.0
COR_STATUS BAD_TLP
HEADER_LOG 0x04000001 0x00200a03 0x05010000 0x00050100
EOF
sudo aer-inject badtlp.aer
dmesg | tail # confirm the injected BadTLP appears AND decodes correctly
```

Inject one COR (RCVR, BAD_TLP, BAD_DLLP, REP_ROLL, REP_TIMER) and one UNCOR (TRAIN, DLP, POISON_TLP, FCP, COMP_TIME, COMP_ABORT, UNX_COMP, RX_OVER, MALF_TLP, End-to-End CRC (ECRC), UNSUP) and assert the five-point test: (a) the correct bit is set in status, (b) the header log matches what you injected, (c) the First Error Pointer points at the right uncorrectable bit, (d) your W1C clear

actually clears it, (e) your kernel-counter delta is exactly one. That **is** the AER self-test — the analog of a golden-unit correlation. A station whose AER decode is subtly wrong (off-by-one bit, doesn't read the header log, races the kernel) mis-triages every real failure; **aer-inject** proves the pipeline correct on a *known* input so you trust it on *unknown* hardware.

4.15 How this maps to manufacturing test

The whole point: turn each PCIe finding into a **test placed at the earliest phase that can catch it**, with a defensible pass/fail and a captured parameter. The two structural facts that shape every PCIe test:

Bit errors are per-BDF and per-direction, evaluated at the receiver. Link downgrades are per-link and seen at both ends.

That’s not pedantry — it’s the data model:

- A link’s two directions are independent differential pairs. **AER on a given BDF reports the errors its receiver saw** — one direction of one link. Four BDFs in a chain = **four independent error counts**, each pinned to a direction. So a bit-error problem is attributed to a **BDF + direction** (the toolkit’s `ChainSegmentResult`), never lumped per link.
- Speed/width (and the LBMS/LABS latches) are a **negotiated link property** — **both ends report the same value**. So a downgrade is read **once, at the downstream port that owns the link** (the toolkit’s `ChainLink` / `ChainLinkResult`), never double-counted across its two BDFs.

Getting this right is what lets a chain diagnostic say “*errors on 02:00.0’s receiver (downstream direction), and the 00:1c.0<->02:00.0 link downgraded to Gen3*” — two distinct, correctly-scoped findings — instead of a vague “the path has errors.”

Worked, on a 3-hop chain (root port → switch → GPU). The endpoint BERT’s traffic traverses every link, so one stress window exercises all of them; you read each link’s own AER and each link’s negotiated speed/width:

```
Root Port Switch GPU (EP)
00:1c.0 ===linkA=== 02:00.0(USP) .
02:01.0(DSP) ===linkB=== 03:00.0
-----
AER receivers (per BDF = one direction of one link):
00:1c.0 AER -> errors the ROOT PORT's RX saw (upstream-pointing traffic on linkA)
02:00.0 AER -> errors the switch USP's RX saw (downstream-pointing traffic on linkA)
02:01.0 AER -> errors the switch DSP's RX saw (upstream-pointing traffic on linkB)
03:00.0 AER -> errors the GPU's RX saw (downstream-pointing traffic on linkB)
=> 4 BDFs = 4 independent, direction-pinned error counts (never summed into "the chain").

Link speed/width (per LINK, read ONCE at the downstream port that owns it):
linkA: owned by 00:1c.0 (root port is a DSP) -> read CLS/NLW + LBMS/LABS here
linkB: owned by 02:01.0 (switch DSP) -> read CLS/NLW + LBMS/LABS here
=> 2 links = 2 downgrade verdicts; 02:00.0 and 03:00.0 (the USPs) report the SAME value,
so you do NOT double-count.
```

So “GPU has errors” actually resolves to, e.g., “*03:00.0 RX correctables (the GPU receiving on linkB, i.e. switch-DSP-TX → GPU-RX) plus linkB autonomously downgraded (LABS at 02:01.0)*” — which points the SI workup at **linkB’s downstream-pointing leg** (switch DSP transmitter, that cable/trace, GPU receiver), not at linkA or the GPU’s transmitter. That is the difference between a reseal-everything afternoon and a one-leg fix. This is the data model behind the toolkit’s `ChainSegmentResult` (per BDF + direction) and `ChainLinkResult` (per link).

4.15.1 Placing PCIe coverage across the four phases

Failure mode	Catchable at	Test
Wrong/missing PCIe device (BOM/stuffing)	PCBA (enumeration) + Module	<code>lspci</code> enumeration vs expected topology config
BGA solder void under a GPU	PCBA (X-ray) + Module (thermal cycling surfaces it)	X-ray; thermal soak + AER monitor

Failure mode	Catchable at	Test
Lane marginal at temperature	Module (<i>not</i> PCBA — room temp passes it)	Stress + AER + lane margining hot/cold
Speed/width fallback (EQ/SI)	Module	Enumerate at expected Gen/width; LABS latch over soak; margining
Marginal eye that still trains	Module	Lane margining vs a UI limit (link-up alone passes it)
Inter-module / cabled-link marginal	System (real cable/connector)	Chain BERT + per-segment margining (retimer localizes board-vs-cable)
Custom card not detected (bifurcation)	PCBA/Module	Enumeration + bifurcation-vs-schematic check

The shift-left rule: a marginal Gen4 lane that only fails at 85 °C **cannot** be caught at room-temperature Printed Circuit Board Assembly (PCBA) — it needs the **module-level thermal stress**, which is where your BERT/AER/margining tools live. A solder short, by contrast, should die at PCBA (ICT), never surface as a PCIe link failure at system test.

4.15.2 Takt-bounded BER confidence with margin

On the line every test is **takt-bound** — seconds to minutes per unit, not hours. The BERT’s confidence target is what makes that tractable: instead of “run forever and hope,” you **transfer exactly the bits needed to prove $BER \leq \text{target}$ at the confidence level, then stop**. Gen4 x16 to 95% at 1e-12 is ≈ 12 s of clean traffic; the engine **fails fast** (rejects the moment the error rate disproves the target) so a bad unit doesn’t burn the full budget, and it **bounds the extend** by a takt budget so an undecided unit can’t run away.

The expert addition is **margin**, two ways:

1. **Run with headroom on the confidence/bits** (`bert.run_bert(margin=1.2)` transfers the target-plus-margin so a borderline-but-good unit still clears, and a clearly-good one passes quickly).
2. **Gate on the *margin number*, not just pass/fail** — the lane-margining UI/mV value, and the BER *upper bound* (not just “no errors”). This is the DV $\leftrightarrow$ MT bridge made concrete: **DV uses the per-lane margin to set a data-driven eye limit across temperature; MT checks that limit per unit**. Pass/fail-on-link-up is replaced by a margin with guard-band — which is exactly the coverage improvement to propose on day one.

4.15.3 The manufacturing PCIe checklist (shape of the module-test gate)

```
#!/bin/bash
set -euo pipefail # -e: stop on any error; -u: unset var is an error; -o pipefail:
# a pipeline fails if ANY stage fails. The single most important
# line in a hardware test script.

# 1. Enumerate everything against the expected topology (count, BDF, vendor/class).
expected_gpus=4
actual_gpus=$(lspci -d 10de: | wc -l)
[[ "$actual_gpus" -eq "$expected_gpus" ]] || die "Expected $expected_gpus GPUs, found $actual_gpus"

# 2. Verify link speed AND width for each device (sysfs, no root needed).
for bdf in $(lspci -d 10de: -D | awk '{print $1}'); do
    speed=$(cat /sys/bus/pci/devices/$bdf/current_link_speed)
    width=$(cat /sys/bus/pci/devices/$bdf/current_link_width)
    [[ "$speed" == "16.0 GT/s" ]] || die "GPU $bdf: speed $speed (expected 16.0 GT/s)"
    [[ "$width" == "16" ]] || die "GPU $bdf: width x$width (expected x16)"
done
```

```
# 3. Arm LBMS/LABS latches + AER, soak under load, re-read: any LABS latch or
# uncorrectable bit = FAIL (catches "trains fine, marginal under load", sec 4.3/sec 13.5).

# 4. Confidence BERT per link to the target (prove BER <= 1e-12 @ 95%, fail fast), sec 11.

# 5. Lane margining per link; gate on >= the per-lane UI limit (sec 10), capture the number.

# 6. Repeat 3-5 hot (thermal chamber) -- the marginal-at-temperature defects PCBA can't see.
```

Every gate above **captures its parameter** (the speed/width, the AER bit-set, the BER bound, the per-lane margin, the temperature) into the test record — because the captured stream is what later sets limits, feeds SPC/Cpk, and flags a bad lot. *Capture the parameter, not just the verdict* is the rule the whole toolkit is built on.

4.16 Command quick-reference

BDF=0000:03:00.0 throughout.

Enumerate & topology

```
lspci -nn # vendor:device IDs at each BDF
lspci -tv # tree: switches, retimers, what's behind what
lspci -nnk -s $BDF # driver bound (-k) + IDs for one device
lspci -vvv -s $BDF # FULL: LnkCap/LnkSta, AER status+header log, DPC, margining
```

Speed/width (sysfs, no root)

```
cat /sys/bus/pci/devices/$BDF/{current,max}_link_speed
cat /sys/bus/pci/devices/$BDF/{current,max}_link_width
# Whole-machine degraded-link scan:
for d in /sys/bus/pci/devices/*; do
  cur=$(cat $d/current_link_speed 2>/dev/null); max=$(cat $d/max_link_speed 2>/dev/null)
  [ -n "$cur" ] && [ "$cur" != "$max" ] && echo "$(basename $d): $cur (max $max)"
done
```

AER via setpci (ECAP_AER alias)

```
setpci -s $BDF ECAP_AER+0x04.L # UNCOR status (read)
setpci -s $BDF ECAP_AER+0x10.L # COR status (read)
setpci -s $BDF ECAP_AER+0x0c.L # UNCOR severity
setpci -s $BDF ECAP_AER+0x08.L ECAP_AER+0x14.L # UNCOR mask, COR mask
setpci -s $BDF ECAP_AER+0x18.L # ERR_CAP (First Error Pointer [4:0])
setpci -s $BDF ECAP_AER+0x1c.L ECAP_AER+0x20.L ECAP_AER+0x24.L ECAP_AER+0x28.L # header log
setpci -s $BDF ECAP_AER+0x10.L=0xffffffff # clear COR (W1C / arm)
setpci -s $BDF ECAP_AER+0x04.L=0xffffffff # clear UNCOR (W1C / arm)
```

Link registers via setpci (CAP_EXP alias = PCIe cap base)

```
setpci -s $BDF CAP_EXP+0x0c.L # LnkCap (max speed/width, DLLLARC bit20)
setpci -s $BDF CAP_EXP+0x12.W # LnkSta (CLS/NLW/LT/DLLLA/LBMS/LABS)
setpci -s $BDF CAP_EXP+0x12.W=0xc000 # arm LBMS/LABS latches (W1C)
setpci -s $BDF CAP_EXP+0x32.W # LnkSta2 (Flit Mode Status [10])
setpci -s $BDF CAP_EXP+0x24.L CAP_EXP+0x28.W # DEVCAP2, DEVCTL2 (CTO value [3:0])
setpci -s $BDF CAP_EXP+0x10.W=0x0020 # Retrain Link (LnkCtl bit5) [perturbs link!]
```

Kernel-side

```
dmesg | grep -iE 'pcie|aer|link|train|dpc|bifurcation'
cat /sys/bus/pci/devices/$BDF/aer_dev_correctable # kernel-maintained counters
cat /sys/bus/pci/devices/$BDF/aer_dev_nonfatal /sys/bus/pci/devices/$BDF/aer_dev_fatal
# cmdline knobs: pcie_ports=native (OS owns AER/DPC), pci=noaer (A/B), pcie_aspm=off
```

Margining & self-test

```
sudo pcilmr --scan # links that can be margined (>=16 GT/s)
sudo pcilmr --margin -TV $BDF # all lanes, timing+voltage
sudo pcilmr --margin -TV -r 1,2,3,6 $BDF # near RX, retimer RXs, far RX
sudo pcilmr -o ./csv --full # every ready link, CSV out
sudo aer-inject badtlp.aer # inject a known error (needs CONFIG_PCIEAER_INJECT)
```

4.17 Accuracy notes & caveats

A few things are version- or silicon-dependent — confirm against *your* hardware before driving it (a wrong offset on a live link turns a diagnosis into an outage):

- **Lane-margining control-register bit offsets.** The *command model* (Receiver Number, Margin Type, Margin Payload; query → set-limit → step → dwell → read) is well-confirmed; the exact *bit positions* are the conventional layout — confirm against the PCIe Base Spec for your silicon. In practice **drive pcilmr and parse its CSV** rather than hand-coding the sequence; it hardcodes per-vendor quirks.
- **MaxVoltageOffset units** — mV vs 10 mV on some parts. Verify before converting steps to mV.
- **Gen6 FLIT layout / FEC error model** — the operational point (FEC-corrected symbols + CRC + replay replace AER-LCRC-retry accounting; DLLPs are gone) is what changes your measurement; the precise FLIT byte-split is spec-final.
- **TX preset dB rounding / P10 definition** — the coefficient *ratios* are authoritative; some references round the dB columns differently.
- **setpci offsets you write** — always validate against `lspci -vvv` for the specific device before a write.

Primary sources (consolidated): Linux kernel `include/uapi/linux/pci_regs.h` (AER, LnkCap/Ctl/Sta, DPC, DEVCTL2 defines) and docs (PCIe AER HOWTO, `sysfs-pci`); `pciutils pcilmr(8)` man page + ChangeLog; lane-margining tools (OCP `pci_lmt`, `google/pcie_lmt`, `oxidecomputer/lmar`); `aer-inject SPEC` + kernel `aer_inject.c`; PCI-SIG/vendor material on equalization, the preset table, and Gen6 PAM4/FLIT/FEC. The BER confidence math is derived in the Math & Statistics chapter.

4.18 Toolkit cross-reference: how computetest realizes the chapter

The `computetest` toolkit (`/toolkit/`) implements the chapter’s diagnostic flow as a first-class library. The pieces and design decisions worth knowing:

- **Generation support is Gen1–Gen6, anchored on Gen5.** `LINK_SPEED_GTPS` covers all six; `_encoding_efficiency` returns 0.8 for Gen1/2 (8b/10b), 128/130 for Gen3–5 (NRZ), and the nominal 242/256 for Gen6 (PAM4 + FLIT). `ber.GEN5_X16_BPS = 504_123_076_923 bps` is the Zoox compute target; `ber.time_estimate()` renders the BERT cost across all four reference rates (Gen3/4/5/6 x16) in the CLI’s `ber` subcommand.
- **The W1C counting model is in C, the decision logic in Python.** `c/pcie_bert.c` is the dumb-fast counter (the “hot loop” calls for) — one config read per iteration, write-1-to-clear when set, count each set bit’s type. Python’s `bert.run_conductor` owns the sequential decision (pass/reject/extend), the idle baseline subtraction (a bit set at idle is a *constant fault*, not a rate error), and the takt-budget cap. The dependency-injection seam (`cfg_io` in `c/pcie_bert_core.h`) lets Unity tests model true W1C semantics off-hardware (see CI/CD chapter).
- **Poll-rate calibration is reported, not assumed.** The C engine emits `poll_rate_hz` in its JSON; the conductor surfaces a calibration note when the achieved rate falls below `10 x target_ber x bps` — the saturation point above which the W1C bit-counting model collapses N same-type errors per window into a single bit-set. On a healthy station Gen5 at 1e-12 needs ~5 polls/sec, trivial; on a loaded host where polls drop to ~1 kHz, the calibration note tells you the count is a lower bound, not a calibrated rate (the verdict — *fail* — is still correct).
- **Security hardening: every BDF is validated.** `RealBackend._check_bdf` rejects anything that doesn’t match the canonical `DDDD:BB:DD.F` shape before interpolating into a sysfs path. Without this, an attacker-controlled `bdf` from a plan file (`bdf: ".../etc/passwd"`) would escape `sys_root`, and with `root + write_config` it was an arbitrary-write primitive. Regression-tested by 10 invalid-shape cases.
- **Verdicts are honest.** `diagnose --no-bert` returns `skip` (`EXIT_UNAVAIL = 5`), not `pass`, because no BER measurement was made — you never claim `PASS` from a measurement you didn’t make. Likewise, a sysfs enumeration failure (`bps == 0`) triggers `skip` immediately rather than wasting `max_seconds` accumulating zero bits.
- **AER snapshot falls back to Device Status.** `aer.snapshot()` on a no-AER device reads Device Status (the universal coarse error source), so a `--no-bert` quick check on a legacy endpoint still catches a NonFatal/Fatal uncorrectable bit. The original snapshot ignored Device Status and would false-PASS that exact case.
- **Lane Margining: Gen4+ capability, Gen5 mandatory on downstream ports.** `margin.py` walks the spec-standard `step→dwell→read` sequence behind a guarded `_real_margin_lane` that requires per-hardware validation; the mock backend produces believable per-lane numbers driven by `injected_ber`. The Gen5 32 GT/s eye is too tight to rely on “the link came up” alone, which is why the spec moved margining from “optional” (Gen4) to “required on downstream ports” (Gen5).
- **Phase 2 binding proof.** `sim/qemu/inject.py` drives QEMU’s QMP socket (`pcie_aer_inject_error`) to inject AER errors into an emulated nvme behind a `pcie-root-port`; the unmodified `pcie_bert` running in the guest reads the resulting W1C-latched bits via the real Linux kernel sysfs and reports the count. That binding (the engine’s read/decode/clear path against real kernel-generated AER config space) is the layer a software mock structurally cannot prove — documented in `toolkit/sim/qemu/README.md`.

Chapter 5

NVMe and Storage

An Non-Volatile Memory Express (NVMe) SSD is two things at once, and you test it as both. It is a **PCIe endpoint** — so everything in the PCIe chapter applies *first*: a NVMe drive that throws Bad-Transaction Layer Packet (TLP) correctable errors, trains x4→x2, or drops to Gen3 is a PCIe problem wearing a storage costume, and you debug it with `lspci`, Advanced Error Reporting (AER), and lane margining, not `nvme-cli`. And it is a **storage controller** with its own command set, health telemetry, self-test engine, and failure modes — which is what this chapter covers. On Zoox’s compute, NVMe holds the OS, the maps and AI models, and the sensor-logging firehose (perception logging alone is enormous and sustained), so “does it hit rated bandwidth and *hold* it under a continuous write soak at temperature” is a real safety-relevant question, not a benchmark.

Rule of thumb for the floor. When a drive misbehaves, ask “PCIe or storage?” before you touch `nvme-cli`. Check the link first: `nvme list`, then the drive’s PCIe Bus/Device/Function (BDF) in `sysfs` (`current_link_speed/current_link_width`) and its AER counters. A throttling-or-errors story on the *link* is a PCIe-chapter problem; a SMART/media/self-test story is a storage problem. Half the “NVMe failures” you will chase are actually link failures.

5.1 Why NVMe (and why the queue model matters to test)

NVMe replaced AHCI as the host-controller interface for flash. AHCI was designed for spinning disks: one command queue, 32 entries. NVMe allows up to **65,535 I/O queues of 65,536 commands each**, which is what lets it exploit flash’s internal parallel channels. That parallelism is also *why* the manufacturing performance test looks the way it does: sequential bandwidth is a single-queue, large-block test, but **IOPS is a queue-depth and parallelism test** — you only see a drive’s true random-read IOPS at high `iodepth` across multiple jobs, because that is what fills all those queues. A drive that hits sequential BW but misses IOPS often has a controller or parallelism problem, not a media problem.

5.2 The architecture, in one paragraph you can act on

The host and controller communicate through **queues that live in host memory**:

- **Submission Queue (SQ):** the host writes a command here, then writes the SQ’s **doorbell register** (in the controller’s Base Address Register (BAR)-mapped Memory-Mapped I/O (MMIO) space) to tell the controller “go look.” The doorbell is the one piece of the model that is a real MMIO register on the device; the queues themselves are host RAM.
- **Completion Queue (CQ):** the controller writes a completion entry here and raises an **Message Signaled Interrupt Extended (MSI-X) interrupt**. The host processes completions and writes

the CQ doorbell to free slots. A phase-bit in each entry tells the host which entries are new without re-reading the doorbell.

- **Admin queue:** exactly one pair (SQ0/CQ0), created at init. It carries *management* commands — Identify, Get Log Page, Get/Set Feature, Format, Firmware Download/Commit, Device Self-Test (DST), Create/Delete I/O Queue. This is the queue your test program lives on.
- **I/O queues:** created via admin commands, typically **one SQ/CQ pair per CPU core** for lock-free parallelism. They carry Read/Write/Flush/Compare. This is where **fio** traffic goes.

```
HOST MEMORY CONTROLLER (the SSD)
+-----+ doorbell write +-----+
| Admin SQ | -----> | fetches cmd via DMA |
| I/O SQ x N | | executes on NAND |
+-----+ | DMA's data to/from |
| Admin CQ | <-- MSI-X IRQ ---- | writes completion |
| I/O CQ x N | +-----+
+-----+
~ PCIe (Gen3 x4 ~3.5 GB/s, Gen4 x4 ~7 GB/s, Gen5 x4 ~14 GB/s)
```

Controller vs namespace (the distinction that bites people in test):

- A **controller** (`/dev/nvme0`) is the NVMe hardware — one chip per SSD, usually. It handles command processing, wear leveling, Error-Correcting Code (ECC) on the NAND, and the PCIe interface.
- A **namespace** (`/dev/nvme0n1`) is a logical collection of LBA blocks under that controller — like a partition, but at the controller level, with its own block-address range and LBA format (block size + metadata). One controller can host **multiple** namespaces, each appearing as a separate block device (`/dev/nvme0n1`, `/dev/nvme0n2`).
- **The test trap:** `nvme list` enumerates **namespaces**, not controllers. A drive whose controller is perfectly alive but has **no namespace configured** shows up in `lspci` and in `/dev/nvme0` but **not** as `/dev/nvme0n1` and **not** in `nvme list`. Some drives ship un-provisioned exactly this way. Your provisioning step may have to *create* the namespace before any data test can run (see Manufacturing Flows below). “Drive is dead” is the wrong conclusion; “drive has no namespace yet” is often the right one.

M.2, U.2/U.3, and EDSFF (E1.S/E1.L/E3) are **physical form factors, not protocols** — they all speak NVMe over PCIe. The form factor matters for *cooling* (and therefore throttling), for hot-swap (U.2/U.3 and EDSFF are hot-swappable — your test may need to verify hot-insert), and for the connector you are qualifying. M.2’s poorer thermal path makes thermal throttling a far more common finding there; Zook’s server-grade assemblies more likely use U.2/U.3 or EDSFF with a real heatsink and airflow.

5.3 Admin vs I/O command sets, and Identify

NVMe splits commands into two sets, matching the two queue types:

Set	Where it runs	Representative commands
Admin	Admin queue (SQ0/CQ0)	Identify, Get Log Page, Get/Set Feature, Format NVM, Firmware Download/Commit, DST, Create/Delete I/O SQ/CQ, Sanitize
NVM I/O	I/O queues	Read, Write, Flush, Compare, Write Zeroes, Dataset Management (TRIM), Write Uncorrectable

Two **Identify** commands anchor every qualification because they are how you confirm you are testing the part you think you are, and what it is *capable* of:

```

nvme id-ctrl /dev/nvme0 -o json # Identify Controller (CNS 0x01)
# Key fields:
# mn model number sn serial number fr firmware revision
# vid PCI vendor ID oacs optional admin cmds supported (bit 4 = DST supported)
# wctemp / cctemp warning / critical composite temperature thresholds (Kelvin)
# tnumcap total NVM capacity (bytes)
# sanicap sanitize capabilities (which erase types are supported)

nvme id-ns /dev/nvme0n1 -o json # Identify Namespace (CNS 0x00)
# Key fields:
# nsze / ncap / nuse namespace size / capacity / utilization (in LBAs)
# flbas formatted LBA size index -> which lbas is active
# lbaf[] the LBA-format table: ds = log2(block size), ms = metadata bytes
# nsfeat, dpc, dps thin-provisioning / protection-information capabilities

```

`id-ctrl` gives you model/serial/FW (traceability — every test record must capture these), the **temperature thresholds** the drive throttles against (`wctemp/cctemp`, in Kelvin), and the **OACS** bitmap that tells you whether DST is even supported (bit 4). `id-ns` gives you the namespace size and the **active LBA format** — which is how you verify a drive was provisioned to 512 B vs 4 KB blocks (a provisioning mistake that silently changes performance and capacity).

5.4 SMART / Health — your primary pass/fail gate (log page 0x02)

`nvme smart-log /dev/nvme0 -o json` is the single most important command in NVMe test. It reads **Log Identifier 0x02**, the SMART / Health Information log. Unlike the rest of this chapter, the byte offsets here are worth knowing cold, because in a pinch you can decode the log from a raw `get-log` dump when the JSON parser hiccups on a vendor field. Offsets are into the 512-byte log page:

Offset	Bytes	Field	New-drive expectation	Why it matters
0x00	1	Critical Warning (bitmap)	0	Any bit set = drive is telling you it is failing
0x01	2	Composite Temperature (Kelvin)	within spec (often 0-70 C)	Out of range during test = thermal/airflow problem
0x03	1	Available Spare (%)	100	Spare blocks remaining; declines as flash wears
0x04	1	Available Spare Threshold (%)	below current spare	If spare drops below this, Critical-Warning bit 0 trips
0x05	1	Percentage Used (%; can exceed 100)	0	Wear indicator; non-zero on a “new” drive = used stock
0x20	16	Data Units Read (x1000 x 512 B)	small/reasonable	Lifetime reads; large on “new” = drive has history
0x30	16	Data Units Written	small/reasonable	Lifetime writes; the endurance-burn signal

Offset	Bytes	Field	New-drive expectation	Why it matters
0x40	16	Host Read Commands	small	Lifetime read-command count (the count behind DUR)
0x50	16	Host Write Commands	small	Lifetime write-command count (the count behind DUW)
0x60	16	Controller Busy Time (min)	small	Minutes the controller had I/O outstanding
0x70	16	Power Cycles	low (single digits)	Re-stock signal
0x80	16	Power On Hours	single-digit hours	The re-stock detector (see below)
0x90	16	Unsafe Shutdowns	typically 0	Context for field returns / power-path issues
0xA0	16	Media and Data Integrity Errors	0	Uncorrectable media errors -> bad NAND. ANY = fail
0xB0	16	Number of Error Information Log Entries	0 (or known-benign)	Total entries in the 0x01 error log
0xC0	4	Warning Composite Temp Time (min)	0	Minutes spent at/above WCTEMP (but below CCTEMP); nonzero = it throttled
0xC4	4	Critical Composite Temp Time (min)	0	Minutes above CCTEMP; a serious thermal finding
0xC8	16	Temperature Sensor 1..8 (Kelvin, 2 B ea)	within spec	Per-sensor temps; a 0 entry = sensor not implemented
0xD8	4	Thermal Mgmt Temp 1 Transition Count	0	Nonzero = drive hit the light-throttle setpoint (TMT1)
0xDC	4	Thermal Mgmt Temp 2 Transition Count	0	Nonzero = drive hit the heavy-throttle setpoint (TMT2)
0xE0	4	Total Time For Thermal Mgmt Temp 1 (sec)	0	Seconds spent in TMT1 throttle
0xE4	4	Total Time For Thermal Mgmt Temp 2 (sec)	0	Seconds spent in TMT2 throttle

These offsets are from the SMART / Health Information Log layout in the NVMe Base Specification (mirrored field-for-field by libnvme's `struct nvme_smart_log` and Microsoft's `NVME_HEALTH_INFO_LOG`). The 16-byte lifetime counters (offsets 0x20 through 0xBF) are little-endian 128-bit values; `nvme-cli` parses them for you. Note the field order on the wire is **Power Cycles (0x70)** then **Power On Hours (0x80)** then **Unsafe Shutdowns (0x90)** then **Media Errors (0xA0)** — a common mistake is to assume Media Errors sits low in the page; it does not. When in doubt, key off the JSON field *name* (`media_errors`, `power_on_hours`, `thm_temp1_trans_count`), not a hand-counted offset.

Critical Warning is a bitmap; **any** set bit fails a new drive. Decode it:

Bit	Meaning
0	Available spare has fallen below the threshold
1	Composite temperature exceeded a threshold (WCTEMP/CCTEMP)
2	NVM subsystem reliability degraded (excessive media errors / internal)
3	Media placed in read-only mode (the drive gave up on writes)
4	Volatile-memory backup device failed (the capacitor that flushes the write cache)
5	Persistent-memory region became read-only / unreliable (if present)

The new-drive manufacturing limits, stated as a gate, and exactly what the toolkit’s `nvme.py` enforces in `_apply_limits()`:

Check	Limit	Rationale
<code>critical_warning == 0</code>	hard fail if any bit	the drive’s own self-assessment
<code>media_errors == 0</code>	hard fail	a brand-new drive has touched no bad NAND
<code>num_err_log_entries == 0</code>	fail / investigate	clean error history expected
<code>percentage_used < 2</code>	fail above	wear; ~0 on new
<code>available_spare >= 100</code>	fail below	full spare pool on new
<code>0 < temperature <= 70 C</code>	fail outside	in spec, and <i>nonzero</i> (a 0 reads as a sensor fault)
<code>power_on_hours <= 50</code>	fail above	re-stock / used-stock detector

The used-vs-new signal — a quality finding, not a drive fault. `percentage_used`, `data_units_written`, `power_on_hours`, and `power_cycles` that are non-zero on a drive that is supposed to be new is how you catch **re-labeled or returned (“re-stock”) drives entering your line**. A “new” drive with 6,200 power-on-hours and 800 power cycles is used inventory — a supply-chain/quality problem, not a defective part. The toolkit treats these correctly: it **gates** on `power_on_hours` (the cleanest single signal) but reports `power_cycles`, `unsafe_shutdowns`, and large lifetime writes as **history flags** (`_history()`), so the fleet data can flag a bad lot without false-failing every drive with a benign power cycle. Log these *even when they pass* — genealogy is how Quality finds a contaminated lot before it spreads.

The already-throttling signal. Any nonzero `thm_temp1/2_trans_count`, `warning_temp_time`, or `critical_comp_time` on a new drive means the drive **throttled during your own test** — that is a cooling/airflow/heatsink-mount problem on *your* fixture or the module, not (necessarily) a drive defect. These are the difference between “the drive is bad” and “your thermal solution is bad,” and only the module-level soak surfaces them. They are the high-value adds beyond a bare `critical_warning==0` check.

The default (non-JSON) `nvme smart-log` print is what you eyeball at the bench; it names every field, so it doubles as a decoder ring for the offsets above:

```
Smart Log for NVME device:nvme0 namespace-id:ffffff
critical_warning : 0
temperature : 41 C (314 Kelvin)
available_spare : 100%
available_spare_threshold : 10%
percentage_used : 0%
```



```

data_units_read : 5,678 (2.90 GB)
data_units_written : 1,234 (631 MB)
host_read_commands : 88,142
host_write_commands : 41,003
controller_busy_time : 0
power_cycles : 3
power_on_hours : 1
unsafe_shutdowns : 0
media_errors : 0
num_err_log_entries : 0
Warning Temperature Time : 0
Critical Composite Temperature Time : 0
Temperature Sensor 1 : 41 C (314 Kelvin)
Temperature Sensor 2 : 44 C (317 Kelvin)
Thermal Management T1 Trans Count : 0
Thermal Management T2 Trans Count : 0
Thermal Management T1 Total Time : 0
Thermal Management T2 Total Time : 0

```

Newer `nvme-cli` prints the human-readable temperature for you (the 41 C (314 Kelvin) form); older versions print only the Kelvin integer, which is the version skew the toolkit’s “subtract 273 if it looks like Kelvin” guard exists to absorb. When you script, take the JSON, not this text — the labels and number formatting shift across `nvme-cli` releases.

5.5 The other logs that a credible qualification reads (Get Log Page)

`smart-log` is one log page. A real NVMe qualification reads several, via `nvme get-log` or the dedicated subcommands. The Get-Log-Page surface:

LID	Log	What it gives you	Subcommand
0x01	Error Information	Ring of recent command errors: status code, command ID, LBA , namespace ID, error count — far more detail than SMART’s <code>num_err_log_entries</code> summary	<code>nvme error-log</code>
0x02	SMART / Health	The core health page above	<code>nvme smart-log</code>
0x03	Firmware Slot Info	Active slot + next slot + per-slot revision strings	<code>nvme fw-log</code>
0x06	DST	Result of the last self-tests (up to 20 entries) + current-operation %	<code>nvme self-test-log</code>
0x07	Telemetry Host-Initiated	Vendor binary blob for FA/RMA, triggered by the host	<code>nvme telemetry-log</code>
0x08	Telemetry Controller-Initiated	Vendor blob the controller captured on its own (e.g., at an internal fault)	<code>nvme telemetry-log</code>

LID	Log	What it gives you	Subcommand
0x0D	Persistent Event Log	Non-volatile, cross-power-cycle history: power cycles, thermal excursions, firmware changes, error bursts — the richest field-return artifact	<code>nvme persistent-event-log</code>

The Error Information Log (0x01) is the detail behind the SMART summary. Where SMART says “7 error log entries,” the error log says *what* those 7 errors were — status code, the LBA involved, the command that triggered it. On a failing drive that is the difference between “errors exist” and “writes to LBA 0x4A1B00 are returning a media error,” which is an actionable defect description. A real **nvme error-log** entry on a drive with a media fault:

```
Error Log Entries for device:nvme0 entries:1
.....
Entry[ 0]
.....
error_count : 41
sqid : 3
cmdid : 0x1a
status_field : 0x2002(INVALID_FIELD: A reserved coded value or an unsupported value in a defined
↳ field)
phase_tag : 0
parm_err_loc : 0xffff
lba : 0x4a1b00
nsid : 0x1
vs : 0
trtype : The transport type is not indicated or the error is not transport related.
cs : 0
.....
```

The fields that matter for triage: **status_field** (the NVMe status code — bit 0 is the phase tag; the status sits in bits 1:15 — Status Code in bits 1:8, Status-Code-Type in bits 9:11. `nvme-cli` prints the raw 16-bit field, so shift right 1 to drop the phase tag before decoding. A media error shows up as SCT 0x2 with codes like 0x81 unrecovered-read-error / 0x80 write-fault), **lba** (the block that faulted — feed it back to `fiio/dd` to confirm it is reproducible), and **error_count** (which is the *running* error counter, not the entry index). Decode the status code against the NVMe spec status tables, or let `nvme-cli` print the parenthetical for you.

Telemetry (0x07/0x08) is a vendor-defined binary dump — you do not parse it on the line; you **capture it on a failure** and hand it to the SSD vendor’s FA team. Capturing it costs you nothing and is the single thing the vendor will ask for on an Return Merchandise Authorization (RMA).

The non-volatile logs are the RMA story. SMART resets some context across power cycles and a `format/sanitize` can clear logs entirely. The **Persistent Event Log (0x0D)** and **Error Information Log (0x01)** carry the history a fresh SMART page hides — prior thermal excursions, firmware changes, error bursts. **Capture both (plus telemetry-log) into the test record on any failure, and capture them before any erase** — a `sanitize` on some drives wipes the logs you would have wanted. The non-volatile history, not the moment-of-test snapshot, is what lets Quality and the vendor root-cause a return.

5.6 Device Self-Test (DST) — free coverage, with one fatal gotcha

The controller can run its own internal diagnostic — a read/verify pass across the media plus internal structural checks — and report pass/fail. It is **coverage you did not have to write**, which makes it valuable. Two modes:

- **Short** (`-s 1`): a few minutes, a bounded sample of the media + internal checks. Run this on every unit.
- **Extended** (`-s 2`): tens of minutes to hours, a full-media pass. Reserve for burn-in / reliability sampling, not per-unit takt time.

```
nvme device-self-test /dev/nvme0 -s 1 # START a short self-test
nvme device-self-test /dev/nvme0 -s 2 # START an extended self-test
nvme self-test-log /dev/nvme0 -o json # POLL: percent complete + pass/fail result code
```

DST is non-blocking — this is the trap. `nvme device-self-test ... -s 1` only *starts* the test and returns immediately. It does **not** wait, and it does **not** return the result. You **must** then poll log page 0x06 (`nvme self-test-log`) for the completion percentage and the **result code of the latest entry** (0 = passed; nonzero = a failure code per the NVMe spec). A test that fires a DST and never reads 0x06 has proven *nothing* — it gives false assurance. The right pattern is: **start a short DST early, run your other tests (fio, link checks) in parallel, then read the 0x06 result at the end.** The toolkit does exactly this — `start_self_test()` kicks it off, `poll_self_test()` polls `self_test_log()` on an interval until `in_progress` clears or a timeout fires, and only then reads the result code; the gate is `result == 0`.

And the parser must not default a missing result to PASS. The trap above has a sharper edge in the *parsing* code. `nvme-cli`'s JSON key names drift across versions (the `self-test-log` array and the result field have been spelled differently), so a parser that does `result = entry.get("Self Test Result", 0)` returns 0 (= passed) the moment the key name doesn't match — a **failing DST silently reads as PASS**. Two rules: (1) if the expected keys are absent, **raise**, don't default to a pass; (2) pin the exact JSON shape with a captured corpus (`nvme self-test-log -o json` from your deployed version, one passing and one failing entry) so a key rename fails a replay test, not a DUT on the line. "Default the absent oracle to good" is the same wrong-PASS pattern the toolkit's audit found here — the fix is to fail loud, then pin the format.

Real `nvme self-test-log` output mid-test and after a clean pass:

```
Device Self Test Log for NVME device:nvme0
Current operation : 0x1 <- 0x1 = short in progress (0x0 = none, 0x2 = extended)
Current Completion : 60% <- poll this until Current operation returns to 0x0
Self Test Result[0]:
  Operation Result : 0x0 <- 0x0 = completed without error; this is the gate
  Self Test Code : 0x1 <- which test produced this entry (short)
  Power on hours (POH) : 0x1
  Vendor Specific : 0 0
```

The **Operation Result** value is a 4-bit code, and only the exact values matter — do not treat “nonzero” uniformly, because some nonzero codes are *aborts* (inconclusive, re-run) while others are genuine *failures* (hard fail + telemetry). The NVMe-spec codes:

Code	Meaning	Verdict
0x0	Completed without error	pass (this is the gate)
0x1	Aborted by a Device Self-test command	inconclusive — re-run
0x2	Aborted by a Controller-Level Reset	inconclusive — re-run
0x3	Aborted, a namespace was removed	inconclusive — re-run
0x4	Aborted by a Format NVM command	inconclusive — re-run

Code	Meaning	Verdict
0x5	A fatal or unknown error occurred during the test	fail + telemetry
0x6	Completed, failed, segment that failed not known	fail + telemetry
0x7	Completed, one or more segments failed	fail + telemetry (read SegmentNumber + FailingLBA)
0x8	Aborted for an unknown reason	inconclusive — re-run
0x9	Aborted due to a Sanitize operation	inconclusive — re-run
0xF	Entry not used (no self-test result here yet)	not a result

So the gate is exactly `result == 0x0`, and 0x5/0x6/0x7 are the real failures (capture telemetry; for 0x7 the entry also carries the `SegmentNumber` and `FailingLBA`). The `Current operation` field reads 0x1/0x2 while a short/extended test runs and returns to 0x0 when done — poll that back to 0x0 before you trust `Self Test Result[0]`, or you will read a stale prior result.

5.7 nvme-cli usage with representative output

```
nvme list # all namespaces: node, model, serial, FW, size
nvme list -o json # machine-readable (script against this, not text)
nvme id-ctrl /dev/nvme0 -o json # controller identify
nvme id-ns /dev/nvme0n1 -o json # namespace identify
nvme smart-log /dev/nvme0 -o json # the health gate
nvme error-log /dev/nvme0 -o json # recent command errors
nvme fw-log /dev/nvme0 # firmware slots + active slot
nvme get-feature /dev/nvme0 -f 0x04 -H # Temperature Threshold feature (over/under, per sensor),
↪ human-readable
nvme self-test-log /dev/nvme0 -o json # DST result
nvme telemetry-log /dev/nvme0 -o telem.bin # capture telemetry blob (on failure)
```

Representative `nvme list` output on a healthy module:

```
Node SN Model Namespace Usage Format FW Rev
-----
/dev/nvme0n1 S5GXNX0R Zoox-Logging-3.84TB 1 0.00 B / 3.84 TB 512 B ZX2.14
```

Representative trimmed `nvme smart-log -o json` (the fields the gate reads):

```
{
  "critical_warning": 0,
  "temperature": 314,
  "avail_spare": 100,
  "spare_thresh": 10,
  "percent_used": 0,
  "data_units_read": 5678,
  "data_units_written": 1234,
  "media_errors": 0,
  "num_err_log_entries": 0,
  "warning_temp_time": 0,
  "critical_comp_time": 0,
  "thm_temp1_trans_count": 0,
  "thm_temp2_trans_count": 0,
  "power_on_hours": 1,
  "power_cycles": 3,
  "unsafe_shutdowns": 0
}
```

```
}
```

Note `temperature: 314` — that is **Kelvin** ($314 - 273 = 41$ C; the exact relation is $C = K - 273.15$, but the NVMe field is an integer Kelvin count so a flat -273 is what every tool uses). The composite temperature and all eight `temperature_sensor` fields are reported in Kelvin per the spec; your code must convert. The toolkit handles this defensively: if the parsed temperature is > 200 it subtracts 273, which covers the Kelvin-vs-Celsius ambiguity across `nvme-cli` versions (some print the converted Celsius, some the raw Kelvin) without guessing wrong on a real 41 C reading.

5.8 PCIe-attach implications (point to the PCIe chapter)

An NVMe drive is a PCIe endpoint, full stop. The whole PCIe chapter — Link Training and Status State Machine (LTSSM), link train/width, AER correctable/uncorrectable decode, the write-1-to-clear arm/stress/read discipline, lane margining, retrain counting — applies to its link **before** any storage test is meaningful. Concretely, on the floor:

- **Verify the link first.** A logging drive should be at, say, Gen4 x4. Find its PCIe BDF from `sysfs` and read `current_link_speed / current_link_width`. A drive that trained Gen4→Gen3 or x4→x2 is a *PCIe* finding (SI, seating, bifurcation), and no amount of SMART reading will explain it.
- **Watch its AER counters across the soak.** Bad-TLP / Replay-Timer correctables climbing during a write soak point at physical-layer Signal Integrity (SI) on the M.2/U.2 connector or trace — identical to the PCIe-chapter triage, just on a drive.
- **“Drive disappeared mid-test”** can be **Downstream Port Containment (DPC)** (DPC) on the root port firing on a fatal error, a surprise-down, or a power glitch — read it the way the PCIe chapter says, not as “the SSD died.”

The toolkit’s NVMe check is explicitly documented to run **alongside** the PCIe diagnostic on the drive’s link — the storage health check and the link check are two halves of one qualification.

5.9 Stress and data integrity with fio

SMART tells you the drive’s *opinion of itself*; `fio` makes you form your own. These are the recipes you keep on the bench:

```
# Sequential WRITE throughput - does it hit rated BW, and does it throttle/heat under sustain?
fio --name=seqwrite --filename=/dev/nvme0n1 --rw=write --bs=128k \
    --iodepth=32 --numjobs=1 --direct=1 --runtime=120 --time_based --group_reporting

# Sequential READ bandwidth
fio --name=seqread --filename=/dev/nvme0n1 --rw=read --bs=128k \
    --iodepth=32 --numjobs=1 --direct=1 --runtime=30 --time_based --group_reporting

# Random 4K READ IOPS - the queue-depth / parallelism test (high iodepth, many jobs)
fio --name=randread --filename=/dev/nvme0n1 --rw=randread --bs=4k \
    --iodepth=256 --numjobs=4 --direct=1 --runtime=120 --time_based --group_reporting

# Mixed 70/30 read/write - a realistic logging-ish workload
fio --name=mixed --filename=/dev/nvme0n1 --rw=randrw --rwmixread=70 --bs=4k \
    --iodepth=64 --numjobs=4 --direct=1 --runtime=300 --time_based --group_reporting

# DATA INTEGRITY - write a known pattern, read it back, verify every byte
fio --name=verify --filename=/dev/nvme0n1 --rw=write --bs=64k --direct=1 \
    --verify=crc32c --verify_fatal=1 --do_verify=1 --size=4G

# Sustained ENDURANCE write (burn-in; large size, time-based)
fio --name=endurance --filename=/dev/nvme0n1 --rw=write --bs=128k \
```

```
--iodepth=32 --numjobs=1 --direct=1 --size=100G --runtime=3600 --time_based
```

--direct=1 bypasses the page cache so you measure the **drive**, not host RAM — non-negotiable for a real measurement. --verify=crc32c is the integrity test that catches silent data corruption: write a CRC-tagged pattern, read it back, compare; --verify_fatal=1 aborts on the first miscompare so a corruption can't hide in a sea of good blocks.

What to watch *during* the soak, not just after:

- **Throughput holding steady.** A drive that does rated BW for 10 s then *halves it* hit the thermal wall — that is throttling, and only a module-level soak (not a 10-second Printed Circuit Board Assembly (PCBA) check) catches it.
- **Composite temperature vs the thresholds.** Poll nvme smart-log temperature against wctemp/cttemp. TMT1 triggers light throttle; TMT2 heavier. A throttle event is a finding about your **airflow/heatsink**, recorded via the thermal transition counters above.
- **AER on its PCIe link** — because it is a PCIe device.

```
# Live temperature monitor during a soak
watch -n 1 'nvme smart-log /dev/nvme0 -o json | jq ".temperature - 273"'
```

Destructive-test discipline — treat this like a loaded tool. Writing to /dev/nvme0n1, nvme format, and nvme sanitize **destroy data irrecoverably**. On the line that is fine — the drive is blank. On any shared or development machine it is a disaster, and “I ran the wrong device node” has wiped engineers’ boot drives. Every destructive step in your toolkit must (a) require an explicit “yes, this exact device, I mean it” confirmation, and (b) refuse to run against a **mounted** filesystem or the **boot** drive. Build the guard once and never bypass it.

5.10 Failure signatures → root cause

Symptom	Most likely cause	First moves
In lspci but not in nvme list (no /dev/nvme0n1)	No namespace provisioned , or controller init failed	ls /dev/nvme* (is /dev/nvme0 there?); nvme id-ctrl /dev/nvme0; nvme list-ns /dev/nvme0; create namespace if none
Trained below expected speed/width (Gen4->Gen3, x4->x2)	PCIe SI / seating / bifurcation — a link problem	PCIe chapter: link speed/width, lane margining, reseal; <i>not</i> a SMART issue
critical_warning != 0	Drive self-reports failing (spare/temp/read-only/backup)	Decode the bit; read SMART; read error-log 0x01; usually RMA
media_errors > 0 or rising	Bad NAND / uncorrectable media	Read error-log 0x01 for the LBAs; fail; capture telemetry
Namespace went read-only (Critical-Warning bit 3)	Spare exhausted, or controller protective lockdown	Hard fail / RMA; the drive stopped accepting writes to protect data
Throughput cliff mid-soak; thm_temp*_trans_count rising	Thermal throttling — airflow/heatsink/mount	Fix cooling on the fixture/module; <i>not</i> inherently a bad drive
Controller timeout / reset in dmesg (nvme nvme0: I/O timeout, resetting controller)	Firmware hang, severe thermal, or a dying drive; sometimes a PCIe link event	dmesg grep nvme ; check link/AER; check temperature; reproduce; if persistent, RMA
power_on_hours/power_cycles high on a “new” drive	Re-stock / returned inventory	Quality/supply-chain finding; flag the lot; not a drive defect
DST result != 0	Controller’s own diagnostic failed	Read the result code; fail; capture logs + telemetry for FA

The dmesg lines worth recognizing on sight:

```
nvme nvme0: I/O 384 QID 3 timeout, aborting
nvme nvme0: Abort status: 0x0
nvme nvme0: I/O 384 QID 3 timeout, reset controller
nvme nvme0: 16/0/0 default/read/poll queues
```

A single timeout under a brutal soak can be a fluke; **repeated** timeouts or a controller reset is a failing unit (or a link that keeps dropping the drive — check AER/DPC first).

5.11 Firmware update flow (you will own this)

“Release updated test programs for new generations of hardware” includes flashing drive firmware on the line:

```
nvme fw-download /dev/nvme0 --fw=image.bin # stage the image (chunked to the controller)
nvme fw-commit /dev/nvme0 --slot=1 --action=1 # commit to slot 1, activate on next reset
# then a controller reset (or power cycle) to run it; re-read fw-log to CONFIRM the active slot
nvme fw-log /dev/nvme0
```

The gotchas: the **commit action code** matters (download-only vs activate-on-reset vs activate-immediately), and some drives need a **full power cycle**, not just a controller reset, to run the new image. Your test must confirm the new version is *running* — **fw-log** shows the new revision in the **active slot** — not merely that it downloaded. Record the firmware version in the test result for traceability (a firmware delta means you re-qualify the test).

5.12 Manufacturing-test flow (provision → test → soak → gate)

The end-to-end NVMe flow at module test, in order:

1. **Enumerate & identify.** `nvme list` — drive present, correct model/FW? If it’s in `lspci` but not here, **provision a namespace** (`nvme create-ns ... --nsze ... --ncap ... --flbas 0`, then `nvme attach-ns`) — un-provisioned drives are common and this is a *step*, not a failure.
2. **Verify the PCIe link.** Expected speed/width (Gen4 x4 etc.); AER clean. (PCIe chapter.)
3. **Format / secure-erase to a known state.** `nvme format /dev/nvme0n1 --ses=1` (secure erase) or a **sanitize** for a stronger crypto/block erase — establishes a clean, known-LBA-format starting point and removes any factory test data. **Destructive — gated.**
4. **SMART gate (baseline).** `critical_warning==0`, `media_errors==0`, `percentage_used~0`, `available_spare==100`, temperature in range — and the **re-stock check** (`power_on_hours/power_cycles` low). Capture model/serial/FW for traceability.
5. **Start a short DST** in the background (it runs while step 6 runs).
6. **Performance + integrity.** `fio` sequential R/W (must meet the **datasheet BW limit**), random 4K IOPS (must meet the **IOPS limit**), and a CRC verify pass. These limits come from the datasheet for go/no-go and from *your fleet data* for tightening (capture the numbers, not just pass/fail).
7. **Soak (module/burn-in).** Sustained write at temperature; watch for the throughput cliff and the thermal transition counters — this is the phase that catches throttling and marginal drives that a room-temp PCBA check passes.
8. **Read the DST result** (poll log 0x06; `result==0`).
9. **Post-stress SMART.** Re-read: **no new media_errors**, no new error-log entries, no new thermal transitions, `available_spare` unchanged. A delta here is the real catch.
10. **Capture logs.** On any failure, dump error-log (0x01), persistent-event-log (0x0D), and telemetry (0x07) **before** any erase, into the test record. On pass, still log the lifetime counters for genealogy.

The limits, summarized: zero `critical_warning`, zero `media_errors`, zero new error-log entries, `percentage_used ~0`, `available_spare` 100%, temperature in range with zero thermal-throttle transitions, `power_on_hours/power_cycles` low (re-stock gate), DST passes, and `fio` BW/IOPS at or above datasheet

— every one captured as a number so Design Verification (DV) can set and Quality can tighten the limit later.

5.13 NVMe health check in Python (toolkit cross-reference)

The toolkit’s `computetest.nvme` module implements this gate. The shape is the same “capture the parameter, then compare to a limit” pattern used across the toolkit:

```
from computetest.nvme import check_nvme

# Reads SMART + Identify, applies new-drive limits, optionally runs + polls a DST.
h = check_nvme("/dev/nvme0", max_temp_c=70, max_power_on_hours=50, run_self_test=True)

print(h.summary())
# /dev/nvme0 Zoox-Logging-3.84TB fw=ZX2.14 temp=41C used=0% media_err=0 poh=1 -> OK dst=pass

if not h.ok:
    failed = [k for k, ok in h.checks.items() if not ok] # e.g. ['media_errors==0']
    # capture logs + telemetry, then fail the unit

for flag in h.history: # used-stock / RMA-history flags that did NOT fail the gate
    log.warning("NVMe history: %s", flag) # e.g. "unsafe_shutdowns=40"
```

Two design points worth lifting from that module into any NVMe test you write:

- **Separate faults from history.** `checks` (faults) drive pass/fail; `history` (high lifetime counters) is logged for fleet/quality but does **not** auto-fail a drive that is functionally fine — except the one `power_on_hours` gate, which is the cleanest re-stock signal. Gating the pass on lifetime counters that a legitimately power-cycled good drive accrues would false-fail good parts.
- **DST is polled to completion, never fire-and-forget.** `run_self_test=True` calls `start_self_test()` then `poll_self_test()`, and only the polled result `== 0` sets the `self_test_passed` check. This is the antidote to the non-blocking gotcha above.
- **Format drift is normalized at the parse layer.** `nvme-cli` 2.10 → 2.11 renamed the SMART keys (`available_spare` → `avail_spare`, `available_spare_threshold` → `spare_thresh`, `percentage_used` → `percent_used`). The toolkit’s `_normalize_smart_keys()` aliases the abbreviated names to the canonical ones at parse time, so a drive talking either dialect produces the same verdict — and the full corpus of both is replayed in `tests/test_parsers_corpus.py` against captured smart-log JSON from each version. A user with `nvme-cli` 2.11 on the station and the guide written against 2.10 doesn’t need to know about the change.
- **Temperature units are normalized too.** Some `nvme smart-log -o json` implementations report temperature in *Kelvin* (Samsung enterprise parts often emit 323 for 50 °C), others in Celsius. `check_nvme` heuristics: if `temperature > 200`, subtract 273. Verified end-to-end by Phase 3 of the QEMU lane, where the emulated nvme reports Kelvin and the toolkit’s parsed verdict shows the converted 50 °C in the guest. The signal “your unit-test mock won’t catch a real-world encoding the parser handled correctly” is exactly what the QEMU+nvme-loop binding exists to prove.
- **`start_self_test(device, *, extended=False)` is keyword-only on `extended`.** A positional bool — `start_self_test("/dev/nvme0", True)` — is a magic-bool at the call site whose meaning is invisible without grepping the signature. The API now refuses by construction: callers write `start_self_test(dev, extended=True)`, and the meaning is at the call site.

Chapter 6

GPUs: Architecture, ECC/RAS, and Manufacturing Test

This is the GPU chapter. A GPU on a Zoox compute board is two things at once: the most expensive, hottest, highest-power part on the assembly, and **a PCIe endpoint first**. Hold both ideas. Most of what you will chase on a GPU — a link sitting at Gen3 when it should be Gen4, a climbing replay count, a unit that “fell off the bus” under thermal load — is a PCIe / power / thermal problem that the GPU happens to report through its own rich telemetry. The rest is genuinely GPU-specific: Error-Correcting Code (ECC) on a huge memory array, row remapping, XID error codes, and a thermal-under-load behavior that idle tells you nothing about.

This chapter assumes the **PCIe chapter** for the link layer (Link Training and Status State Machine (LTSSM), Advanced Error Reporting (AER), lane margining, the arm/stress/read discipline) and the **Memory chapter** for the host-Dynamic Random-Access Memory (DRAM) Error Detection and Correction (EDAC) story — GPU ECC is the on-package analog of both. What you get here: the architecture you actually need (not a graphics-programming tour), ECC in real depth, the XID table with an action per code, the throttle-reasons bitmask decoded, NVLink, the health-check command set (`nvidia-smi -q, DCGM`), failure signatures mapped to root cause, and the manufacturing flows — what is a screen versus what is an Return Merchandise Authorization (RMA). The companion code is `toolkit/src/computetest/gpu.py`; read a section here, then read the function that does it.

6.1 GPU Architecture, the Parts That Matter for Test

You are not writing CUDA kernels. But you cannot test a part you cannot reason about, and the failure signatures map directly onto the architecture — a double-bit ECC error lives in a specific memory array, a thermal slowdown throttles specific clock domains, an NVLink error is a specific SerDes. So: the architecture, filtered to what changes how you test.

6.1.1 The compute hierarchy

NVIDIA’s datacenter and automotive parts (A100, H100, L40/L40S, the Orin System-on-Chip (SoC)’s integrated GPU, plus whatever Zoox’s roadmap lands on) all share the same hierarchy:

```
GPU die
+-- GPC (Graphics Processing Cluster) a few per die
+-- TPC (Texture Processing Cluster)
+-- SM (Streaming Multiprocessor) the core compute unit
```

```

+-- CUDA cores (FP32/INT) -- ALUs
+-- Tensor cores -- matrix-multiply units (the AI workhorse)
+-- register file, L1 cache / shared memory (per-SM)
+-- L2 cache (shared across all SMs)
+-- Memory controllers --> HBM or GDDR stacks
+-- Copy/DMA engines, NVENC/NVDEC (video), NVLink, PCIe interface

```

What matters for test:

- **SMs are the redundancy and binning unit.** A die has dozens of SMs; vendors fuse off defective ones and sell the result as a lower SKU. You will not usually test at SM granularity, but understand that “the same chip” can ship with different SM counts, and a compute stress test exercises *all* enabled SMs — which is how you surface a marginal one that a light load misses.
- **Tensor cores dominate the perception workload.** Zoox’s stack is inference-heavy, so the parts that get hot and draw power under real load are the tensor cores. A thermal soak that only loads FP32 CUDA cores under-stresses the part; `gpu-burn --tensor` and DCGM’s targeted-stress plugins exist precisely to drive the tensor path.
- **L2 and on-chip RAMs have ECC too**, not just the external memory. When you read ECC counters you are reading errors aggregated across HBM/GDDR *and* the internal SRAMs.

6.1.2 Memory: HBM vs GDDR, and why you care

The GPU’s memory is the part most likely to throw a hard error in test, so know which kind you are looking at:

	HBM2e / HBM3	GDDR6 / GDDR6X
Where	A100, H100, high-end datacenter	L40, consumer, many automotive parts
Construction	Stacked DRAM dies on a silicon interposer, in-package	Discrete chips around the GPU on the PCB
Bandwidth	Very high (TB/s), wide bus (1024-bit+ per stack)	High (hundreds of GB/s), narrower bus
ECC	Native, side-band ECC on extra dies	Often inline/soft ECC — capacity carved from the array
Test implication	A bad stack is unrepairable in-package -> RMA	Can sometimes be a single discrete chip

The ECC distinction bites you. On older GDDR parts, enabling ECC **reduces usable memory and bandwidth** because the ECC bits are carved out of the same array (you will see total memory drop when ECC is on). HBM parts carry ECC on dedicated dies, so enabling it is “free.” For a datacenter/AV part you run **with ECC enabled, always** — turning it off to win a benchmark is exactly the kind of thing that must never happen on a safety-relevant box. Verify it is on.

6.1.3 How the GPU attaches: PCIe vs NVLink, and the driver stack

A GPU connects to the host over **PCIe Gen4/Gen5 x16** — that is the link your PCIe chapter tools margin and Bit Error Rate Test (BERT). Multi-GPU boards may *additionally* wire GPUs to each other (or to an NVSwitch) over **NVLink**, a separate, faster, NVIDIA-proprietary GPU-to-GPU SerDes. Keep them straight:

- **PCIe** is host↔GPU: how the CPU feeds data in and reads results out. Every GPU has it. This is the link that matters for *your* per-GPU qualification.
- **NVLink** is GPU↔GPU: peer bandwidth for multi-GPU workloads (model/tensor parallelism). NVLink 3 (A100) $\approx$ 600 GB/s, NVLink 4 (H100) $\approx$ 900 GB/s bidirectional — far above PCIe. It

has its **own** link-up, CRC, replay, and recovery state to verify, summarized by XID 74 and read with `nvidia-smi nvlink / DCGM`.

The software stack you are testing through, bottom to top:

```
hardware
NVIDIA kernel driver (nvidia.ko, nvidia-vm.ko, nvidia-drm.ko)
+-- emits "NVRM: Xid (...)" lines to dmesg <- your XID source (Section 5)
GSP firmware (GPU System Processor) -- on-GPU microcontroller running much of the
driver logic; XID 119/120 are GSP faults
NVML (libnvidia-ml) -- the C API that backs both tools below
+-- nvidia-smi -- human/CLI front end
+-- DCGM -- the daemon + dcgmi for fleet/automated health (Section 6)
CUDA runtime/toolkit -- what gpu-burn, cuda-memtest, dcgmi diag stress with
```

The two facts to carry: **XID errors come from the kernel driver into dmesg** (so your health check scrapes the kernel log), and **nvidia-smi/DCGM both sit on NVML** (so the fields you query are NVML fields — which is why scripting against `--query-gpu` is stable and parsing free-text `-q` output is fragile;).

Discrete GPU vs Tegra iGPU — almost none of applies to a Jetson. Everything below (ECC, the volatile/aggregate counters, row remapping) and the `nvidia-smi/NVML` tooling above assume a **discrete datacenter GPU** on a PCIe link with its own HBM/GDDR. NVIDIA’s automotive parts — Jetson **Orin/Thor**, and the Jetson dev kit you’ll bring up on — are a different animal: an **integrated GPU on an SoC**, sharing **LPDDR** with the CPU, over no PCIe link. Concretely, on a Tegra: **nvidia-smi does not exist** (L4T ships `tegrastats / jtop`, and only a partial NVML); the iGPU has **no GPU-ECC, no row-remapping, and no PCIe replay counter** to read (those are HBM/GDDR and PCIe-link concepts); and it is **ARM64**, not x86. So a “GPU health” check written against `nvidia-smi -q -d ECC` or `-d ROW_REMAPPER` returns *nothing* on a Jetson — you read die temperatures, power rails, and the GR3D (GPU) / EMC (memory-controller) load from `tegrastats` instead, and you keep the one signal both worlds share: kernel **XID** faults from `dmesg`. Know which class of part is in front of you before you trust a GPU script; on the SoC the fields simply aren’t there, and “clean because absent” is a silent escape.

6.2 The ECC Model in Depth

Datacenter GPUs run **ECC** over their external memory and internal RAMs. This is the GPU’s error-detection conscience, and getting the gating right is the single most common place a GPU test is written wrong. Treat this section as the GPU analog of the AER chapter: same philosophy (arm → stress → read; corrected vs uncorrected; volatile vs lifetime), different registers.

6.2.1 Single-bit (SBE) vs double-bit (DBE)

ECC on these parts is SECDED-class (Single-Error-Correct, Double-Error-Detect), with the high-end memory controllers adding stronger codes:

- **Corrected error — single-bit (SBE).** A single flipped bit the ECC fixed. Data is fine. The exact analog of a PCIe *correctable* error. A handful over a long soak can be normal (cosmic rays, a marginal cell); a **high or climbing** SBE rate is a degrading-memory finding, and NVIDIA raises **XID 92** when the rate crosses a threshold.
- **Uncorrectable error — double-bit (DBE).** Two (or more) bits flipped — detected but *not* correctable. Data is corrupt. The analog of a PCIe *uncorrectable* error, and it raises **XID 48** (plus the contained/uncontained XIDs 94/95 on newer parts). **Any DBE on a unit under test is a fail.**

6.2.2 Volatile vs aggregate — the gating gotcha

This is the distinction that breaks naive tools. NVIDIA keeps **two** sets of ECC counters:

Counter set	Resets on	Lives in	What it tells you
Volatile	reboot / driver reload / <code>nvidia-smi -r</code>	RAM	Errors since the last reset – i.e. during <i>this</i> test
Aggregate	never (lifetime)	InfoROM on the GPU	The part’s whole-life error history

The rule:

Gate the manufacturing pass on VOLATILE uncorrected == 0. Treat a nonzero AGGREGATE as investigate / RMA-history, NOT an automatic fail.

Why this matters: a perfectly good GPU that took one DBE years ago, remapped the row, and has run clean ever since carries that DBE in its **aggregate** count *forever*. If your test queries `ecc.errors.uncorrected.aggregate.total` and fails on nonzero — a real and common bug — you scrap good, already-healed parts on their lifetime history. Conversely, gating only on aggregate can *miss* a fresh error if the part was reset between insertion and test.

The correct flow mirrors AER’s arm/stress/read:

1. **Baseline / arm** — read (or reset) the volatile counters so you are counting *your* stress window, not boot + enumeration + the previous unit.
2. **Stress** — `gpu-burn / dcgmi diag -r 3` for the soak interval.
3. **Read volatile** — `ecc.errors.uncorrected.volatile.total` must be 0; log volatile SBE as a trend metric.
4. **Separately log aggregate** — for history / re-stock detection, never as the pass gate.

The toolkit encodes exactly this split — `_apply_limits()` gates on `ecc_uncorrected_volatile == 0`, while `_history()` records `ecc_uncorrected_aggregate` and remapped-row count as **history flags, not failures** (see the module docstring in `gpu.py`).

A real `nvidia-smi -q -d` ECC block makes the two-counter structure obvious — note the parallel **Volatile** and **Aggregate** subtrees, each with its own SBE/DBE split:

```
Ecc Mode
Current : Enabled
Pending : Enabled
ECC Errors
Volatile
SRAM Correctable : 0
SRAM Uncorrectable : 0
DRAM Correctable : 0
DRAM Uncorrectable : 0
Aggregate
SRAM Correctable : 0
SRAM Uncorrectable : 0
DRAM Correctable : 14
DRAM Uncorrectable : 1
```

This exact part **passes**: volatile uncorrectable is 0 (nothing happened during your window), even though aggregate DRAM uncorrectable is 1 — one lifetime DBE it took, remapped, and has run clean through since. A tool that gates on `ecc.errors.uncorrected.aggregate.total` scraps this good part; the correct tool gates on the **Volatile** subtree and logs the **Aggregate** subtree as genealogy. Two field-name details that trip up parsers: the counters are split **SRAM vs DRAM** (internal RAMs vs framebuffer) — the CSV `--query-gpu .total` fields sum them, so use those when you just want a gate; and the exact label is “Uncorrectable” in

the `-q` tree but “uncorrected” in the `--query-gpu` field name (`ecc.errors.uncorrected.volatile.total`). Match the right one.

6.2.3 Row remapping, retired pages, and remap-pending

When the GPU takes an uncorrectable error (or enough correctables on one row), it does not just log it — it **retires the bad memory** so it is never used again. Two mechanisms by era:

- **Page retirement (Volta and older):** the driver retires the offending 64 KiB page, permanently blacklisting it. Read with `nvidia-smi -q -d PAGE_RETIREMENT / retired_pages.*`. Retirements due to DBE are the serious kind; due to SBE-threshold are the softer kind.
- **Row remapping (Ampere and newer — A100/H100):** finer-grained. The GPU has **spare rows** per memory bank and remaps a failing row to a spare, in hardware, so capacity is preserved. Read with `nvidia-smi -q -d ROW_REMAPPER`. The states you must parse:

Field	Meaning	Test action
Correctable / Uncorrectable remap count	Rows already remapped (lifetime)	History flag, not a fail by itself
Remap Pending : Yes	A remap is queued but needs a GPU reset to take effect	The part is in a degraded state -> reset and re-verify; on a new unit, a finding
Remap Failure Occurred : Yes	A remap was attempted and failed -> no spare row, or the remap mechanism itself failed	Hard fail / RMA . The part can no longer protect itself

When a remap fails is the case that must always fail a unit: it means either the spare rows for that bank are exhausted (the memory is badly degraded) or the remapping hardware is broken. Either way the GPU has lost its ability to heal, and a future DBE will be unrecoverable. **XID 64** is the kernel signature of this; **XID 63** is the benign sibling (remap *succeeded*, reset pending). The toolkit’s `_apply_limits()` fails on `row_remap_failure` and `row_remap_pending`, and `_query_row_remap()` parses the three fields above out of `nvidia-smi -q -d ROW_REMAPPER`.

What actually trips the remap-failure flag is worth knowing, because it tells you *how degraded* a part is and it is also NVIDIA’s stated RMA criterion. Per NVIDIA’s GPU memory error management docs, every DRAM bank ships with a fixed pool of spare rows, and a remapping **failure** is raised when any of these happens:

- a remap is attempted for an uncorrectable error on a bank that **already has 8 rows remapped for uncorrectable errors** (the per-bank spare pool for the UCE class is used up);
- a remap is attempted on a **row that was already remapped** (the remap did not stick);
- **512 total uncorrectable-error remappings** have accumulated across the GPU.

Any one of those sets **Remap Failure Occurred : Yes**, and NVIDIA’s policy is that a part with the row-remap-failure flag set — once **confirmed by the NVIDIA Field Diagnostic** — qualifies for RMA. That is the clean dividing line for the floor: a remap that *succeeded* (pending reset) is a heal; a remap that *failed* is an RMA. Note the contrast with the lifetime *count*: a handful of successfully remapped rows is just history, but the failure flag means the heal machinery itself is out of headroom or broken.

A real `nvidia-smi -q -d ROW_REMAPPER` block looks like this (the fields the parser keys on):

```
Remapped Rows
Correctable Error : 0
Uncorrectable Error : 2
Pending : No
Remapping Failure Occurred : No
Bank Remap Availability Histogram
Max : 95 bank(s)
High : 1 bank(s)
```

```
Partial : 0 bank(s)
Low : 0 bank(s)
None : 0 bank(s)
```

Two remapped uncorrectable rows with **Pending** : No and **Failure Occurred** : No is a part that took two UCEs, healed both, and reset clean — a **history flag, not a fail**. The **Bank Remap Availability Histogram** is the underused field: it buckets banks by how many spare rows remain (Max = full spare pool, down to None = exhausted). A bank in **Low** or **None** is a degrading-part early warning even when the failure flag is still No — log it as a trend metric, the same way you trend SBE rate.

The one-line ECC gate. *Fail on: volatile DBE > 0, remap-failure, remap-pending, or a critical XID. Log (do not fail) on: aggregate DBE, lifetime remap count.* That single rule is what separates a correct GPU test from one that either passes corrupt parts or scraps good ones.

6.3 Clocks, Power, Thermal, and the Throttle Bitmask

A GPU at idle tells you almost nothing. The defects you are catching at module test — bad heatsink mount, wrong or insufficient thermal interface material, a dead fan, paste pump-out, a marginal power stage — only appear **under sustained compute load**. So the test is the load, and the instrument is the set of `nvidia-smi` telemetry fields plus the throttle bitmask.

6.3.1 The fields you read (and their limits)

```
nvidia-smi --query-gpu=index,name,temperature.gpu,temperature.memory,\
power.draw,power.limit,enforced.power.limit,\
clocks.current.sm,clocks.max.sm,clocks.current.memory,\
utilization.gpu,pstate,clocks_event_reasons.active --format=csv
```

Field	What it is	Manufacturing expectation
<code>temperature.gpu</code>	GPU core temp (degC)	Below the slowdown threshold under load (commonly < 83-87 degC; part-specific)
<code>temperature.memory</code>	HBM/GDDR temp	Below its own limit (HBM throttles ~95 degC)
<code>power.draw</code> vs <code>power.limit</code>	Live draw vs cap	Should reach near TDP under burn, not exceed enforced.power.limit
<code>clocks.current.sm</code> vs <code>clocks.max.sm</code>	SM clock vs P0 boost	Stays near P0 under load – a sustained drop means throttling
<code>pstate</code>	Performance state (P0 = max, P8 = idle)	P0 under full load; if it sags to P2/P3 while hot, investigate why
<code>clocks_event_reasons.active</code>	Throttle bitmask (hex)	Decode it – this is the <i>why</i> behind any clock drop

Note the field rename: older drivers call it `clocks_throttle_reasons.active`; newer ones `clocks_event_reasons.active`. Query the one your driver exposes (the toolkit queries `clocks_throttle_reasons.active`). Either way the bits are identical.

6.3.2 Decoding the throttle bitmask

`clocks_event_reasons.active` is a **bitmask** — multiple reasons can be set at once. This is the highest-signal field for any “why are clocks low / why is it slow” question, and reading the raw hex correctly is the skill. The NVML bit definitions:

Bit (hex)	Reason	Benign or a finding?
0x0001	GPU Idle	Benign – nothing running
0x0002	Applications Clocks Setting	Benign – clocks were set by the operator/app
0x0004	SW Power Cap	Driver capping to stay under the power limit . Often normal at full load; suspicious if the limit is set too low
0x0008	HW Slowdown	Finding. Hardware forced a slowdown – thermal <i>or</i> power-brake <i>or</i> a failing voltage regulator. Look at the more specific bits below
0x0010	Sync Boost	Benign – clocks synced across a group of GPUs
0x0020	SW Thermal Slowdown	Finding. Driver throttled because temp hit the soft limit – a cooling problem
0x0040	HW Thermal Slowdown	Serious finding. Hardware throttled at the <i>hard</i> thermal limit (TLIMIT). Cooling is badly inadequate or a sensor/mount problem
0x0080	HW Power Brake Slowdown	Finding. An external power-brake signal (EDPp / PWR_BRAKE#) fired – the PSU/VRM asserted it. Power-delivery problem
0x0100	Display Clock Setting	Benign (not relevant on headless compute)

The toolkit's `_THROTTLE_BAD` map is exactly the four that matter for a fail: {0x8: `hw_slowdown`, 0x20: `sw_thermal`, 0x40: `hw_thermal`, 0x80: `hw_power_brake`} — and `_apply_limits()` fails the unit if **any** of them is set (`no_bad_throttle`). The benign bits (idle, app-clocks, sync-boost) are deliberately *not* in that set, so a GPU that is merely idle or operator-clocked does not false-fail.

How to read it by hand: take the hex value, e.g. 0x0000000000000060. Mask off the bits: 0x60 = 0x40 | 0x20 → **HW Thermal Slowdown + SW Thermal Slowdown** are both set. That is a unit cooking under load — a heatsink/TIM/fan problem, not a silicon defect. Contrast 0x0004 alone (SW Power Cap) under a `gpu-burn` at a deliberately low power limit, which is expected and benign.

6.3.3 Power and clock limits

```
nvidia-smi -q -d POWER # min/max/default/enforced power limits, current draw
nvidia-smi -pl 300 -i 0 # set GPU 0 power limit to 300 W (within min/max)
nvidia-smi -q -d CLOCK # current/max/default clocks per domain
nvidia-smi -lgc 1400,1400 -i 0 # lock SM clock to 1400 MHz (repeatable test point)
nvidia-smi -rgc -i 0 # reset locked clocks back to default
nvidia-smi -pm 1 # PERSISTENCE MODE on -- keeps the driver resident
```

Two operational notes that save you time on the floor:

- **Persistence mode (-pm 1) belongs in every test station's setup.** Without it, the driver unloads when no client is using the GPU, so the *first* query after idle pays a multi-second init penalty and your enumeration/timing looks flaky. Turn it on once at station start. (On the newest drivers this is the `nvidia-persistenced` daemon rather than the deprecated flag — same effect.)
- **Locking clocks (-lgc) makes a power/thermal measurement repeatable.** For a characterization or a station-to-station correlation, pinning the SM clock removes boost-algorithm variance so

two runs are comparable. For a normal go/no-go soak you leave boost free and watch the throttle bits instead.

6.3.4 Confirm ECC is enabled (it is part of the power/clock setup)

```
nvidia-smi -q -d ECC | grep -A2 "Ecc Mode" # Current: Enabled / Pending: Enabled
nvidia-smi -e 1 -i 0 # ENABLE ECC on GPU 0 -- needs a GPU reset to apply
```

ECC mode is per-GPU persistent state in the InfoROM, and toggling it requires a reset (the **Pending** line shows the value that takes effect after reset). A datacenter/AV part **must** run with ECC enabled; a station check that confirms **Ecc Mode: Current: Enabled** belongs in your baseline, because a part that shipped (or got flashed) with ECC off has no memory protection at all.

6.4 The Manufacturing Test Sequence

Now assemble the pieces into the flow you actually run at module test. The shape is the same as every other interface in this guide: **enumerate** → **verify link** → **baseline counters** → **stress hot** → **re-read counters** → **decode** → **verdict**.

```
# 1. ENUMERATE -- correct GPU count present?
gpu_count=$(nvidia-smi -L | wc -l)
[[ "$gpu_count" -eq 4 ]] || fail "Expected 4 GPUs, found $gpu_count"

# 2. PCIe LINK per GPU -- right gen and width (this is a PCIe test surfaced via nvidia-smi)
nvidia-smi -q -d PCIe | grep -E "Link (Gen|Width)|Replay"
# Link Gen Max: 4 Current: 4 (Gen4) -- a new board at Gen3 is your first SI finding
# Link Width Max: 16x Current: 16x -- x8 = a dead lane / bifurcation problem
# Replay Count: 0 -- nonzero/climbing = SI problem (see PCIe chapter)

# 3. BASELINE -- ECC enabled, volatile counters armed, no pre-existing critical state
nvidia-smi -q -d ECC | grep -E "Ecc Mode|Volatile"
nvidia-smi -q -d ROW_REMAPPER | grep -E "Pending|Failure" # must be No / No
dmesg --since "$(uptime -s)" | grep "NVRM: Xid" || true # no pre-existing XIDs

# 4. STRESS -- real compute + memory load, hot (the actual test)
dcgmi diag -r 3 # structured per-subsystem pass/fail (memory, SM stress, PCIe, power)
# and/or: gpu-burn -tc 300 # 5 min tensor-core max-thermal soak

# 5. MONITOR during stress -- watch temp, power, and the throttle bitmask
nvidia-smi dmon -s pucvm -d 1 # power+temp / util / clock / violations / mem, 1 s interval
# (add 'e' for ecc+pcie-replay; 't' is PCIe throughput, NOT temperature -- temp rides on 'p')
# temp must stay under the slowdown threshold; throttle reasons must decode to benign-only

# 6. POST-STRESS verdict
# - volatile uncorrected ECC (DBE) == 0 (Section 2)
# - row-remap pending == No, failure == No (Section 2.3)
# - PCIe replay count not climbing (PCIe chapter)
# - no critical XID in dmesg during the window (Section 5)
# - throttle bitmask had no HW/SW-thermal/power-brake bit set (Section 3.2)
# - GPU still enumerable (did NOT fall off the bus -> XID 79)
```

The `nvidia-smi dmon` field codes are worth memorizing because it is your live floor view, and one of them is a classic trap. The official `-s` metric groups are:

-s flag	What it actually selects (per the nvidia-smi docs)
p	Power (W) AND GPU/memory temperature (degC) – temp lives here, not under t
u	Utilization (SM, memory, encoder, decoder, JPEG, OFA) in %
c	Proc (SM) and memory clocks (MHz)
v	Power violations (%) and thermal violations (boolean)
m	Frame-buffer + BAR1 (+ confidential-compute) memory used (MB)
e	ECC errors (aggregate SBE/DBE counts) AND PCIe replay errors – not ECC alone
t	PCIe Rx/Tx throughput (MB/s) – this is the trap: t is <i>throughput</i> , NOT temperature

The gotcha that bites people: **t** is **PCIe throughput**, and temperature comes from **p**. A monitor string written as “**pucvmet** for temp+power+...” still happens to show temperature (because **p** carries it), but if you reach for a bare **-s t** expecting degrees you get PCIe MB/s instead. For a soak the high-signal set is **p** (power+temp), **c** (clocks), **v** (violations) and **m** (memory); add **e** when you want live ECC/replay counts in the same view. `nvidia-smi dmon -s pucvm -d 1` is the one-line “show me everything once a second” you leave running on a second terminal during a burn.

6.5 XID Errors — the Codes That Matter and the Action for Each

XID errors are NVIDIA’s catch-all hardware/driver error codes, emitted by the kernel driver into `dmesg` as:

```
NVRM: Xid (PCI:0000:65:00): 48, pid=12345, name=python, ...
```

Learning to read them is the GPU analog of decoding AER bits: **each code maps to a root-cause bucket** (app vs memory vs bus vs NVLink vs firmware), which is what makes XID monitoring the single highest-signal GPU manufacturing check. You scrape `dmesg` (or `journald`) for `NVRM: Xid` across the stress window and **bucket by code**. The toolkit’s `_scan_xids()` does exactly that with the regex `NVRM:\s*Xid\s*\([^\]]*\):\s*(\d+)`, counting occurrences per code.

The codes that actually matter, with the action:

XID	Name	Bucket	Action on the floor
13	Graphics Engine Exception (GR: SW Notify Error)	App (usually)	Out-of-bounds / illegal instruction in the workload. Re-run under compute-sanitizer ; rarely HW. Not a unit fail by itself
31	GPU memory page fault (FIFO: MMU Error)	App (usually)	Illegal address access. Usually the test app’s bug; can be driver/HW if it repeats across apps
43	GPU stopped processing (Channel Reset Verification Error)	SW teardown	The driver stopped a misbehaving context (app abort/SIGKILL). GPU stays healthy. Not a fail
45	Preemptive cleanup / channel removal (due to previous errors)	SW teardown	Robust-channel recovery after an app was killed. Benign for the GPU

XID	Name	Bucket	Action on the floor
48	Double-Bit ECC (DBE)	Memory HW	Uncorrectable memory error. Reset/reboot to clear; fail the unit. Repeated across resets -> RMA
62	Internal micro-controller halt	Firmware HW	Firmware fault -> GPU reset. Repeated -> RMA
63	Row-remap / page-retirement event (succeeded)	Memory HW	A row was successfully remapped; reset pending. On a <i>new</i> unit this is a finding (why did a fresh part remap?) -> reset, re-verify, log
64	Row-remap / page-retirement FAILURE	Memory HW	Remap failed – no spare row or broken remap HW. Hard fail / RMA (Section 2.3)
74	NVLink error	NVLink HW	CRC/replay/recovery problem on a GPU-to-GPU or NVSwitch link. Read <code>nvidia-smi nvlink -e</code> ; can be HW -> fail/RMA
79	GPU has fallen off the bus	PCIe / power / thermal HW	GPU is no longer accessible over PCIe. Almost always a hardware failure – link, power droop, or thermal. Hard fail ; correlate with rail/temp logs
92	High single-bit ECC rate (Excessive SBE Interrupts)	Memory (degrading)	SBE rate crossed threshold. Degrading memory -> investigate; trend it, fail if it climbs
94	Contained ECC error	Memory HW	Uncorrectable error isolated to one app (the rest of the GPU keeps running). Restart the app; fail the unit on a new part
95	Uncontained ECC error	Memory HW	Uncorrectable error that escaped containment -> affects multiple apps -> GPU reset required. Hard fail
119 / 120	GSP RPC Timeout / GSP Error	Firmware HW	GPU System Processor (on-die microcontroller) fault -> reset. Repeated -> RMA

The XID gate. In burn-in, **any of XID 48, 63, 64, 74, 79, 92, 94, 95** is a hard fail with a clear root-cause bucket already attached. XID 13/31/43/45 are app/SW teardown noise *unless* they repeat across different workloads. This is precisely the toolkit's `_XID_CRITICAL` set: {48: DBE, 63: remap-pending, 64: remap-failure, 74: NVLink, 79: fell off bus, 92: high-SBE-rate, 94: contained, 95: uncontained} — and `no_critical_xid` fails the unit if any of those appear in the dmesg scan. (XID 92 is the high single-bit-rate code, *not* a

containment code — 94/95 are the contained/uncontained pair.)

A subtlety worth internalizing: **XID 79 (“fell off the bus”) is a PCIe/power/thermal story, not a GPU-internal one.** When you see it, you do not start by suspecting the silicon — you correlate with the PCIe AER counters on that GPU’s root port (PCIe chapter), the rail voltages (a droop under load can drop the GPU off the link), and the thermal log (an over-temp can do the same). The XID tells you *what* happened; the surrounding telemetry tells you *which layer*.

Why 94/95 exist at all — the containment story. On Volta and older, a single uncorrectable ECC error took down **every** running context on the GPU; there was no isolation. Starting with the A100, NVIDIA added **error containment**: the driver tries to confine an uncorrectable error to just the offending application (XID 94, *contained* — the rest of the GPU keeps running) and only falls back to a full GPU reset when it cannot (XID 95, *uncontained*). For *your* purposes both are a fail on a new part — a DBE is a DBE — but the 94-vs-95 split tells you whether the GPU’s containment machinery did its job, which matters when you are deciding screen-vs-RMA on a part that throws these repeatedly.

What the scrape actually looks like, and how to bucket it. The driver writes one line per event; a real soak log with a memory fault in it reads:

```
NVRM: Xid (PCI:0000:65:00): 48, pid=12345, name=python3, Row Remapper: ...
NVRM: Xid (PCI:0000:65:00): 63, pid=0, Row remap: pending, reset required
NVRM: Xid (PCI:0000:b3:00): 31, pid=22871, name=cuda-memtest, Ch 00000008, ...
```

The toolkit’s regex `NVRM:\s*Xid\s*\([^)]*\):\s*(\d+)` pulls the bus-id-and-code shape and keys on the integer, so the three lines above bucket to {48: 1, 63: 1, 31: 1}. The verdict logic then says: 48 and 63 are in `_XID_CRITICAL` -> **fail**, with the bus IDs telling you *which* GPU (65:00 took the DBE-and-remap; b3:00 only logged an app-level MMU fault that is noise unless it repeats). One bench-level gotcha: scrape `dmesg` with a **timestamp filter bounded to the stress window** (`dmesg --since / journalctl --since`), not the whole boot log, or you will pick up enumeration-time and previous-unit XIDs and false-fail. The companion `_scan_xids()` is run against the captured window for exactly this reason.

6.6 Health Checks: `nvidia-smi -q` and DCGM

6.6.1 Parsing `nvidia-smi`: CSV for machines, `-q` for humans

There are two query modes and you must use the right one. `nvidia-smi -q` dumps a deep, human-readable tree (`-d ECC|POWER|CLOCK|TEMPERATURE|PCI|ROW_REMAPPER|PERFORMANCE` to scope it). It is what you read at the bench. But **do not parse it with regex in a test program if you can avoid it** — it is free-form and changes between driver versions.

For automation, `--query-gpu=<fields> --format=csv,noheader,nounits` returns clean CSV that maps straight into a parser, no regex:

```
nvidia-smi --query-gpu=index,name,pci.bus_id,\
pci.link.gen.current,pci.link.width.current,\
temperature.gpu,power.draw,clocks.current.sm,utilization.gpu,\
ecc.errors.corrected.volatile.total,ecc.errors.uncorrected.volatile.total,\
ecc.errors.corrected.aggregate.total,ecc.errors.uncorrected.aggregate.total,\
pci.replay.counter,clocks_throttle_reasons.active \
--format=csv,noheader,nounits
```

This is exactly the field list the toolkit’s `_query_nvidia_smi()` requests. Note it asks for **both** `.volatile.total` and `.aggregate.total` for corrected and uncorrected — so the code can gate on volatile and log aggregate. The few things CSV cannot give you (the row-remapper detail, the ECC *mode*) you fetch with a scoped `-q -d ROW_REMAPPER` and parse narrowly, which is why `_query_row_remap()` is a separate function.

The minimal Python pattern (the toolkit’s `gpu.py` is the full version):

```
import subprocess, csv, io

FIELDS = ["index", "name", "pcie.link.gen.current", "pcie.link.width.current",
"temperature.gpu", "ecc.errors.uncorrected.volatile.total",
"ecc.errors.uncorrected.aggregate.total", "pcie.replay.counter",
"clocks_throttle_reasons.active"]

def query_gpus() -> list[dict]:
    out = subprocess.run(
        ["nvidia-smi", f"--query-gpu='{','.join(FIELDS)}'",
        "--format=csv,noheader,nounits"],
        capture_output=True, text=True, timeout=10, check=True).stdout
    # csv with manual fieldnames since we asked for noheader:
    reader = csv.DictReader(io.StringIO(out), fieldnames=FIELDS, skipinitialspace=True)
    return [dict(row) for row in reader]
```

6.6.2 DCGM: the manufacturing-grade tool

nvidia-smi is for humans; **DCGM (Data Center GPU Manager)** is for test programs. It is a daemon (nv-hostengine) plus the dcgmi CLI, and it is what you wrap for an automated station because it produces **structured, per-subsystem pass/fail** and machine-readable output.

```
dcgmi discovery -l # enumerate GPUs + topology (NVLink/PCIe)
dcgmi diag -r 1 # quick readiness (seconds): driver/NVML/sanity
dcgmi diag -r 2 # medium (~2 min): + PCIe/NVLink, memory bandwidth, integration
dcgmi diag -r 3 # long (several min): full HW diag + stress -- THE qualification run
dcgmi diag -r 4 # extra-long (DCGM >= 2.4): + memtest + Pulse (power-spike) test
dcgmi diag -r 3 -j # JSON output -- parse this in your harness
```

Level	Flag	Coverage (approx.)
1	-r 1	Quick readiness: SW/driver, NVML, basic sanity (seconds)
2	-r 2	Medium: + PCIe/NVLink checks, memory bandwidth, integration (~2 min)
3	-r 3	Long: full HW diag + stress – Memory (Targeted), SM/Targeted Power & Stress, PCIe. The standard qualification run
4	-r 4	Extra-long: adds memtest (walking-1s + pattern) and the Pulse Test (PSU power-spike stress)

The named plugins **-r 3** runs (these are the keys you will see in the output) are: **Memory**, **Memory Bandwidth**, **PCIe** (+ NVLink), **SM Stress**, **Targeted Stress** (drives a target gigaflops via large cuBLAS GEMMs), and **Targeted Power** (drives the part to its power limit). **-r 4** adds **Memtest** and **Pulse Test**. A plaintext run prints a pass/fail grid:

```
+-----+
| Diagnostic | Result |
+=====+
|---- Deployment -----+-----|
| Denylist | Pass |
| NVML Library | Pass |
| Persistence Mode | Pass |
+---- Integration -----+-----|
```

```

| PCIe | Pass - All |
+----- Hardware -----+-----+
| GPU Memory | Pass - All |
| Memtest | Pass - All |
+----- Stress -----+-----+
| Targeted Stress | Fail - GPU 2 |
| Targeted Power | Pass - All |
| SM Stress | Pass - All |
+-----+-----+

```

In a harness you do not parse that grid — you run `dcgmi diag -r 3 -j` and read the JSON, where each test carries a **status** plus, on failure, a **warnings** array with the specific reason (a thermal-violation message, a NVVS error code, an ECC count). The **Fail - GPU 2** on Targeted Stress above is the hook: the JSON for that entry will say *why* (most often a thermal throttle the stress provoked, which points you straight back to the throttle bits for that GPU). Fold the per-test status into your pytest result and attach the warning text to the failure so the bench sees the bucket, not just “DCGM failed.”

`dcgmi diag -r 3` is close to a turnkey GPU module test: memory tests (catch ECC/memory defects), compute stress (drives thermal + power), PCIe checks, all with structured pass/fail. But it is not the *whole* test. Your job is to **wrap it**: set the right plugin thresholds, parse `-j` JSON, fold it into the pytest harness, and **add what it does not cover** — your PCIe lane margining (PCIe chapter), thermal-correlated AER, XID bucketing across the soak, and rail-voltage measurements with a real DMM (power chapter). DCGM complements **gpu-burn**: gpu-burn is a brute max-thermal soak that proves the *cooling solution*; `-r 3/4` gives you structured per-subsystem coverage. Run both.

6.6.3 NVLink and multi-GPU checks

On a multi-GPU board, NVLink is a separate interface to verify. The commands:

```

nvidia-smi topo -m # topology matrix: which GPUs are NVLink (NV#) vs PCIe (PIX/PXB/SYS)
nvidia-smi nvlink -s # per-link state: active/inactive + per-link speed (GB/s)
nvidia-smi nvlink -e # per-link ERROR counters: CRC, replay, recovery
nvidia-smi nvlink -ec # per-LANE CRC error counters (finer than -e; isolates one lane)
dcgmi nvlink --link-status # DCGM view; DCGM_FI_DEV_NVLINK_* fields for fleet monitoring
nvbandwidth # measured GPU-to-GPU / host bandwidth (separate from PCIe BW)

```

What to check: the topology matches the board design (a full mesh or the specific tree Electrical Engineering (EE) specified — a missing NV# entry means a link did not come up), every expected link is **active**, the NVLink **CRC/replay/recovery** counters are zero or not climbing under load, and peer bandwidth hits expected. **XID 74** is the kernel’s summary of an NVLink fault; `nvidia-smi nvlink -e` is where you read the detail that XID 74 points at. Multi-GPU boards also have a **power-budget** test: 4 GPUs at ~300 W each is ~1.2 kW — start all GPUs on max compute simultaneously, watch the PSU rails for droop (should not sag >5%), and confirm no GPU drops off (XID 79) when the whole board is loaded at once.

`nvidia-smi nvlink -e` reports **four** counter types per link; know what each one implies:

Counter	What it counts	Reading it
CRC FLIT Error	Receive flow-control-digit (header/control) CRC errors	A few isolated ones can be recovered; a climbing rate is a marginal lane/SerDes
CRC Data Error	Receive data-payload CRC errors	Same – physical-layer integrity; climbing == SI/SerDes problem
Replay Error	Transmit-side replays (a flit had to be re-sent)	The NVLink analog of a PCIe replay – climbing == marginal link
Recovery Error	Link had to run its recovery/retrain sequence	The serious one: the link went down far enough to need recovery; repeated == failing link

These mirror the PCIe correctable story exactly: an occasional CRC the link corrected is one thing; a **rate that climbs under thermal load** is the finding, and the fix path is the same (check the lane with `-ec`, soak hot, suspect the SerDes / the connector / the peer device). A non-zero **Recovery Error** is the most alarming of the four because it means the link dropped, not merely flickered.

The link-count arithmetic is worth carrying for the topology check. NVLink aggregate bandwidth is *per-link speed x number of links*, so a partly-trained part is easy to spot by the number, not just the missing NV#:

- **A100 / NVLink 3:** 12 links x 50 GB/s = **600 GB/s** total bidirectional.
- **H100 / NVLink 4:** 18 links x 50 GB/s = **900 GB/s** total bidirectional.

So if `nvidia-smi nvlink -s` on an H100 shows only 16 active links instead of 18, you are looking at ~800 GB/s and two links that never trained — a finding even though the GPU enumerates and most links are up. Cross-check the active-link *count* against the part’s spec, not just “are there some links.”

6.7 Failure Signatures → Root Cause

The payoff table. A GPU finding rarely arrives labeled; you read the telemetry and map it to a layer. This is the GPU analog of the PCIe triage table, and it is what you keep open when a unit fails on the fixture.

Symptom (what you observe)	Most likely root cause	First moves
Link at Gen3 (expected Gen4) / x8 (expected x16)	PCIe SI margin, bent pin, bifurcation, thermal	It’s a PCIe problem – compare LnkCap both ends, retest hot/cold, lane-margin per lane (PCIe chapter)
Replay count climbing under load	PCIe physical-layer SI, marginal lane, temperature	Decode AER bits on the root port; thermal soak; margining (PCIe chapter)
Volatile DBE > 0 / XID 48	Uncorrectable memory error (HBM/GDDR or SRAM)	Fail the unit. Check row-remapper state; repeated across resets -> RMA
XID 64 / Remap Failure = Yes	Spare rows exhausted or broken remap HW	Hard fail / RMA – the part can no longer self-heal (Section 2.3)
Remap Pending = Yes on a new unit	A remap is queued (recent uncorrectable)	Reset, re-verify; ask <i>why a fresh part remapped</i> – a finding
High/climbing SBE rate / XID 92	Degrading memory cells	Trend it across the soak; fail if it climbs; correlate with temp
Clocks sag under load, throttle bits 0x40/0x20	Cooling problem – bad TIM/mount, dead fan, pump-out	Re-seat heatsink, re-paste, check fan RPM; <i>not</i> a silicon defect
Clocks sag, throttle bit 0x80 (power brake)	Power-delivery problem – PSU/VRM asserted brake	Scope the rails under load; check PSU sizing and the power-brake net
Clocks sag, only 0x04 (SW power cap) at high load	Often normal – hitting the configured power limit	Check the limit is set correctly (<code>-q -d POWER</code>); raise if too conservative
XID 79 – “fell off the bus”	PCIe / power / thermal HW (not GPU-internal)	Correlate AER (root port) + rail voltage + temp; reseal; hard fail
XID 74 / NVLCRC/replay climbing	NVLink SerDes problem	<code>nvidia-smi nvlink -e</code> ; check the peer GPU and NVSwitch; can be HW
<code>temperature.memory</code> high but core OK	HBM/GDDR cooling path (separate from die)	Check memory thermal pads/contact; HBM throttles ~95 degC
GPU not enumerated at all (<code>nvidia-smi -L</code> short)	Power rail, PCIe link dead, seating, BIOS	Rails first; <code>dmesg</code> for LTSSM-stuck/training-fail; reseal; bifurcation (PCIe chapter)
XID 119/120 – GSP fault	GPU firmware (GSP)	Reset; reflash/match driver+firmware; repeated -> RMA

The throughline: **the throttle bitmask and the XID code each name a layer**. Thermal-bit throttle → cooling. Power-brake bit → power delivery. XID 48/64/92/94/95 → memory. XID 79 → link/power/thermal (go correlate). XID 74 → NVLink. Your test should never report only “GPU failed” — it should report *which bit / which XID*, because that is the first fork in the debug tree, and it is what lets EE fix the board instead of guessing.

6.8 Manufacturing Flows: Screen vs RMA

Tie it to the four phases (platform chapter). A GPU defect is catchable at different phases, and the organizing principle is to put each test at the earliest phase that can catch its defect — the earlier a defect is caught, the cheaper the rework and the less value has been added to a part that was going to fail anyway.

At Printed Circuit Board Assembly (PCBA) (bare board, before the GPU’s thermal solution is even mounted, often at the Contract Manufacturer (CM)): - Enumeration and basic link train — does `nvidia-smi -L` see it, does it reach the expected gen/width at room temperature. Catches gross assembly defects (dead lane, bad solder under the package — pair with X-ray). - Cannot catch: thermal-mount defects (no heatsink yet), marginal-at-temperature behavior.

At module (the heart of GPU test — the unit is sealed with its heatsink/TIM/fan): - **Burn-in / thermal soak**. `gpu-burn` or `dcmi diag -r 3` for a sustained interval (typically 15+ minutes to reach thermal steady state). This is where a bad TIM application, a dead fan, or paste pump-out surfaces as a thermal throttle — defects that **cannot** be caught at room-temperature PCBA. - **ECC stress**. `dcmi diag -r 3/4` memory tests (and `cuda-memtest --stress` if available) drive the memory array to surface weak cells → volatile DBE / row-remap events. - **Thermal-correlated link + XID monitoring**. Run the burn while watching the throttle bitmask, AER on the PCIe link, and `dmesg` XIDs — the marginal-lane-at-85degC and fell-off-the-bus-under-load defects only appear here. - **Power characterization**. Measure the rails with a DMM/scope under full load; confirm no droop, no power-brake assertion.

The module burn-in is the highest-yield step in the whole flow, so it is worth being precise about *why each parameter is set where it is*:

- **Why 15+ minutes, not 2.** A heatsink/TIM defect does not show up cold — the die has to reach **thermal steady state** before a bad thermal path expresses itself as a throttle. Steady state on a big GPU under full tensor load is several minutes of rising temperature; the soak has to run *past* the knee so the temperature has plateaued, then hold so a slow failure (paste pump-out, a fan spinning down) has time to appear. A 2-minute run can pass a part that throttles at minute 8. Pick the dwell from the thermal curve, not a round number.
- **Why tensor load specifically.** `gpu-burn --tensor` / **DCGM Targeted Stress** drive the tensor path, which is the hottest, highest-power region. An FP32-only load under- drives the part and under-tests the cooling. The point of burn-in is to provoke the worst- case thermal, so use the worst-case workload.
- **What “good” looks like during the soak.** Temperature rises and **plateaus** below the slowdown threshold; `clocks.current.sm` holds near P0 boost (a *sustained* sag is the throttle signature); `clocks_event_reasons.active` decodes to benign-only (idle/app-clocks/ sync-boost, or 0x4 SW power cap if you deliberately capped power); volatile DBE stays 0; no new XID in the windowed `dmesg` scrape. A bad TIM mount shows the opposite: temperature keeps climbing to the hard limit and 0x40 (HW thermal) latches.
- **ECC-stress as a distinct objective.** Thermal soak proves the *cooling*; the memory test (DCGM **Memtest** / **Memory**, walking-1s and pattern writes) proves the *array*. They are different defects — a part can cool perfectly and still have weak cells — so a complete module test runs both, not one as a proxy for the other. The ECC-stress pass criterion is `volatile uncorrected == 0` and no new remap-pending/failure across the test.
- **Repeatability across stations.** When you are correlating station-to-station or chasing a marginal part, **lock the clock** (`-lgc`) and **pin the power limit** (`-p1`) so boost variance does not muddy the

temperature/power numbers; for a normal go/no-go soak leave boost free and judge on the throttle bits instead.

At system (multiple GPUs/modules integrated): the **simultaneous** full-load power and thermal test — all GPUs at max at once, system airflow, PSU under aggregate load — plus NVLink/peer bandwidth across the real topology. Single-module test should already have proven each GPU in isolation, so a system-test failure points at integration (power budget, airflow, inter-module links).

At vehicle (EOL): the compute runs in the car; you confirm the GPUs come up and run the real perception workload in the real thermal/vibration environment.

6.8.1 What is a screen vs an RMA

The distinction that decides what happens to a failing unit:

- **A SCREEN is a defect introduced in *your* build that rework can fix** — re-seat the heatsink, re-apply TIM, replace a fan, re-flash firmware, reseal the card. A thermal throttle from a bad mount, an XID 119 from mismatched firmware, a Gen3 link from a poorly seated card: you rework and re-test, and the unit passes. The defect never leaves your line.
- **An RMA is a defect in the GPU itself that you cannot fix** — a DBE that recurs across resets, a remap *failure* (no spare rows / broken remap HW, XID 64), a recurring NVLink HW fault (XID 74), a part that repeatedly falls off the bus after power/thermal/seating have been cleared (XID 79). The part goes back to the vendor.

The judgment call lives in the middle: **a part with nonzero *aggregate* ECC or a lifetime remap count is not automatically either.** It is RMA *history* — most often it tells you a **used or returned part entered your new-build line** (a supply-chain/genealogy finding), which is why you *log* aggregate and remap counts on every unit even when they pass. A fresh part should have a clean lifetime history; one that does not is a flag for quality, not necessarily a scrap. That logging discipline — capture the parameter, not just the verdict (Design Verification (DV)-vs-Manufacturing Test (MT) chapter) — is what lets the fleet data later flag a bad lot or a re-stocked tray before it becomes a field problem.

Chapter 7

DRAM and Memory (EDAC/RAS)

Memory test has the same shape as every other interface in this guide: **clear the counters** → **stress it (hot)** → **read the counters** → **decode an error to a physical part you can Return Merchandise Authorization (RMA)**. The counter system for Dynamic Random-Access Memory (DRAM) is **Error Detection and Correction (EDAC)**, and it is the DRAM analog of PCIe Advanced Error Reporting (AER) and GPU Error-Correcting Code (ECC) — same mental model, different sysfs. On Zoox’s server-grade compute the DRAM is almost certainly **ECC** (RDIMM/LRDIMM), and verifying that ECC *actually detects and corrects* — not just that the box boots — is a functional- safety requirement, not a nicety: ECC is a fault-tolerance mechanism the manufacturing test must prove engages.

The one-sentence version. A single-bit corrected error (CE) is the DRAM analog of a PCIe correctable — a few over a long soak can be benign, a high or concentrated rate is a finding. An uncorrectable error (UE) is the DRAM analog of a PCIe uncorrectable — **any UE on a new unit is a fail**, and it can crash the box. Everything below is how you count them, run the system hot enough to provoke them, and turn one into a Dual Inline Memory Module (DIMM) silkscreen label.

7.1 DDR4/DDR5 fundamentals relevant to test

You do not need to design a memory controller, but a handful of facts change what you test and how you read a failure:

- **The bus is 64 data bits; ECC adds 8** for a **72-bit** channel. That extra DRAM device per rank is what carries the ECC syndrome — enough for **SECEDED** (Single-Error-Correct, Double-Error-Detect) per 64-bit word.
- **Ranks and channels.** A DIMM has one or more **ranks** (a set of DRAM chips the controller activates together to make a full data word). The controller has multiple **channels**, each driving one or more DIMMs. EDAC reports per-**csrow**/per-**channel** (DDR4 style) or per-**dim**/per-**rank** (DDR5 style), which is the granularity at which you localize a fault.
- **DDR5 splits each DIMM into two independent 32-bit sub-channels** (so a DDR5 DIMM presents as *two* narrower channels), runs at higher data rates, moves voltage regulation **onto the DIMM** (the PMIC), and adds **on-die ECC** (below). The sub-channel split matters because a fault localizes to a sub-channel, and the higher rates make signal-integrity and thermal margin tighter — DDR5 is *less* forgiving, which is exactly why the hot soak matters more.
- **Refresh and timing.** DRAM is leaky; cells are refreshed on an interval (tREFI). Marginal cells fail when refresh can’t keep them charged — which is strongly **temperature dependent** (hotter = leakier = more refresh stress). This is the physical reason “run it hot.”

7.2 ECC: SECCDED, on-die ECC, and scrubbing

- **SECCDED (system ECC)**: the controller computes an 8-bit Hamming-style code over each 64-bit word. It can **correct any single-bit** error and **detect (not correct) any double-bit** error. A corrected single-bit event is a **CE**; a detected-but-uncorrectable double-bit (or worse) event is a **UE**.
- **Chipkill / SDDC (Single Device Data Correction)**: high-end controllers go beyond SECCDED and can **correct an entire failed x4 (or x8) DRAM device** by spreading Reed-Solomon symbols across devices. This is a stronger scheme than basic 72-bit SECCDED — a whole chip can die and the system keeps running and correcting. Know whether the platform has it, because it changes what “a CE” means (the controller may be masking a dead device).
- **On-die ECC (ODECC, DDR5)**: *internal* single-bit correction **inside each DRAM die**, before data leaves the chip. DDR5 mandates it (it is a *yield* feature — it lets the fab ship die with isolated weak cells): each die computes a SECCDED-class code over an internal ~128-bit word with 8 extra check bits and corrects single-bit errors on read. The check bits are **not** transmitted on the bus, so the correction is **invisible to the host and does not replace system ECC** — the host’s EDAC counters never see an error ODECC silently fixed. DDR5 also adds **on-die Error Check and Scrub (ECS)**, an internal scrub the die runs on its own array, again invisibly. The danger is mistaking “DDR5 has ECC” (on-die, internal, invisible) for “system ECC is on and clean” (controller-level, the 72-bit channel ECC, which is what your EDAC counters actually monitor). They are different layers, and only the second one is observable to your test.
- **Scrubbing** keeps single-bit errors from *accumulating* into uncorrectable double-bit errors over time:
- **Patrol scrub**: the controller proactively walks all of memory in the background, reads each location, and **rewrites the corrected data** so a latent single-bit flip is fixed before a second bit in the same word flips and makes it uncorrectable.
- **Demand scrub**: when a read hits a correctable error, the controller writes back the corrected value immediately (scrub-on-demand).
- Why you care: patrol scrub is *why* a system can run for months without a CE turning into a UE, and a **disabled** scrubber is a latent reliability bug your test should verify is on. It also means CE counts you read are “errors found and fixed,” not “errors waiting.”

The DDR5 masking gotcha (state it back the way it bites). DDR5 on-die ECC can **hide a marginal cell from system-level EDAC counters** — the die corrects it internally and the host sees nothing. So a DDR5 system can look *perfectly clean* in EDAC while a die is silently working overtime to correct a degrading cell. The takeaway: **system ECC + EDAC remain necessary** to catch what escapes on-die correction, and a clean EDAC count on DDR5 is weaker evidence than on DDR4. Do not read “DDR5 has ECC” as “I can trust a clean EDAC count” — lean harder on the *stress* (miscompare detection in **stressapptest**) to provoke the cell past what ODECC can hide.

Reading “is this module ECC?” from the SPD — DDR5 moved the byte. When a tool reads the module’s SPD EEPROM to record density/ECC for the RAS profile, the **SPD layout changed from DDR4 to DDR5**. DDR4 carried the bus-width extension (the ECC indicator) in SPD **byte 13**; DDR5 (JESD400-5) puts the Memory Channel Bus Width — bus-width extension at **byte 235, bits [4:3]** — and byte 13 is now thermal/refresh options. A parser ported from DDR4 that still reads byte 13 reports ECC from an unrelated field on a DDR5 module, and may not read far enough into the (longer) DDR5 SPD to reach byte 235 at all. Read the generation’s byte and guard the buffer length. (A real DDR5-SPD audit find — and a textbook *tautology trap*: the test corpus had been hand-built to satisfy the byte-13 read, so it stayed green while decoding the wrong byte.)

7.3 The memory controller (where the counters come from)

On modern server silicon the **integrated memory controller (iMC)** lives on the CPU die, one or more per socket, each owning several channels. When ECC corrects or detects an error, the iMC logs it in **machine-check (MCA) registers**, and the platform reports it either via a **CMCI** (Corrected Machine

Check Interrupt) for CEs or an **MCE** for UEs. The Linux EDAC subsystem (and `rasdaemon`) consumes those events and surfaces them as the counters you read. This is why the model is identical to AER: a hardware block latches errors into registers, an OS layer drains them, and your test arms/stresses/reads. The chipset-specific EDAC driver (`skx_edac`, `i10nm_edac`, `amd64_edac`, etc.) is what knows your controller's topology and must be **loaded** for `/sys/devices/system/edac/mc/` to populate — a missing driver looks like “no memory errors ever,” which is a silent test escape (verify the `mc*` nodes exist).

7.4 EDAC sysfs — reading memory errors in Linux

The kernel surfaces the controller's error counters under `/sys/devices/system/edac/mc/`:

```
# Per-controller totals (mc0, mc1, ... one per integrated memory controller)
cat /sys/devices/system/edac/mc/mc0/ce_count # corrected (single-bit) errors
cat /sys/devices/system/edac/mc/mc0/ue_count # uncorrectable (double-bit+) errors

# Per-DIMM / per-rank breakdown (the localization granularity)
cat /sys/devices/system/edac/mc/mc0/dimm0/dimm_ce_count # CE on this specific DIMM
cat /sys/devices/system/edac/mc/mc0/dimm0/dimm_label # the silkscreen label (if populated)
cat /sys/devices/system/edac/mc/mc0/dimm0/dimm_location # channel/slot location
cat /sys/devices/system/edac/mc/mc0/dimm0/size # MB

# Older / csrow-style controllers expose csrowN/chX_ce_count instead of dimmN
ls /sys/devices/system/edac/mc/mc0/
```

The structure under each `mcN` (these attribute names are the kernel EDAC sysfs ABI):

```
/sys/devices/system/edac/mc/mc0/
mc_name <- the EDAC driver / controller name (e.g. "skx_edac")
size_mb <- total memory this controller manages, in MB
ce_count <- controller-total corrected errors
ue_count <- controller-total uncorrectable errors
ce_noinfo_count <- CEs the controller could not attribute to a DIMM
ue_noinfo_count <- UEs with no location info
reset_counters <- write-only: writing here zeroes this mc's counters
seconds_since_reset <- seconds since the last counter reset (your baseline clock)
sdram_scrub_rate <- scrub bandwidth in bytes/sec (0 or absent = scrub off/unsupported)
max_location <- the deepest topology string this mc can report
dimm0/ dimm1/ ... (or rank0/ ... or csrow0/ ... on older drivers)
dimm_ce_count <- per-DIMM corrected count
dimm_ue_count <- per-DIMM uncorrectable count
dimm_label <- e.g. "DIMM_A1" (only if you registered the label DB)
dimm_location <- e.g. "memory controller 0 channel 1 slot 0"
dimm_mem_type <- e.g. "Registered-DDR5"
dimm_edac_mode <- the ECC scheme in force, e.g. "S4ECD4ED" or "SECDDED"
size <- DIMM size in MB
```

Two of these directly serve the arm/stress/read discipline below: `reset_counters` (write-to-zero, when the driver supports it) and `seconds_since_reset` give you a clean baseline and a rate denominator, and `sdram_scrub_rate` lets the test **prove the patrol scrubber is on** (a zero here on a platform that should scrub is the “disabled scrubber” latent bug called out earlier). `dimm_edac_mode` is worth logging too — it tells you whether the controller is running plain SECDDED or a Chipkill/SDDC-class code (S4ECD4ED = single-x4-device correct, double-x4-device detect), which changes what a clean CE count means.

A real listing on a two-controller DDR5 box looks like:

```
$ ls /sys/devices/system/edac/mc/
mc0 mc1
$ ls /sys/devices/system/edac/mc/mc0/
ce_count ce_noinfo_count dimm0 dimm1 dimm2 dimm3 max_location
```

```
mc_name reset_counters sdram_scrub_rate seconds_since_reset size_mb
ue_count ue_noinfo_count
$ cat /sys/devices/system/edac/mc/mc0/dimm0/dimm_location
memory controller 0 channel 0 slot 0
$ cat /sys/devices/system/edac/mc/mc0/dimm0/dimm_edac_mode
S4ECD4ED
```

To read it in a test: sum `ce_count/ue_count` across every `mcN` for the totals, and walk each `dimmN/rankN` for the per-DIMM breakdown, falling back to the directory name (`dimm0`) when `dimm_label` is empty. That is exactly what the toolkit’s `_read_edac()` does:

```
# computetest.memory._read_edac() (real-hardware path)
def _read_edac():
    mcs = sorted(glob.glob("/sys/devices/system/edac/mc/mc[0-9]*"))
    total_ce = total_ue = 0
    per_dimm = {}
    for mc in mcs:
        total_ce += _read_int(f"{mc}/ce_count")
        total_ue += _read_int(f"{mc}/ue_count")
    for dimm in sorted(glob.glob(f"{mc}/dimm[0-9]*") + glob.glob(f"{mc}/rank[0-9]*")):
        label = _read_str(f"{dimm}/dimm_label") or os.path.basename(dimm)
        per_dimm[label] = per_dimm.get(label, 0) + _read_int(f"{dimm}/dimm_ce_count")
    return len(mcs), total_ce, total_ue, per_dimm
```

Note the two robustness moves you should copy: glob **both** `dimm*` and `rank*` (different EDAC drivers name the leaf nodes differently), and fall back to the **node name** when the silkscreen label has not been populated — so the test still localizes to *something* even on a board whose label DB you have not built yet.

Arm/stress/read for memory. EDAC counters are **monotonic** — they count from boot and do not auto-clear, so a raw read includes errors from POST, boot, and prior tests. For a clean per-unit measurement you must **baseline before the stress window** (read the counts, or reset them where the driver allows) and compare the *delta* after the soak. Reading the absolute count and gating on it is the EDAC version of forgetting to Write-1-to-Clear (W1C)-clear AER before a Bit Error Rate Test (BERT) — you end up failing units on errors that predate your test.

7.5 rasdaemon and decoding an error to a physical DIMM silkscreen label

This is the highest-value capability in the whole chapter, because it turns a vague “the board has memory errors” into an **actionable RMA**: “*DIMM_A1 is throwing 40 CE/hour, swap that stick.*” `rasdaemon` is the userspace daemon that consumes the kernel’s Reliability, Availability, Serviceability (RAS) tracepoints, decodes each CE/UE, and logs it — with the DIMM label and a timestamp — to a SQLite DB.

```
ras-mc-ctl --summary # totals of CE/UE seen since rasdaemon started
ras-mc-ctl --error-count # CE/UE per DIMM (from sysfs)
ras-mc-ctl --errors # per-error detail: which DIMM, rank, syndrome, WHEN
ras-mc-ctl --layout # the memory topology (controllers, channels, slots, sizes)
```

`ras-mc-ctl --errors` is the one you screenshot into a failure report:

```
1 2026-05-24 11:42:07 -0700 1 Corrected error(s) memory read error at
  ↳ CPU_SrcID#0_MC#1_Chan#0_DIMM#0 ... label="DIMM_A1"
2 2026-05-24 11:42:09 -0700 1 Corrected error(s) memory read error at
  ↳ CPU_SrcID#0_MC#1_Chan#0_DIMM#0 ... label="DIMM_A1"
```

`--summary` rolls the same data into per-DIMM totals (the `location`: tuple is `mc:top:mid:low`, i.e. controller : channel : slot, with `-1` meaning “not applicable at this level”):

```
Memory controller events summary:
Corrected on DIMM Label(s): 'DIMM_A1' location: 1:0:0:0 errors: 42
Corrected on DIMM Label(s): 'DIMM_B1' location: 1:1:0:0 errors: 1
No uncorrectable errors.
No PCIe AER errors.
No MCE errors.
```

and `--error-count` is the columnar CE/UE per DIMM you parse for the gate (labeled once the label DB is registered; bare `mc#0csrow#2channel#0`-style rows before that):

```
Label CE UE
DIMM_A1 42 0
DIMM_B1 1 0
```

The path from a kernel CE to the silkscreen label `DIMM_A1` (so the operator knows which physical slot to pull):

1. The iMC logs the error; the EDAC driver attributes it to a (controller, channel, slot) tuple — by default an abstract location like `CPU_SrcID#0_MC#1_Chan#0_DIMM#0`.
2. To turn that into the **board’s silkscreen label** (`DIMM_A1`, the text printed next to the socket), you write a board-specific label map keyed by **DMI/SMBIOS identity**. `ras-mc-ctl` reads `/sys/class/dmi/id/board_vendor` and `/sys/class/dmi/id/board_name` (falling back to `dmidecode`), matches them against a config file under `/etc/ras/dimm_labels.d/` (or `/etc/ras/dimm_labels.db`), and `ras-mc-ctl --register-labels` writes the names into each `dimm*/dimm_label` sysfs node. The config format is a `Vendor: / Model:` header followed by `label: mc.top.mid.low` lines. You build this mapping **once per board revision** — read the schematic/mechanical to learn which controller/channel/slot maps to which silkscreen, write the file, and **every station inherits it** (register at boot via the `ras-mc-ctl` systemd unit).
3. With the labels registered, `dimm_label` reads `DIMM_A1` and `ras-mc-ctl --errors`, `--summary`, and `--error-count` all print it.

Without the label DB you still get the abstract `Chan#0_DIMM#0` location — usable, but an operator can’t act on it without a translation sheet. Building the label map per revision is the difference between a test that says “memory error somewhere” and one that says “pull the stick in slot A1.” Wire `ras-mc-ctl` into the test and **log the label on every error**, pass or fail.

7.6 RAS concepts: CE vs UE, predictive failure, PPR, row-hammer

The error classes and what each means for a verdict:

- **CE (Corrected Error):** a single-bit flip the ECC fixed. The data was *correct* — nothing crashed. A handful over a long soak can be cosmic-ray noise; the signal is **rate** and **concentration** (many on one DIMM), not the existence of one CE.
- **UE (Uncorrectable Error):** ECC detected an error it could not fix (double-bit, or a failure beyond the code’s strength). The data is *wrong*; the consequence is a machine-check — typically a kernel panic or an application crash. **Any UE on a new unit fails it.**

The Reliability, Availability, Serviceability (RAS) features built to manage these:

- **Predictive Failure Analysis (PFA):** rather than wait for a UE, the platform watches the **CE rate per DIMM/row** and flags a DIMM as *predicted-to-fail* when its CE rate crosses a threshold — the assumption being that a cell throwing rising single-bit errors will eventually throw an uncorrectable one. In the field this triggers a proactive RMA before a crash; in manufacturing it is *why* per-DIMM CE concentration is a gate, not just total CE.

- **Post-Package Repair (PPR):** DDR4/DDR5 DRAM ships with **spare rows** inside each device. When a row goes bad, the controller/BIOS can **remap a failing row to a spare** — **soft PPR** (volatile, until next boot) or **hard PPR** (a permanent, one-time fuse blow). This is the DRAM analog of Non-Volatile Memory Express (NVMe) spare blocks or GPU row-remapping. Test relevance: a board that has **already consumed PPR resources** on a “new” DIMM, or that *needs* PPR to pass, is a marginal part — log the PPR/repair state, don’t just let BIOS quietly repair around a defect and ship it.
- **Row-hammer awareness:** repeatedly activating one DRAM row at high rate can disturb charge in **adjacent** rows and flip their bits — a reliability and security concern. Mitigations (TRR / Target Row Refresh, RFM / Refresh Management in DDR5, increased refresh) live in the DRAM and controller. You usually do not row-hammer test on the line, but know that a cluster of CEs in physically adjacent rows can be a row-hammer signature rather than a single weak cell, and that disabling refresh-management mitigations to “speed up” a test is a mistake.

7.7 Stress and soak: stressapptest and memtester

The stress *provokes* errors; EDAC *counts* them. The two tools, and when to use each:

- **stressapptest** (Google’s “stressful application test”) — the manufacturing favorite, because it runs **under Linux**, inside your normal pytest/test environment. It hammers memory **bandwidth and patterns** using many threads, and it detects errors **two ways**: by **miscompare** (it writes known data, reads it back, and compares — catching errors ECC might mask *and* errors on non-ECC paths) and via the system’s ECC/EDAC counters. It also exercises some I/O and cache coherency. Typical soak invocation:

```
stressapptest -s 120 -M 28000 -W # 120 s, use ~28 GB, CPU-stressful copy routine (-W)
stressapptest -s 600 -W # 10 min soak, auto-size memory, CPU-stressful copy (-W)
```

-M caps the memory footprint (MB) so you don’t OOM the test station; omit it to let it auto-size to most of free RAM. **-W** switches to a more CPU-stressful copy routine (vector/FP); **-m N** sets the number of copy threads (default one per CPU). **-s** is the soak duration. The toolkit’s **stress_memory(seconds, mb)** wraps exactly this: `["stressapptest", "-s", str(seconds), "-W"]` plus **-M** if a footprint is given.

- **memtester** — a lighter userspace tool that mmaps a buffer and walks it through many pattern tests (walking ones/zeros, checkerboard, address-in-address, etc.). Good for a quick targeted pattern sweep of a *region*; weaker than **stressapptest** for whole-system bandwidth stress and multi-threaded coherency. **memtester 4G 3** tests 4 GB for 3 passes.
- **memtest86+** (awareness) — boots **instead of** the OS and walks the *full* address space with the most thorough patterns. Highest coverage of pattern-sensitive defects, but it is **offline** (cannot run inside your pytest harness) and slow — reserve it for a debug bring-up step or a fielded-failure deep dive, not volume line test.

Interpreting the results. A clean run is **stressapptest** exiting 0 **and** zero new UE **and** total CE under your limit **and** no single-DIMM CE concentration. The tail of a clean run ends with the status line you gate the exit on:

```
Log: Seconds remaining: 0
Stats: Found 0 hardware incidents
Stats: Completed: 1835008.00M in 600.02s 3058.25MB/s, with 0 hardware incidents, 0 errors
Status: PASS - please verify no corrected errors
```

A **stressapptest miscompare** is a hard fail — it means data read back wrong, which on an ECC system implies the error exceeded ECC’s correction strength (effectively a UE) or hit a path ECC doesn’t cover. The failure line names the address, the expected vs actual bits, and the worker thread:

```
Hardware Error: miscompare on CPU 5(0x2) at 0x7f3a1c004000(0x...): read:0x0000000000000000,
↳ reread:0xfffffffffffffff expected:0x0000000000000000
Hardware Error: miscompare on CPU 5 ... ECC? (re-read matched expected -> transient/marginal)
Status: FAIL - 1 hardware incidents
```

The re-read tells you something: if the **first** read was wrong and the **re-read** matched expected, the bit flipped and self-corrected — a transient/marginal cell or a soft error, still a fail on a new unit but more “marginal DIMM” than “stuck cell.” A miscompare whose re-read *also* reads wrong is a hard/stuck fault. Always pair the stress run with a **before/after EDAC read**: the tool provokes, EDAC attributes the error to a DIMM. The tool says “something is wrong”; EDAC + the label DB say “DIMM_A1 is wrong.” Note that on an ECC system, a single-bit flip stressapptest provokes is usually *corrected before stressapptest ever sees it* — it shows up as a **CE in EDAC**, not a stressapptest miscompare. So the two detectors are complementary: EDAC catches the corrected single-bit events, the miscompare catches what got past ECC. That is exactly why the gate reads **both**.

7.8 Temperature and voltage dependence — run it hot

DRAM marginality is **strongly temperature- and voltage-dependent**, and this is the single most important operational fact in this chapter. The physics: hotter cells leak charge faster, so a marginal cell that holds its value at 25 C loses it before the next refresh at 70 C; voltage droop (a sagging VDD/VDDQ rail under load) shrinks the noise margin the same way. The consequence for test:

- **A room-temperature memory test is a weak test.** The classic escape is **room-temp pass / hot fail** — a DIMM that is clean on the bench and throws CEs (or UEs) at operating temperature in the vehicle. So you soak **at temperature** (thermal chamber, or under a load that self-heats the platform) and/or **at worst-case voltage** if the platform lets you margin the rail.
- **Correlate the rail.** If CEs appear under load, measure VDDQ with a DMM/scope at the same moment — a CE burst coincident with a rail droop is a **power-delivery** finding (a weak VRM/PMIC), not a bad DIMM. This is the same “is it the part or the support circuitry?” discipline as the NVMe thermal-throttle-vs-bad-drive split.
- **DDR5 ties this together:** higher data rates (tighter margins) plus on-die ECC (which hides early degradation) means the hot soak is doing *more* of the catching on DDR5 than it did on DDR4. Lean on it.

7.9 Failure signatures and RMA decisions

Symptom	Most likely cause	Decision / first moves
Any UE (ue_count > 0)	A real uncorrectable memory fault	Hard fail. Find the DIMM via ras-mc-ctl --errors ; RMA that stick
CEs concentrated on one DIMM	That DIMM/rank is marginal (PFA signal)	Fail / RMA the named DIMM even if the <i>total</i> is under budget
CEs spread evenly , low rate, no concentration	Possibly cosmic-ray noise / benign	Pass if under the total CE limit; log for fleet trend
CEs only under load / heat	Marginal cell at temperature, or rail droop	Run hot; measure VDDQ during the burst — droop = power, not DIMM
stressapptest miscompare	Data read back wrong (beyond ECC, or non-ECC path)	Hard fail; treat as effectively a UE; capture which address
UE escalating to MCE / kernel panic mid-soak	Severe uncorrectable fault	Hard fail; the panic log + ras-mc-ctl identify the DIMM
No mc* nodes in EDAC sysfs at all	EDAC driver not loaded (silent escape)	Fix the test environment — load the chipset EDAC driver; a “clean” no-data result is invalid

Symptom	Most likely cause	Decision / first moves
Board needed/consumed PPR to pass on a new DIMM	Marginal DIMM that BIOS repaired around	Log PPR state; flag as marginal — don't ship a part that needed repair to pass
Cluster of CEs in adjacent rows	Possible row-hammer signature	Note it; verify refresh-management mitigations are enabled

The RMA discipline: memory failures are *componentized* — you do not RMA “the board,” you RMA the **DIMM** (or, soldered-down, you flag the specific device location for the failure- analysis lab). The whole reason for the label DB and `rasdaemon` is to make that decision crisp: a failure report that says “*DIMM_B1, 3 UE during 10-min soak at 70 C, syndrome attached*” is a closed-loop RMA; “*board has memory errors*” is a week of back-and-forth.

7.10 Manufacturing-test limits and the gate

The memory gate at module test, stated as limits — and exactly what the toolkit's `_limits()` enforces:

Check	Limit	Why
<code>no_uncorrectable</code>	<code>total_ue == 0</code>	Any UE = fail. Non-negotiable on a new unit
<code>ce_total <= max_ce_total</code>	bounded total CE (default 100)	A small CE count over a soak can be benign; a flood is a finding
<code>no_ce_concentration</code>	worst DIMM <= <code>max_ce_per_dimm</code> (default 20)	A hot DIMM even within the total budget is suspect (PFA) — concentration localizes a marginal stick

The third check is the subtle, high-leverage one: **total CE under budget is not enough**. A board with 100 CE spread across 8 DIMMs is plausibly benign noise; a board with 100 CE *all on DIMM_A1* is a marginal DIMM that happens to fit under the total — and PFA says it will fail in the field. So you gate on **both** total CE and per-DIMM concentration. The toolkit:

```
# computetest.memory._limits()
def _limits(total_ce, total_ue, per_dimm, max_ce_total, max_ce_per_dimm):
    worst = max(per_dimm.values()) if per_dimm else 0
    return {
        "no_uncorrectable": total_ue == 0,
        f"ce_total<={max_ce_total}": total_ce <= max_ce_total,
        f"no_ce_concentration(<={max_ce_per_dimm}/dimm)": worst <= max_ce_per_dimm,
    }
```

Set the limit from fleet data, not a guess. The 100-total / 20-per-DIMM defaults are starting points. The right values come from **Design Verification (DV) characterization** (soak good units across temperature, see what the healthy CE distribution actually is) and then from **production fleet data** (the CE distribution of known-good shipped units). Capture the per-DIMM CE counts on **every** unit — pass or fail — so the limit can be tightened to the real population, and so a *lot-wide* shift in CE rate (a bad DRAM batch) shows up as a trend before it shows up as field returns. This is the same “capture the parameter, not just the verdict” principle the rest of the toolkit runs on.

The full memory flow at module test, in order:

1. **Verify EDAC is alive.** `ls /sys/devices/system/edac/mc/` shows `mc0` (etc.) — the chipset EDAC driver is loaded. No nodes = invalid test, fix the environment first.

2. **Confirm topology.** `ras-mc-ctl --layout / dmidecode --type memory` — the right number, size, and speed of DIMMs are present (a missing or down-clocked DIMM is its own defect).
3. **Baseline the counters.** Read CE/UE per controller and per DIMM (or reset where allowed) so you measure the *delta* across the soak, not boot-time noise.
4. **Soak hot.** `stressapptest -s <soak> -W` (most of RAM, CPU-stressful copy) at temperature — long enough and hot enough to provoke marginal cells. This is the catching step.
5. **Read the counters.** Delta CE/UE total and per-DIMM; `ras-mc-ctl --errors` for any error detail and the DIMM label.
6. **Apply the gate.** `total_ue == 0, total_ce <= limit`, no single-DIMM concentration; a `stressapptest` miscompare is a hard fail regardless.
7. **Log & decide.** Capture per-DIMM CE counts and any error detail (with label) into the test record. On fail, the named DIMM is the RMA; on pass, the numbers feed the fleet trend.

For step 2, `dmidecode --type 17` (DMI type 17 = Memory Device) is the topology gate — it tells you populated slots, size, configured speed, and part number per slot, which is how you catch a DIMM that is missing, undersized, or running below its rated speed (a down-clock is its own defect: a single slow stick can drag the whole channel to the lowest common speed):

```
Memory Device
Locator: DIMM_A1
Size: 32 GB
Type: DDR5
Speed: 4800 MT/s <- rated speed of the part
Configured Memory Speed: 4800 MT/s <- what it is actually clocked at (gate on this)
Manufacturer: <vendor>
Part Number: <pn>
Rank: 2
```

Gate on **Configured Memory Speed** matching the expected rate (not just the rated **Speed**), on the populated-slot count and size matching the BOM, and on the part number matching the qualified part — a substituted or down-binned DIMM is a supply-chain finding the same way a re-stock NVMe drive is. The **Locator** (`DIMM_A1`) is the same silkscreen string you want the EDAC label DB to reproduce, so cross-checking the two also validates your label map.

7.11 Memory health check in Python (toolkit cross-reference)

```
from computetest.memory import check_memory, stress_memory

# Hot soak first (provokes), then read EDAC (counts):
stress_memory(seconds=600, mb=28000) # stressapptest -s 600 -M 28000 -W
h = check_memory(max_ce_total=100, max_ce_per_dimm=20)

print(h.summary())
# memory: 2 mc, CE=4 UE=0 worst=DIMM_A1(2) -> OK

if not h.ok:
    failed = [k for k, ok in h.checks.items() if not ok] # e.g. ['no_uncorrectable']
    # worst_dimm names the stick to pull:
    print("RMA target:", h.worst_dimm) # e.g. 'DIMM_B1'

for note in h.history: # actionable per-DIMM notes, logged even on pass
    log.info(note) # e.g. "DIMM_A1 has 2 CE (swap that stick)"
```

The `MemoryHealth` dataclass mirrors the NVMe one: `checks` (the three limits) drive pass/fail via `.ok`, `per_dimm` carries the localization, `worst_dimm` names the RMA target, and `history` records the actionable “swap that stick” note **even when the unit passes** — so the per-DIMM signal feeds fleet trending and a marginal-

but-passing DIMM is still visible to Quality. Same pattern as everywhere in this toolkit: arm/stress/read, capture the number, compare to a limit, and hand a failure to whoever fixes it with the part already named.

Chapter 8

Automotive and Serial Buses: GMSL, CAN, Ethernet, I2C/SPI/UART

Four families of bus carry everything on a Zoox compute board that *isn't* PCIe. **Gigabit Multimedia Serial Link (GMSL)** brings camera pixels in from the harness. **Automotive Ethernet** brings radar, lidar, and inter-module traffic. **Controller Area Network (CAN)/Controller Area Network Flexible Data-Rate (CAN-FD)** is the vehicle's control nervous system — brakes, steering, power distribution. And **I2C/SPI/UART** are the housekeeping buses that configure, identify, and monitor every die on the board. The first three are high-speed serial links and share a single mental model with PCIe (the Networking and PCIe chapters); the last three are low-speed but are where the *reason* for a high-speed failure usually turns up.

This chapter goes deepest on **GMSL**, because on a robotaxi the camera path is both the highest-channel-count interface you own and the one whose failures are most likely to surface late — on the vehicle, over 15 m of coax, at temperature — where they cost the most to catch. Everything else here is real and you'll use it daily, but GMSL is where a Compute Test Engineer earns the title.

The one mental model. PCIe, GMSL, automotive Ethernet, NVLink, USB, SATA — all the same kind of thing: a serial differential SerDes link that recovers a clock from the data (CDR), encodes/scrambles for DC balance, equalizes to fight channel loss (TX emphasis + RX Continuous-Time Linear Equalizer (CTLE)/Decision Feedback Equalizer (DFE)), *trains* both ends up together, and *counts errors*. The failure physics are identical — channel loss, jitter, Inter-Symbol Interference (ISI), reflections, temperature, connector/cable quality. Learn the model once; what changes per link is the **vocabulary and the tooling**. So your debugging instinct is always the same: *is it trained at the right rate, stress it, watch the error counters, suspect the channel and temperature first.*

8.1 GMSL — Cameras Over Coax

8.1.1 What GMSL is and why a robotaxi lives on it

GMSL is a SerDes designed to move **uncompressed** sensor video from a remote camera to a host System-on-Chip (SoC) over a single inexpensive cable, while *simultaneously* carrying bidirectional control, power, and synchronization on that same cable. That “everything on one coax” property is the whole point: a robotaxi has dozens of cameras scattered around the body, each up to ~15 m of harness away from the compute box, and you cannot afford a fat multi-conductor cable, a separate power run, a separate I2C run, and a separate sync wire to every one of them.

The end-to-end path you are protecting:

```

Image sensor (e.g. ON Semi AR0820)
| MIPI CSI-2 (short board trace, camera module internal)
v
GMSL SERIALIZER (MAX9295A / MAX96717 / MAX96717F)
| ===== single 50 Ohm coax (or 100 Ohm STP) =====
| forward: 3 or 6 Gbps uncompressed video + embedded data ----->
| reverse: 187.5 Mbps control (I2C/UART), GPIO, frame-sync <-----
| PoC: DC power rides the same conductor <-----
v
GMSL DESERIALIZER (MAX9296A dual / MAX96712 quad / MAX96724 quad)
| MIPI CSI-2 D-PHY or C-PHY (multiple virtual channels)
v
Compute SoC capture / VI / ISP ----PCIe----> GPU(s), NVMe, NIC

```

Read it as: **sensor** → **serializer** → **coax** → **deserializer** → **CSI-2** → **SoC**. That chain is the unit under test. A frame that lands in `/dev/video0` proves *all* of it works — sensor power and config, the forward link, the reverse control channel that programmed the sensor, the coax, the deserializer, the CSI-2 output, and the SoC’s capture block. That is why a real frame capture is the gold end-to-end check and not just a “lock” bit.

8.1.2 GMSL1 vs GMSL2 (vs GMSL3) — the generations you’ll meet

	GMSL1	GMSL2	GMSL3
Forward rate	up to ~3.125 Gbps	3 or 6 Gbps	3 / 6 / 12 Gbps
Reverse rate	~1 Mbps (slower control)	187.5 Mbps (1.5 Gbps option)	187.5 Mbps
Signaling	NRZ	NRZ	NRZ (<=6G), PAM4 at 12G
Forward EQ	fixed/limited	continuous adaptive	continuous adaptive
Control tunnel	basic I2C	I2C + UART, GPIO tunneling	same + more
Typical AV part	MAX9271/MAX9286	MAX9295/MAX96717 + MAX9296/MAX96712	MAX96793 + MAX96792A

(Rates: ADI GMSL1/GMSL2 channel-spec user guides and the GMSL Wikipedia summary — GMSL1 downlink up to 3.125 Gbps; GMSL2 forward 3 or 6 Gbps, reverse 187.5 Mbps with a 1.5 Gbps reverse option on some parts; GMSL3 forward 12 Gbps using Pulse Amplitude Modulation 4-level (PAM4) above 6 Gbps.)

The forward rate is **fixed/selectable**, not auto-negotiated — set by CFG-strap resistors at power-on or by register writes. This is a key difference from Ethernet: there is no rate-fallback negotiation, so a GMSL2 link either locks at the configured rate or it doesn’t lock at all. The generations are backward compatible (a GMSL2 deserializer can run a GMSL1 serializer in GMSL1 mode), and a deserializer like the MAX9296A can even run **mixed** GMSL1 and GMSL2 links on its two inputs — which matters because the lock register and error counters live at *different addresses* depending on which mode a given link came up in (more below).

Zoos-relevant silicon, the parts you will actually probe over I2C:

Part	Role	Capability
MAX9295A / MAX9295D	Serializer (on camera)	GMSL2/1, single/dual CSI-2 in; 3/6 Gbps fwd, 187.5 Mbps rev
MAX96717 / 96717F	Serializer	GMSL2, CSI-2 in; F = ISO 26262 functional-safety variant

Part	Role	Capability
MAX96793	Serializer	GMSL3/2, CSI-2 in; 3/6/12 Gbps fwd
MAX9296A	Dual deserializer	2x GMSL2/1 -> CSI-2; pairs with MAX9295/96717
MAX96712	Quad deserializer	4x GMSL2/1 -> CSI-2; coax or STP; the AV workhorse
MAX96724	Quad deserializer	4x GMSL2/1 -> CSI-2, tunneling focus
MAX96792A	Dual deserializer	GMSL3/2 -> CSI-2

The “F”/“R” suffixes matter on a safety vehicle: the **F** variants add functional-safety features (CRC/Error-Correcting Code (ECC) on internal memories, register-readback verification, lock-step diagnostics, a dedicated error pin) so the SerDes can participate in the Automotive Safety Integrity Level (ASIL) chain. If Electrical Engineering (EE) specs a 96717F/96724F, your test must read the safety status registers too, not just lock.

8.1.3 The SerDes model and link establishment

The serializer takes a parallel CSI-2 stream and **serializes** it onto the differential forward channel; the deserializer recovers the clock from the data stream (no separate clock wire), **deserializes** it back to CSI-2, and drives the SoC. Two facts drive everything you do with GMSL2:

1. **The deserializer initiates and owns link training.** At power-on the deserializer drives the link and the serializer responds; the handshake is automatic and needs *no software*. This is why a GMSL link can lock before your test program has even run — and why “lock” alone is necessary but not sufficient (you still have to configure the sensor over the now- locked reverse channel before pixels flow). Lock typically establishes in **5–50 ms** depending on cable length and electrical conditions.
2. **GMSL2 runs *continuous adaptive equalization*.** At 3/6 Gbps over many meters of coax the channel badly attenuates the high-frequency content — the raw eye is closed. The deserializer’s receiver continuously adapts its equalizer (CTLE + decision-feedback equalization, DFE — the receiver cancels inter-symbol interference using its own recent bit decisions) and **re-optimizes roughly once per second** to track temperature drift, cable aging, and connector wear. It also runs an **eye-opening monitor**: a built-in margin measurement with programmable alarm thresholds that fires a run-time alert when the link eye degrades *before* it actually loses lock.

Why the eye monitor is a high-leverage test lever. It is the GMSL analog of PCIe lane margining — an on-die eye measurement with no scope. A link that *locks* but whose eye monitor sits near its alarm threshold is a marginal unit that will drop at temperature in the vehicle. Reading the eye-monitor margin (not just the lock bit) turns “it locked” into “it locked with X margin” — a captured *parameter* you can set a data-driven limit on, exactly the Design Verification (DV)-sets-the-limit / Manufacturing Test (MT)-checks-it pattern from the test-strategy chapter.

A *compliant GMSL2 channel* is specified to deliver a **Bit Error Rate (BER) of 1e-15 or better** under worst-case conditions (longest cable, aged cable, temperature extremes, PCB impedance variation, min/max PoC load). That number is your acceptance bar: GMSL is essentially an error-free pipe when healthy, so any nonzero decode/CRC error count on a soak is a finding, not noise.

Two ways an eye/channel check fakes a pass — both real audit finds. (1) If the eye-opening-monitor verdict compares against a *fixed default* threshold instead of the **threshold you configured**, a tightened limit doesn’t actually gate — the check reports PASS at a margin you meant to fail. Make the verdict read the per-link threshold you set. (2) Channel-compliance against the S-parameter masks (insertion/return loss, e.g. ADI’s AN-2585) is meaningless without the *real* mask numbers; if you don’t have them, the check must **raise / refuse to pass**, never

quietly pass against a placeholder mask. A green “channel compliant” with no mask behind it is the worst kind of result — confidently wrong, and it ships a marginal camera link. “I don’t have the limit yet” is an honest skip; a faked pass is a latent field failure.

8.1.4 Forward and reverse control channels

GMSL is **full-duplex on one conductor**. The two directions are:

- **Forward channel** (3/6 Gbps): serializer → deserializer. Carries the video plus embedded data (statistics lines, the sensor’s embedded metadata rows).
- **Reverse channel** (187.5 Mbps): deserializer → serializer. Carries *control*: the I2C/UART tunnel, GPIO state, frame-sync triggers, and link-management traffic.

The reverse channel is what makes GMSL more than a video pipe. The image sensor and the serializer sit at the *far* end of 15 m of coax, but the SoC must configure them — set exposure, gain, the output resolution/format, enable the sensor’s streaming, arm frame-sync. GMSL solves this by **tunneling I2C** (and optionally UART) over the reverse channel so the SoC’s *local* I2C controller can read and write registers in the remote serializer and sensor **as if they were on the local bus**. The deserializer is the local I2C device; it forwards each transaction up the reverse channel to the serializer, which replays it on the camera-module-local I2C bus.

```
SoC I2C controller
| local I2C (e.g. /dev/i2c-1)
v
DESERIALIZER (e.g. 0x48 or 0x29) <-- you i2cdetect this locally
| reverse channel over coax (I2C tunnel)
v
SERIALIZER (e.g. 0x40) <-- appears as a *translated* local address
| camera-module-local I2C
v
IMAGE SENSOR (e.g. 0x10) <-- also appears at a translated local address
```

Two consequences own a big slice of your camera debugging:

- **A camera that “won’t configure” is often a reverse-channel problem, not a sensor problem.** If the forward link is locked but I2C writes to the sensor fail (NACK / -ENXIO in `dmesg`), suspect the reverse channel: serializer not locked in the reverse direction, I2C tunnel not enabled, or an address-translation mistake — before you ever suspect the sensor itself.
- **I2C address translation lets identical cameras share one bus.** Four identical sensors all ship with the *same* hardwired I2C address (say 0x10). A quad deserializer performs **address translation** so each remote sensor and serializer appears at a *distinct* address on the local bus. When you `i2cdetect` a quad-camera carrier you see the deserializer plus four translated serializer addresses and four translated sensor addresses — not four devices colliding at 0x10.

8.1.5 GPIO tunneling and the I2C/UART control tunnel

Beyond register access, the reverse (and forward) channel can **tunnel GPIO**: a logic level on a pin at one end is reproduced on a mapped pin at the other end, with no separate wire. On the Maxim parts these are the **MFP (multi-function pin) / GPIO** lines. You map, e.g., deserializer GPIO_x → serializer GPIO_y, and a level or pulse driven into the deserializer pin appears at the serializer pin across the coax. This is how:

- **Frame-sync** triggers reach the sensor (the host’s FSYNC pulse is tunneled to each camera’s trigger input — see below).
- A camera’s **error/interrupt** line is brought back to the SoC (sensor fault → serializer GPIO → tunneled → deserializer GPIO → SoC interrupt).
- **Reset / power-enable** of the remote module can be driven from the host side.

The **UART tunnel** is the same idea for a byte stream — useful when a camera module has its own MCU

that speaks UART rather than (or in addition to) I2C. For test you mostly care that the **I2C tunnel** is alive (you can read a known sensor ID register through it) and that the **GPIO/frame-sync tunnel** is alive (the cameras actually fire together).

8.1.6 Video transport: tunnel mode vs pixel mode, data types, virtual channels

The deserializer reconstructs a MIPI Alliance (MIPI) **CSI-2** stream for the SoC, and how the video crosses the GMSL link is configured in one of two modes — a real EE/firmware decision you must understand to debug a “frames are corrupt / wrong format” failure:

- **Pixel mode** (the original GMSL2 mode). The serializer *strips* the CSI-2 packet header and footer, converts the payload to GMSL’s internal **pixel** representation, sends that, and the deserializer rebuilds a fresh CSI-2 packet (new header/footer) on the far side. It understands the data type — so it can do bits-per-pixel packing, **watermarking**, and per-stream manipulation. Supported types: RAW8/10/12/14/16/20, RGB565/666/888, YUV422 8/10-bit, embedded (EMB8), user-defined, generic long-packet.
- **Tunnel mode** (a.k.a. CSI-2 forwarding). The serializer re-packetizes the *entire* CSI-2 structure and forwards it verbatim; the deserializer emits it unchanged. It is data-type agnostic (*any* CSI-2 type, including ones the pixel-mode logic doesn’t model) and lower latency, but does no per-pixel processing. The MAX96724 is explicitly the “tunneling” variant.

Why this matters in a corrupt-frame debug. A format mismatch — sensor outputs RAW12 but the pipe is configured for RAW10, or pixel-mode bits-per-pixel set wrong — produces a frame that *captures* (lock is fine, frames > 0) but looks sheared, miscolored, or wrong size. That is a **configuration** bug in the video-pipe/data-type setup, not a link or cable fault. Lock + frames-captured + *correct resolution and format* must all be checked together; the toolkit’s **resolution_ok** check exists precisely so a wrong-format capture doesn’t pass as good.

Video pipes, streams, and virtual channels. Inside the link, video moves in **pipes**; each pipe carries one or more **streams**, and each stream is tagged with a CSI-2 **virtual channel (VC)** and data type. A quad deserializer aggregates up to four cameras’ streams and multiplexes them onto its CSI-2 output, assigning each a distinct VC (the MAX96714 family supports up to **16 virtual channels**; a dual-4-lane CSI-2 deserializer like the MAX9296 can decode up to 16 VC IDs). On the SoC side, each VC typically lands as its own `/dev/videoN`. This is the mechanism behind “one deserializer, four cameras, four video nodes”:

```
cam0 --GMSL link0--> pipe Z --VC0--+
cam1 --GMSL link1--> pipe Y --VC1---> CSI-2 (D-PHY/C-PHY) --> SoC --> /dev/video0..3
cam2 --GMSL link2--> pipe X --VC2--+
cam3 --GMSL link3--> pipe W --VC3---+
```

When you debug “camera 3 is missing,” the question is whether **link3** failed to lock, whether its **stream/VC mapping** is wrong (locked but routed to the wrong VC or dropped at the pipe), or whether the SoC’s CSI-2 receiver isn’t configured for that VC. Three different root causes, three different fixes.

8.1.7 FrameSync — multi-camera shutter alignment

For sensor fusion, all the cameras in a cluster must expose **at the same instant** — otherwise a moving object lands at inconsistent positions across cameras and perception mis-fuses it. Software timestamp matching is far too jittery; this has to be **hardware** frame sync, and GMSL provides it through GPIO tunneling:

```
Host FSYNC source (SoC GPIO timer, or deserializer's internal generator)
| drives the deserializer FSYNC / GPIO pin at the exact frame rate (e.g. 30 Hz)
v
DESERIALIZER -- broadcasts the FSYNC pulse over each link's reverse/control channel -->
v
each SERIALIZER -- drives its tunneled GPIO to the sensor's trigger/FSYNC input -->
v
every sensor exposes on the same edge ==> frames are shutter-aligned
```

The FSYNC master can be the **deserializer’s own internal frame-sync generator** (free- running at a programmed rate) or an **external** source (a SoC GPIO timer, or a vehicle-wide clock for cross-cluster alignment). The key point for test: a single pulse is fanned out over the *control channel* to every camera — there is no separate sync wire per camera. So frame-sync depends on (a) every link being locked and (b) the GPIO/frame-sync tunnel being configured on every link.

The failure mode that a single-camera test misses entirely. Every camera can lock, enumerate, and stream perfect individual frames — and still be **out of frame-sync**, if the FSYNC pulse isn’t reaching one camera (its GPIO-tunnel mapping is wrong, or that link’s reverse channel is marginal). Each camera looks healthy in isolation; only when you check *cross-camera alignment* do you catch it. This is exactly the **all-links-locked-but-not-synchronized** case, and it is why the toolkit models frame-sync as a property *separate from* per-link lock — see `check_deserializer` below.

8.1.8 Coax vs STP cabling and Power-over-Coax (PoC)

GMSL runs over either **50 Ω coax** or **100 Ω shielded twisted pair (STP)**. Coax has lower insertion loss per meter and can support runs up to ~50% longer for the same link margin, so robotaxi camera harnesses are typically coax. STP shows up where routing or weight favors it. The cable choice changes the channel-loss budget and the termination, both of which your hardware team designs — your job is to know which one a given camera uses so you can read its cable diagnostics correctly.

Power-over-Coax (PoC) runs the camera’s DC power up the *same* conductor that carries the GHz video. A **PoC filter network** (series ferrite/inductor + shunt caps, a bias-tee) on each end separates the DC power band from the high-frequency signal band so they coexist without the inductor loading the signal or the signal coupling into the power rail. This is elegant — one cable does power, video, control, and sync — but it concentrates failure modes onto one connector:

- **A bad coax connector (cold solder joint, not fully clicked in, corrosion) kills power *and* signal** at once: the camera looks completely dead — no PoC, no lock, no I2C. Don’t chase the sensor; check the connector and the PoC rail first.
- **A PoC filter problem** (open inductor, shorted cap, wrong-value part) can drop the remote power, or — more insidiously — pass enough DC to power the camera but couple noise into the signal band and *degrade the eye* (rising decode errors, intermittent lock at temperature).
- **PoC line-fault detection.** GMSL parts include **line-fault detection** — an on-chip multilevel comparator that classifies the cable condition as **normal, open (disconnected), short-to-ground, or short-to-battery** (ADI design note *How to Use GMSL Line-Fault Detection for Power Over Coax*). Reading that status localizes a cabling fault without a scope. Two caveats worth knowing: it needs a **dedicated line-fault pin/divider circuit**, and on many parts it **cannot run simultaneously with PoC on the simple bias-tee** — the datasheet specifies an *alternate* PoC+line-fault filter topology if you want both. So whether your board exposes line-fault at all is an EE schematic question; confirm it before a test step depends on it.

The PoC mental checklist. Camera totally dead → PoC/connector (power gone). Camera powers but never locks → PoC noise into signal band, or link-rate/mode mismatch, or reverse-channel dead. Camera locks but decode errors climb → marginal signal eye (cable loss, PoC noise, temperature). The *symptom* of “dead vs no-lock vs errors” already points at the layer.

8.1.9 Link-lock and frame-sync status registers — the bits that matter

These are the registers your driver/sysfs/I2C reads resolve to, and knowing them lets you diagnose by hand when sysfs is missing or stale. (Exact addresses are part-specific; these are the canonical Maxim/ADI ones.)

What	Where (MAX9296/96712 family)	Meaning
GMSL2 link lock	reg 0x0013, bit 3 (LOCKED)	1 = PLLs locked and the forward receive datapath is operational
GMSL1 link lock	a <i>separate</i> , mode-specific lock reg (see below)	1 = locked when the link came up in GMSL1 mode (different reg from the GMSL2 bit)
LOCK pin	open-drain output pin	hardware mirror of lock; high = locked. A board-level “is it up” you can scope
Decode / line-CRC errors	per-link error-count registers	accumulate on a marginal channel; the GMSL analog of PCIe AER correctable
Video-pipe / packet status	pipe status regs	per-pipe “video detected”, overflow, line-length errors
PoC / line-fault status	line-fault detect regs	open/short/short-to-battery on the cable (alternate circuit; see PoC section)
(F-parts) safety status	dedicated error/CRC regs + ERRB pin	memory CRC/ECC, register-readback mismatch, lock-step fault

The GMSL2 lock bit 0x0013[3] (LOCKED) is well-documented and is what the Jetson/ADI flows read (e.g. `i2cget -y <bus> 0x48 0x0013, mask 0x08`). The single most important *subtlety* is that **lock lives at a different register depending on the mode the link negotiated**. A link that came up in GMSL2 reports at 0x0013[3]; a link that came up in GMSL1 (because the serializer is a GMSL1 part, or a mode mismatch forced it) reports through a *different*, *GMSL1-mode* lock register — not the GMSL2 bit. If your check reads only the GMSL2 lock bit and the link is actually in GMSL1, you wrongly report “no lock.” This is a classic mixed-fleet bug on a deserializer that supports both.

Confirm the GMSL1 lock address against your exact part’s datasheet — do not hard-code it from memory. The GMSL1-mode lock register is *not* publicly documented as a single portable address across the GMSL2/1 deserializer family the way 0x0013[3] is, and it differs by part. Concrete anchor: a pure GMSL1 deserializer like the **MAX9286** reports “all enabled links locked” at reg **0x27 bit 7** (MAX9286_LOCKED in the mainline Linux `max9286.c` driver); a GMSL2/1 part such as the MAX9296/MAX96712 exposes its own GMSL1-mode lock status that you must read out of *that* device’s register map (or, more safely, via the vendor driver’s `sysfs link_status`, which abstracts the mode). The robust rule for test code: prefer the driver’s mode-agnostic lock attribute; only fall to raw I2C when you’ve pulled the exact address+bit for the exact silicon from its datasheet.

(Lock is fundamentally the detection of valid **sync words** on the serial stream; the Phase-Locked Loop (PLL)-lock + sync-word condition is what sets the bit and drives the LOCK pin.)

The **LOCK pin gotcha** worth knowing from the field: if the LOCK output reads asserted while the deserializer is *not even connected* to a serializer, the device is mispowered or damaged (or stuck in a board-specific reset/programming mode) — a high LOCK with no link partner is not “good,” it’s “broken.”

What the error counters are actually counting is worth knowing so you read them right. GMSL2 protects its traffic with multiple CRCs: each **control packet** carries a 4-bit sequence number and a 16-bit CRC, and the link checks (and strips) the **CSI-2 packet ECC** on the header and the **CSI-2 CRC** on the footer as it converts to/from pixel form. So the per-link “decode error” / line-CRC counters increment on *physical-layer* decode failures and CRC mismatches — the marginal-channel symptom. (End-to-end **Video Line CRC**, VID_PXL_CRC, is a *GMSL3* addition; on GMSL2 your end-to-end pixel-integrity proof is the captured frame plus the sensor’s own embedded-stats CRC, not a single link counter.) Net for test: a nonzero decode/CRC counter is a channel finding; for true pixel-payload integrity you still rely on the frame capture and any sensor-side CRC, especially on GMSL2.

8.1.10 Multi-camera deserializer topologies

A quad deserializer (MAX96712) is the AV building block: four coax inputs, four cameras, one CSI-2 output (often split across two D-PHY/C-PHY ports) to the SoC. Common topologies you'll test:

```
(A) Quad deser, 4 independent cameras (the common AV cluster)
cam0..3 --coax--> MAX96712 --CSI-2(2x4-lane)--> SoC (4 video nodes, frame-synced)

(B) Aggregation / GMSL switch board (a "custom PCIe device" at Zoox)
many cameras --coax--> [multiple quad desers] --PCIe--> SoC
(a sensor-interface card; each deser is its own I2C device on the carrier)

(C) Daisy / line-fanout variants
sensor clusters share trunk runs; address translation keeps identical sensors distinct
```

The test implication is constant: **you enumerate and verify each link, not “the deserializer.”** A quad part with three good links and one dead camera is a *failing* unit, and a per-deserializer “is it there” check would pass it. Your check must be per-link (lock + video node + frames + errors for *each* of the four) *and* cross-link (frame-sync across all four). That two-level structure is exactly how the toolkit models it.

8.1.11 Toolkit cross-reference: gmsl.py

The toolkit's `gmsl.py` (`toolkit/src/computetest/gmsl.py`) implements this model in two layers, with a real-hardware path (`sysfs lock/error + v4l2-ctl`) and a mock path for laptop/CI demos.

- `check_gmsl(link, video_device, ...)` → `GmslHealth` checks *one* camera link. It reads link lock from `/sys/bus/i2c/devices/<link>/link_status`, the error count from `.../error_count`, then uses `v4l2-ctl --get-fmt-video` to confirm resolution and `v4l2-ctl --stream-mmap --stream-count=N` to actually **capture frames** to `/dev/null`. The four pass/fail limits encode exactly the right gate:

```
link_locked : the lock bit is set (PCIe-L0 analog; necessary, not sufficient)
resolution_ok : width/height == expected (catches wrong-format/data-type config)
frames_captured : frames > 0 (the end-to-end sensor->...->SoC proof)
no_link_errors : decode/CRC error_count == 0 (marginal-channel catch)
```

This is the GMSL equivalent of the PCIe rule “don’t just report `errors=5` — report *which* layer.” Lock without frames = reverse-channel/sensor-config problem; frames with wrong resolution = pipe/data-type config; lock + frames + rising errors = marginal coax.

- `check_deserializer(addr, n_links=4, ...)` → `GmslDeserHealth` is the **multi-camera** layer. It runs `check_gmsl` per link and then computes a *separate* `frame_sync_ok = all(link locked)` and (not `DESYNC`). The crucial design choice: **ok requires every link healthy *and* frame_sync_ok** — so the dataclass deliberately models the **all-links-locked-but-not-frame-synchronized** case. In the mock, a “DESYNC” address yields four locked, streaming, error-free links that *still* fail overall, because frame-sync is false. That is the fusion-breaking failure a naive per-camera test cannot see, encoded as a first-class state. (A “BAD” address instead drops `link0` to model a single dead camera; the deserializer is still partly up but `ok` is false.)

The `summary()` line a station logs reads like the bench truth you want:

```
GMSL deser 1-0029: 4/4 links locked, DESYNC -> FAIL
GMSL 1-0029:link0 lock=True 1920x1080 frames=5 err=0 -> OK
GMSL 1-0029:link1 lock=True 1920x1080 frames=5 err=0 -> OK
...
```

Four OK links, one FAIL deserializer — because they aren’t shutter-aligned. That one line is the whole argument for testing frame-sync as its own thing.

A few **production-hardening decisions** in the toolkit's `_real_check_gmsl` worth calling out — each was a real audit finding, not a hypothetical:

- **Every v4l2-ctl call has an explicit timeout.** `--get-fmt-video` uses `timeout=10`; the hot path, `--stream-mmap --stream-count=N`, uses `timeout=max(30, frames * 2)` (≥ 2 s per frame). The streaming case is the most likely hang in the whole toolkit — a locked link with no frames flowing (the classic intermittent FrameSync scenario this test exists to catch) wedges `v4l2-ctl` indefinitely. On timeout the toolkit treats it as `captured = 0` so the existing `frames_captured > 0` gate fails honestly instead of the test station wedging.
- **link and video_device are regex-validated before they reach sysfs/argv.** `_LINK_RE = r"^\d{1,4}-[0-9a-fA-F]{4}$"` matches a Linux I²C device id; `_VIDEO_RE = r"^/dev/video\d+$"`. Without these, a hostile link from a plan file (link: `"../../proc/self/environ"`) would be interpolated into `/sys/bus/i2c/devices/{link}/link_status` and read whatever the path resolved to — a sysfs information-disclosure primitive.
- **Missing v4l2-ctl raises, not silent-fails.** Earlier behavior was to fall through to `captured = 0`, which then read as a hardware fault on the station log (“0 frames captured = camera dead”); the actual cause was just an unininstalled

CLI. Now the toolkit raises `RuntimeError("v4l2-ctl not found ...")` which the CLI maps to `EXIT_UNAVAIL` (5) — matching `nvme-cli` / `ethtool` / `nvidia-smi`’s “tool-missing is not a hardware fail” contract.

8.1.12 Diagnostic flow: common GMSL failure signatures → root cause

This is the table you keep open on the bench. Read the *signature* (what lock/frames/errors/ sync are doing) and it points at the layer.

Signature	Most likely cause	First moves
No lock, camera totally dead (no I2C either)	PoC/power gone: connector not clicked, cold solder, open PoC inductor	Check coax seating; measure PoC rail at the camera; PoC line-fault status; swap a known-good cable
No lock, camera <i>is</i> powered	Rate/mode mismatch (GMSL1 vs 2, 3 vs 6 Gbps), reverse channel dead, wrong term	Confirm ser/deser generation + rate match; read lock via sysfs <code>link_status</code> (mode-agnostic) or <code>0x0013[3]</code> for GMSL2 (GMSL1 lock is a separate part-specific reg); check 50/100 ohm termination; verify CFG straps
Intermittent lock (drops and re-locks, worse hot)	Marginal eye: cable loss/length, connector, PoC noise, temperature	Read eye-opening monitor margin + alarm; soak hot/cold and watch lock + error count; reseal/replace coax; check PoC filter
All links locked, but DESYNC (cameras stream individually, fusion off)	Frame-sync not reaching one camera: GPIO/FSYNC tunnel mapping, marginal reverse channel on one link	Verify FSYNC source + per-link GPIO-tunnel config; confirm every camera fires on the pulse; check the one link’s reverse channel
Locked, frames captured, but corrupt/wrong size/color	Video-pipe config: wrong data type (RAW10 vs RAW12), pixel-mode bpp, VC/stream mapping	Compare sensor output format vs pipe config; check tunnel vs pixel mode; verify VC -> <code>/dev/videoN</code> mapping
Locked, but decode/CRC error count climbing	Marginal channel — the GMSL “links up but fails BERT” pattern	Trend <code>error_count</code> over a soak; eye-monitor margin; suspect cable/connector/temperature; this is a <i>fail</i> even though it locked
Camera won’t configure (lock OK, I2C NACKs)	Reverse-channel / I2C-tunnel / address-translation problem	<code>i2cdetect</code> the deserializer locally; check tunnel enabled; verify translated sensor/ser addresses; <code>dmesg</code> for <code>-ENXIO</code>

Signature	Most likely cause	First moves
One sensor of an identical set unreachable	Address-translation collision/misconfig	Check each remote's <i>translated</i> address is distinct; two cameras translated to the same address collide

The Printed Circuit Board Assembly (PCBA)→vehicle escape this is all built around. On the bench you test GMSL through a *short* cable; a marginal eye passes. In the vehicle the same link runs **15 m of coax through a real harness with real connectors and Electromagnetic Interference (EMI), at temperature** — and that's where a marginal link drops or its error count climbs. This is the GMSL twin of the PCIe “passes at 25 °C, fails at 85 °C” escape, and it's exactly why GMSL earns *real* coverage at the **vehicle/EOL** phase, not just a lock-bit check at module test. Reading the eye-monitor margin at module test is the lever that pulls some of that catch *earlier* (capture the margin parameter, set a limit), instead of waiting for the vehicle to find it.

8.1.13 Mapping GMSL to manufacturing-test phases

How the GMSL checks above spread across the line — and which numbers you trend for yield:

Phase	What runs	The yield/quality signal you watch
Bare-board / ICT	continuity + impedance on the coax/STP traces, PoC-rail presence	shorts/opens before any silicon is stressed; bad-trace boards never reach lock test
Module / PCBA functional	per-link lock, v412 resolution + frame capture, error_count==0, eye-monitor margin captured, frame-sync across the cluster	camera-lock first-pass yield (locked-and-streaming / attempted); eye-margin distribution (a left-shifting histogram = a marginal lot or a connector/PCB change); DESYNC rate
Cabling / harness screen	PoC line-fault status (open/short/short-to-batt) on every conductor; lock + error_count through the <i>real</i> harness cable, not a bench pigtail	PoC fault rate by conductor/connector ; lock-fail localized to “cable” vs “module” by swapping a golden cable
Soak / thermal	lock held + error_count and eye-margin trended over hours, hot and cold	drop events per camera-hour and error-count slope ; a unit that locks at 25 degC but drops or climbs errors at 85 degC is the classic late escape
Vehicle / EOL	full-harness lock, frame-sync across clusters, end-to-end capture into the perception stack	final escape rate — what the earlier phases with margin limits are trying to drive to zero

Three field-tested screens worth calling out:

- **Camera-lock yield is a per-link metric, never per-board.** A quad part with one dead link is one failed unit *and* one failed link — track both, because a single link that fails across many boards points at a specific channel/connector (a fixturing or layout problem), while scattered single-link fails point at modules or cables.
- **The eye-margin histogram is the early-warning instrument.** Lock is binary and hides the cliff; the eye-monitor margin captured at module test is a continuous parameter, so a lot whose margin distribution has shifted toward the alarm threshold flags a marginal build *before* any unit actually fails lock — the whole point of capturing the parameter.

- A golden-cable / golden-module swap is the fastest “module vs cable” split. When lock or error_count fails through the production harness, re-run with a known-good coax (or move the suspect module to a known-good slot): if it passes, the harness/connector is the fault; if it still fails, the module is. This two-swap routine resolves most “no-lock, powered” fails on the line without a scope.

8.1.14 The hands-on sequence

```
# 1. Find the deserializer (and translated sensor/serializer addresses) on the I2C bus
i2cdetect -y -r 1 # e.g. deser at 0x29 or 0x48; translated cams alongside

# 2. Read link lock. Prefer the driver's sysfs (mode-agnostic); fall to raw I2C if needed.
cat /sys/bus/i2c/devices/1-0029/link_status # "locked" / "1" == forward link up
# raw GMSL2 lock bit by hand (bit 3 of reg 0x0013, the well-documented LOCKED bit):
i2cget -y 1 0x48 0x0013 # 0x08 -> GMSL2 locked
# if the link is in GMSL1 mode, lock lives in a DIFFERENT, part-specific register --
# look up the exact address+bit for your silicon in its datasheet (MAX9286 GMSL1
# deser, for reference, uses reg 0x27 bit 7), or just trust sysfs link_status above.

# 3. Enumerate the video nodes the SoC sees (one per camera/VC)
v4l2-ctl --list-devices
v4l2-ctl -d /dev/video0 --get-fmt-video # resolution/format/framerate sane?

# 4. Capture real frames -- the end-to-end proof (sensor->ser->coax->deser->CSI-2->SoC)
v4l2-ctl -d /dev/video0 --stream-mmap --stream-count=10 --stream-to=/tmp/cam0.raw

# 5. Read the physical-layer error counter; trend it over a soak (should stay 0)
cat /sys/bus/i2c/devices/1-0029/error_count # nonzero == decode/line-CRC errors

# 6. (multi-camera) repeat 2-5 for every link, THEN verify frame-sync across them
```

What the i2cdetect of a healthy quad-camera carrier actually looks like makes the address-translation story concrete — one deserializer plus four *translated* serializer addresses and four *translated* sensor addresses, none of them colliding at the sensors’ shared hardware address:

```
$ i2cdetect -y -r 1
 0 1 2 3 4 5 6 7 8 9 a b c d e f
00: -- -- -- -- -- -- -- -- --
10: -- -- -- -- -- -- -- -- --
20: -- -- -- -- -- -- -- 48 -- -- -- -- -- -- <- 0x29 deserializer (the local device)
30: -- -- -- -- -- -- -- -- --
40: 40 41 42 43 -- -- -- -- -- -- -- -- -- -- -- -- <- 4 serializers, translated 0x40..0x43
50: -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
60: 60 61 62 63 -- -- -- -- -- -- -- -- -- -- -- -- <- 4 sensors, translated 0x60..0x63
70: -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
```

(Real carriers vary; some show UU instead of the hex value where a kernel driver has already claimed the device — UU still means “present and bound,” not “missing.”) The diagnostic read of this grid: all four serializer *and* sensor addresses present = every reverse channel is tunneling and address translation is configured. A **missing serializer address** points at that link’s reverse channel or lock; a serializer present but its **sensor address missing** points at sensor power (PoC) or the camera-module-local I2C; **two cameras at the same translated address** is an address-translation misconfig (the “one sensor unreachable” row in the signature table).

The toolkit’s `check_deserializer` wraps steps 2–6 into a single per-link-plus-frame-sync verdict so the operator gets one PASS/FAIL with the per-camera detail attached.

8.2 CAN and CAN-FD — The Vehicle Control Bus

CAN is the vehicle's low-bandwidth, ultra-reliable control bus: brake controllers, steering modules, power distribution, body electronics. Your compute board has **CAN transceivers** you must prove work — send actuator commands, read vehicle state, run diagnostics over UDS/DoIP. The compute platform's *high-bandwidth* sensor data rides Ethernet/PCIe/GMSL; CAN-FD is the control-plane link to the rest of the car.

8.2.1 The bus, physically

Two wires, **CAN_H** and **CAN_L**, differential, multi-drop. Bits are **dominant (0)** or **recessive (1)**:

```
Dominant (logical 0): CAN_H ~3.5V, CAN_L ~1.5V -> differential ~2V (actively driven)
Recessive (logical 1): CAN_H ~2.5V, CAN_L ~2.5V -> differential ~0V (idle/released)
```

```
Dominant ALWAYS wins: if any node drives dominant, the bus is dominant.
Only when ALL nodes release does the bus stay recessive. <- this is what makes
bitwise arbitration work without collisions.
```

Termination: 120 Ω at each physical end of the bus. With both terminators present you measure ~60 Ω across CAN_H/CAN_L with the bus powered off (two 120 Ω in parallel). This DMM check is the single fastest, highest-value CAN test on the line:

- ~60 Ω → both terminators present, healthy.
- ~120 Ω → one terminator missing (you're reading a single resistor).
- **open / very high** → wiring fault, both terminators missing, or bus not connected.
- **near 0 Ω** → CAN_H/CAN_L shorted.

8.2.2 Arbitration — how the bus shares without collisions

Every node monitors the bus *while* transmitting. When a node sends recessive (1) but reads back dominant (0), a higher-priority message is on the bus — that node **loses arbitration, backs off, and retries later**, while the winner transmits *uninterrupted*. Lower ID = more dominant bits earlier = higher priority. The winner never even knows there was contention; no time is wasted (nondestructive arbitration).

```
Node A wants ID 0x100 = 001 0000 0000
Node B wants ID 0x050 = 000 0101 0000
^bit where they differ
Both start together. At the bit where A sends recessive(1) and B sends dominant(0):
A reads back dominant -> A LOST -> A stops and waits. B continues, frame intact.
B (0x050, lower ID) wins because its dominant bit came first.
```

8.2.3 Frame formats — Classic CAN and CAN-FD

Field	Classic CAN	Notes
SOF	1 bit	Start of frame (dominant)
ID	11 bit (std) / 29 bit (ext)	lower = higher priority
RTR / control	a few bits	remote request, IDE, reserved
DLC	4 bit	data length code (0–8 bytes)
Data	0–8 bytes	payload
CRC	15 bit	CRC-15 over frame content
ACK	2 bit	receivers acknowledge
EOF	7 bit	end of frame (recessive)

CAN-FD extends Classic CAN where you need more data but Ethernet is overkill (e.g., faster ECU firmware flashing):

Feature	Classic CAN	CAN-FD
Max data rate	1 Mbit/s	up to ~8 Mbit/s in the data phase
Max payload	8 bytes	64 bytes
CRC	CRC-15	CRC-17 (≤ 16 B) or CRC-21 (> 16 B)
Bit-rate switching	No	Yes (arbitration at nominal rate, data phase faster)

CAN-FD keeps arbitration at the nominal bit rate (so priority/arbitration still works across the whole bus) and switches to the faster rate only for the data + CRC phase, flagged by the **BRS** (bit-rate switch) bit. CAN-FD frames carry the **FDF** bit; a Classic-CAN-only node sees an FD frame as an error, so mixed FD + classic on one bus is delicate (you generally don't send FD frames while a classic-only node is present).

CAN-FD DLC mapping (non-linear above 8): DLC 0–8 $\rightarrow$ 0–8 bytes, then 9 $\rightarrow$ 12, 10 $\rightarrow$ 16, 11 $\rightarrow$ 20, 12 $\rightarrow$ 24, 13 $\rightarrow$ 32, 14 $\rightarrow$ 48, 15 $\rightarrow$ 64 bytes.

8.2.4 Bit timing and the sample point

A CAN bit is divided into time quanta with a programmable **sample point** — the fraction of the bit time at which the receiver samples the level (commonly **75–87.5%** of the bit). Both ends must agree on bit rate *and* roughly on sample-point placement, or you get intermittent errors that look like noise. A **wrong bit-rate/timing config** is a top cause of a node that goes bus-off the moment it tries to transmit — the symptoms look like a hardware fault but the fix is in the timing registers (`ip link set ... type can` parameters, or the controller's `tq/prop_seg/phase_seg` settings).

8.2.5 Error frames and the three error states

Every node keeps a **Transmit Error Counter (TEC)** and **Receive Error Counter (REC)**. CAN's brilliance is *self-healing fault confinement* — a node that's causing errors progressively removes itself:

```
ERROR-ACTIVE (TEC and REC < 128) normal; sends ACTIVE error flags (dominant)
| errors accumulate ^ returns here when TEC and REC are
v | both back <= 127
ERROR-PASSIVE (TEC or REC >= 128) -----+ sends PASSIVE error flags; backs off more
| TEC keeps climbing
v
BUS-OFF (TEC > 255, i.e. >= 256) node removes itself from the bus entirely
(recover: 128 occurrences of 11 recessive bits,
then both counters reset to 0)
```

The boundaries are exact and worth memorizing: error-passive at **TEC or REC ≥ 128** , bus-off at **TEC > 255** (the 8-bit counter's top). A node drops back from error-passive to error-active only once *both* counters fall to ≤ 127 again. (Sources: the CAN error- confinement rules as documented by can-wiki.info and the CSS Electronics CAN-errors intro.)

The five error-detection mechanisms that drive those counters: **bit monitoring** (sent $\neq$ read back), **bit stuffing** (a complementary bit after 5 identical — a violation is an error), **CRC check**, **frame-format check** (fixed-form fields), and **ACK check** (at least one receiver must acknowledge). Detecting an error makes the node transmit an **error frame** (6 dominant bits for an active flag), which destroys the current frame on the bus and forces a retransmit — and bumps the counters.

The single-node test trap. A node alone on a bus **cannot** transmit successfully: after each frame it checks the ACK slot, finds no other node drove it dominant, flags an ACK error, retries, and marches TEC straight to BUS-OFF. So testing a compute board's CAN port with *nothing else on the bus* always “fails” — you need at least one other node (a CAN tool, another board, or a loopback). This catches people constantly; it's not a defect, it's physics.

8.2.6 Linux: SocketCAN and can-utils

Linux treats CAN as a network interface (SocketCAN), so the tooling is `ip` + `can-utils`:

```
# Bring up the interface (classic, 500 kbit/s -- the common vehicle rate)
ip link set can0 up type can bitrate 500000
# CAN-FD: nominal 500k arbitration + 2M data phase
ip link set can0 up type can bitrate 500000 dbitrates 2000000 fd on

# State + error counters -- the CAN analog of AER/EDAC; READ THIS FIRST when debugging
ip -details -statistics link show can0
# look for: state ERROR-ACTIVE (good) / ERROR-PASSIVE / BUS-OFF (bad)
# berr-counter tx <TEC> rx <REC>
# bus error / restart counts

candump can0 # watch all traffic (hex)
candump -t d can0 # with delta timestamps (timing analysis)
candump -e can0 # show error frames
candump -l can0 # log to file (candump-<timestamp>.log)

cansend can0 123#DEADBEEF # standard ID 0x123, 4 data bytes
cansend can0 00000123#01.02.03.04.05.06.07.08 # extended ID, 8 bytes
cansend can0 123##0.01.02.03 # CAN-FD frame (## then flags byte)

cangen can0 -g 10 -I 100 -L 8 -D random # generate: every 10ms, ID 0x100, 8 random bytes
canbusload can0@500000 # bus utilization at 500 kbit/s
```

What those tools actually print is worth recognizing on sight. A healthy `candump` is one column of interface, ID, length-in-brackets, then hex bytes:

```
$ candump can0
can0 100 [8] 01 02 03 04 05 06 07 08
can0 1A0 [3] DE AD BE
can0 123 [8] 00 00 00 00 00 00 00 2A
```

The single most important read is the interface-state line. On a *healthy* bus the counters sit at zero and the state is `ERROR-ACTIVE`:

```
$ ip -details -statistics link show can0
2: can0: <NOARP,UP,LOWER_UP,ECHO> mtu 16 ... state UP
link/can promiscuity 0
can <FD> state ERROR-ACTIVE (berr-counter tx 0 rx 0) restart-ms 100
bitrate 500000 sample-point 0.875
...
RX: bytes packets errors dropped ...
... 0 0 0
TX: bytes packets errors dropped ...
... 0 0 0
```

On a *sick* bus those same fields are the diagnosis — note `state BUS-OFF` and a saturated `TEC`, and the `bus-error/restart` counts climbing:

```
can <FD> state BUS-OFF (berr-counter tx 255 rx 0) restart-ms 100
...
re-started bus-errors arbit-lost error-warn error-pass bus-off
3 511 0 4 2 3
```

And with `-e`, an error frame decodes in human-readable form, error counters and all — this is the exact moment the controller flagged a fault, with the cause spelled out:


```
$ candump -e can0
can0 20000088 [8] 00 00 80 19 00 00 00 00
ERRORFRAME protocol-violation{error-on-tx}{acknowledge-slot}}
bus-error error-counter{tx{128}rx{97}}
```

That `acknowledge-slot / error-on-tx` with the TEC at 128 is the textbook **single-node / no-ACK** signature from the trap above — the board transmitted, nobody ACKed, TEC jumped.

The DBC database. Raw CAN is just IDs and bytes; a **DBC** file (Vector’s format) is the schema that says “ID 0x100 byte 0 bits 0–7 is `WheelSpeed`, scale 0.1, offset 0, unit km/h.” Tools like `cantools` (Python) decode live traffic against a DBC so your test reads *signals* (“wheel speed = 42.3 km/h”), not bytes. Manufacturing tests use a DBC to assert that a board emits the right signals at the right rates, and to craft stimulus frames by signal name rather than hand-packing bytes.

8.2.7 What a manufacturing CAN test actually does

1. **Termination check** — DMM across CAN_H/CAN_L, expect $\sim 60\ \Omega$ (5-second, highest-yield check).
2. **Bring-up at the configured bitrate** with a known partner on the bus (CAN tool / another board / loopback) — never solo.
3. **Loopback / known-frame exchange** — send known frames, verify received intact in both directions; confirm the bitrate is right (wrong bitrate $\rightarrow$ immediate errors).
4. **Watch TEC/REC and state** — must stay **ERROR-ACTIVE**, TEC/REC at/near 0, never go **BUS-OFF** under traffic.
5. **Stress** — `cangen` near max bus load, confirm no error frames and counters stay clean.
6. **Bus-off recovery** — deliberately inject errors, confirm the controller recovers.
7. (**Deeper**) scope CAN_H/CAN_L for clean dominant/recessive levels ($\sim 2.5\text{ V}$ common mode, $\sim 2\text{ V}$ differential dominant).

8.2.8 Toolkit cross-reference: `check_can`

`ethernet.py`’s `check_can(iface)` $\rightarrow$ `CanHealth` reads exactly the right state from `ip -details -statistics` link show: it classifies state (**BUS-OFF** / **ERROR-PASSIVE** / **ERROR-WARNING** / **ERROR-ACTIVE**), parses `berr-counter tx/rx` into **TEC/REC**, and detects FD mode. Its three checks encode the gate: `not_bus_off`, `error_active` (the *only* fully healthy state), and `low_errors` (TEC and REC both < 96 — a margin below the 128 error-passive threshold, so a unit that’s *trending* toward trouble fails before it actually crosses into error-passive). That sub-threshold limit is the CAN version of “capture the parameter, set the limit below the cliff.”

Higher layers are awareness-level for you day one. **UDS** (ISO 14229) diagnostics — sessions, security access, Read/Write Data By Identifier, Routine Control (self-tests), firmware download — ride on CAN (ISO 15765 / ISO-TP) or on Ethernet via **DoIP** (ISO 13400, TCP port 13400, for fast firmware flashing). **SOME/IP** and **DDS** (ROS 2’s transport) are service-oriented middleware over Automotive Ethernet. You’ll meet them, but the compute-board CAN test is about the transceiver, termination, and error counters.

8.3 Automotive Ethernet — One Pair, Master/Slave

Radar, lidar, and inter-module traffic ride **automotive Ethernet**. It is standard Ethernet at the MAC/IP layer (so everything in the **Networking chapter** about IP, sockets, `ip`, `tcpdump`, `iperf` applies) but with a radically different **physical layer**, and a few gotchas that bite specifically on the line.

8.3.1 The physical layer: single twisted pair, PAM3

Standard	Speed	PHY	Use at Zoox
100BASE-T1 (BroadR-Reach)	100 Mbit/s	single twisted pair, full-duplex, PAM3	body/chassis, gateways, slower sensors
1000BASE-T1	1 Gbit/s	single twisted pair, full-duplex, PAM3	camera/radar/lidar streams, inter-module
10BASE-T1S	10 Mbit/s	single pair, multidrop	low-speed multidrop (CAN replacement)
2.5/5/10GBASE-T1	multi-Gbit	single pair	compute backbone, high-rate sensors

Why single pair? Weight and connector size. A standard 4-pair RJ45 link is unthinkable across a vehicle harness with hundreds of connections; a single unshielded/shielded twisted pair, full-duplex (Transmit (TX) and Receive (RX) share the pair via echo cancellation), with **PAM3** line coding (three voltage levels, more bits per symbol than Non-Return-to-Zero (NRZ)) gets 100 Mbit/s–multi-Gbit over one lightweight pair. Contrast with consumer Ethernet (the Networking chapter): 4 pairs, RJ45, auto-MDI-X, and clock auto-negotiation — none of which apply here.

8.3.2 Master/slave — a config, not a cable

This is the automotive-Ethernet gotcha that wastes the most bench time. Unlike consumer Ethernet, one PHY on each link is the **MASTER** (it provides the clock) and the other is the **SLAVE** (it recovers the clock). The two ends **must be opposite**. A “no link” between two correctly-cabled, healthy PHYs is very often a **master/master or slave/slave misconfiguration**, not a wiring fault.

First move on a dead automotive-Ethernet link: check the *role*, not the cable. Run `ethtool` and read master/slave on both ends *before* you suspect the harness. Two masters or two slaves never link, no matter how perfect the cable.

8.3.3 The PHY and MDIO

The PHY (Marvell **88Q2112**, TI **DP83TG721-Q1**, Broadcom multi-Gig families) is configured over **MDIO** (the management bus — clause-22 for legacy 100 Mbit, clause-45 for Gig+), and Linux exposes it through the `netdev` + **phylib**. The master/slave role, link status, and the PHY’s error/diagnostic counters all live in PHY registers reachable over MDIO; `ethtool` surfaces the common ones, and `mdio-tool/phytool` reach raw registers when you need them. Many automotive PHYs (e.g., DP83TG721) also carry **TSN/AVB** support and **TC10** (OPEN Alliance coordinated sleep/wake) — words you’ll see in the datasheet; not your day-one ownership.

8.3.4 AVB/TSN basics

TSN (Time-Sensitive Networking, the IEEE 802.1 suite) makes Ethernet *deterministic* — bounded latency and jitter for real-time sensor traffic alongside best-effort traffic, which an AV stack needs so sensor frames arrive in their time window. The pieces you’ll hear: **gPTP** (802.1AS, a 1588 PTP profile) distributes a grandmaster clock for sensor fusion; **802.1Qbv** time-aware shaping schedules traffic into time slots; **802.1Qbu/802.3br** frame preemption lets urgent frames interrupt long ones. Validate gPTP with `ptp4l/phc2sys` and check **offset-from-master** convergence and `PHC↔system-clock` sync. You don’t own the TSN config day one, but the test hooks (is the clock locked, what’s the offset) shouldn’t surprise you.

8.3.5 Diagnostics

```
ethtool eth1 # link up? speed? duplex? -- and master/slave role
ethtool -i eth1 # driver + firmware version (re-qualify on FW change)
ethtool -S eth1 # stats: rx/tx errors, CRC/align errors, dropped -> error counters
ethtool -t eth1 online # PHY/MAC self-test (offline is more thorough but drops link)
```

```

ethtool --cable-test eth1 # TDR cable diagnostics on supported PHYs
ethtool --cable-test-tdr eth1 # TDR with fault type + DISTANCE TO FAULT
iperf3 -c <partner> -t 30 # throughput: 1000BASE-T1 should sustain ~0.95 Gbps

```

The fields the toolkit (and you) actually parse out of `ethtool eth1` — note the master/slave line that consumer Ethernet never has, and the Speed line `_parse_speed` reads:

```

$ ethtool eth1
Settings for eth1:
  Supported ports: [ TP ]
  Supported link modes: 1000baseT1/Full
  Speed: 1000Mb/s
  Duplex: Full
  Port: Twisted Pair
  PHYAD: 0
  master-slave cfg: forced master
  master-slave status: master
  Link detected: yes

```

master-slave status: master on *both* ends is the no-link bug from above; one must read `slave`. The error counters live in `ethtool -S` and are the marginal-Signal Integrity (SI) tell when they creep on an otherwise-up link:

```

$ ethtool -S eth1 | grep -iE 'err|crc|symbol'
rx_errors: 0
tx_errors: 0
rx_crc_errors: 0
SymbolErrorDuringCarrier: 0

```

`ethtool --cable-test-tdr` is the **automotive-Ethernet killer app** — the single-pair cousin of PCIe lane margining, scope-free. It pulses the pair and times the reflection to report **fault type + distance to fault**: **OK**, **Open Circuit** (with distance), **Short** (to another pair), **Impedance Mismatch** (a reflection from a discontinuity), or **Noise** (the test couldn't complete). On a vehicle harness the *distance* pinpoints which connector or segment is bad. The result reads back per-pair, with the distance that localizes the fault:

```

$ ethtool --cable-test-tdr eth1
Cable test completed for device eth1.
Pair A code Open Circuit
Pair A, fault length: 4.50m

```

(A clean run reports Pair A code OK. On an unsupported PHY the command errors out instead — which is exactly why `check_ethernet` treats “unsupported” as *skipped*, never a fail.) The diagnostic triad: a link that's *up* with *low iperf throughput* and *rising CRC counts* (`ethtool -S`) is marginal SI — confirm it with the Time-Domain Reflectometry (TDR), which localizes the fault on the cable.

Raw PHY registers when ethtool isn't enough. The master/slave bit and link state live in standard PHY registers you can read directly over MDIO with `phytool/mdio-tool` — useful when a driver doesn't surface a field or you're bringing up a board pre-driver. The 1000BASE-T1 role lives in the PMA control register; the generic status register 0x01 (BMSR) bit 2 is link-up:

```

# phytool read <iface>/<phyaddr>/<reg> (clause-22) or /<devad>/<reg> for clause-45
$ phytool read eth1/0/0x01 # BMSR; bit 2 (0x0004) = Link Status (1 = up)
0x796d
$ mdio eth1 phy 0x00 # raw register dump via the 'mdio' tool, if present

```

(Exact register/bit for the master/slave *role* is PHY-specific — Marvell 88Q2112 vs TI DP83TG721 differ; pull it from that PHY's datasheet. `ethtool`'s `master-slave status` is the portable read; raw MDIO is the fallback.)

8.3.6 Toolkit cross-reference: check_ethernet

ethernet.py's `check_ethernet(iface)` → EthHealth parses `ethtool` for **link**, **speed** (handling “2.5G” → 2500 Mbps), and **master/slave** role, reads **Receive (RX)/Transmit (TX)** errors from `ethtool -S`, optionally runs an `iperf3` throughput leg, and — on supported PHYs — runs a **TDR cable test** (`--cable-test-tdr`), parsing fault `pair/code/distance_m`. The checks: `link_up`, `speed_ok` (`>=` expected — catches a 1000BASE-T1 link that came up at 100), `low_errors` (`rx+tx < 10`), `throughput_ok`, and `cable_ok` (TDR not a fault). Note the deliberate design that the real TDR path treats *unsupported* as “skipped,” **never a fail**, so a PHY that can't do TDR doesn't false-fail a good link — the same “don't punish a missing capability” discipline you want everywhere in MT code.

Three **production-hardening** decisions in the toolkit's implementation worth knowing:

- **The interface name is regex-validated before it reaches `ethtool` / `ip`.** `_IFACE_RE = r"^[A-Za-z0-9_][A-Za-z0-9._-]{0,14}$"` — Linux's `IFNAMSIZ - 1` of 15 characters, with the **leading character constrained** to alnum/underscore so a value like `--help` can't slip through into `argv` as an option to `ethtool/ip`. A plan-file `ethernet: ["--help"]` would otherwise emit `ethtool --help` and parse the help text as if it were the device's link state. The leading-char constraint was the audit fix; the bare-char-class regex caught everything else but missed that.
- **Every subprocess.run has an explicit timeout.** `ethtool/ethtool -S` and `ip -details -statistics link show use timeout=10`; without it, a wedged PHY (a driver bug, a flaky MDIO controller) wedges the test station indefinitely. The sibling calls `_real_iperf` and `_real_cable_test` had timeouts since day one; this is the audit-1 finding that closed the inconsistency.
- **Parser hardening from Hypothesis.** `_parse_speed` was crashing on "Speed: . G" (regex `[\d.]+` matched the bare . and fed `float('.')` to the parser) and `_stat` was crashing on "weird: 5²\n" (Unicode digit, `str.isdigit()` returns True, `int()` raises). Both were found by property tests in `tests/test_parsers_property.py` and pinned with `@example(...)` decorators. The real-world cases came from `ethtool` output captured on a marginal NIC.

Contrast with the Networking chapter. That chapter owns the IP/socket/tcpdump/iperf layer and standard Ethernet. Here the *additions* are: single-pair PAM3 PHYs, the **master/slave** role failure mode, **MDIO/phylib** access, **TDR cable test**, and **TSN/gPTP** determinism. Same MAC/IP debugging on top; different wire underneath.

8.4 I2C, SPI, UART — The Housekeeping Buses

These three carry *configuration, identity, and health*, not sensor bandwidth: who are you (EEPROM), what's your temperature (sensor), set your registers (sensor/SerDes config), are your rails in spec (power monitor), what time is it (RTC). They're far simpler than PCIe — and that's exactly why you must know them cold: **when a high-speed link won't come up, the reason is frequently sitting in a register you read over I2C**. (Full depth lives in the Embedded Buses chapter; this is the test-floor essentials and the GMSL tie-in.)

Property	I2C	SPI	UART
Wires	2: SDA, SCL	4: MOSI, MISO, SCLK, CS	2: TX, RX
Clock	shared, from controller	shared, from controller	none — async, agreed baud
Topology	multi-controller, multi-target (shared)	1 controller, N targets (one CS each)	point-to-point
Addressing	7-bit (or 10-bit) device address	none — CS selects target	none
Duplex	half	full	full

Property	I2C	SPI	UART
Drive	open-drain + pull-ups (wired-AND)	push-pull	push-pull
Typical speed	100k / 400k / 1M / 3.4M Hz	1–100 MHz	9600–115200 (to a few Mbaud)

Mental model: **I2C trades speed for wires** (two wires, address-multiplexed, slow). **SPI trades wires for speed** (more pins, no addressing, fast). **UART trades a clock wire for a timing agreement** (no clock, both ends must already know the baud).

8.4.1 I2C — the test floor’s most-used bus

Both lines are **open-drain**: a device can only pull low; external pull-ups pull high (wired-AND — any device holding low wins). A transaction: **START** (SDA falls while SCL high) → 7-bit address + R/W bit → target **ACK** (pulls SDA low) → data bytes each ACK’d → **STOP** (SDA rises while SCL high). A register read uses a **repeated START**: write the register pointer, repeated START, then read.

The #1 I2C gotcha: 7-bit vs 8-bit address. Datasheets list a **7-bit** address (e.g. 0x48); the byte on the wire is `addr << 1 | r/w` (so 0x90 write / 0x91 read). **Linux tools take the 7-bit address.** “The device is at 0x48 but my code talks to 0x90 and gets nothing” is the classic mistake.

```
i2cdetect -l # list all I2C buses the kernel knows about
i2cdetect -y 1 # scan bus 1: grid of addresses that respond
# "UU" = claimed by a bound kernel driver (don't poke)
# "48" = ACKs, no driver bound (free to probe)
# "--" = no response
i2cget -y 1 0x48 0x00 # read register 0x00 from device 0x48
i2cget -y 1 0x48 0x00 w # read a 16-bit word (watch byte order!)
i2cset -y 1 0x48 0x01 0xFF # write 0xFF to register 0x01
i2cdump -y 1 0x48 # dump all registers (great for exploring an unknown part)
i2ctransfer -y 1 w2@0x48 0x01 0xFF r1@0x48 # explicit write-then-read (repeated START)
```

Debug outside-in: `i2cdetect` first (ACK at the right address? -- everywhere = dead bus/pull-ups; wrong address = check ADDR strap pins). Then scope SDA+SCL together: no edges = not clocking; stuck low = a device holding the bus or a hung target stretching SCL forever (fix: clock 9+ SCL pulses, a bus-recovery sequence); slow rounded rising edges = pull-ups too weak / bus capacitance too high; missing ACK = wrong address / unpowered / in reset / the 7-vs-8-bit mistake. `dmesg` logs `-ETIMEDOUT/-ENXIO` (no ACK) with the bus and address.

The GMSL tie-in (why I2C is *the* camera-bring-up bus). As covered in the GMSL section, the deserializer is a *local* I2C device that **tunnels** transactions over the reverse channel to the remote serializer and sensor, with **address translation** so identical cameras don’t collide. The whole camera diagnostic flow is I2C: talk to the deserializer (lock asserted at 0x0013[3]?), then through the tunnel to the sensor (configured? powered via PoC?). The single highest-value tunnel probe is reading a **known sensor ID register** through the translated address — if it returns the datasheet’s chip-ID, the entire reverse path (deser tunnel → coax → serializer → sensor I2C) is proven in one read:

```
# deserializer GMSL2 lock first (bit 3 of 0x0013):
$ i2cget -y 1 0x48 0x0013
0x08 # bit 3 set -> GMSL2 locked

# then the sensor's chip-ID register THROUGH the tunnel, at its TRANSLATED address (0x60).
# (ON-Semi AR-series sensors conventionally expose chip-version at 0x3000 -- use YOUR
# sensor's actual ID register + expected value from its datasheet; values below illustrate.)
$ i2cget -y 1 0x60 0x3000 w # 16-bit chip-ID register read through the tunnel
0xAABB # == datasheet chip-ID -> reverse tunnel + sensor alive
```

```
# a sensor that won't answer through the tunnel NACKs (reverse channel / translation / PoC):
$ i2cget -y 1 0x60 0x3000 w
Error: Read failed # -> dmesg shows i2c -ENXIO on bus 1, addr 0x60
```

A dead I2C reverse channel = no sensor config = no video, even over a perfect coax — and lock can be asserted while decode-error counters climb on a marginal coax, the PCIe “links up but fails Bit Error Rate Test (BERT)” pattern read over I2C instead of Advanced Error Reporting (AER). (The sensor’s ID-register address and expected value are part-specific; pull them from the image-sensor datasheet, not memory.)

8.4.2 SPI — fast, point-to-point-ish

Four push-pull lines: **SCLK** (clock), **MOSI** (out), **MISO** (in), **CS** (chip select, active-low, one per target). No addresses — asserting a target’s CS selects it. Full-duplex: a bit goes out on MOSI and a bit comes in on MISO on every clock.

The SPI gotcha: CPOL/CPHA mode. Both ends must agree on clock polarity (idle high/low) and phase (sample on leading/trailing edge), or every byte is garbage:

Mode	CPOL (idle)	CPHA	Sample on
0	0 (low)	0	leading edge
1	0 (low)	1	trailing edge
2	1 (high)	0	leading edge
3	1 (high)	1	trailing edge

Modes 0 and 3 dominate. Wrong mode → data shifted a bit or scrambled while clock/CS look perfect on the scope. Bit order (MSB/LSB-first) and word size must match too.

```
ls /dev/spidev* # devices are /dev/spidevB.C = bus.chip-select
spidev_test -D /dev/spidev0.0 -s 1000000 -v -p "\x9F\x00\x00\x00"
# 0x9F = JEDEC READ-ID on most SPI NOR flash; reply = manufacturer/device ID -> link proven
flashrom -p linux_spi:dev=/dev/spidev0.0,spispeed=1000 # read/probe a SPI NOR flash
```

Debug: verify with a **known command** — a flash’s READ-ID (0x9F) returns a fixed manufacturer/device ID; the right ID is the fastest end-to-end proof. 0x00/0xFF everywhere = nothing driving MISO (dead/unselected target, wrong CS, wrong mode so the command is never recognized).

8.4.3 UART — the debug console

Two lines **Transmit (TX)** and **Receive (RX)** (cross-connected: each device’s TX → the other’s RX) + common ground. **Asynchronous** — no clock, both ends preconfigured to the same **baud**. Frame: idle-high, **start bit** (high→low), data bits (LSB-first), optional parity, 1–2 **stop bits**. “**115200 8N1**” = 115200 baud, 8 data, No parity, 1 stop — the de-facto console default.

Two UART hazards. (1) **Voltage:** board debug UARTs are TTL/CMOS **3.3 V (or 1.8 V)**; true **RS-232** swings ± 3 –15 V with *inverted* logic. Connecting a ± 12 V RS-232 line to a 3.3 V pin **destroys it** — use a level-shifter/USB-TTL adapter at the right voltage, confirmed against the schematic. (2) **Baud mismatch** is the #1 failure: garbage characters on a perfect cable. Try the ladder (9600/19200/38400/57600/115200/921600). And **TX/RX swap** is the eternal bug — both ends “transmit on TX,” so you must cross them.

```
dmesg | grep tty # which serial devices enumerated
stty -F /dev/ttyUSB0 115200 cs8 -cstopb -parenb # configure 115200 8N1
picocom -b 115200 /dev/ttyUSB0 # interactive console (exit: C-a C-x)
screen /dev/ttyUSB0 115200 # alternative terminal
```

For programmatic/instrument use, `pyserial` — and **always set a timeout** so a UART that never replies can’t hang the test:

```
import serial
ser = serial.Serial("/dev/ttyUSB0", 115200, timeout=2,
    bytesize=8, parity="N", stopbits=1)
ser.write(b"*IDN?\n")
print(ser.readline().decode(errors="replace"))
ser.close()
```

Common test uses: the boot console (watch a board power up, catch a bootloader hang or a kernel panic in real time) is the highest-value UART; also GPS modules, instrument links, and a camera module’s MCU when it speaks UART (possibly tunneled over GMSL).

8.5 Putting It Together at Zoox

Every bus in this chapter maps to a manufacturing-test phase and a captured parameter, the same way PCIe and Non-Volatile Memory Express (NVMe) do:

Bus	What you prove	Earliest phase that catches it	Captured parameter
GMSL	each link locks, streams frames, error-free, frame-synced	lock/format at module ; marginality at vehicle (real 15 m harness)	per-link lock, eye-monitor margin , decode-error count, frame-sync state
CAN	transceiver works, no bus-off under load	module (with a bus partner); vehicle EOL talks to real ECUs	state, TEC/REC , termination ohms
Auto Ethernet	link at rate, low errors, cable healthy	module ; harness faults at vehicle	speed, master/slave role, rx/tx errors, TDR distance-to-fault
I2C/SPI/UART	every housekeeping device present and readable	PCBA/module (enumeration + known-ID reads)	i2cdetect map, known-ID readbacks

The throughline is the same one the rest of this guide hammers: **capture the parameter, not just the verdict**. “Camera locked” / “CAN is up” / “Ethernet linked” are binary and hide the margin. The GMSL eye-monitor margin, the CAN TEC/REC trend, the Ethernet TDR distance, the GMSL decode-error count over a soak — those are the *numbers* that let DV set a data-driven limit and MT check it, and that turn a vehicle-level “passes at 25 °C, drops at 85 °C” escape into a module-level catch with evidence already attached. Most of the expensive failures here are *only detectable* late (over the full harness, at temperature) but were *introducible* early — which is the whole reason GMSL gets the deepest coverage of the four, and why it gets real teeth at the vehicle phase.

Chapter 9

Manufacturing Test Principles

This is the discipline that the rest of the guide serves. The interface chapters teach you *how* to make a measurement (read Advanced Error Reporting (AER), margin a lane, count Error-Correcting Code (ECC) errors); this chapter is *why* you make it, *where* you make it, *what limit* you judge it against, and *how* you turn the captured numbers into yield, root cause, and a test program you can release to a contract manufacturer and trust.

Manufacturing test exists to answer one question on every unit: “**Is this unit good enough to ship?**” Every decision in the chapter — phase placement, limit setting, soak duration, parallelization, correlation — is a balance of three forces: **coverage** (catch the defect), **time** (hit takt / throughput), and **cost** (equipment, labor, floor space, and the cost of being wrong). The expert skill is holding all three at once.

The math behind everything quantitative here — capability (C_{pk}/P_{pk}), Statistical Process Control (SPC) and the Western Electric rules, Gage R&R, yield (First Pass Yield (FPY)/Rolled Throughput Yield (RTY)), cost-of-test, and the Bit Error Rate Test (BERT) confidence formulas — is *derived* in the **Math & Statistics chapter**. This chapter uses those results to make decisions; it points there for the derivations rather than repeating them.

9.1 The Four Phases, In Depth

The phases are **Printed Circuit Board Assembly (PCBA)** → **module** → **system** → **vehicle**, and the organizing rule is **place each test at the earliest phase that can catch its defect**. Each phase tests a different *thing*, owned by a different group, proving a different property.

9.1.1 PCBA — “was it built correctly?”

The unit is a bare board just off the Surface-Mount Technology (SMT) line: right components, good solder, no shorts/opens, powers on, reaches a basic boot. Mostly run **at the Contract Manufacturer (CM)**, mostly not your code — but you must understand it, because a defect that escapes here costs 10× more at the next phase, and because the *coverage gaps* here are what your module test must cover.

- **Automated Optical Inspection (AOI)** — cameras check for missing / misplaced / wrong / tombstoned parts and gross solder defects. Fast, before anything is powered.
- **In-Circuit Test (ICT)** — a bed-of-nails fixture probes nets to measure R/C, find shorts and opens, and verify component values. The workhorse PCBA test for high-volume, *fixed* boards.
- **Boundary scan / JTAG** (IEEE 1149.1) — shifts test patterns through device scan chains to test interconnects you cannot physically probe (BGA balls under the package) and to program flash/CPLD. Often the *first* test on a new board, before you try to boot it. (See Design for Testability (DFT), .)

- **Flying probe** — ICT without a custom fixture, for low volume and prototypes (slower, no fixture cost).
- **X-ray (AXI)** — sees solder voids and bridges *under* BGAs that AOI cannot. The only way to catch a void under the GPU or a large connector at PCBA.
- **First power-on / boot** — does it come up, draw the right current, reach a prompt. Often programs the initial bootloader/firmware here.

9.1.2 Module — “does every interface work, at speed, under load, hot?”

The unit is now a sealed, functional module *with its thermal solution*. **This is the heart of your job.** Almost every interface test in the guide runs here:

- **Enumerate everything** — `lspci`, `nvme list`, `nvidia-smi`, NIC/CAN/GMSL presence.
- **Prove each link trains at the *expected* speed and width** — PCIe Gen4 x16, Non-Volatile Memory Express (NVMe) Gen4 x4, etc. Trained-below-max is your first Signal Integrity (SI) finding (PCIe chapter).
- **Stress + error counting** — drive traffic and watch AER / SMART / ECC / Error Detection and Correction (EDAC) counters (the BERT and diagnostic tools live here; PCIe/NVMe/GPU/Memory chapters).
- **Thermal / burn-in** — soak at temperature and/or under load, re-check links and errors. **This is the phase that catches marginal-at-temperature defects PCBA structurally cannot.**
- **Power characterization** — measure rail voltages/currents under load with a DMM/scope (Instruments chapter); out-of-window-under-load is a power-delivery defect and a common root cause of link errors.
- **Firmware / version verification** — flash and confirm firmware/Board Support Package (BSP) versions; record them for traceability.
- **Calibration** where applicable.

9.1.3 System — “do the modules work *together*?”

Modules are integrated into the compute box/rack. You now test the *interactions*: inter-module PCIe/Ethernet links, the system power budget under full load, system-level thermals (fans, airflow, the whole box), and that the integrated system boots and runs the stack. Failures here are expensive to localize — which is exactly why module test should have already proven each module *in isolation*. If you are debugging “which of six modules is marginal” at system test, the right fix is usually upstream: tighten the module-phase coverage so it never gets here.

9.1.4 Vehicle (End-of-Line (EOL) — End Of Line) — “does it work in the car, end to end?”

Compute is installed in the vehicle with real sensors. Cameras must lock over *real* GMSL harnesses (15 m of coax, real connectors, real Electromagnetic Interference (EMI)), all sensors stream, CAN talks to the vehicle bus, and the vehicle-level functional checks pass before it ships. **This is where channel-marginal SerDes defects surface** — a GMSL link that locks on a short bench cable can drop over a full harness at temperature, the GMSL version of the PCIe “passes at 25 °C” escape (Automotive buses chapter). You cannot move this coverage earlier because the *real channel* only exists at the vehicle.

9.1.5 Design for Testability (DFT) — earning the coverage before the board exists

DFT means designing the hardware so it *can* be tested effectively in manufacturing. If DFT is neglected, you get boards that work but can’t be verified — and you discover the defects in the field instead of the factory. As the test engineer you push for DFT *in design reviews*, before there is a board to test. What you ask for:

- **Test points** on critical signals — power rails (voltage + ripple), clocks (frequency + amplitude), high-speed SerDes (scope access for debug). Physically reachable by bed-of-nails / flying probe, ≥ 25 mil pad, not buried under a heatsink.
- **JTAG / boundary scan** accessible on the production fixture — interconnect test + BIST + flash programming.
- **Loopback paths** — PCIe / Ethernet / UART loopback to test a full signal path (serializer $\rightarrow$ driver $\rightarrow$ receiver $\rightarrow$ deserializer) *without* a second device, saving fixture cost and test time.
- **BIST** (Built-In Self-Test) — memory BIST (walking-ones, checkerboard patterns) and logic BIST run *at full speed* and cover internal paths external tests cannot reach.
- **On-board identity (EEPROM/flash)** — serial number, MAC, board revision, manufacturing date, test results, calibration. Travels with the board through its lifecycle; the anchor for traceability.
- **Debug interfaces** — a UART console that works during boot (the #1 bring-up tool), JTAG header, I2C/SPI taps — accessible on *production* boards for field-return failure analysis.

DFT review checklist (carry this into the review): Can every power rail be measured without board mods? Is JTAG reachable on the production fixture? Can every high-speed link run in loopback? Is there a boot-time UART console? Can board identity + test history live on-board? Can firmware be updated without special equipment? Each “no” is a coverage gap or a debug blind spot you will pay for at volume.

9.2 The 10 $\times$ Rule and Test-Coverage Allocation

The cost to find and fix a defect rises $\sim 10\times$ at each phase it escapes to. This single curve drives the entire test strategy.

Caught at	Rough relative cost	Why
PCBA	1x	Rework a single board in the line, often automatically
Module	10x	Disassemble enclosure/heatsink, rework, re-test
System	100x	Tear down an integrated system, isolate which module
Vehicle	1000x	Pull compute from a vehicle, diagnose in situ
Field (RMA)	10,000x+	Truck roll, downtime, brand/safety risk

The economic version of the same curve (the classic semiconductor framing): wafer test $\sim \$1$, package test $\sim \$10$, board test $\sim \$100$, system test $\sim \$1,000$, field $\sim \$10,000+$. The numbers are illustrative; the *order of magnitude per phase* is the durable point.

The allocation skill. When Electrical Engineering (EE) hands you a new board, the expert question for *each way it can fail* is: what is the cheapest phase that can catch this, and what test detects it there? You build a small matrix:

Failure mode	Catchable at	Test
Missing/wrong component	PCBA	AOI / ICT
BGA solder void under the GPU	PCBA (X-ray) + Module (thermal cycling surfaces it)	AXI; thermal soak + link-error monitor
PCIe lane marginal at temperature	Module (not PCBA)	Stress + AER + lane margining, hot and cold
NVMe throttles under sustained write	Module	<code>fio</code> soak + SMART temperature/throttle log
GMSL won't lock over full harness	Vehicle (real harness)	Link-lock + frame capture at End-of-Line (EOL)
Inter-module link marginal	System	System link enumeration + stress

The expensive mistakes are defects that are only *detectable* late but were *introducible* early — a workmanship defect on a SerDes trace that doesn’t show until a hot system soak. You will not get this matrix perfect on day one. Thinking in it is the point: **test coverage is a placement problem.**

9.3 Design Verification vs Manufacturing Test, In Depth

Design Verification (DV) and Manufacturing Test (MT) share instruments, code, and physics but differ in goal, statistics, and consumer. (The framing is introduced in the Platform chapter, on DV vs MT; this is the working depth.)

Dimension	DV	MT
Question	How good is the <i>design</i> ? Where are its margins/edges?	Is <i>this unit</i> good enough — and fast?
Output	Characterization data, margin maps, the limits themselves	A go/no-go verdict (+ a few captured parameters)
Sample size	Small N (EVT ~20-50; DVT ~50-500)	Every unit (PVT ~300-2,000, then full volume)
Method	Characterization, shmoo, margining, corner/stress sweeps	Go/no-go against fixed limits, fast
Conditions	Voltage/temp/frequency corners, worst-case combos	Nominal (+ targeted stress where a defect demands it)
Time budget	Hours-days per unit acceptable	Seconds-minutes per unit (takt-bound)
Run by	Test/EE engineers in the lab	Operators on the line / at the CM
Statistic	Distribution shape, design margin, C_{pk} of the <i>design</i>	FPY, escape/false-fail rates, C_{pk}/P_{pk} vs limits

The build-phase vocabulary maps onto this. EVT (Engineering Validation, ~20-50 units, “does it meet functional requirements”) and DVT (DV, ~50-500 units, “can it be *manufactured* to spec” — heavy characterization/margining) are DV work. PVT (Production Validation, ~300-2,000 units, “can the *line* hit its metrics”) is where MT is proven out before mass production runs it.

9.3.1 Shmoo, margining, go/no-go

- A **shmoo plot** is a 2-D pass/fail map across two operating parameters (classically supply voltage $\times$ clock frequency), shading where the part works. It is a *design characterization* tool — it shows the design is stable across process so it “can be manufactured with virtually zero yield loss.” You produce shmoos in **DV**; you do **not** shmoo every unit on the line.
- **Margining** is the continuous-parameter cousin: step an operating point (sampling time/voltage, a Transmit (TX) preset) until errors appear and record *how much margin* there was. In DV you margin across corners to characterize; in MT you margin once at nominal and compare to a limit. PCIe **lane margining** (PCIe chapter) is exactly this.
- **Go/no-go** is the MT default: run, compare each measured value to its limit, emit PASS/FAIL. Fast, repeatable, operator-runnable.

9.3.2 The unifying idea — capture the parameter, not just the verdict

This is the single most important design principle for your test code:

Capture the parameter, not just the verdict. In DV you sweep and *plot* the captured parameter (the shmoo, the margin-vs-temperature curve). In MT you compare that *same* captured parameter to a limit for a fast pass/fail. **Same measurement code, same captured field; the only difference is whether you sweep-and-plot (DV) or compare-to- limit (MT).**

Concretely:

- A **BERT** measures (**errors, bits**) $\rightarrow$ a Bit Error Rate (BER) upper bound. **DV**: run it across voltage/temperature corners and Transmit (TX) presets and *plot the surface*. **MT**: run it once to a confidence target (prove $\text{BER} < 1\text{e-}12$ at 95% and stop) and emit pass/fail. Same engine. (Confidence math: Math chapter, the BER/BERT confidence section.)
- **Lane margining** yields a per-lane **timing margin in UI**. **DV**: sweep it across temperature to characterize the eye and *set* the limit. **MT**: compare the one nominal number to that limit.

The lane-margin number is the bridge: DV uses it to *set* a data-driven per-lane eye limit; MT uses it to *check* that limit per unit — replacing “pass on link-up” with a margin number. That is why MT must capture parameters: the captured stream is what later feeds SPC, C_{pk} , and guard-banding. **You cannot set a good limit on data you didn’t keep.**

9.4 Test Limits — Guard-Banding and Cpk-Driven Limits

A test limit is a number that turns a measurement into a verdict. Setting it well is where false-fails and escapes are won or lost. The derivations (capability indices, PPM-from- C_{pk} , the normal tables) are in the **Math chapter**, in the capability section; here is the *engineering* of it.

9.4.1 Guard-banding — tighten the test limit inside the spec limit

The spec limit is the customer/design requirement (e.g., “rail must stay ≥ 11.4 V under load”). Your **measurement is not perfect** — it has gauge uncertainty (; Gage R&R derivation in the Math chapter). If you test directly at the spec limit, gauge error will pass units that are truly out of spec (an escape) and fail units that are truly in spec (a false fail). A **guard band** moves the *test* limit *inside* the *spec* limit by an amount tied to that uncertainty:

```
test_limit = spec_limit - guard_band
guard_band ~= k * sigma_gauge # k from the confidence you need (often ~2-3)
```

So a 11.4 V spec floor with a 50 mV gauge uncertainty might get a 11.55 V *test* floor. The guard band trades a little yield (some good-but-marginal units fail) for protection against the escape — and in a safety product that trade is correct. The guard band comes *out of the Gage R&R study*: you cannot pick $k * \text{sigma_gauge}$ until you have measured `sigma_gauge`.

9.4.2 Cpk-driven, data-driven limits

Do not pull a limit from a guess or a datasheet round number. Set it from the *measured fleet distribution*, then check that the limit gives an acceptable capability:

1. **Capture the parameter on many units across corners (DV)**. This is the EVT/DVT characterization data.
2. **Build the distribution**. Mean, spread, shape, tails.
3. **Set the MT limit from the distribution + a guard band**. Place it so the process sits comfortably inside it — i.e. so C_{pk} is healthy. *Example phrasing you will actually write in a limit-justification doc*: “fleet timing margin is 0.42 ± 0.04 UI; a 0.25 UI limit gives $C_{pk} \approx 1.4$ with room for gauge error.”
4. **Monitor with SPC and re-tune when the process moves**.

The capability bars you target (Math chapter, capability section): $C_{pk} \geq 1.33$ is the industry “capable” floor (~32 PPM one-sided), ≥ 1.67 is strong, ≥ 2.0 is world-class. A **low C_{pk} is itself a finding**, not a limit problem: it means the process spread and the spec are too close, and you will bleed yield *no matter how good the test is* — hand that back to EE/process as “the design margin is too tight,” not “loosen the limit.” Note also P_p/P_{pk} vs C_p/C_{pk} : use the long-term P_{pk} in DVT/PVT *before* the process is proven stable, C_{pk} once control charts show stability (Math chapter, capability section).

9.5 Per-Unit Data, Traceability, and SPC

Every unit a station tests must leave a record richer than PASS/FAIL. That record is the raw material for limit-setting, yield analysis, SPC, Return Merchandise Authorization (RMA) root-cause, and the safety case.

9.5.1 Traceability and genealogy

Every unit carries a **serial number** (1-D/2-D barcode, Direct Part Marking, or RFID), scanned to *start* the test — which both removes keystroke error and stamps each result row with **which unit, which station, which test-program version, when**. **Genealogy** records which component lots and sub-assembly serials went into each finished unit, so when a bad lot or a failing test mode appears you can trace *every affected unit* quickly (top-down: “which units got lot X”; bottom-up: “what went into this failing unit”). For Zoxx compute this is also a *quality* lever — it is how logging a NVMe drive’s **power_on_hours / data_units_written** catches re-labeled or returned stock entering the line even when the unit otherwise passes (NVMe chapter). And it is a **safety-case requirement**: serial → genealogy → program version → measured results → disposition must be an auditable chain (functional-safety chapter).

The structured result a station emits per unit:

```
station_id, dut_serial, test_program_version, operator, timestamp,
per_test: { name, measured_value, limit_low, limit_high, result },
artifacts_on_failure: { decoded_AER, dmesg_snippet, margin_matrix, ... }
```

A test that records only PASS/FAIL throws away the data you need for SPC and limit-setting — and forces the next engineer to *reproduce* a failure to diagnose it instead of reading its attached evidence.

9.5.2 Measurement-system trust — Gage R&R (MSA)

Before you trust *any* limit, prove the *measurement system*. **Gage R&R** decomposes observed variation into the gauge versus the part:

- **Repeatability** — same operator, same part, same equipment, repeated. Variation = equipment noise.
- **Reproducibility** — different operators/stations, same part. Variation = operator/station-to-station.

The AIAG study is **10 parts × 3 operators × 3 repeats** (90 measurements). Acceptance: **%GRR < 10%** good, **10-30%** conditional, **> 30%** unacceptable (and **ndc >= 5**). If gauge variation is large relative to the tolerance, *your pass/fail is noise*. This is where **sigma_gauge** for the guard band comes from, and — crucially — **cross-CM correlation is a reproducibility study across sites**. (Derivation and the variance math: Math chapter, Gage R&R section.)

9.5.3 SPC — the process talks to you before it makes scrap

Capability (C_{pk}) is a snapshot; **SPC** is the movie. Plot each captured parameter over time with center line and $\pm 1/2/3\sigma$ zones. For per-unit test data the natural chart is the **I-MR** (individuals / moving-range) pair. The control limits are the *process’s own* voice (mean $\pm 3\sigma$), **not** the spec limits — a key distinction:

A process can be **in control yet not capable** (stable but too wide for the spec), or **capable yet out of control** (fits the spec today but drifting). Control charts and capability are a pair; you need both.

The **Western Electric rules** flag special-cause variation *before* it becomes scrap (Math chapter, Western Electric rules): 1 point beyond 3σ ; 2 of 3 beyond 2σ (same side); 4 of 5 beyond 1σ (same side); 8 in a row on one side. (A 6-in-a-row monotonic trend is a *Nelson* rule, not Western Electric.) **Reading the yield chart**: a *sudden* drop says process change, equipment failure, or bad incoming material; a *gradual* decline says drift — tool calibration, fixture wear. The chart points; Root Cause Analysis (RCA) finds the cause.

9.5.4 Cross-site correlation with golden units

Keep **golden units**: characterized **known-good** and **known-bad** references. Two uses:

- **Release gate.** Before and after every release, prove the test still **passes the known-good and fails the known-bad**. This catches a too-tight (false-fail) or too-loose (escape) change *before* the line, not on it.
 - **Cross-station / cross-site correlation.** Run the *same* golden unit at Zoox and at the CM; the stations must agree. A correlation gap is a fixture / calibration / environment difference you must resolve before you trust their yield numbers. Operationally this *is* a Gage R&R reproducibility study across sites.
-

9.6 Yield and Test Economics

These are the three numbers a test engineer is judged on, and the JD names them: **test deployment, test runtime, and yield**. (Yield/RTY/throughput/cost-of-test derivations: Math chapter, yield and cost-of-test.)

9.6.1 Yield

- **FPY** — fraction passing the *first* time, no retest/rework. The primary KPI.
- **RTY** — product of FPY across all steps. Five steps each at 98% $\rightarrow 0.98^5 \approx 90\%$. **This is why every added test step costs yield**, and why you do not add coverage casually.
- **Pareto analysis** — bar chart of failure modes by frequency with a cumulative line. 80% of failures come from ~20% of causes; **always attack the tallest bar first**.

A test program hurts yield in two opposite ways: **false fails** (good units failing — gauge noise, too-tight limits, flaky tests) and **escapes** (bad units passing — too-loose limits, missing coverage). The whole limit/guard-band discipline is about minimizing both.

9.6.2 Runtime — test time is money

Test time sets line throughput and station count. **Takt time** = available production time $\div$ required output is the drumbeat the line must hit; **your test's cycle time must fit inside takt** or the station becomes the bottleneck and you need more stations (more capital). Scale intuition: a station producing one record per cycle at a 30 s takt generates ~100,000 records/month — that sets the data-volume scale you design the results store for. The levers:

- **Parallelize.** Run independent tests concurrently — NVMe `fio` soak || GPU `gpu-burn` || PCIe AER monitor — and test **N DUTs at once** on one station (multisite / multi-up). This is how you amortize a fixed soak across throughput.
- **Right-size soaks.** The **BERT confidence target** is an *economic* tool: run exactly long enough to prove 1e-12 at 95% confidence and **stop**, not a padded fixed duration (Math chapter, BER/BERT confidence). Choosing a confidence level instead of a wall-clock time is a runtime optimization *with a statistical guarantee*.
- **Adaptive testing.** If a unit passes a quick screen, skip the extended version.
- **Move tests left** — run a test at the cheapest phase that still catches its defect (balanced against the 10 $\times$ escape cost; never move a test earlier than the phase that can *catch* the defect).
- **Cut tests that never catch anything.** If the fleet data shows a 30 s test has caught zero real defects across 10,000 units, it is a candidate to drop or sample. The data tells you which tests earn their runtime.

9.6.3 The false-fail vs escape trade — asymmetric here

Normally false-fail vs escape is a pure economic optimization. **In a robotaxi it is asymmetric.** A **false fail** costs throughput, retest labor, and (at a CM) remote firefighting. An **escape** costs the 10×-per-phase curve *and*, for safety-critical compute, a field/safety event whose cost is effectively unbounded. So the standing rule is “**ship only good units**”: you do **not** buy yield by loosening a limit that lets a real defect through. You buy yield by *reducing false fails* — better limits via data + guard-banding, less gauge noise, less test flakiness — **never** by raising the escape rate.

9.6.4 Deployment

How fast and how reliably a new/updated test reaches every station and CM. Levers: config over code (no code change to retarget a board revision or a CM), versioned releases with rollback, golden-unit correlation gating a rollout, remote station update, and clear release notes. A test that takes a week to deploy to a CM is operationally *worse* than a slightly-less-thorough one that deploys in an hour — which is why deployment is a first-class metric, not an afterthought.

9.7 Bring-Up vs Production

The same hardware passes through two very different regimes, and the test engineer works both.

9.7.1 Board bring-up (a DV-adjacent activity)

When a new board revision arrives, you do a structured bring-up *before* production test development begins. The order is “fail fast, cheap first”:

1. **Visual inspection** — obvious assembly defects (missing parts, solder bridges, wrong orientation); verify the BOM matches the design.
2. **Power-on (the scary part)** — apply power with a **current-limited supply** and watch the current draw: a short pulls max current instantly. Verify every rail comes up to spec; check for components getting hot (thermal camera / touch test).
3. **Boot to console** — UART debug console; watch boot messages (BIOS POST, kernel, login). Each line that scrolls is another subsystem that came up; if it stops, the *last* message tells you what failed.
4. **Device enumeration** — `lspci -vvv`, `lsusb`, `ip link show`, `dmesg | grep -i error`; compare against the expected device list from the schematic.
5. **Basic functional** — each subsystem individually: GPU `nvidia-smi`, NVMe `nvme list`, NIC `ethtool`, memory `free -h / dmidecode --type memory`, CAN a test frame.
6. **Characterization** — stress, thermals, PCIe equalization, power under load. **This data informs the production test limits** — bring-up is where the DV characterization that *sets* limits happens.

For *custom* PCIe cards (the JD’s “custom PCIe devices”) you work from the schematic: which root port feeds the slot, is the slot **bifurcated** (a top cause of “device-not-detected” when BIOS bifurcation doesn’t match layout), where are the retimers, which rails feed the PHY. Bring-up is where you and EE are closest; your job is to produce evidence sharp enough that a layout or stuffing fix is *obvious* (PCIe chapter has the full bring-up checklist).

9.7.2 Production

The unit is no longer a question, it is a verdict. The test is locked, versioned, operator-run, takt-bound, and measured on FPY; the properties that matter flip from “deep and exploratory” to “fast, robust, repeatable, operator-proof, correlated across stations.” The *measurements* are often the same code as bring-up — the difference is sweep-and-characterize (bring-up) vs compare-to-limit-and-move-on (production): the DV→MT transition made concrete in the life of one board.

9.8 The Debug-to-Root-Cause Workflow

When a unit fails — or worse, when *yield* drops — you need a method, not a hunch. The goal is always to get from a **symptom** to a **physical root cause** you can hand to EE, process, or the supply chain as an actionable correction.

9.8.1 The reflex (single-unit failure)

```
failure
-> enumerate # is it even there? lspci / nvme list / nvidia-smi / ip link
-> dmesg # what did the kernel see? link-down, AER, XID, training
-> counters # arm -> stress -> read (AER / SMART / ECC / EDAC / TEC-REC)
-> isolate # swap / reseal / known-good unit / known-good slot
-> measure # scope the rail, DMM the current, thermal-force it
-> decode # which AER bit / XID code / DIMM label -> the layer -> the part
```

The meta-skill is not memorizing flags; it is the *reflex* and the discipline of **arming counters before you measure** (clear → stress → read) so you count errors from *your* stress window, not from boot. The interface chapters supply the decode tables (which AER bit means which layer; which XID code means which GPU subsystem; which Dual Inline Memory Module (DIMM) label a CE maps to). Many “digital” failures are really **power** / **SI** failures wearing a digital costume — the engineer who reaches for the scope *and* the AER decode together is the one who closes the hard intermittent bugs (Instruments / Power chapters). ### Structured RCA (yield drop / repeated failure)

When the problem is a *population*, not one unit, use a structured method so you find the true cause, not the first plausible one:

Fishbone (Ishikawa) — categorize candidate causes (the 6 M’s):

Category	Examples
Man (operator)	Training gap, wrong procedure, skipped step
Machine (equipment)	Fixture failure, instrument drift, cable wear
Material	Bad component lot, wrong revision, incoming quality
Method (process)	Procedure error, wrong test limit, missing test
Measurement	Instrument accuracy, Gage R&R failure, wrong probe point
Environment	Temperature, humidity, vibration, ESD

5 Whys — drill from symptom to root cause. A worked example:

1. Why did the GPU fail thermal test? → Temperature exceeded 90 °C.
2. Why did it exceed 90 °C? → Heatsink thermal resistance was too high.
3. Why was thermal resistance too high? → Insufficient thermal-paste coverage.
4. Why was coverage insufficient? → The stencil aperture was undersized.
5. Why was the aperture undersized? → The stencil spec wasn’t updated for the new heatsink design.

Root cause: stencil specification not updated. *Fix:* update the stencil spec, add incoming inspection for paste coverage. Notice the symptom (“GPU too hot”) and the root cause (“a document wasn’t updated”) are five steps apart — stopping at “bad heatsink” would have treated a symptom.

Pareto + 80/20 — attack the tallest bar first. When you have a list of failure modes, compute the cumulative percentage and fix the top contributors for the biggest yield gain per engineering hour:

Failure mode	Count	%	Cumulative %
PCIe link degraded	45	36%	36%
NVMe not detected	25	20%	56%
GPU ECC error	15	12%	68%
Thermal throttle	12	10%	78%
NIC link down	8	6%	84%
All others	20	16%	100%

Fixing the top two here (PCIe + NVMe = 56% of all failures) is where the leverage is. **Always verify the fix moved the bar** — re-Pareto after the corrective action; if the top bar didn’t shrink, you fixed a symptom, not the cause.

9.9 Reliability and Stress Screening

“Passes at 25 °C, fails at 85 °C” is the defining manufacturing-test reality, and it is why a thermal forcer / chamber is on the bench. Two physics facts to keep in hand:

1. **High-speed links lose margin with temperature.** Conductor loss and jitter rise with temperature, so a SerDes link (PCIe, GMSL, automotive Ethernet) that equalizes to an open eye at 25 °C can have a *closed eye* — errors, retrains, or a speed/width fallback — at 55-85 °C. **This is why lane margining and AER/ECC monitoring must be done hot**, not just at ambient, and why module-phase burn-in exists.
2. **Failure rate is Arrhenius in temperature** — temperature-activated mechanisms follow an exponential law; rule of thumb, **failure rate roughly doubles per ~10 °C**. Elevated-temperature life tests are processed *through* the Arrhenius equation to predict normal-temperature behavior. This is the theory under burn-in and the infant-mortality (left) side of the **bathtub curve** (Math chapter, reliability): early-life failures are screened by a powered, elevated-temperature soak so they happen *in the factory*, not in a vehicle.

The stress techniques and — the part people get wrong — **where each belongs** (HALT is a *design* tool; HASS/ESS/burn-in are *production* screens):

Technique	Stress	Applied to	Where in flow
HALT (Highly Accelerated Life Test)	Temp + multi-axis vibration <i>beyond</i> spec, to destruction	Prototypes (DV)	NPI / reliability — finds the design’s limits and <i>derives the HASS profile</i>
HASS (Highly Accelerated Stress Screen)	HALT-derived profile, near/just beyond operating limits	Production units	Production screen (post-assembly)
ESS (Environmental Stress Screening)	Thermal cycling + vibration <i>within</i> spec	Production units	Production screen — infant-mortality / workmanship escapes
Burn-in	Steady elevated temp, powered / under load, hours	Production units	Module/system — screens infant mortality
Thermal cycling	Repeated hot<->cold ramps	Both	DV reliability + production ESS — solder-fatigue / CTE-mismatch

Your contribution to every one of these is the in-soak functional monitor. The screen *precipitates* the latent defect (the oven/shaker supplies the stress); *your* test supplies the **at-temperature**

link/error/throttle checks — margin the lanes hot, watch the AER/ECC/EDAC/SMART/XID deltas vs temperature — that turn “we baked it” into “we baked it *and proved every interface still trains clean hot.*” A thermal test that never actually gets the part hot, or a soak with no functional monitor running during it, is a test that cannot catch the defect it exists for.

Electrostatic Discharge (ESD) discipline belongs here too: a fixture without proper ESD grounding makes *you* the failure mechanism — a board can pass test with latent ESD damage and die in the field. Wrist-strap monitors, dissipative mats, controlled humidity, ESD-safe fixture contacts; verify the strap monitors work on every station (Power/Safety chapter).

9.10 The CM Relationship: NPI to Mass-Production Ramp

Contract Manufacturers (CM) (EMS partners — Flex/Jabil-type) build at volume on units you may never physically touch. Releasing a test program *to* a CM and supporting it remotely is a large part of the job, and it spans the whole product life cycle: **New Product Introduction (NPI)** → **sustaining**.

9.10.1 NPI (New Product Introduction)

Test development, **first-article testing**, yield-target establishment. You work closely with the CM to validate that your test coverage works on *real* boards built on *their* line with *their* fixtures — not just on the golden unit in your lab. **FAI (First Article Inspection)** is the production-side gate: the first unit(s) off a new line/process/revision get a thorough, documented verification before volume is released, proving the line is set up correctly.

9.10.2 Sustaining

Ongoing test maintenance, yield improvement, test-time reduction, handling field returns, and updating tests for board revisions. The recurring ritual is the **yield meeting**: review the CM’s (CM) yield data, Pareto the failures, trend-analyze, and assign corrective actions. This is where the captured per-unit data and the Pareto/RCA discipline earn their keep against a partner you cannot stand next to.

9.10.3 The release package

A test program is a **released artifact**, versioned and tagged like firmware (the Bash/Linux chapter covers the git mechanics). What you hand a CM:

- **Versioned code + config** — config separate from code so the *same* code runs at Zoox and at the CM with different fixtures.
- **Setup / runbook** — how to provision a station, connect the fixture, run the program.
- **Fixture specification** — what hardware the station needs.
- **Acceptance criteria** — the limits and what each test proves.
- **Golden-unit correlation data** — the reference results their station must match.
- **Triage guide** — symptom → likely cause → first moves, so the CM can self-serve common failures instead of escalating every one.

Robustness and clear logs matter *ten times more* when you cannot walk over to the station. A failure must arrive with its evidence already attached so you can diagnose a board in another country from the result row.

9.10.4 MES / OEE — the system the line runs on

The CM’s (CM) floor runs on a **MES (Manufacturing Execution System)** that owns work-order release, electronic work instructions, serialization, genealogy/traceability, quality/NCR handling, and **OEE** (Overall Equipment Effectiveness = Availability × Performance × Quality). Your station typically **checks in/out**

with MES (is this serial allowed to test here? record the verdict back) and streams parameters to a test-data-management/analytics layer. Note how your three metrics map onto OEE: a station drags OEE down through **downtime** (Availability), **slow cycles** (Performance), and **false-fails/retests** (Quality). The factory-automation/SCADA layer you may meet here (Ignition, OPC-UA, PLC data) is the same distributed pattern as any station-dashboard system: stations POST status to a central server, structured data lands in a SQL database, a web frontend renders real-time yield/throughput/station-status dashboards.

The MES check-in/out handshake is worth making concrete, because it is the contract between your station code and the factory:

```
1. operator scans DUT serial
2. station -> MES: "may serial SN123 run test-program PCBA_v4.2 at station S07?"
3. MES -> station: ALLOW (correct routing, not already passed, work order open)
   or DENY (wrong step / already shipped / lot on hold / rework loop)
4. station runs the program
5. station -> MES: verdict + per-test results + program version + timestamp
6. MES advances the unit's route state (or routes a FAIL to rework/quarantine)
```

That handshake is what enforces **route control** (a unit cannot skip a station or be tested out of order) and what makes the per-unit record auditable. If your station does not check in, a unit can be re-tested until it passes by luck — the classic “test until pass” escape that route control exists to kill.

Why traceability is not optional here. For automotive compute the genealogy chain is a *compliance* requirement, not just a debugging convenience. Two standards drive it:

- **IATF 16949** (the automotive quality-management standard, built on ISO 9001) requires a traceability system that can identify product lots and tie them to manufacturing records, so a nonconforming population can be **contained** — i.e. given a bad component lot you can name every finished unit that received it, and given a failing unit you can name everything that went into it.
- **ISO 26262** (functional safety) requires end-to-end traceability across the safety lifecycle — each safety requirement linked through design, implementation, and verification. At the manufacturing layer this shows up as the demand that *every* unit’s serial → genealogy → test-program version → measured results → disposition is an auditable, durable chain. A “we think it passed” with no record is a safety-case hole.

Practically: this is why the per-unit record must be **immutable and retained for years** (often the vehicle’s service life plus a margin), why test-program versions are captured on every row (a result is meaningless without knowing which limits produced it), and why “test until pass” is forbidden. The infrastructure that stores all this — the parametric warehouse, the dashboards, the analytics — is the subject of the next two sections.

9.11 The Test Station and Its Instruments

Everything above assumes a *station*: the physical + software assembly that holds a DUT, applies stimulus, measures, and emits a verdict. Knowing the layers of a station — and which instrument makes which measurement — is what lets you turn “test the board” into a concrete bench.

9.11.1 The anatomy of a station

A production test station is a stack you can name top to bottom:

```
operator UI / barcode scanner # start-on-scan, PASS/FAIL light, retest control
|
test executive (sequencer) # runs steps, applies limits, logs results
|
test code (Python / C / drivers) # the measurement logic per step
```

```

|
instrument layer (the bench) # PSU, DMM, scope, BERT, thermal forcer, switch
| (controlled over GPIB / USB / LAN-VISA / PCIe / serial)
fixture / DUT carrier # power, signal, thermal, ESD contact to the DUT
|
DUT (the unit under test)

```

The **fixture** is where most station bugs live: a worn pogo pin, a marginal connector, a ground loop, or an ESD-grounding gap turns into “intermittent fails” that look like a DUT problem. When a station’s yield drops with no design change, suspect the fixture before the boards (it is a *Machine* cause on the fishbone,) and prove it with a **golden unit**: if the golden unit now fails or shifts, the station moved, not the DUTs.

9.11.2 Instrument control — how the code talks to the bench

Bench instruments are almost universally driven over **SCPI** (Standard Commands for Programmable Instruments — ASCII command strings like MEAS:VOLT:DC?) carried on a transport: **GPIB/IEEE-488** (legacy but everywhere), **USB-TMC**, **LAN/LXI** (often via **VISA** or raw sockets), or RS-232. In Python the common stack is **PyVISA** (or a vendor SDK) wrapping VISA; the pattern is the same regardless of instrument:

```

# open -> configure -> trigger -> read -> close, with explicit ranges and timeouts
import pyvisa
rm = pyvisa.ResourceManager()
dmm = rm.open_resource("TCPIP::192.168.1.50::INSTR")
dmm.timeout = 5000 # ms; a hung instrument must not hang the line
dmm.write("CONF:VOLT:DC 20,0.001") # fixed range + resolution => repeatable, fast
rail_v = float(dmm.query("READ?"))

```

Two production rules that separate a robust station from a flaky one: **never leave an instrument on autorange in production** (autorange re-hunts each reading — slow and non-repeatable; fix the range from the expected value), and **always set a timeout and handle the dead-instrument case** (a GPIB hang with no timeout stops the whole line).

9.11.3 Which instrument makes which measurement

Instrument	Measures	Where it shows up in this guide
Programmable PSU / electronic load DMM (6.5-digit)	Supply the DUT; sweep/limit V and I; sink current to load a rail DC rail voltage, current (shunt), resistance	Bring-up power-on; power-under-load characterization Rail-in-window checks, ICT-style continuity
Oscilloscope	Time-domain: ripple, rise/fall, clocks, glitches, eye diagrams (with the right probe/SW)	SI debug, ripple-vs-AER correlation, clock integrity
BERT / built-in eye+margining	Bit-error ratio and eye/timing margin on a SerDes lane	The PCIe/GMSL margining story
Protocol analyzer/exerciser	Decoded PCIe/CAN/Ethernet/USB traffic, inject + capture	Link bring-up, CAN bus-off, packet-level faults
Thermal forcer / chamber	Force the DUT (or a part) to a set temperature	The “passes at 25 C, fails at 85 C” screen
Thermal/IR camera	Surface temperature map; find the hot part	Power-on “what’s getting hot,” heatsink coverage
Switch / multiplexer matrix	Route one instrument to many nets or many DUTs	Multisite stations, sharing a costly instrument
Power analyzer / DAQ	Many channels of V/I/temp logged over a soak	Burn-in monitoring, power budgets

The deep how-to for each (probing, bandwidth, eye reading, thermal-forcer setup) lives in the **Instruments**

/ **Power** chapter; the point here is the *mapping* — a measurement implies an instrument, and a station’s bill of materials is just that mapping made physical. A recurring failure-analysis move is reaching for the **scope + the protocol decode together**: many “digital” failures are a power or SI problem wearing a digital costume, and you only see it when the rail trace and the error counter are on the same screen.

9.12 The Manufacturing-Test Tooling Landscape

A working test engineer does not write everything from scratch; you assemble a *stack* of tools, and a large part of the job is knowing which layer each tool belongs to and where the boundaries are. The mistakes here are category errors — running deep stats in the wrong tool, or letting a Continuous Integration (CI) server think it is a production sequencer. The layers, top to bottom:

```
yield analytics / SPC / deep stats yieldWerx, PDF Exensio, JMP, notebooks
^ (reads the parametric warehouse, not the line directly)
|
dashboards / monitoring Grafana
^
|
parametric data store + format Postgres/TimescaleDB, a warehouse; STDF on ATE
^
|
MES + traceability/genealogy route control, serialization, OEE
^
|
test executive (sequencer) NI TestStand -or- custom Python sequencer
^
|
test code + instrument drivers your measurement logic

--- separate lifecycle, NOT in the per-unit path ---
CI for the test *software* GitLab CI / GitHub Actions / Jenkins
```

9.12.1 The test executive (sequencer) — buy vs build

The **test executive** is the layer that owns *sequencing*: run steps in order, branch on results, apply limits, handle retries, log a structured result, and present an operator UI. You either buy it or build it.

- **NI TestStand** is the dominant commercial test executive. It gives you sequencing, built-in limit evaluation, parallel/multi-UUT models (run N DUTs at once on one station), operator interfaces, and report/result generation (HTML/XML/ATML/ASCII, or a database) for free. It calls test code written in Python, C/C++, .NET, or LabVIEW — so “TestStand vs Python” is a false binary; the common pattern is **TestStand sequencing with Python step modules**. It is heavily used on automotive End-of-Line (EOL) lines (e.g. ECU End-of-Line testers commonly pair LabVIEW/TestStand for sequencing and reporting). The cost is licensing and a degree of lock-in.
- **A custom Python sequencer** trades that out-of-the-box machinery for full control and no license cost. You get to own the data model, the result schema, and the deployment story — but you must *build* the parts TestStand gives you: looping/branching/retry logic, parallel-UUT execution, the operator UI, and result logging. For a Python-first shop testing custom hardware (custom PCIe cards with no vendor test plan), this is often the chosen path precisely because the flexibility matters more than the prebuilt sequencer — **pytest** is sometimes bent into this role for its fixtures and parametrize, though **pytest** is a *developer* test runner and a production line wants an operator UI, hard takt behavior, and an immutable result record on top.

The decision is the classic build-vs-buy: buy when your need is standard and you value time-to-line and vendor support; build when your hardware is unusual, your team is software-strong, and you need to own the

whole pipeline. Either way the executive’s job is the same — and either way it must emit the rich per-unit record of , not just a PASS/FAIL.

9.12.2 Parametric data formats — STDF and the warehouse

The captured parameters have to land somewhere with a schema, or they are not analyzable later. Two worlds meet here:

- **STDF (Standard Test Data Format)** is the semiconductor industry’s near-universal *binary* format for ATE (Automated Test Equipment) results — it stores parametric, functional, and datalog records (part records, test results with limits, bin results) and is what wafer/package test equipment emits. If Zoox compute touches die/package-level test data from a silicon vendor, or runs any ATE-style station, STDF is the lingua franca, and every yield-analytics tool ingests it. For board/module functional test you more often emit your *own* structured record into a relational/time-series store rather than STDF, but you should recognize STDF on sight and know it is parseable (open-source and vendor parsers exist).
- **A parametric warehouse** is where per-unit records accumulate for analysis: commonly **Postgres** (with **TimescaleDB** when the access pattern is time-series: station heartbeats, yield-over-time, soak telemetry), a column store / data-lake table for large parametric history, and **Prometheus** for short-retention operational metrics (station up/down, cycle time, queue depth). The shape that matters: **one row per (unit, test, parameter)** with limits attached, so any later question — “show the timing- margin distribution for last week’s lot,” “is rail ripple drifting on station 7” — is a query, not a re-test. (This is the data-volume scale sizes: a 30 s-takt station is ~100k records/month, ×N stations ×many parameters.)

9.12.3 Yield analytics and deep statistics

Above the warehouse sit the tools that turn stored parameters into yield decisions. These read the warehouse; they are **not** in the per-unit test path.

- **yieldWerx** and **PDF Solutions Exensio** are commercial yield-management / test-data-analytics platforms (Exensio is the bigger, fab-oriented one, with a stated automotive-semiconductor push; both ingest STDF and other formats). They provide automated SPC, parametric outlier/bin rules, wafer-map and cross-lot correlation, and the alerting that flags a yield signature before it becomes scrap. A board/module shop may not run a full fab-grade platform, but the *capabilities* — automated SPC on every parameter, outlier detection, cross-site correlation — are the target, whether bought or built on the warehouse + Grafana + notebooks.
- **JMP** (from SAS) is the analyst’s bench for *deep* statistics — the tool you open to do the work the dashboard cannot: a real Gage R&R study, a DOE to find why a parameter drifts, distribution fitting and capability analysis, a regression to correlate a failure with a process variable. Grafana answers “is something wrong, now”; JMP (or a Python/**pandas+statsmodels** notebook) answers “*why*, with statistical rigor.” They are complementary: monitoring is continuous and shallow, JMP is occasional and deep.

9.12.4 Version control for test programs

A test program is not a script you run once — it is a **released, versioned artifact** that deploys to multiple identical stations, at Zoox and at contract manufacturers you may never physically visit, and decides whether a safety-critical unit ships. Three forces make version control stricter here than in ordinary application development:

1. **Traceability (a safety-case requirement).** Every result row a station writes must record *which program version* produced it. When a field issue or a bad lot surfaces, you trace serial → genealogy → test-program version → measured results → disposition. If “the test version” is “whatever was on the laptop that day,” that chain is broken and the safety argument collapses.

2. **Reproducibility across sites.** Many identical stations run the same version and report to the same dashboard. A yield difference between Zoox and a CM must be a *fixture/calibration* difference, not a *code* difference — which you can only assert if you can prove both ran the same tagged commit.
3. **Deployment and rollback are first-class.** “Push v2.4.1 to all stations, then revert to v2.4.0 if FPY drops” must be a one-command operation. A clean tag-and-release process is what makes that safe.

Daily workflow. Review before you stage — `git add -p` forces a hunk-by-hunk pass that stops a stray debug print or a hardcoded station IP from shipping.

```
git status # working-tree state: staged / unstaged / untracked
git diff / git diff --staged
git add -p # interactively stage hunks, REVIEWING each change
git commit -m "Raise NVMe fw-activate reset wait to 10s; Micron 7450 needs ~8s to re-enumerate"
git push origin feature/gmsl-timeout
```

Write commit messages a CM engineer can use at 2 a.m. during a line-down: *what changed and why the value is what it is*. “Fix bug” is useless; the message above is a debugging document.

Branching and review discipline for a manufacturing-test repo:

Element	Practice	Why
main	always deployable to production	a station can be re-provisioned from main to a known-good state at any time
feature branches	one per driver, board revision, or test change	isolates in-progress work from the deployable tip
pull requests	review before merge	a second set of eyes on a change that can scrap good units or pass bad ones
CI	pytest + lint on every PR	a red gate blocks merge (see the next section)
golden-unit gate	re-run known-good + known-bad references before release	catches a too-tight (false-fail) or too-loose (escape) change <i>before</i> the line, not on it
tags	mark each version deployed to the line	the traceability anchor

Config over code. Limits, bus/topology maps, station IDs, and CM-specific fixture settings live in versioned *config*, not in the Python. A retarget to a new CM or board revision is then a config change, not a code release that re-qualifies the whole program.

Tags turn a commit into a release. Prefer annotated tags (they carry tagger, date, and message, and are what you sign):

```
git tag -a v2.4.0 -m "Release 2.4.0: add MAX96712 quad-cam config, hot-margin path"
git push origin v2.4.0
git describe --tags # "v2.4.0-3-gA1B2C3D" -- embed in every result row so even an
# unreleased dev build is uniquely identifiable
```

Semantic versioning maps cleanly onto test programs: **MAJOR** = incompatible change (new limit schema, dropped test, new result format the dashboard must understand), **MINOR** = added coverage or board config (backward compatible), **PATCH** = bug fix or limit re-tune within the same schema. The shipped artifact is more than code: versioned code + config + runbook + fixture spec + acceptance criteria + golden-unit correlation data + triage guide, with the git tag as the spine that proves what was shipped.

Recovery and forensics:

```
git log --oneline --graph # branch/merge history at a glance
git blame limits.yaml # who set this limit, when, in which commit
```

```
git revert <hash> # NEW commit that undoes <hash> -- SAFE on shared branches
# (does not rewrite history); how you roll back a bad release
git bisect start # binary-search the commit that introduced a regression
```

Rule of thumb: never `reset --hard` or force-push a branch a station or CM might be pulling from — use `revert` for shared history. Rewriting history is fine only on a private feature branch you have not shared.

9.12.5 CI for the test *software* — a separate lifecycle

This is the category error to avoid, so it gets its own callout. **Continuous Integration (CI — GitLab CI, GitHub Actions, Jenkins)** belongs to the *software development lifecycle of the test code*, **not** to per-unit production execution. The test program is a released artifact; Continuous Integration is what builds, unit-tests, lints, packages, and versions that artifact when you push a change — and ideally runs it against a **golden unit** on a hardware-in-the-loop runner as a release gate before it is allowed to ship to a station. What CI does **not** do is run on every DUT on the line: the **test executive** does that, takt-bound, on the factory floor. Jenkins building your test program nightly is correct; Jenkins being asked to test 10,000 units per the takt clock is a category error. Keep the two mental models separate:

	Test executive (TestStand / custom)	CI (Jenkins / GitLab CI / Actions)
Runs	Per DUT, on the line, at takt	Per code change / nightly
Triggers	Operator scans a serial	A git push / a schedule
Output	A per-unit PASS/FAIL + parametric record	A built, tested, versioned test-program release
Lives on	The station / factory floor	A build server
Hardware	The fixture + bench + DUT	Usually none, or one golden-unit HIL runner

9.13 Dashboards and Grafana for Manufacturing Test

You cannot manage a line you cannot see. Once stations emit the per-unit record into a store, the **dashboard** is how the data becomes a live picture of fleet health — and **Grafana** is the de-facto open-source tool for it. This section is concrete because “we’ll put up a dashboard” is where a lot of test-data value is won or lost.

9.13.1 What Grafana is, and exactly where it sits

Grafana is an open-source visualization-and-alerting front end that queries one or more data sources and renders panels (time series, stat tiles, tables, bar/Pareto, heatmaps) into dashboards, with an alerting engine on top. It **stores no data itself** — it sits on top of whatever you already write results into. Place it precisely:

```
station -> result record -> data store -> Grafana (read-only views + alerts)
^
MES owns route control & genealogy;
Grafana visualizes; it does NOT gate a unit.
```

The boundary that matters: **Grafana is observation, not control**. The **test executive** decides PASS/FAIL on a unit; the **MES** decides whether a unit may proceed; **Grafana** tells *humans* how the line and the fleet are doing so they can intervene. A Grafana panel never passes or fails a board — confusing the dashboard with the gate is a real mistake. It is the “web frontend renders real-time dashboards” layer names, made specific.

9.13.2 The data sources it sits on

Grafana speaks to several backends at once, and a real MFG-test setup uses more than one:

- **Postgres / TimescaleDB** — the primary parametric + result store. TimescaleDB (a Postgres extension) is the natural home for the *time-series* views (yield-over-time, station cycle time, soak telemetry) because it gives time-bucketing, continuous aggregates (pre-rolled-up summaries for fast long-range queries), and per-metric retention. Most yield/parametric panels are plain SQL against this.
- **Prometheus** — short-retention *operational* metrics scraped from stations: station up/down, cycle-time gauges, queue depth, instrument errors. Prometheus is for “is the line healthy right now”; it is not where you keep a year of parametric history (that is the warehouse). The Prometheus/Grafana pairing is the standard operational-monitoring stack.
- **A parametric warehouse / data lake** — for deep historical parametric queries across millions of units, sometimes fronted by its own SQL engine.
- **Loki (logs)** and occasionally other sources — to pull a failing unit’s `dmesg`/log snippet next to its result row.

The practical pattern: **TimescaleDB/Postgres for parametric + yield, Prometheus for live station ops, one Grafana** stitching them into role-specific dashboards.

9.13.3 The panels that actually matter

A useful MFG-test Grafana deployment is usually a few focused dashboards, not one giant wall. The panels that earn their place:

- **Live fleet & per-station FPY.** Today’s first-pass yield overall and broken out by station and by product, as stat tiles + a trend line. This is the number the line is run on. Example (TimescaleDB SQL, last 24 h FPY by station):

```
-- first-pass yield = first-attempt PASS / first attempts, bucketed for a trend panel
SELECT time_bucket('1 hour', first_seen) AS t,
       station_id,
       100.0 * sum((first_result = 'PASS')::int) / count(*) AS fpy_pct
FROM (
  SELECT DISTINCT ON (dut_serial) dut_serial, station_id,
         result AS first_result, ts AS first_seen
  FROM test_runs
  WHERE ts > now() - interval '24 hours'
  ORDER BY dut_serial, ts -- earliest attempt per unit = "first pass"
) first_attempts
GROUP BY t, station_id
ORDER BY t;
```

- **Station heartbeats / liveness.** When did each station last report a result? A station that has gone quiet is either starved (no units) or down (Availability,) — either way you want a tile that flips red. Example with Prometheus:

```
# alert when a station hasn't pushed a result in 15 minutes during a shift
time() - max by (station_id) (mfgtest_last_result_timestamp_seconds) > 900
```

- **Failure Pareto.** A bar chart of failure modes by count for the selected window, so the tallest bar is obvious at a glance. This is the yield-meeting view made live:

```
SELECT failure_mode, count(*) AS n
FROM test_runs
WHERE result = 'FAIL' AND ts > now() - interval '7 days'
GROUP BY failure_mode
ORDER BY n DESC; -- render as a sorted bar panel; optionally add a cumulative line
```

- **Parametric SPC / control charts.** Per-parameter I-MR-style control charts with the process’s own center line and $\pm 3\sigma$ zones — *not* the spec limits. A timing-margin or rail-voltage panel with control limits drawn lets you *see* a Western Electric violation (a run, a 2-of-3-beyond- 2σ) before it makes scrap.
- **Parametric drift / distribution.** A heatmap or time-series of a key parameter’s distribution (e.g. PCIe per-lane margin, NVMe soak temperature) over days/lots. A slow slide of the mean is the *gradual* signal calls out — tool wear, fixture aging, incoming-material shift — visible long before yield drops.
- **Throughput / cycle time vs takt.** Units/hour and per-station cycle time against the takt line, so a station drifting toward the takt ceiling (a future bottleneck) is visible before it actually blocks the line.

9.13.4 Alerting — turn the dashboard into a pager

A dashboard nobody is staring at is useless at 2 a.m. Grafana’s alerting engine evaluates rules on the same queries and routes notifications (Slack/PagerDuty/email). The alerts a MFG-test setup wants are the *Western-Electric-on-the-fleet* analogs:

- **Yield-drop alert** — FPY on any station falls below a floor (a *sudden* drop = process change / equipment / bad material,) → page the on-call test engineer.
- **Station-down alert** — no result in N minutes during a shift (the heartbeat query above).
- **SPC-violation alert** — a control parameter trips a Western Electric rule (point beyond 3σ , or a run), catching *drift* before it becomes a yield event.
- **New-failure-mode alert** — a failure mode that was rare this month suddenly climbs the Pareto.

The discipline mirrors : tune alerts so they fire on *real* signal, not noise — a flapping yield alert that pages on normal small-N variation gets muted, and then the real event is missed. Alert thresholds are limits too, and they earn the same data-driven, guard-banded treatment as a test limit.

9.13.5 Grafana vs the test executive vs the MES — the one-paragraph map

Hold these three apart, because the interview-grade (and the on-the-job) clarity is in the boundaries: the **test executive** runs the test and decides PASS/FAIL *per unit*; the **MES** owns route control, serialization, and genealogy and decides whether a unit may *proceed*; **Grafana** (this section) reads the resulting data and shows *humans* how the line and fleet are trending, and pages them when something moves. Executive = verdict, MES = routing + traceability, Grafana = visibility + alerting. They share the per-unit record as the common substrate, but only the first two are *in* the production control path.

9.13.6 What is publicly known about Zoox and AV-compute manufacturing test

A grounding note, deliberately hedged — treat the specific *internal* tool choices below as **unverified**; what is public is the shape, not the stack:

- **Zoox builds in-house at scale.** Zoox opened a ~220,000 sq ft robotaxi production facility in Hayward, California (publicly reported 2025), described as a serial-production line designed to scale toward ~10,000 vehicles/year. Public descriptions say the site houses robotaxi engineering, hardware/software integration, component storage, and **end-of-line testing** before deployment — i.e. the vehicle/End-of-Line (EOL) phase is done by Zoox, on-site, which is consistent with the JD’s “test solutions for manufacturing the compute platform.”
- **Vehicle-level End-of-Line is physical and sensor-centric.** Reporting confirms a sensor **calibration bay** (aligning all sensors to one world model) and an **outdoor test track** (drive-quality / build verification). Beyond those two confirmed steps, *typical* automotive vehicle-EOL also includes optical/lighting checks and a water-ingress (rain) test — plausible here, but not something I could source for Zoox specifically, so treat them as the general EOL pattern rather than confirmed Zoox steps. Either way it maps onto the EOL realities this chapter names: real sensors, real harnesses, environmental checks — the GMSL-over-full-harness and end-to-end streaming that *only* exist at the vehicle.

- **The compute itself is “data-center parts in a car.”** The role and Zoox’s public description point at server-grade compute assemblies, **custom PCIe devices**, networking, storage and memory — which is exactly the module/system test surface (-) the guide centers on.
- **Two real signals about the data layer — and the honest limits.** Two things *are* publicly visible. (1) A Zoox **Test Infrastructure Engineer** job posting describes **implementing RESTful APIs to provide access to manufacturing data** and integrating test infrastructure across teams — i.e. Zoox exposes its test/manufacturing data as a service (the same REST shape the companion **toolkit**/dashboard uses). (2) A **zoox.grafana.net** org instance exists, so Zoox runs Grafana *somewhere* — but it is access-gated and there is **no public confirmation it is used for manufacturing test specifically**, so treat “Zoox runs Grafana on the line” as a reasonable expectation, not a sourced fact. Beyond those two signals the stack is **not public**, so reason from verifiable industry practice: automotive EOL lines widely use a test executive (NI TestStand/LabVIEW common) for sequencing and reporting; automotive electronics manufacturing is governed by **IATF 16949** (quality/traceability) and **ISO 26262** (functional-safety traceability), which force the genealogy chain; and Grafana on a Postgres/TimescaleDB + Prometheus stack is the standard pattern for live yield dashboards. Honest framing in a design review: *“Zoox exposes manufacturing data via REST APIs (per their own job posts) and runs Grafana; the rest of the pipeline isn’t disclosed, but the industry-standard shape is a sequencer → MES → parametric warehouse → Grafana under IATF 16949 / ISO 26262 traceability.”*

Do not assert internal Zoox tool names you cannot source. “I’d expect *X* because it is the industry norm, and here’s why” is a strong answer; “Zoox uses *X*” (when you cannot cite it) is a weak one.

9.14 Putting It Together — The Expert Mental Model

Strip the chapter to its load-bearing sentences:

- **Place each test at the earliest phase that can catch its defect** — coverage is a placement problem, governed by the 10× curve and by what is physically detectable where (-).
- **DV sets the limits; MT checks them — same measurement code.** Capture the parameter, not just the verdict, because the captured stream is what tunes the limits and proves the safety case.
- **Set limits from data + a guard band, sized by the Gage R&R.** A low C_{pk} is a design/process finding, not a reason to loosen the limit (-).
- **Yield is FPY/RTY; runtime is takt; deployment is config-over-code + rollback.** Buy yield by killing false-fails, never by raising the escape rate — because for a robotaxi the trade is asymmetric.
- **Debug to a *physical* root cause** with the enumerate → dmesg → arm/stress/read → isolate → measure → decode reflex, and use Pareto/5-Whys/fishbone on a population.
- **Bring-up characterizes and *sets* limits; production *checks* them, fast and correlated, at Zoox and at the CM across the NPI→sustaining life cycle (,).**
- **Know the stack and its boundaries:** a sequencer runs the test, the MES routes and traces the unit, the warehouse keeps every parameter, Grafana shows humans the trend, and Continuous Integration (CI) versions the test *software* — never the units (-). Most failures of a test *organization* are category errors between these layers.

That is the JD said back in one paragraph — and every later chapter is the depth behind one of these sentences.

Chapter 10

Math, Probability, and Statistics for Test

This is the working math of the test floor. Not interview trivia — the formulas you reach for when you set a limit, size a sample, pick a confidence target, or defend a yield/escape number in a quality review. Every section ties a method to a decision: *where do I put the spec line, how many units do I pull, how long do I run the Bit Error Rate (BER) soak, when do I stop the line.*

The companion **PCIe chapter** carries the link-test detail; this chapter carries the statistics that turn a Bit Error Rate Test (BERT) run, a Cpk study, or an Statistical Process Control (SPC) chart into a pass/fail call you can sign your name to. The opening sections — probability, distributions, and the sigma bridge — fix the probability and distribution vocabulary once; read them first. The middle sections (geometry/sensor FOV, logs and dB, the GT/s $\rightarrow$ GB/s bandwidth math, vectors, and the core physical relationships) are supporting reference math you reach for less often but want defensible when you do. The daily tools are the back half: **BER and confidence**, **process capability and limits**, **SPC**, **gauge R&R**, **sampling and AQL**, **yield and throughput**, and **reliability**. Sections cross-reference each other by name throughout; the one-page formula sheet at the end collapses the whole chapter into something you can pin to a station.

10.1 Probability Fundamentals

Everything downstream — Cpk, control limits, AQL, BER confidence — is a probability statement dressed in engineering units. Get the four rules right and the rest follows.

10.1.1 The rules you actually use

- **Sample space S** — every outcome. One board test: $S = \{\text{pass}, \text{fail}\}$.
- **Probability** of equally likely outcomes: $P(A) = |A|/|S|$.
- **Axioms:** $0 \leq P(A) \leq 1$; $P(S) = 1$; $P(\emptyset) = 0$.
- **Complement:** $P(A^c) = 1 - P(A)$. If $P(\text{defective}) = 0.03$ then $P(\text{good}) = 0.97$. On a high-yield line you almost always compute the rare event through its complement — it is numerically stabler.

Addition (OR). General: $P(A \cup B) = P(A) + P(B) - P(A \cap B)$. Drop the overlap term **only** when A and B are mutually exclusive. $P(\text{GPU fails OR Non-Volatile Memory Express (NVMe) fails})$ subtracts $P(\text{both})$ so you do not double-count the boards that fail both.

Multiplication (AND). General: $P(A \cap B) = P(A)P(B | A)$. Drop the conditional **only** when A and B are independent. *If GPU and NVMe failures are independent, $P(\text{both}) = 0.02 \times 0.01 = 0.0002$.*

Independence is not mutual exclusivity. Independent means knowing A leaves $P(B)$ unchanged. Mutually exclusive means A happening forces B not to ($P(A \cap B) = 0$). Two mutually exclusive events are therefore *dependent* — learning A occurred tells you B did not. Mixing these up is the single most common probability error in a review.

10.1.2 Conditional probability and Bayes — the “defective given a FAIL” engine

$$P(A | B) = \frac{P(A \cap B)}{P(B)}.$$

This reversal is the workhorse of test statistics. The test gives you $P(\text{FAIL} | \text{defective})$ (sensitivity) and $P(\text{FAIL} | \text{good})$ (false-reject rate). What you actually need to make a disposition is the *reverse*: $P(\text{defective} | \text{FAIL})$. Bayes flips it:

$$P(A | B) = \frac{P(B | A) P(A)}{P(B)}, \quad P(B) = P(B | A) P(A) + P(B | A^c) P(A^c).$$

MSA vocabulary mapped to Bayes (memorize this row of equivalences — quality and medical-test language both show up at Zoox):

Test term	Meaning	Bayes piece
Sensitivity (true-positive rate)	P(FAIL given truly defective)	P(B given A)
Specificity (true-negative rate)	P(PASS given truly good)	P(B ^c given A ^c)
False-positive / false-reject rate	1 - specificity	P(B given A ^c)
PPV (precision)	P(defective given FAIL)	the answer Bayes returns
Prevalence	true defect rate	P(A)

Worked — the PPV problem. A test has 96% sensitivity and a 2% false-positive rate (98% specificity). True defect rate (prevalence) is 3%.

$$P(\text{FAIL}) = 0.96(0.03) + 0.02(0.97) = 0.0288 + 0.0194 = 0.0482,$$

$$\text{PPV} = P(\text{def} | \text{FAIL}) = \frac{0.0288}{0.0482} = 59.8\%.$$

The lesson that runs a high-yield line: even at 96% detection, only ~60% of your FAILs are real defects. The 2% false-positive rate acting on the *large* good population (97%) manufactures most of the alarms. On a mature line where prevalence is low, **false-positive rate dominates PPV, not sensitivity** — it is what drives your scrap-good cost and your retest burden. Tightening a flaky test limit to kill false rejects often buys more than chasing the last 1% of detection.

Worked — which station made the defect? Station A makes 60% of boards at 2% defect; Station B makes 40% at 5%. A board is defective — which station?

$$P(\text{def}) = 0.02(0.60) + 0.05(0.40) = 0.032, \quad P(B | \text{def}) = \frac{0.020}{0.032} = 62.5\%.$$

B makes fewer boards but the larger *share of the defects* — that is where you point the containment.

10.1.3 Counting (combinatorics)

- **Factorial:** $n! = n(n-1) \cdots 1$; $0! = 1$.
- **Permutations (order matters):** $P(n, k) = \frac{n!}{(n-k)!}$. Assign 3 priority tests across 8 stations:
 $8 \cdot 7 \cdot 6 = 336$.

- **Combinations (order does not):** $\binom{n}{k} = \frac{n!}{k!(n-k)!}$. Pull 3 boards from 10 for teardown: $\binom{10}{3} = 120$.
- **Identity:** $\binom{n}{k} = \binom{n}{n-k}$. **Handy:** $\binom{n}{2} = \frac{n(n-1)}{2}$; $\binom{5}{2} = 10$, $\binom{6}{3} = 20$.
- **Multiplication principle:** 4 GPU types $\times$ 3 NVMe $\times$ 2 Network Interface Card (NIC) = 24 configs.

The binomial coefficient is the $\binom{n}{k}$ that shows up in the binomial distribution and in every “how many of these n boards fail” calculation below.

10.1.4 Expected value and variance — and the rule that catches people

$$\mathbb{E}[X] = \sum_i x_i P(x_i), \quad \text{Var}(X) = \mathbb{E}[X^2] - (\mathbb{E}[X])^2, \quad \sigma = \sqrt{\text{Var}(X)}.$$

Properties you will use to roll up multi-stage test times and stacked tolerances:

$$\mathbb{E}[aX + b] = a \mathbb{E}[X] + b, \quad \mathbb{E}[X + Y] = \mathbb{E}[X] + \mathbb{E}[Y] \text{ (always),}$$

$$\text{Var}(aX + b) = a^2 \text{Var}(X), \quad \text{Var}(X + Y) = \text{Var}(X) + \text{Var}(Y) \text{ (independent).}$$

Variances add; standard deviations do not. $\sigma_{X+Y} = \sqrt{\sigma_X^2 + \sigma_Y^2}$, never $\sigma_X + \sigma_Y$. This is the same root-sum-square that governs tolerance stack-ups and uncertainty propagation.

Three independent test stages (μ, σ) : (5, 1), (10, 2), (5, 1.5) min. Total mean = 20 min; total variance = $1 + 4 + 2.25 = 7.25$; total $\sigma = 2.69$ min — **not** 4.5. If you quote the linear sum you over-budget your test-time spread by 67%.

10.2 Distributions and When to Reach for Each

The whole game is matching the physical situation to the right distribution. Pick wrong and your limits and confidence numbers are wrong. Here is the decision map, then the formulas.

Situation on the floor	Distribution	Why
Pass/fail of one unit	Bernoulli	single trial
# passing in a fixed batch of n	Binomial	n fixed, independent, constant p
# boards tested until first fail	Geometric	trials to first success
# rare events in a fixed window (defects/board, fails/shift, bit errors)	Poisson	rare, independent, constant rate
A measured continuous parameter (voltage, temp, time)	Normal	sum of many small effects (CLT)
Time between random failures (useful life)	Exponential	constant hazard rate
Time to wear-out failure (aging, NAND, fans)	Weibull	shape parameter bends the hazard

10.2.1 Binomial — counting failures in a batch

$$P(X = k) = \binom{n}{k} p^k (1-p)^{n-k}, \quad \mathbb{E}[X] = np, \quad \text{Var}(X) = np(1-p).$$

Test 20 boards, each 95% pass. Exactly 18 pass? $\binom{20}{18} (0.95)^{18} (0.05)^2 = 190(0.3972)(0.0025) = 0.189$ (18.9%). All 20? $0.95^{20} = 0.359$. At least 18? $0.189 + 0.377 + 0.359 = 0.925$ (92.5%). Use the binomial directly for accept-on-zero sampling and for redundancy/k-of-n reliability (the *Reliability* section).

10.2.2 Geometric — trials to the first event

$$P(X = k) = (1 - p)^{k-1}p, \quad \mathbb{E}[X] = \frac{1}{p}.$$

Defect rate 3%. Expected boards until a defect? $1/0.03 = 33.3$. First defect on the 5th board? $0.97^4(0.03) = 2.7\%$. This is how you reason about “how long until the next escape” when the rate is steady.

10.2.3 Poisson — the rare-event distribution behind BER and defect counts

$$P(X = k) = \frac{\lambda^k e^{-\lambda}}{k!}, \quad \mathbb{E}[X] = \text{Var}(X) = \lambda.$$

Poisson is the most important distribution for a compute-test engineer because **bit errors are Poisson** (*BER and Confidence*) and **defect counts per board are Poisson** (*c-charts, SPC*). Use it whenever events are rare, independent, and arrive at a roughly constant rate.

Station averages $\lambda = 2$ failures/shift. $P(0) \stackrel{?}{=} e^{-2} = 0.135$. $P(X \geq 5) \stackrel{?}{=}$ with $P(0..4) = 0.135, 0.271, 0.271, 0.180, 0.090$ summing to 0.947, $P(X \geq 5) = 0.053$ — a $(5 - 2)/\sqrt{2} = 2.1\sigma$ excursion, worth an investigation.

Poisson approximates the binomial when n is large and p small, with $\lambda = np$. 1000 solder joints at 0.01% each: $\lambda = 0.1$, $P(\text{board clean}) = e^{-0.1} = 0.905$. At 5000 joints, $\lambda = 0.5$, only $e^{-0.5} = 0.607$ are clean. High-density boards demand tighter process control purely from the joint count.

10.2.4 Normal — the reference for any measured parameter

$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{(x - \mu)^2}{2\sigma^2}\right), \quad Z = \frac{X - \mu}{\sigma}.$$

68–95–99.7 rule: $\mu \pm 1\sigma/2\sigma/3\sigma$ holds 68% / 95% / 99.7%. The Z -table is how you turn “how far is the mean from the limit” into a defect fraction — the bridge to Cpk in *Process Capability and Setting Limits*.

Z	P(Z < z)	use
1.645	0.9500	95th percentile (one-sided)
1.96	0.9750	two-sided 95%
2.326	0.9900	99th percentile
2.576	0.9950	two-sided 99%
3.0	0.9987	1350 ppm one-sided tail

PCIe link speed $\sim N(15.98, 0.05^2)$ GT/s, spec 15.8–16.2: in-spec fraction = $P(Z < 4.4) - P(Z < -3.6) \approx 99.98\%$. Excellent capability (compute the Cpk in *Process Capability and Setting Limits*).

Why Normal is the default — the Central Limit Theorem. Average n independent samples from *any* finite-variance distribution and the sample mean tends to $N(\mu, \sigma^2/n)$ as n grows ($n \geq 30$ is the rule of thumb). Test times are right-skewed by retests, yet *batch averages* are nearly normal — which is exactly what lets $\bar{X}$ charts and Cpk math work on real, non-normal data. Times $\mu = 20, \sigma = 5$ min, batch of 50: averages are $\approx N(20, 0.707^2)$; a 22-min batch is $Z = 2.83$, flag it.

10.2.5 Exponential — the flat bottom of the bathtub

$$f(x) = \lambda e^{-\lambda x}, \quad P(X > t) = e^{-\lambda t}, \quad \mathbb{E}[X] = \frac{1}{\lambda}.$$

Models time between *random* failures during useful life, where the hazard rate is constant. **Memoryless:** $P(X > s + t \mid X > s) = P(X > t)$ — a unit that has survived does not “age” in this regime, which is exactly why MTBF is meaningful only here. $MTBF = 500$ h $\Rightarrow \lambda = 0.002/h$: $P(\text{survive 24 h}) = e^{-0.048} = 0.953$; a 168-h week = $e^{-0.336} = 0.715$.

10.2.6 Weibull — the wear-out distribution (and bathtub in one equation)

The exponential assumes a constant hazard. Real hardware does not: solder fatigues, NAND wears, fans seize, capacitors dry out. **Weibull** generalizes the exponential with a **shape parameter** β that lets the hazard rate rise or fall:

$$P(X > t) = \exp\left[-\left(\frac{t}{\eta}\right)^\beta\right], \quad h(t) = \frac{\beta}{\eta} \left(\frac{t}{\eta}\right)^{\beta-1},$$

where η is the **characteristic life** (the age by which 63.2% have failed) and $h(t)$ is the instantaneous **hazard rate**. The single parameter β tells you *which region of the bathtub you are in*:

beta	Hazard trend	Bathtub region	Physical cause
< 1	decreasing	infant mortality	latent manufacturing defects
= 1	constant	useful life	random (= exponential exactly)
> 1	increasing	wear-out	fatigue, electromigration, aging

Why a test engineer cares. Plot field/reliability-lab failure times on Weibull paper, read β off the slope. $\beta < 1$ says your escapes are infant mortality — **burn-in screens them out**, so add or extend burn-in. $\beta > 1$ says wear-out is arriving — burn-in does nothing; you need a design fix or a life limit. Reading β is how you decide whether more screening even helps. *Example:* a fan population fits $\beta = 2.5$, $\eta = 40,000$ h — clearly wear-out, and burn-in would just consume useful life. By contrast a connector batch fitting $\beta = 0.6$ is begging for a longer burn-in to flush the weak ones before they ship.

10.3 The “Sigma Level” Bridge

Before capability indices, fix the link between *number of sigmas to the nearest limit* and *defect fraction*. This table is the spine of *Process Capability and Setting Limits* and *Sampling and AQL*.

Sigma to spec	One-sided tail	~PPM (one side)	Cpk equivalent
1 sigma	15.866%	158,655	0.33
2 sigma	2.275%	22,750	0.67
3 sigma	0.135%	1,350	1.00
4 sigma	0.0032%	31.7	1.33
4.5 sigma	0.00034%	3.4	1.50
5 sigma	0.000029%	0.29	1.67
6 sigma	0.000001%	0.001	2.00

The “Six Sigma = 3.4 PPM” reconciliation. A perfectly centered 6σ process has a 0.001 PPM tail. The famous **3.4 PPM** assumes a long-term 1.5σ **mean shift** (processes drift over weeks), leaving 4.5σ of effective margin. That 1.5σ shift is exactly the Cp-vs-Ppk gap in the worked capability examples — short-term potential minus long-term drift.

10.4 Geometry and Spatial Reasoning

Not the daily statistics, but the reference geometry a compute-test engineer reaches for when a fixture is laid out, a sensor pose has to be checked, or a field-of-view number has to be sanity-checked against a datasheet. At Zoox the sensors (camera, lidar, radar) live in 3D space with positions and orientations, so the coordinate-transform and similar-triangle math below is the same math the perception fixtures use.

10.4.1 Distance formulas

$$d_{2D} = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}, \quad d_{3D} = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2 + (z_2 - z_1)^2}.$$

Camera at (2.0, 0.5, 1.8) m, lidar at (0, 0, 2.1). Separation? $d = \sqrt{4.0 + 0.25 + 0.09} = \sqrt{4.34} = 2.083$ m.

Point to a line (2D). Line $ax + by + c = 0$, point (x_0, y_0) :

$$d = \frac{|ax_0 + by_0 + c|}{\sqrt{a^2 + b^2}}.$$

Reference line $y = 2x + 1$ (i.e. $2x - y + 1 = 0$), sensor at (3, 4): $d = \frac{|2(3) - 4 + 1|}{\sqrt{5}} = \frac{3}{2.236} = 1.342$.

10.4.2 Areas

Triangle by coordinates (shoelace):

$$\text{Area} = \frac{1}{2} |x_1(y_2 - y_3) + x_2(y_3 - y_1) + x_3(y_1 - y_2)|.$$

Points (0, 0), (4, 0), (2, 3): $\frac{1}{2} |0 + 12 + 0| = 6$.

Circle: area $= \pi r^2$, circumference $= 2\pi r$. **Sector** (angle θ in radians): area $= \frac{1}{2} r^2 \theta$, arc length $= r\theta$. A radar that covers a 60° cone out to 100 m on a flat plane sweeps $\frac{1}{2} r^2 \theta = \frac{1}{2} (100^2) (\pi/3) = 5236 \text{ m}^2$ — the footprint you reason about when laying out a calibration target field.

10.4.3 Coordinate transforms (sensor placement)

Every sensor has a pose relative to the vehicle frame; you transform between frames with a rotation then a translation.

2D rotation by θ :

$$x' = x \cos \theta - y \sin \theta, \quad y' = x \sin \theta + y \cos \theta.$$

Object at (5, 3) in the sensor frame, sensor rotated 30° ($\cos = 0.866$, $\sin = 0.5$): $x' = 5(0.866) - 3(0.5) = 2.83$, $y' = 5(0.5) + 3(0.866) = 5.098$. If the sensor sits at (1.5, 0.8) in the vehicle frame, add the offset: vehicle-frame position = (4.33, 5.898).

10.4.4 Trigonometry quick reference

Function	SOH-CAH-TOA	Key values at 0,30,45,60,90 deg
sin	opp/hyp	0, 0.5, 0.707, 0.866, 1
cos	adj/hyp	1, 0.866, 0.707, 0.5, 0
tan	opp/adj	0, 0.577, 1, 1.732, inf

- **Pythagoras (right triangles):** $a^2 + b^2 = c^2$.
- **Law of cosines (any triangle):** $c^2 = a^2 + b^2 - 2ab \cos C$ (reduces to Pythagoras at $C = 90^\circ$).
- **Radians:** $360^\circ = 2\pi \text{ rad}$; $\text{rad} = \text{deg} \times \pi/180$.

10.4.5 Regular polygons — the hexagon pattern

Working *backwards* from area to side length is the geometry move that recurs whenever a target tile, a panel cell, or a packing layout is specified by area rather than dimension.

Equilateral triangle (the building block), side s : height $h = \frac{\sqrt{3}}{2}s$, area $= \frac{\sqrt{3}}{4}s^2$. Quick: $s=1 \rightarrow 0.433$, $s=2 \rightarrow 1.732$, $s=10 \rightarrow 43.3$.

Regular hexagon = six equilateral triangles around a center: perimeter = $6s$,

$$\text{Area} = 6 \cdot \frac{\sqrt{3}}{4} s^2 = \frac{3\sqrt{3}}{2} s^2 \approx 2.598 s^2, \quad \text{interior angle} = 120^\circ.$$

Working backwards: given area A , each triangle is $A/6$, so

$$\frac{A}{6} = \frac{\sqrt{3}}{4} s^2 \Rightarrow s^2 = \frac{2A}{3\sqrt{3}} = \frac{2A\sqrt{3}}{9} \Rightarrow s = \sqrt{\frac{2A\sqrt{3}}{9}}, \quad \text{Perimeter} = 6s.$$

Worked: $A = 100$. $s^2 = 2(100)(1.732)/9 = 38.49$, $s = 6.204$, perimeter = 37.2. Check: $2.598(6.204^2) = 99.99$. Correct.

Other regular polygons: square area = s^2 ($P=4s$); pentagon $\approx 1.72 s^2$; octagon = $2(1 + \sqrt{2})s^2 \approx 4.828 s^2$.

10.4.6 Angular size and similar triangles — the FOV math

A camera's field of view is a similar-triangles problem, and so is any “how big does a target need to be at range R ” question on a perception fixture.

Small-angle approximation (angle $\lesssim 15^\circ$):

$$\theta \text{ (rad)} \approx \frac{\text{physical size}}{\text{distance}} \iff \text{distance} \approx \frac{\text{physical size}}{\theta}.$$

Similar triangles: if a near object (size d_1 at distance D_1) just covers a far object (size d_2 at distance D_2), then $\frac{d_1}{D_1} = \frac{d_2}{D_2}$, so $D_2 = d_2 \frac{D_1}{d_1}$.

Camera FOV. A 50 mm lens on a 36 mm-wide sensor:

$$\text{FOV} = 2 \arctan \frac{36}{2(50)} = 2 \arctan(0.36) = 2(19.8^\circ) \approx 39.6^\circ.$$

At 100 m it covers $2(100) \tan(19.8^\circ) = 72$ m of width — the kind of number you check a camera-aiming fixture against. For a lidar or radar the same arc-length relation (width = $2R \tan(\text{half-FOV})$) sizes the target board at a given standoff.

10.4.7 3D spatial counting

Organized counting beats guessing: categorize by direction, then sum. In a $3 \times 3 \times 3$ grid the count of collinear triples (the same bookkeeping you use to enumerate paths or positions in a 3D array of sensors or test points):

```
Axis-aligned (parallel to an axis):
X-direction: 3 (y) * 3 (z) = 9
Y-direction: 3 (x) * 3 (z) = 9
Z-direction: 3 (x) * 3 (y) = 9 subtotal 27
Face diagonals (within a 2D plane):
XY planes: 2 diag * 3 layers = 6
XZ planes: 2 diag * 3 layers = 6
YZ planes: 2 diag * 3 layers = 6 subtotal 18
Space diagonals (corner to opposite corner): 4
TOTAL 49
```

General n^3 formula: $3n^2 + 6n + 4$; for $n = 3$, 49. The center cell lies on 13 lines, corners on 7, edges on 4 — a consistency check on the count.

10.5 Algebra and Signal Math

Practical algebra: log and dB scales, SNR, and the signaling-rate math you actually do on a high-speed link. The GT/s $\rightarrow$ GB/s conversion below is the bridge between the BER work in *BER and Confidence* and the bandwidth numbers quoted on a PCIe/NVMe spec sheet.

10.5.1 Logarithms and decibels

A **decibel** is a log ratio. For **power**, $10 \log_{10}$; for **amplitude/voltage**, $20 \log_{10}$ (because power $\propto V^2$, and $\log V^2 = 2 \log V$):

$$\text{dB}_{\text{power}} = 10 \log_{10} \frac{P_1}{P_2}, \quad \text{dB}_{\text{amp}} = 20 \log_{10} \frac{V_1}{V_2}.$$

Memorize these and you can do most dB arithmetic in your head:

Ratio	Power dB	Note
x2	+3.01	“3 dB = double the power”
x10	+10	one decade
x100	+20	two decades
x0.5	-3	half power
x1000	+30	

dB add when ratios multiply: a x20 power gain is x2 then x10, so $3 + 10 = 13$ dB. A 30 dB amplifier on a 1 mW input: $30/10 = 3$ decades $\Rightarrow$ x1000 $\Rightarrow$ 1 W.

Log scales (Bode/decades). A decade is x10 in frequency; an octave is x2. A first-order low-pass rolls off -20 dB/decade ($= -6$ dB/octave) above its corner $f_c = \frac{1}{2\pi RC}$.

10.5.2 Signal-to-noise ratio

$$\text{SNR} = \frac{P_{\text{signal}}}{P_{\text{noise}}}, \quad \text{SNR}_{\text{dB}} = 10 \log_{10} \frac{P_{\text{signal}}}{P_{\text{noise}}}.$$

3.3 V signal, 10 mV RMS noise (amplitude ratio, so use $20 \log$): $\text{SNR} = 3.3/0.01 = 330 \Rightarrow 20 \log_{10} 330 = 50.4$ dB. (As a power ratio 330^2 , $10 \log_{10} (330^2)$ gives the same 50.4 dB — the two forms agree because a voltage ratio of 330 is a power ratio of 330^2 .) On a SerDes link a higher SNR margin is what shows up downstream as the lower BER you confidence-test in *BER and Confidence*.

10.5.3 Bandwidth math: GT/s to GB/s

High-speed serial links quote a **raw symbol rate** in GT/s (giga-transfers per second). Usable data rate is lower because of **line coding** overhead. The two encodings to know:

- **8b/10b** (PCIe Gen1/2, USB 3.0, SATA, older SerDes): 10 line bits carry 8 data bits, efficiency $= 8/10 = 80\%$.
- **128b/130b** (PCIe Gen3/4/5/6): 130 line bits carry 128 data bits, efficiency $= 128/130 = 98.46\%$.

Convert raw rate to usable bytes/s per lane:

$$\text{GB/s per lane} = \frac{\text{GT/s} \times \text{coding efficiency}}{8 \text{ bits/byte}}.$$

PCIe Gen	GT/s	Coding	Bytes/s per lane	x16 (GB/s)
1	2.5	8b/10b	0.25	4.0

PCIe Gen	GT/s	Coding	Bytes/s per lane	x16 (GB/s)
2	5.0	8b/10b	0.50	8.0
3	8.0	128b/130b	0.985	15.75
4	16.0	128b/130b	1.969	31.5
5	32.0	128b/130b	3.938	63.0
6	64.0	PAM4 + FEC	7.563	121

Worked: PCIe Gen4 x4 NVMe usable rate. $16 \times (128/130)/8 = 1.969$ GB/s per lane $\times 4 = 7.88$ GB/s. (Gen6 switches to Pulse Amplitude Modulation 4-level (PAM4) — 2 bits/symbol — plus Fixed-size Link Packet (FLIT)-mode Forward Error Correction (FEC), so a “transfer” no longer equals one bit; the table value already accounts for the encoding. This is also why Gen6 BER acceptance in *BER and Confidence* tests the *post-FEC* error count.)

10.5.4 Averaging to reduce noise

Averaging N independent readings of the same quantity reduces the random-noise standard deviation by $\sqrt{N}$ (means add as N , noise variance adds as N , so σ of the mean scales as $1/\sqrt{N}$):

$$\sigma_{\text{avg}} = \frac{\sigma_{\text{single}}}{\sqrt{N}}.$$

$\sigma = 0.5^\circ\text{C}/\text{reading}$, average 25: $\sigma_{\text{avg}} = 0.5/5 = 0.1^\circ\text{C}$. To cut noise 10x you need 100x the samples — diminishing returns, and the direct argument for fixing a noisy gauge (the *Gauge R&R* section) rather than averaging around it.

10.5.5 Propagation of uncertainty

$$f = a \pm b : \quad \sigma_f = \sqrt{\sigma_a^2 + \sigma_b^2} \quad (\text{add in quadrature}),$$

$$f = a \cdot b \text{ or } a/b : \quad \frac{\sigma_f}{f} = \sqrt{\left(\frac{\sigma_a}{a}\right)^2 + \left(\frac{\sigma_b}{b}\right)^2} \quad (\text{relative, in quadrature}).$$

$P = VI$, $V = 48.0 \pm 0.2$, $I = 10.0 \pm 0.1$, $P = 480$ W: $\frac{\sigma_P}{P} = \sqrt{(0.2/48)^2 + (0.1/10)^2} = 0.0108$, so $\sigma_P = 480 \times 0.0108 = \pm 5.2$ W. This is the same root-sum-square that governs tolerance stack-ups and the variance addition in *Expected value and variance*.

10.5.6 Sampling and aliasing (Nyquist)

To capture a signal of frequency f you must sample at $f_s \geq 2f$ (the Nyquist rate); below that the signal **aliases** to a false lower frequency. *Vibration content to 500 Hz*: sample at ≥ 1 kHz; in practice use 2.5x (~1.25 kHz) for margin against imperfect anti-alias filtering. The same rule sets the minimum scan/strobe rate when a moving part is sampled on a line — sample a 1000 rev/min fan below 33 Hz and a strobed camera will show it crawling backward (a temporal alias), the rotational version of the same trap.

10.6 Vectors and Linear Algebra Basics

Sensor data is vector-valued — camera pixels, lidar point clouds, force/torque readings — and sensor calibration is matrix algebra. Awareness level: enough to read a calibration routine and reason about a pose.

10.6.1 Vectors

A vector has magnitude and direction: $\mathbf{v} = (v_x, v_y, v_z)$.

- **Magnitude:** $|\mathbf{v}| = \sqrt{v_x^2 + v_y^2 + v_z^2}$.
- **Unit vector:** $\hat{\mathbf{v}} = \mathbf{v}/|\mathbf{v}|$.
- **Addition:** component-wise (tip-to-tail). **Scalar multiply:** scales length.

10.6.2 Dot product

$$\mathbf{a} \cdot \mathbf{b} = a_1 b_1 + a_2 b_2 + a_3 b_3 = |\mathbf{a}| |\mathbf{b}| \cos \theta.$$

Uses: angle $\cos \theta = \frac{\mathbf{a} \cdot \mathbf{b}}{|\mathbf{a}| |\mathbf{b}|}$; projection of $\mathbf{a}$ onto $\mathbf{b}$ is $\frac{\mathbf{a} \cdot \mathbf{b}}{|\mathbf{b}|}$; $\mathbf{a} \cdot \mathbf{b} = 0 \iff$ perpendicular. $\mathbf{a} = (3, 4)$, $\mathbf{b} = (4, -3)$: $\mathbf{a} \cdot \mathbf{b} = 12 - 12 = 0 \Rightarrow$ perpendicular. $\mathbf{a} = (1, 0)$, $\mathbf{b} = (1, 1)$: $\cos \theta = 1/\sqrt{2} = 0.707 \Rightarrow \theta = 45^\circ$.

10.6.3 Cross product (3D)

$$\mathbf{a} \times \mathbf{b} = (a_2 b_3 - a_3 b_2, a_3 b_1 - a_1 b_3, a_1 b_2 - a_2 b_1), \quad |\mathbf{a} \times \mathbf{b}| = |\mathbf{a}| |\mathbf{b}| \sin \theta.$$

The result is perpendicular to both. Uses: parallelogram area $= |\mathbf{a} \times \mathbf{b}|$; triangle area $= \frac{1}{2} |\mathbf{a} \times \mathbf{b}|$; surface normal from two edge vectors; and shortest point-to-line distance in 3D, $\frac{|\mathbf{AP} \times \mathbf{d}|}{|\mathbf{d}|}$.

10.6.4 Matrices (awareness level)

Matrix multiply: $(m \times n)(n \times p) = (m \times p)$, row-by-column dot products; order matters ($AB \neq BA$ in general). **2D rotation matrix:**

$$R(\theta) = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix}.$$

A full sensor-to-vehicle transform is a 4×4 matrix combining rotation and translation (homogeneous coordinates); calibration is the process of solving for those matrices.

10.7 Core Physical Relationships

A brief reference for the physics that backs the fixtures and the thermal/power test conditions. Reconstruct these from first principles rather than recall them blind.

Relationship	Formula	Floor use
Force / pressure / area	$F = P * A$ [lbf]=[psi]*[in ²]	linear / pneumatic fixture force
Ohm's law and power	$V = IR$; $P = VI = I^2 R = V^2/R$	supply sizing, heat load
Torque	$\tau = F * d$ (moment arm)	air-brake / load-cell fixtures
Heat	$Q = mcdT$	thermal soak energy budget

- **Force.** *290 psi regulator, 20 in² piston cap:* $F = 290 \times 20 = 5800$ lbf. The rod side has a smaller effective area (piston minus rod cross-section), so the same pressure makes *less* force in tension than in compression.
- **Power.** *Supply at 48 V, 25 A:* $P = 1200$ W — hence active cooling on a high-power compute board.
- **Torque.** *Load cell, 6.6 in arm, 18.2 lbf:* $\tau = 18.2 \times (6.6/12) = 10.0$ ft-lb. An air-brake fixture fits $\tau = \text{slope} \cdot \text{PSI} + \text{intercept}$, slope set by $\mu \cdot (\text{pad area}) \cdot (\text{eff. radius})$ — a regression fit (*Regression and Correlation*) turned into a test limit.
- **Heat.** Thermal testing matters because at -40°C switching is slower (higher V_{th}) and at $+85^\circ\text{C}$ leakage grows and noise margins shrink — the extremes that catch timing and power-margin defects, and the basis for the burn-in and Arrhenius math in the *Reliability* section.

10.8 BER and Confidence — the PCIe BERT Math

This is the centerpiece for a compute/high-speed-link test engineer, and the statistical core of the PCIe BERT tool. The interview and the job both ask the same thing: **“You ran a SerDes/PCIe link for N bits and saw E errors — what BER can you claim, and how long must you run to prove a target?”** The link-layer detail lives in the **PCIe chapter**; the statistics live here.

10.8.1 Why bit errors are Poisson

Errors on a healthy link are rare and effectively independent, so the error count over n transmitted bits is **Poisson** with mean $\lambda = np$, where p is the true bit error ratio (BER). A BERT runs a known pattern (PRBS) and counts mismatches; that count is your Poisson observation.

Confidence level (CL) = the probability that, *if the true BER were as bad as your target p* , you would have seen *more than* the E errors you observed. High CL means a truly bad link would almost certainly have shown more errors than you saw — so a clean run is strong evidence the real BER is below the target.

$$\text{CL} = 1 - \text{PoissonCDF}(E; np) = 1 - \sum_{k=0}^E \frac{(np)^k e^{-np}}{k!}$$

Frame it correctly. CL is *not* “the probability the link is good.” It is the probability a link *at the target BER* would have produced at least $E + 1$ errors. Stating it the wrong way in a design review is a credibility tell.

10.8.2 The zero-error case — the “3/BER” rule you know cold

Most acceptance runs target zero errors. With $E = 0$ the sum collapses:

$$\text{CL} = 1 - e^{-np} \implies np = -\ln(1 - \text{CL}) \implies n = \frac{-\ln(1 - \text{CL})}{p} \quad (E = 0).$$

Memorize the constant $-\ln(1 - \text{CL})$:

CL	$-\ln(1 - \text{CL})$
90%	2.303
95%	2.996
99%	4.605
99.9%	6.908

The rule of thumb: for 95% confidence with zero errors you need about $3/p$ **bits**; for 99%, about $4.6/p$. This is the bit-domain twin of the *rule of three* — with 0 failures in n trials, the upper 95% bound on the failure rate is $\approx 3/n$.

Worked — bits to prove $\text{BER} \leq 10^{-12}$ at 95% CL, zero errors.

$$n = \frac{2.996}{10^{-12}} = 2.996 \times 10^{12} \text{ bits} \approx 3 \times 10^{12}.$$

On one PCIe Gen5 lane at 32 GT/s (use the raw 32×10^9 bit/s for the link test): $t = 3 \times 10^{12} / 32 \times 10^9 \approx 94$ s. So a ~95-second clean run on one lane proves $\leq 10^{-12}$ at 95% CL. For 99% CL scale by $4.605/2.996 = 1.54 \rightarrow \sim 145$ s.

Worked — what BER did I prove? A run of $n = 10^{13}$ bits with $E = 0$:

$$p_{95} = \frac{2.996}{10^{13}} = 3.0 \times 10^{-13}.$$

You have demonstrated $\text{BER} \leq 3.0 \times 10^{-13}$ at 95% confidence.

10.8.3 Allowing observed errors — the chi-squared upper bound

If you saw $E > 0$ errors and still want a confidence-bounded BER, the exact one-sided upper limit on a Poisson mean is a clean chi-squared expression:

$$\text{BER}_{\text{upper}} = \frac{\chi_{1-\alpha, 2(E+1)}^2}{2n}$$

where $\chi_{q,\nu}^2$ is the q -quantile with ν degrees of freedom. This is the form lab software and the Telcordia/standards methods use. Two checks that it is the right formula:

- For $E = 0$: $\nu = 2$, and $\chi_{1-\alpha,2}^2 = -2 \ln(\alpha) = -2 \ln(1-\text{CL})$, so $\text{BER}_{\text{upper}} = -\ln(1-\text{CL})/n$ — **identical** to the zero-error “3/BER” rule.
- It is exact for any E , where the normal approximation $\hat{p} \pm z\sqrt{\hat{p}/n}$ falls apart in the small-count, low- p regime BER lives in.

Worked — $E = 2$ errors at 95% CL. $n = 2 \times 10^{12}$, $\nu = 2(2+1) = 6$, $\chi_{0.95,6}^2 = 12.59$:

$$\text{BER}_{\text{upper}} = \frac{12.59}{2(2 \times 10^{12})} = 3.15 \times 10^{-12}.$$

Even with 2 errors you can claim $\text{BER} \leq 3.15 \times 10^{-12}$ at 95% (the point estimate is only $E/n = 10^{-12}$; the bound is higher because 2 errors is a tiny sample). Useful $\chi_{0.95,\nu}^2$: $\nu = 2$: 5.99, 4: 9.49, 6: 12.59, 8: 15.51, 10: 18.31. For 99% CL ($\alpha = 0.01$): $\nu = 2$: 9.21, 4: 13.28, 6: 16.81.

10.8.4 The CL-given-errors check

Going the other way — you fixed the run length and want the confidence achieved — use the CDF form. Target $p = 10^{-12}$, ran $n = 3 \times 10^{12}$ bits ($np = 3$), saw $E = 1$:

$$\text{CL} = 1 - [e^{-3} + 3e^{-3}] = 1 - 4e^{-3} = 1 - 0.199 = 0.801.$$

Only **80% CL** — one error in 3×10^{12} bits does *not* clear 10^{-12} at 95%; run longer. To reach 95% with $E = 1$, solve $1 - (1 + np)e^{-np} = 0.95$ to get $np = 4.74$, i.e. $n = 4.74 \times 10^{12}$ bits.

10.8.5 Sequential test — pass / continue / reject

A fixed- n acceptance run wastes time: a great link passes long before n , and a dead link should be rejected almost immediately. **Wald’s Sequential Probability Ratio Test (SPRT)** evaluates after every error (or every block) and emits one of three verdicts — **pass**, **continue**, or **reject** — drawing two parallel boundary lines in the (bits, cumulative-errors) plane. This is what a good BERT does instead of always running to the bitter end.

The test discriminates between an acceptable BER p_0 and a rejectable BER $p_1 > p_0$, with producer’s risk α (reject a good link) and consumer’s risk β (accept a bad one). Plot cumulative errors E against transmitted bits n ; the two decision lines are parallel with the same slope:

```
slope s = (p1 - p0) / ln(p1/p0)
accept (PASS) line: E = s*n - h_a, h_a = ln((1-a)/b) / ln(p1/p0)
reject line: E = s*n + h_r, h_r = ln((1-b)/a) / ln(p1/p0)
```

```
E cumulative errors
^ . reject region (above)
| ____/ reject line
| ____/ ____/
| continue ____/ ____/ accept line
| ____/ ____/
| ____/ ____/ accept region (below)
+-----> n bits
```

Decision each step: if E crosses *above* the reject line, **stop and fail**; if it stays *below* the accept line as n grows, **stop and pass**; in between, **keep running**. The payoff: a clean link earns its PASS in a fraction of the fixed- n time, and a marginal link gets caught early instead of soaking a station for 40 hours.

Worked — SPRT boundaries for a PCIe lane. Discriminate an acceptable $p_0 = 10^{-12}$ from a rejectable $p_1 = 10^{-11}$ (10x worse) with $\alpha = 0.05$, $\beta = 0.10$. Here $\ln(p_1/p_0) = \ln 10 = 2.303$, so the slope is $s = (10^{-11} - 10^{-12})/2.303 = 3.91 \times 10^{-12}$ errors/bit, with intercepts $h_a = \ln(0.95/0.10)/2.303 = 0.978$ and $h_r = \ln(0.90/0.05)/2.303 = 1.255$. A perfectly clean link ($E = 0$) crosses the accept line at $n = h_a/s = 2.5 \times 10^{11}$ bits — about **8 s** on a 32 GT/s lane, versus ~94 s for the fixed- n 95% run, a ~12x time saving on good links. A truly bad link ($\gg p_1$) accumulates errors fast and trips the reject line ($h_r \approx 1.3$, so as soon as a couple of early errors land) in seconds rather than soaking the full run.

The tradeoff is variable test time — fine for engineering bring-up and margining, less ideal for a fixed-takt production line where you usually pin the run length with the zero-error “3/BER” rule instead.

10.8.6 Practical notes (cross-ref the PCIe chapter)

- **Per-lane vs aggregate.** A x16 link is 16 lanes; testing them in parallel feels like a 16x speedup, and for *time* it is — to prove 10^{-12} at 95% CL you still need 3×10^{12} bits **on each lane**, but 16 lanes deliver those bits simultaneously, so the ~94 s single-lane run covers all 16 at once. The trap is in the *accounting*: if you pool the 16 lanes’ errors and divide by the aggregate bit count, you prove only the *aggregate* BER, which hides a single sick lane. Concretely, one lane running at 10^{-11} (10x over spec) alongside 15 clean lanes at 10^{-13} gives a pooled BER of $(10^{-11} + 15 \times 10^{-13})/16 \approx 7.2 \times 10^{-13}$ — under 10^{-12} , so the aggregate **passes** while a lane is an order of magnitude out. Always **count errors per lane and margin the worst one**; report per-lane BER, never the link average.
- **Targets.** PCIe Gen1–5 spec raw BER $\leq 10^{-12}$; **Gen6 (PAM4)** relaxes the *raw* target to $\leq 10^{-6}$ and leans on **FEC** for an effective post-FEC BER $\leq 10^{-12}$ — so for Gen6 you confidence-test the *post-FEC* error count, not the raw symbol errors. Note what the relaxed raw target does to run time: at $p = 10^{-6}$ the zero-error 95% run is only $n = 2.996/10^{-6} \approx 3 \times 10^6$ bits — microseconds — so the meaningful Gen6 acceptance run is the *post-FEC* one against the 10^{-12} effective target, back to the ~minutes-per-lane regime.

```
BERT cheat-sheet (zero-error acceptance, single lane)
bits needed: n = -ln(1 - CL) / p
time: t = n / line_rate_bits_per_sec
proven BER: p = -ln(1 - CL) / n (from a completed clean run)
with errors: BER_upper = chi2(1-alpha, 2*(E+1)) / (2*n)
CL achieved: CL = 1 - PoissonCDF(E; n*p)
-ln(1-CL): CL=90% ->2.303 95% ->2.996 99% ->4.605 99.9% ->6.908
```

10.9 Process Capability and Setting Limits

Capability indices compare the **voice of the process** (its spread) to the **voice of the customer** (the spec width). They answer the question every limit review asks: *given how this parameter actually behaves, will the spec line scrap good units or pass bad ones?*

10.9.1 Descriptive stats — and why $n-1$

For a sample $x_1, \dots, x_n$:

$$\bar{x} = \frac{1}{n} \sum x_i, \quad s^2 = \frac{1}{n-1} \sum (x_i - \bar{x})^2, \quad s = \sqrt{s^2}.$$

Why $n-1$ (Bessel’s correction)? Deviations are taken from the *sample* mean, which is itself pulled toward the data, so dividing by n underestimates spread. $n-1$ (the degrees of freedom)

makes s^2 unbiased. Use n only for a full population — test-floor data is always a sample, so always $n - 1$.

The mean chases outliers; a single stuck reading moves $\bar{x}$ but barely moves the median. Watch for that when a fixture glitches.

10.9.2 The four indices

- **Cp** / **Cpk** use **short-term** / **within-subgroup** sigma $\hat{\sigma}_{ST}$, estimated from a control chart as $\hat{\sigma} = \bar{R}/d_2$. These describe process *potential* when stable.
- **Pp** / **Ppk** use the **overall** / **long-term** sample s across all data. These describe *actual* performance including drift.

$$C_p = \frac{USL - LSL}{6 \hat{\sigma}_{ST}}, \quad C_{pk} = \min\left(\frac{USL - \mu}{3 \hat{\sigma}_{ST}}, \frac{\mu - LSL}{3 \hat{\sigma}_{ST}}\right),$$

$$P_p = \frac{USL - LSL}{6 s}, \quad P_{pk} = \min\left(\frac{USL - \mu}{3 s}, \frac{\mu - LSL}{3 s}\right).$$

How to read them:

- C_p ignores centering — the *best you could do* if perfectly centered.
- $C_{pk} \leq C_p$ always; the gap is the **centering penalty**, zero only when μ sits midway between the limits.
- $C_{pk} \gg P_{pk}$ means capable short-term but **drifting** between subgroups — chase the special cause (tool wear, shift-to-shift, ambient temperature), do not tighten the machine.
- One-sided spec (e.g. a max-temperature limit): use only the relevant half; C_p is undefined.

Cpk	Margin	One-sided defect	~PPM	Verdict
1.00	3 sigma	0.135%	1,350	minimum / marginal
1.33	4 sigma	0.0032%	31.7	industry “capable” floor
1.67	5 sigma	0.000029%	0.29	strong
2.00	6 sigma	~1e-7 %	0.001	world-class

10.9.3 Worked capability examples

One-sided (GPU temperature). $USL = 85^\circ\text{C}$, no LSL, $\mu = 72$, $\sigma = 3$. $C_{pk} = (85 - 72)/9 = 1.44$ — good; ~7 PPM exceed 85°C . Drift to $\mu = 76$: $C_{pk} = 9/9 = 1.00$ — marginal, ~1,350 PPM. A 4° mean shift (just over one sigma) moved the escape rate ~184×. The leverage is brutal because the defect rate lives in the *tail* of the normal: out there the curve is dropping near-exponentially, so a shift measured in fractions of a sigma multiplies the escape count by orders of magnitude. This is why thermal limits get guardbanded and why a slow $\bar{X}$ drift (caught by the SPC charts in *SPC*) is worth chasing long before any single board fails.

Two-sided, off-center. Spec 45–55, $\mu = 50$, $\sigma = 2$: $C_p = 10/12 = 0.833$, centered so $C_{pk} = 0.833$ — not capable; out-of-spec = $2P(Z > 2.5) = 1.24\%$. Shift to $\mu = 52$: C_p unchanged, but $C_{pk} = \min(0.5, 1.167) = 0.5$ — the centering penalty halved the index.

Cpk vs Ppk (drift). Within-subgroup $\hat{\sigma}_{ST} = 0.10$ but overall $s = 0.18$ because the mean wanders. With $\mu = 10.0$, $USL = 10.5$, $LSL = 9.5$: $C_{pk} = 0.5/0.30 = 1.67$ but $P_{pk} = 0.5/0.54 = 0.93$. The process *could* be world-class; right now it is below the capable floor. The fix is killing the between-subgroup drift, not tightening the tool.

Cpk to PPM, fast. $PPM \approx 10^6[P(Z > 3C_{pk,upper}) + P(Z < -3C_{pk,lower})]$. One-sided/symmetric: $PPM \approx 10^6(1 - \Phi(3C_{pk}))$. Memorize $3 \times 1.33 = 4\sigma \rightarrow 32$ ppm and $3 \times 1.67 = 5\sigma \rightarrow 0.3$ ppm and you can eyeball any Cpk.

10.9.4 Setting limits and guardbands

A spec line is not just the customer number — it must account for *your measurement uncertainty* so you do not pass parts that are actually out, or scrap parts that are actually in.

- **Spec limits** come from the customer / design (the LSL/USL).
- **Control limits** come from the *process* and go on the SPC chart — never put spec limits on a control chart (*SPC*).
- **Test (guardband) limits** are the lines your tester actually uses, pulled *inside* the spec by a **guard-band** g to protect against gauge error:

$$\text{upper test limit} = \text{USL} - g, \quad \text{lower test limit} = \text{LSL} + g.$$

A defensible guardband is tied to measurement uncertainty: a common rule is $g = k \cdot U$ where U is the gauge's expanded uncertainty (often the GR&R standard deviation times a coverage factor), with k chosen for the risk you will tolerate. The tradeoff is direct and unavoidable:

Guardband	Effect on escapes	Effect on yield (false rejects)
Wider (limits pulled in more)	fewer bad parts pass	more good parts scrapped
Narrower (toward spec)	more escapes	fewer good parts scrapped

Example. USL = 85°C, gauge GR&R $\sigma = 0.5^\circ\text{C}$. A 2σ guardband $g = 1.0^\circ\text{C}$ sets the **test limit at 84.0°C** — you fail anything reading above 84 to be 95%-confident the true value is under 85. You knowingly scrap a sliver of good parts between 84 and 85 to drive the escape rate down. That sliver is the price of a noisy gauge — which is the direct argument for the *Gauge R&R* section: fix the gauge and you can move the guardband back out and recover yield.

10.10 Statistical Process Control (SPC)

Capability is a snapshot; SPC is the movie. A control chart plots a statistic over time against a centerline (CL) and control limits (UCL/LCL) set at $\pm 3\sigma$ **of the plotted statistic** — derived from the process, not the spec. A point outside the limits, or any rule trip, signals an **assignable cause**: stop, investigate, correct. Random scatter inside the limits is **common cause** — do *not* react to it (over-adjusting an in-control process *adds* variance; Deming's funnel).

10.10.1 Variables charts (measurements)

Run in pairs — one for location, one for spread:

- $\bar{X}$ and R (subgroup means and ranges, $n = 2\text{--}9$):

$$\text{UCL/LCL}_{\bar{X}} = \bar{\bar{X}} \pm A_2 \bar{R}, \quad \text{UCL}_R = D_4 \bar{R}, \quad \text{LCL}_R = D_3 \bar{R}, \quad \hat{\sigma} = \bar{R}/d_2.$$

- $\bar{X}$ and s (preferred for $n \gtrsim 10$): $\hat{\sigma} = \bar{s}/c_4$, with B_3, B_4 for the s -chart limits.
- **I-MR** (individuals + moving range) when $n = 1$ — one expensive board per run: $\hat{\sigma} = \overline{MR}/d_2$ with $d_2 = 1.128$.

n	A2	D3	D4	d2	c4
2	1.880	0	3.267	1.128	0.7979
3	1.023	0	2.574	1.693	0.8862
4	0.729	0	2.282	2.059	0.9213
5	0.577	0	2.114	2.326	0.9400

Worked $\bar{X}/R$. $n = 5$, $\bar{X} = 20.0$ min, $\bar{R} = 1.2$: $UCL_{\bar{X}} = 20 + 0.577(1.2) = 20.69$, $LCL = 19.31$; $UCL_R = 2.114(1.2) = 2.54$; $\hat{\sigma} = 1.2/2.326 = 0.516$ min — and *that* $\hat{\sigma}$ is the short-term sigma you feed into Cp/Cpk.

10.10.2 Attribute charts (counts / pass-fail)

- ***p*-chart** — fraction defective, variable subgroup size: limits $\bar{p} \pm 3\sqrt{\bar{p}(1-\bar{p})/n}$.
- ***np*-chart** — number defective, fixed subgroup size.
- ***c*-chart** — defect *count* per unit, constant area of opportunity (Poisson): limits $\bar{c} \pm 3\sqrt{\bar{c}}$.
- ***u*-chart** — defects per unit, variable area.

The $\sqrt{\bar{c}}$ in the *c*-chart is the Poisson standard deviation from the Poisson distribution — the same math that flags a bad-solder excursion on a board.

10.10.3 CUSUM and EWMA — catching small, slow shifts

Shewhart $\pm 3\sigma$ limits are fast on big jumps but slow on small sustained drifts (a 1σ shift can take ~40 points to trip). Two charts fix that by remembering history:

- **CUSUM** accumulates the running sum of deviations from target; a small persistent bias builds a visible slope long before any single point goes out. Best for detecting a known shift size quickly.
- **EWMA** plots an exponentially weighted moving average $z_t = \lambda x_t + (1 - \lambda)z_{t-1}$ (typically $\lambda = 0.2$); it smooths noise and reacts to gradual drift while staying robust to non-normality.

Use these on a mature, stable parameter where the failure mode is slow drift (tool wear, calibration creep), not sudden breakage.

10.10.4 Western Electric / Nelson rules

$\pm 3\sigma$ alone misses patterns. The WE rules add sensitivity by zone — **A** = $2-3\sigma$, **B** = $1-2\sigma$, **C** = $0-1\sigma$:

```
WE Rule 1: any 1 point beyond 3 sigma -> large shift / outlier
WE Rule 2: 2 of 3 consecutive in Zone A or beyond, same side-> moderate shift
WE Rule 3: 4 of 5 consecutive in Zone B or beyond, same side-> small sustained shift
WE Rule 4: 8 consecutive points on one side of CL -> mean has shifted
```

Common Nelson additions: 6 steadily rising/falling (trend — tool wear); 14 alternating (over-control or two interleaved streams); 15 in Zone C (unnaturally low spread — often a *measurement* problem or wrong limits).

The false-alarm tradeoff. Rule 1 alone gives $\alpha \approx 0.0027$ (1 in 370 points). Stacking all rules pushes the combined false-alarm rate past ~1%, so pick a rule set deliberately. On a stable, mature line many teams run Rules 1, 2, 3, and the 8-in-a-row.

10.10.5 Part Average Testing (PAT) — outlier screening

SPC watches the *process*; **Part Average Testing (PAT)** watches the *part*. PAT is an outlier screen that flags a unit which passes every spec limit but sits far from its peers — the classic latent-defect signature on a compute board (a marginal solder joint, a leaky cap, a device drawing slightly more current than the population). The unit is “in spec but not like its neighbors,” and on a safety-critical platform that is exactly the part you want to pull before it ships. PAT was formalized in automotive (AEC-Q001) and maps directly onto Zoox compute boards.

Robust statistics first. PAT limits are built on **robust** estimators so a few outliers do not inflate the very limits meant to catch them:

- **Median** (50th percentile) for center — unmoved by a stuck or extreme reading.
- **MAD** (median absolute deviation) for spread: $MAD = \text{median}(|x_i - \tilde{x}|)$, scaled to a robust sigma by $\hat{\sigma}_{\text{robust}} = 1.4826 \cdot MAD$ (the 1.4826 makes it match σ for normal data).

Static PAT sets fixed limits from a qualified characterization population, placed *inside* the spec to catch latent outliers:

$$\text{PAT limits} = \text{robust mean} \pm 6 \hat{\sigma}_{\text{robust}} \quad (\text{tightened inside the spec line}).$$

The ± 6 robust-sigma band is the common default; if it falls outside the spec the spec governs (PAT never loosens a spec). Anything inside spec but outside the PAT band is a PAT reject.

Dynamic PAT recomputes the center and limits **per lot / per panel / per wafer** from that population's own robust mean and MAD, so the screen tracks normal lot-to-lot shifts (a different reel of caps, a new solder lot) instead of false-failing a whole good lot that simply ran a hair off the historical center. Use dynamic PAT when the part-to-part distribution legitimately moves between lots but the *within-lot* outliers are still the defect signal.

Worked — board supply current. Characterization gives robust mean = 2.00 A, MAD = 0.012 A, so $\hat{\sigma}_{\text{robust}} = 1.4826(0.012) = 0.0178$ A and the static PAT band is $2.00 \pm 6(0.0178) = 2.00 \pm 0.107 = [1.893, 2.107]$ A. The customer spec is 1.7–2.3 A. A board reading 2.18 A **passes spec** but **fails PAT** — it is a 10σ -robust outlier from its peers, pulled as a latent-defect risk. On the next lot, dynamic PAT recomputes the center (say robust mean 2.03 A) and re-centers the band so a uniformly higher-but-tight lot is not wrongly scrapped.

PAT vs SPC vs spec. Spec limits protect the *customer* (fixed, from design). Control limits (the variables-charts discussion) protect the *process* (from $\bar{R}/d_2$, on subgroup statistics). PAT limits protect against the *individual latent outlier* (from robust part-population statistics, tightened inside spec). Three different jobs — do not substitute one for another.

10.11 Gauge R&R / Measurement Systems Analysis

Before trusting a single measurement, prove the *measurement system* itself is capable. A wandering fixture inflates apparent part variation and tanks your Cpk — so you GR&R the station *before* you blame the product. Observed variance decomposes as

$$\sigma_{\text{total}}^2 = \sigma_{\text{part}}^2 + \sigma_{\text{measurement}}^2, \quad \sigma_{\text{measurement}}^2 = \sigma_{\text{repeatability}}^2 + \sigma_{\text{reproducibility}}^2.$$

- **Repeatability (EV, equipment variation):** same operator, same part, same gauge, repeated — the gauge's own scatter.
- **Reproducibility (AV, appraiser variation):** different operators / stations / fixtures on the same parts — setup-to-setup variation.
- **Gauge R&R** = $\sqrt{\text{EV}^2 + \text{AV}^2}$.

A standard crossed study: 10 parts $\times$ 3 operators $\times$ 3 trials = 90 measurements (ANOVA method preferred). Two acceptance metrics:

$$\% \text{GR\&R} = \frac{\sigma_{\text{GRR}}}{\sigma_{\text{total}}} \times 100\%, \quad \text{ndc} = 1.41 \frac{\sigma_{\text{part}}}{\sigma_{\text{GRR}}}.$$

%GR&R (study var)	Verdict
< 10%	acceptable
10 - 30%	marginal - accept on cost/criticality
> 30%	unacceptable - fix the gauge/fixture first

ndc (number of distinct categories) should be ≥ 5 — fewer and the gauge can barely tell parts apart.

Worked. $\sigma_{\text{total}} = 1.00$, $\sigma_{\text{GRR}} = 0.25$: %GR&R = 25% (marginal). $\sigma_{\text{part}} = \sqrt{1.00^2 - 0.25^2} = 0.968$, $\text{ndc} = 1.41(0.968/0.25) = 5.46 \rightarrow 5$ — just acceptable. The lesson connects straight to *Setting limits and guardbands*: that 0.25 of gauge sigma is the uncertainty your guardband must cover.

Pin the GR&R math to a worked example, and demand ≥ 2 operators. A GR&R *implementation* is easy to get subtly wrong: an EMS-divisor swap (dividing a variance component by the wrong $o \cdot r$ vs $p \cdot r$) shifts the part/operator split *without* breaking the sum-of-squares decomposition identity — so a test that only checks “the SS components add up to the total” stays green on a wrong answer. Pin the full ANOVA table (SS / MS / F / variance components / %GR&R) to a **published AIAG worked example** so a divisor drift fails the test against absolute numbers. And reproducibility (AV) is **not estimable with one operator** — a single-appraiser study has zero operator degrees of freedom; reject $o < 2$ rather than silently reporting $\text{AV} = 0$ and “capable.” (Both were real gaps the toolkit’s MSA audit closed.)

10.11.1 Bland-Altman and tester-to-tester / Contract Manufacturer correlation

GR&R answers “is this one station’s measurement system capable?” The next question is “do two stations — or my station and the Contract Manufacturer (CM)’s (CM) — *agree*?” When the same boards run on tester A and tester B (or in-house vs CM), you must prove the two read the same value before you trust a number that crosses sites. A naive correlation coefficient r is the wrong tool: two testers can have $r = 0.99$ yet a constant 0.3 A offset, which r is blind to. The right tool is the **Bland-Altman** (difference-vs-mean) plot.

For each part i measured on both, compute the **difference** and the **mean**:

$$d_i = A_i - B_i, \quad m_i = \frac{1}{2}(A_i + B_i).$$

Plot d_i against m_i . Three numbers come off it:

- **Bias** (mean difference) $\bar{d}$ — the systematic offset between the two testers. A non-zero $\bar{d}$ is a calibration difference to fix, not noise.
- **Limits of agreement (LoA)**: $\bar{d} \pm 1.96 s_d$, where s_d is the standard deviation of the differences. ~95% of part-to-part disagreements fall inside this band.
- **Trend**: if d_i grows with m_i (a fan-out or slope on the plot), the testers disagree more at one end of the range — a gain/scale mismatch, not just an offset.

You then ask the engineering question the plot frames: **is the LoA band narrow enough to be acceptable** relative to the spec width and the guardband (*Setting limits and guardbands*)? If the limits of agreement eat a meaningful fraction of the tolerance, a board passing at the CM could fail in-house (or vice versa) — an escape or a yield-loss source purely from measurement disagreement.

Worked — in-house vs CM, board supply current. Across 30 boards the differences (in-house minus CM) average $\bar{d} = +0.04$ A with $s_d = 0.05$ A. Bias = +0.04 A (in-house reads high — chase the shunt calibration), and LoA = $0.04 \pm 1.96(0.05) = [-0.058, +0.138]$ A. Against a 1.7–2.3 A spec (0.6 A wide), the LoA spans about 0.20 A — roughly **a third of the full tolerance** (each half-band is ~0.1 A, ~17% of the spec width), and the $s_d = 0.05$ A of disagreement is itself ~2x the 0.025 A GR&R sigma of a single station. That is large enough that the two sites need a correlation offset applied (or the in-house shunt re-calibrated) before either site’s pass is honored at the other: with the bias uncorrected, a board reading 2.28 A in-house ships, while the same board at the CM reads ~2.24 A and also ships — but a board near the low edge can straddle the limit and pass at one site, fail at the other. The same difference-vs-mean method validates a new tester against the incumbent before it joins the fleet.

Why not just r ? Correlation measures *association*, agreement measures *sameness*. Two testers tracking each other perfectly with a fixed offset have $r \approx 1$ and a non-zero bias; Bland-Altman shows the offset, r hides it. Report bias and LoA for tester-to-tester and CM correlation, not r .

The CI on the bias is a mean — use Student- t , not 1.96. The limits of agreement $\bar{d} \pm 1.96 s_d$ are a *reference interval* (where ~95% of differences fall), so the normal 1.96 is correct there. But

the **confidence interval on the bias itself** is a CI on a mean, so it takes $t_{0.975, n-1}$: 12.7 at $n = 2$, 2.23 at $n = 10$, reaching 1.96 only as $n \rightarrow \infty$. Using 1.96 for a small- n bias CI quietly understates it. The slope/intercept complement to Bland-Altman is **Deming regression** (an errors-in-variables fit, because *both* testers are noisy — ordinary least squares assumes a perfect x and is wrong here). If you bootstrap a Deming CI, **drop degenerate resamples**: a resample whose x -values are all equal has zero covariance and no defined slope; counting it as a spurious slope-0 “fit” drags the CI’s lower edge to 0, so the slope CI spuriously contains 1 and you falsely conclude the two testers agree. (That false-agreement-at-small- n bug was a real find in the toolkit’s station-correlation audit.)

10.12 Sampling and AQL

You rarely test 100% — you pull a sample and infer the lot. Sampling plans formalize the risk you take by *not* inspecting everything.

10.12.1 Confidence intervals on a proportion (yield)

For yield $\hat{p} = x/n$, large-sample normal approximation:

$$\hat{p} \pm z_{\alpha/2} \sqrt{\frac{\hat{p}(1-\hat{p})}{n}}.$$

Yield $\hat{p} = 0.95$ on $n = 400$: SE = 0.0109, 95% CI = (0.929, 0.971). You cannot honestly claim “96% yield” from this lot at 95% confidence. For small x or p near 0/1, switch to an exact method — the zero-failure case is exactly the **rule of three**: 0 fails in n gives an upper 95% bound $\approx 3/n$ (the bit-domain twin of the zero-error “3/BER” rule in *BER and Confidence*).

Sample size to hit a margin E : $n \approx z^2 p(1-p)/E^2$, worst case at $p = 0.5$. 95% CI, $\pm 3\%$: $n = 1.96^2(0.25)/0.03^2 \approx 1068$. Note margin shrinks only as $\sqrt{n}$ — halving the CI width needs 4× the units.

10.12.2 AQL, LTPD, and the OC curve

An **acceptance sampling plan** is defined by sample size n and accept number c : inspect n units, accept the lot if defects $\leq c$. Its behavior is the **Operating Characteristic (OC) curve** — P(accept) vs the true lot defect rate:

```
P(accept)
1.0 |*****__ <- AQL: good lots accepted (high P)
| *_
| *_ <- steeper c=0 curve discriminates harder
| *_
| *__
0.0 |_____*****--> lot fraction defective
AQL LTPD
```

- **AQL (Acceptable Quality Level)**: the defect rate you want *accepted* almost always. The **producer’s risk** α ($\approx 5\%$) is rejecting a lot that is actually at the AQL.
- **LTPD (Lot Tolerance Percent Defective)**: the bad rate you want *rejected* almost always. The **consumer’s risk** β ($\approx 10\%$) is accepting a lot at the LTPD.
- The plan (n, c) is chosen so the OC curve passes near $(\text{AQL}, 1 - \alpha)$ and (LTPD, β) .

P(accept) is just a binomial tail: $P(\text{accept}) = \sum_{k=0}^c \binom{n}{k} p^k (1-p)^{n-k}$. Plan $n = 50$, $c = 1$, lot at $p = 2\%$: $P(\text{accept}) = 0.98^{50} + 50(0.02)(0.98^{49}) = 0.364 + 0.372 = 0.736$.

The c=0 plan (accept-on-zero). Setting $c = 0$ gives the steepest possible OC curve for a given n and is the modern default for safety-critical hardware: a single defect rejects the lot.

$P(\text{accept}) = (1 - p)^n$, so to be 95% sure of catching a 1% lot you need n with $(0.99)^n \leq 0.05 \Rightarrow n \approx 300$. Zero-acceptance is unforgiving by design — which is the point on a robotaxi compute board.

10.13 Yield: FPY, RTY, Throughput, and Cost of Test

Yield is where probability meets the P&L, and it sits right next to capability (*Process Capability and Setting Limits*) and sampling (*Sampling and AQL*) because the same defect fractions drive all three. The trap to avoid: **step yields multiply, they do not average.**

10.13.1 First Pass Yield and Rolled Throughput Yield

- **First Pass Yield (FPY)** of one step = $\frac{\text{units passing without rework}}{\text{units in}}$.
- **Rolled Throughput Yield (RTY)** across k independent steps = $\prod_{i=1}^k \text{FPY}_i$ — the probability a unit clears the *entire* line clean the first time.
- **Final / Test Yield** counts units that eventually pass (after rework). $\text{FPY} \leq \text{final yield}$.
- **Normalized Yield** = $\sqrt[k]{\text{RTY}}$ — the average per-step yield, for comparing lines with different step counts.

Worked. Two independent steps fail 3% and 5%: $\text{RTY} = 0.97 \times 0.95 = 0.9215$ (92.15%). Five steps each at 99%: $\text{RTY} = 0.99^5 = 0.951$ — a line that is “99% at every step” still loses ~5% end-to-end. Ten steps at 99%: $0.99^{10} = 0.904$. Step count is a yield tax. *On a Zoox compute board* the in-line gates (PCIe link, GPU Error-Correcting Code (ECC) scrub, NVMe/Double Data Rate (DDR) soak, Gigabit Multimedia Serial Link (GMSL) camera-lock) each carry an FPY; their product is what you start-quantity against to hit a ship target.

10.13.2 DPMO and DPPM — defects per million

Two related “per million” metrics convert defect counts into a comparable scale and bridge straight to the Cpk-to-PPM table in *The “Sigma Level” Bridge* and the four-indices discussion:

- **DPMO (defects per million opportunities)**. A unit can fail in several *independent ways* (opportunities); DPMO normalizes by them. With D defects over U units at O opportunities each:

$$\text{DPMO} = \frac{D}{U \cdot O} \times 10^6.$$

- **DPPM / DPM (defective parts per million)**. Counts *defective units*, not defects, regardless of how many ways each could fail:

$$\text{DPPM} = \frac{\text{defective units}}{\text{total units}} \times 10^6.$$

A board with many components has many opportunities, so its DPMO is far smaller than its DPPM — quoting the wrong one flatters or damns a line unfairly. Use **DPPM** for “what fraction of boards are bad” (the customer’s view) and **DPMO** for “how clean is each solder joint / placement / net” (the process view).

Worked. 2 defects across 100 boards, 50 opportunities each: $\text{DPMO} = 2/(100 \cdot 50) \times 10^6 = 400$. If those 2 defects landed on 2 distinct boards, $\text{DPPM} = 2/100 \times 10^6 = 20,000$ — same data, two very different headline numbers.

The bridge to capability. A DPMO (or one-sided PPM) is just a normal-tail defect fraction, so it maps onto the sigma/Cpk table in *The “Sigma Level” Bridge* directly. With the 1.5σ **long-term shift** convention, the *long-term sigma level* is

$$\text{sigma level} \approx 0.8406 + \sqrt{29.37 - 2.221 \ln(\text{DPMO})}.$$

This is the inverse of the conversion table, but note the convention: the **sigma level it returns already includes the 1.5 σ shift**, so it is *not* the same number as the short-term “sigma to spec” column in that table. The canonical anchor points (long-term sigma level, then DPMO):

$$3\sigma \leftrightarrow 66,800, \quad 4\sigma \leftrightarrow 6210, \quad 5\sigma \leftrightarrow 233, \quad 6\sigma \leftrightarrow 3.4.$$

Strip the 1.5 σ shift back out and the famous **6 σ level = 4.5 σ short-term = C_{pk} of 1.5 = 3.4 PPM** reconciliation falls right out — the same 1.5 σ -shift logic as the Cp-vs-Ppk gap in *Process Capability and Setting Limits*. So a DPMO target and a Cpk target are the same requirement in two dialects — but confirm *which* convention (short-term Cpk vs shifted sigma level) the other party is quoting before you agree to a number.

Worked — a GMSL camera-lock station. A new station logs 18 camera-lock failures across 60 boards, each board exercising 4 GMSL links (so 4 lock opportunities per board): $DPMO = 18/(60 \cdot 4) \times 10^6 = 75,000$. Plugging in, sigma level = $0.8406 + \sqrt{29.37 - 2.221 \ln(75,000)} = 0.8406 + \sqrt{29.37 - 24.93} = 0.8406 + 2.11 = 2.95$ — call it a **3 σ -level** station, squarely in “needs-improvement” territory and nowhere near the 4 σ ($C_{pk} \approx 1.33$) capable floor you would gate a robotaxi compute board at.

10.13.3 Throughput, takt, and station count

Capacity per station per shift = $\frac{\text{available minutes}}{\text{cycle time}}$. Stations needed = $\lceil \frac{\text{demand} \times \text{cycle time}}{\text{available minutes}} \rceil$. **Takt time** = $\frac{\text{available time}}{\text{demand}}$ is the drumbeat the line must hit.

Worked. Test = 20 min, 3% fail and get a 10 min retest, demand 200 boards in an 8-h shift, 2 stations. Per-station capacity = $480/20 = 24$ first-pass tests; total $2 \times 480 = 960$ station-minutes vs first-pass need $200 \times 20 = 4000$ min. $4000/960 = 4.17$ shifts of work — **you cannot do 200 in one shift with 2 stations**. Minimum stations = $\lceil 200 \times 20/480 \rceil = \lceil 8.33 \rceil = 9$ (before even counting the 3% retests). A long BER or NVMe/DDR soak (*BER and Confidence*, the *Reliability* section) blows up the cycle time, so a soak station is usually parallelized — many DUTs per station — rather than run in series at takt.

10.13.4 Cost of test and where to put the gate

False-reject (scrap good, the α cost) and false-accept (ship bad, the β /escape cost) trade against test time. The escape cost compounds downstream: catching a defect at board test might cost \$50; the same defect found at vehicle integration costs orders of magnitude more (the classic 10x-per-stage rule of thumb). That asymmetry is why the Bayes math in *Conditional probability and Bayes* matters — and why on a high-yield line a high false-positive rate, not low sensitivity, is usually what bleeds money. Place the gate where the marginal escape cost first exceeds the marginal test cost: cheap, high-coverage screens early; expensive soaks (BER, HTOL) reserved for what the cheap screens cannot see.

10.14 Reliability: Bathtub, MTBF, FIT, and Acceleration

Reliability is where probability meets the field-return rate. The failure rate over life follows the **bathtub curve** (the three Weibull regions from the Weibull distribution):

1. **Infant mortality** (decreasing rate): manufacturing defects. **Burn-in** (operate hot, 55–85°C, hours–days) accelerates and screens these so survivors land on the flat.
2. **Useful life** (constant rate): random failures — exponential applies, MTBF is meaningful.
3. **Wear-out** (increasing rate): electromigration, NAND wear, aging — Weibull, $\beta > 1$.

10.14.1 MTBF and FIT

For the constant-hazard region, $\text{MTBF} = 1/\lambda$. The industry quotes the same λ as a **FIT rate** — **F**ailures **I**n **T**ime, failures per 10^9 device-hours:

$$\text{FIT} = \lambda \times 10^9 \frac{\text{failures}}{\text{device-hour}}, \quad \text{MTBF (h)} = \frac{10^9}{\text{FIT}}.$$

A part rated 100 FIT: $\lambda = 10^{-7}/\text{h}$, $\text{MTBF} = 10^7$ h. FIT is additive across parts (like λ), which is why component datasheets quote it — you sum the board.

FIT to the number that matters: field returns. MTBF in hours is abstract; the program manager wants an **annualized failure rate (AFR)** and a spares budget. For a small annual hazard, $\text{AFR} = 1 - e^{-\lambda \cdot 8760} \approx \lambda \cdot 8760$ (8760 h/yr), and the expected returns from a fleet of N continuously-running boards over a year is just $N \cdot \lambda \cdot 8760$. *A board summing to 50 FIT* ($\lambda = 5 \times 10^{-8}$): $\text{AFR} \approx 5 \times 10^{-8} \times 8760 = 4.4 \times 10^{-4}$, i.e. $\sim 0.044\%$ /yr, or ~ 4.4 returns per 10,000 boards per year — the number you size the Return Merchandise Authorization (RMA) pipeline against. The inverse direction sets a *requirement*: “no more than 100 returns/yr across a 50,000-board fleet” is $\lambda \leq 100/(50,000 \times 8760) = 2.3 \times 10^{-7}/\text{h}$, i.e. a **board FIT budget of ~ 228** that you then allocate down to components.

10.14.2 Series and redundant systems

Series (any one failure kills the system) — failure rates add:

$$\lambda_{\text{sys}} = \sum_i \lambda_i, \quad \text{MTBF}_{\text{sys}} = 1/\lambda_{\text{sys}}.$$

Board: 4 GPU (50,000 h each), 2 NVMe (200,000 h), 1 NIC (500,000 h): $\lambda = 8 \times 10^{-5} + 1 \times 10^{-5} + 2 \times 10^{-6} = 9.2 \times 10^{-5}/\text{h}$, $\text{MTBF} = 10,870$ h ≈ 1.24 yr continuous. In FIT: 92,000 FIT for the board. Run that through the AFR bridge above and it is sobering: $\text{AFR} = 1 - e^{-9.2 \times 10^{-5} \times 8760} = 55\%$ — more than half of these boards would fail within a year of continuous operation. The 4 GPUs at 50,000 h each dominate (8 of the 9.2 in the λ sum), and *that* is the quantitative argument for the redundancy below: a serial stack of high-power parts simply cannot hit a robotaxi availability target on its own.

Redundant / k-of-n — system works if enough units do (binomial, the binomial distribution): $R_{\text{sys}} = 1 - \prod_i (1 - R_i)$ for full parallel. *Need ≥ 3 of 5 GPUs, each 99%:* $P(\geq 3) = 0.9510 + 0.0480 + 0.00097 = 0.9999$. Redundancy crushes the failure probability — the architectural reason a safety-critical compute platform carries spare lanes and compute.

10.14.3 Acceleration models — Arrhenius

Reliability lab tests run hot to fail parts faster, then you extrapolate to use temperature. The **Arrhenius model** governs temperature-driven (chemical/diffusion) failure mechanisms:

$$\text{AF} = \exp \left[\frac{E_a}{k} \left(\frac{1}{T_{\text{use}}} - \frac{1}{T_{\text{stress}}} \right) \right],$$

with activation energy E_a (eV), Boltzmann $k = 8.617 \times 10^{-5}$ eV/K, and temperatures in **kelvin**. AF is the multiplier by which life shrinks at the stress temperature.

Worked. $E_a = 0.7$ eV, use $55^\circ\text{C} = 328$ K, stress $125^\circ\text{C} = 398$ K:

$$\text{AF} = \exp \left[\frac{0.7}{8.617 \times 10^{-5}} \left(\frac{1}{328} - \frac{1}{398} \right) \right] = \exp[8124 \times 5.36 \times 10^{-4}] = e^{4.36} \approx 78.$$

So **1 hour at $125^\circ\text{C} \approx 78$ hours at 55°C** — a 1000-hour HTOL soak demonstrates $\sim 78,000$ use-hours (~ 9 years) of thermal life. That single number is how a burn-in or HTOL plan gets justified to a program manager. (Voltage and humidity stresses use companion models — Eyring, Coffin-Manson for thermal-cycle fatigue — but Arrhenius is the one you will quote most.)

AF is exponentially sensitive to E_a — pick it honestly. Hold the same $55 \rightarrow 125^\circ\text{C}$ but vary the activation energy: $E_a = 0.3$ eV gives $\text{AF} = 6.5$, 0.7 eV gives 78 , 1.0 eV gives 504 . The *same* 1000-hour soak then “demonstrates” anywhere from 0.74 to 57 years of life depending on a number you assumed. Use the *mechanism*’s published E_a (electromigration ~ 0.7 eV, oxide/dielectric breakdown ~ 0.3 – 0.7 eV, some ionic-contamination mechanisms ~ 1.0 eV), and when a failure mode is unknown, use a **conservative low E_a** — guessing high inflates your demonstrated life and is exactly how an under-screened part reaches a robotaxi. A NVMe or DDR soak gated on Arrhenius is only as trustworthy as the E_a behind it.

10.15 Regression and Correlation

When you need to relate two test parameters — predict a final value from an in-line reading, or check whether a fixture knob actually moves a measurement — you reach for correlation and linear regression.

Correlation r measures linear association, $-1 \leq r \leq 1$:

$$r = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum (x_i - \bar{x})^2} \sqrt{\sum (y_i - \bar{y})^2}}.$$

$r = 0$ means no *linear* relationship (it can still be curved); $|r|$ near 1 means tight linear fit. r^2 is the fraction of variance in y explained by x .

Least-squares line $\hat{y} = b_0 + b_1x$:

$$b_1 = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sum (x_i - \bar{x})^2} = r \frac{s_y}{s_x}, \quad b_0 = \bar{y} - b_1\bar{x}.$$

Use on the floor. Fit final post-burn-in leakage against an in-line room-temperature reading; if $r^2 = 0.9$ the cheap early measurement predicts the expensive late one well enough to screen early and save burn-in slots. The air-brake fixture model $\tau = \text{slope} \cdot \text{PSI} + \text{intercept}$ from the platform chapter is exactly this — a regression fit turned into a test limit.

Correlation is not causation, and watch your range. A strong r over a narrow tested range can vanish or flip outside it; never extrapolate a fit past the data that built it. And a lurking variable (ambient temperature drifting with time of day) can manufacture correlation between two unrelated readings.

10.16 Fast Estimation and Mental Math for the Floor

Half the math you do at a station is a 10-second sanity check, not a derivation. These are the shortcuts that keep you from chasing a phantom or quoting a number that is off by a decade.

Orders of magnitude and the rule of 72-ish. - Powers of two: $2^{10} \approx 10^3$, so $2^{20} \approx 10^6$, $2^{30} \approx 10^9$. A 32-bit counter wraps at $\sim 4 \times 10^9$. - dB shortcuts: $\times 2 = +3$ dB, $\times 10 = +10$ dB. A $\times 8$ gain is $3 \times 3 = 9$ dB. - “3/BER” for run time (the zero-error “3/BER” rule); “3/n” rule of three for a zero-failure upper bound (the proportion-CI discussion).

BER run time in your head. $n \approx 3/p$ for 95% CL, then divide by the line rate. 10^{-12} at 32 Gb/s: 3×10^{12} bits / 3×10^{10} bit/s ≈ 100 s. Deeper targets scale linearly: 10^{-15} is $1000\times$ longer — tens of hours, or run many lanes in parallel.

Yield multiplies, it never averages. Step yields *compound*: ten steps at 99% is $0.99^{10} \approx 0.90$, not 99%. Quick rule for small loss: total loss $\approx$ sum of per-step losses, so $10 \times 1\% \approx 10\%$ — close to the exact 9.6%. To ship N good units, *start* N/RTY and budget the scrap.

Poisson zero-defect feel. $P(\text{zero}) = e^{-\lambda}$ with $\lambda = np$. At $\lambda = 1$ you have a 37% chance of a clean unit; at $\lambda = 0.1$, 90%; at $\lambda = 3$, only 5%. Lets you eyeball “is a defect-free lot plausible?” instantly.

Sigma-to-ppm anchors. $3\sigma \rightarrow 1350$ ppm, $4\sigma \rightarrow 32$ ppm, $5\sigma \rightarrow 0.3$ ppm — each extra sigma is roughly a 30–100× drop. So Cpk 1.0 $\rightarrow$ 1.33 is two orders of magnitude fewer escapes; that is the payoff you cite when arguing for a process improvement.

Throughput / takt. Capacity per station = available min/cycle time; stations needed = $\lceil \text{demand} \times \text{cycle/available} \rceil$. 20-min test, 480-min shift, 200 boards: $200 \times 20/480 = 8.3 \rightarrow$ **9 stations** — before retests. Do this before promising a build rate.

Sanity-check every answer. Units, order of magnitude, and direction. “Cpk went *up* when I widened the spec” — correct. “BER bound got *lower* when I saw *more* errors” — wrong, recheck. Catching your own decade error is the cheapest quality gate on the floor.

10.17 One-Page Formula Sheet

PROBABILITY / BAYES

$P(A \text{ or } B) = P(A) + P(B) - P(A \text{ and } B)$ $P(A \text{ and } B) = P(A)P(B|A)$
 Bayes: $P(A|B) = P(B|A)P(A)/P(B)$ $P(B) = P(B|A)P(A) + P(B|A')P(A')$
 PPV = $P(\text{defective} | \text{FAIL})$ (sensitivity, specificity, prevalence)
 $E[aX+b] = aE[X] + b$ $\text{Var}(aX+b) = a^2 \text{Var}(X)$ $\text{sigma_sum} = \sqrt{\sum \text{sigma}_i^2}$

DISTRIBUTIONS

Binomial: $C(n,k) p^k (1-p)^{(n-k)}$ mean np
 Poisson: $\text{lam}^k e^{-\text{lam}} / k!$ mean=var= lam (BER, defect counts)
 Geometric: $(1-p)^{(k-1)} p$ mean $1/p$
 Normal: $Z = (X - \mu) / \text{sigma}$ 68-95-99.7
 Exponential: $P(X > t) = e^{-(\text{lam } t)}$ mean $1/\text{lam}$, memoryless (useful life)
 Weibull: $P(X > t) = \exp(-(t/\eta)^\beta)$ $\beta < 1$ infant, $\beta = 1$ random, $\beta > 1$ wear-out

GEOMETRY

hexagon area = $(3 \sqrt{3} / 2) s^2 \sim 2.598 s^2$ perimeter $6s$
 equilateral triangle area = $(\sqrt{3}/4) s^2$
 small angle: distance = size / angle(rad)
 camera FOV = $2 \text{atan}(\text{sensor_width} / (2 f))$; width@R = $2 R \tan(\text{half-FOV})$
 3D n^3 collinear-triple lines = $3n^2 + 6n + 4$ ($n=3 \rightarrow 49$)
 point-to-line (3D): $|AP \times d| / |d|$
 2D rotation: $x' = x \cos - y \sin$, $y' = x \sin + y \cos$

SIGNAL / ALGEBRA

dB_power=10 $\log_{10}(P1/P2)$ dB_amp=20 $\log_{10}(V1/V2)$ x2=3dB x10=10dB
 GB/s per lane = GT/s * coding_eff / 8 (8b10b=0.8, 128b130b=0.9846)
 noise averaging: $\text{sigma_avg} = \text{sigma} / \sqrt{N}$
 uncertainty: add (sigma) or relative-(sigma) in quadrature
 Nyquist: $f_s \geq 2 f$
 vectors: $a \cdot b = |a||b|\cos \theta$; $|a \times b| = |a||b|\sin \theta$
 physical: $F = P \cdot A$; $V = I \cdot R$, $P = VI = I^2 R = V^2/R$; $\tau = F \cdot d$; $Q = m \cdot c \cdot dT$

BER / BERT (zero-error acceptance)

$n = -\ln(1-CL)/p$ $t = n/\text{line_rate}$ proven BER = $-\ln(1-CL)/n$
 with E errors: BER_upper = $\chi^2(1-\alpha, 2(E+1)) / (2n)$
 $CL = 1 - \text{PoissonCDF}(E; n \cdot p)$
 $-\ln(1-CL)$: 90%=2.303 95%=2.996 99%=4.605 99.9%=6.908 ($\sim 3/p$ rule)
 SPRT: pass below accept line, fail above reject line, else continue

CAPABILITY / LIMITS / OUTLIERS

```

Cp = (USL-LSL)/(6 sigma_ST)
Cpk= min(USL-mu, mu-LSL)/(3 sigma_ST) Pp/Ppk use overall s
Cpk 1.33 ~ 32ppm 1.67 ~ 0.3ppm 2.0 ~ 0.001ppm (3.4 w/1.5 shift)
test limit = spec +/- guardband; guardband ~ k * gauge uncertainty
robust sigma = 1.4826 * MAD MAD = median(|x - median|)
static PAT = robust_mean +- 6 robust_sigma (tightened inside spec)
dynamic PAT: recompute center/limits per lot/panel

```

SPC

```

Xbar: xbarbar +- A2 Rbar R: D4 Rbar, D3 Rbar sigma_ST = Rbar/d2
c-chart: cbar +- 3 sqrt(cbar) p-chart: pbar +- 3 sqrt(pbar(1-pbar)/n)
CUSUM / EWMA catch small slow shifts; WE rules: 1past3 / 2of3 A / 4of5 B / 8side

```

YIELD

```

RTY = product of step FPYs normalized = RTY^(1/k) start N/RTY to ship N
DPMO = D/(U*0) * 1e6 DPPM = defective_units/total * 1e6
capacity/station = avail_min / cycle stations = ceil(demand*cycle / avail)
takt = available_time / demand

```

GAUGE / MSA / SAMPLING / RELIABILITY

```

GR&R = sqrt(EV^2+AV^2) %GRR<10 good >30 bad ndc=1.41 part/GRR >=5
Bland-Altman: d=A-B, m=(A+B)/2; bias=mean(d), LoA = mean(d) +- 1.96 sd(d)
CI(prop) = phat +- z sqrt(phat(1-phat)/n) n ~ z^2 p(1-p)/E^2
AQL plan (n,c): P(accept)=sum_{k=0..c} C(n,k)p^k(1-p)^(n-k) c=0 = accept-on-zero
series: lam_sys=sum lam_i MTBF=1/lam FIT=lam*1e9 MTBF=1e9/FIT
Arrhenius AF = exp[(Ea/k)(1/Tuse - 1/Tstress)] k=8.617e-5 eV/K, T in kelvin

```

REGRESSION

```

r = cov / (sx sy) b1 = r sy/sx b0 = ybar - b1 xbar r^2 = var explained

```

Chapter 11

Networking: Fundamentals to the Manufacturing Floor

Networking shows up in this role in four concrete ways, each pulling the material in a different direction:

1. **The test station talks to instruments and dashboards over TCP/IP.** The ACT3 power-supply client, `equipment_rpc` JSON-over-TCP, the heartbeat HTTP POST, and the FastAPI results dashboard are all ordinary network programs. When one “hangs” or reports “Disconnected,” you debug it as a network problem.
2. **You test Network Interface Cards (NICs) on compute boards as a manufacturing step.** Enumerate the card, confirm link/speed/duplex, run a self-test, push traffic to a partner with `iperf3`, and gate on error counters. A Network Interface Card (NIC) that links but quietly drops 0.1% of frames is a reject — catch it on the line, not in the field.
3. **The Zoxx compute platform uses automotive Ethernet between modules** — single-pair 100/1000BASE-T1 — which behaves differently enough from office RJ45 that it gets its own section. See also the Automotive chapter for the full 100/1000BASE-T1 context; this chapter focuses on the test and diagnostic angle.
4. **You debug failures with command-line tools.** `ip`, `ethtool`, `ping`, `ss`, `tcpdump`, `iperf3`, `nmap`, `mtr`, `dig`, `nc` — used with a layered methodology so you isolate the fault fast.

The throughline of this chapter: **think in layers**. Almost every networking failure, whether on the bench or in a test script, is answered fastest by asking “which layer is broken?” and starting from the bottom.

11.1 The OSI and TCP/IP Models

11.1.1 Two Models, One Reality

The Open Systems Interconnection (OSI) 7-layer model is the teaching model and the shared vocabulary (“that’s a Layer 2 problem”). The TCP/IP 4-layer model is what actually runs on every machine you touch. Know both; **think in TCP/IP, speak in OSI layer numbers**.

TCP/IP layer	OSI layer(s)	Example protocols	What it does	Address
Application	5-7 (Session, Presentation, Application)	HTTP, DNS, DHCP, SSH, MQTT, Modbus/TCP, your RPC	Meaning of the bytes; your code lives here	URL / hostname

TCP/IP layer	OSI layer(s)	Example protocols	What it does	Address
Transport	4 (Transport)	TCP, UDP	End-to-end delivery; reliability (TCP) or speed (UDP); multiplexing by port	Port number
Internet	3 (Network)	IP, ICMP, ARP*	Logical addressing and routing between networks	IP address
Link	1-2 (Data Link, Physical)	Ethernet, Wi-Fi, PPP, 1000BASE-T1	Framing + physical transmission on the local link	MAC address

* Address Resolution Protocol (ARP) straddles the boundary — it maps L3 IP to L2 MAC and is often called “L2.5.”

A quick mnemonic for OSI bottom-to-top: **Please Do Not Throw Sausage Pizza Away** (Physical, Data Link, Network, Transport, Session, Presentation, Application).

11.1.2 What Each Layer Actually Does

- **L1 Physical** — voltages, light, modulation, connectors, cable. “Is there a signal on the wire?” Bits only. (PAM3 on 1000BASE-T1 lives here.)
- **L2 Data Link** — frames with MAC addresses; error detection via CRC/Frame Check Sequence (FCS); media access (who talks when). Switches operate here. A CRC error or a duplex mismatch is L2.
- **L3 Network** — IP addressing, subnets, routing between networks. Routers live here. ICMP (ping) is L3. “Wrong subnet / no route to host” is L3.
- **L4 Transport** — segments the byte stream, multiplexes apps via ports, and for TCP adds reliability, ordering, flow and congestion control. “Connection refused” is L4.
- **L5 Session / L6 Presentation** — establishing/resuming sessions (TLS session resumption), encoding, serialization (JSON/UTF-8), encryption/compression. Your `json.dumps(...).encode()` is presentation work.
- **L7 Application** — the protocol your code speaks: HTTP verbs, DNS queries, your custom RPC framing.

11.1.3 Encapsulation: How a Packet Is Built and Torn Down

When your Python script calls `socket.sendall(data)`, each layer wraps the layer above in its own header (L2 also appends a trailing CRC):

```
APP | [ JSON payload ]
L4 | [ TCP hdr | JSON payload ] <- src/dst PORT, seq/ack
L3 | [ IP hdr | TCP hdr | JSON payload ] <- src/dst IP, TTL
L2 | [ Eth hdr | IP hdr | TCP hdr | JSON payload | FCS ] <- src/dst MAC, CRC-32
L1 | ...serialized as PAM/NRZ symbols on the wire...
```

The receiver does the reverse — decapsulation — stripping headers bottom-up and handing the payload to the next layer, until the peer’s `recv()` returns the JSON. Each layer reads only its own header; the payload is opaque. This is why a switch can forward a frame without understanding HTTP, and why an L4 firewall that filters on ports does not need to parse your application protocol.

Precise terminology that comes up in packet captures:

- **Frame** = L2 unit (Ethernet frame).
- **Packet** = L3 unit (IP packet, also called IP datagram).
- **Segment** = L4 TCP unit; **datagram** = L4 UDP unit.

11.1.4 Frame, MTU, and Jumbo Frames

A standard Ethernet frame: 6-byte dst MAC, 6-byte src MAC, 2-byte EtherType (0x0800 = IPv4, 0x0806 = ARP, 0x86DD = IPv6), 46–1500-byte payload, 4-byte FCS (CRC-32). The **Maximum Transmission Unit (MTU)** is the largest L3 payload — **1500 bytes** by default on Ethernet. **Jumbo frames** raise this to ~9000 bytes; they reduce per-packet overhead and CPU load and are common on storage/10GbE test links — but **every device in the path (NIC, switch, partner) must agree**, or you get silent black-holing of large packets (“ping works, big transfers hang”). That mismatch is a classic test-station gotcha: small control packets pass, bulk `iperf3` stalls. Check it:

```
ping -M do -s 1472 192.168.1.50 # don't-fragment, 1472B payload -> total IP pkt 1500B
ping -M do -s 8972 192.168.1.50 # tests jumbo (8972+28 = 9000B)
```

If the 8972-byte ping fails but the 1472-byte ping succeeds, an intermediate device is not jumbo-capable.

11.2 Ethernet, MAC Addresses, and VLANs

11.2.1 MAC Addresses

A **MAC address** is a 48-bit (6-byte) Layer-2 hardware address, written as six hex pairs: 00:1b:21:3c:4d:5e. The **first 3 bytes are the OUI** (Organizationally Unique Identifier) — the vendor. The last 3 bytes are vendor-assigned. ff:ff:ff:ff:ff:ff is the **broadcast** MAC; frames sent to it are delivered to every device on the L2 segment.

MAC addresses are **link-local** — they only matter within one broadcast domain. As a packet crosses routers, the IP addresses stay the same end-to-end but **the src/dst MAC is rewritten at every hop**. On each link, the destination MAC is the *next hop's* MAC (the router's interface), not the final destination's MAC. This is the most commonly missed point: if you capture traffic leaving a router, the dst MAC is the next router's interface, while the dst IP is still the original target.

11.2.2 ARP — Mapping IP to MAC

To send an IP packet on the local link, the OS needs the destination's MAC. ARP resolves it:

```
Host A wants to send to 192.168.1.50 (same subnet) but only knows its IP.
A broadcasts: "ARP: who has 192.168.1.50? Tell 192.168.1.10" (dst MAC = broadcast)
B replies (unicast): "192.168.1.50 is at 00:1b:21:aa:bb:cc"
A caches that mapping in its ARP table and sends the IP packet in an Ethernet frame.
```

If the destination is on a different subnet, A ARPs for the **gateway's** MAC and sends the frame there; the gateway routes onward. Inspect and manage the table:

```
ip neigh show # ARP / neighbor table (modern)
arp -n # legacy equivalent
ip neigh flush dev eth0 # clear stale entries (useful after re-cabling or IP change)
```

ARP entry states: **REACHABLE** (recently confirmed), **STALE** (cached, unverified), **FAILED** (no answer — host down, wrong subnet, or wrong Virtual Local Area Network (VLAN)), **INCOMPLETE** (resolution in progress). A neighbor stuck **INCOMPLETE** or **FAILED** for an IP you expect to reach is an L2/L3 problem: wrong VLAN, wrong subnet, dead link, or the device is off.

Gratuitous ARP: a host announces its own IP→MAC mapping unsolicited, typically on boot or after an IP change, so switches and peers update their caches. Relevant on the floor when a station is re-imaged and the old MAC lingers in a switch's forwarding table.

11.2.3 VLANs and Factory Network Segmentation

A **VLAN** (IEEE 802.1Q) logically partitions one physical switch into multiple isolated broadcast domains. Frames carry a 4-byte 802.1Q **tag** containing a 12-bit **VLAN ID** (1–4094) and a 3-bit PCP priority field. Devices in different VLANs cannot communicate at L2 — they must pass through a **router or L3 switch** (“inter-VLAN routing”), which is exactly where you enforce policy between test stations and the corporate network.

- **Access port** — belongs to one VLAN; the connected device is unaware of tags (the switch adds/removes the tag transparently).
- **Trunk port** — carries multiple VLANs tagged between switches or between a switch and a server. A test server hosting several VLAN sub-interfaces sits on a trunk.

Typical segmentation on a Zoox-style test floor:

- **Test-station VLAN** — controllers running your software, isolated from corporate.
- **Instrument VLAN** — power supplies, DMMs, BERTs at known static IPs.
- **DUT VLAN** — the board under test; a misbehaving DUT cannot disrupt other stations.
- **Monitoring/dashboard VLAN** — FastAPI dashboard, logging, results database.
- **Corporate VLAN** — tightly firewalled from the others.

Benefits: broadcast/fault isolation (a DUT broadcast storm stays in its VLAN), security (a buggy device is contained), determinism (test traffic is not competing with office traffic), and clean per-VLAN subnets. This is also why a station that “can’t reach the dashboard” sometimes just landed on the wrong VLAN after re-cabling.

Configuring a VLAN sub-interface on Linux:

```
# Create a tagged sub-interface for VLAN 100 on eth0
ip link add link eth0 name eth0.100 type vlan id 100
ip addr add 10.10.100.1/24 dev eth0.100
ip link set eth0.100 up

# Inspect (shows the VLAN ID and protocol):
ip -d link show eth0.100

# Capture only VLAN-tagged traffic for that VLAN:
tcpdump -i eth0 vlan 100
```

The switch port feeding `eth0` must be a trunk that permits VLAN 100. A frequent miss: configuring the Linux sub-interface correctly while the switch port remains an access port for a different VLAN — result is complete silence with no obvious error.

Switch-config basics a test engineer touches:

Most managed switches are configured via a web UI, a proprietary CLI (Cisco IOS-style or similar), or a REST API. The operations you’ll actually perform:

- Assign a port to an access VLAN: `switchport mode access; switchport access vlan 100` (IOS syntax).
- Configure a trunk: `switchport mode trunk; switchport trunk allowed vlan 100,200`.
- Check a port’s operational state and speed: `show interfaces GigabitEthernet0/1` — look for “connected,” the negotiated speed, and any input/output error counts.
- Check the MAC address table to see which device is learned on which port: `show mac address-table` (useful when a DUT is unresponsive and you want to confirm it is even communicating at L2).
- Enable/disable spanning-tree PortFast on test ports that connect directly to end devices (avoids the 30-second Spanning Tree Protocol (STP) listening/learning delay that makes newly connected DUTs appear dead on boot).

11.3 IP Addressing and Subnetting

11.3.1 The Basics

IPv4 is 32 bits, written as four 8-bit **octets** in dotted-decimal: 192.168.1.100. A **subnet mask** (or CIDR prefix) splits the address into a **network portion** (left, the 1-bits of the mask) and a **host portion** (right, the 0-bits).

- CIDR /24 means “the first 24 bits are network.”
- The mask 255.255.255.0 is the same /24 (24 leading 1-bits = 11111111.11111111.11111111.00000000).

In every subnet, two addresses are reserved:

- **Network address** — all host bits 0 (names the subnet).
- **Broadcast address** — all host bits 1 (reaches every host on the subnet).

So a subnet with H host bits has 2^H total addresses and $2^H - 2$ usable host addresses. Exception: /31 point-to-point links (RFC 3021) have no network/broadcast reserved — both addresses are usable. A /32 is a single-host route.

11.3.2 Reference Table

CIDR	Mask	Host bits	Usable hosts	Block size	Typical use
/8	255.0.0.0	24	16,777,214	—	10.0.0.0/8 large private
/16	255.255.0.0	16	65,534	—	172.16.0.0/16 medium site
/24	255.255.255.0	8	254	256	typical LAN / VLAN
/25	255.255.255.128	7	126	128	half a /24
/26	255.255.255.192	6	62	64	quarter /24, lab segment
/27	255.255.255.224	5	30	32	small rack
/28	255.255.255.240	4	14	16	tiny segment, mgmt
/29	255.255.255.248	3	6	8	handful of instruments
/30	255.255.255.252	2	2	4	point-to-point link
/31	255.255.255.254	1	2*	2	P2P (RFC 3021, no net/bcast)
/32	255.255.255.255	0	1	1	single-host route

* /31: both addresses are usable as the two ends of a point-to-point link.

Private ranges (RFC 1918) — never routed on the public Internet:

- 10.0.0.0/8 — 10.0.0.0 through 10.255.255.255
- 172.16.0.0/12 — 172.16.0.0 through 172.31.255.255 (note: /12, not /16)
- 192.168.0.0/16 — 192.168.0.0 through 192.168.255.255

Also: 127.0.0.0/8 loopback (127.0.0.1 = localhost); 169.254.0.0/16 link-local / APIPA — an interface that requested Dynamic Host Configuration Protocol (DHCP) and got no answer self-assigns here. Seeing a 169.254.x.x address is a dead giveaway that **DHCP failed**.

11.3.3 The Fast Method (Block Size / “Magic Number”)

You almost never need to go to binary. For any subnet:

1. Find the octet where the mask is not 0 and not 255 — the “interesting octet.”
2. **Block size = 256 - (mask value in that octet)**. This is the increment between subnet network addresses.
3. Subnet boundaries are multiples of the block size: 0, block, 2*block, ...
4. The given IP falls in whichever block bracket contains it.

5. **Network address** = lower boundary. **Broadcast** = next boundary - 1. **Usable** = network+1 through broadcast-1.

11.3.4 Worked Example A — /26

Given: 192.168.1.100/26. Find mask, network, broadcast, usable range, host count.

- /26 → 6 host bits → $2^6 = 64$ total, **62 usable**.
- Mask: 255.255.255.192 (4th octet = 11000000 = 192).
- Block size = 256 - 192 = **64**. Boundaries in 4th octet: 0, 64, 128, 192. 100 falls in the **64–127** block.
- **Network = 192.168.1.64, Broadcast = 192.168.1.127.**
- **Usable = 192.168.1.65 – 192.168.1.126** (62 addresses).

The four /26 subnets of 192.168.1.0/24:

Subnet	Network	Usable range	Broadcast
1	192.168.1.0	.1 – .62	192.168.1.63
2	192.168.1.64	.65 – .126	192.168.1.127
3	192.168.1.128	.129 – .190	192.168.1.191
4	192.168.1.192	.193 – .254	192.168.1.255

11.3.5 Worked Example B — /28 (Smaller Blocks)

Given: 10.10.50.37/28.

- /28 → 4 host bits → 16 total, **14 usable**. Mask 255.255.255.240 (4th octet = 240).
- Block size = 256 - 240 = **16**. Boundaries: 0, 16, 32, 48, 64... 37 is in **32–47**.
- **Network = 10.10.50.32, Broadcast = 10.10.50.47.**
- **Usable = 10.10.50.33 – 10.10.50.46.**

A /28 is a natural fit for a small test cell: 14 usable addresses cover a controller, a few instruments, a NIC-under-test, and the partner box.

11.3.6 Worked Example C — Interesting Octet in the Third Octet (/22)

Given: 172.16.20.5/22. People stumble here because the boundary is not in the last octet.

- /22 → mask 255.255.252.0. Interesting octet = **3rd** (11111100 = 252).
- Block size in the 3rd octet = 256 - 252 = **4**. Boundaries: 0, 4, 8, 12, 16, **20**, 24... The 3rd octet is 20, which is exactly a boundary — the subnet starts there.
- **Network = 172.16.20.0**. It spans 4 values of the 3rd octet: 20, 21, 22, 23.
- **Broadcast = 172.16.23.255** (3rd octet 23, 4th octet 255).
- **Usable = 172.16.20.1 – 172.16.23.254.**
- Host count: 10 host bits → $2^{10} - 2 = 1022$ usable.

11.3.7 Worked Example D — “How Many /27s Fit, and What Is the 5th One?”

Given: Subnet 192.168.1.0/24 into /27s.

- Block size 32 → $256 / 32 = 8$ subnets, starting at 0, 32, 64, 96, 128, 160, 192, 224.
- **5th subnet** (1-indexed): start = $4 \times 32 = 128 \rightarrow 192.168.1.128/27$.
- Network 192.168.1.128, broadcast 192.168.1.159, usable .129–.158 (30 hosts).

11.3.8 “Are These Two Hosts on the Same Subnet?” (The AND Trick)

Two hosts can communicate directly (L2, no router) only if they share the same network address. Compute `IP AND mask` for each and compare.

Given: A = 10.0.5.10/22, B = 10.0.7.200/22. Same subnet?

- /22, interesting octet = 3rd, block = 4. For A: 3rd octet 5 → block **4–7** → net 10.0.4.0. For B: 3rd octet 7 → block **4–7** → net 10.0.4.0. **Same — direct L2.**

Now B = 10.0.8.200/22: 3rd octet 8 → block **8–11** → net 10.0.8.0 $\neq$ 10.0.4.0 → **different subnets, traffic must go through the gateway**. This is the bug behind “I set a static IP and now I can’t reach the instrument” — the host and instrument ended up in different subnets, or the mask is wrong.

11.3.9 Routing Basics

The host’s kernel routing table decides where to send each packet. Each entry is essentially: “for packets destined to network X/Y, send them to next-hop Z on interface W.” The kernel matches the **most specific (longest prefix)** route first. If no specific route matches, the **default route** (0.0.0.0/0) is used, which points to the gateway.

```
ip route show
# typical output:
# default via 10.10.100.1 dev eth0 proto dhcp
# 10.10.100.0/24 dev eth0 proto kernel scope link src 10.10.100.50
# 172.16.0.0/16 via 10.10.100.254 dev eth0
```

The `proto kernel` route is auto-added for the interface’s own subnet (direct, no next hop needed). The `default via` line is what you check when an instrument on a different subnet is unreachable — if there is no route to that subnet and no default, packets are silently discarded with “No route to host.”

Add/delete routes temporarily (lost on reboot unless persisted via NetworkManager or `/etc/network/interfaces`):

```
ip route add 192.168.50.0/24 via 10.10.100.1 # add a static route
ip route del 192.168.50.0/24 # remove it
ip route add default via 10.10.100.1 # set the default gateway
```

11.3.10 IPv6 at Awareness Level

You will mostly live in IPv4 on the factory floor, but know the shape: **128 bits**, written as eight groups of four hex digits, e.g., 2001:0db8:0000:0000:ff00:0042:8329. Rules: drop leading zeros in a group and collapse one run of all-zero groups to `::` → 2001:db8::ff00:42:8329. `::1` is loopback (= IPv4 127.0.0.1); `fe80::/10` is link-local (every interface has one, used by neighbor discovery). There is **no ARP** in IPv6 — it uses **NDP** (Neighbor Discovery Protocol, Internet Control Message Protocol (ICMP) v6) instead. IPv6 link-local addresses are assigned automatically, so you may see `fe80::` addresses even on networks with no IPv6 infrastructure — they are normal.

11.4 ARP, DHCP, and DNS

11.4.1 DHCP — Dynamic Address Assignment

DHCP hands out IP, mask, gateway, DNS server, and lease time. The exchange is **DORA**:

```
Client (no IP) --- DHCPDISCOVER (broadcast, UDP src=68 dst=67) ---> server(s)
Client <-- DHCPOFFER (here is an IP, lease time) ----- server
Client --- DHCPREQUEST (I'll take that IP) -----> server
```

```
Client <-- DHCPACK (it's yours, lease=T) ----- server
```

DHCP rides on **UDP**, ports 67 (server) and 68 (client). If a host self-assigns `169.254.x.x`, no DHCP OFFER ever arrived — DHCP server down, wrong VLAN, or a cable/link problem.

Manufacturing practice: instruments, the DUT-facing NIC, and the dashboard get **static IPs** (or DHCP reservations keyed to MAC) so that “the power supply is always at 192.168.10.5” is an invariant your test code can rely on. A station that grabs a new DHCP lease mid-run and changes IP is a flaky-test root cause.

11.4.2 DNS — Names to Addresses

DNS turns `dashboard.factory.local` into an IP. Resolution order on a typical Linux host: **in-memory stub cache** → `/etc/hosts` → **configured DNS servers**. The order is governed by `/etc/nsswitch.conf` (`hosts:` line).

Record types worth knowing: **A** (name → IPv4), **AAAA** (name → IPv6), **CNAME** (alias), **PTR** (reverse: IP → name), **SRV** (service discovery), **TXT** (metadata).

```
dig +short dashboard.factory.local # just the answer (scriptable)
dig dashboard.factory.local # full answer with TTL and sections
dig -x 192.168.1.50 # reverse lookup (PTR)
dig @8.8.8.8 example.com # query a specific resolver directly
nslookup dashboard.factory.local # interactive / legacy
getent hosts dashboard.factory.local # resolve the way the OS would (honors /etc/hosts)
cat /etc/resolv.conf # which DNS servers + search domains configured
cat /etc/hosts # static local overrides
```

Factory-relevant gotchas:

- “Can ping the IP but not the hostname” → **DNS** problem, not connectivity. Start at `/etc/resolv.conf` and `/etc/hosts`.
- “ping-by-name works but the app fails” → not DNS. Look at the application level.
- **Best practice for test stations:** pin critical instrument and dashboard names in `/etc/hosts` so the line never depends on a DNS server being up during a production run.

11.5 TCP vs UDP

11.5.1 Side-by-Side

Feature	TCP	UDP
Connection	Connection-oriented (3-way handshake)	Connectionless
Reliability	Guaranteed delivery + ordering + retransmit	Best-effort; no retransmit
Ordering	In-order byte stream	None (app must handle reordering)
Flow control	Yes (receiver window)	No
Congestion control	Yes (slow start, cwnd)	No
Message boundaries	None — byte stream	Preserved per datagram
Header size	20-60 bytes	8 bytes
Typical use	HTTP(S), SSH, instrument sockets, <code>equipment_rpc</code> , <code>ACT3</code>	DNS, DHCP, NTP, video, gPTP, <code>iperf3 -u</code>

The deepest practical difference for your code: **TCP is a byte stream**. A single `recv(4096)` gives you “up to 4096 bytes that have arrived so far,” which may be a part of one logical message, one whole message, or

pieces of several. A UDP `recvfrom()` gives you exactly one datagram per call. This is why the RPC/ACT3 `recv` loop (see the Sockets section) must frame messages itself.

11.5.2 The TCP 3-Way Handshake (Connection Setup)

```
Client Server
| ---- SYN (seq=x) ----->| "I want to talk; my starting seq is x"
| <--- SYN-ACK (seq=y,ack=x+1)| "OK; my seq is y; I received your x"
| ---- ACK (ack=y+1) ----->| "Got it; connection established"
|===== data flows =====|
```

The handshake exchanges initial sequence numbers and negotiates options (MSS, window scaling, SACK). Two failure signatures that instantly localize a problem:

- **“Connection refused”** — the SYN reached the host but nothing was listening on that port; the host sent a RST. Network is fine; the service is not running or is bound to the wrong port/interface.
- **“Connection timed out”** — the SYN got no answer at all. Host is down, wrong IP, or a firewall is silently dropping it. That single distinction (refused vs timeout) narrows a large fraction of test-station faults without touching a cable.

11.5.3 Teardown and the State Machine

```
Active closer Peer
| ---- FIN ----->|
| <--- ACK -----| (peer now in CLOSE_WAIT until it close(s))
| <--- FIN -----|
| ---- ACK ----->|
| (TIME_WAIT ~2*MSL, ~60s) | (CLOSED)
```

Two states that come up as operational symptoms:

- **TIME_WAIT** — on the side that closed first. It lingers ~60s to absorb delayed/ duplicate packets. Normal, but if a server is restarted rapidly, the listening port may be briefly unavailable → “Address already in use.” Fix: `SO_REUSEADDR` in the server socket setup.
- **CLOSE_WAIT** — the remote end closed but your code never called `close()`. These accumulate and are a **file-descriptor and socket leak** — a real bug, not a transient state. Many `CLOSE_WAIT` sockets piling up on a service is the signature of a missing `close()` or an unclosed context manager in a server loop.

```
ss -tan # all TCP sockets with state
ss -tan state time-wait # just TIME_WAIT
ss -tan state close-wait # CLOSE_WAIT -- socket leak hunting
ss -s # summary counts by state
```

11.5.4 Flow Control vs Congestion Control (Do Not Conflate Them)

- **Flow control** protects the *receiver*: the receiver advertises a window (how much it can buffer); the sender never has more unacknowledged data in flight than that. Prevents a fast sender from overrunning a slow receiver’s buffers.
- **Congestion control** protects the *network*: the sender maintains a congestion window (`cwnd`) that grows on success and shrinks on detected loss. Phases: **slow start** (exponential `cwnd` growth), **congestion avoidance** (linear growth), and on loss, fast retransmit/recovery or (on timeout) back to slow start. Linux defaults to the CUBIC algorithm; BBR is an alternative optimized for high-BDP links.

Why a test engineer cares: if `iperf3` throughput is below spec but stable, suspect the physical link or a receiver window too small; if it sawtooths (climbs, collapses, climbs), that is congestion control reacting to packet loss — chase the loss (errors, a flaky switch port, duplex mismatch) rather than the NIC itself.

11.5.5 When to Use Which Protocol

- **TCP** when correctness matters and a little latency is acceptable: controlling an instrument, submitting a test result, an HTTP API, file transfer. Almost all station-to-service traffic is TCP.
 - **UDP** when timeliness beats completeness or you need to *see* loss rather than have TCP hide it: telemetry, time sync (gPTP/PTP), streaming sensor data, Controller Area Network (CAN)-over-IP, and `iperf3 -u` to measure loss/jitter on a link.
-

11.6 Sockets and the Equipment RPC Pattern

11.6.1 The 5-Tuple

Every active connection is uniquely identified by a **5-tuple**: (`protocol`, `local_IP`, `local_port`, `remote_IP`, `remote_port`). That is why one server port (e.g., `:5000`) can simultaneously serve many clients — each connection differs in the remote IP/port pair.

11.6.2 TCP Server — the `InstrumentServer` / `equipment_rpc` Pattern

```
import socket, json

server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
# AF_INET = IPv4. SOCK_STREAM = TCP (reliable ordered byte stream).
# For UDP: SOCK_DGRAM.

server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
# SO_REUSEADDR: allow binding a port still in TIME_WAIT from a prior run.
# Without it, restarting the server within ~60s fails:
# "OSError: [Errno 98] Address already in use"

server.bind(("0.0.0.0", 5000))
# 0.0.0.0 = listen on ALL interfaces.
# 127.0.0.1 would accept only connections from localhost --
# a common mistake that makes the service invisible to remote clients.

server.listen(5) # backlog: connections queued before accept() runs

while True:
    conn, addr = server.accept() # blocks until a client connects
    with conn: # context manager guarantees close()
        data = recv_message(conn) # frame-aware receive (see below)
        request = json.loads(data)
        response = handle(request)
        conn.sendall(json.dumps(response).encode())
```

The `with conn:` block is not cosmetic — it is what prevents the `CLOSE_WAIT` leak described above. A server loop that forgets to close per-connection sockets leaks file descriptors until it hits `EMFILE` (“too many open files”) and stops accepting new ones.

11.6.3 TCP Client — the `act3_api_client` Pattern with the `recv` Fix

```
import socket, json

def rpc_call(host, port, cmd, timeout=5.0):
    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as sock:
        sock.settimeout(timeout)
```

```

# settimeout: if no data arrives within timeout seconds, raise socket.timeout.
# ALWAYS set a timeout on instrument sockets. Without it, a hung instrument
# blocks the program forever -- the #1 cause of a "frozen" test station.
sock.connect((host, port))
sock.sendall(json.dumps(cmd).encode())

# The recv() gotcha: TCP is a byte STREAM, not a message stream.
# One recv(4096) may return only PART of a large response.
# Loop and accumulate until the peer closes the connection.
chunks = []
while True:
    chunk = sock.recv(4096)
    if not chunk: # b'' means the peer closed the connection
        break
    chunks.append(chunk)
return json.loads(b"".join(chunks))

```

This pattern fixes a real bug: a ~4 KB instrument response arrived in two TCP segments; a single `recv(4096)` returned ~1500 bytes; `json.loads` raised `JSONDecodeError` and the call “randomly” failed under load. Accumulating until close (or until a length/delimiter is satisfied) is mandatory for any TCP `recv` loop.

11.6.4 Message Framing (Because the Stream Has No Boundaries)

“Read until the peer closes” works for one-shot request/response, but a long-lived connection that sends many messages needs explicit framing. Two standard approaches:

```

# (a) Newline-delimited JSON -- simple, human-debuggable:
def recv_line(sock):
    buf = b""
    while not buf.endswith(b"\n"):
        chunk = sock.recv(4096)
        if not chunk:
            break
        buf += chunk
    return buf

# (b) Length-prefixed -- robust for binary payloads:
# send 4-byte big-endian length, then the payload.
import struct

def send_message(sock, payload):
    sock.sendall(struct.pack(">I", len(payload)) + payload)

def recv_exact(sock, n):
    buf = b""
    while len(buf) < n:
        chunk = sock.recv(n - len(buf))
        if not chunk:
            raise ConnectionError("peer closed mid-message")
        buf += chunk
    return buf

def recv_message(sock):
    (length,) = struct.unpack(">I", recv_exact(sock, 4))
    return recv_exact(sock, length)

```

11.6.5 UDP — for Telemetry and Measurement

UDP preserves message boundaries — one `sendto` = one `recvfrom` — so no framing loop is needed, but delivery is not guaranteed. Your app must tolerate loss and reordering.

```
# UDP server
srv = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
srv.bind(("0.0.0.0", 9000))
while True:
    data, addr = srv.recvfrom(65535) # one complete datagram per call
    srv.sendto(handle(data), addr)

# UDP client
cli = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
cli.settimeout(2.0) # still set a timeout; recvfrom blocks
cli.sendto(b"PING", ("192.168.1.50", 9000))
try:
    reply, _ = cli.recvfrom(65535)
except socket.timeout:
    reply = None # datagram lost; handle or retry
```

11.6.6 The Linux Networking Stack (What Happens Between `sendall` and the Wire)

When `sendall()` is called, the kernel TCP stack:

1. Appends data to the send buffer (size controlled by `SO_SNDBUF` / `net.core.wmem_max`).
2. Packetizes into segments no larger than the MSS (maximum segment size, negotiated at handshake, typically MTU - 40 bytes for TCP/IP headers = 1460 bytes).
3. Applies any offloads configured on the NIC: **T**SO (TCP Segmentation Offload, the NIC segments rather than the CPU), **G**SO (Generic Segmentation Offload, SW equivalent), **G**RO (Generic Receive Offload, coalesces incoming segments).
4. Passes through the Netfilter/iptables/nftables hooks for firewall rules.
5. Hands the packet to the NIC driver ring buffer; the NIC Direct Memory Access (DMA)-fetches and transmits.

On receive, the path reverses: NIC DMA into ring buffer → driver interrupt/NAPI poll → IP reassembly → TCP reorder buffer → application `recv()`. Tunable receive buffers (`net.core.rmem_max`, `net.ipv4.tcp_rmem`) matter for high-throughput links. The `ethtool -k` command shows which offloads are active; disabling them can be useful when debugging whether a throughput problem is in hardware or software.

11.7 Linux Networking Tools — Command-First Reference

11.7.1 `ip` — Interfaces, Addresses, Routes, Neighbors (L1–L3)

```
ip link show # all interfaces: admin/oper state, MAC, MTU
ip link show eth0 # one interface
ip addr show # IP addresses per interface (alias: ip a)
ip addr show eth0
ip route show # routing table (alias: ip r)
ip neigh show # ARP / neighbor table (IP <-> MAC)
ip -s link show eth0 # interface counters: RX/TX packets, errors, dropped
ip -br addr # brief one-line-per-interface summary
ip -d link show eth0.100 # detailed: shows VLAN ID and protocol
```



```
# Temporary configuration (lost on reboot):
ip link set eth0 up
ip addr add 192.168.10.20/24 dev eth0
ip route add default via 192.168.10.1
ip neigh flush dev eth0 # clear stale ARP entries
```

ip link state semantics:

- UP — interface is administratively enabled.
- NO-CARRIER — interface is up but there is **no physical link signal** (cable unplugged or PHY problem). Pure L1 symptom.
- DOWN — interface is administratively disabled.
- LOWER_UP — physical layer is up (carrier present).

```
# Representative output of ip addr show eth0:
# 2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc mq state UP group default
# link/ether 00:1b:21:3c:4d:5e brd ff:ff:ff:ff:ff:ff
# inet 10.10.100.50/24 brd 10.10.100.255 scope global dynamic eth0
# valid_lft 86391sec preferred_lft 86391sec
```

The LOWER_UP flag confirms physical carrier; its absence while UP is set means NO-CARRIER — cable, SFP, or PHY issue.

11.7.2 ethtool — NIC and PHY Details (L1–L2, Critical for NIC Test)

```
ethtool eth0 # link, speed, duplex, autoneg, supported modes
ethtool -S eth0 # per-driver stats: rx_errors, tx_errors, rx_crc_errors, drops
ethtool -i eth0 # driver name + version, firmware version, bus-info (PCIe addr)
ethtool -k eth0 # offload features (TSO, GSO, GRO, checksum offload)
ethtool -t eth0 online # NIC built-in self-test PASS/FAIL (online = non-disruptive)
ethtool -m eth0 # SFP/optics DDM diagnostics: temperature, Tx/Rx power (dBm)
ethtool --cable-test eth0 # pass/fail per twisted pair
ethtool --cable-test-tdr eth0 # TDR: fault type + distance to fault (meters)
ethtool -p eth0 5 # blink port LED for 5 seconds (physically identify the port)
```

```
# Representative ethtool eth0 output for a 1GbE link:
# Settings for eth0:
# Supported ports: [ TP ]
# Supported link modes: 10baseT/Half 10baseT/Full 100baseT/Full 1000baseT/Full
# Speed: 1000Mb/s
# Duplex: Full
# Auto-negotiation: on
# Link detected: yes
```

Speed is not always a clean number. Parsing ethtool output for speed requires handling 2.5Gbit/s, 10000Mb/s, and Unknown!. A naive “strip all non-digits” parse turns 2.5G into 25 — a real bug that has caused a test to falsely pass a 2.5G NIC when 10G was expected. Always capture the value and unit separately and multiply G by 1000.

```
# ethtool -S eth0 output excerpt:
# rx_packets: 1234567
# rx_errors: 0
# rx_crc_errors: 0
# rx_dropped: 0
# tx_packets: 987654
# tx_errors: 0
# collisions: 0
```

All error/drop counters should be zero (or zero-delta over a run). Any non-zero value after a clean iperf3 run is a manufacturing reject trigger.

11.7.3 ss and netstat — Sockets and Listening Services (L4)

```
ss -tuln # TCP+UDP, listening only, numeric (what is serving?)
ss -tan # all TCP sockets with state
ss -tan state time-wait # TIME_WAIT sockets
ss -tan state close-wait # CLOSE_WAIT (socket leak hunting)
ss -tnp # show owning process/PID (needs root)
ss -s # summary counts by state
ss -tan '( dport = :8080 or sport = :8080 )' # filter by port
```

ss -tuln | grep 8080 answers “is my dashboard actually listening?” If it is not in the output, either the service is not running or it is bound to 127.0.0.1 only (loopback), so remote stations on the VLAN cannot reach it. Fix: bind to 0.0.0.0.

```
# Representative ss -tuln output:
# Netid State Recv-Q Send-Q Local Address:Port Peer Address:Port
# tcp LISTEN 0 5 0.0.0.0:5000 0.0.0.0:*
# tcp LISTEN 0 128 0.0.0.0:8080 0.0.0.0:*
# udp UNCONN 0 0 0.0.0.0:67 0.0.0.0:*
```

11.7.4 tcpdump — Packet Capture (All Layers)

```
tcpdump -i eth0 -c 100 -w capture.pcap # save 100 packets to file (open in Wireshark)
tcpdump -i eth0 -nn host 192.168.1.100 # traffic to/from one host, no name resolution
tcpdump -i eth0 -nn port 502 # Modbus/TCP traffic
tcpdump -i eth0 -A port 8080 # print ASCII payload (read HTTP inline)
tcpdump -i eth0 -nn 'tcp[tcpflags] & tcp-syn != 0' # only SYN packets (watch conn attempts)
tcpdump -i eth0 -nn arp # watch ARP: who-has / is-at
tcpdump -i eth0 vlan 100 # only VLAN 100 tagged frames
tcpdump -i eth0 -nn 'tcp port 5000 and (tcp-syn or tcp-fin or tcp-rst)' # connection lifecycle
```

-nn disables name and port resolution (faster, unambiguous output). Capture to .pcap when you need to hand it off or open Wireshark; print inline with -A for quick eyeballing of HTTP or ASCII protocols. Watching ARP (-nn arp) is how you confirm an L2 problem: if you never see an “is-at” reply, the peer is not answering on this segment (wrong VLAN, wrong subnet, or the device is off).

Useful Wireshark display filters (apply after loading a .pcap):

```
ip.addr == 192.168.1.100 -- all traffic to/from a host
tcp.port == 8080 -- filter by port
tcp.flags.syn == 1 && tcp.flags.ack == 0 -- SYN packets only (connection attempts)
tcp.analysis.retransmission -- retransmissions (network trouble)
http.response.code >= 400 -- HTTP errors
arp -- ARP traffic
```

11.7.5 ping — Reachability and RTT (L3, ICMP)

```
ping -c 4 192.168.1.1 # 4 echoes then stop
ping -c 4 -I eth0 192.168.1.50 # force out a specific interface
ping -c 100 -i 0.2 192.168.1.50 # 100 pings at 0.2s spacing (loss/jitter sample)
ping -M do -s 1472 192.168.1.50 # MTU probe: don't-fragment, 1472B payload
```

Interpreting results:

- **0% loss, steady RTT** → healthy L3 path.
- **Some loss** → marginal link or congestion.
- **100% loss, “Destination Host Unreachable”** → ARP or routing failure (L2/L3).
- **100% loss, silent timeout** → firewall dropping ICMP, or host is down.
- **MTU probe** (-M do -s 1472): if 1472B succeeds but 1473B fails with “frag needed,” path MTU is exactly 1500 (1472 payload + 28 IP/ICMP headers). If 8972B fails, jumbo is not working somewhere in the path.

11.7.6 mtr — Path Analysis, Hop by Hop (L3)

```
mtr 10.20.30.40 # live per-hop RTT and loss% (interactive)
mtr -n 10.20.30.40 # skip DNS resolution
mtr -rwc 100 10.20.30.40 # report mode: 100 cycles, printable summary
```

mtr is the best single tool for intermittent path problems: it shows which hop first starts dropping packets, at what rate. On a flat factory subnet you will see one or two hops; on a routed multi-VLAN network it pinpoints the offending router or link.

11.7.7 nc (netcat) — Fastest “Can I Reach This Port?” Test (L4)

```
nc -zv 192.168.1.100 8080 # -z scan (no data), -v verbose
nc -zv 192.168.1.100 5000-5010 # scan a port range
nc -l 5000 # listen on a port (throwaway test server)
echo '{"cmd":"ping"}' | nc 192.168.1.50 5000 # send a line, read the reply
```

Result interpretation:

- **“succeeded”** → TCP handshake completed; service is up and firewall allows it.
- **“refused”** → host reachable but nothing listening on that port (RST received).
- **“timed out”** → host unreachable or firewall dropping packets.

This single command disambiguates a large fraction of “can’t connect” tickets. “Refused” vs “timed out” is the same distinction as “connection refused” vs “connection timed out” in Section TCP vs UDP — it immediately localizes the problem to L4 (service) vs L3/firewall.

11.7.8 iperf3 — Throughput and Bandwidth (L4, the NIC-Test Workhorse)

```
# On the partner / golden unit:
iperf3 -s # start server (listens on port 5201)

# On the DUT:
iperf3 -c 192.168.1.100 -t 30 -P 4 # TCP, 30s, 4 parallel streams
# -P 4: parallel streams often needed to saturate a high-speed link
iperf3 -c 192.168.1.100 -u -b 950M -t 30 # UDP at 950 Mbps -> reports loss% + jitter
iperf3 -c 192.168.1.100 -R # reverse: server sends to DUT
iperf3 -c 192.168.1.100 --get-server-output # pull server-side numbers too

# Representative iperf3 output (TCP, 10 GbE):
# [ ID] Interval Transfer Bitrate
# [ 5] 0.00-30.00 sec 33.3 GBytes 9.54 Gbits/sec
# - - - - -
# [ ID] Interval Transfer Bitrate Retr
# [ 5] 0.00-30.01 sec 33.3 GBytes 9.54 Gbits/sec 0 sender
```

Expected results: 10 GbE → >9.5 Gbps with parallel streams; 1000BASE-T1 → ~940 Mbps. Retr (re-transmit count) should be 0 or near-zero on a good link. UDP mode reports **loss percentage and jitter**

directly — use it when you suspect a marginal link, because TCP hides loss by retransmitting (you’d see low throughput but not the loss count).

Throughput interpretation:

- **Near line rate, Retr=0, errors 0** → pass.
- **Below spec, stable, error counters rising** → physical link marginal (cable/connector/ PHY signal integrity). Confirm with `ethtool -S diff` and `--cable-test-tdr`.
- **Below spec, sawtooth** → packet loss triggering TCP congestion control. Chase the loss (flaky port, duplex mismatch, MTU mismatch) rather than tuning TCP.
- **Below spec, errors clean, no jitter** → single TCP stream not saturating the link (add `-P 4`), or CPU/IRQ bottleneck, or offloads disabled.

11.7.9 nmap — Discovery and Port Scanning (L3–L4)

```
nmap -sn 192.168.1.0/24 # ping sweep: which hosts are alive on the subnet
nmap -p 5000-5010 192.168.1.100 # which ports are open on this host
nmap -sV -p 8080 192.168.1.100 # probe service/version on a port
```

On a test floor, `nmap -sn` inventories what is actually on a VLAN — useful during station bring-up to confirm instruments have their expected static IPs. The port scan confirms an instrument’s control port is open before writing an RPC client.

11.7.10 Quick “Which Tool for Which Symptom” Map

Symptom	First tool(s)
Is the cable / link physically up?	<code>ip link show</code> , <code>ethtool eth0</code>
Correct IP / subnet / gateway?	<code>ip addr show</code> , <code>ip route</code>
Can I reach the host at all?	<code>ping -c4</code>
Where in the path does it drop?	<code>mtr -rwc 100</code> , <code>traceroute -n</code>
Is the service listening?	<code>ss -tuln</code>
Can the client open the port?	<code>nc -zv host port</code>
Is the app returning the right thing?	<code>curl -v</code>
Name does not resolve?	<code>dig +short</code> , <code>getent hosts</code> , <code>/etc/resolv.conf</code>
Throughput below spec?	<code>iperf3 -c partner -P 4</code> , then <code>ethtool -S</code>
What is actually on the wire?	<code>tcpdump</code> , Wireshark
Socket leak / wrong TCP state?	<code>ss -tan state close-wait</code>
ARP not resolving?	<code>ip neigh show</code> , <code>tcpdump -nn arp</code>
Which NIC is which physical port?	<code>ethtool -p eth0 5</code> (blink LED)
Cable integrity / fault location?	<code>ethtool --cable-test-tdr eth0</code>

11.8 PXE, TFTP, NFS, and HTTP — Test-Station Provisioning and DUT Netboot

In a manufacturing environment, DUTs often boot from the network rather than from on-board flash, and test stations themselves are re-imaged from a central provisioning server. Understanding the provisioning stack is essential because a DUT that will not netboot is a connectivity problem, not a firmware problem, until proven otherwise.

11.8.1 The PXE Boot Chain

PXE (Preboot eXecution Environment) allows a machine with a network-bootable NIC to get an OS or bootloader image from the network. The chain:

1. **DHCP** — the booting machine sends a DHCP request; the DHCP server responds with the standard IP/gateway/DNS assignment plus **option 66** (TFTP server address) and **option 67** (bootfile name, e.g., `pxelinux.0` or `grubx64.efi`).
2. **TFTP** (Trivial File Transfer Protocol, UDP port 69) — the client fetches the bootloader from the TFTP server. TFTP is intentionally simple and unauthenticated — it is transport for bootstrap only.
3. **Bootloader** — retrieves its configuration (e.g., `pxelinux.cfg/default`) via TFTP, then chains to the kernel (`vmlinuz`) and initial ramdisk (`initrd.img`), also via TFTP (or HTTP in modern iPXE setups).
4. **Root filesystem** — the booted kernel mounts its root over **NFS** (Network File System) for a live/diskless environment, or downloads a disk image via **HTTP/HTTPS** to flash.

```
DUT (booting) DHCP server TFTP / HTTP server
|-- DHCPDISCOVER -->| |
|<- DHCPOFFER -----| (IP + option 66/67) |
|-- DHCPREQUEST --> | |
|<- DHCPACK -----| |
|----- TFTP RRQ pxelinux.0 ----->|
|<----- TFTP DATA (bootloader) -----|
|----- TFTP RRQ pxelinux.cfg/default -->|
|<----- TFTP DATA (menu config) -----|
|----- TFTP/HTTP vmlinuz + initrd ----->|
| [kernel boots, mounts NFS or fetches img] |
```

11.8.2 Diagnosing a Failing Netboot

Symptoms and root causes:

Symptom	Layer	Likely cause
DUT says “PXE-E11: ARP timeout”	L2/L3	DHCP server not reachable (wrong VLAN, no DHCP relay, server down)
DUT gets DHCP IP but “TFTP timeout”	L3/UDP	TFTP server IP wrong (option 66), firewall blocking UDP 69, TFTP service down
TFTP starts but stalls / times out	L4/app	File not found on server, wrong filename in option 67, TFTP blocksize negotiation failure
Kernel loads but root mount fails	L3/NFS	NFS export not configured for the DUT's IP, NFS ports blocked, wrong mount path
Works from test VLAN, not DUT VLAN	VLAN/routing	PXE broadcast not relayed across VLANs (need DHCP relay / IP helper)

Debugging steps:

```
# On the provisioning server: is TFTP responding?
ss -tuln | grep 69 # TFTP listens on UDP 69

# From a test machine on the same VLAN as the DUT:
tftp <server-ip>
# tftp> get pxelinux.0
# Confirms TFTP is accessible and the file exists

# Watch DHCP and TFTP traffic while the DUT boots:
tcpdump -i eth0 -nn '(port 67 or port 68 or port 69)'
```

```
# Is NFS exported?
showmount -e <nfs-server-ip> # lists NFS exports

# Check NFS mounts are accessible:
mount -t nfs <server>:/exports/rootfs /mnt/test
```

DHCP relay (IP helper): PXE DHCPDISCOVER is a broadcast — it will not cross a VLAN boundary unless a **DHCP relay agent** (Cisco: `ip helper-address`, most switches have an equivalent) is configured on the DUT's VLAN to forward DHCP broadcasts to the DHCP server's unicast IP. A DUT on a new VLAN that gets 169.254.x.x while the provisioning server is on a different VLAN is almost always a missing helper-address, not a broken DUT.

Watching a netboot live from the provisioning server is the fastest way to see exactly where the chain breaks — the packet capture catches each step:

```
# Run on the provisioning server's interface (or any machine on the DUT's VLAN):
tcpdump -i eth0 -nn '(port 67 or port 68 or port 69 or port 4011)'
# port 4011 = ProxyDHCP (used by some UEFI/iPXE environments)

# What a healthy boot looks like in the capture:
# 0.000s DHCP Discover (DUT, broadcast)
# 0.003s DHCP Offer (server -> DUT: 10.20.30.50, next-server 10.20.1.5)
# 0.004s DHCP Request (DUT -> server)
# 0.005s DHCP Ack (server -> DUT: confirmed)
# 0.010s TFTP RRQ "pxelinux.0" (DUT -> 10.20.1.5 :69)
# 0.012s TFTP DATA block 1 (512 bytes, old mode -- or 1468 bytes, negotiated blksize)
# ...

# If you only see Discover and no Offer: DHCP server is unreachable on this VLAN.
# Check: is the DHCP server running? Is the DUT VLAN in its scope?
# Is the relay agent (ip helper-address) configured on the DUT VLAN's SVI?

# If Offer appears but no TFTP RRQ: DUT did not get option 66 (next-server).
# Check: DHCP server is sending siaddr / next-server field and option 67 (filename).
ss -tuln | grep ':69' # is tftpd listening?
ls /var/lib/tftpboot/ # is pxelinux.0 actually there?
```

TFTP blocksize and timeouts: Classic TFTP (RFC 1350) transfers in 512-byte blocks and requires an ACK after every block — at 50 ms RTT this limits throughput to ~10 KB/s. The TFTP options extension (RFC 2347/2348) allows negotiating larger blocksizes (up to 65464 bytes). Some DUT boot ROMs or TFTP servers silently reject the negotiation and fall back to 512 bytes; in this case a ~30 MB kernel takes minutes rather than seconds. If netboot is “working but slow,” check whether blocksize negotiation is occurring (it appears in the packet capture as TFTP OACK). iPXE avoids this entirely by using HTTP after the initial chainload.

11.8.3 HTTP-Based Provisioning

Modern provisioning tools (iPXE, Foreman, MAAS, custom scripts) fetch images via HTTP rather than TFTP for reliability (TCP vs UDP) and bandwidth. The pattern:

```
# DUT iPXE script fetches kernel/initrd over HTTP:
# kernel http://provision.factory.local/images/vmlinuz
# initrd http://provision.factory.local/images/initrd.gz
# boot

# Debug: can the DUT reach the HTTP server?
curl -v http://provision.factory.local/images/vmlinuz --output /dev/null
```

```
# Watch HTTP requests while DUT boots (from provisioning server):
tcpdump -i eth0 -A port 80 | grep -E 'GET|POST|HTTP'
```

11.9 Time Synchronization: NTP and PTP/IEEE 1588

Time sync is not a luxury feature — it is a functional requirement for any system that correlates events across nodes. On the Zoox compute platform, every sensor and compute module must share a common time base so that camera, LiDAR, and radar data can be correctly fused. An offset of even a few hundred microseconds between modules produces ghost objects or missed detections in the sensor fusion stack.

11.9.1 NTP (Network Time Protocol)

NTP (RFC 5905) synchronizes system clocks using UDP port 123. A **stratum 0** source is a hardware reference clock (GPS, atomic); **stratum 1** servers are directly connected to stratum 0; clients are stratum 2+. NTP achieves **1–10 ms accuracy** on a LAN under normal conditions.

```
timedatectl status # system time, NTP sync status, timezone
timedatectl timesync-status # detailed systemd-timesyncd info
chronyc tracking # chrony: offset, RMS jitter, stratum, sources
chronyc sources -v # all NTP sources and their health
ntpq -p # NTPd peers and their offsets
```

```
# chronyc tracking representative output:
# Reference ID : COA8 0101 (192.168.1.1)
# Stratum : 2
# Ref time (UTC) : Thu May 21 18:22:10 2026
# System time : 0.000004231 seconds slow of NTP time
# Last offset : +0.000003812 seconds
# RMS offset : 0.000004109 seconds
# Frequency : 15.232 ppm slow
# Residual freq : +0.003 ppm
# Skew : 0.031 ppm
# Root delay : 0.001234567 seconds
# Root dispersion : 0.000876543 seconds
# Update interval : 64.2 seconds
# Leap status : Normal
```

NTP is adequate for test station timestamps, log correlation, and result timestamping. It is **not adequate** for sensor fusion timestamps in an autonomous vehicle — that requires PTP.

11.9.2 PTP / IEEE 1588 — Precision Time Protocol

IEEE 1588 PTP synchronizes clocks over a local network to **sub-microsecond accuracy** by using hardware timestamps at the NIC level (or switch level) to cancel out software jitter. **IEEE 802.1AS (gPTP)** is a constrained profile of IEEE 1588 designed for bridged LANs and used in automotive networks as part of the TSN suite. The original 802.1AS-2011 was based on IEEE 1588-2008; the current 802.1AS-2020 revision references IEEE 1588-2019 and adds support for multiple domains and hot-standby grandmasters. In practice: when you see “gPTP” on the floor, expect 802.1AS-2020.

11.9.2.1 How PTP Works

```
Grandmaster (GM) Slave (boundary/ordinary clock)
|-- Sync (t1) ----->| GM records t1 (transmit HW timestamp)
|-- Follow_Up (t1) ----->| delivers t1 to slave (2-step mode)
```



```
|<-- Delay_Req (t3) -----| slave sends Delay_Req, records t3 (HW tx timestamp)
|-- Delay_Resp (t4) ----->| GM records t4 (HW rx timestamp), sends to slave
```

Slave computes:

```
mean path delay = ((t2 - t1) + (t4 - t3)) / 2
offset from master = (t2 - t1) - mean path delay
```

The slave then applies this offset to slew its clock (gradually adjusting the clock rate rather than stepping, to avoid timestamp discontinuities).

gPTP uses *peer delay*, and the rate-ratio scales the responder’s turnaround. The formula above is the end-to-end (Delay_Request) mechanism. 802.1AS instead uses the **peer-delay (P2P)** mechanism between adjacent ports: the requestor sends Pdelay_Req (t_1), the responder timestamps receive (t_2) and transmit (t_3), the requestor receives the response (t_4), and $\text{meanLinkDelay} = [(t_4 - t_1) - r(t_3 - t_2)]/2$, where r is the **neighborRateRatio**. The subtlety that’s easy to get backwards: r multiplies the *responder’s* turnaround ($t_3 - t_2$) — which is measured in the neighbor’s clock — to convert it into the local timebase; it is **not** applied to the round-trip ($t_4 - t_1$). At a non-unity ratio the two placements give different delays, so this is a real correctness trap (an automated audit even “corrected” the right formula to the wrong one). When in doubt the reference is linuxptp’s `tsproc: delay = ((t2-t3)*rr + (t4-t1))/2`. Hardware timestamping is critical: software timestamps have jitter of tens of microseconds; hardware timestamps (taken at the moment the SFD crosses the wire at the NIC’s MAC layer) have jitter of nanoseconds.

11.9.2.2 Why PTP Matters for Autonomous Vehicles

- **Camera / LiDAR / radar fusion** requires knowing precisely when each sensor frame was captured. A 200 μs timing error at 100 km/h corresponds to ~5.6 mm of vehicle motion — meaningful for sensor alignment.
- **Event correlation** across compute modules (e.g., matching a CAN event to a camera frame) requires all nodes to share the same time domain.
- **TSN traffic shaping** (IEEE 802.1Qbv, time-aware gating) relies on every switch knowing the same time to open and close transmission windows at the right instant.
- **Log forensics:** after an incident, correlating logs from multiple ECUs requires synchronized timestamps — NTP-level accuracy makes millisecond event ordering ambiguous; PTP-level accuracy does not.

11.9.2.3 linuxptp — Running PTP on Linux

`ptp4l` runs the PTP state machine (Best Master Clock Algorithm selects the grandmaster, then ordinary or boundary clock slaves synchronize to it). `phc2sys` synchronizes the system clock (`CLOCK_REALTIME`) to the NIC’s hardware clock (PHC, PTP Hardware Clock).

```
# Start ptp4l as a slave on eth0 (using hardware timestamps):
ptp4l -i eth0 -m -f /etc/ptp4l.conf
# -i eth0: interface -m: print to stdout -f: config file

# Sync the system clock to the PHC (wait for ptp4l to lock, then auto-fetch TAI-UTC offset):
phc2sys -s eth0 -c CLOCK_REALTIME -w -m
# -w: wait for ptp4l to be synchronized before starting, then retrieve the TAI-to-UTC
# offset from ptp4l automatically. Omitting -w and using -O 0 manually forces a
# zero offset, which is wrong: PTP operates in TAI, the system clock in UTC, and
# the current TAI-UTC difference is 37 seconds. Always use -w in production.

# Check PHC capability on a NIC:
ethtool -T eth0
# Should show: hardware-transmit, hardware-receive, hardware-raw-clock
```



```
# Monitor offset from master (ptp4l log lines):
# ptp4l[123.456]: master offset -1234 s2 freq +5678 path delay 12345
# master offset: nanoseconds of clock difference (converges toward 0)
# freq: current clock rate correction in ppb
# path delay: one-way network delay in nanoseconds
```

11.9.2.4 PTP Health Metrics

Metric	Healthy	Warning
master offset	Converging to 0, then < +-100 ns steady	> +-1000 ns or diverging
path delay	Stable (constant switch latency)	Large variation = switch overloaded or asymmetric path
phc2sys offset	< +-1 us	> +-10 us = system clock / PHC drift
Clock state	s2 (synchronized)	s1 (uncertain) or FAULTY

11.9.2.5 PTP Bring-Up on the Floor — Practical Sequence

When a new compute board lands on the test bench, this is the sequence to verify its PTP subsystem before it ever runs sensor fusion:

```
# Step 1: confirm the NIC has a hardware PHC (PTP Hardware Clock)
ethtool -T eth0
# Must show: hardware-transmit, hardware-receive, hardware-raw-clock
# If only software-transmit/receive appear, the driver lacks HW timestamp support
# -- you will only get microsecond-class accuracy, not sub-microsecond.

# Step 2: start ptp4l as slave, hardware timestamping, verbose so you can watch it
ptp4l -i eth0 -m -s --summary_interval -1 -f /etc/ptp4l.conf
# -s: slave-only mode (do not try to become grandmaster)
# --summary_interval -1: log every second for rapid debugging

# Step 3: watch the log and check for these stages:
# ptp4l[0.000]: selected /dev/ptp0 as PTP clock
# ptp4l[2.134]: port 1: LISTENING (no grandmaster yet seen)
# ptp4l[2.960]: port 1: UNCALIBRATED (grandmaster found, measuring delay)
# ptp4l[4.123]: port 1: SLAVE (s2: synchronized)
# ptp4l[5.456]: master offset -234 s2 freq +1234 path delay 4567
# ^^^ must be s2; offset converges toward 0 within 30-60 seconds

# Step 4: sync the system clock to the PHC after ptp4l locks
phc2sys -s eth0 -c CLOCK_REALTIME -w -m
# -w waits until ptp4l is in s2 before starting, and reads the TAI-UTC
# offset automatically. The phc2sys log should show:
# phc2sys[10.234]: CLOCK_REALTIME phc offset -456 s2 freq +789

# Step 5: automated pass/fail in a test script
ptp4l_log=$(journalctl -u ptp4l --since "2 minutes ago" --no-pager)
# Pass criterion: s2 state reached, master offset < 1000 ns sustained for 30 s
```

Common floor failures and their signatures:

Symptom in log	Cause	Fix
Stays in LISTENING forever	No grandmaster visible	VLAN isolation, wrong domain, PTP multicast blocked
Oscillates UNCALIBRATED -> SLAVE	Path delay unstable	Switch not transparent clock / boundary clock; excessive queue variation
<code>offset > 10000 ns</code> , not converging	SW timestamps only (no HW)	<code>ethtool -T</code> – driver does not support hardware-transmit/receive
<code>phc2sys</code> reports wrong time of day	TAI/UTC offset wrong	Use <code>-w</code> not <code>-0 0</code> ; current TAI-UTC = 37 s
<code>ptp4l</code> crashes or errors on <code>-T</code>	Config file missing or wrong interface	Verify <code>-i</code> matches actual interface name with <code>ip link show</code>

If `ptp4l` never reaches `s2` (synchronized state): check that the NIC supports hardware timestamping (`ethtool -T eth0`), that the grandmaster is reachable and its priority/ domain matches, that the switch in the path supports transparent clock or boundary clock operation (a switch that does not handle PTP will introduce variable queuing delay that prevents convergence), and that no firewall is blocking PTP multicast packets. PTP uses:

- **UDP ports 319** (event messages: Sync, Delay_Req, Pdelay_Req, Pdelay_Resp) and **320** (general messages: Announce, Follow_Up, Delay_Resp, management)
- **IPv4 multicast** 224.0.0.129 (end-to-end delay) and 224.0.0.107 (peer delay)
- **L2 multicast** 01:1b:19:00:00:00 (standard PTP multicast) and 01:80:c2:00:00:0e (peer-delay / link-local, not forwarded by bridges unless configured)

gPTP (802.1AS) uses exclusively the **peer-delay** mechanism — each port independently measures the delay to its directly attached neighbor, rather than measuring the full end-to-end path. This distinction matters for switch configuration: a switch in the path must run a boundary clock or transparent clock to prevent the variable queuing delay from corrupting the peer-delay measurement.

```
# Confirm PTP multicast traffic is flowing (on the DUT or any node on the link):
tcpdump -i eth0 -nn '(udp port 319 or udp port 320) or ether proto 0x88f7'
# 0x88f7 is the PTP Ethertype for L2 PTP frames (used when not running over UDP/IP)

# Show which clock state the node is in and the current offset:
journalctl -u ptp4l -f
# or read the log directly if running in foreground
```

11.9.2.6 PTP vs NTP Summary

	NTP	PTP / IEEE 1588
Typical accuracy (LAN)	1-10 ms	100 ns - 1 us
Hardware timestamping needed	No	Yes (for sub-us)
Used for	Log timestamps, general sync	Sensor fusion, TSN, automotive
Linux tools	<code>chrony</code> , <code>ntpd</code> , <code>systemd-timesyncd</code>	<code>ptp4l</code> , <code>phc2sys</code>
Profile for automotive	—	IEEE 802.1AS (gPTP)

11.10 NIC Manufacturing Test

This is the heart of the role: take a board off the line, prove its NIC(s) are good, and **reject the marginal ones**. Gate on measurable thresholds, not on “it seems to work.” A link that comes up but drops frames, negotiates the wrong speed, or accumulates CRC errors under load is a field failure waiting to happen.

11.10.1 The Standard NIC Test Sequence

```
# 1. Enumerate -- is the NIC present on the PCIe bus?
lspci | grep -i ethernet
lspci -vvv -s <bus:dev.fn> | grep -iE 'LnkCap|LnkSta'
# LnkSta should match LnkCap: Speed and Width (e.g., 8GT/s x4)

# 2. Link / speed / duplex -- did it negotiate the expected rate?
ethtool eth0
# Want: "Link detected: yes", correct Speed (e.g., 10000), "Duplex: Full"

# 3. Driver / firmware -- correct, expected versions
ethtool -i eth0
# Captures driver name, version, firmware-version, bus-info

# 4. Self-test -- NIC's built-in diagnostics
ethtool -t eth0 online
# online = non-disruptive; result must be PASS

# 5. Throughput to a partner / golden unit
iperf3 -c <partner-ip> -t 30 -P 4
# 10 GbE -> >9.5 Gbps; 1000BASE-T1 -> ~940 Mbps

# 6. Error counters -- must be ~0 after the throughput run
ethtool -S eth0 | grep -iE 'err|drop|crc|collision'

# 7. Cable integrity (copper / automotive)
ethtool --cable-test-tdr eth0
# OK, or reports: fault type (open/short/impedance mismatch) + distance in meters
```

11.10.2 What “Good” Looks Like and the Gates

The test gates, expressed as boolean checks in code:

```
checks = {
    "link_up": link_detected, # ethtool "Link detected: yes"
    "speed_ok": negotiated_mbps >= expected_mbps, # e.g., >= 1000 for 1GbE
    "duplex_full": duplex == "Full",
    "self_test": self_test_result == "PASS",
    "low_errors": rx_errors + tx_errors < 10, # combined threshold after iperf3
    "throughput_ok": measured_mbps >= min_mbps, # e.g., >= 900 for a 1G link
    # for automotive / copper:
    "cable_ok": cable_test_status != "fault",
}
# Unit passes only if ALL checks pass.
```

Design points that prevent false results:

- **Speed parsing:** 2.5G must be parsed as 2500, not 25 (strip non-digits gives the wrong answer). Capture value and unit separately; multiply G-suffix by 1000.
- **Master/slave role:** captured from `ethtool master-slave cfg/status`: line and surfaced in the test summary, because role misconfiguration is the top “no link” cause on automotive PHYs.
- **Cable test is best-effort:** PHY/driver Time-Domain Reflectometry (TDR) support varies. An unsupported result is reported as `skipped`, not `fault` — do not false-fail a good board.
- **Delta counters:** snapshot `ethtool -S` before and after `iperf3`, diff the values. A NIC that links and passes self-test but accumulates CRC errors *under load* is the marginal unit you exist to catch. A minimal Python implementation:

```

import subprocess, re

def ethtool_stats(iface):
    out = subprocess.check_output(["ethtool", "-S", iface], text=True)
    stats = {}
    for line in out.splitlines():
        m = re.match(r"\s+(\S+):\s+(\d+)", line)
        if m:
            stats[m.group(1)] = int(m.group(2))
    return stats

def delta(before, after):
    return {k: after[k] - before.get(k, 0) for k in after}

ERROR_KEYS = re.compile(r"err|crc|drop|collision|miss", re.I)

def check_nic(iface, run_iperf3):
    before = ethtool_stats(iface)
    run_iperf3() # call your iperf3 fixture here
    after = ethtool_stats(iface)
    d = delta(before, after)
    bad = {k: v for k, v in d.items() if ERROR_KEYS.search(k) and v > 0}
    return bad # empty dict = pass; any non-zero key = reject

```

11.10.3 Error Counters: What Each Means

Counter	Meaning	Non-zero implies
rx_errors / tx_errors	Total receive/transmit errors	Physical or driver problem
rx_crc_errors	Frames failing CRC check	Signal integrity: cable, connector, PHY
rx_dropped / tx_dropped	Frames dropped (no buffer or no link)	Ring buffer overrun, link flap, CPU can't keep up
collisions / late_collision	(Half-duplex) collisions	Duplex mismatch (force full-duplex)
rx_missed_errors	NIC ring overflow	Host not reading fast enough: CPU/IRQ, driver

11.10.4 Loopback Options When No Partner Is Available

- **Internal/PHY loopback** via the driver (`ethtool -t offline` on some drivers; some drivers expose a loopback mode) — exercises the MAC/PHY datapath without a cable.
- **Physical loopback plug** on copper — connects Tx pairs back to Rx for a self-link.
- **Two NICs on the same DUT cabled together** — run `iperf3` server on one and client on the other for a self-contained throughput test.

A partner/golden unit on a known-good link is still the gold standard, because it exercises the actual cable and the real far-end PHY negotiation.

11.11 Automotive Ethernet for Test Engineers

The Zoox compute platform connects modules with automotive Ethernet. This section focuses on the test and diagnostic angle. For full 100/1000BASE-T1 architecture and in-vehicle networking design, see the Automotive chapter.

11.11.1 Physical Layer — Single Pair, Special Line Codes

Standard	Rate	Medium / line code	IEEE
100BASE-T1	100 Mb/s	single pair, PAM3	802.3bw
1000BASE-T1	1 Gb/s	single pair, PAM3	802.3bp
2.5/5/10GBASE-T1	2.5-10 Gb/s	single pair, PAM4	802.3ch
10BASE-T1S	10 Mb/s	short multidrop, half-duplex	802.3cg

Key differences from consumer BASE-T Ethernet:

- **One pair, full-duplex with echo cancellation.** 1000BASE-T uses 4 pairs (8 wires); ***BASE-T1** runs full-duplex over a **single twisted pair**. Both transmit and receive share the wire; the PHY cancels its own echo. Fewer wires means lighter, cheaper vehicle harness.
- **Line coding.** 1000BASE-T1 uses **PAM3** (three voltage levels: -1, 0, +1); 802.3ch multi-Gig uses **Pulse Amplitude Modulation 4-level (PAM4)** (four levels). Eye diagram and Signal Integrity (SI) analysis concepts from SerDes carry over.
- The trailing 1 in ***BASE-T1** is the tell: single pair, automotive.

11.11.2 Master/Slave Clock Roles — the Number-One “No Link” Cause

Each automotive PHY link is configured as **master** or **slave** (a PHY-level timing role for clock recovery). **The two ends must be opposite** — one master, one slave.

A “no link” between correctly cabled, otherwise-healthy automotive PHYs is very often a **master/master or slave/slave misconfiguration**, not a cable fault.

The first thing to check on a dead ***BASE-T1** link is the role on each end, before suspecting the harness. This is different from consumer Ethernet where roles are auto-negotiated and invisible.

```
ethtool eth0
# Look for:
# master-slave cfg: master (or slave, or preferred master/slave)
# master-slave status: master (the actual negotiated result)
```

If both sides show **master** or both show **slave**, the link will not come up regardless of cable quality.

11.11.3 Real PHYs — Recognize the Part Numbers

Vendor	Part	Capability
Marvell	88Q2112	100/1000BASE-T1; integrates MDI termination
Marvell	88Q222x / 88Q42xx	newer multi-Gig automotive PHY/switch family
TI	DP83TG721-Q1	1000BASE-T1 with TSN/AVB + TC10 sleep/wake
Broadcom	BCM8989x / BCM8957x	automotive multi-Gig PHY/switch

11.11.4 MDIO, ethtool, and the Cable Test TDR

- **MDIO** (Management Data I/O, clause 22/45) is the sideband management bus for reading and writing PHY registers: link status, master/slave role, error counters. Linux exposes PHYs through the netdev + **phylib** layer; most register access is via ethtool.
- **Cable test (TDR)** is the highest-value automotive manufacturing test, because it pinpoints the bad segment of a harness:

```
ethtool --cable-test eth0 # pass/fail per pair
ethtool --cable-test-tdr eth0 # TDR: fault type + distance to fault (meters)
```

TDR sends a pulse and times the reflection. Result codes:

Code	Meaning
OK	Cable is good
Open	Open circuit; distance shown is meters to the break
Short	Short circuit between pairs
Impedance mismatch	Reflection from discontinuity: bad connector or crimp
Noise	Test could not complete

The **distance-to-fault** is what makes TDR powerful on a vehicle harness: it tells you *which connector or segment* is bad, rather than “the cable is broken somewhere.” On a manufacturing line this eliminates guesswork in rework.

11.11.5 The Combined Diagnostic Chain

Good link + low iperf3 throughput + rising CRC errors → marginal signal integrity
(cable / connector / PHY). Confirm with `ethtool --cable-test-tdr` to localize it.

The link is up (roles and basic negotiation are fine); throughput is below spec; and `ethtool -S` shows CRC errors climbing under load. That combination points squarely at the analog/physical path. TDR gives you the distance to the fault so a tech fixes the right connector.

11.11.6 TC10 — Coordinated Sleep and Wake

OPEN Alliance TC10 standardizes coordinated sleep and wake-up over automotive Ethernet: **local wake**, **remote wake**, **wake-forwarding**, and **sleep negotiation**, so an entire in-vehicle network can power down and a single event can wake it back up. As a test step: verify the PHY enters sleep, wakes on the defined event, and forwards wake correctly to downstream nodes. The TI DP83TG721-Q1 supports TC10.

11.11.7 TSN — Time-Sensitive Networking

TSN is the suite of IEEE 802.1 standards that adds:

- **IEEE 802.1Qbv** — time-aware shaping: scheduled transmission windows so safety-critical frames are never delayed by bulk traffic.
- **IEEE 802.1Qbu / 802.3br** — frame preemption: high-priority frames can interrupt a large low-priority frame mid-transmission.
- **IEEE 802.1CB** — frame replication and elimination for redundancy.
- **IEEE 802.1AS (gPTP)** — the time synchronization backbone (see the PTP section above).

The test engineer’s TSN checklist: (1) PTP converges and holds offset < 100 ns; (2) Qbv gates open/close at the right times and high-priority traffic meets its latency budget; (3) the switch’s credit-based shaper does not starve low-priority streams.

11.12 Layer-by-Layer Troubleshooting Methodology

The discipline: **start at Layer 1 and climb**. Confirm each layer is sound before blaming the one above it. This turns “I can’t reach the instrument” into a deterministic procedure. You can also divide and conquer (ping first to cover L1–L3 at once), but when stuck, the bottom-up sweep never lies.

11.12.1 The Ladder

L1 Physical — is there a link?

```
ip link show eth0 # UP? NO-CARRIER? LOWER_UP flag set?
ethtool eth0 # "Link detected: yes"? Correct speed?
ethtool -p eth0 5 # blink LED to confirm which physical port
```

Cable seated? Link LED on? Right port? For automotive single-pair: check master/slave roles before touching the harness.

L2 Data Link — is the framing clean?

```
ethtool eth0 # Duplex: Full?
ethtool -S eth0 | grep -iE 'err|drop|crc|collision' # all should be ~0 or zero-delta
ip neigh show # ARP resolved? REACHABLE vs FAILED
```

Rising CRC/align errors = physical-layer signal integrity issue (bad cable/connector/ PHY). A **duplex mismatch** shows as late collisions and poor throughput on an apparently good link.

L3 Network — addressing and routing correct?

```
ip addr show eth0 # correct IP and mask? Not a stray 169.254.x.x?
ip route # default gateway present? Route to target subnet?
ping -c3 <gateway> # gateway reachable?
ping -c3 <target> # target reachable?
```

Same-subnet check: if the target is on a different subnet and there is no matching route, packets are dropped with “No route to host.”

L4 Transport — is the service up and reachable?

```
ss -tuln | grep 8080 # on the server: is anything LISTENING?
nc -zv <host> 8080 # from the client: does the TCP handshake complete?
```

“Refused” → service not running, or wrong port, or bound to 127.0.0.1. “Timed out” → firewall or L3 problem (go back down a rung).

Firewall — is traffic being blocked?

```
sudo iptables -L -n -v # legacy firewall rules with hit counters
sudo nft list ruleset # nftables (modern)
```

A silent drop rule looks identical to a dead link from the client’s side — nc “timed out” with the host clearly up via ping is the tell. Check firewall on both ends and any host in the path.

L7 Application — is the protocol behaving?

```
curl -v http://<host>:8080/api/stations # correct status code and body?
```

422/400 → client request shape is wrong (Pydantic validation failure: wrong fields or types in the JSON body). 500 → server-side exception (read the server logs). Hostname-only failure while IP works → DNS.

11.12.2 Full Diagnostic Sequence — Paste and Run

```
ip link show eth0 # L1: interface up? carrier?
ethtool eth0 # L1/L2: link detected, speed, duplex
ethtool -S eth0 | grep -iE 'err|drop' # L2: error counters (should be 0)
ip addr show eth0 # L3: correct IP/mask?
ip route # L3: default gateway / route to target?
ping -c 3 192.168.1.1 # L3: gateway reachable?
ping -c 3 192.168.1.100 # L3: target reachable?
```

```
nc -zv 192.168.1.100 8080 # L4: port open?
curl -v http://192.168.1.100:8080/ # L7: app responding correctly?
```

If step N is the first to fail, the fault is at that layer. Worked example: steps 1–7 pass, step 8 returns “refused” → network is perfect; the dashboard process is not listening (crashed, or bound to 127.0.0.1). No need to touch a cable.

Automating the L3/L4 check from a test script — useful for a pre-run connectivity gate before the station starts a test sequence:

```
import socket, subprocess, sys

def check_l3(host, count=3, timeout=1.0):
    """Returns True if host responds to ICMP echo (ping)."""
    rc = subprocess.call(
        ["ping", "-c", str(count), "-W", str(int(timeout)), host],
        stdout=subprocess.DEVNULL, stderr=subprocess.DEVNULL,
    )
    return rc == 0

def check_l4(host, port, timeout=2.0):
    """Returns ('open'/'refused'/'timeout') for the TCP port."""
    try:
        with socket.create_connection((host, port), timeout=timeout):
            return "open"
    except ConnectionRefusedError:
        return "refused"
    except (socket.timeout, OSError):
        return "timeout"

# Pre-run gate: fail fast with a meaningful message
for host, port, label in [
    ("192.168.10.5", 5000, "power supply RPC"),
    ("10.10.100.50", 8080, "results dashboard"),
]:
    if not check_l3(host):
        sys.exit(f"PRE-RUN FAIL: {label} at {host} not reachable at L3 (ping failed)")
    state = check_l4(host, port)
    if state != "open":
        sys.exit(f"PRE-RUN FAIL: {label}:{port} TCP {state} -- service may be down")
```

11.12.3 Common Failure Signatures

Symptom	Root cause	Diagnostic step
NO-CARRIER on ip link	Cable unplugged, PHY dead, SFP missing	Seat cable, check LED, <code>ethtool eth0</code>
169.254.x.x address	DHCP failed	Wrong VLAN, DHCP server down, or link problem
ping works, name fails	DNS broken	<code>/etc/resolv.conf</code> , <code>/etc/hosts</code> , <code>dig @server name</code>
nc refused, ping works	Service not listening	<code>ss -tuln</code> , check process is running and bind address
nc timed out, ping works	Firewall dropping packets	<code>iptables -L -n -v</code> on both hosts
iperf3 sawtooths	Packet loss → congestion control	<code>ethtool -S</code> CRC errors, duplex check, MTU check
iperf3 stable but low	PHY signal integrity marginal	<code>ethtool --cable-test-tdr</code> , check connector/cable

Symptom	Root cause	Diagnostic step
Many <code>CLOSE_WAIT</code> sockets	Missing <code>close()</code> in server loop	Code audit for unclosed context managers
“Address already in use” on restart	<code>TIME_WAIT</code> from prior run	<code>SO_REUSEADDR</code> before <code>bind()</code>
Duplex mismatch	Autoneg failure, forced mismatch	<code>ethtool -S</code> late collisions, force both to full
Automotive PHY no link	Master/slave misconfiguration	<code>ethtool eth0</code> master-slave status, reconfigure
Big transfers fail, small work	MTU/jumbo mismatch	<code>ping -M do -s 8972 host</code> , check switch MTU config
<code>ptp4l</code> stuck in <code>s1</code> (uncertain)	No HW timestamps, switch no transparent clock, firewall blocking PTP multicast	<code>ethtool -T eth0</code> , switch config, <code>tcpdump</code> PTP ports 319/320

11.13 HTTP, REST, and the FastAPI Dashboard

11.13.1 HTTP Request/Response Anatomy

HTTP is a text-based request/response protocol over TCP (port 80; HTTPS = HTTP over TLS, port 443). A request is `METHOD path HTTP/1.1` followed by headers and an optional body; a response is `HTTP/1.1 status reason` followed by headers and a body.

11.13.2 Methods and Idempotency

Method	Meaning	Idempotent	Dashboard example
GET	Retrieve a resource	Yes	<code>GET /api/stations</code> – list stations
POST	Create / submit / trigger	No	<code>POST /api/heartbeat</code> – station heartbeat
PUT	Replace a resource at a known URI	Yes	<code>PUT /api/stations/s1</code> – overwrite config
PATCH	Partial update	No	<code>PATCH /api/stations/s1</code> – change one field
DELETE	Remove a resource	Yes	<code>DELETE /api/runs/42</code>

Idempotent = doing it twice has the same effect as once — matters for safe retries. A heartbeat POST that is retried on network error must not corrupt state by recording duplicate events.

11.13.3 Status Codes

- **2xx success** — 200 OK, 201 Created (after POST), 204 No Content.
- **4xx client error** — 400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, 422 Unprocessable Entity (FastAPI/Pydantic validation failure — wrong field types or missing required fields), 429 Too Many Requests.
- **5xx server error** — 500 Internal Server Error (unhandled exception), 502 Bad Gateway, 503 Service Unavailable, 504 Gateway Timeout.

Rule: **4xx = the client sent something wrong; 5xx = the server blew up.** A 422 from FastAPI almost always means the JSON body does not match the Pydantic model — fix the request, not the server.

11.13.4 curl as an HTTP Multimeter

```
# GET
curl http://dashboard:8080/api/stations

# POST with JSON body
curl -X POST http://dashboard:8080/api/heartbeat \
  -H "Content-Type: application/json" \
  -d '{"hostname":"station1","fat_state":"RUNNING"}'

# Verbose: see both request and response headers
curl -v http://dashboard:8080/api/stations

# Print only the HTTP status code (scripting-friendly)
curl -s -o /dev/null -w "%{http_code}\n" http://dashboard:8080/

# Timing breakdown: separate DNS, connect, and total time
curl -w "dns:%{time_namelookup}s connect:%{time_connect}s total:%{time_total}s\n" \
  -s -o /dev/null http://dashboard:8080/
```

The timing breakdown (`-w`) is useful for “the dashboard is slow” reports: it separates DNS resolution time, TCP connect time, and total time, pinpointing whether to blame name resolution, the network, or the server-side handler.

11.14 Pointer to the Automotive Chapter

100/1000BASE-T1 physical layer design, vehicle Ethernet topology, the switch fabric architecture, domain controller interconnects, and the full in-vehicle networking stack are covered in the Automotive chapter. This chapter’s NIC test sequence, master/slave diagnostics, TDR cable test, and PTP/TSN sections are the manufacturing-floor view of that same technology.

Chapter 12

Power, Bring-up, and Functional Safety

The most common pattern in compute-board debugging is this: what looks like a PCIe enumeration failure, a SerDes lock problem, or an Error-Correcting Code (ECC) storm turns out to be a power problem in disguise. A 0.8V core rail that sequenced 2 ms late, a 1.1V Double Data Rate (DDR) rail drooping 6% under load, a switching regulator coupling 40 mVpp into an analog Phase-Locked Loop (PLL) supply — all of these manifest first as link errors and GPU faults. They do not look like power problems because the digital subsystem is the part that screams. This chapter builds the instincts to reach for the right tool — scope, DMM, IPMI, `dmesg`, the sequencing diagram — and name the physical cause rather than chasing the symptom.

The functional safety framing at the end explains *why* the test coverage has to be as thorough as it is, and why the records you keep are themselves part of the safety argument. Neither section is purely theoretical: on a real Zoox board these disciplines interact daily.

12.1 Board Power Architecture

12.1.1 The Power Distribution Network

A compute board is not powered from one voltage; it is powered from a *tree* of domains that each feed a different set of loads with different tolerance requirements. Understanding the topology is the starting point for both hardware bring-up and manufacturing test.

Typical domain stack on an automotive compute board:

Domain	Nominal	Typical Tolerance	Load Character
Primary input	12V (or 48V in newer designs)	+/-5%	From vehicle power bus via connector
5V standby	5.0V	+/-3%	Always-on for BMC, RTC, wake logic
3.3V peripheral	3.3V	+/-5%	PHYs, SerDes, flash, GPIOs
1.8V I/O	1.8V	+/-3%	SoC I/O, LPDDR support, PCIe side-band
1.1V DDR	1.1V	+/-3%	DDR5 VDD; critical for array stability

Domain	Nominal	Typical Tolerance	Load Character
0.8-1.0V core	varies by chip	+/-3%	SoC/GPU/FPGA compute core; highest current
PLL / analog	1.0-1.8V (clean)	+/-2%	SerDes, PLL; ripple spec tighter than logic
PoC (Power over Coax)	6-12V	+/-10%	GMSL camera supply through the coax link

The primary input arrives from the vehicle power bus (12V in traditional vehicle architectures, 48V in some newer designs) through a connector, possibly an eFuse or hot-swap controller, and into the input bulk capacitors. From there a set of switching regulators (buck converters, multi-phase VRMs) step down to the intermediate and load rails. Each rail then feeds one or more domains, often gated by a load switch so the sequencer can power them on and off independently.

The tighter-tolerance rails — core, DDR, PLL — are almost always the last to come up and the first to expose a process problem. They are also the rails where a 4-wire DMM measurement and an AC-coupled scope are mandatory tools, not optional.

12.1.2 Rail Tolerances and Guard-Banding in Test

Every rail has a tolerance stated in the component datasheet and system requirements: commonly +/-3% for core/DDR and +/-5% for less sensitive domains. Those are the *specification limits*. Manufacturing test typically uses **guard-banded test limits** that sit inside the spec window to account for DMM uncertainty, fixture contact resistance, and process drift:

```
Spec limit: [V_nom * (1 - 0.03), V_nom * (1 + 0.03)]
Test limit: [V_nom * (1 - 0.025), V_nom * (1 + 0.025)] -- 2.5% guard vs 3% spec
```

The guard band is not arbitrary; it follows from the measurement system analysis (MSA) that characterizes the meter, the fixture, and the combined measurement uncertainty. Any limit change must go through that analysis — loosening a rail limit to recover yield without understanding the measurement uncertainty is how you ship boards with marginal power delivery.

12.1.3 PMICs, VRMs, and Point-of-Load Regulators

PMIC (Power Management IC): A single chip that integrates multiple regulated outputs, the sequencing logic, and fault monitoring. Common in lower-power SoCs (automotive-grade Maxim/Analog Devices, TI, Renesas PMICs). The PMIC receives an enable from a supervisor or CPLD and raises its outputs in a programmed order per internal sequencing registers. In manufacturing test, the PMIC's fault status registers are readable over I2C or Power Management Bus (PMBus) — they tell you *which* rail tripped and *why* (OV/UV/OC event) rather than just that PWRGOOD went low.

VRM / multi-phase buck (Voltage Regulator Module): For high-current rails (GPU/System-on-Chip (SoC) core can demand 50-200A), the board uses a multi-phase synchronous buck VRM. Each phase delivers a fraction of the total current and interleaves switching at an offset phase angle, which multiplies the effective ripple frequency and reduces per-phase inductor size. The output LC filter plus the Power Delivery Network (PDN) (power delivery network: planes, vias, package inductance, die capacitance) determines load-transient response and ripple at the load.

Point-of-load regulators (POLs): Small LDOs or single-phase bucks close to a sensitive load (SerDes PLL, Dynamic Random-Access Memory (DRAM) VTT, Field-Programmable Gate Array (FPGA) analog supply). They often have tighter ripple specs than the main rails and are the ones where a bad probe tip with a long ground clip will fool you into thinking there is more noise than there actually is.

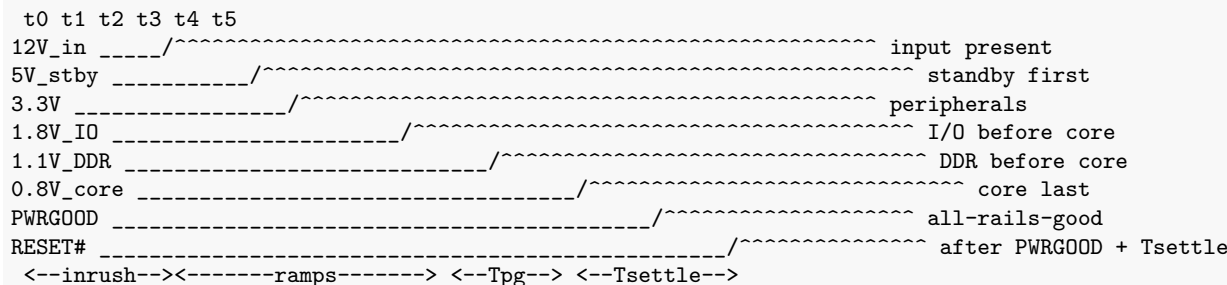
Load switches and eFuses: Load switches (FET + driver) gate a rail on/off under sequencer control. eFuses (electronic fuses with programmable OCP and slew-rate control) sit at the input and handle inrush limiting and over-current protection without a one-time-blow physical fuse. An eFuse latching off means an OCP event occurred — the board is asserting it cannot draw the current it needs, which is distinct from a regulator fault.

12.1.4 Power Sequencing — The Datasheet Is Law

Multi-rail devices require rails to power on and off in a **specified order with specified inter-rail delays**. The sequencing diagram in the SoC/GPU/DDR datasheet is not a suggestion. Violating it causes:

- **Latch-up:** I/O rail present before core → the I/O Electrostatic Discharge (ESD) protection diodes forward-bias into the unpowered core supply, injecting current into the substrate and N-wells and triggering the parasitic PNP thyristor structure inherent in CMOS processes. Once the thyristor fires, it latches into a low-impedance short between VDD and GND that sustains itself — even if the triggering condition is removed — until power is cycled. The classic bench sign is a rail that was in spec at initial warm-up and now reads 0.2V while the bench supply current-limits at 5A: that is a latched device drawing the supply's maximum current through the parasitic short. Apply no further power until you understand which rail violated sequence; the part may already be destroyed.
- **Strap miscapture:** Strapping pins and configuration inputs are sampled at de-reset. If the supply providing pull-up or pull-down voltage on strap pins is not stable before reset deasserts, the device boots with the wrong PCIe width, wrong bus ID, or wrong SerDes mode.
- **DDR training failures:** DDR controllers must see VDD, then VPP/VTT, then begin training against a stable array. Out-of-order rail arrival causes training to run against an unstable reference → intermittent ECC events that look random because they only appear under combinations of temperature and load that stress the marginally-trained DDR.

A typical compute-board sequence (illustrative):



PWRGOOD is the AND of all individual rail power-good outputs. RESET# deasserts only after PWRGOOD has been asserted for a minimum settling time (Tsettle, often 1-5ms).

What to probe on the bench to verify sequencing:

Set up four to eight scope channels simultaneously (multi-channel scope or two scopes synced via trigger-out). Assign one channel each to the primary input, the two or three critical mid-level rails, core, PWRGOOD, and RESET#. Capture on a single shot triggered by the 12V edge. Then:

1. Verify each rail rises before the one that depends on it.
2. Measure the inter-rail delays and compare against the datasheet min/max windows.
3. Verify PWRGOOD does not assert until *all* monitored rails are in regulation.
4. Verify RESET# deasserts only after PWRGOOD + Tsettle.

A sequencing fault is the kind of defect that passes at room temperature and fails in the field after a cold soak, because the regulator startup times shift with temperature. That is why sequencing must be verified at both cold and hot temperature extremes, not just ambient.

12.1.5 Inrush Current

At power-on, the board's bulk and decoupling capacitance charges from approximately zero, drawing a large transient current that can be 5-20x the steady-state idle current. Concerns:

- **Tripping the supply's current limit:** the supply folds back, the rail never reaches nominal, and the board appears dead. On a bench supply this is easy to diagnose (voltage collapses, current pegs the limit). On a vehicle bus the symptom is a connector-level voltage droop during startup.
- **Blowing eFuses:** an eFuse programmed too aggressively for the actual inrush will latch off, leaving the board dead until reset.
- **Connector stress:** repeated high inrush cycles stress the connector contacts.

Mitigation is usually a soft-start (the VRM ramps its output over a set period, limiting dV/dt and therefore dI/dt on the input), or an inrush-limiting circuit on the eFuse controller. On boards with a hot-swap controller (e.g., LTC4260-class), the controller explicitly limits inrush to a programmed current.

Measuring inrush: current probe clipped over the 12V input wire, scope single-shot triggered on the rising edge of the 12V rail. Measure peak amplitude (must be below the PSU's limit and the eFuse threshold) and duration. On a current-limited bench supply during bring-up, set the current limit just above the expected inrush: inrush trips it harmlessly instead of letting a latent short survive.

12.2 Measuring Rails Correctly

Getting a rail measurement right requires the right instrument for the right quantity and disciplined technique. Using the wrong instrument or wrong technique for the physical quantity being measured is one of the most common engineering mistakes.

12.2.1 DMM for DC Accuracy

A calibrated 6.5-digit DMM gives you the accurate DC value. For low-voltage, high-current rails, the series resistance of your test leads (a typical DMM lead pair may have 50-200m Ω) introduces a voltage drop that is significant at 0.8V with a 100A load. **Use 4-wire (Kelvin) sensing:**

```
2-wire (incorrect for low-V/high-I rails):
DMM + o--[R_lead]---+---[load]---+
| (meas) | reads V_nominal - I*R_lead
DMM - o--[R_lead]---+-----+ ERROR proportional to I

4-wire / Kelvin (correct):
Force+ o--[R_lead]---+---[load]---+
Sense+ o-----+ | Sense leads carry no current
Sense- o-----+ | reads true V_load (at the device)
Force- o--[R_lead]---+-----+
```

At 0.8V nominal with 100A draw through 100 $\mu\Omega$ (0.1 m Ω) of force-path/contact resistance, the 2-wire error is 10mV — a non-trivial fraction of a 24mV one-sided spec allowance (3% of 0.8V). 4-wire eliminates it. (100A across the 100m Ω of a DMM lead pair would be a 10V drop, not 10mV — the m Ω -scale figure belongs to the no-load lead context, not a 100A force path.)

Additional DMM discipline:

- **Let the reading settle** (1-3 seconds on autorange; faster on a fixed range).
- **Use a fixed range** rather than autorange for repeatable measurement timing and to avoid the transient glitch when the meter range-switches.
- **Log the instrument IDN string and calibration due date** alongside every measured value. NIST-traceable calibration is part of the safety case.

- The DMM tells you the DC value. It tells you nothing about ripple or transient events — that is the scope’s job.

12.2.2 Oscilloscope for Everything Time-Varying

The scope is the right instrument for sequencing order and timing, inrush current (with a current probe), ripple and noise on a rail, and load-transient droop and recovery.

Measuring ripple — the discipline that separates production-ready measurements from bench hacks:

1. **AC-couple the channel.** The DC level (e.g., 0.8V) forces a coarse V/div setting that buries millivolt ripple. AC coupling removes the DC offset so you can use 1-5 mV/div to see the ripple directly.
2. **Enable the 20 MHz bandwidth limit.** This is the most commonly omitted step. Without it, the probe tip acts as an antenna and couples radiated RF into the measurement, making clean rails look noisy. Rail ripple specs are defined up to 20 MHz; broadband noise above 20 MHz is not part of the specification and will make a passing rail appear to fail.
3. **Use the shortest possible ground return.** A long ground-clip lead is a loop antenna. The loop area times dB/dt of nearby switching currents induces a voltage into your measurement. For mV-level work, use a spring-tip barrel that grounds at the probe tip, or clip the ground barrel directly across the decoupling cap pads near the load.
4. **Probe at the load’s bulk decoupling capacitor,** not at the regulator output. The spec is what the device sees, which is at its own decoupling caps, downstream of the PDN inductance.
5. **Measure Vpp and note the frequency.** Ripple at the regulator’s switching frequency is normal switching ripple. Ripple synchronous with a load event is droop from insufficient bulk capacitance or high PDN impedance. Broadband fuzz that disappears when you improve grounding is a measurement artifact.

Poor probing (long ground clip, no BW limit):

```
Vpp reads 85 mV -- probe loop antenna + RF coupling dominate
"FAILS 30 mVpp spec" <-- the measurement is lying
```

Correct probing (AC coupled, 20 MHz BW, barrel ground at cap):

```
Vpp reads 18 mV -- actual switcher ripple at fSW, clean and periodic
"PASSES 30 mVpp spec" <-- true result
```

Load transient measurement: trigger the scope on a GPIO that fires when the GPU stress-test starts. Capture the core rail droop as the load steps from idle to full. Measure: - Peak droop (how far below nominal) - Recovery time (time from step to return within 1% of nominal) - Overshoot after recovery

A well-designed VRM with adequate bulk capacitance droops less than 3% and recovers within a few microseconds. A weak PDN droops through the spec limit and may not recover before the next load step — which is exactly when the GPU starts reporting PCIe errors.

12.2.3 Probing Checklist for a Rail Under Test

A concise field reference for scope work on a low-voltage rail. Each item has a reason; omitting any one of them produces a measurement you cannot defend.

Step	Action	Why
1	AC-couple the channel	Removes the DC offset so you can use 1-5 mV/div and see ripple directly
2	Enable 20 MHz BW limit	Rail ripple specs are defined to 20 MHz; RF above that is measurement artifact

Step	Action	Why
3	Shorten the ground return to the probe tip	Long ground clips are loop antennas; a 1 cm spring-tip barrel reduces loop area by ~100x vs a 15 cm clip
4	Probe at the bulk cap nearest the load pins	The spec is at the device, not at the regulator output; PDN inductance between the two is real
5	Note the ripple frequency and character	Switching-frequency ripple = normal; load-synchronous droop = PDN impedance problem; fixed-frequency ring = loop instability
6	Record Vpp, frequency, and load state	A passing Vpp at idle and a failing Vpp under load are different results; document both
7	If measuring DC: switch to DC-couple, widen V/div	AC coupling blocks DC; never report a DC rail value from an AC-coupled channel
8	Log scope settings and probe type with the result	Calibration and measurement context are part of the safety record

Power-rail probes vs. standard passive probes: Dedicated power-rail probes (Tektronix TPR series, Keysight N7020A-class) have 50 ohm input impedance, built-in DC offset range of +60V or more, and extremely low probe tip inductance. They eliminate the AC-coupling step and let you see DC level and AC ripple simultaneously on a single channel. On a production test station where you are measuring many rails repeatedly, the power-rail probe is the right tool; the passive-probe-with-AC-coupling technique is acceptable for bench bring-up when a dedicated probe is unavailable.

12.2.4 Power Analyzer for Efficiency and Long-Term Telemetry

A power analyzer (Yokogawa WT series, Keysight N6700-class source-measure units) adds continuous input power, output power, and efficiency measurements. The source-measure units also serve as programmable loads for automated DC margining. On a test station for a power-sensitive compute board, the power analyzer data feeds into the production result record alongside the rail measurements — efficiency at full load is itself a quality screen and a thermal predictor.

12.2.5 Reading Rails via BMC and sysfs (Unattended Line Use)

Probing every rail with a DMM on a production line is impractical. Instead, the board’s power monitors (typically PMBus or INA-class ICs hanging off I2C) report rail voltages and currents digitally, readable through the BMC via IPMI or through the kernel hwmon subsystem:

```
# IPMI (if a BMC is present)
ipmitool sensor list # all sensors: voltage rails, currents, temps
ipmitool sdr list # sensor data repository with threshold values

# hwmon sysfs
cat /sys/class/hwmon/hwmon*/name # identify the power-monitor IC
cat /sys/class/hwmon/hwmon0/in1_input # voltage in millivolts
cat /sys/class/hwmon/hwmon0/in1_min # low threshold
cat /sys/class/hwmon/hwmon0/in1_max # high threshold
cat /sys/class/hwmon/hwmon0/curr1_input # rail current in milliamps
```

These power monitor ICs live on I2C — closing the loop that a “digital” power reading is itself a low-speed bus transaction. The station reads them at idle, then under load, and compares to the guard-banded test limits:


```
def verify_power_rails(expected_rails: dict) -> dict:
    """
    expected_rails: {'12V': (11.4, 12.6), '0.8V_core': (0.776, 0.824)}
    Tuple is (spec_min, spec_max) in volts.
    Capture measured value always -- pass/fail alone loses the distribution.
    """
    results = {}
    for name, (vmin, vmax) in expected_rails.items():
        measured = read_voltage_from_hwmon(name)
        midpoint = (vmin + vmax) / 2.0
        half_window = (vmax - vmin) / 2.0
        results[name] = {
            "measured": measured,
            "spec_min": vmin,
            "spec_max": vmax,
            "pass": vmin <= measured <= vmax,
            "margin_pct": round(100.0 * (measured - midpoint) / half_window, 1),
        }
    return results
```

Capture the measured value, not just the pass/fail verdict. A rail that sits at 0.780V today and 0.776V next week is on a trajectory, and you only see it if you kept the numbers. The measured values feed the Statistical Process Control (SPC) charts and the data-driven-limit review that is part of the ongoing safety argument.

12.3 Thermal Management and Thermal Test

12.3.1 Junction Temperature and Thermal Resistance

Every semiconductor die has a maximum rated junction temperature (T_J) — the temperature inside the chip at the active junctions. Die junction temperature is not directly measurable in the field; it is inferred from the package case or heatsink temperature using the thermal resistance network:

```
P_die (heat source)
|
| theta_jc (junction-to-case resistance, per datasheet)
|
T_case (measurable on the package lid)
|
| theta_cs (case-to-heatsink: TIM + mounting)
|
T_heatsink
|
| theta_sa (heatsink-to-ambient: fin area, airflow)
|
T_ambient
```

$$T_J = T_{ambient} + P \times (\theta_{jc} + \theta_{cs} + \theta_{sa})$$

On a compute board under GPU stress at 200W with θ_{jc} of 0.1°C/W and a well-mounted heatsink, junction temperature can be 20-30°C above the measurable case temperature. The manufacturing test must therefore measure the *case* temperature and use the thermal resistance chain to infer junction temperature, or rely on the die's on-chip temperature sensor (the GPU's DTS, readable via `nvidia-smi`).

Thermal resistance and the test engineer:

- θ_{jc} is a silicon and package property. It varies unit-to-unit but you cannot change it.

- θ_{cs} (case-to-heatsink) is dominated by thermal interface material (TIM) quality and quantity, mounting pressure, and surface flatness. A poorly applied TIM pad or a heatsink not seated to torque spec will show up as an elevated case temperature at a given power level — a manufacturing defect screenable by measuring case temperature under a fixed load.
- θ_{sa} is an airflow and heatsink design problem. On a compute board tested in a fixture, the airflow conditions must match or bracket what the board sees in service.

12.3.2 Thermal Throttling

All modern compute SoCs and GPUs implement a thermal-throttle hierarchy. As the die temperature approaches limits, the firmware/hardware steps down clock frequency and/or voltage to reduce power and protect the junction:

- **First throttle level:** reduce GPU clock to a lower P-state; reduce CPU boost clocks.
- **Second throttle level:** shut off further cores; reduce memory bandwidth.
- **Thermal shutdown:** at the absolute maximum junction temperature, the device asserts a thermal trip signal and the system shuts down.

In manufacturing test, **throttling under the qualification load is a defect signal**. A board that throttles at ambient temperature under a load that field vehicles will sustain means the thermal solution is inadequate — bad TIM application, missing thermal pad, partial heatsink contact. The test must run a fixed, repeatable compute load (e.g., GPU matrix-multiply loop) and verify that neither throttling events (readable via `nvidia-smi -q | grep Throttle` or the `hwmon` throttle sysfs) nor junction temperature limits are exceeded under the expected field environment.

```
# Monitor GPU temperature and throttling in real time during stress
nvidia-smi --query-gpu=temperature.gpu,clocks_throttle_reasons.active,\
clocks.gr,power.draw --format=csv --loop=1

# Check hwmon for SoC die temperature
cat /sys/class/hwmon/hwmon2/temp1_input # millidegrees C
cat /sys/class/hwmon/hwmon2/temp1_crit # critical trip point

# Check kernel thermal zone and throttle state
cat /sys/class/thermal/thermal_zone*/temp
cat /sys/class/thermal/cooling_device*/cur_state
```

A test that measures clocks and computes performed while the device is throttling is measuring a degraded state, not the nominal state — the result is meaningless for its intended purpose. The test must confirm the device is running unthrottled before capturing performance metrics.

12.3.3 Thermal Soak and Temperature Corners

Thermal soak refers to stabilizing the board at a target temperature (hot or cold) before running a test, to ensure the entire board — not just the die surface — is at temperature. A soak time of 10-30 minutes is typical for a large, multi-chip board. Without a soak, the DRAM and peripheral ICs may still be at ambient even though the GPU die has reached its target temperature, producing results that do not represent a truly hot or cold board.

Temperature corners for manufacturing test:

Corner	Typical Target	What It Stresses
Cold soak	-40°C to -20°C	Crystal oscillator frequency, PLL lock range, DDR training window, regulator dropout at low Vout
Ambient	25°C	Baseline; characterization reference point

Corner	Typical Target	What It Stresses
Hot (operating limit)	85-105°C (board ambient)	Thermal throttle threshold, rail droop under load+heat, electromigration onset, SerDes eye closure at temperature

The Arrhenius rule of thumb is that failure rate roughly doubles per 10°C rise in junction temperature. This is why hot-corner screening is non-negotiable for a safety-critical compute board: defects that do not manifest at ambient become infant mortalities and field failures at operating temperature.

12.4 DC and Transient Margining (Shmoo Testing)

12.4.1 Voltage Margining

Voltage margining deliberately shifts a rail above and below nominal to find the functional operating margin. The VRM output is adjusted either through a PMBus command or by changing the feedback-resistor DAC on a supported regulator. A shmoo sweeps both voltage and (where possible) temperature simultaneously:

- **Pass/fail shmoo:** for each (V, T) point, run the critical test (PCIe link train, DDR ECC test, Gigabit Multimedia Serial Link (GMSL) lock). Mark pass or fail. The boundary of the passing region is the functional operating region — compare it to the spec window.
- **Margin shmoo:** instead of pass/fail, capture a margin metric (PCIe eye height, Bit Error Rate Test (BERT) error count, Advanced Error Reporting (AER) correctable-error count) as a function of V and T. The margin degrades continuously with voltage and temperature; the test limit is set where the margin has headroom to the spec boundary.

On a compute board, the SoC core rail shmoo is the primary screen: confirm that the device meets its functional requirements with the core rail at +5% and -5% of nominal across the full temperature range. A unit that fails at -5% core voltage at 85°C has a silicon speed bin problem — it passed at nominal but has insufficient margin. That is a screen-and-reject decision, not a limit-relaxation decision.

The shmoo is a characterization tool at bringup, producing the distribution from which limits are set. On a production line, the shmoo itself is too slow; instead the production test exercises the nominal voltage at both temperature corners, plus a brief voltage-step check (margin in, hold for 100ms, verify no errors) to screen for units near the margin cliff.

12.4.2 Transient Margining

Transient margining adds controlled perturbations to test stability under dynamic conditions:

- **Load transients:** step the compute load from idle to full and back while monitoring the rail. The load-step is triggered programmatically (launch GPU kernel; measure before and after).
- **VRM enable/disable cycling:** cycle the rail on and off under the sequencer and verify the sequencing timing and PWRGOOD behavior repeat within spec for 10-100 cycles.
- **Ripple injection:** a stimulus injection network adds a known-amplitude AC component to the rail feedback path to characterize loop bandwidth. This is advanced bench characterization, not production test, but the Design Verification (DV) results set the bounds on what the production ripple test must catch.

12.5 Board Bring-up — Structured and Gated

The goal of bring-up is to take a bare Printed Circuit Board Assembly (PCBA) from powered-off on the bench to fully-enumerated and characterized, while never letting a defect destroy the board. The method is cheapest-safest-first, gated: you do not move to the next phase until the current phase passes cleanly.

12.5.1 Phase 0 — Documentation and Bench Preparation

Have the **schematic, board layout, BOM, power-sequencing diagram, and all connector pinouts** open before touching the board. Know the expected device list: which PCIe endpoints at which root ports, bifurcation settings, retimers, Network Interface Card (NIC)/Non-Volatile Memory Express (NVMe)/GPU devices, which I2C addresses and buses, which rails and their tolerances. Set up at an ESD-safe station: wrist strap on and verified, dissipative mat grounded to the station common point. Stage: current-limited bench supply, 4-wire DMM, scope with current probe, USB-TTL console adapter at the correct logic voltage (3.3V or 1.8V — confirm before connecting), and thermal camera if available.

Do not skip Phase 0. The engineer who brings up a board without the schematic in hand and discovers a dead short at power-on has no idea which net is shorted, which IC is damaged, or how to find it. Phase 0 is the difference between a two-hour bring-up and a two-week archaeology project.

12.5.2 Phase 1 — Visual Inspection and Continuity

Before any power:

- Inspect under magnification for assembly defects: **missing components, solder bridges (especially on BGA escape routes and fine-pitch connectors), tombstoned passives, wrong orientation** (pin-1 markers on ICs, polarity marks on electrolytics and tantalums, diode direction), bent or missing connector pins.
- Cross-check placed parts against the BOM for obvious wrong values where it matters (0Ω jumpers, filter resistors, key passives on regulated rails).
- With the DMM in resistance/continuity mode: **measure each power rail to GND**. A near-zero ohm reading on a rail before power is applied means do not power up — find the short first. This check catches shorted bulk caps, solder bridges to GND, and dead-shortened ICs. It takes five minutes and prevents the most expensive possible bring-up mistake.

12.5.3 Phase 2 — Controlled Power-On

Apply power from a **current-limited bench supply** with the limit set just above the expected inrush-plus-idle current. Watch the current draw constantly:

- **Dead short:** current hits the limit immediately, voltage folds back to near zero. Kill power. A short on the 12V input looks like this immediately; a short on a secondary rail may let the primary come up but that rail's regulator current-limits. Find the short (thermal camera, resistance measurement with supply off) before re-applying power.
- **Normal power-on:** a brief inrush spike (a few milliseconds), then the current settles to the expected idle level.
- **Excessive idle current:** primary came up, but idle draw is 2-3x expected. A short or damage is present even if the voltage reads nominal — the regulator is just strong enough to hold the rail while sourcing fault current.

Once the current looks right, verify **every rail** with the DMM (4-wire on any rail at or below 1.8V), and verify **sequencing** with the scope (capture all rails simultaneously on a single shot triggered by the 12V edge). Check PWRGOOD and RESET# timing against the datasheet. Run a hand or thermal camera over the board to find any hot parts — a regulator that should be warm is fine; an IC that has no reason to be hot is a fault even if all rails read in spec.

Do not leave Phase 2 until rails, sequencing, and thermal are clean. Every sequencing bug you do not fix here will show up as a boot failure or an intermittent hard-to-reproduce fault later, at exactly the wrong time.

12.5.4 Phase 3 — Boot to Console

Connect the **UART debug console** at the correct voltage (almost always 3.3V for a debug header, sometimes 1.8V on tight-packaged SoCs). Settings: 115200 baud, 8 data bits, no parity, 1 stop bit (115200 8N1) is the default for nearly every embedded Linux platform; confirm from the schematic or reference design. Use `picocom (picocom -b 115200 /dev/ttyUSB0)` for a lightweight interactive terminal that exits cleanly with Ctrl-A Ctrl-X.

Power on and watch the boot messages. Every line that scrolls past represents a subsystem initializing successfully. The bring-up log is a record of the board’s health — save it. When the boot stops:

Where it stops	What failed
Immediately, blank screen	No clock, no reset release, or a severe sequencing problem
During POST/BIOS self-test	Firmware crash; CPU or chip-level fault
“Initializing memory controller” hang	DDR rail marginal, DDR training failure; check 1.1V rail
“PCI probe” hang	PCIe root port or endpoint power problem; check that device’s rail and PERST# (PERST#)
Kernel panic on “loading initramfs”	Storage (NVMe/eMMC) issue; check NVMe rail and reset
After full kernel boot, hang on device driver	Driver probe failure; read dmesg carefully

The console is the most valuable single instrument in a bring-up. Every other measurement is cross-referenced against what the console reported.

12.5.5 Phase 4 — Device Enumeration

Once at a Linux prompt, verify every device the schematic says should be present actually appears:

```
lspci -nnvvv # PCIe: present at expected BDF? Link speed/width correct? AER clean?
lsusb -t # USB topology
ip link show # NICs enumerated and reported up?
lsblk # block devices (NVMe, eMMC)
nvme list # NVMe drives: model, serial, firmware version
i2cdetect -y 1 # I2C bus 1: expected sensors ACKing at expected addresses?
v4l2-ctl --list-devices # GMSL camera pipelines: deserializers enumerated?
dmesg --level=err,warn # kernel errors during boot (probe failures, timeouts, -ENXIO)
lshw -short # full hardware inventory
dmidecode # SMBIOS: board revision, memory config, firmware versions
```

A missing device is the thread to pull. Cross-reference its power rail (is it in spec?), its reset line (PERST# / RESET# asserted or deasserted correctly at the right time?), its configuration straps (PCIe width, address), and its I2C presence signal. The absence almost always traces to power, reset timing, a wrong strap, or a solder defect on that specific block. Start with the schematic, not with guessing.

12.5.6 Phase 5 — Subsystem Functional Tests

Exercise each major block individually so a failure is unambiguous and not attributed to an interaction:

```
nvidia-smi # GPU: driver bound, PCIe link speed, ECC mode, temp
nvidia-smi -q | grep -E "ECC|Error" # GPU ECC errors (should be zero on a new unit)
```

```

nvme smart-log /dev/nvme0n1 # NVMe SMART: media errors, unsafe shutdowns, temp
fio --name=seqread --rw=read --bs=128k \
  --filename=/dev/nvme0n1 --direct=1 \
  --size=4G --numjobs=1 # NVMe sequential read: throughput check
ethtool eth0 # NIC: link speed, duplex, negotiated mode
iperf3 -c <test-server> -t 10 # NIC throughput: must hit line rate or near it
free -h # memory size matches expected?
memtester 1G 1 # basic DRAM integrity check
cansend can0 123#DEADBEEF # CAN: inject frame
candump can0 & # CAN: verify it loops back
v4l2-ctl -d /dev/video0 --stream-mmap \
  --stream-count=120 \
  --stream-to=/dev/null # GMSL: capture frames; proves ser->coax->deser->SoC

```

Run each test to completion and verify the output matches the expected result before moving to the next. A thermal camera pass at the end of Phase 5 (under load) is a good practice: the thermal picture under functional load is more informative than the idle-power-on picture from Phase 2.

12.5.7 Phase 6 — Characterization (Data Drives Limits)

This phase generates the distribution data that sets production test limits. You cannot set a defensible limit from a guess; you set it from a distribution of known-good units measured under the full range of expected conditions.

- Run **CPU+GPU stress** (`stress-ng`, CUDA matrix-multiply loops) and capture: core temperatures, throttle onset (note the temperature and clock-step), rail voltages under load (AC-coupled ripple + DC droop), power consumption.
- **Re-verify all rails under full load:** check that no rail droops below its spec window at maximum draw. The rails-under-load measurement is the load-regulation screen; idle measurements alone are insufficient.
- **PCIe margin characterization:** run `lspci` to confirm link width and speed under load, use PCIe lane-margining (PCIe 4.0+ receivers support `PCIeLinkMarginReq` via the margining registers) to measure voltage and timing margin per lane, capture AER counters before and after a 30-minute stress run.
- **Temperature corners:** soak at cold, run stress, capture; soak at hot, run stress, capture. The distribution at each corner is the input to the hot and cold test limits.
- **Inrush characterization:** measure peak current and duration; compare to eFuse threshold and PSU limit.

The output of Phase 6 is a set of measured distributions. Guard-banded test limits are derived from those distributions (mean +/- N-sigma, or a percentile plus tolerance accounting for measurement uncertainty). Any limit that is not traceable to this measured data is not a defensible limit for a safety-critical product.

12.5.8 Bring-up Checklist

```

[ ] Phase 0 Schematic/BOM/sequence/pinout in hand; ESD station verified
[ ] Phase 1 Visual: no bridges/shorts/missing/mis-oriented; rail-to-GND resistance OK
[ ] Phase 2 Current-limited power-on; I draw normal; all rails in spec; sequencing OK;
  nothing hot
[ ] Phase 3 UART console: boots to login; log saved; (stop = last line names the fault)
[ ] Phase 4 lspci/lsusb/ip/nvme/i2cdetect/v4l2/dmesg match expected; no errors
[ ] Phase 5 GPU/NVMe/NIC/CAN/memory/GMSL each exercised individually; pass
[ ] Phase 6 Stress + thermals; rails under load + ripple; PCIe margin hot+cold;
  power/inrush; corner sweep -> distributions -> guard-banded limits

```

12.6 Functional Safety (ISO 26262) — What a Compute Test Engineer Must Know

12.6.1 The Standard and Its Purpose

ISO 26262 is the international standard for functional safety of automotive electrical/electronic (E/E) systems, derived from IEC 61508 and adapted for the vehicle domain. Its 2018 revision (second edition) added coverage for semiconductors (Part 11) and motorcycles (Part 12). The standard addresses the *entire safety lifecycle*: concept, system design, hardware design, software design, integration, verification, validation, and — critically for manufacturing engineers — **production and field operation**. Safety does not stop at the end of design verification; it continues through every board that ships.

The central concept is **Automotive Safety Integrity Level (ASIL)**, ranging from A (least stringent) through D (most stringent):

ASIL	Hazard Class	Requirements Rigor	Illustrative Example
QM	Quality management only; no specific FuSa requirement	Normal quality processes	Infotainment cosmetic display
A	Lowest FuSa level	Defined coverage, documentation	Seat heater control
B	Moderate	Increased coverage, FMEDA	Power steering assist monitoring
C	High	Significant coverage, diagnostic requirements	Automated emergency braking component
D	Maximum	Maximum coverage, fault injection evidence, full traceability	AV compute path: loss of vehicle control

A failure in Zoox’s compute platform that could contribute to **loss of vehicle control** is ASIL-D. That single fact explains why every test limit must be defensible, every result must be traceable, and every safety mechanism must be verified — not just the happy-path functionality.

12.6.2 The Safety Lifecycle

ISO 26262 defines a V-model lifecycle. The left side is decomposition (concept → system → hardware/software); the right side is integration and verification at increasing levels of integration. Manufacturing test sits at the bottom-right of the V: it is the final verification step before a unit enters the field, and it must produce evidence that the unit as built meets the requirements defined on the left side.

The safety lifecycle phases relevant to a test engineer:

1. **Hazard Analysis and Risk Assessment (HARA)**: defines the ASIL for each safety goal. You consume the ASIL classification; you do not derive it. But you need to understand it because it determines what your test must prove.
2. **Safety Requirements**: derived from the safety goals. System-level safety requirements flow down to hardware and software. Hardware safety requirements include things like “the ECC memory shall detect and correct single-bit errors” and “the compute module shall detect loss of a GMSL camera within 50ms.”
3. **Failure Mode and Effects Analysis (FMEA) / FMEDA (FMEA / Failure Modes, Effects, and Diagnostic Analysis)**: a systematic enumeration of all hardware failure modes, their effects at the system level, and the safety mechanisms that detect or control them. FMEDA also computes:
 - **Diagnostic coverage (DC)**: the fraction of the failure mode’s random hardware failure rate that the safety mechanisms detect. ISO 26262-5 Table 14 defines three reference levels: Low (60%), Medium

(90%), and High (99%) per the standard. The word “high” in FMEDA reports corresponds to the 99% tier (90% is Medium); claims of high DC require design evidence (architecture, test result, or analysis) that actually achieves that coverage in the fielded hardware.

- **SPFM (Single-Point Fault Metric) and LFM (Latent Fault Metric):** fractions of random hardware failures that do not cause a safety violation (either because they are covered by a safety mechanism or because they are not safety-relevant). ASIL-D targets SPFM $\geq 99\%$ and LFM $\geq 90\%$ (and a PMHF below 10 FIT = 10^{-8} failures/hour). These are hardware architectural metrics computed from the FMEDA failure rate table, not things you measure at a test station — but the validity of the calculation depends entirely on the safety mechanisms being functional in every shipped unit, which is what manufacturing test validates.
4. **Safety Mechanisms:** the hardware and software features that detect or tolerate faults. Examples on a compute board:
 - **ECC / Error Detection and Correction (EDAC):** detects and corrects single-bit DRAM errors; detects (but does not correct) double-bit errors.
 - **PCIe AER:** detects correctable and uncorrectable PCIe errors.
 - **Watchdog timers:** detect software hangs by requiring a periodic heartbeat.
 - **Redundant GMSL links:** tolerate a single coax failure.
 - **Voltage monitors / PWRGOOD:** detect rail out-of-spec events.
 - **Thermal trip logic:** prevents thermal runaway from reaching catastrophic junction temperatures.
 - **CRC / HMAC on safety-relevant data:** detects data corruption.
 5. **Fault Injection Testing:** at the hardware verification stage, faults are deliberately induced (bit-flips in DRAM, signal disruptions on PCIe, power glitches) to confirm the safety mechanisms respond as designed. Manufacturing test may include a lightweight version of this (inject a correctable ECC error, verify the correction counter increments) as part of the safety-mechanism verification coverage.

12.6.3 FMEA and FMEDA — What They Demand of Manufacturing Test

The FMEDA produces a list of failure modes and their safety mechanism coverages. For each safety mechanism, there must be evidence that it actually functions in every shipped unit. That evidence is produced by manufacturing test. The connection is direct:

- **FMEDA says:** “ECC provides high DC ($\approx 99\%$) for single-bit DRAM failures; ECC must function to achieve SPFM $\geq 99\%$.”
- **Manufacturing Test (MT) must prove:** ECC is enabled, ECC detects a correctable error (inject one or observe during memtest), and the EDAC driver reports it. Reading the corrected-error counter is not sufficient — you must verify it increments in response to an actual error event during test.
- **FMEDA says:** “Redundant GMSL links provide fault tolerance for coax failure; both paths must be independently functional.”
- **MT must prove:** both paths carry error-free video independently — not just that the active path works, but that if you disable the primary, the secondary is already functional (not just present).
- **FMEDA says:** “Watchdog timer provides DC for software hangs.”
- **MT must prove:** the watchdog is enabled at the firmware revision shipped, its timeout is set within the required interval, and it fires correctly when not serviced.

The point for daily work: your test does not only find manufacturing defects; it **verifies that the safety mechanisms enumerated in the FMEDA are functioning in this specific unit**. Testing the safety features is as important as testing the functionality, because a robotaxi depends on those mechanisms to fail safe.

12.6.4 ASIL Decomposition and Dual-Channel Architectures

When a single component cannot meet the required ASIL alone, the standard allows **ASIL decomposition**: split the safety requirement between two independent channels, each achieving a lower ASIL, such that the combination meets the original:

```
ASIL-D -> ASIL-B (channel A) + ASIL-B (channel B) -- most common
ASIL-D -> ASIL-C (channel A) + ASIL-A (channel B)
ASIL-D -> ASIL-D (channel A) + QM (channel B) -- asymmetric; QM channel is monitoring only
```

The notation in ISO 26262-9 is ASIL X(Y), where Y is the parent safety goal ASIL and X is the reduced ASIL of the decomposed element (e.g., ASIL-B(D) means “ASIL-B methods applied to a requirement that originated from an ASIL-D safety goal”). The key constraint is **independence**: decomposition is invalid if the two channels share a power supply, clock, common-cause failure path, or any single point of failure. A shared 12V input without independent over-current protection is enough to invalidate a decomposition.

Zoox-style compute platforms frequently decompose safety requirements across redundant compute modules or across primary and secondary compute paths. This means:

- Manufacturing test must verify **both channels independently** — a dual-channel decomposition with one dead channel is effectively undecomposed and does not meet the ASIL-D requirement.
- Failure of the interface between channels (the cross-channel monitoring link) is itself a safety-relevant failure mode that must have a safety mechanism and be tested.

12.6.5 What ISO 26262 Means for Records and Traceability

ISO 26262 defines the required traceability chain for a safety-critical unit. This is not merely a quality-management aspiration; it is an audit requirement with legal liability implications for a vehicle in service:

- **Unit serial number** → assembly genealogy (which lot of each component went into this unit) → **test-program version** → **measured test results** → **disposition** (pass/reject/rework).
- Every link in this chain must be auditable and queryable. If a field failure occurs or a bad component lot is discovered, you must be able to identify *every unit that contains a component from that lot* within hours, not weeks.
- The test-program version is part of the traceability record because a limit change in the test program is a change to the safety argument. The history of what limits were in force when a unit was tested is permanently relevant.

Change control under ISO 26262: a change to a test limit, a test procedure, or the test program itself touches the safety argument and requires:

- Version control and a tagged release (see the version-control discipline in the companion embedded-buses chapter).
- A golden-unit gate: re-test a known-good unit and a known-marginal unit with the new limits to confirm they still pass and still fail respectively.
- Documentation of the rationale for the change, the analysis supporting it, and who approved it.

“We loosened a limit to recover yield” is not an acceptable rationale under ISO 26262 unless accompanied by analysis showing the loosened limit does not increase the escape rate for safety-relevant failures. The standing principle: **ship only good units; improve yield by reducing false fails, never by accepting more real fails.**

12.6.6 Diagnostic Coverage and What it Implies for Screening

Diagnostic coverage (DC) quantifies how thoroughly safety mechanisms detect the random hardware failure modes they are supposed to cover. DC is a calculation, not a measurement — but the *inputs* to that calculation must be validated by test:

- If the FMEDA claims high DC ($\approx 99\%$) for ECC on DRAM, that claim depends on ECC being enabled, functional, and correctly configured in every shipped unit. Manufacturing test provides that validation.
- If the FMEDA claims DC for a voltage monitor detecting rail out-of-spec events, that depends on the voltage monitor thresholds being set correctly and the monitor being accessible (not hung/dead) in every shipped unit.

A safety mechanism that exists in the design but is disabled or misconfigured in a specific unit contributes zero DC for that unit, regardless of what the FMEDA says. Manufacturing test is the gate that ensures the design's safety-mechanism coverage is actually realized in every unit.

FMEDA-to-test mapping in practice: the FMEDA document is the specification for what your test must exercise. For each safety mechanism row in the FMEDA, ask: (a) is there a test step that confirms this mechanism is present and functional in this unit? (b) does that step produce a binary result (mechanism fires / does not fire) or only a proxy (register readable but never stimulated)? The standard distinguishes between diagnostic coverage claimed because a mechanism *exists in the design* and coverage claimed because the mechanism *is proven functional at test*. For ASIL-D, only the latter counts. A practical cross-reference table format:

FMEDA Failure Mode	Safety Mechanism	MT Step	Acceptance Criterion
DRAM single-bit error	ECC SECDED	ECC-inject	EDAC counter +1, no crash
Rail undervoltage event	PMBus UV threshold	MV-margin	FAULT# asserts at V _{nom} -4%
Watchdog expiry	WDT reset	WDT-test	Reset occurs within timeout+10%
GMSL coax open	Redundant path failover	SI-fault	Secondary stream valid <100ms

Build this table at bringup and keep it alive. Every FMEDA revision that adds or changes a safety mechanism requires a corresponding MT step change — that linkage is part of the change-control argument.

12.6.7 RAS Requirements and Their Interaction with Manufacturing Test

Reliability, Availability, Serviceability (RAS) — captures the operational requirements that sit alongside and overlap with functional safety:

- **Reliability** requirements (e.g., MTTF targets, infant-mortality budgets) are supported by thermal screening (burn-in, HTOL) and voltage-margining screens that accelerate and expose weak or damaged devices before they reach the field.
- **Availability** requirements (the system must be available >99.X% of operating hours) drive the redundancy architecture that MT must verify both paths of, as discussed above.
- **Serviceability** requirements (failures must be diagnosable without returning the vehicle to a depot) drive the need for the Design for Testability (DFT) hooks — JTAG, UART console, IPMI, in-system test points — that manufacturing test also uses. These hooks must work in every unit.

The interplay at manufacturing test: a unit that passes all functional tests but has a dead UART console debug port, a disabled BMC IPMI interface, or a non-functional JTAG chain has reduced serviceability in the field, which means failures will be harder to diagnose and repair. These are screenable defects at manufacturing test, not field-discovery failures.

12.6.8 The Manufacturing Test Safety Case Contribution

The total argument that a compute board is safe to ship is a **safety case** — a structured argument with evidence. Manufacturing test contributes evidence at several levels:

1. **Hardware conformance:** the board as built matches the design (correct components, correct assembly, no manufacturing defects).
2. **Functional correctness:** all functions operate within specification under the full range of environmental conditions (voltage corners, temperature corners).
3. **Safety mechanism verification:** each safety mechanism enumerated in the FMEDA is functioning in this specific unit.

4. **Parametric traceability:** measured values for safety-relevant parameters are recorded and traceable to the unit serial number, component genealogy, and test program version.

These four contributions, taken together, are the manufacturing test’s contribution to the safety case. A test program that only checks pass/fail on happy-path functionality makes no contribution to items 3 and 4, and an incomplete contribution to item 2. The test engineer who understands ISO 26262 designs a test that covers all four, and defends that design against the “we only need to test what breaks during assembly” objection with the argument that the standard requires it.

What a safety auditor looks for in a manufacturing test record:

- Test-program version number and the git commit or release tag it corresponds to.
- A link from each test step to the requirement it satisfies (requirement ID from the safety requirements document, or at minimum the FMEDA row it validates).
- Measured values, not just PASS/FAIL verdicts, for all safety-relevant parametric tests.
- Evidence that safety mechanism activation tests (ECC inject, watchdog fire, GMSL failover) produced the correct observable response — not just that the test step ran.
- Re-test traceability: if a unit was reworked and re-tested, the record must show which steps were re-run, with the new results linked to the same serial number, with the rework action documented.

A test record that cannot answer “what firmware version was loaded, which test-program version ran, and what did each safety-relevant rail measure at hot-corner full load?” for an arbitrary serial number is not a compliant safety case contribution.

Concession and deviation discipline: if a unit is dispositioned as acceptable with a test result outside the normal pass window (a “concession”), the concession document must include the safety analysis showing the out-of-window result does not increase safety risk. Blanket “use-as-is” dispositions without analysis are a common audit finding and a liability exposure.

12.7 Correlating Rail Problems with Digital Failures

This is the “digital failures are often power failures” instinct made concrete and operational. The pattern occurs regularly enough that it should be reflexive:

Symptom: PCIe link retrains or falls back in width/speed under load. Look first at the core rail of the endpoint device under load. AC-couple the scope and measure ripple; DC-couple and measure droop during the load step. A rail that sags 60mV at the moment of link retrain is the Root Cause Analysis (RCA). Also check the SerDes PLL supply: a switching regulator near the SerDes that couples into the PLL reference creates deterministic jitter at the switcher frequency, visible as a periodic component in the PCIe eye.

Symptom: GMSL camera loses lock under heavy load or at temperature. Check the PoC (Power over Coax) voltage at the connector: it may be drooping if the PoC regulator is shared with a heavily loaded domain. Check the serializer supply at the camera end (read via I2C through the GMSL reverse channel). A serializer undervoltage causes re-initialization events that look identical to a coax signal- integrity problem.

Symptom: ECC correctable errors increasing with temperature and time under load. Check the DDR VDD and VTT rails under load. DDR5 at 1.1V with insufficient bulk capacitance or a weak VRM droops during burst accesses — that droop widens the DDR timing eye and causes correctable (then uncorrectable) errors. Also check the DDR reference voltage (VREF) stability. If the ECC errors appear exclusively at temperature, scope the rail hot: a rail that passes at ambient may droop to $V_{nominal} - 5\%$ at 85°C due to regulator thermal derating.

Symptom: Intermittent boot failures, especially after cold starts. Scope the 3.3V peripheral rail and the 1.8V I/O rail during cold power-on. LDOs and linear regulators can have sluggish startup at -40°C; a rail that comes up 5ms late at cold causes the SoC to sample incorrect strap pins or begin DDR training

against an unstable reference. The failure is inconsistent because most cold soaks only go to -20°C, not the -40°C corner where the startup time shifts enough to violate the sequencing window.

The diagnostic discipline: when a digital symptom appears, check the relevant rails under the same conditions (load, temperature, time-into-test) that produce the symptom. Correlate the scope timebase with the error log timestamp. A rail that droops at the exact moment the error log records a correctable error is root-cause evidence, not circumstantial evidence.

Chapter 13

Control Systems

A compute test engineer is not a controls engineer, but test fixtures, the board under test, and the vehicle systems around it all close control loops — thermal forcers, fan speed regulators, power-supply feedback loops, and the higher-level servo control in the AV stack. You need fluency: enough to hold a five-minute technical conversation, justify a fixture’s control design with engineering reasoning, and recognize the control-loop signatures that show up in compute board behavior.

13.1 Feedback, Open-Loop, and Closed-Loop

Open-loop control applies a command based purely on a model of the plant, with no measurement of the actual output. A heater set to a fixed duty cycle is open-loop: it does not know whether the board temperature is rising, falling, or already at target. Open-loop is simple, fast, and noise-free — it is also blind to disturbances and plant variation.

Closed-loop (feedback) control measures the output, compares it to a setpoint, and generates a correction based on the error. A PWM fan controller that reads a temperature sensor and adjusts duty cycle is closed-loop. Feedback rejects disturbances (a sudden load increase on the board) and tolerates plant variation (every board has slightly different thermal resistance). The cost is measurement noise, the risk of instability, and the latency introduced by sensing and actuation.

Most real systems combine both: **feedforward** provides an open-loop estimate that gets close immediately; **feedback** trims the residual error. This is the design choice that appears in well-engineered test fixtures and in the regulators on the board under test.

13.2 PID — Intuition for Each Term

The three PID terms each address a different aspect of the error signal:

13.2.1 Proportional — Present Error

```
correction = Kp * error
```

The proportional term reacts to *right now*. Larger K_p means a stronger response to error, which speeds up convergence but also drives toward oscillation as K_p increases. The fundamental problem: as the error shrinks, so does the correction, and the system settles *near* the setpoint but not exactly at it. That permanent

residual offset is called **steady-state error**, and proportional-only control always leaves it (unless K_p is impractically large).

Tuning symptom: **K_p too low** — slow convergence, large steady-state error. **K_p too high** — oscillates around the setpoint, possibly goes unstable.

13.2.2 Integral — Accumulated Error

```
integral += error * dt
correction = Ki * integral
```

The integral accumulates error over time. As long as any steady-state error remains, the integral keeps growing and the correction keeps increasing, driving the output until the error reaches zero. This is the only term that eliminates steady-state error from constant disturbances (a constant load on a thermal system, a constant valve bleed in a pneumatic press).

The pathology: **integral windup**. If the actuator saturates (maximum heater duty, or maximum regulator voltage trim) while a large error persists, the integral keeps accumulating even though additional integration changes nothing. When the error finally clears, the over-wound integral drives a violent overshoot. **Anti-windup** prevents this by clamping the integral to a physically meaningful range:

```
integral += error * dt
integral = max(-max_integral, min(integral, max_integral)) # clamp
correction = Ki * integral
```

Tuning symptom: **K_i too low** — slow elimination of steady-state error, may never fully converge. **K_i too high** — oscillates with a growing amplitude (unstable), or winds up and overshoots badly. **Missing anti-windup** — well-tuned steady state but dramatic overshoot after saturation periods.

13.2.3 Derivative — Rate of Change of Error

```
correction = Kd * d(error)/dt
```

The derivative predicts where the error is going and applies a braking correction. It damps oscillation and speeds convergence near the setpoint. The two serious problems: it **amplifies measurement noise** (high-frequency noise has a large derivative), and it is **destabilizing in the presence of dead time** (transport delay between command and response). With dead time, the derivative acts on stale information and can drive the system toward instability at exactly the moment the command is reaching the plant.

Tuning symptom: **K_d too high** — high-frequency chatter, sensor noise amplified into actuator wear. **D in a dead-time-dominated plant** — makes stability worse, not better. The correct response for significant dead time is to *omit* the derivative term entirely.

Practical design choice for fixture control: a pneumatic force actuator with 100-200ms regulator dead time and noisy load cell signals uses feedforward + integral only. The feedforward provides the initial estimate using the known cylinder area model; the integral trims the constant valve-bleed offset. The derivative is explicitly *left out* because dead time makes it harmful. This is the correct engineering judgment: match the tool to the physics, and omit terms that hurt.

13.3 Stability, Poles, Bandwidth, and Margins

13.3.1 Transfer Functions and Poles (Conceptual)

A linear control system can be described by a transfer function — a ratio of polynomials in the Laplace variable s . The roots of the denominator are **poles**. Where the poles sit in the complex plane determines

stability:

- Poles in the **left half-plane** (negative real part) → stable; the response decays exponentially.
- Poles on the **imaginary axis** → marginally stable; sustained oscillation.
- Poles in the **right half-plane** (positive real part) → unstable; growing oscillation.

You do not need to derive transfer functions for compute-board work. You need the intuition: a system with poles close to the imaginary axis is “almost oscillating” and has poor disturbance rejection; pole locations move as gains change, which is why increasing K_p or K_d too far eventually crosses the imaginary axis and goes unstable.

13.3.2 Bandwidth

Loop bandwidth is the frequency range over which the feedback loop effectively rejects disturbances. A higher bandwidth means the loop responds faster to errors, but also means it amplifies high-frequency measurement noise more. For a thermal control loop (slow plant, slow sensor), the bandwidth is naturally low — a few Hz at most. For a power-supply voltage regulation loop (fast plant), bandwidth can be tens of kHz. The test engineer encounters bandwidth implicitly: if the thermal controller is too slow to correct a sudden load change, the temperature overshoots; if the power supply loop bandwidth is too low, load-transient response is sluggish and the rail droops further than it should.

13.3.3 Phase Margin and Gain Margin

Stability margins quantify how far the system is from instability:

- **Phase margin (PM):** at the frequency where the open-loop gain is 0 dB (the gain crossover frequency), how many more degrees of phase lag would cause instability? A PM of 45° is the conventional minimum for “adequate” damping; 60° is comfortable. PM below $\sim 20^\circ$ means the step response overshoots significantly and rings.
- **Gain margin (GM):** at the frequency where the open-loop phase is -180° (the phase crossover frequency), how much more gain increase would cause instability? Typically want $GM \geq 6$ dB.

Where this appears in practice: a power supply’s loop compensation is designed for a specific output capacitance and load range. If the actual capacitance (from board layout, parallel decoupling cap combinations) differs from the design target, the phase margin changes. A supply that was stable on the original board may be marginally stable or oscillating on a revised layout — which shows up as rail ripple at the resonant frequency that does not go away at any load point, and that changes when you add or remove bulk capacitors. Recognizing this signature is the practical use of the stability-margin concept.

13.3.4 Measuring Loop Stability on the Bench (Bode Plot)

For bring-up or debugging of a switching regulator, the loop gain can be measured directly with a network analyzer or a function-generator-plus-oscilloscope setup:

1. Inject a small AC perturbation into the feedback network (typically through a 10-50 ohm resistor inserted in series with the feedback divider).
2. Sweep the perturbation frequency from below the LC resonance up to above the switching frequency (e.g., 1 kHz to 500 kHz for a 500 kHz switcher).
3. Measure the gain (V_{out}/V_{in}) and phase at each frequency.
4. Read phase margin at the 0 dB crossover; read gain margin at the -180 deg phase crossing.

Dedicated loop analyzers (Ridley AP300 series, AP Lab FRA-series) automate this sweep and plot the Bode diagram directly. The result tells you whether the supply has adequate stability margins with the actual capacitors on the board, not with whatever the datasheet reference design assumed.

Practical rule for test engineers: you will rarely measure Bode plots in production. You will recognize the *symptom* of a stability problem (sustained oscillation on the rail at a frequency unrelated to the switching frequency, ripple that changes when you clip a scope probe to the output, or rail that rings at a fixed frequency

independent of load). That recognition is what drives you to escalate to the power design team with a specific, actionable observation rather than a vague “the rail looks noisy.”

13.3.5 Dead Time and Its Effect on Stability

Dead time (also called transport delay) is any fixed latency between when a control command is issued and when the plant output begins to respond. It is a pure phase lag:

```
phase lag from dead time = 360 * f * T_dead (degrees, f in Hz, T_dead in seconds)
```

At a crossover frequency of 10 kHz with 10 us of dead time, the dead time alone contributes 36 degrees of phase lag — enough to eat half of a 60 degree phase margin. Sources of dead time in compute-board control loops:

- Pneumatic actuators in test fixtures: gas propagation delay from valve to cylinder (10-200 ms typical for long runs)
- PWM-to-mechanical lag in fans: the tachometer only reports speed after the next blade passes, introducing up to one revolution of delay
- Digital control loop computation: the ADC conversion time plus the controller computation cycle plus the DAC/PWM update latency

When dead time is significant relative to the loop time constant, the derivative term makes stability *worse* (not better) because it acts on stale information. The correct responses are: reduce dead time where possible (shorten pneumatic lines, increase sampling rate), reduce loop bandwidth to stay well below the frequency where dead-time phase lag dominates, or for large fixed dead times consider a Smith predictor — an inner-loop model that predicts the plant output ahead of the dead time and feeds a corrected error signal to the controller. The Smith predictor is advanced fixture control design territory, but knowing the concept prevents the mistake of tuning a high-gain controller on a dead-time-dominated plant and wondering why it is unstable.

13.4 Sampling and the Nyquist Rate

A digital controller samples the measured output at a discrete time interval T_s (sampling period); the sampling frequency is $f_s = 1/T_s$.

Nyquist theorem: to represent a signal of frequency f without aliasing, the sampling rate must be greater than $2f$ (strictly — at exactly $2f$ a band-edge sinusoid is ambiguous). The **Nyquist frequency** is $f_s/2$ — the highest frequency that can be unambiguously represented in the sampled signal.

For control systems, the practical rule is stricter: sample at least **10-20x the closed-loop bandwidth** to avoid the degradation in phase margin that discrete sampling introduces. A digital thermal controller with a 1 Hz bandwidth loop should sample at 10-20 Hz minimum (not 2 Hz, even though Nyquist would technically allow it).

Aliasing: if the sensor signal contains frequency content above $f_s/2$, those components fold back (alias) into the sampled data as false low-frequency content. An anti-aliasing filter before the ADC removes signal content above $f_s/2$ before sampling. The 20 MHz bandwidth limit on the scope for ripple measurement is the analog of this: you remove content above the relevant frequency before drawing conclusions.

13.4.1 Discrete-Time Control and the Z-Domain (Conceptual)

Continuous-time controllers are analyzed with the Laplace variable s . When a controller is implemented digitally — sampled, computed, and output at discrete time steps — the equivalent analysis uses the Z-transform variable z . The relationship is:

$z = \exp(s * T_s)$ where T_s is the sampling period

For conceptual fluency, the key facts are:

- The left-half s-plane (stable continuous poles) maps to the *inside* of the unit circle in the z-plane. A discrete controller is stable if all its poles lie strictly inside the unit circle.
- Sampling adds a computational delay of at least one sample period, which shows up as additional phase lag in the discrete loop — this is part of why the 10-20x oversampling rule exists.
- A continuous PID translated naively to discrete form (the “Euler backward” or “bilinear / Tustin” approximation) is generally stable when the sampling rate is well above the loop bandwidth, and degrades in performance as the sampling rate approaches the Nyquist minimum.

You will not need to derive z-domain transfer functions in daily work. What you need is the intuition: a digital controller sampled too slowly does not just fail to track fast disturbances; it can become unstable in ways that have no analogue in the continuous design it was based on.

Fan controller example: a BMC fan controller implemented in firmware samples the DTS temperature sensor at 1 Hz and adjusts PWM duty. At 1 Hz sampling, the controller can only respond to thermal events at timescales of seconds — appropriate for a slow thermal plant. If someone “improves” the controller by increasing the gain to speed up response without increasing the sample rate, the discrete phase lag from the 1 Hz sample may push the loop into oscillation: fan hunts between high and low speed, temperature oscillates, and the root cause is the combination of high gain and inadequate sampling rate. The fix is to increase the sampling rate before increasing the gain, not the other way around.

13.5 Sensors and Actuators in an Autonomous-Vehicle Context

Sensors relevant to compute-board test:

Sensor	Physical Quantity	Interface	Test Relevance
On-die DTS	Junction temperature	MMIO / hwmon sysfs	Primary thermal monitoring
Thermistor / thermocouple	Board ambient / case temp	ADC / I2C	Heatsink and case temperature
Shunt + INA-class IC	Rail current	I2C / PMBus	Power consumption monitoring
Hall-effect / tachometer	Fan speed (RPM)	GPIO pulse count	Fan health and speed control
Voltage monitor (INA219)	Rail voltage	I2C	In-circuit rail monitoring
Load cell	Applied force	Strain-gauge bridge / ADC	Fixture force control

Actuators:

Actuator	Physical Action	Control Signal	Used In
PWM fan	Air cooling	PWM duty cycle (25 kHz typical)	Thermal control loop
Heater element	Heat application	PWM or switched DC	Thermal soaker / chamber
Power supply trim	VRM output voltage	PMBus / DAC feedback	DC margining
Load switch (FET)	Rail enable/disable	GPIO	Sequencing and fault injection
Pneumatic regulator	Applied force	Proportional valve current	Fixture press

In the AV stack above the compute board, sensors (cameras, radar, lidar, IMU) feed perception algorithms, which output state estimates that feed planning and control. The actuators are the vehicle's steering motor, brake actuators, and throttle. The compute board is the central processing node in this chain. A test engineer's job is to verify that the board correctly receives sensor data and outputs commands in the required format — the control loop runs in software, and the test verifies the I/O correctness at the boundaries.

13.6 Where Test Engineers Touch Control Loops

13.6.1 Thermal Control and Fan PWM

The board's BMC or System-on-Chip (SoC) implements a closed-loop thermal controller that adjusts fan PWM to maintain die temperature within operating limits. The control loop is:

```
setpoint (T_target) --> [+] --> [controller] --> [PWM fan] --> board thermal plant
^ |
|_____ [temperature sensor (DTS)] _____|
```

In manufacturing test, verify: - Fan responds to a PWM command: read the tachometer back and confirm RPM vs duty curve is within spec. - The thermal control loop converges: under a fixed thermal load, the temperature settles and does not oscillate. - Throttling does not occur at the nominal test ambient temperature and load — if it does, the thermal solution is defective.

```
# Write fan PWM via hwmon (typical paths vary by board)
echo 200 > /sys/class/hwmon/hwmon3/pwm1 # set PWM level (0-255)
cat /sys/class/hwmon/hwmon3/fan1_input # read RPM
cat /sys/class/hwmon/hwmon3/temp1_input # read temperature (millidegrees C)
```

13.6.2 Power Regulation Loops

Every switching regulator is a closed-loop controller: the output voltage is the measured variable; the PWM duty cycle is the control output. The loop compensator (Type II or Type III op-amp compensator, or a digital controller in a Power Management Bus (PMBus) VRM) is designed for a specific phase margin and transient response. From a test perspective:

- **Steady-state rail accuracy** is the DC gain of the loop — verify with DMM.
- **Ripple** is the loop's rejection of the switching frequency — verify with AC-coupled scope at 20 MHz BW.
- **Load transient response** is the loop's dynamic tracking — verify with scope, trigger on load step, measure droop and recovery time.
- **Loop instability** appears as sustained oscillation on the rail at a frequency unrelated to the switcher frequency — typically caused by a capacitance out of the compensation design range. It shows up as a ring on the scope that does not correlate with switching frequency or load events.

13.6.3 Fixture Control Loops — Design Fluency for Test Engineers

A production test fixture commonly contains one or more explicit control loops:

Thermal forcing loop: a Peltier or resistive heater controlled by a PID to drive the board to a target temperature for hot-corner testing. Design considerations: - The plant (board thermal mass) is slow and first-order-like; integral action is needed to eliminate steady-state offset against a fixed ambient. - Derivative action may help if the sensor is low-noise and the plant has no significant dead time; omit it if the thermocouple signal is noisy or the heater-to-sensor path has a transport lag. - Overshoot on heat-up is a real concern for thermal-sensitive components; limit the heat-up rate (ramp-rate limiter on the setpoint, not just on the actuator) to avoid overshooting the target corner by 10-15 deg.

Pneumatic press loop: a proportional valve controlled by a PI loop to maintain a target contact force on a board-under-test during pogo-pin engagement. Design considerations: - Dead time from valve-to-cylinder gas transit (often 50-200 ms) dictates that bandwidth must stay well below $1/(2 * T_{\text{dead}})$. - Use integral-only or feedforward+integral; derivative is counterproductive. - Implement an anti-windup clamp at the valve's physical limits (0% to 100% open); windup during large force steps produces violent overshoot that can crack boards.

Fan speed loop in the fixture enclosure: a closed-loop fan controller maintaining airflow to simulate field conditions during test. The main failure modes are: - Fan fault (stall or bearing failure): tachometer reads zero or drops below expected RPM for the commanded duty. Screen by confirming tachometer reads within $\pm 20\%$ of the duty-RPM curve at each tested duty point. - Thermal leak in fixture: fan at full duty but board temperature rising — the fixture thermal design is inadequate for the board's power level. Flag as a fixture maintenance issue, not a board failure.

Commissioning a fixture control loop: before using a new fixture for production test, verify the control loops independently:

1. Step-response test: command a step to the target setpoint from ambient; measure rise time, overshoot, and settling time. Confirm overshoot < 5 deg for thermal, $< 5\%$ for force.
2. Disturbance rejection: with the loop at setpoint, apply a known disturbance (increase board load for thermal; add a fixed weight for force). Verify the loop corrects within the specified time.
3. Saturation recovery: drive the actuator to saturation (100% duty or max valve), then step to setpoint. Verify the anti-windup clamp prevents runaway overshoot.
4. Long-duration stability: run the loop for 30 minutes at setpoint under nominal load. Temperature or force should not drift or oscillate. Any slow drift indicates either a calibration error in the sensor or a leak/loss in the actuator.

These tests are part of fixture validation and are documented alongside the board test procedure. A fixture with a poorly tuned or uncalibrated control loop introduces process variation that cannot be distinguished from board-level variation — it becomes a phantom source of yield loss.

13.6.4 Control Loop Tuning Symptoms — Quick Reference

Observed Symptom	Likely Cause	First Fix
Slow convergence, permanent offset	Ki too low; or missing integral term	Increase Ki gradually; confirm integral is active
Overshoot then slow settle	Kp or Ki too high relative to system time constant	Reduce Ki first; then Kp if still overshooting
High-frequency chatter on actuator	Kd too high; sensor noise amplified	Reduce Kd; add low-pass filter on derivative path
Growing oscillation (unstable)	Gain too high; phase margin exhausted	Reduce Kp until oscillation stops; then retune
Correct at idle; oscillates under load	Phase margin collapses with added capacitance; or supply loop bandwidth near resonance	Check for capacitance added at load; reduce loop bandwidth
Rail oscillates at one frequency regardless of load	Loop instability (not switching ripple)	Identify frequency; measure vs switcher frequency; consult power design if different
Overshoot only after long saturation periods	Integral windup	Add or tighten anti-windup clamp to actuator physical limits
Temperature never reaches setpoint	Fan at max duty; thermal solution inadequate; thermistor fault	Check fan RPM at max duty; verify thermistor calibration; inspect TIM
Correct steady-state but sluggish load response	Loop bandwidth too low for load step rate	Increase Kp or reduce derivative filter; or increase switching regulator bandwidth

Observed Symptom	Likely Cause	First Fix
Intermittent oscillation correlated with load steps	Dead time + high derivative gain	Reduce or remove derivative term; increase damping

13.7 Control Concepts Applied to the Compute Board — Summary

The control-systems vocabulary is not abstract decoration in a compute test context. Every closed loop that runs through or around the board under test has a gain, a bandwidth, and stability margins, and when any of those are wrong the symptom appears in the data you are collecting:

- A rail that oscillates at 8 kHz independent of switching events is a regulator loop instability problem, diagnosable with a scope and the stability-margin framework.
- A thermal forcer that overshoots by 15 deg on every heat-up cycle is an anti-windup problem in the fixture PID, not a board defect — but if you do not distinguish the two, you will spend time chasing phantom thermal failures.
- A Gigabit Multimedia Serial Link (GMSL) camera that intermittently loses lock during high GPU load is a PoC voltage control problem: the PoC regulator’s load-transient response is insufficient, which is a loop-bandwidth problem in the power supply.
- A fan controller that hunts between 3000 and 6000 RPM under steady load is a discrete-time instability in the BMC thermal loop — too much gain at too low a sampling rate.

Fluency here means you can name the control-loop property that is failing, produce the evidence (scope trace, temperature log, RPM log, frequency annotation), and hand the right problem to the right team with the right framing. That is the practical boundary between someone who says “the board is unstable” and someone who says “the 1.0V SerDes supply has a 7 kHz sustained oscillation that is not at the 500 kHz switching frequency, onset correlates with adding 47 uF of bulk capacitance on the revised layout, and the regulator datasheet shows a minimum capacitance of 22 uF for stability at the current compensation setting.”

Chapter 14

Embedded Systems for the Compute Test Engineer

Embedded systems are not the center of this role, but they surround it. An autonomous-vehicle compute platform is itself a large embedded system, and scattered around the big SoCs and GPUs are dozens of microcontrollers you will test through or alongside: the BMC that owns power sequencing and telemetry, the safety MCU that watches the compute, power-sequencer and PMIC controllers, fan controllers, and the MCUs inside cameras, GMSL deserializers, and sensors. Every one of them runs firmware, talks a low-speed bus, and has a bring-up and update story. A test engineer who is fluent in embedded — registers, real-time behavior, firmware flashing, ESD — reads board behavior faster, writes better fixtures, and debugs the seams between “the chip” and “the software” where manufacturing defects love to hide.

This chapter is the embedded layer beneath the rest of the guide. It cross-references rather than repeats: the low-speed buses are covered in depth in the Automotive and Serial Buses chapter, rails and bring-up phases in the Power, Bring-up, and Functional Safety chapter, fixture control loops in the Control Systems chapter, and register/bit-field mechanics in the PCIe and Python chapters.

14.1 What “Embedded” Means on a Compute Platform

A useful split:

- **MCU (microcontroller)** — CPU + flash + SRAM + peripherals on one die, runs bare-metal or an RTOS from on-chip flash, boots in milliseconds, deterministic. Think BMC, power sequencer, safety supervisor. Tens of KB to a few MB of memory, no MMU (or a simple MPU for protection regions).
- **MPU / application processor** — external DRAM, an MMU, runs Linux. The big compute SoC is here. Covered by the Linux chapter.
- **SoC** — a system-on-chip blends both: application cores plus on-die microcontrollers (a safety island, a sensor hub, power-management cores) running their own firmware.

The distinctions that matter at test: an MCU’s behavior is **deterministic and owned by its firmware**, it comes up before Linux does (so it is often what you talk to *first* during bring-up), and its failures are register- and timing-level, not log-file-level. When a board “won’t power on,” the answer usually lives in an MCU’s firmware and a power-sequencing register, not in `dmesg`.

Where MCUs show up on an AV compute board, and why you care:

MCU role	What it owns	Test-engineer touchpoint
BMC (board management controller)	power sequencing, fan/thermal, rail telemetry, recovery	read rails/temps unattended over IPMI/sysfs; firmware version is part of genealogy
Safety MCU / supervisor	watchdog of the compute, fault reaction	fault-injection response, watchdog behavior, ASIL evidence
PMIC / sequencer	rail enable order and timing	the power-on sequence is “the datasheet is law” (Power chapter)
Sensor / camera / GMSL MCUs	sensor config, link bring-up	I2C config sequences, firmware revs, link training

14.2 Microcontrollers and the Memory Map

You already think in registers from PCIe config space; embedded is the same skill one level lower. An MCU exposes its peripherals as **memory-mapped registers**: fixed addresses where a load reads hardware state and a store changes hardware behavior. There is no driver stack in the way — you read and write the silicon directly.

```
/* A memory-mapped 32-bit peripheral register. 'volatile' is mandatory: it tells the
 * compiler the value can change outside program flow (hardware sets it) and must not
 * be cached in a register or optimized away. Omitting volatile is the classic
 * embedded bug -- a poll loop that never sees the bit change. */
#define GPIOA_BASE 0x40020000u
#define GPIO_ODR (*(volatile uint32_t *) (GPIOA_BASE + 0x14)) /* output data reg */
#define GPIO_IDR (*(volatile uint32_t *) (GPIOA_BASE + 0x10)) /* input data reg */

GPIO_ODR |= (1u << 5); /* set pin 5 high (read-modify-write) */
GPIO_ODR &= ~(1u << 5); /* clear pin 5 */
while (!(GPIO_IDR & (1u << 3))) /* spin until input pin 3 reads high */
;
```

The bit-field extract/insert idioms are identical to the PCIe register work — `(reg >> lsb) & mask` to read a field, `reg = (reg & ~(mask << lsb)) | (val << lsb)` to write one. Two embedded-specific hazards:

- **Read-modify-write races.** `REG |= bit` is three operations (load, OR, store). If an interrupt also writes `REG` between the load and the store, one update is lost. Guard shared registers with a critical section, or use hardware atomic set/clear registers where the silicon provides them (many MCUs expose separate “set” and “clear” register aliases exactly to avoid RMW).
- **Write-1-to-clear status bits** — the same semantics as PCIe AER status: you clear a latched flag by writing a 1 to it, not a 0. Writing the whole register back can clear bits you did not mean to. (The PCIe chapter and the toolkit’s BERT engine lean on this.)

Memory layout is fixed by a **linker script** that places code/constants in flash and variables/stack in SRAM, and defines the regions the startup code initializes (copy `.data` from flash to RAM, zero `.bss`) before `main()` runs. Knowing the map is how you read a fault address: a hard fault at an address outside any valid region is a wild pointer; one inside the peripheral block is a bad register access.

14.3 Bare-Metal, RTOS, and Real-Time

There are three execution models, in increasing order of structure:

- **Bare-metal super-loop** — `init(); for(;;) { do_work(); }` with interrupts handling time-critical events. Simple, fully deterministic, no scheduler overhead. Fine for a power sequencer or a fan controller.
- **RTOS** (FreeRTOS, Zephyr, ThreadX) — preemptive **tasks** with priorities, a scheduler, queues/semaphores/mutexes for inter-task communication. You reach for it when you have several activities at different rates and priorities (telemetry, comms, control) that a super-loop can no longer juggle cleanly.
- **Embedded Linux** — full MMU, processes, drivers; the application SoC. The Linux chapter covers it.

Real-time means deterministic, not fast. A hard-real-time task must meet its deadline *every* time; missing it is a system failure (a motor controller, a safety reaction). Soft real-time tolerates occasional misses (a UI, a log flush). What you reason about:

- **Latency and jitter** — the delay from event to response, and its variation. Jitter is often worse than latency: a control loop can design around a fixed 100 μ s delay but not around one that randomly spikes to 5 ms.
- **WCET (worst-case execution time)** — hard-real-time analysis is about the worst case, not the average. Caches, branch prediction, and DMA contention make the worst case much larger than the typical case.
- **Priority inversion** — a high-priority task blocked on a mutex held by a low-priority task that itself is preempted by a medium-priority task. The classic fix is **priority inheritance** (the holder temporarily inherits the waiter’s priority). This is famous because it stalled the Mars Pathfinder.

Two safety mechanisms you will meet on every compute board:

- **Watchdog timer** — a counter the firmware must “kick” periodically; if the firmware hangs and stops kicking, the watchdog resets the system. The safety MCU’s watchdog over the compute is part of the fault-reaction path (Functional Safety section of the Power chapter). At test, you verify the watchdog actually fires when starved — a watchdog that never bites is worse than none, because it gives false confidence.
- **Brown-out detection** — holds the MCU in reset until the supply is in spec, so it never executes on a marginal rail. A board that resets under load, not at power-on, often points here.

14.4 Interrupts and Concurrency on an MCU

Interrupts are the embedded concurrency model, and they bite the same way threads do (the Python chapter’s race conditions, one layer down and with no GIL to soften them).

```
/* ISR + main-loop hand-off via a ring buffer. The shared indices are volatile; the
 * ISR is the sole producer, main() the sole consumer (single-producer/single-consumer
 * is lock-free if head/tail are written by only one side each). */
#define RB_SZ 256
static volatile uint8_t rb[RB_SZ];
static volatile uint16_t head, tail; /* head: ISR writes; tail: main writes */

void UART_RX_IRQHandler(void) { /* keep ISRs SHORT: no printf, no malloc, no blocking */
    uint16_t next = (head + 1) % RB_SZ;
    if (next != tail) { rb[head] = UART_DR; head = next; } /* drop on full, never block */
}

int main(void) {
    for (;;) {
        if (tail != head) { /* data available */
            uint8_t byte = rb[tail];
            tail = (tail + 1) % RB_SZ;
            process(byte);
        }
    }
}
```

```
}  
}  
}
```

The rules that keep this correct:

- **ISRs must be short and non-blocking** — no `printf`, no dynamic allocation, no waiting on a lock. Do the minimum (grab the byte, set a flag) and let the main loop do the work.
- **volatile on anything shared with an ISR**, or the compiler will cache the stale value in a register.
- **Critical sections** — to touch a multi-byte shared structure from both ISR and main safely, briefly disable interrupts (`__disable_irq()/__enable_irq()`), or use a hardware atomic. Keep them as short as possible; disabling interrupts adds latency to everything else.
- **volatile is not a memory barrier**. It prevents caching of *that* variable but does not order writes to *different* variables across cores or DMA. Multi-core/DMA hand-offs need real barriers (`__DMB()`), not just `volatile`.

14.5 Firmware Lifecycle: Build, Flash, Boot

Firmware is C (sometimes C++/Rust) cross-compiled on your workstation for the target's architecture:

```
arm-none-eabi-gcc -mcpu=cortex-m4 -mthumb -O2 -ffunction-sections \  
-T stm32f4.ld -nostartfiles startup.s app.c -o app.elf # -T = linker script (the memory map)  
arm-none-eabi-objcopy -O binary app.elf app.bin # strip ELF to a raw flashable image  
arm-none-eabi-size app.elf # flash/SRAM usage vs the part's budget
```

The boot flow: on reset the CPU loads the initial stack pointer and the **reset vector** from the start of flash, runs startup code (init `.data/.bss`, set the clock tree), then `main()`. Many products boot a small **bootloader** first, which can accept a new image over a bus before jumping to the application — the basis of field/manufacturing update.

Getting an image onto the part, two routes:

- **Debug-port programming (JTAG/SWD)** — an external probe writes flash directly. This is the bring-up and factory path: deterministic, recoverable, and how you flash a virgin part that has no bootloader yet.
- **In-system / DFU** — the running firmware (or a resident bootloader) accepts an image over USB-DFU, UART, CAN, or the network. This is the field/line update path; it requires the device to already be bootable.

Security and integrity matter increasingly: **secure boot** has the bootloader verify a cryptographic signature on the image before running it (so only signed firmware executes), and **anti-rollback** prevents flashing an older, vulnerable version. At test you confirm signed images are accepted, unsigned/tampered ones are rejected, and the version meets the anti-rollback floor.

Version control for firmware images. Treat every firmware image as a released artifact: each shipped binary maps to a tagged commit, built reproducibly, with the version string baked into the image *and* readable at runtime. Tag releases (`fw-2.3.1`), never ship a binary you cannot rebuild from a tag, and record the firmware version in the per-unit results so you can trace which firmware touched each serial number. Version control of the *test programs* that flash and verify these images — branching, releasing, and rollback across stations — is covered in the Manufacturing Test chapter.

14.6 Firmware Update in Manufacturing

Manufacturing frequently includes **flashing production firmware** as a test step: the board arrives with factory-default (or older) firmware, the station flashes the production image, then **verifies** it. Done wrong, you brick units at volume; done right, it is a clean staged operation with a rollback path. The canonical safe pattern has four phases — **stage** → **activate** → **verify** → (**rollback on failure**) — and **records the resulting version for traceability** every time.

14.6.1 The staged (A/B) update model

Robust update mechanisms support **A/B (dual-bank) slots**: write the new image to the *inactive* slot (stage), switch the boot pointer and reset (activate), confirm it runs (verify), and if activation/verification fails, **roll back** by pointing at the still-intact previous slot. This is exactly how NVMe firmware slots and modern UEFI capsule / BMC redundant-image schemes work, and it is why a botched flash does not have to brick the unit.

```
stage activate (reset) verify rollback
+-----+ +-----+ +-----+ +-----+
| write new | | switch boot ptr | | read version| | revert ptr |
| -> slot B | -----> | to slot B; reset| ---> | == expected?| -No->| to slot A |
| (slot A | | (slot A intact) | | functional? | | (good image)|
| stays | +-----+ +-----+ +-----+
| bootable)| | Yes
+-----+ v
record version
in test results
```

14.6.2 Commands by component

```
# --- GPU VBIOS ---
nvidia-smi --query-gpu=vbios_version --format=csv # current VBIOS
# update via nvidia-smi / nvflash (NVIDIA proprietary) -- vendor-specific

# --- NVMe firmware (slot model: download -> activate -> verify) ---
nvme fw-log /dev/nvme0n1 # slots + active firmware revision
nvme fw-download /dev/nvme0n1 --fw=prod_fw.bin # STAGE into a slot
nvme fw-activate /dev/nvme0n1 --slot=2 --action=1 # ACTIVATE slot 2 (may need reset)
nvme fw-log /dev/nvme0n1 # VERIFY active rev == expected
nvme smart-log /dev/nvme0n1 # confirm healthy after activation

# --- BMC firmware (via IPMI) ---
ipmitool mc info # current BMC firmware version
# update is vendor-specific (HPM.1 or vendor tool); BMCs often keep a redundant image

# --- BIOS/UEFI ---
dmidecode -t bios | grep Version # current BIOS version
# flashing via vendor flasher or a signed UEFI capsule update; verify version after
```

14.6.3 The manufacturing firmware test pattern

1. **Read** the incoming version (record the *before* too).
2. **Stage** the production image (download to the inactive slot where supported).
3. **Activate** (reset if required).
4. **Verify the version matches expected and run a functional test** — a version string is necessary but not sufficient. Prove the device still enumerates, links, and passes its functional check on the new firmware.

5. **Roll back** to the known-good image if verify fails, and fail the unit cleanly with a decoded reason in the log.
6. **Record the final firmware version in the results** for traceability.

Robustness notes for an unattended CM line: **timeout every flash/activate/reset call** (a hung flasher must not wedge the station), never flash without a verified-good rollback path, and make the **failure state unambiguous** so a remote operator knows the unit failed and why.

14.7 Debug and Bring-Up Tooling

The embedded debug kit, and what each is for:

- **JTAG / SWD** — the hardware debug port. SWD (Serial Wire Debug) is the 2-pin ARM variant; JTAG the older multi-pin standard that can also chain multiple devices. Through it you halt the core, read/write memory and registers, set breakpoints, and flash. This is ground truth when there is no console.
- **gdb + OpenOCD** — OpenOCD drives the JTAG/SWD probe and exposes a gdb server; you debug the MCU from `gdb` exactly as you would a Linux program (`target remote :3333`, `load`, `break`, `mon reset halt`). Same muscle memory as the Linux chapter's `gdb`.
- **UART console / semihosting** — a serial `printf` is the cheapest telemetry. Semihosting routes `stdio` through the debug probe when no UART is free, at the cost of halting the core per call (never leave it on in timed code).
- **Logic analyzer** — decodes I2C/SPI/UART traffic so you can see whether the bus transaction actually happened and matched the datasheet timing. Indispensable for “the device isn’t responding” on a control bus.
- **Oscilloscope** — for anything analog or timing-level (rail ramp, signal integrity, the inrush and sequencing covered in the Power chapter).

The embedded bring-up reflex mirrors the board bring-up phases (Power chapter): confirm power and clocks before blaming firmware; confirm the bus transaction on a logic analyzer before blaming the device; read the fault status registers before guessing. Most “dead firmware” is a clock, a power rail, or a bus that never acked.

14.8 ESD Awareness and Handling

Electrostatic discharge silently destroys — or, worse, *damages without killing* — semiconductor devices. A static event from a person can be thousands of volts: far below what you feel, far above what a sub-1 V gate oxide tolerates. For a test engineer the danger is twofold: a board you mishandle now, and — more insidiously — a **test fixture that makes you the failure mechanism**.

14.8.1 The two failure modes (latent is the dangerous one)

- **Catastrophic** — the part dies immediately; the unit fails test. Bad, but you *catch* it.
- **Latent** — ESD weakens a junction or oxide without an immediate functional failure. The unit **passes test and ships**, then fails *in the field* weeks later. For a robotaxi this is the unacceptable case: a latent-damaged unit is a field/safety event waiting to happen, and your test never flagged it. This is precisely why ESD control is a *process* requirement, not a “be careful” suggestion.

14.8.2 Standard controls (the ESD Protected Area)

- **Grounded wrist straps** on every operator (1 M Ω series resistor for safety), bonded to a common ground point.

- **Dissipative work surfaces** — conductive/dissipative mats, grounded.
- **ESD-safe packaging** in transit — shielding (Faraday) bags, dissipative trays; never bare boards in ordinary plastic (plastic generates charge).
- **Humidity control** — dry air builds far more static; EPAs control relative humidity.
- **ESD footwear/flooring and smocks**; ionizers where charge cannot be bled off directly (insulators).
- **Common-ground discipline** — DUT, fixture, instruments, mat, and person all bonded to the *same* ground, so there is no potential difference to discharge through the part.

14.8.3 Why it matters specifically for test engineering

If your **fixture** lacks proper ESD grounding, the act of inserting or contacting the DUT can zap it. So ensure fixture contacts and probes are ESD-safe and grounded, that the DUT shares the station ground *before* signal pins mate, and that **wrist-strap monitors are verified working on every station, every shift** — a strap with a broken cord gives false confidence, so a continuous monitor beats a once-a-day check. An ESD escape is invisible at test and catastrophic in the field; the process controls are the safety net, and verifying them is part of keeping the line honest.

14.9 Low-Speed Control Buses (Cross-Reference)

The embedded world runs on three low-speed buses — **I2C** (multi-drop, 2-wire, for configuration and telemetry), **SPI** (fast, point-to-point, for flash and ADCs), and **UART** (the debug console and simple links). On a compute board they configure cameras and deserializers, read PMIC telemetry, and carry the console. From the embedded side they are just register read/write sequences over a bus: an I2C device exposes registers you address and read/write, often with a documented power-on init sequence the firmware must replay.

Their protocol depth — addressing, arbitration, clock stretching, modes, and the test-floor patterns for each — is in the **Automotive and Serial Buses chapter**; their use inside fixture and instrument control is in the **Python chapter** (pyserial, the `equipment_rpc` socket pattern). This chapter's point is only that, on an MCU, a bus transaction is a register operation you can see on a logic analyzer and must match to the datasheet.

14.10 Throughlines

- The AV compute platform is an embedded system; the MCUs around the SoCs (BMC, safety supervisor, sequencers, sensor hubs) come up first and own the behavior that `dmesg` never sees.
- Embedded is the register skill you already have from PCIe, one level lower and without a driver in the way — with `volatile`, read-modify-write races, and write-1-to-clear as the recurring hazards.
- Real-time means *deterministic*, not fast: reason about jitter, WCET, priority inversion, and the watchdog/brown-out mechanisms that make a board fail safe.
- Firmware is a lifecycle — build reproducibly, flash recoverably (JTAG/SWD for virgin parts, A/B staged update for the line), verify functionally not just by version string, and record the version for traceability.
- ESD control is a process, not a caution: latent damage ships and fails in the field, so verifying the EPA and the fixture grounding is part of the test engineer's job.

Chapter 15

Continuous Integration and Test-Engineering Quality Tooling

A test program is itself a piece of software. If the *bench-side* code (the BERT, the plan harness, the parsers, the dashboard) isn't held to the same engineering bar as the code under test, the program is the bottleneck — false rejects, hidden regressions, nightly mysteries — and you end up “trusting tools” you can't actually trust. This chapter is the engineering bar: the tooling stack that turns “the suite passes on my laptop” into a repeatable, auditable, defensible verdict the manufacturing line, the Quality team, and the next on-call engineer all rely on.

The Python chapter introduced the basics of `pytest`, mocking, and a smattering of CI patterns. This chapter is the **deep, holistic** version: what each tool exists for, how to actually use it (not toy examples), and **why** you reach for it instead of the nearest substitute. Each section is dense on purpose — every paragraph is meant to change what you reach for tomorrow.

The Zoox toolkit in `/toolkit/` is the running example throughout. When a section says “see `tests/test_parsers_property.py`” or “`.github/workflows/test.yml`”, those are real files in the repo you can open and trace.

15.1 The shape of a quality pipeline (and why a test engineer must own it)

A **quality pipeline** is the ordered set of automated checks every change passes through between an engineer's keyboard and a production-rack-mounted release. For a manufacturing-test program it has four nested layers, in increasing cost and decreasing frequency:

1. **Local pre-commit** — Lint + format + type-check + the fast unit subset, in sub-second. Run on every Save / before every `git commit`. **Goal:** never push the wrong shape of code.
2. **CI on every push and pull request** — Full unit suite + coverage gate + types + lint + C unit tests, in single-digit minutes. **Goal:** never *merge* a regression.
3. **Nightly** — Mutation tests, slow integration / corpus suites, real-hardware verification (where applicable), corpus refresh from a real station. **Goal:** catch the bug classes the per-PR layer is too costly to run.
4. **Release** — Tag + changelog + reproducible artifact + signed distribution. **Goal:** what the line runs is what was reviewed.

The defining property of a good pipeline is **honesty**: a green build means the code is good *to the standard*

you publicly committed to. Every shortcut (skipping a flaky test instead of fixing it, lowering the coverage gate to land a refactor, suppressing a real warning) is a debt that compounds. The pipeline must surface every honest finding and refuse to lie about a green that wasn't earned.

The test engineer **must own this pipeline end-to-end** because nobody else has the context that the bench code is itself life-critical instrumentation. A flaky pytest in a web app is a bad day; a flaky test in a station-side BERT means the wrong DUTs ship. The rest of this chapter is the toolset for owning that pipeline.

15.2 pytest, at the depth that matters in CI

The Python chapter showed `pytest` fixtures, `parametrize`, and basic mocking. This section is what you do **next** — the tools and patterns that turn a 50-test toy suite into a 500-test production pipeline.

15.2.1 Discovery and the layout that scales

`pytest` discovers tests by walking `testpaths` (set in `pyproject.toml`'s `[tool.pytest.ini_options]`) and importing any file matching `test_*.py` / `*_test.py`. **Layout decisions you'll regret if you skip them:**

- Put production code in `src/` (the **src layout**), not at the repo root. With `pythonpath = ["src"]` in `pyproject.toml`, `pytest` imports your package the same way `pip` does — meaning a missing `__init__.py` or a circular import breaks the suite the way it'll break a fresh install, not the way it works in your editor's `PYTHONPATH`.
- One test file per source file is the default; allow yourself one *suffix* file for the “edge cases / property tests / corpus replay” content (e.g. `test_parsers.py` + `test_parsers_property.py` + `test_parsers_corpus.py`). The suffix tells future-you what's expensive without having to read the file.
- Shared fixtures live in `conftest.py` at the **closest** scope they're useful. A station-config fixture used by every test goes in `tests/conftest.py`; a fake AER fixture used only by AER tests goes in `tests/aer/conftest.py`.

`conftest.py` is *not* an import path — `pytest` reads it automatically based on the directory walk. That is exactly why misplaced fixtures are the most common silent “why does this work in `test_a` but not `test_b`?” — they live above only one of them in the tree.

15.2.2 Marks: the vocabulary of selective execution

A **mark** is metadata on a test that lets you query, skip, or treat it specially. Three you must use fluently:

- `@pytest.mark.skip(reason=...)` / `@pytest.mark.skipif(condition, reason=...)` — declare *why* the test isn't running. `skipif(sys.platform == "win32")` is honest; bare `skip` without a reason is debt.
- `@pytest.mark.xfail(reason=..., strict=True)` — “I expect this to fail.” `strict=True` is mandatory in CI: it turns the test red the day the bug gets fixed, forcing you to delete the `xfail`. Without `strict`, an `xfail` that starts passing silently joins the “passing” bucket and you lose the diagnostic.
- **Custom marks** (`@pytest.mark.slow`, `@pytest.mark.realhw`) — register them in `pyproject.toml`'s `[tool.pytest.ini_options] markers = [...]` to avoid the “PytestUnknownMarkWarning”. Then `pytest -m "not slow"` is your sub-second PR gate and `pytest -m slow` is your nightly. Toolkit example: the real-hardware Phase 3 tests are not in the per-PR `test.yml`; they live in `qemu.yml` and run inside a Linux guest.

15.2.3 Fixture resolution and the rule of explicit dependency

Fixtures resolve **by name**: when a test function takes `nvme_device` as an argument, `pytest` walks up `conftest.py` until it finds a fixture by that name. Two things go wrong constantly:

- **Implicit dependency.** A fixture quietly mutates global state and a later test reads it. Solution: fixtures should be *pure* (return a value, never `import` your production module's `_global_singleton` and mutate it). When teardown is required, use `yield` — `pytest` guarantees teardown even on test failure.
- **Scope mismatch.** A `scope="session"` fixture that calls a `scope="function"` fixture is a `ScopeMismatch` error `pytest` raises when the fixture is requested at test setup (collection still succeeds). The rule: **a higher-scoped fixture cannot depend on a lower-scoped one** — function can use module/session; session cannot use module/function. Get this right and the fixture graph stays a DAG; get it wrong and you create test interdependence `pytest`'s parallel runners will surface randomly.

Two patterns the Zoox toolkit uses heavily:

- **The fixture factory.** `def make_device(injected_ber, ...)` returns a fresh mock device per call, inside the test body, when the test wants to build the topology itself. Avoids the parametrize explosion and reads more like a story.
- **The `tmp_path` fixture for I/O isolation.** Every test that writes to disk takes `tmp_path` (built-in) and writes there. `tmp_path_factory` (session-scoped) is the variant when you want shared-but-isolated.

15.2.4 The plugin ecosystem you'll actually use

- **`pytest-cov`** vs running coverage manually: `pytest-cov` integrates `coverage.py` with `pytest`'s plugin hooks, so a coverage failure aborts the run. The toolkit uses bare `coverage run -m pytest && coverage report` instead — slightly less integrated, but means coverage doesn't fight `pytest-xdist` if you add parallelism later. **Decision rule:** if you want parallel-test execution, prefer bare `coverage`; if you want the simpler single command, use `pytest-cov`.
- **`pytest-xdist`** for parallel execution (`pytest -n auto`). Worth it once your suite hits 30+ seconds. Two warnings: (a) tests that mutate shared state will fail randomly under `-n auto`; (b) coverage with `xdist` needs `coverage[toml]` and `parallel = true` in `[tool.coverage.run]`.
- **`pytest-timeout`** — a global timeout that kills a hung test. Mandatory the day you have any real-hardware or socket-bound test; without it, a wedge in production code wedges CI until the runner times the whole job out, and you lose the diagnostic. *Concretely:* during the 2026 audit a new thermal-chamber settle-poll spun to its 600-second timeout against a mock that never reported “settled,” and the whole suite hung — `--timeout=60` would have failed *that test* in a minute with its name, instead of a silent 10-minute wall. The toolkit doesn't ship `pytest-timeout` yet; that hang is the argument for adding it. Two lessons it drove home: a hang is not a slow test (you need a per-test deadline, not a bigger job timeout), and the per-step “just run the tests I touched” gate could not see it — only the *full* suite did. Run the whole suite before you trust the green; targeted gates hide cross-test interactions (that same audit had a second one — a corpus count that only broke when an unrelated module's fix changed a shared expectation).
- **`hypothesis`** — property-based testing. Doesn't conflict with `pytest`; registers as a plugin and adds the `@given(...)` decorator.

15.2.5 Anti-patterns to spot in PRs

These are the test-engineer-blocking findings you should send back without merging:

- **A `try/except` inside the test** that swallows an assertion. Tests should fail loud; if you have to catch something, the assertion goes outside the `try`.
- **`time.sleep(N)`** in a test (other than a few-millisecond `yield`). It's slow and flaky. Use a monkeypatched clock, a polled condition with a timeout, or a pre-computed deterministic timeline (the toolkit uses an injectable `clock=` and `sleep=` on `bert.run_bert` for exactly this).
- **Hard-coded paths.** `open("/tmp/foo")` collides under parallel runs and fails on Windows CI runners. Use `tmp_path`.
- **Tests that import production code with side effects on import.** A module that opens a serial port or connects to an instrument *at import time* is broken design — but it'll only get caught when the

test that imports it runs first on a fresh machine. Test for it explicitly with `import myproduction.module` in a fixture.

15.3 unittest — when stdlib is enough, and migrating away

`unittest` is Python’s standard-library xUnit framework (`TestCase` subclasses, `assertEqual`, `setUp/tearDown`, `discover`). You will meet it, but you should reach for it **only** in two cases:

1. **You cannot add a dependency** — secure-environment Python where `pip install pytest` is blocked. (Rare in test-engineering contexts, but real for some manufacturing partners.)
2. **You’re maintaining a legacy suite written in `unittest`** and the migration cost exceeds the benefit. Note: `pytest` discovers and runs `unittest.TestCase` classes natively, so you can run a mixed suite during migration.

For everything else, `pytest`’s friction is dramatically lower. Compare:

```
# unittest
class TestSpeed(unittest.TestCase):
    def setUp(self):
        self.fake = FakeBackend()
    def test_gen5_x16(self):
        self.assertEqual(self.fake.bps(5, 16), 5.041e11, places==9)

# pytest
def test_gen5_x16():
    assert FakeBackend().bps(5, 16) == pytest.approx(5.041e11, rel=1e-3)
```

The `pytest` version is half the lines, more readable, and `pytest.approx` is more expressive than `assertAlmostEqual`’s `places=` argument (which counts decimal *places*, not significant figures, a footgun on values that span orders of magnitude).

Migration recipe. When you inherit a `unittest` suite:

1. Add `pytest` as a dep; commit.
2. Run `pytest` against the existing `unittest` files — they pass unchanged.
3. Replace one `TestCase` at a time: remove the class, replace `setUp` with a fixture, `assert*` with `assert` Add the new test, delete the old, commit per conversion. The suite stays green throughout.

A test-engineering note on `pytest.approx` vs `unittest.assertAlmostEqual`: in BER math (`tests/test_ber.py`) you assert `gammaincinv(...)` to `rel=1e-9` — that’s relative tolerance, the right concept for floating-point. `assertAlmostEqual` with `places=` is wrong here. Knowing the right tolerance for the right measurement is half the battle of writing tests that don’t flake.

15.4 Unity — unit testing in C

Unity is a minimal, single-pair-of-files C test framework (`unity.h` + `unity.c`, no build system, no allocator). The toolkit uses it for the BERT engine (`c/test/test_pcie_bert.c`). It exists because **C test runners exist and you should use one**: the alternative is `assert(...)` in `main()` and `printf` — which is fine for one file and untenable across modules.

15.4.1 The runner shape

```
#include "unity.h"
```

```

#include "pcie_bert_core.h"
#include "fake_cfgspace.h"

void setUp(void) { /* per-test setup */ }
void tearDown(void) { /* per-test teardown */ }

void test_wlc_clears_only_set_bits(void) {
    fake_cfg_t fake = {0};
    fake_cfg_init(&fake, /* aer at offset */ 0x100);
    cfg_io io = fake_cfg_io(&fake);
    fake.aer_corr_status = (1u << 6) | (1u << 12); /* BadTLP + ReplayTO */

    uint32_t got = read_and_clear(&io, 0x100 + AER_CORR_STATUS, 0xFFFFFFFFu, 4);

    TEST_ASSERT_EQUAL_HEX32((1u << 6) | (1u << 12), got);
    TEST_ASSERT_EQUAL_HEX32(0, fake.aer_corr_status); /* both bits cleared */
}

int main(void) {
    UNITY_BEGIN();
    RUN_TEST(test_wlc_clears_only_set_bits);
    return UNITY_END();
}

```

Build: `cc -g -O0 -I. -Itest test/test_pcie_bert.c c/pcie_bert_core.c test/fake_cfgspace.c test/unity.c -o test/runner`. Run: `./test/runner`. Exit code = number of failures.

15.4.2 The dependency-injection seam (why Unity unit tests aren’t optional)

The toolkit’s C engine doesn’t talk to sysfs directly — it goes through a `cfg_io` struct of function pointers (`read`, `write`, `ctx`). Production passes a `cfg_io` whose `read` is `pread()` on a `/sys/.../config` file descriptor; tests pass a `cfg_io` whose `read` reads from an in-memory `fake_cfgspace_t`. This is the same idea as a Python `unittest.mock.MagicMock` but at the C ABI level.

The seam is **non-negotiable** for testable C. Without it your test has to actually open `/sys/bus/pci/devices/.../config` — which means running as root, on a specific Linux box, with a real PCIe device, while not being able to inject specific bit patterns. Unity tests against a `cfg_io` seam can: (a) verify W1C semantics (set-cleared-by-write-1) bit-for-bit; (b) walk capability lists with malformed-loop inputs that real hardware can’t safely produce; (c) test the `find_ext_cap` linked-list walk by directly setting “next” pointers. These are the bugs hardware *will* eventually present you with, and Unity catches them weeks before silicon does.

15.4.3 When to step up to cmocka

cmocka is Unity’s heavier cousin: stack mocks (function-replacement at link time), parameterized tests, group setup/teardown. Reach for it when (a) you need to mock syscalls deeper than a function-pointer seam can reach (e.g. mocking `ioctl`), or (b) your test program crosses translation-unit boundaries enough that link-time substitution is cleaner than a `cfg_io` per file. For the toolkit’s BERT engine the seam is enough; cmocka would be appropriate the day we add a kernel-driver test.

15.4.4 Memory and undefined-behavior checking: where the *real* wins are

Unity catches *logic* bugs. The wins in C come from the *language*: out-of-bounds writes, use-after-free, signed overflow. Build the test runner twice — once with `-fsanitize=address,undefined` (AddressSanitizer + UBSan), once without. Run both in CI. The sanitizer build catches every bug the logic tests can’t see; the unsanitized build catches the bugs the sanitizer accidentally hides (rare, but real).


```
test/runner-asan: $(TEST_SRC) ; $(CC) -g -O1 -fsanitize=address,undefined $^ -o $@
ctest-asan: test/runner-asan ; ./test/runner-asan
```

For the bench, this is the equivalent of running the BERT under a logic analyzer: strictly more information for one extra build step.

15.5 Hypothesis — property-based testing

A property-based test asserts **invariants over a generated input distribution** instead of asserting one specific output for one specific input. The framework (hypothesis for Python) tries hundreds of inputs and, when it finds a failing one, **shrinks** it to the minimum failing case. Two minutes after you wrote the test you know not “an input failed” but “*the smallest possible* input that violates your invariant is X.”

15.5.1 The mental flip: pinning behavior vs proving a property

Example/regression tests (the body of pytest) say “input X produces output Y.” A property test says “for *any* input drawn from this distribution, output Z holds.” Either alone is incomplete; together they cover orthogonal classes of bug.

The toolkit pins two real bugs via Hypothesis in `tests/test_parsers_property.py`:

- `_parse_speed`: Speed: . G (a malformed Speed line a real `ethtool` produced) fed `float('.')` and crashed. Property: **`_parse_speed` must never raise for any string** (real test below).
- `_stat`: '5²' (a value containing a Unicode digit, which `str.isdigit()` accepts but `int()` does not) crashed. Property: **`_stat` returns an int for any (stats, key) pair**.

```
from hypothesis import given, strategies as st
```

```
@given(st.text())
def test_parse_speed_never_raises(text):
    assert isinstance(_parse_speed(text), int)
```

This is **four lines** of test that exhaust an input space pytest’s hand-written `parametrize` never could. When the toolkit added this test in the audit pass, Hypothesis found and shrank the two bugs above in seconds. The property generalizes; the manual regression doesn’t.

15.5.2 Strategies — describing the input distribution

A **strategy** is “how to draw an input.” Built-in: `st.text()`, `st.integers(min, max)`, `st.lists(of)`, `st.dictionaries(keys=, values=)`, `st.binary(max_size=4096)`, `st.from_regex(r"...")`. Composable: `st.lists(st.integers())` is a list of ints.

For PCIe register tests, `st.binary(max_size=4096)` is exactly the input shape of a config space; the toolkit’s `test_find_ext_cap_terminates_and_never_raises` passes a random 4 KB blob to `find_ext_cap` and asserts the capability walk always terminates (no infinite loops, no crashes). That’s a class of bug — the malformed “next” pointer that loops back — that real hardware *can* produce under bus error, and that a regression test would need to know about ahead of time.

15.5.3 Shrinking, settings, and `@example` for permanent regressions

When a property fails, Hypothesis shrinks the input. The output you see in CI is the *minimal* failing case — exactly the regression you want to pin. The right next step is to **save it**:

```
@given(st.text())
@example(text="Speed: . G") # the shrunk case Hypothesis found
```

```
def test_parse_speed_never_raises(text):
    assert isinstance(_parse_speed(text), int)
```

`@example` runs in addition to generated inputs; the shrunk case becomes a permanent, named regression. The toolkit pins the `'rx_errors_phy'` shadows `'rx_errors'` and `'52'` cases this way.

`@settings(max_examples=300)` — for slower properties, raise the count. Default 100 is fine for fast ones; for `find_ext_cap` over 4 KB blobs we use 300.

15.5.4 The HealthCheck argument and why large_base_example will bite you

Hypothesis emits warnings when a strategy is “unhealthy” (too many filter-rejected examples, base example too large, etc.). The default is to fail the test on these warnings. For a strategy of `st.binary(max_size=4096)` Hypothesis complains about `large_base_example` — the “first example” is the empty string, which is small; when we ask for 4 KB blobs the variance triggers the heuristic.

The fix is **never** to suppress the warning globally. It’s to set `min_size=0` on the strategy, so the empty case is a legal example, and the framework stops complaining. The toolkit learned this the hard way in the Phase 0 commit; the correct strategy is `st.binary(max_size=4096)` with the implicit `min_size=0`.

15.6 mutmut — mutation testing, and what it actually buys you

Coverage tells you what *executed*. Mutation testing tells you what your tests **would detect a change in**. The difference matters: a line can be 100 % “covered” by a test that asserts nothing about it (touched but not validated). Mutation testing exposes that by *modifying* your code, re-running the suite, and reporting which mutations the suite *failed to notice*.

15.6.1 Mechanics

`mutmut run` parses every source file in `paths_to_mutate` (the toolkit configures this in `setup.cfg` to `ber.py`, `bert.py`, `aer.py`, `linkstate.py`, `topology.py`, `diagnostics.py` — the decision/math/logic modules) and emits **one mutant at a time**: `x < y` becomes `x <= y`; `+ 1` becomes `+ 2`; `True` becomes `False`; strings get prefixed with `XX` and suffixed with `XX`. For each mutant the runner applies the change, runs `pytest -x -q`, and records:

- **ok_killed** — the test suite caught the change (failed). Good.
- **bad_survived** — the test suite still passed despite the change. **A gap.**
- **bad_timeout** — the change broke the suite into a loop. Usually the same signal as killed.

The aim is the **survivor count** trending toward zero. **It will never reach zero** — mutations of equivalent code (a tolerance constant that doesn’t change results within float precision, a `_HAVE SCIPY = True/False` import probe on a machine where `scipy` is installed) cannot be killed by *any* test that doesn’t change the runtime environment. Those are called **equivalent mutants**, and you document them in your audit notes; the goal is “no *meaningful* survivor”, not zero survivors.

15.6.2 What a survivor actually tells you

Three categories, in priority order:

1. **A real gap.** The toolkit had a survivor on `GEN4_X16_BPS = link_bits_per_second(4, 16)` — `mutmut` mutated `(4, 16)` to `(5, 16)`, every test still passed, because no test pinned the constant’s value. The fix is a one-line test: `assert ber.GEN4_X16_BPS == pytest.approx(2.520e11, rel=1e-3)`. Survivor killed.

2. **An equivalent mutant.** `tiny = 1e-300` mutated to `tiny = 1e-299` doesn't change the continued-fraction convergence in `_gcf` within float precision; no test can kill this without artificially restricting the input. Document and accept.
3. **A spec-internal constant you've been carrying for years.** The audit-2 round turned up several of these in `ber.py`'s `scipy-fallback` path — they survived because the `scipy` path is the default and the fallback only runs when `scipy` is uninstalled. The kill: a test that `monkeypatch.setattr(ber, "_HAVE SCIPY", False)` and asserts the fallback produces the same numbers. Real value.

15.6.3 The runner cost, and why mutmut belongs in nightly CI

A killed mutant exits as soon as the first test fails (with `pytest -x`), so ~1 second. A survivor runs the whole suite to discover *nothing* fails — at the toolkit's 13 second baseline, ~13 seconds. With ~1264 mutants and a 30 % survivor rate, that's about 105 minutes of wall-clock. **Mutmut belongs nightly, never on the per-PR gate.** The toolkit's `.github/workflows/mutmut.yml` runs at 07:00 UTC, sharded into six parallel per-module jobs each at `timeout-minutes: 90` (wall-clock $\approx$ the slowest module, not the serial sum); the result is an artifact a human triages weekly, not a blocker on the PR cycle.

The toolkit deliberately scopes `paths_to_mutate` to the *decision/math/logic* modules. Mutating `nvme.py`'s `subprocess.run` line just turns the call into a crash that the test suite has no business catching (the real path is pragma'd out of coverage). Scoping mutmut to the modules the unit suite *fully* exercises is what keeps the survivor count meaningful.

15.6.4 When NOT to run mutation testing

If your suite has *coverage* gaps, mutmut will produce a flood of “survivors” that are really “lines no test ever touched.” Fix coverage first. Mutation testing is the next layer up: it asks “is each covered line *asserted on*?”, which is only a useful question once coverage is solidly above 90 %.

15.7 coverage.py — what to measure, what *not* to

Coverage (Python's `coverage.py`, integrated by `pytest-cov`) measures which lines your tests execute. The Zoox toolkit measures **branch coverage** (`[tool.coverage.run] branch = true`), which counts the **decisions** (both legs of each `if/while`) rather than the lines. Branch coverage is **strictly stronger**: a test that runs `if x: foo()` with only truthy `x` shows 100 % line coverage but 50 % branch coverage; line coverage hides the missed false-branch.

15.7.1 The `exclude_lines` philosophy: declare what you're NOT measuring

Some code is *deliberately* not unit-testable. The toolkit's real-hardware paths (`subprocess.run(["nvme", ...])`) are validated on the bench, not in the unit suite; the unit suite would have to mock or assume what `nvme-cli` produces, neither of which gives you confidence. The honest move is to **declare** that those lines are deliberately uncovered and exclude them from the coverage *gate*:

```
[tool.coverage.report]
exclude_lines = [
    "pragma: no cover", # real-hw sysfs/subprocess paths
    "if __name__ == '__main__':", # module demos
    "raise NotImplementedError", # placeholder paths
    "if TYPE_CHECKING:", # static-typing-only imports
]
```

Then `# pragma: no cover` on the real path's branch and the gate counts only what *should* be covered. The toolkit sits at ~97 % branch coverage with this rule; without the rule it would sit far lower and everyone would learn to ignore the warning, which is much worse than the honest 97 %.

15.7.2 fail_under is the gate, not the goal

`fail_under = 95` in `[tool.coverage.report]` makes `coverage report` exit non-zero below 95 %; CI's "Run coverage" step fails. You set this **just below where you actually sit** (we're at 98 %, gate is 95 %) — that gives a small honest buffer for the corpus or fixture work that occasionally swings ± 1 %. Setting it *above* where you actually sit makes CI red on day one; setting it *equal* to where you sit makes refactors that legitimately swing -0.1 % red the next day. Both errors destroy operator trust in the gate; the right answer is "a few percent below."

15.7.3 Coverage isn't quality. Read the misses, not the percentage.

`coverage report -m` produces the line-by-line miss list. When you do that on the toolkit you see things like `src/computetest/cli.py:226-227` (the `KeyboardInterrupt` handler) and `backend.py:268` (the speed-code fall-through for an unknown `speed_str`). Those are the gaps mutation testing will eventually exploit — closing them is more valuable than chasing the last 0.5 %.

The audit-2 round turned coverage misses **into bug fixes**: the `KeyboardInterrupt` miss was "no test exists, because no test had been written for this exit semantic"; once we wrote `test_keyboard_interrupt_maps_to_exit_130` the miss closed *and* a real CLI behavior got pinned.

15.8 The static-analysis stack — ruff, black, mypy

Static analysis runs your code through a tool that **infers properties without executing it**. The wins are speed (millisecond, not second), independence from tests (you'd never catch some of these bugs at runtime), and the discipline of "the code can't even merge in this shape." The three tools below each target a different property; you want all of them, run as a single pre-commit / PR gate.

15.8.1 ruff — the linter and now-also-the-formatter

`ruff` (Rust-written, $\sim 100\times$ faster than the Python `flake8`/`pylint` tools it replaces) does **lint** (warn on bad code shapes) and, since 0.1, **format** (rewrite to a canonical style). The toolkit configures it in `pyproject.toml`:

```
[tool.ruff]
line-length = 100
target-version = "py310"

[tool.ruff.lint]
select = ["E", "F", "W", "I", "B", "UP"]
```

The `select = [...]` is the rule families:

- **E/W** (pycodestyle) — whitespace and style consistency. Not a value judgment, but predictable formatting reduces review friction.
- **F** (pyflakes) — *unused imports, undefined names, shadowed variables*. These catch real bugs, not style: an unused import means a deleted feature left a stub.
- **I** (isort) — sorted, grouped imports. Mechanical, but `--fix` does it for free and PR diffs stop containing churn-changes.
- **B** (flake8-bugbear) — **real-bug findings**: B904 (`raise ... from None` vs nothing inside an except — preserves traceback), B008 (function call in argument default — the mutable-default antipattern), B905 (zip without `strict=` — silently drops elements if lengths differ). Audit-1 found two of these in real toolkit code; both were real bugs, not style issues.
- **UP** (pyupgrade) — keeps you on modern Python idioms: `% format` $\rightarrow$ f-string, redundant `()` in type annotations, `Tuple[int]` $\rightarrow$ `tuple[int]` on py310+. Easy, mechanical, no behavior change.

What ruff *won't* catch: anything that needs to know your types. That's what mypy is for.

15.8.2 mypy — types as a category of test you don't have to write

mypy reads your type hints (`def f(x: int) -> str:`) and reports inconsistencies. A useful way to think about types: **types are the cheapest property tests in the language**. Where a Hypothesis test would say “for any `str` input, this never raises,” a type hint says “this only takes `str` — the caller passing `bytes` is a PR-time error, not a runtime crash.”

The toolkit runs mypy with three strictness knobs from `pyproject.toml`:

```
[tool.mypy]
python_version = "3.10"
files = ["src"]
strict_optional = true # `None` is never silently a `str`
no_implicit_optional = true # `def f(x: int = None)` becomes an error
warn_unused_ignores = true # not the `# type: ignore` comments out over time
```

`strict_optional` is the one that catches the most real bugs. Without it, mypy treats `None` as compatible with any type — so `f(): -> str` returning `None` when the file isn't found compiles. With it, you get a type error at the location your code is silently producing the wrong thing, and you write the explicit `str | None` or `assert x is not None`.

Audit-1 (`margining.py:67`, `diagnostics.py:59`) found real `AttributeError-on-None` bugs this way: `worst_lane` returned `LaneMargin | None`, and the code accessed `.lane` without checking. The right fix is a real `assert ... is not None` with a comment explaining the invariant, not a `# type: ignore`. Those `asserts` now document the property and crash early if the property breaks.

Reasonable mypy goal: zero errors on `src/` with `strict_optional = true`, ratcheting up to `disallow_untyped_defs` once the codebase is fully annotated. The toolkit hits the former; not yet the latter.

15.8.3 black (and why ruff format mostly replaces it)

`black` is Python's most popular auto-formatter — opinionated, near-zero config, “the resulting style is the same regardless of who pushed the commit.” Its value isn't aesthetic; it's that **review diffs are about logic, not formatting**. Once your codebase is black-formatted, every PR's diff is signal.

`ruff format` (since ruff 0.1) implements the same formatting model as `black`, with the same defaults; the toolkit could use either. For a new project today, just use `ruff format` — one tool, one config, one CI step. Keep `black` only if you're in a codebase that already uses it and the team prefers it.

The unenforceable disagreement: `black` enforces double-quote strings; the toolkit doesn't use either formatter and the codebase has mixed quotes. **This is fine until it isn't** — the moment a PR review starts including “fix the quotes,” add the formatter and resolve the entire codebase in one commit.

15.8.4 Composition: pre-commit, the run-this-locally orchestrator

`pre-commit` (a separate tool, `pip install pre-commit`) is the local-side orchestrator: a `.pre-commit-config.yaml` lists the hooks (ruff, mypy, etc.) to run on every `git commit`. Configured well, it catches the lint/type findings **before** they leave your laptop, so CI on push never sees them.

A minimal `pre-commit` config for a Python toolkit:

```
repos:
- repo: https://github.com/astral-sh/ruff-pre-commit
  rev: v0.6.0
  hooks:
  - id: ruff
  - id: ruff-format
```

```
- repo: https://github.com/pre-commit/mirrors-mypy
  rev: v1.10.0
  hooks:
  - id: mypy
  additional_dependencies: [pytest, types-PyYAML]
```

Then `pre-commit install` writes a git hook; `pre-commit run --all-files` reruns everything on demand. The Zoox toolkit doesn't yet ship a `.pre-commit-config.yaml` — a reasonable next addition once the CI gates are settled.

15.9 GitHub Actions, from first principles

A **workflow** is a YAML file under `.github/workflows/` that GitHub runs on specified events (`push`, `pull_request`, `schedule`, `workflow_dispatch`). The workflow contains **jobs**, each running on a fresh **runner** (GitHub-hosted VM by default). Each job is a sequence of **steps**: a checkout, a Python setup, a `run`: command. **You read a workflow top-down, like a script.**

15.9.1 The minimum viable workflow

```
name: test
on: [push, pull_request]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
    - uses: actions/checkout@v4
    - uses: actions/setup-python@v5
    with: { python-version: '3.12', cache: 'pip' }
    - run: pip install -e '[dev]'
    - run: python -m pytest
```

Eight lines of config, ten seconds of wall-clock per push. **Every test repo should have this**, before anything else.

15.9.2 The matrix — why you fan out

```
strategy:
  fail-fast: false
  matrix:
    os: [ubuntu-latest, macos-latest]
    python-version: ['3.10', '3.11', '3.12', '3.13']
```

Eight combinations run in parallel. `fail-fast: false` is mandatory: without it, the first failure cancels the rest and you can't see *which* combinations broke. The toolkit's matrix found two real CI-only bugs (a `mypy import yaml` failure absent locally, a macOS-only flake in the BERT test) that single-config CI would never have caught.

The shape of a matrix follows your customer surface. The toolkit supports Linux and macOS station-side; not Windows. The matrix reflects that.

15.9.3 Triggers, paths, and the cost of running on every push

```
on:
  push:
  paths: ['toolkit/**', '.github/**']
  pull_request:
  workflow_dispatch: # let humans trigger from the UI
  schedule:
    - cron: '0 7 * * *' # 07:00 UTC daily
```

`paths:` is the underrated cost-control: a docs-only PR doesn't need to spin up the full matrix. `workflow_dispatch` lets you run the workflow on demand from the Actions UI — the toolkit uses this for `mutmut.yml` so you can re-run nightly off the schedule. `schedule:` runs unattended; **always set `timeout-minutes`** on scheduled jobs because a runaway one consumes your minute budget.

15.9.4 Secrets, artifacts, and the test-engineer's CI hygiene

- **Secrets** (`{{ secrets.NAME }}`) — never check a token into source. The Actions UI is the only place secrets live; jobs can reference them by name.
- **Artifacts** (`actions/upload-artifact`) — files the job produces (`.mutmut-cache`, coverage XML, serial-log dump on failure). The job uploads them; a human downloads via the run page. The toolkit's `qemu.yml` uploads `.work/serial.log` if: `failure()` so a failed boot is debuggable without rerunning.
- **Caching** (`actions/setup-python@v5` with: `cache: 'pip'`) — `pip install` runs in seconds instead of minutes on warm cache. The Python version is the cache key, so the matrix correctly caches per-version.

15.9.5 continue-on-error and honest CI

A step with `continue-on-error: true` shows in the UI as a warning instead of a fail. **Use it sparingly and honestly:** the toolkit marks the Phase 2 e2e step `continue-on-error: true` because the x86 TCG path is documented-best-effort (*not* because we want to hide a failure). The README explains why. **The moment you set `continue-on-error: true` to “make CI green,” you're lying to the team about a failure,** and the next person to read the workflow has to discover the lie. Don't do that.

15.9.6 Mapping workflow files to intents

The toolkit ships four workflows; the design is intentional:

- **`test.yml`** — the per-PR gate. Lint + types + tests + coverage. Eight-cell matrix. **Blocks merges.**
- **`qemu.yml`** — the privileged real-hardware-path verification: boot a Linux guest, drive QMP from the host, run the toolkit's real backend inside the guest. Slow (10–15 min). Runs on `toolkit/sim/qemu/**` or `toolkit/c/**` path changes.
- **`mutmut.yml`** — nightly mutation testing, sharded into six parallel per-module jobs (each `timeout-minutes: 90`). Uploads a `.mutmut-cache` artifact per shard; a human triages weekly. **Doesn't block merges.**
- **`corpus-refresh.yml`** — nightly schema-drift capture: re-runs the parsers against pinned fixtures and opens a PR if captured output drifts. **Doesn't block merges.**

This split — **fast + blocking on PR**, **slow + not-blocking on schedule** — is the ergonomic shape every test-engineering repo should converge to.

15.10 Test data: fixtures, fakes, corpus, contract tests

Where do your test inputs come from? Three sources, in increasing strength:

1. **Hand-written fixtures** (a literal in the test file). Fast and clear; **only valid for cases small enough to read at a glance**.
2. **Verified fakes** — a mock that behaves like the real thing, validated by a *contract test* that runs the same assertions over the mock *and* the real (or real-shape) backend.
3. **Corpus** — actual output from the real tool, captured to a versioned text file, replayed in tests.

15.10.1 Verified fakes and contract tests (the toolkit’s RealBackend+fixture pattern)

A mock that’s only ever tested against itself is **not** a substitute for the real thing — it’s a fixed point of confusion that drifts from reality. The toolkit’s `tests/sysfs_fixture.py` builds a tmp-dir fake `/sys/bus/pci` tree with byte-accurate config-space blobs and real symlinks, and `tests/test_backend_contract.py` runs the **same 22 assertions** against the MockBackend and a RealBackend(`sys_root=fake`). That’s a contract test: any property the real backend must have is asserted against both implementations. Drift between mock and real becomes a red CI immediately, not a Heisenbug at the bench.

The pattern generalizes. Whenever you have a “fake” anything, write the assertion in a function and call it twice — once for the fake, once for whatever you have of the real thing.

15.10.2 Corpus tests — pinning real-tool output as ground truth

`tests/test_parsers_corpus.py` opens `corpus/nvme/2.10/samsung-pm9a3/smart-log.json` (literal output of `nvme smart-log -o json` captured from a real station) and asserts the toolkit’s parser produces the expected verdict. Two reasons to do this:

1. **Format drift.** `nvme-cli` emits the wire key `avail_spare`; the toolkit normalizes it internally to `available_spare` (see `nvme/health.py`). The corpus captures the real wire key across `nvme-cli` versions and runs the parser against each; the day a parser quietly stops handling the wire name, a corpus replay catches it.
2. **Bug pinning.** The “abbreviated keys” bug (the toolkit’s `_normalize_smart_keys`) is recorded as a corpus entry — the exact JSON shape that triggered the regression — so the fix is permanent.

Corpus tests are the right answer to “we don’t trust the mock, we don’t have the hardware in CI.” Capture once, replay forever, refresh on a schedule.

15.10.3 Refreshing corpus — the `corpus-refresh.yml` workflow

A nightly workflow can run `nvme smart-log` inside a container with the latest `nvme-cli` and diff against the committed corpus. When the schema shifts, it opens a PR; the parser test that fails on the new corpus is your warning to update the parser. **You learn about a format change before manufacturing does**, on the tool maintainer’s release rhythm, not yours.

15.10.4 The tautology trap — the oracle must be the spec, not the code

asked “is each covered line *asserted* on?” This goes one level deeper: **what is the assertion compared against?** A test’s *oracle* is the source of its expected value, and the single most common way a green suite hides a spec-*wrong* program is an oracle **derived from the implementation itself**:

- A corpus `expected.json` generated by *running the parser you’re testing* and saving its output. The replay then asserts the parser reproduces what the parser produced — it can never fail, and it freezes in whatever bug the parser had the day you captured it.
- A round-trip: `decode(encode(x)) == x`. That proves the codec is self-*consistent*, not that the wire format is *correct*. A register field packed at the wrong bit offset round-trips perfectly and ships.

- A “golden” number copied from the code’s current output instead of computed from the datasheet.

Each passes on day one and stays green through every refactor — while the device on the bench, which speaks the *actual* spec, is decoded wrong. The toolkit’s 2026 audit found dozens: a DDR5 SPD corpus built to satisfy a parser reading the wrong byte (ECC at DDR4’s byte 13, not DDR5’s byte 235); an NVMe-MI header round-trip that “passed” with every field at the wrong bit position; CXL error-bit maps asserted against the code’s own off-by-two table. **Anchor the oracle outside the code:**

- **Golden vectors from the standard.** Hand-derive the expected bytes from the spec figure or a published worked example, write them as literals, assert the code reproduces *those*. The toolkit re-anchored its NVMe-MI test to a golden NMP byte (0x09 for a request / MI-command / CSI=1) computed from the spec, not from the encoder.
- **A second, independent implementation.** Cross-check against `linuxptp`, the Linux kernel decode tables, `libnvme`, QEMU’s QAPI schema. Agreement with an independent source is evidence; agreement with yourself is not.
- **A real-hardware capture** whose ground truth is hand-verified once, replayed forever.

Litmus test for any test you inherit: *if the implementation were subtly wrong, could this test still pass?* If yes, its oracle is the code, and it is theater.

A tool that flags a bug is itself a fallible oracle — verify before you “fix.” When you run a static analyzer, a coverage report, or (as the toolkit did) a large automated audit over your own test program, the findings are *hypotheses*, not facts. The toolkit’s audit had a meaningful **false-positive rate** — several “bugs” were the audit misreading a spec (a density table that was already correct, a gPTP delay formula that matched `linuxptp`, a coherency rule that was right). Applying those blindly would have *introduced* the bug the audit imagined. Verify every finding against the authoritative source before changing code; a test engineer who “fixes” an unverified finding has just shipped a regression with a confident commit message.

15.11 The pipeline you build at the bench (a concrete worked example)

Stitch the above into the pipeline the Zoox toolkit’s `.github/workflows/` codifies:

On every commit (locally, via pre-commit): `ruff + mypy` on the staged files. Sub-second; runs the format-fixer too.

On every push and PR (`test.yml`): 1. **Lint** (job 1, fastest, fails earliest) — `ruff check src tests`. 2. **Types** (still job 1) — `mypy src`. 3. **Tests** (job 2, the matrix, parallel) — `coverage run -m pytest && coverage report with fail_under = 95`. Ubuntu+macOS × py3.10–3.13. 4. **C tests** (job 3, ubuntu only) — `make ctest`.

Total wall-clock: ~2 min on a warm cache. Blocks merge.

On every push to a sim-changing path (`qemu.yml`): 1. **Unit** (job 1, same machine) — `pytest tests/test_qmp_inject.py`. 2. **E2E** (job 2, ubuntu, 60-min timeout) — boot a TCG guest, run Phase 2 AER injection, run Phase 3 real-kernel-path vdev checks.

Total: ~12 min. Doesn’t block merge for Phase 2 (TCG-vs-HVF caveat); does for the unit + Phase 3 paths.

Nightly (`mutmut.yml`): 1. Six parallel shards (one per scoped module) each spin a ubuntu runner, install dev deps, run `mutmut run` on their module with `timeout-minutes: 90`, and upload a per-shard `.mutmut-cache` artifact.

Total: ≈ the slowest module’s shard (~35–45 min), not the serial sum. Doesn’t block anything; produces a triage queue.

Release (when you cut one): 1. Tag the commit. CI runs the test workflow against the tag. Tag-bound artifact gets attached to the GitHub release. The line consumes that artifact.

This is the pipeline shape **every** Zoox test-engineering repo should converge to. The compositions matter:

- **Lint blocks types blocks tests.** If lint fails, types and tests don't run — why pay for an 80-second matrix when the formatter would have caught it?
 - **Tests block the matrix scope.** Run *one* fast cell first (Ubuntu / 3.12); if it passes, the matrix runs in parallel. If it fails, you save runner minutes.
 - **Nightly is for what's too slow to gate.** Mutation testing, corpus refresh, long-soak real-hardware. Their finding rate is “weekly” not “per-PR”, and their cost is multi-minute — wrong gate, right schedule.
-

15.12 Reading CI output: what to ignore, what to fix today

A real CI failure history reads like a debugger session — you should be able to look at the run page and infer *what* broke, *where*, and *why*. Build the habit of treating the run page as data, not noise:

- **Read the failed step's output bottom-up.** Test failures print the stack and the assertion at the bottom; everything above is noise. The toolkit's `test/macos-3.13` failure in the audit round was *one line at the bottom* of a 3000-line log: `assert 'pass' == 'fail'`, with the test name. Diagnosing it took 90 seconds because the test was small and the assertion was clear.
- **A failure that reproduces only on CI is real.** Don't write it off as “CI weirdness.” The macOS-only BERT flake was a wall-clock dependency the test shouldn't have had; CI surfaced it because CI's wall-clock is different from yours. The fix (inject a deterministic clock) made the test better forever, not just for CI.
- **A failure on *one* cell of a matrix is information.** Python 3.13 on Ubuntu failing while Python 3.12 on Ubuntu passes tells you the bug is a 3.13-typing-only finding (a `mypy` strict-optional false negative resolved in 3.13's typeshed, e.g.). Read the matrix as a coordinate system.
- **A timeout is a hung test, not a slow test.** The right fix is `pytest-timeout` per-test, not bumping the workflow timeout. A test that should take 100 ms and takes 5 minutes is broken; making CI wait longer for the broken test hides the break.

The CI output is the only person on the team who's seen every failure. Read it carefully.

15.13 What this entire stack does for you

Pick any production toolkit and ask: *can you delete a 300-line module, refactor the caller, and merge it with confidence by Tuesday?* If yes — your tests + types + coverage + linting + mutation testing + CI gate make the rewrite **provable** rather than scary. If no — you've been running on hope, and hope is what fails on the production line at 03:00 in a manufacturing context where calling a senior engineer isn't possible.

That is what every section in this chapter, taken seriously, buys: **the budget to refactor**. A test program isn't done when it works; it's done when the next person can change it. The path to “the next person can change it” is the stack above.

The Zoox compute toolkit's metrics today — ~1,166 tests, 97 % branch coverage, `ruff` + `mypy` clean, a nightly mutation lane, four CI workflows (two gating, ~11 build-gate minutes per push) — are not the goal. **Trust** is the goal. The metrics are how you and the rest of the manufacturing organization decide whether to trust the verdict the BERT just printed. Build the pipeline so that every signal — lint clean, types clean, tests green, coverage above the gate, no surviving mutants in your scoped modules — is one a downstream operator can act on without re-checking your work.